



Agent Handbook

with LangChain, LangGraph & DeepAgents

AI 에이전트 개발 가이드

Agent Handbook with LangChain, LangGraph & DeepAgents

Copyright © 2026 BAEUM.AI. All rights reserved.

이 책의 코드 예제는 교육 목적으로 자유롭게 사용할 수 있습니다.
본문의 무단 복제 및 배포는 금지됩니다.

초판 발행: 2026

서문

AI 에이전트는 단순한 챗봇을 넘어, 도구를 사용하고, 계획을 세우며, 복잡한 작업을 자율적으로 수행하는 지능형 시스템입니다. 이 책은 세 가지 주요 프레임워크 – **LangChain**, **LangGraph**, **Deep Agents** – 를 활용하여 실전 AI 에이전트를 구축하는 방법을 체계적으로 안내합니다.

이 책의 특징

- **실습 중심** – 59개의 Jupyter 노트북 기반 예제와 실행 결과를 포함합니다
- **단계적 학습** – 기초부터 프로덕션 배포까지 점진적으로 난이도가 올라갑니다
- **프레임워크 비교** – 각 프레임워크의 강점과 적합한 사용 사례를 비교합니다
- **실전 응용** – RAG, SQL, 데이터 분석, ML, 딥 리서치 에이전트를 구현합니다

대상 독자

- Python 기본 문법을 아는 개발자
- LLM 기반 애플리케이션을 구축하려는 엔지니어
- AI 에이전트 아키텍처를 체계적으로 배우고 싶은 분

이 책의 구성

파트	주제	챕터
I	에이전트 입문 – 환경 설정부터 세 프레임워크 기초까지	8
II	LangChain – 모델, 도구, 메모리, 미들웨어, 멀티 에이전트	13
III	LangGraph – 상태 그래프, 워크플로, 영속성, 서브그래프	13
IV	Deep Agents – 백엔드, 서브에이전트, 메모리, 하네스	10
V	고급 패턴 – 멀티 에이전트, RAG, SQL, 프로덕션	10
VI	실전 응용 – 5개 실전 에이전트 프로젝트	5

각 챕터는 학습 목표 → 이론 설명 → 코드 실습 → 요약 구조로 구성되어 있습니다.

이 책의 사용법

코드 블록 읽기

이 책의 코드 블록은 두 가지로 구분됩니다:

```
# Python 코드 블록 - 민트 좌측 보더
from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
result = model.invoke("Hello, Agent!")
```

```
실행 결과 블록 - 코랄 좌측 보더
Hello! I'm an AI agent ready to help.
```

박스 유형

TIP

유용한 팁이나 추가 정보를 제공합니다.

· NOTE

중요한 참고 사항이나 배경 지식을 설명합니다.

! WARNING

주의가 필요한 사항이나 일반적인 실수를 경고합니다.

실습 환경

모든 코드는 Python 3.11+ 환경에서 테스트되었습니다. 필요한 패키지 설치와 API 키 설정은 Part I의 첫 번째 챕터에서 다룹니다.

목차

Part I: 에이전트 입문	6
0. 환경 설정	8
1. LLM 기초	12
2. LangChain 입문	15
3. LangChain 대화	18
4. LangGraph 입문	21
5. Deep Agents 입문	24
6. 세 프레임워크 비교 & 다음 단계	27
7. 미니 프로젝트	33
Part II: LangChain	36
1. LangChain 소개	38
2. 첫 번째 에이전트	45
3. 모델과 메시지 시스템	51
4. 도구와 구조화된 출력	58
5. 메모리와 스트리밍	65
6. 미들웨어와 가드레일	72
7. 사람 개입(HITL)과 런타임	80
8. 멀티 에이전트 패턴	87
9. 커스텀 워크플로와 RAG	96
10. 프로덕션	102
11. MCP (Model Context Protocol)	111
12. 프론트엔드 스트리밍	123
13. 가드레일	133
Part III: LangGraph	145
1. LangGraph 소개	147
2. Graph API 기초	153
3. Functional API	162
4. 워크플로 패턴	170
5. 에이전트 구축	177
6. 지속성과 메모리	182
7. 스트리밍	191
8. 인터럽트와 타임 트래블	197
9. 서브그래프	204
10. 프로덕션	210
11. 로컬 서버	220
12. 내구성 실행	230
13. API 선택 가이드와 Pregel	241
Part IV: Deep Agents	252
1. Deep Agents 소개	254
2. 첫 번째 에이전트 만들기	263
3. 에이전트 커스터마이징	270
4. 스토리지 백엔드	279
5. 서브에이전트와 태스크 위임	292
6. 장기 메모리 & 스킬	304
7. 고급 기능	315
8. 에이전트 하네스	328
9. 외부 프레임워크 비교	341
10. 샌드박스 와 ACP	350

11. 비동기 서브에이전트	362
12. 프로덕션 준비	368
13. 컨텍스트 엔지니어링	375
14. 스트리밍	381
15. 권한 관리	386
Part V: 고급 패턴	391
0. v0 → v1 마이그레이션 가이드	393
1. 미들웨어 시스템 심화	401
2. 멀티에이전트: Subagents	419
3. 멀티에이전트: Handoffs & Router	430
4. 컨텍스트 엔지니어링 & 메모리 심화	444
5. Agentic RAG	459
6. SQL 에이전트 심화	474
7. 데이터 분석 에이전트	485
8. 보이스 에이전트	495
9. 프로덕션 배포	508
Part VI: LangSmith	523
1. LangSmith Quickstart	525
2. 에이전트 트레이스 구조	534
3. 데이터셋과 평가 루프	542
4. Prompt Hub 버전 관리	550
5. 프로덕션 모니터링	555
Part VII: 실전 응용	561
1. RAG 에이전트	563
2. SQL 에이전트	569
3. 데이터 분석 에이전트	574
4. 머신러닝 에이전트	580
5. 딥 리서치 에이전트	585
Part VIII: Integrations	590
1. 통합 카테고리 개요	592
2. Anthropic Prompt Caching	596
3. Claude Bash Tool	600
4. Claude Text Editor	604
5. Claude Memory	607
6. Anthropic File Search	611
7. Bedrock Prompt Caching	615
8. OpenAI Moderation	619
부록 A. 용어집	623

P A R T

I

에이전트 입문

Foundations of AI Agents

이 파트에서 다루는 내용

- 0. 환경 설정
- 1. LLM 기초
- 2. LangChain 입문
- 3. LangChain 메모리
- 4. LangGraph 입문
- 5. Deep Agents 입문
- 6. 세 프레임워크 비교
- 7. 미니 프로젝트

00 환경 설정

시작하기 전에

이 책은 AI 에이전트 개발을 세 가지 프레임워크를 통해 단계적으로 학습하는 과정입니다. **LangChain**은 100 개 이상의 LLM 프로바이더와 도구를 연결하는 인터페이스 레이어를 제공하고, **LangGraph**는 상태 기반 워크플로 오케스트레이션을 추가하며, **Deep Agents**는 이 모든 것을 사전 구축된 에이전트 하네스로 감싸줍니다. 세 프레임워크는 동일한 모델 인터페이스를 공유하므로, 이 장에서 설정하는 환경은 책 전체에 걸쳐 그대로 사용됩니다.

이 모든 것을 시작하려면 먼저 개발 환경을 구성해야 합니다.

학습 목표

LEARNING OBJECTIVES

- `.env` 파일로 API 키를 안전하게 관리하는 방법을 익힌다
- `ChatOpenAI` 로 LLM 모델을 초기화한다
- 모델에 간단한 질문을 보내 정상 동작을 확인한다

0.1 API 키 설정

LLM API를 호출하려면 프로바이더별 API 키가 필요합니다. 프로젝트 루트의 `.env.example` 을 `.env` 로 복사하고, 발급받은 키를 입력하세요.

```
OPENAI_API_KEY=sk-...  
TAVILY_API_KEY=tvly-... # 선택
```

키	용도	발급처
OPENAI_API_KEY	LLM 호출 (필수)	https://platform.openai.com/api-keys
TAVILY_API_KEY	웹 검색 도구 (선택)	https://tavily.com

! WARNING

`.env` 파일은 절대 버전 관리 시스템에 커밋하지 마세요. 프로젝트의 `.gitignore` 에 `.env` 가 포함되어 있는지 반드시 확인하세요. 프로덕션 환경에서는 `.env` 파일 대신 AWS Secrets Manager, GCP Secret Manager 같은 전용 시크릿 매니저를 사용하는 것이 권장됩니다.

```
from dotenv import load_dotenv
import os

load_dotenv(override=True)

assert os.environ.get("OPENAI_API_KEY"), "OPENAI_API_KEY가 설정되지 않았습니다!"
print("\u2713 API 키 로드 완료")
```

`load_dotenv(override=True)` 에서 `override=True` 매개변수는 `.env` 파일의 값이 시스템 환경 변수보다 우선하도록 합니다. 프로젝트마다 다른 API 키나 설정을 사용해야 할 때 유용합니다.

✓ API 키 로드 완료

Observability (관측 가능성) 설정

에이전트는 LLM 호출, 도구 실행, 상태 전이 등 여러 단계를 거쳐 작업을 수행합니다. 이 과정에서 어떤 단계에서 오류가 발생했는지, 각 단계에 토큰이 얼마나 소비되었는지 파악하려면 트레이싱(tracing)이 필수적입니다.

LangSmith는 `LANGSMITH_TRACING=true` 환경 변수 하나만 설정하면 코드 수정 없이 자동으로 활성화되며, Langfuse는 오픈소스 대안으로 콜백 핸들러를 통해 동작합니다.

```
# Observability 설정 (선택) - LangSmith 또는 Langfuse
# .env에 키를 설정하거나, 아래 주석을 해제하여 직접 입력하세요.
# os.environ["LANGFUSE_SECRET_KEY"] = "sk-lf-..."
# os.environ["LANGFUSE_PUBLIC_KEY"] = "pk-lf-..."
# os.environ["LANGFUSE_HOST"] = "https://lf.ddok.ai"
import os

# LangSmith: LANGSMITH_TRACING=true 시 자동 활성화 (코드 수정 불필요)
if os.environ.get("LANGSMITH_TRACING", "").lower() == "true":
    project = os.environ.get("LANGSMITH_PROJECT", "default")
    print(f"LangSmith tracing ON \u2014 project: {project}")

# Langfuse: invoke/stream 호출 시 config={"callbacks": [langfuse_handler]} 전달
langfuse_handler = None
if os.environ.get("LANGFUSE_SECRET_KEY"):
    from langfuse.langchain import CallbackHandler
    langfuse_handler = CallbackHandler()
    print(f"Langfuse tracing ON \u2014 {os.environ.get('LANGFUSE_HOST', '')}")

# Langfuse config: pass to invoke/stream/batch calls
lf_config = {"callbacks": [langfuse_handler]} if langfuse_handler else {}
```

Langfuse tracing ON – <https://lf.ddok.ai>

API 키가 준비되었으니, 이제 이 키를 사용할 LLM 모델을 초기화합니다.

0.2 모델 초기화

LangChain에서 모델을 초기화하는 방법은 두 가지입니다. 첫 번째는 프로바이더별 클래스를 직접 사용하는 방식으로, `ChatOpenAI` 가 대표적입니다. `temperature`, `max_tokens`, `timeout`, `max_retries` (기본값: 6) 등 프로바이더 고유 매개변수에 직접 접근할 수 있어 세밀한 제어가 가능합니다.

두 번째는 `init_chat_model()` 팩토리 함수로, 모델 이름에서 프로바이더를 자동 감지합니다. 런타임에 프로바이더를 전환해야 하는 경우에 유용하지만, 이 책에서는 명시성을 위해 `ChatOpenAI` 를 사용합니다.

```
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-5.4")
print("\u2713 모델 설정 완료:", model.model_name)
```

✓ 모델 설정 완료: gpt-5.4

TIP

프로바이더를 동적으로 전환하고 싶다면 `from langchain.chat_models import init_chat_model` 을 사용하세요.

`init_chat_model("gpt-5.4")` 은 모델 이름에서 OpenAI를 자동 감지하고,
`init_chat_model("claude-sonnet-4-6")` 은 Anthropic을 자동 감지합니다.

모델 객체가 준비되었습니다. 실제로 동작하는지 간단한 호출로 확인해 봅시다.

0.3 동작 확인

`model.invoke()` 는 단순한 문자열이 아니라 `AIMessage` 객체를 반환합니다. `.content` 속성으로 응답 텍스트를 얻을 수 있고, `.usage_metadata` 에는 토큰 사용량(`input_tokens`, `output_tokens`, `total_tokens`)이, `.response_metadata` 에는 모델 버전이나 종료 사유 같은 프로바이더별 메타데이터가 담겨 있습니다. 이 메시지 체계는 다음 장에서 자세히 다룹니다.

```
response = model.invoke("안녕하세요! 한 문장으로 답해주세요.", config=lf_config)
print("\u2713 모델 응답:", response.content)
```

✓ 모델 응답: 안녕하세요! 무엇을 도와드릴까요?

요약

항목	내용
환경 변수	<code>load_dotenv()</code> 로 <code>.env</code> 파일 로드

모델	<code>ChatOpenAI(model="gpt-5.4")</code>
테스트	<code>model.invoke("...")</code> → <code>AIMessage</code> 객체 반환 확인

환경이 정상적으로 구성되었습니다. 다음 장에서는 이 `model` 객체를 사용하여 LLM의 메시지 시스템과 호출 방식을 자세히 살펴봅니다.

01 LLM 기초

메시지, 프롬프트, 스트리밍

현대의 LLM은 근본적으로 채팅 완성(chat completion) 모델입니다. 단순히 텍스트를 이어 쓰는 과거의 텍스트 완성 모델과 달리, 메시지 시퀀스를 입력으로 받아 다음 메시지를 예측합니다. 이 메시지 기반 인터페이스를 통해 대화 맥락, 페르소나, 멀티턴 상호작용을 정밀하게 제어할 수 있습니다. 에이전트, 도구, 워크플로 – 이 책에서 다루는 모든 것은 이 메시지 시스템 위에 구축됩니다.

에이전트를 만들기 전에, 먼저 LLM과 대화하는 기본 방법을 익힙니다.

학습 목표

LEARNING OBJECTIVES

- 메시지의 역할(`system` , `human` , `ai`)을 이해한다
- 시스템 메시지로 모델의 행동을 제어한다
- `model.stream()` 으로 실시간 응답을 받는다

1.1 환경 설정

```
from dotenv import load_dotenv
load_dotenv(override=True)

from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
print("\u2713 모델 준비 완료")
```

✓ 모델 준비 완료

1.2 메시지의 세 가지 역할

LLM은 메시지 리스트를 입력으로 받습니다. 각 메시지에는 역할이 있으며, 역할(role), 콘텐츠(content), 메타데이터(metadata) 세 가지 핵심 요소로 구성됩니다.

역할	클래스	설명
system	SystemMessage	모델의 행동 지침을 설정합니다. 페르소나, 응답 톤, 규칙 등을 정의하여 모델의 초기 동작을 결정합니다.
human	HumanMessage	사용자의 입력을 나타냅니다. 텍스트뿐만 아니라 이미지, 오디오, 파일 등 멀티모달 콘텐츠를 지원합니다.
ai	AIMessage	모델의 응답입니다. 텍스트 응답 외에도 <code>tool_calls</code> (도구 호출), <code>usage_metadata</code> (토큰 사용량) 등의 속성을 포함합니다.

`AIMessage` 는 단순한 텍스트 응답 이상의 정보를 담고 있습니다. `.content` 로 응답 텍스트를 얻는 것 외에도, `.tool_calls` 에는 에이전트가 사용하려는 도구 호출 요청(이름, 인자, 고유 ID)이 담겨 있고, `.usage_metadata` 에는 토큰 소비량(`input_tokens` , `output_tokens` , `total_tokens`)이, `.response_metadata` 에는 모델 버전이나 종료 사유 같은 프로바이더별 정보가 포함됩니다. 스트리밍 중에는 `AIMessageChunk` 객체가 생성되며, 이들은 `+` 연산자로 점진적으로 결합되어 최종 `AIMessage` 를 구성합니다.

이 외에도 도구 실행 결과를 모델에 전달하는 `ToolMessage` 가 있습니다. 도구 호출의 전체 흐름은 다음과 같습니다: (1) 모델이 `tool_calls` 가 포함된 `AIMessage` 를 반환하고, (2) 코드가 해당 도구를 실행한 뒤, (3) 결과를 매칭되는 `tool_call_id` 와 함께 `ToolMessage` 로 감싸서 (4) 대화에 추가합니다. `ToolMessage` 에는 선택적으로 `artifact` 필드를 사용할 수 있는데, 모델에게는 보여줄 필요 없지만 후속 코드에서 활용할 보조 데이터(예: 문서 ID)를 저장할 때 유용합니다.

이 메시지 시스템을 구성하는 핵심 요소부터 살펴봅시다.

1.3 시스템 메시지로 행동 제어

`SystemMessage` 를 바꾸면 같은 질문에도 전혀 다른 응답을 받습니다. 이것이 프롬프트 엔지니어링의 핵심입니다. 메시지 역할 중 가장 강력한 것은 `SystemMessage` 입니다. 같은 질문이라도 시스템 메시지에 따라 완전히 다른 답변을 받을 수 있습니다.

1.4 딕셔너리 형식

메시지 객체 대신 딕셔너리으로도 전달할 수 있습니다. LangChain은 메시지 입력을 세 가지 형식으로 지원합니다:

1. 문자열(String): 단순 텍스트 프롬프트에 적합 (예: `model.invoke("Hello")`)
2. 메시지 객체(Message objects): `SystemMessage` , `HumanMessage` 등 타입이 지정된 인스턴스 리스트
3. 딕셔너리(Dictionary): OpenAI Chat Completion API와 동일한 `{"role": ..., "content": ...}` 구조

세 가지 형식 모두 동일한 결과를 반환하므로, 상황에 맞는 형식을 선택하면 됩니다. 딕셔너리 형식은 기존 OpenAI 코드를 LangChain으로 마이그레이션할 때 특히 유용합니다.

딕셔너리 형식의 `content` 필드는 텍스트뿐 아니라 멀티모달 콘텐츠도 담을 수 있습니다. `HumanMessage` 는 텍스트 외에 이미지(URL, base64, 프로바이더 파일 ID), 오디오, 비디오, 문서(PDF) 등 다양한 형식을 지원합니다. LangChain은 이러한 멀티모달 입력을 프로바이더 간에 표준화하므로, OpenAI, Anthropic, Google 중 어떤 프로바이더를 사용하더라도 동일한 코드가 동작합니다.

1.5 스트리밍

`model.stream()` 을 사용하면 토큰이 생성되는 대로 실시간으로 출력됩니다. 사용자 체감 속도가 크게 향상됩니다.

LangChain 모델은 세 가지 호출 방식을 제공합니다:

- `invoke()` : 동기 호출로 전체 응답을 한 번에 반환
- `stream()` : 토큰 단위로 `AIMessageChunk` 객체를 순차 반환하여 실시간 출력 가능
- `batch()` : 여러 요청을 동시에 처리하여 효율성 향상

스트리밍 중에는 각 `AIMessageChunk` 가 점진적으로 결합되어 최종 메시지를 구성하며, 토큰 사용량도 점진적으로 추적할 수 있습니다. `stream()` 은 동기 메서드이며, 비동기 코드에서는 `astream()` 을 사용합니다.

스트리밍이 단일 요청의 응답성을 높인다면, `batch()` 는 여러 요청의 처리량을 높입니다.

1.6 배치 호출

`model.batch()` 로 여러 질문을 한 번에 보낼 수 있습니다. `batch()` 는 요청을 순차적으로 처리하는 것이 아니라 병렬로 처리하므로, `invoke()` 를 루프로 반복 호출하는 것보다 훨씬 빠릅니다. 여러 문서를 분류하거나 다수의 텍스트를 동시에 번역하는 등의 대량 처리 작업에 유용합니다.

요약

개념	설명
<code>SystemMessage</code>	모델의 페르소나·규칙 설정
<code>HumanMessage</code>	사용자 입력
<code>model.invoke()</code>	동기 호출 (전체 응답)
<code>model.stream()</code>	토큰 단위 실시간 출력
<code>model.batch()</code>	여러 요청 동시 처리

이 장에서 다룬 메시지 시스템과 호출 방식은 다음 장에서 배울 도구(Tool) 호출과 에이전트의 기반이 됩니다.

02 LangChain 입문

첫 번째 에이전트

에이전트(Agent)란 LLM이 어떤 행동을 취할지 스스로 판단하고, 도구를 통해 실행하고, 결과를 관찰하여, 작업이 완료될 때까지 반복하는 시스템입니다. 단순한 프롬프트-응답 상호작용과 근본적으로 다릅니다 – 모델이 어떤 도구를 어떤 순서로 사용할지 추론하기 때문입니다. LangChain v1의 `create_agent()` 함수는 이 과정을 ReAct(Reasoning + Acting) 루프로 구현합니다.

이 장에서는 LangChain v1의 핵심 API로 도구를 갖춘 첫 번째 에이전트를 만들어 봅니다.

학습 목표

LEARNING OBJECTIVES

- `@tool` 데코레이터로 커스텀 도구를 정의한다
- `create_agent()` 로 에이전트를 생성한다
- `invoke()` 로 에이전트를 실행하고 결과를 확인한다

2.1 환경 설정

```
from dotenv import load_dotenv
load_dotenv(override=True)

from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
print("\u2713 모델 준비 완료")
```

✓ 모델 준비 완료

에이전트가 사용할 도구를 먼저 정의합니다. 도구란 에이전트가 외부 세계와 상호작용할 수 있게 해주는 함수입니다.

2.2 도구 만들기

`@tool` 데코레이터를 붙이면 일반 함수가 에이전트 도구가 됩니다.

도구를 정의할 때 알아야 할 핵심 규칙:

- Docstring이 가장 중요합니다. 모델은 이 설명을 읽고 도구 사용 여부를 결정합니다. 동료에게 설명하듯 명확하게 작성하세요 — 이 도구가 무엇을 하는지, 언제 사용해야 하는지, 제약 조건은 무엇인지.
- 타입 힌트(Type Hints): 함수 파라미터의 타입 힌트가 도구의 입력 스키마를 자동으로 정의합니다.
`Literal` 타입으로 허용 값을 제한할 수도 있습니다 (예: `units: Literal["celsius", "fahrenheit"]`).
- 커스텀 이름/설명: `@tool("custom_name")` 형태로 함수명을 덮어쓰거나, `@tool("calc", description="...")` 형태로 이름과 설명 모두 직접 지정할 수 있습니다.
- 복잡한 입력: Pydantic `BaseModel` 과 `Field(description="...")` 를 사용하여 필드별 설명이 포함된 복잡한 입력 스키마를 정의할 수 있습니다.

! WARNING

`config` 와 `runtime` 은 LangChain이 내부적으로 사용하는 예약된 매개변수 이름입니다. 도구 함수의 인자 이름으로 사용하지 마세요.

```
from langchain.tools import tool

@tool
def add(a: int, b: int) -> int:
    """두 수를 더합니다."""
    return a + b

@tool
def multiply(a: int, b: int) -> int:
    """두 수를 곱합니다."""
    return a * b

print("도구 목록:")
for t in [add, multiply]:
    print(f" - {t.name}: {t.description}")
```

도구 목록:

- add: 두 수를 더합니다.
- multiply: 두 수를 곱합니다.

도구가 준비되었으니, 이제 모델과 도구를 결합하여 에이전트를 생성합니다.

2.3 에이전트 생성 & 실행

`create_agent()` 는 모델과 도구를 결합하여 에이전트를 만듭니다. 에이전트는 내부적으로 다음과 같은 ReAct(Reasoning + Acting) 루프를 실행합니다:

1. 사용자가 메시지를 보냅니다.
2. 모델이 메시지를 분석하고 판단합니다: 도구를 호출할 것인가, 바로 응답할 것인가?
3. 도구 호출이 필요하다면, 모델은 텍스트 대신 `tool_calls` 가 포함된 `AIMessage` 를 반환합니다.
4. 에이전트 프레임워크가 도구를 실행하고 결과를 `ToolMessage` 로 생성합니다.
5. `ToolMessage` 가 대화에 추가되고, 모델이 다시 호출됩니다.
6. 2~5단계를 반복합니다.
7. 모델이 텍스트로 응답하거나(도구 호출 없음), 최대 반복 횟수에 도달하면 루프가 종료됩니다.

에이전트의 핵심 구성 요소:

- 모델(Model): LLM이 어떤 도구를 호출할지 판단합니다. 문자열("openai:gpt-5.4") 또는 모델 객체를 전달할 수 있습니다. 문자열 형식은 "provider:model_name" 으로, 빠른 프로토타이핑에 편리합니다.
- 도구(Tools): 에이전트가 수행할 수 있는 액션입니다. 순차 호출, 병렬 실행, 재시도를 모두 지원합니다.
- 시스템 프롬프트(System Prompt): 에이전트의 행동을 안내하는 지침입니다. 수학 에이전트라면 “당신은 수학 도우미입니다. 항상 계산 과정을 보여주세요.”처럼 구체적으로 작성할수록 유용한 결과를 얻습니다.

```
from langchain.agents import create_agent

agent = create_agent(
    model=model,
    tools=[add, multiply],
    system_prompt="당신은 수학 도우미입니다.",
)
print("\u2713 에이전트 생성 완료")
```

✓ 에이전트 생성 완료

요약

핵심 API	역할
<code>@tool</code>	함수를 에이전트 도구로 변환
<code>create_agent()</code>	모델 + 도구 → 에이전트 생성
<code>agent.invoke()</code>	에이전트 실행, 결과 반환
<code>agent.stream()</code>	중간 단계를 실시간으로 확인

TIP

장시간 실행되는 에이전트의 경우 `agent.stream({"messages": [...]}, stream_mode="values")` 를 사용하면 각 도구 호출과 응답을 실시간으로 확인할 수 있습니다.

이 장에서 만든 에이전트는 메모리가 없어 대화를 기억하지 못합니다. 다음 장에서는 `InMemorySaver` 를 사용하여 에이전트에 대화 기억 능력을 추가합니다.

03 LangChain 대화

멀티턴 메모리

앞 장에서 만든 에이전트는 매 호출마다 모든 것을 잊습니다. “내 이름은 철수야”라고 말한 뒤 “내 이름이 뭐야?”라고 물으면, 에이전트는 답하지 못합니다. `invoke()` 는 기본적으로 상태를 유지하지 않기 때문입니다. 이 장에서는 체크포인트를 사용하여 에이전트에 대화 기억 능력을 추가합니다.

학습 목표

LEARNING OBJECTIVES

- `InMemorySaver` 로 대화 상태를 저장한다
- `thread_id` 로 대화 세션을 구분한다
- 에이전트가 이전 문맥을 기억하는 멀티턴 대화를 실행한다

3.1 환경 설정

```
from dotenv import load_dotenv
load_dotenv(override=True)

from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
print("\u2713 모델 준비 완료")
```

✓ 모델 준비 완료

먼저 메모리가 없는 에이전트의 한계를 직접 확인해 봅시다.

3.2 메모리 없는 에이전트의 한계

기본 에이전트는 상태를 저장하지 않습니다. 매번 새로운 대화로 취급하므로, 이전 대화에서 사용자가 제공한 정보(이름, 선호도 등)를 전혀 기억하지 못합니다. `invoke()` 는 입력 메시지를 처리하고 결과를 반환한 뒤 아무것도 보존하지 않기 때문입니다.

이 문제를 해결하기 위해 LangGraph의 체크포인트 시스템을 사용합니다.

3.3 InMemorySaver로 메모리 추가

단기 메모리(Short-term Memory)란 단일 대화 스레드 내에서 이전 상호작용의 정보를 유지하는 기능입니다.

`InMemorySaver` 는 그래프의 전체 상태(메시지 히스토리 포함)를 Python 딕셔너리에 저장하는 체크포인트입니다. 각 항목은 `thread_id` 로 구분됩니다.

에이전트가 `thread_id` 와 함께 호출되면, `LangGraph`는 이전 상태를 로드하고, 새 메시지를 추가하고, 에이전트를 실행한 뒤, 업데이트된 상태를 다시 저장합니다. 이것이 의미하는 바는:

- 같은 `thread_id` = 연속된 대화 (이전 맥락 유지)
- 다른 `thread_id` = 완전히 별개의 대화
- `InMemorySaver` 의 데이터는 Python 프로세스가 종료되면 사라집니다 (메모리 내 저장)

프로덕션 환경에서는 프로세스 재시작 후에도 대화가 유지되어야 합니다. `LangGraph`는 `PostgresSaver` (`PostgreSQL`)와 `SqliteSaver` (`SQLite`) 같은 영속적 체크포인트를 제공합니다. API는 동일하므로

`InMemorySaver()` 를 `PostgresSaver(connection_string)` 으로 교체하기만 하면 됩니다. 체크포인트는 “타임 트래블” 기능도 지원하여, 이전 체크포인트에서 에이전트 실행을 재생할 수 있습니다.

대화 히스토리 관리 전략

LLM은 유한한 컨텍스트 윈도우를 가지고 있으므로, 긴 대화는 토큰 제한을 초과하여 오류를 발생시킬 수 있습니다. 이를 방지하기 위한 세 가지 전략이 있습니다:

- 트리밍(Trimming): 최근 N개의 메시지만 유지합니다. 구현이 단순하지만 초기 맥락이 손실됩니다.
- 요약(Summarization): 오래된 메시지를 주기적으로 하나의 요약 메시지로 압축합니다. 핵심 정보를 보존하면서 토큰을 절약합니다.
- 삭제(Deletion): 특정 메시지(예: 중간 도구 호출)를 제거하여 노이즈를 줄입니다.

```
from langgraph.checkpoint.memory import InMemorySaver

agent = create_agent(
    model=model,
    tools=[add],
    checkpointer=InMemorySaver(),
)

config = {"configurable": {"thread_id": "session-1"}}
print("\u2713 메모리 에이전트 생성 완료")
```

✓ 메모리 에이전트 생성 완료

메모리가 추가된 에이전트를 스트리밍으로 실행하면, 각 단계에서 에이전트가 이전 대화를 기억하는 모습을 실시간으로 관찰할 수 있습니다.

3.4 스트리밍으로 실시간 확인

`agent.stream()` 으로 에이전트의 각 단계를 실시간으로 관찰합니다. `LangGraph`는 여러 스트리밍 모드를 제공합니다:

모드	설명
----	----

"updates"	각 노드의 변경 사항만 반환 – “모델이 도구 X를 호출함”, “도구가 Y를 반환함” 등. 가장 간결한 출력
"values"	각 노드 실행 후 전체 상태를 반환. 더 자세하지만 전체 그림 파악에 유용
"messages"	개별 LLM 토큰과 메타데이터를 반환. 실시간 UI 렌더링에 최적

요약

개념	설명
<code>InMemorySaver</code>	메모리 내 대화 상태 저장 (체크포인트)
<code>thread_id</code>	대화 세션 구분 키
<code>checkpointer=</code>	<code>create_agent()</code> 에 체크포인트 전달
<code>stream(mode="updates")</code>	에이전트 실행 단계를 실시간 확인

지금까지 LangChain의 `create_agent()` 가 내부적으로 처리해주던 것들을 직접 구성해보고 싶다면, 다음 장에서 배울 LangGraph의 `StateGraph` 가 그 방법입니다.

04 LangGraph 입문

워크플로 만들기

2장의 `create_agent()` 는 단순한 ReAct 루프를 제공하지만, 실제 애플리케이션에서는 조건 분기(예: 긴급도에 따라 고객 이메일을 다른 처리기로 라우팅), 병렬 실행, 인간 승인 단계, 오류 복구 등이 필요합니다. LangGraph 는 이러한 복잡한 워크플로를 방향 그래프(directed graph)로 모델링합니다. 노드는 처리 단계이고, 엣지는 노드 간의 흐름을 정의합니다. 이 분해(decomposition) 방식을 통해 스트리밍, 일시 중지/재개가 가능한 내구적 실행, 단계 간 상태 검사를 통한 명확한 디버깅이 가능해집니다.

이 장에서는 LangGraph의 `StateGraph` 로 노드와 엣지를 연결하는 워크플로를 직접 만들어 봅니다.

학습 목표

LEARNING OBJECTIVES

- `StateGraph` 로 상태 기반 그래프를 정의한다
- 노드(함수)를 등록하고 엣지로 연결한다
- `compile()` → `invoke()` 로 그래프를 실행한다

4.1 환경 설정

```
from dotenv import load_dotenv
load_dotenv(override=True)

from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
print("\u2713 모델 준비 완료")
```

✓ 모델 준비 완료

4.2 첫 번째 그래프

LangGraph의 기본 흐름은 5단계입니다:

```
StateGraph(State) → add_node() → add_edge() → compile() → invoke()
```

StateGraph의 핵심 개념:

LangGraph는 에이전트 워크플로를 그래프로 모델링하며, 세 가지 기본 구성 요소를 사용합니다:

1. State(상태): 애플리케이션의 현재 스냅샷을 나타내는 공유 데이터 구조입니다. 보통 `TypedDict` 나 `Pydantic` 모델로 정의합니다.
2. Node(노드): 상태를 받아 연산을 수행하고 업데이트된 상태를 반환하는 함수입니다. 즉, 노드가 실제 작업을 수행합니다.
3. Edge(엣지): 현재 상태를 기반으로 다음에 실행할 노드를 결정합니다. 즉, 엣지가 다음 할 일을 지시합니다.

`StateGraph` 는 주요 그래프 클래스로, 사용자 정의 State 객체를 매개변수로 받습니다. 그래프는 반드시 `.compile()` 메서드를 통해 컴파일한 후 사용해야 하며, 컴파일 시 도달 불가능한 노드나 누락된 엣지 등 구조적 오류가 검증됩니다.

`compile()` 은 선택적 매개변수도 지원합니다:

- `checkpointer=InMemorySaver()` - 상태 영속성 활성화 (3장에서 다룸)
- `interrupt_before=["node_name"]` - 특정 노드 실행 전 일시 중지 (휴먼 인 더 루프)
- `interrupt_after=["node_name"]` - 특정 노드 실행 후 일시 중지

LangGraph 워크플로 설계 5단계

LangGraph로 워크플로를 설계할 때는 다음 5단계 프로세스를 따릅니다:

1. 워크플로를 개별 단계로 분해: 각 독립적 작업을 노드 후보로 식별합니다.
2. 작업 유형 분류: LLM 단계(추론), 데이터 단계(검색), 액션 단계(부수 효과), 사용자 입력 단계(인간 개입)로 구분합니다.
3. 상태 구조 설계: 단계 간에 유지해야 할 데이터를 정의합니다. 원시 데이터를 저장하고, 포매팅은 노드 내부에서 수행합니다.
4. 노드 함수 구현: 각 함수는 상태를 받아 변경된 키만 포함한 상태 업데이트를 반환합니다.
5. 연결 및 실행: 엣지로 연결하고, 컴파일하고, 실행합니다.

아래 예제는 텍스트의 단어 수를 세는 간단한 1노드 그래프입니다.

1개 노드 그래프로 기본 구조를 이해했으니, 노드를 연결하여 파이프라인을 구성해 봅시다.

4.3 2노드 그래프

두 개의 노드를 순서대로 연결합니다. 첫 번째 노드가 텍스트를 대문자로 변환하고, 두 번째 노드가 단어 수를 셉니다.

```
START → uppercase → counter → END
```

지금까지는 노드가 항상 같은 다음 노드로 진행했습니다. 하지만 실제 워크플로에서는 조건에 따라 다른 경로로 분기해야 할 때가 많습니다. `add_conditional_edges()` 를 사용하면 라우팅 함수가 현재 상태를 검사하고 다음 노드의 이름을 반환하여 동적 분기를 구현할 수 있습니다. 예를 들어, LangGraph가 제공하는 `tools_condition` 함수는 LLM이 도구 호출을 요청했으면 "tools" 노드로, 아니면 END로 라우팅합니다. 조건부 엣지는 5장 이후에서 본격적으로 다룹니다.

조건 분기의 가장 흔한 사용 사례는 LLM이 도구를 호출할지 결정하는 것입니다. `MessagesState` 를 사용하면 이를 쉽게 구현할 수 있습니다.

4.4 LLM을 노드로 사용하기

`MessagesState` 를 사용하면 LLM 대화를 그래프로 구성할 수 있습니다.

`MessagesState`란?

`MessagesState` 는 LangGraph가 제공하는 사전 정의 상태 클래스로, `messages` 라는 단일 키를 가지며 `add_messages` 를 리듀서로 사용합니다. 내부적으로 다음과 같이 정의되어 있습니다:

```
class MessagesState(TypedDict):
    messages: Annotated[list, add_messages]
```

`add_messages` 리듀서는 메시지 ID를 추적하여 중복 없이 메시지를 누적하고, JSON 딕셔너리를 LangChain `Message` 객체로 자동 역직렬화합니다. 추가 필드(예: `documents: list`, `user_id: str`)가 필요하면 `MessagesState` 를 서브클래싱하여 메시지 처리 동작을 유지하면서 커스텀 필드를 추가할 수 있습니다.

노드의 구조:

노드는 현재 상태(`state`)를 받아 상태 업데이트를 반환하는 일반 Python 함수(동기/비동기)입니다. LangGraph는 노드를 자동으로 `RunnableLambda` 객체로 변환하여 배치 처리, 비동기 지원, 네이티브 트레이싱 기능을 추가합니다.

요약

핵심 API	역할
<code>StateGraph(State)</code>	상태 스키마로 그래프 빌더 생성
<code>add_node()</code>	노드(함수) 등록
<code>add_edge()</code>	노드 간 연결
<code>compile()</code>	실행 가능한 그래프 생성
<code>invoke()</code>	그래프 실행
<code>add_conditional_edges()</code>	조건부 분기 – 라우팅 함수로 다음 노드 결정

LangGraph로 워크플로를 세밀하게 제어할 수 있지만, 매번 노드와 엣지를 직접 구성하는 것은 번거로울 수 있습니다. 다음 장에서는 이 모든 것을 한 줄로 해결하는 Deep Agents를 소개합니다.

05 Deep Agents 입문

올인원 에이전트

Deep Agents SDK의 `create_deep_agent()` 로 도구·메모리·백엔드가 내장된 에이전트를 한 줄로 만들어 봅니다.

학습 목표

LEARNING OBJECTIVES

- `create_deep_agent()` 로 에이전트를 생성한다
- `invoke()` 로 에이전트를 실행한다
- 커스텀 도구를 추가한 에이전트를 만든다

5.1 환경 설정

```
from dotenv import load_dotenv
load_dotenv(override=True)

from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
print("\u2713 모델 준비 완료")
```

✓ 모델 준비 완료

5.2 에이전트 생성

`create_deep_agent()` 는 LangChain 모델을 받아, 파일 읽기/쓰기/검색 등의 빌트인 도구가 자동으로 포함된 에이전트를 반환합니다. 반환 타입은 LangGraph의 `CompiledStateGraph` 이므로 `invoke()` , `stream()` 등을 그대로 사용할 수 있습니다.

Deep Agents란?

Deep Agents는 에이전트 개발을 간소화하기 위해 설계된 프레임워크로, 일종의 “에이전트 하네스(harness)” 역할을 합니다. 내부적으로 LangChain과 LangGraph 위에 구축됩니다. `create_deep_agent()` 를 호출하면 내부적으로 LangGraph의 `StateGraph` 에 계획, 도구 실행, 컨텍스트 관리를 위한 사전 구성된 노드가 생성됩니다.

반환되는 `CompiledStateGraph` 는 4장에서 `builder.compile()` 로 얻는 것과 동일한 타입이므로, 스트리밍, 체크포인트링 등 모든 `LangGraph` 기능이 자동으로 작동합니다. 따라서 4장에서 `LangGraph` 기초를 이해한 것이 `Deep Agents`의 내부 동작을 파악하는 데 도움이 됩니다.

핵심 내장 기능:

기능	설명
태스크 플래닝	<code>write_todos</code> / <code>read_todos</code> 도구로 구조화된 태스크 리스트를 생성하고, 진행하면서 항목을 체크합니다
컨텍스트 관리	<code>write_file</code> / <code>read_file</code> 도구로 중간 결과를 가상 파일시스템에 저장합니다. 대화에 모든 것을 유지하는 대신 데이터를 파일로 오프로드하여 토큰 오버플로를 방지합니다
유연한 저장소	기본적으로 인메모리 파일시스템을 사용하지만, 로컬 디스크나 영구 저장소로 교체할 수 있는 플러그블 백엔드를 지원합니다
서브에이전트 위임	<code>create_subagent</code> 도구로 독자적인 컨텍스트 윈도우, 도구, 시스템 프롬프트를 가진 전문 서브에이전트를 생성합니다. 서브에이전트는 작업을 완료하고 결과를 부모에게 반환합니다
영구 메모리	<code>LangGraph</code> 의 메모리 인프라를 활용하여 여러 대화에 걸쳐 정보 유지

에이전트 생성 방법:

`create_deep_agent()` 에 모델, 도구, 시스템 프롬프트를 전달하여 에이전트를 생성합니다. 도구 호출을 지원하는 모델이 필요하며, Anthropic, OpenAI 등 다양한 모델 프로바이더를 사용할 수 있습니다.

```
from deepagents import create_deep_agent

agent = create_deep_agent(model=model)
print(f"\u2713 에이전트 생성 완료 (타입: {type(agent).__name__})")
```

✓ 에이전트 생성 완료 (타입: `CompiledStateGraph`)

빌트인 도구만으로 충분하지 않을 때, 커스텀 도구를 추가하여 에이전트의 능력을 확장할 수 있습니다.

5.3 커스텀 도구 추가

Python 함수에 docstring과 타입 힌트를 작성하면 그대로 도구가 됩니다.

커스텀 도구는 일반 Python 함수로 작성하며, 다음 두 가지가 자동으로 변환됩니다:

- docstring → 도구 설명 (에이전트가 도구의 용도를 이해하는 데 사용)
- 타입 힌트 → 파라미터 스키마 (에이전트가 올바른 인자를 전달하는 데 사용)

`create_deep_agent()` 의 `tools` 파라미터에 함수 리스트를 전달하면, 빌트인 도구를 대체하는 것이 아니라 추가됩니다. 에이전트는 같은 실행에서 커스텀 도구와 빌트인 파일/계획 도구를 함께 사용할 수 있습니다.

`system_prompt` 파라미터는 에이전트가 모든 도구를 어떻게 활용할지 안내합니다 – 항상 계획을 먼저 세우도록 하거나, 특정 도구를 선호하도록 지시할 수 있습니다.

 TIP

단순한 Q&A 에이전트에 도구 하나둘만 필요하다면 2장의 `create_agent()` 가 더 단순하고 오버헤드가 적습니다. 여러 단계, 대량 데이터 처리, 자동 계획이 필요한 작업에는 Deep Agents가 설정 시간을 크게 절약합니다.

요약

핵심 API	역할
<code>create_deep_agent(model)</code>	빌트인 도구가 포함된 에이전트 생성
<code>create_deep_agent(model, tools, system_prompt)</code>	커스텀 도구 + 시스템 프롬프트
<code>agent.invoke()</code>	에이전트 실행

지금까지 LangChain, LangGraph, Deep Agents를 각각 살펴보았습니다. 다음 장에서는 세 프레임워크를 나란히 비교하여 어떤 상황에서 어떤 것을 선택해야 하는지 정리합니다.

06 세 프레임워크 비교 & 다음 단계

각 프레임워크를 개별적으로 학습한 후, 자연스럽게 떠오르는 질문은 “어떤 것을 사용해야 하나?”입니다. 정답은 프로젝트의 복잡도, 팀 규모, 일정에 따라 달라집니다. 이 장에서는 단순한 기능 비교를 넘어, 구체적인 시나리오 기반의 의사결정 프레임워크를 제공합니다.

학습 목표

LEARNING OBJECTIVES

- LangChain, LangGraph, Deep Agents 세 프레임워크의 핵심 차이점을 이해한다
- 각 프레임워크의 적합한 사용 사례를 판단할 수 있다
- 중급 과정으로의 학습 경로를 선택할 수 있다

핵심 질문

이 문제는 결국 하나로 귀결됩니다. “지금 내 문제에 필요한 것이 빠른 시작인지, 정밀한 제어인지, 아니면 계획·파일·서브에이전트까지 포함한 작업형 하네스인지?”

한눈에 보는 핵심

- LangChain은 빠른 시작과 단순한 도구 호출에 가장 가볍습니다.
- LangGraph는 상태 전이, 분기, 병렬 처리, interrupt/resume 같은 정밀 제어에 강합니다.
- Deep Agents는 계획, 파일시스템, 메모리, 서브에이전트가 필요한 복합 작업에 잘 맞습니다.

6.1 프레임워크 비교

추상화 수준	높음	중간	매우 높음
핵심 개념	에이전트 + 도구	그래프 + 상태 + 노드	올인원 에이전트
에이전트 생성	<code>create_agent()</code>	<code>StateGraph</code> → <code>compile()</code>	<code>create_deep_agent()</code>
실행	<code>agent.invoke()</code>	<code>graph.invoke()</code>	<code>agent.invoke()</code>

커스터마이징	도구/프롬프트/메모리	노드/엣지/상태/리듀서	도구/백엔드/서브에이전트
적합 상황	빠른 프로토타이핑	복잡한 워크플로	파일/태스크 관리가 필요한 에이전트

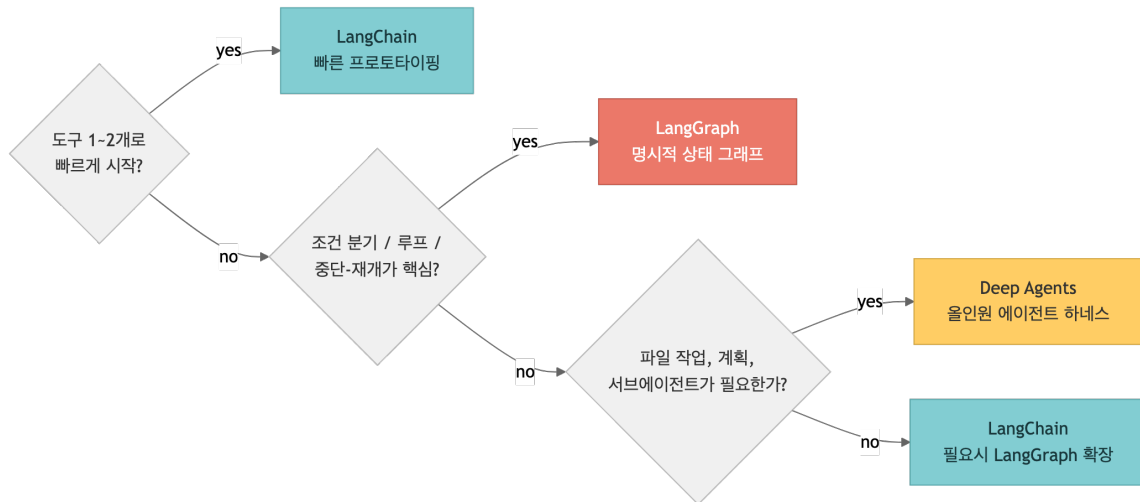
추가 비교 정보:

특성	LangChain	LangGraph	Deep Agents
모델 지원	모델 무관 (100개 이상 프로바이더)	LangChain 모델 공유	LangChain 모델 공유
라이선스	MIT	MIT	MIT
샌드박스 통합	기본 지원 없음	기본 지원 없음	에이전트가 샌드박스에서 작업 실행 가능
상태 관리	메모리 기반	체크포인트 기반 (타임 트래블 지원)	LangGraph 체크포인트 활용
관측성	LangSmith 연동	LangSmith 네이티브 트래이싱	LangSmith 지원

세 프레임워크는 서로 배타적이지 않습니다. Deep Agents는 내부적으로 LangGraph를 사용하고, LangChain의 모델/도구 인터페이스를 공유합니다. 따라서 LangChain으로 기본기를 다지고, LangGraph로 복잡한 워크플로를 설계한 뒤, Deep Agents로 프로덕션급 에이전트를 구축하는 것이 자연스러운 학습 경로입니다.

표만으로는 선택이 어려울 수 있습니다. 구체적인 시나리오별로 어떤 프레임워크가 적합한지 살펴봅시다.

6.2 어떤 걸 선택해야 할까?



의사결정 트리를 읽는 가장 쉬운 방법은 “어디까지 직접 제어하고 싶은가?” 를 먼저 묻는 것입니다. 도구 호출 몇 개와 프롬프트 조합만으로 충분하면 LangChain, 상태 전이와 재개 시점을 직접 설계해야 하면 LangGraph, 파일/계획/서브에이전트가 기본 탑재된 작업형 에이전트가 필요하면 Deep Agents가 자연스럽게 됩니다.

· NOTE

3줄 규칙: **빠르게 시작**이면 LangChain, **분기/루프/HITL**이 핵심이면 LangGraph, **코딩·리서치·파일 작업**까지 한 번에 다뤄야 하면 Deep Agents를 우선 검토하세요.

"간단한 도구 호출 에이전트가 필요해" → LangChain
 "조건 분기·루프가 있는 워크플로가 필요해" → LangGraph
 "파일 조작 + 계획 기능까지 한 번에" → Deep Agents

LangChain 시나리오: 주문 상태 조회와 FAQ 응답이 필요한 고객 지원 챗봇을 만들고 있습니다. 워크플로는 단순합니다 – 사용자가 질문하면, 에이전트가 적절한 도구를 호출하고, 결과를 반환합니다. `create_agent()` 에 도구 두 개와 시스템 프롬프트만 있으면 20줄 미만의 코드로 완성됩니다.

LangGraph 시나리오: 문서 검토 파이프라인을 구축하고 있습니다. 문서를 분류하고, 유형에 따라 다른 검토 큐로 라우팅하고, LLM이 검토한 뒤, 승인하거나 인간 검토자에게 에스컬레이션해야 합니다. 조건 분기, 휴먼 인 더 루프 일시 중지, 서로 다른 처리 경로 — `StateGraph`의 자연스러운 활용 사례입니다.

Deep Agents 시나리오: 웹을 검색하고, 여러 문서를 읽고, 노트를 작성하고, 접근 방식을 계획하고, 구조화된 보고서를 생성하는 리서치 어시스턴트를 만들고 있습니다. 빌트인 계획(`write_todos`), 파일 관리(`write_file` / `read_file`), 서브에이전트 위임이 각 단계를 수동으로 조율하지 않아도 되게 해줍니다.

· NOTE

언제 쓰고, 언제 멈출까?

- **LangChain**은 빠르게 시작하는 단일 에이전트와 비교적 단순한 도구 호출에 적합합니다. 반대로 조건 분기와 장기 실행 상태를 직접 설계해야 한다면 한 단계 아래인 **LangGraph**가 더 낫습니다.
- **LangGraph**는 결정론적 단계와 에이전트 단계를 함께 다루는 복잡한 워크플로에 적합합니다. 하지만 직선형 흐름만 있는 문제를 굳이 그래프로 쪼개면 코드와 운영 비용이 함께 늘어납니다.
- **Deep Agents**는 계획, 파일시스템, 서브에이전트, 컨텍스트 관리가 필요한 복합 작업에 강합니다. 단순 FAQ나 짧은 도구 호출 문제라면 오히려 하네스의 오버헤드가 과할 수 있습니다.

프레임워크	이럴 때 쓴다	이럴 때는 과하다
LangChain	빠른 시작, 표준 모델/도구 추상화, 복잡한 오케스트레이션이 없는 에이전트	상태 전이, durable execution, 병렬/조건부 그래프가 핵심인 경우
LangGraph	세밀한 제어, 장기 실행, 결정론적 단계 + 에이전트 단계 혼합, 사람 개입	단순 ReAct 루프만 있거나 선형 절차만 필요한 경우
Deep Agents	계획 수립, 파일 작업, 서브에이전트, 장시간 자율 작업, 컨텍스트 격리	짧고 단순한 도구 호출 문제, 중간 컨텍스트를 메인 에이전트가 계속 직접 봐야 하는 경우

→ TIP

세 프레임워크는 함께 사용하도록 설계되었습니다. 공통 패턴은 LangChain의 모델/도구 인터페이스를 기반으로, LangGraph로 정밀한 워크플로를 제어하고, Deep Agents로 고수준 에이전트 작업을 처리하는 것입니다. Deep Agents 에이전트를 LangGraph 워크플로의 노드로 사용할 수도 있습니다.

이론적 비교를 넘어, 동일한 작업을 세 가지 방식으로 구현한 코드를 직접 비교해 봅시다.

6.3 코드 비교 — 같은 질문, 세 가지 방식

아래는 동일한 작업을 세 프레임워크로 처리하는 최소 코드입니다.

LangChain

```
from langchain.agents import create_agent
from langchain.tools import tool

@tool
def add(a: int, b: int) -> int:
    """Add two numbers."""
    return a + b

agent = create_agent(model=model, tools=[add])
agent.invoke({"messages": [{"role": "user", "content": "3+4?"}]})
```

LangGraph

```
from langgraph.graph import StateGraph, START, END, MessagesState

def chatbot(state):
    return {"messages": [model.invoke(state["messages"])]}

builder = StateGraph(MessagesState)
builder.add_node("chat", chatbot)
builder.add_edge(START, "chat")
builder.add_edge("chat", END)
graph = builder.compile()
graph.invoke({"messages": [{"role": "user", "content": "3+4?"}]})
```

Deep Agents

```
from deepagents import create_deep_agent

agent = create_deep_agent(model=model)
agent.invoke({"messages": [{"role": "user", "content": "3+4?"}]})
```

이 단순한 예제에서는 세 접근 방식 모두 같은 결과를 생성하지만, 복잡도가 증가하면 차이가 극적으로 벌어집니다. LangChain의 `create_agent()` 는 단순함을 유지하지만 내부가 블랙박스입니다. LangGraph는 완전한 제어권을 주지만 더 많은 코드가 필요합니다. Deep Agents는 복잡성을 자동으로 처리하지만, 내부에서 정확히 무슨 일이 일어나는지 파악하기 어려울 수 있습니다.

요약

항목	LangChain	LangGraph	Deep Agents
핵심 역할	에이전트 생성 (ReAct 루프)	워크플로 오케스트레이션 (상태 그래프)	올인원 에이전트 (빌트인 도구)
상태 관리	미들웨어 기반	StateGraph 명시적 상태	자동 (파일시스템 + 메모리)
적합 대상	빠른 프로토타이핑, 도구 호출	복잡한 워크플로, 조건 분기	코딩 에이전트, 데이터 분석
학습 곡선	낮음	중간	낮음

미니 프로젝트

→ [07_mini_project.ipynb](#): 검색 + 요약 에이전트를 만드는 실습

중급 과정

과정	설명	노트북 수
LangChain 중급	모델 설정, 고급 도구 패턴(ToolRuntime, 상태 접근, 스트림 라이터), 전처리/후처리 미들웨어, 멀티에이전트 아키텍처	10
LangGraph 중급	복합 조건 라우팅, <code>interrupt()</code> 를 사용한 휴먼 인 더 루프, 모듈형 서브 그래프, 데이터베이스 체크포인트를 활용한 영속적 상태, 재시도 전략	10
Deep Agents 중급	커스텀 백엔드, 고급 서브에이전트 패턴, 대화 간 영속 메모리, 샌드박스 실행, 프로덕션 배포	7

권장 학습 순서:

1. LangChain – 모델 설정, 도구 패턴, 미들웨어의 기본기를 다진다
2. LangGraph – 조건 분기, 루프, 인간 검토 등 정밀한 그래프 기반 워크플로를 설계한다
3. Deep Agents – 계획, 파일 관리, 서브에이전트를 활용한 프로덕션급 에이전트를 구축한다

07 미니 프로젝트

검색 + 요약 에이전트

이 장은 앞선 여섯 개 장의 모든 내용을 하나의 실전 프로젝트로 종합합니다. 0장의 모델 초기화, 1장의 메시지 시스템, 2장의 도구 정의 패턴, 3장의 스트리밍 관찰, 5장의 Deep Agents 프레임워크를 활용하여, 웹에서 정보를 검색하고 구조화된 요약을 생성하는 리서치 에이전트를 만듭니다.

학습 목표

LEARNING OBJECTIVES

- Tavily 검색 도구를 직접 정의한다
- Deep Agents로 리서치 에이전트를 만든다
- 스트리밍으로 에이전트 실행 과정을 실시간 관찰한다
- LangChain 에이전트로도 같은 작업을 수행하여 비교한다

7.1 환경 설정

이 노트북에는 `TAVILY_API_KEY` 가 필요합니다. <https://tavily.com> 에서 무료로 발급받을 수 있습니다.

```
from dotenv import load_dotenv
import os

load_dotenv(override=True)

assert os.environ.get("OPENAI_API_KEY"), "OPENAI_API_KEY 필요!"
assert os.environ.get("TAVILY_API_KEY"), "TAVILY_API_KEY 필요!"

from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
print("\u2713 환경 준비 완료")
```

✓ 환경 준비 완료

7.2 검색 도구 정의

Tavily 클라이언트를 래핑하는 검색 함수를 만듭니다. docstring과 타입 힌트가 에이전트에 도구 스키마를 알려줍니다.

도구 함수 작성 규칙:

`create_deep_agent()`의 `tools` 파라미터에 전달할 검색 함수를 정의합니다. Deep Agents는 함수의 docstring을 도구 설명으로, 타입 힌트를 파라미터 스키마로 자동 변환합니다. 따라서:

- docstring: 에이전트가 “이 도구를 언제 사용해야 하는지” 판단하는 근거가 됩니다. 명확하고 구체적으로 작성하세요.
- 타입 힌트: 에이전트가 올바른 타입의 인자를 전달하도록 합니다. `Literal` 타입을 사용하면 허용 값을 제한할 수 있습니다.
- Args 섹션: 각 파라미터의 용도를 설명하면 에이전트가 더 정확하게 인자를 선택합니다. 없으면 에이전트가 `topic`의 의미를 잘못 추측할 수 있습니다.

이 패턴 — 제약된 타입, 합리적 기본값, 상세한 docstring — 은 도구 설계의 모범 사례입니다.

`Literal["general", "news"]` 타입 힌트는 에이전트가 유효한 값만 전달하도록 강제하고, `max_results: int = 3` 기본값은 에이전트가 특별히 더 많은 결과가 필요하지 않으면 이 매개변수를 지정할 필요가 없음을 의미합니다.

```
from typing import Literal
from tavily import TavilyClient

tavily = TavilyClient(api_key=os.environ["TAVILY_API_KEY"])

def internet_search(
    query: str,
    max_results: int = 3,
    topic: Literal["general", "news"] = "general",
) -> dict:
    """인터넷에서 정보를 검색합니다.

    Args:
        query: 검색 쿼리
        max_results: 최대 결과 수
        topic: 검색 주제 카테고리
    """
    return tavily.search(query, max_results=max_results, topic=topic)

print("\u2713 검색 도구 준비 완료")
```

✓ 검색 도구 준비 완료

검색 도구가 준비되었습니다. 이제 이 도구를 사용할 Deep Agents 에이전트를 만듭니다.

7.3 Deep Agents 리서치 에이전트

`create_deep_agent()`에 검색 도구와 시스템 프롬프트를 전달합니다. 시스템 프롬프트는 에이전트의 행동을 결정적으로 좌우합니다. “한국어로 요약하세요”도 좋지만, “3개의 핵심 포인트로 정리하세요”처럼 출력 형식을 구체적으로 지정하면 더 일관된 결과를 얻습니다. “검색 결과가 불충분하면 추가 검색을 수행하세요”와 같은 행동 지침을 추가하면 에이전트가 반복 검색을 통해 더 철저한 리서치를 수행합니다.

에이전트의 자동 워크플로:

에이전트는 사용자의 요청을 받으면 다음과 같은 과정을 자동으로 수행합니다:

1. 계획 수립: 빌트인 `write_todos` 도구로 작업을 단계별로 분해합니다.
2. 리서치 수행: 전달된 검색 도구(`internet_search`)를 사용하여 웹에서 정보를 수집합니다.
3. 컨텍스트 관리: 필요 시 파일 시스템 도구(`write_file` , `read_file`)로 중간 결과를 저장하여 토큰 한도를 관리합니다.
4. 결과 종합: 수집한 정보를 분석하고 일관된 보고서로 종합합니다.

복잡한 작업의 경우, 에이전트는 전문 서브에이전트를 생성하여 특정 하위 작업의 컨텍스트를 격리할 수도 있습니다.

```
from deepagents import create_deep_agent

research_agent = create_deep_agent(
    model=model,
    tools=[internet_search],
    system_prompt="당신은 전문 리서처입니다. 웹을 검색한 후 결과를 한국어로 요약하세요.",
)
print("\u2713 리서치 에이전트 생성 완료")
```

✓ 리서치 에이전트 생성 완료

에이전트를 실행하면서 내부에서 어떤 일이 벌어지는지 스트리밍으로 관찰해 봅시다.

7.4 스트리밍으로 과정 관찰

`stream(mode="updates")` 로 에이전트가 어떤 단계를 거치는지 실시간으로 확인합니다. 스트리밍 출력에서는 다음과 같은 과정을 관찰할 수 있습니다: 먼저 에이전트의 “사고” 단계 – 바로 검색할지, 계획을 세울지 판단합니다. 이어서 도구 호출이 나타나며, 에이전트가 선택한 쿼리로 `internet_search` 가 실행됩니다. 검색 결과가 돌아오면, 에이전트는 정제된 쿼리로 추가 검색을 수행할 수도 있습니다. 마지막으로 모든 결과를 종합하여 응답을 생성합니다. 각 단계는 노드 이름(예: “agent”, “tools”)과 함께 별개의 `updates` 이벤트로 나타납니다.

LangGraph 스트리밍 시스템:

LangGraph는 완전한 응답이 준비되기 전에 진행 상황을 점진적으로 표시하여 애플리케이션의 반응성을 높이는 포괄적인 스트리밍 시스템을 제공합니다.

스트림 모드	용도
<code>values</code>	각 그래프 단계 후 전체 상태를 스트리밍
<code>updates</code>	각 단계 후 상태 변경분만 스트리밍
<code>messages</code>	LLM 토큰을 메타데이터와 함께 스트리밍
<code>custom</code>	노드에서 사용자 정의 데이터를 스트리밍
<code>debug</code>	포괄적인 실행 정보를 스트리밍

`stream()` (동기) 또는 `astream()` (비동기) 메서드로 스트리밍에 접근하며, 여러 모드를 리스트로 전달하여 동시에 사용할 수도 있습니다. 아래 예제에서는 `updates` 모드를 사용하여 에이전트의 각 단계(도구 호출, 최종 응답)를 실시간으로 출력합니다.

같은 작업을 LangChain의 `create_agent()` 로도 수행하여, 두 접근 방식의 차이를 직접 비교합니다.

7.5 LangChain 에이전트로 비교

같은 검색 도구를 LangChain `create_agent()` 로도 사용해 봅니다. `@tool` 데코레이터를 사용하면 LangChain의 도구 인터페이스에 맞게 함수를 변환할 수 있습니다.

실행 결과를 비교하면 실질적인 차이를 관찰할 수 있습니다. LangChain 에이전트는 검색 도구를 호출하고 바로 요약합니다 — 계획을 세우거나 파일을 관리하지 않습니다. 반면 Deep Agents 에이전트는 투두 리스트를 생성하고, 여러 번 검색하고, 중간 노트를 파일에 기록하여 더 철저한 결과를 생성할 수 있습니다.

트레이드오프는 명확합니다: Deep Agents는 더 철저하지만 더 많은 토큰을 소비합니다(비용이 높아집니다). LangChain은 단순한 쿼리에 더 빠르고 저렴합니다. 이 특정 작업(단일 주제 리서치)에서는 차이가 작을 수 있지만, 복잡한 멀티 토픽 리서치에서는 Deep Agents의 계획 능력이 큰 차이를 만듭니다.

요약

이 미니 프로젝트에서 사용한 기술:

기술	출처
<code>ChatOpenAI + load_dotenv</code>	00_setup
메시지 역할, 스트리밍	01_llm_basics
<code>@tool</code> , <code>create_agent()</code>	02_langchain_basics
<code>InMemorySaver</code> , <code>thread_id</code>	03_langchain_memory
<code>StateGraph</code> , <code>compile()</code>	04_langgraph_basics
<code>create_deep_agent()</code>	05_deep_agents_basics

도전 과제

이 프로젝트를 확장해 보세요:

1. 계산기나 날짜 유틸리티 같은 두 번째 도구를 추가하고, 두 도구가 모두 필요한 리서치를 요청해 보세요.
2. `InMemorySaver` 를 추가하여 에이전트가 이전 리서치 세션을 기억하도록 만들어 보세요.
3. 시스템 프롬프트를 수정하여 마크다운 형식(섹션, 불릿 포인트, 출처 링크 포함)의 보고서를 생성하도록 해 보세요.

축하합니다! 입문 과정을 완료했습니다. 이제 6장에서 안내한 중급 과정으로 넘어가, 각 프레임워크의 고급 기능을 깊이 있게 학습할 수 있습니다.

P A R T

II

LangChain

Building Blocks for AI Applications

이 파트에서 다루는 내용

- 1. LangChain 소개
- 2. 첫 번째 에이전트
- 3. 모델과 메시지 시스템
- 4. 도구와 구조화된 출력
- 5. 메모리와 스트리밍
- 6. 미들웨어와 가드레일
- 7. 사람 개입과 런타임
- 8. 멀티 에이전트 패턴
- 9. 커스텀 워크플로와 RAG
 - 10. 프로덕션
 - 11. MCP
- 12. 프론트엔드 스트리밍
 - 13. 가드레일

01 LangChain 소개

Part I에서 `create_agent()` 를 간단히 사용해 봤다면, 이제 그 내부를 깊이 이해할 차례입니다. LangChain v1은 100개 이상의 LLM 프로바이더와 도구를 10줄 미만의 코드로 연결할 수 있는 통합 프레임워크로, 에이전트 개발의 기반 레이어 역할을 합니다. 이 장에서는 프레임워크의 전체 구조, 핵심 설계 철학, 그리고 v1에서 달라진 주요 사항을 살펴봅니다.

학습 목표

LangChain 프레임워크의 구조와 핵심 컴포넌트를 이해합니다.

이 장을 완료하면 다음을 이해할 수 있습니다:

- LangChain v1 프레임워크의 3가지 레이어 구조
- ReAct 에이전트 패턴의 동작 방식
- 핵심 컴포넌트와 주요 API
- 개발 환경 설정 및 검증 방법

1.1 LangChain 프레임워크 개요

LangChain v1은 LLM 기반 에이전트를 구축하기 위한 통합 프레임워크입니다. 3가지 레이어로 구성되어 있으며, 각 레이어는 서로 다른 수준의 추상화를 제공합니다.

3가지 레이어 구조

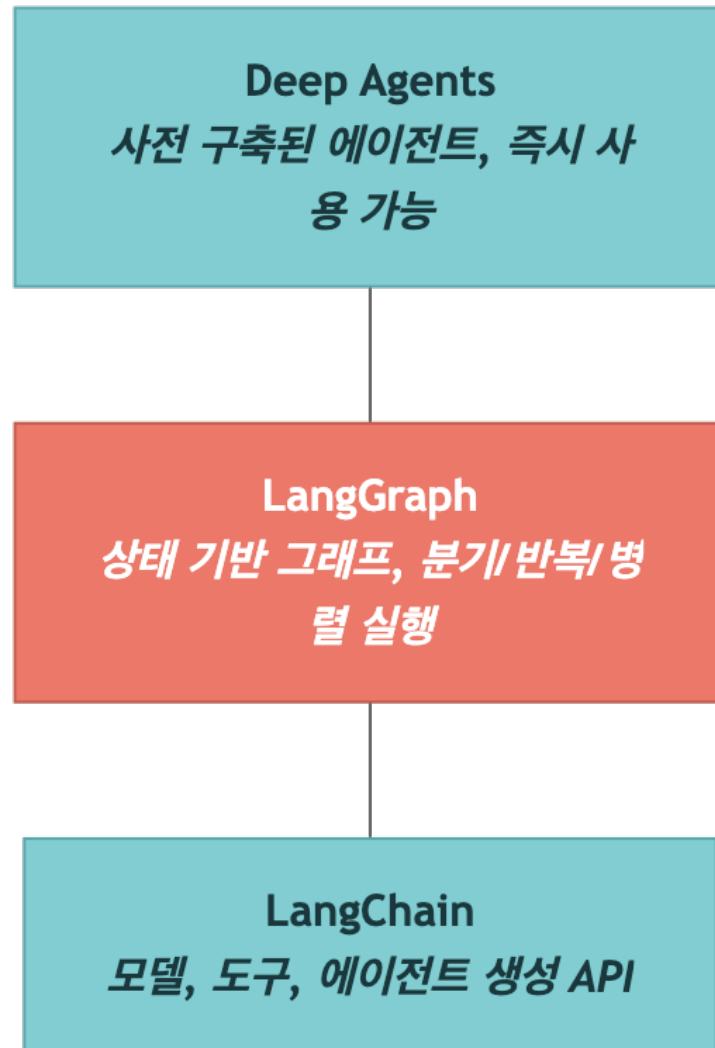


그림 읽는 법

위에서 아래로 읽으면 됩니다. **LangChain**은 개발자가 직접 만지는 상위 API, **LangGraph**는 실행과 상태 관리, **Deep Agents**는 그 위에 계획·파일·서브에이전트를 얹은 하네스입니다.

레이어	역할	대상 사용자
LangChain	에이전트 생성의 핵심 API (<code>create_agent</code> , <code>tool</code> , <code>ChatOpenAI</code>)	모든 개발자
LangGraph	복잡한 워크플로 구현 (상태 그래프, 체크포인트, 스트리밍)	중급 이상
Deep Agents	사전 구축된 에이전트 (코딩, 리서치 등)	빠른 프로토타이핑
LangSmith	어떤 프레임워크로 만든 에이전트든 추적·디버깅·평가하는 관측 플랫폼	모든 개발자 / 운영자

TIP

LangSmith는 LangChain·LangGraph·Deep Agents에 한정되지 않습니다. OpenAI SDK·LlamaIndex·자체 구현 에이전트까지 트레이스를 전송하면 동일한 UI로 디버깅·평가할 수 있어, v1부터는 네 번째 핵심 도구로 다룹니다.

LangChain v1에서 변경된 주요 사항

항목	이전 (v0.x)	현재 (v1)
에이전트 생성	<code>create_react_agent()</code>	<code>create_agent()</code>
에이전트 임포트	<code>from langchain.agents import ...</code> (다양)	<code>from langchain.agents import create_agent</code>
모델 초기화	<code>ChatOpenAI(...)</code> 직접 사용	<code>init_chat_model()</code> 또는 <code>ChatOpenAI(...)</code>
메모리	<code>ConversationBufferMemory</code> 등	<code>InMemorySaver</code> (LangGraph 체크포인트)
실행 엔진	<code>AgentExecutor</code>	LangGraph 그래프 (내부적으로)

핵심 설계 철학

LangChain v1의 핵심 설계 철학은 모든 에이전트가 LangGraph 그래프로 실행된다는 것입니다.

`create_agent()` 로 생성된 에이전트는 내부적으로 LangGraph의 `StateGraph` 로 구현되며, 이를 통해:

- 스트리밍: `stream()` 메서드로 실시간 응답
- 상태 관리: 체크포인트를 통한 대화 히스토리 유지
- 확장성: 커스텀 노드와 엣지 추가 가능

멀티프로바이더 지원

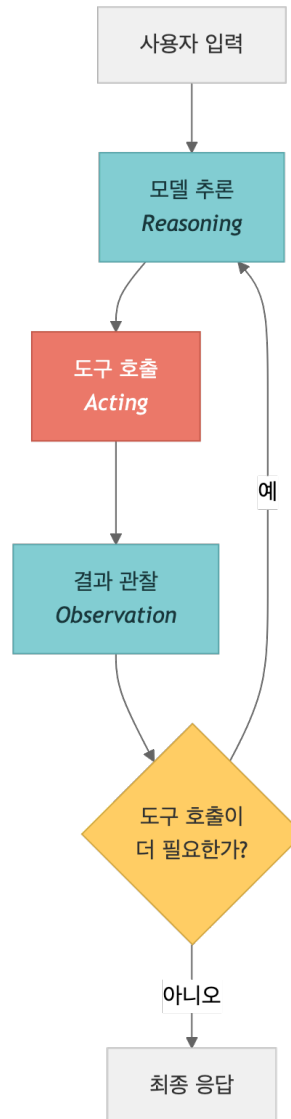
LangChain v1은 100여 개의 LLM 프로바이더를 동일한 인터페이스로 다룹니다. 본 책에서 자주 등장하는 모델 ID 예시는 다음과 같습니다:

- OpenAI: `openai:gpt-5.4` , `openai:gpt-4.1`
- Anthropic: `anthropic:claude-sonnet-4-6` , `anthropic:claude-opus-4-6`
- Google: `google:gemin-2.5-flash-lite` , `google:gemin-2.5-pro`
- OpenRouter: `openrouter:meta-llama/llama-3.3-70b-instruct` 등 100+ 모델
- AWS Bedrock: `bedrock:anthropic.claude-sonnet-4-6`
- Ollama: `ollama:llama3.3` (로컬 실행)

본 책의 예제는 별도 명시가 없는 한 OpenAI `gpt-5.4` 또는 Anthropic `claude-sonnet-4-6` 을 기본 모델로 사용합니다.

1.2 ReAct 에이전트 패턴

ReAct (Reasoning + Acting) 패턴은 LangChain v1 에이전트의 기본 동작 방식입니다. 에이전트는 다음과 같은 루프를 반복합니다:



내부적으로 에이전트는 LangGraph `StateGraph` 의 두 노드 – “model”과 “tools” – 로 구현됩니다. 루프의 각 단계는 다음과 같습니다:

1. **모델 호출**: 시스템 프롬프트, 대화 메시지, 사용 가능한 도구 목록을 LLM에 전달합니다. 모델은 최종 텍스트 응답(루프 종료) 또는 도구 호출 요청(`tool_calls` 가 포함된 `AIMessage`)을 반환합니다.
2. **도구 실행**: `ToolNode` 가 요청된 도구를 실행하고, 결과를 매칭되는 `tool_call_id` 와 함께 `ToolMessage` 로 대화에 추가합니다. 여러 도구가 요청되면 병렬로 실행됩니다.
3. **반복**: 모델이 충분한 정보를 얻어 텍스트로 응답하거나, 최대 반복 횟수에 도달할 때까지 1~2단계를 반복합니다.

이 흐름을 결정하는 핵심 라우팅 함수가 `tools_condition` 입니다. 모델의 응답에 `tool_calls` 가 있으면 “tools” 노드로, 없으면 `END` 로 라우팅합니다.

핵심 특징

- 자율적 판단: 에이전트가 도구 사용 여부를 스스로 결정합니다
- 다단계 추론: 복잡한 작업을 여러 단계로 분해하여 처리합니다

- 관찰 기반 학습: 도구 결과를 관찰하고 다음 행동을 결정합니다
- 병렬 도구 호출: 모델이 여러 도구를 동시에 요청하면 `ToolNode` 가 병렬로 실행합니다

1.3 주요 컴포넌트 개요

LangChain v1의 핵심 컴포넌트를 표로 정리합니다. 이 컴포넌트들은 이후 장에서 하나씩 깊이 다룹니다:

컴포넌트	설명	주요 API
Model	LLM 또는 Chat 모델. 에이전트의 “두뇌” 역할	<code>ChatOpenAI</code> , <code>init_chat_model()</code>
Tools	에이전트가 사용할 수 있는 함수. 검색, 계산, API 호출 등	<code>\@tool</code> 데코레이터, <code>TavilySearch</code>
Agent	모델과 도구를 결합한 실행 단위. 내부적으로 LangGraph 그래프	<code>create_agent()</code>
Memory	대화 히스토리를 저장하고 관리하는 체크포인트	<code>InMemorySaver</code> , <code>SqliteSaver</code>
Middleware	요청/응답 처리 파이프라인에 로직 삽입	<code>prompt</code> , <code>before_tool</code> , <code>after_model</code>
State	에이전트 실행 중 관리되는 상태 (메시지, 컨텍스트 등)	<code>AgentState</code> , <code>messages</code>
Streaming	실시간 응답 스트리밍 지원	<code>stream()</code> , <code>stream_mode="updates"</code>

1.4 환경 설정 및 설치 확인

LangChain v1 개발에는 4개의 핵심 패키지가 필요합니다: `langchain` (코어 프레임워크), `langchain-openai` (OpenAI 프로바이더), `langchain-community` (커뮤니티 통합), `langgraph` (실행 엔진). 이 패키지들이 정상적으로 설치되었는지 확인합니다.

```
# 환경 설정
import subprocess
import importlib

packages = {
    "langchain": "langchain",
    "langchain_openai": "langchain-openai",
    "langchain_community": "langchain-community",
    "langgraph": "langgraph",
}

print("=" * 50)
print("LangChain v1 환경 확인")
print("=" * 50)

for module_name, package_name in packages.items():
    try:
        mod = importlib.import_module(module_name)
        version = getattr(mod, "__version__", "installed")
        print(f"\u2713 {package_name}: {version}")
    except ImportError:
        print(f"\u2717 {package_name}: 미설치 \u2192 pip install {package_name}")
```

```
=====
LangChain v1 환경 확인
=====
✓ langchain: 1.2.10

✓ langchain-openai: installed
✓ langchain-community: 0.4.1
✓ langgraph: installed
```

```
# API 키 확인
from dotenv import load_dotenv
import os

load_dotenv(override=True)

required_keys = ["OPENAI_API_KEY"]
optional_keys = ["TAVILY_API_KEY", "LANGSMITH_API_KEY"]

print("필수 API 키:")
for key in required_keys:
    status = "\u2713 설정됨" if os.environ.get(key) else "\u2717 미설정"
    print(f" {key}: {status}")

print("\n선택 API 키:")
for key in optional_keys:
    status = "\u2713 설정됨" if os.environ.get(key) else "- 미설정 (선택)"
    print(f" {key}: {status}")
```

```
필수 API 키:
OPENAI_API_KEY: ✓ 설정됨

선택 API 키:
TAVILY_API_KEY: ✓ 설정됨
LANGSMITH_API_KEY: - 미설정 (선택)
```

```
# LangChain v1 핵심 import 확인
from langchain.agents import create_agent
from langchain.tools import tool
from langchain_openai import ChatOpenAI
from langgraph.checkpoint.memory import InMemorySaver

print("\u2713 모든 핵심 모듈 임포트 성공")
print(" - create_agent: LangChain v1 에이전트 생성")
print(" - tool: 도구 데코레이터")
print(" - ChatOpenAI: OpenAI 호출 모델")
print(" - InMemorySaver: 메모리 체크포인트")
```

```
✓ 모든 핵심 모듈 임포트 성공
- create_agent: LangChain v1 에이전트 생성
- tool: 도구 데코레이터
- ChatOpenAI: OpenAI 호출 모델
- InMemorySaver: 메모리 체크포인트
```

요약

항목	설명
LangChain v1	에이전트 중심 프레임워크, <code>create_agent()</code> API 기반

3계층 구조	모델 → 도구 → 미들웨어
ReAct 패턴	추론(Reasoning) → 행동(Action) → 관찰(Observation) 반복
핵심 API	<code>create_agent()</code> , <code>@tool</code> , <code>invoke()</code> , <code>stream()</code>

다음 장에서는 `create_agent()` 를 사용하여 도구, 메모리, 스트리밍이 포함된 완전한 에이전트를 직접 구축합니다.

02 첫 번째 에이전트

앞 장에서 LangChain v1의 아키텍처를 개괄적으로 살펴보았습니다. 이 장에서는 그 핵심 API인 `create_agent()`를 사용하여 도구, 메모리, 스트리밍이 포함된 완전한 에이전트를 처음부터 끝까지 구축합니다. Part I에서 다른 기본 패턴을 확장하여, 프로덕션에 가까운 에이전트의 기반을 만들어 봅니다.

`create_agent()`는 LangChain v1에서 에이전트를 만드는 단일 진입점(single entry point)입니다. 내부적으로 LangGraph의 `CompiledStateGraph`를 반환하며, 모델·도구·메모리·미들웨어를 하나의 실행 그래프로 결합합니다. 이 장에서는 가장 기본적인 매개변수(`model`, `tools`, `system_prompt`)부터 시작하여, 메모리(`checkpointers`)와 스트리밍까지 단계별로 확장해 나갑니다.

학습 목표

LangChain v1의 `create_agent()`로 에이전트를 생성하고 실행합니다.

이 장을 완료하면 다음을 수행할 수 있습니다:

- `@tool` 데코레이터로 커스텀 도구를 정의
- `create_agent()`로 에이전트를 생성
- `invoke()`로 에이전트를 실행하고 결과를 확인
- `stream()`으로 실시간 스트리밍 응답을 받기
- `InMemorySaver`로 멀티턴 대화를 구현

2.1 환경 설정

이번 장의 예제를 실행하려면 LangChain v1과 Deep Agents 패키지를 먼저 설치합니다. 다음 명령 중 환경에 맞는 하나를 선택하세요:

```
# uv 사용 시
uv add langchain deepagents

# pip 사용 시
pip install -U langchain deepagents
```

TIP

`langchain`을 설치하면 `langchain-core`, `langgraph`, `langchain-openai` (OpenAI 통합)가 함께 설치됩니다. Anthropic·Google 등 다른 프로바이더는 `langchain-anthropic`, `langchain-google-genai`를 별도로 추가하세요.

본격적인 에이전트 구축에 앞서, 모델 객체를 준비합니다. `create_agent()` 의 `model` 매개변수는 `ChatModel` 인스턴스뿐 아니라 `"openai:gpt-5.4"` 같은 프로바이더:모델명 문자열도 받을 수 있습니다. 여기서는 명시적으로 `ChatOpenAI` 인스턴스를 생성하여 사용합니다.

OpenAI를 통해 모델을 설정합니다. `ChatOpenAI` 는 OpenAI 호환 API를 지원하므로, `base_url` 을 변경하여 OpenAI를 사용할 수 있습니다.

```
from dotenv import load_dotenv
import os

load_dotenv(override=True)

from langchain_openai import ChatOpenAI

model = ChatOpenAI(
    model="gpt-5.4",
)
print("\u2713 모델 설정 완료:", model.model_name)
```

✓ 모델 설정 완료: gpt-5.4

2.2 간단한 도구 만들기

모델이 준비되었으니, 에이전트가 호출할 수 있는 도구(tool)를 정의합니다. 도구는 에이전트가 외부 세계와 상호 작용하는 수단이며, LangChain은 `@tool` 데코레이터 하나로 일반 Python 함수를 에이전트용 도구로 변환합니다.

`@tool` 데코레이터로 에이전트가 사용할 도구를 정의합니다.

도구를 정의할 때 중요한 점:

- docstring은 필수입니다. 에이전트가 도구의 용도를 이해하는 데 사용됩니다.
- 타입 힌트를 사용하면 에이전트가 올바른 인자를 전달할 수 있습니다.
- 도구 이름은 함수명에서 자동으로 생성됩니다.

```
from langchain.tools import tool

@tool
def add(a: int, b: int) -> int:
    """두 수를 더합니다."""
    return a + b

@tool
def multiply(a: int, b: int) -> int:
    """두 수를 곱합니다."""
    return a * b

print("도구 목록:")
for t in [add, multiply]:
    print(f" - {t.name}: {t.description}")
```

도구 목록:
 - add: 두 수를 더합니다.
 - multiply: 두 수를 곱합니다.

2.3 에이전트 생성

도구가 준비되었으면, 이제 모델과 도구를 하나의 에이전트로 결합합니다. `create_agent()` 는 다음과 같은 전체 시그니처를 가집니다:

```
create_agent(model, tools, system_prompt, name, response_format, state_schema, checkpoint, store,
             middleware, context_schema)
```

이 중 `model` 과 `tools` 만 필수이며, 나머지는 모두 선택입니다. `system_prompt` 는 에이전트의 행동 지침을 설정하고, `checkpoint` 는 대화 상태 저장, `middleware` 는 실행 파이프라인 혹은 담당합니다. 이 장에서는 기본 세 가지(`model` , `tools` , `system_prompt`)만 사용하고, 나머지 매개변수는 이후 장에서 하나씩 추가합니다.

생성된 에이전트는 내부적으로 LangGraph 그래프로 구현되며, `invoke()` , `stream()` 등의 메서드를 제공합니다.

TIP

LangChain v1에서는 `create_react_agent()` 대신 `create_agent()` 를 사용합니다.

TIP

에이전트에 `name="research_assistant"` 처럼 snake_case 이름을 주면 LangSmith 트레이스·서브에이전트 라우팅·로그에서 식별이 쉬워집니다. 공백·하이픈·대문자는 피하고, 의미 있는 짧은 이름을 권장합니다.

TIP

`state_schema` 를 직접 정의할 때는 반드시 `langgraph.graph.MessagesState` 또는 `langchain.agents.AgentState` 를 상속한 `TypedDict` 서브클래스여야 합니다. 일반 `dataclass` 나 `Pydantic BaseModel` 은 허용되지 않습니다.

```
from langchain.agents import create_agent

agent = create_agent(
    model=model,
    tools=[add, multiply],
    system_prompt="당신은 수학 도우미입니다. 제공된 도구를 사용하여 계산하세요.",
)
print("\u2713 에이전트 생성 완료")
print(f"타입: {type(agent).__name__}")
```

✓ 에이전트 생성 완료
타입: CompiledStateGraph

2.4 에이전트 실행

에이전트가 생성되었으니, 실제로 질문을 던져 결과를 확인해 봅니다. `invoke()` 는 동기 호출 메서드로, 에이전트가 모든 추론과 도구 호출을 완료한 뒤 최종 결과를 한 번에 반환합니다.

`invoke()`의 반환값은 `dict`이며, 핵심 키는 `"messages"`입니다. 이 리스트에는 대화의 전체 흐름 — `HumanMessage` → `AIMessage` (도구 호출) → `ToolMessage` (도구 결과) → `AIMessage` (최종 응답) — 이 순서대로 담겨 있습니다. `response_format`을 설정한 경우 `"structured_response"` 키도 추가됩니다.

에이전트에 메시지를 전달하면, 내부적으로 ReAct 루프가 실행됩니다:

1. 모델이 질문을 분석하고 도구 호출을 결정
2. 도구가 실행되고 결과를 반환
3. 모델이 결과를 바탕으로 최종 응답을 생성

! WARNING

메모리를 사용하지 않는 에이전트에서는 `invoke()`에 `config`를 전달하지 않아도 됩니다. 하지만 `InMemorySaver`를 사용하는 경우, 반드시 `{"configurable": {"thread_id": "..."} }` 형태의 `config`를 함께 전달해야 합니다. 이 부분은 2.6절에서 다룹니다.

```
# 전체 메시지 흐름 확인
print("전체 메시지 흐름:")
print("=" * 50)
for msg in result["messages"]:
    role = msg.type if hasattr(msg, 'type') else msg.get('role', 'unknown')
    content = msg.content if hasattr(msg, 'content') else msg.get('content', '')
    print(f"[{role}] {content[:200]}")
print("-" * 50)
```

전체 메시지 흐름:

```
=====
[human] 15 + 27은 얼마인가요?
-----
[ai]
-----
[tool] 42
-----
[ai] 15 + 27은 42입니다.
-----
```

2.5 스트리밍 실행

`invoke()`가 결과를 한 번에 반환하는 반면, `stream()`은 에이전트의 실행 과정을 단계별로 실시간 전달합니다. 챗봇 UI처럼 사용자에게 즉각적인 피드백을 제공해야 하는 경우에 필수적입니다.

`stream()`으로 실시간 응답을 받습니다.

스트리밍을 사용하면 에이전트의 각 단계(모델 추론, 도구 호출, 최종 응답)를 실시간으로 확인할 수 있습니다.

`stream_mode="updates"`를 사용하면 각 노드의 업데이트를 순차적으로 받을 수 있습니다. 호출 형태는 `agent.stream(input, config, stream_mode="updates")`이며, 이터레이터를 반환합니다. 각 이터레이션에서 어떤 노드(모델 또는 도구)가 어떤 출력을 생성했는지 확인할 수 있습니다.

➡ TIP

LangChain v1은 `values`, `updates`, `messages`, `custom`, `debug` 등 5가지 스트리밍 모드를 지원합니다. 이 장에서는 가장 직관적인 `updates` 모드를 사용하며, 전체 스트리밍 모드는 5장에서 상세히 다룹니다.

2.6 멀티턴 대화

지금까지의 에이전트는 매 호출이 독립적이었습니다. 즉, 이전 대화 내용을 기억하지 못합니다. 실제 어시스턴트라면 “아까 계산한 결과에 5를 곱해줘” 같은 후속 질문을 처리할 수 있어야 합니다. 이를 위해 체크포인트를 추가합니다.

`InMemorySaver` 로 대화 상태를 유지합니다.

`InMemorySaver` 는 메모리 내에서 상태를 저장하며, `thread_id` 로 대화 세션을 구분합니다. 에이전트 생성 시 `checkpointer=InMemorySaver()` 를 전달하고, `invoke()` 호출 시 `{"configurable": {"thread_id": "my-session"}}` 형태의 config를 함께 전달하면 됩니다. 동일한 `thread_id` 를 사용하는 한, 이전 대화의 모든 메시지가 자동으로 복원됩니다.

TIP

LangChain v1에서는 LangGraph의 체크포인트를 사용하여 대화 히스토리를 관리합니다.

! WARNING

`InMemorySaver` 는 프로세스 메모리에 상태를 저장하므로, 서버 재시작 시 데이터가 사라집니다. 프로덕션 환경에서는 `SqliteSaver` , `PostgresSaver` 등 영구 체크포인트를 사용하세요.

2.7 Tavily 검색 도구 연동 (선택)

지금까지는 산술 도구라는 단순한 예제를 사용했습니다. 이 절에서는 외부 API를 호출하는 실전형 도구를 연동하여, 에이전트가 실시간 정보에 접근할 수 있도록 합니다.

웹 검색 도구를 추가하여 실제 정보를 검색합니다.

Tavily는 AI 에이전트를 위해 설계된 검색 API입니다. 검색 결과를 LLM이 소비하기 좋은 형태로 반환하므로, RAG 파이프라인이나 에이전트의 검색 도구로 널리 사용됩니다.

`TAVILY_API_KEY` 가 설정된 경우에만 이 셀이 실행됩니다.

2.8 런타임 컨텍스트 (`context_schema`)

`thread_id` 가 대화 세션을 구분한다면, `context` 는 요청 단위 메타데이터(사용자 ID, 권한, 테넌트 등)를 도구·미들웨어에 주입하는 통로입니다. `create_agent()` 에 `context_schema=` 를 등록한 뒤, `invoke()` 호출 시 `context=` 로 값을 넘기면 도구의 `ToolRuntime.context` 로 접근할 수 있습니다.

```
from dataclasses import dataclass
from langchain.agents import create_agent

@dataclass
class Context:
    user_id: str
    tenant: str = "default"
```

```
agent = create_agent(
    model=model,
    tools=[add, multiply],
    system_prompt="...",
    context_schema=Context,
)

result = agent.invoke(
    {"messages": [{"role": "user", "content": "..."}]},
    context=Context(user_id="u-1234", tenant="acme"),
)
```

TIP

`context_schema` 는 4장의 `ToolRuntime` , 7장의 HITL과 짝을 이룹니다. 여기서는 “주입 통로가 있다”는 점만 기억하고, 구체적 활용은 4장과 7장에서 다룹니다.

요약

이 장에서는 `create_agent()` 의 핵심 매개변수 세 가지(`model` , `tools` , `system_prompt`)와 `checkpointner` 를 사용하여 완전한 에이전트를 구축했습니다. 이 노트북에서 다른 내용을 정리합니다:

주제	핵심 API	설명
도구 정의	<code>\@tool</code>	함수에 데코레이터를 추가하여 에이전트 도구로 변환
에이전트 생성	<code>create_agent()</code>	모델 + 도구 + 시스템 프롬프트를 결합
동기 실행	<code>agent.invoke()</code>	완전한 응답을 한 번에 반환
스트리밍 실행	<code>agent.stream()</code>	각 단계의 업데이트를 실시간으로 반환
멀티턴 대화	<code>InMemorySaver</code> + <code>thread_id</code>	체크포인트로 대화 상태를 저장/복원
검색 도구	<code>TavilySearch</code>	웹 검색을 통한 실시간 정보 접근

이 장에서는 `ChatOpenAI` 인스턴스를 직접 생성하여 모델을 사용했습니다. 다음 장에서는 `init_chat_model()` 을 통한 프로바이더 통합 초기화, 메시지 타입의 세부 속성, 그리고 멀티모달 입력까지 – 모델과 메시지 시스템의 전체 그림을 살펴봅니다.

03 모델과 메시지 시스템

에이전트의 성능은 기반 모델의 설정과 메시지 구성 방식에 크게 좌우됩니다. 이 장에서는 LangChain v1이 지원하는 다양한 LLM 프로바이더를 통합된 인터페이스로 관리하는 방법과, 메시지 타입별 특성을 깊이 있게 다룹니다. Part I의 1장에서 다룬 기초를 확장하여, 멀티모달 입력, 토큰 사용량 추적, 프로바이더 간 전환까지 학습합니다.

앞 장에서는 `ChatOpenAI` 를 직접 생성하여 에이전트에 전달했습니다. 그런데 프로젝트가 커지면 OpenAI에서 Anthropic으로, 또는 로컬 Ollama 모델로 전환해야 할 때가 옵니다. LangChain v1은 `init_chat_model()` 함수를 통해 프로바이더 문자열 하나로 모델을 초기화할 수 있으며, 런타임 config를 통한 동적 전환까지 지원합니다. 이 장에서는 그 메커니즘과 함께, 대화의 기본 단위인 메시지 객체의 구조를 상세히 파악합니다.

학습 목표

LEARNING OBJECTIVES

- LangChain v1의 모델 초기화 방법(`init_chat_model` , `ChatOpenAI`)을 이해합니다
- `invoke()` , `stream()` , `batch()` 세 가지 호출 패턴을 학습합니다
- `SystemMessage` , `HumanMessage` , `AIMessage` , `ToolMessage` 등 메시지 타입을 이해합니다
- 멀티모달 메시지(이미지 입력)를 구성하는 방법을 익힙니다

3.1 환경 설정

이 장 전체에서 사용할 모델 인스턴스를 준비합니다. `ChatOpenAI` 의 주요 매개변수는 다음과 같습니다: `model` (모델 이름), `temperature` (창의성 조절, 0 2), `max_tokens` (최대 출력 토큰), `timeout` (요청 타임아웃), `max_retries` (실패 시 자동 재시도 횟수, 기본값 6), `api_key` (API 키). 대부분의 경우 `model` 만 지정하면 나머지는 합리적인 기본값이 적용됩니다.

`.env` 파일에서 API 키를 로드하고, OpenAI를 통해 모델을 초기화합니다.

```
import os
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI

load_dotenv(override=True)

# OpenAI를 통한 모델 초기화
model = ChatOpenAI(
    model="gpt-5.4",
```

```
)
print("모델 초기화 완료:", model.model_name)
```

모델 초기화 완료: gpt-5.4

3.2 모델 프로바이더 비교

LangChain v1은 `init_chat_model()` 을 통해 다양한 프로바이더의 모델을 통합된 방식으로 초기화할 수 있습니다.

프로바이더	모델 문자열 형식	필요 패키지	환경 변수
OpenAI	"openai:gpt-5"	langchain-openai	OPENAI_API_KEY
Anthropic	"anthropic:claude-sonnet-4-6"	langchain-anthropic	ANTHROPIC_API_KEY
Google	"google:gemini-2.5-flash-lite"	langchain-google-genai	GOOGLE_API_KEY
AWS Bedrock	"bedrock:anthropic.claude-v3"	langchain-aws	AWS credentials
Azure	"azure:gpt-4o"	langchain-openai	AZURE_OPENAI_API_KEY
Ollama	"ollama:llama3"	langchain-ollama	(로컬 실행)

· NOTE

참고: OpenAI를 사용하는 경우, ChatOpenAI 에 `base_url` 과 `api_key` 를 직접 지정하여 OpenAI API 형식의 서비스(vLLM, LMStudio, Ollama 등)에 접근할 수 있습니다.

3.3 init_chat_model() 사용법

프로바이더 비교표를 확인했으니, 이제 실제로 `init_chat_model()` 을 사용하여 모델을 생성해 봅니다.

`init_chat_model()` 은 LangChain v1에서 제공하는 통합 모델 초기화 함수입니다. 프로바이더별 패키지가 설치되어 있으면, 문자열 하나로 모델을 생성할 수 있습니다.

이 함수의 핵심 장점은 프로바이더 자동 감지입니다. "openai:gpt-5.4" 처럼 `프로바이더:모델명` 형식의 문자열을 전달하면, LangChain이 자동으로 해당 프로바이더의 ChatModel 클래스를 찾아 인스턴스를 생성합니다. 더 나아가, `configurable_fields` 를 사용하면 런타임에 프로바이더와 모델을 동적으로 전환할 수도 있어, A/B 테스트나 비용 최적화에 유용합니다.

OpenAI를 사용하는 경우에는 ChatOpenAI 를 직접 사용하는 것이 더 간편합니다.

➡ TIP

`init_chat_model()` 은 `create_agent()` 의 `model` 매개변수에 직접 전달할 수 있습니다. 또는 `create_agent(model="openai:gpt-5.4", ...)` 처럼 문자열을 직접 전달하면 내부적으로 `init_chat_model()` 이 호출됩니다.

3.4 invoke(), stream(), batch() 패턴

모델 초기화 방법을 살펴봤으니, 이제 모델을 호출하는 세 가지 방법을 알아봅니다. LangChain v1의 모든 모델은 동일한 인터페이스를 따르므로, 프로바이더를 바꿔도 호출 코드는 그대로 유지됩니다.

LangChain v1의 모든 모델은 세 가지 호출 패턴을 지원합니다:

메서드	설명	반환 타입
<code>invoke()</code>	단일 입력에 대한 단일 응답	<code>AIMessage</code>
<code>stream()</code>	토큰 단위로 스트리밍 응답	<code>Iterator[AIMessageChunk]</code>
<code>batch()</code>	여러 입력을 동시에 처리	<code>List[AIMessage]</code>

`stream()` 이 반환하는 `AIMessageChunk` 는 부분 응답을 나타내며, `+` 연산자로 결합할 수 있습니다. 예를 들어, 스트리밍 중 받은 여러 chunk를 `chunk1 + chunk2 + ...` 로 합치면 최종 `AIMessage` 와 동일한 내용이 됩니다. 각 메서드에는 비동기 버전(`ainvoke()` , `astream()` , `abatch()`)도 있어 `asyncio` 기반 애플리케이션에서 사용할 수 있습니다.

3.5 메시지 타입

호출 패턴을 이해했으니, 이제 호출의 입력과 출력을 구성하는 메시지 객체를 자세히 살펴봅니다. LangChain v1의 메시지 시스템은 대화의 각 역할을 명확히 구분합니다:

메시지 타입	역할	설명
<code>SystemMessage</code>	시스템	모델의 행동 방식을 지시하는 시스템 프롬프트
<code>HumanMessage</code>	사용자	사용자가 입력하는 메시지
<code>AIMessage</code>	AI	모델이 생성한 응답
<code>ToolMessage</code>	도구	도구 실행 결과를 모델에 전달

메시지 리스트를 구성하여 `model.invoke()` 에 전달하면, 대화 맥락을 유지한 응답을 받을 수 있습니다.

각 메시지 타입은 고유한 속성을 가집니다. 특히 `AIMessage` 는 단순한 `content` 외에도 다음과 같은 유용한 메타데이터를 포함합니다:

- `tool_calls` : 모델이 요청한 도구 호출 목록 (4장에서 상세히 다룸)
- `usage_metadata` : 토큰 사용량 정보 (`input_tokens` , `output_tokens` , `total_tokens`)
- `response_metadata` : 프로바이더별 응답 메타데이터 (모델 버전, finish reason 등)

`ToolMessage` 는 도구 실행 결과를 모델에 전달하는 메시지입니다. `content` (결과 텍스트), `tool_call_id` (어떤 도구 호출에 대한 응답인지 식별), `name` (도구 이름)을 필수로 가지며, 선택적으로 `artifact` 필드를 통해 파일이나 이미지 같은 비텍스트 결과물을 첨부할 수 있습니다.

3.6 멀티모달 메시지 (이미지 입력)

텍스트 메시지를 넘어, 최신 모델들은 이미지·오디오·비디오·PDF 등 다양한 모달리티를 입력으로 받을 수 있습니다. LangChain v1은 이를 통합된 메시지 형식으로 지원합니다.

LangChain v1에서는 `HumanMessage` 의 `content` 에 텍스트와 이미지를 함께 전달할 수 있습니다. 이미지는 URL, base64 인코딩, 또는 파일 ID(일부 프로바이더) 중 하나로 전달하며, 비전(Vision)을 지원하는 모델에서만 동작합니다.

TIP

멀티모달 지원 범위는 프로바이더마다 다릅니다. OpenAI GPT-5.4는 이미지·PDF·오디오를, Google Gemini 2.5는 이미지·오디오·비디오를 지원합니다. 사용 전 해당 프로바이더의 문서를 확인하세요.

```
content = [
    {"type": "text", "text": "설명 텍스트"},
    {"type": "image_url", "image_url": {"url": "이미지_URL"}},
]
```

3.7 표준 출력 포맷 — `output_version="v1"`

LangChain v1부터 모든 프로바이더가 동일한 content blocks 표준 직렬화를 제공합니다.

`init_chat_model("openai:gpt-5.4", output_version="v1")` 처럼 명시하거나, 환경변수 `LC_OUTPUT_VERSION="v1"` 을 설정하면 `AIMessage.content_blocks` 속성으로 텍스트·도구 호출·추론(reasoning)·사고 과정·인용을 통일된 스키마로 받을 수 있습니다.

```
import os
from langchain.chat_models import init_chat_model

os.environ["LC_OUTPUT_VERSION"] = "v1"

model = init_chat_model("openai:gpt-5.4", output_version="v1")
result = model.invoke("LangChain v1을 한 줄로 설명하세요.")

for block in result.content_blocks:
    print(block["type"], "→", block.get("text", block))
```

스트리밍에서도 동일합니다. `chunk.content_blocks` 로 부분 블록을 받아 UI에 매핑할 수 있습니다.

```
for chunk in model.stream("멀티모달 입력을 설명해 줘."):
    for block in chunk.content_blocks:
        if block["type"] == "text":
            print(block["text"], end="")
```

TIP

`output_version="v1"` 은 프로바이더 간 응답 차이를 흡수하는 권장 설정입니다. 기존 코드가 `result.content` 문자열에 의존한다면 호환 모드(`v0`)로도 동작하지만, 새 프로젝트는 `v1`을 사용하세요.

3.8 토큰 사용량 추적 — UsageMetadataCallbackHandler

여러 단계에 걸친 에이전트 실행의 누적 토큰 사용량은 콜백 핸들러로 집계합니다. `UsageMetadataCallbackHandler` 를 `config={"callbacks": [...]}` 로 전달하면 모델 호출마다 `input_tokens` / `output_tokens` / `total_tokens` 가 누적됩니다.

```
from langchain_core.callbacks import UsageMetadataCallbackHandler

cb = UsageMetadataCallbackHandler()
result = agent.invoke(
    {"messages": [{"role": "user", "content": "15 x 27 + 3 은?"}]},
    config={"callbacks": [cb]},
)

print(cb.usage_metadata)
# {'gpt-5.4': {'input_tokens': 312, 'output_tokens': 48, 'total_tokens': 360, ...}}
```

3.9 Runtime Config — 실행 단위 옵션

`invoke()` / `stream()` 의 `config=` 인자는 단일 호출의 실행 메타데이터를 지정합니다. LangSmith 트레이스 이름, 태그·메타데이터, 도구·서브에이전트 동시 실행 한도 등을 한 번에 설정할 수 있습니다.

```
result = agent.invoke(
    {"messages": [{"role": "user", "content": "..."}]},
    config={
        "run_name": "monthly-report",
        "tags": ["batch-2025-05", "finance"],
        "metadata": {"user_id": "u-1234", "tenant": "acme"},
        "max_concurrency": 5,
    },
)
```

TIP

`run_name` / `tags` / `metadata` 는 LangSmith UI에서 그대로 필터·검색에 사용됩니다. `max_concurrency` 는 한 그래프 실행 내에서 병렬 도구·서브에이전트 호출 수를 제한합니다.

3.10 Connection Resilience — `max_retries` / `timeout`

프로덕션 환경에서는 OpenAI·Anthropic 등 외부 API의 일시적 장애를 견뎌야 합니다. `init_chat_model()` 또는 `ChatOpenAI()` 에 `max_retries` 와 `timeout` 을 지정하면 LangChain이 지수 백오프로 재시도합니다.

```
from langchain.chat_models import init_chat_model

model = init_chat_model(
    "openai:gpt-5.4",
    max_retries=15, # 기본값 6 → 장애 대비를 위해 상향
    timeout=120,   # 초 단위, 긴 응답 대비
)
```

TIP

재시도는 5xx 에러·연결 오류·429 Too Many Requests 에서 발생합니다. 422 같은 클라이언트 오류는 재시도하지 않습니다. 한도를 너무 크게 잡으면 비용·지연이 증폭되므로, 보통 10~20 범위에서 운영합니다.

3.11 프롬프트 캐싱 — Implicit vs Explicit

긴 시스템 프롬프트나 RAG 컨텍스트를 반복 사용할 때 프롬프트 캐싱으로 입력 토큰 비용을 크게 줄일 수 있습니다.

- Implicit caching: 프로바이더가 동일한 프리픽스를 자동 인식하여 캐싱합니다. OpenAI `gpt-5.4` 는 1024 토큰 이상 동일 프리픽스에서 자동 동작합니다.
- Explicit caching: Anthropic은 `prompt_cache_key` (또는 `cache_control` 블록)로 캐시 키를 명시합니다. LangChain은 `model_kwargs={"prompt_cache_key": "..."}` 로 전달합니다.

```
from langchain_anthropic import ChatAnthropic

model = ChatAnthropic(
    model="claude-sonnet-4-6",
    model_kwargs={"prompt_cache_key": "system-v3"},
)
```

TIP

캐시 적중 시 `usage_metadata.input_tokens_details` 에 `cache_read` · `cache_creation` 값이 따로 표시됩니다. `UsageMetadataCallbackHandler` 로 캐시 적중률을 운영 지표로 추적하세요.

요약

이 노트북에서 학습한 핵심 내용을 정리합니다.

항목	설명
<code>init_chat_model()</code>	프로바이더 문자열로 모델을 통합 초기화
<code>ChatOpenAI(base_url=...)</code>	OpenAI 등 커스텀 엔드포인트 사용
<code>invoke()</code>	단일 입력 → 단일 응답
<code>stream()</code>	토큰 단위 스트리밍 응답
<code>batch()</code>	여러 입력을 동시에 처리
<code>SystemMessage</code>	시스템 지시사항 설정
<code>HumanMessage</code>	사용자 입력 메시지
<code>AIMessage</code>	AI 응답 메시지 (대화 이력용)
<code>ToolMessage</code>	도구 실행 결과 전달
멀티모달 메시지	<code>content</code> 에 텍스트와 이미지를 함께 전달

이 장에서는 모델 초기화와 메시지 시스템의 전체 구조를 파악했습니다. 모델이 무엇을 할 수 있는지 이해했으나, 다음 장에서는 모델이 어떻게 외부 세계와 상호작용하는지 – `@tool` 데코레이터의 고급 기능, Pydantic 스키마, `ToolRuntime`, 그리고 `with_structured_output()` 을 통한 출력 구조화 – 를 깊이 있게 다룹니다.

04 도구와 구조화된 출력

도구는 에이전트가 외부 세계와 상호작용하는 유일한 수단입니다. 이 장에서는 `@tool` 데코레이터의 고급 기능 — Pydantic 스키마, `ToolRuntime` 을 통한 런타임 컨텍스트 접근, 동적 도구 등록 — 과 함께, `with_structured_output()` 으로 모델의 응답을 Pydantic 모델이나 JSON 스키마에 맞게 강제하는 방법을 학습합니다.

앞 장에서 메시지 시스템을 통해 모델과 대화하는 방법을 배웠습니다. 하지만 모델이 텍스트만 생성한다면 진정한 에이전트라 할 수 없습니다. 에이전트의 핵심은 도구 호출을 통해 계산, 검색, API 호출 등 실질적인 작업을 수행하는 데 있습니다. 이 장에서는 도구 정의부터 에이전트 연결, 런타임 컨텍스트 주입, 출력 구조화까지 도구 시스템의 전체를 다룹니다.

학습 목표

LEARNING OBJECTIVES

- `@tool` 데코레이터로 도구를 만들고 스키마를 확인합니다
- Pydantic 모델을 사용하여 복잡한 입력 스키마를 정의합니다
- `create_agent()` 에 도구를 연결하여 에이전트를 구성합니다
- `ToolRuntime` 을 통해 도구에서 런타임 컨텍스트에 접근합니다
- `with_structured_output()` 으로 구조화된 출력을 설정합니다
- `ToolStrategy` 와 `ProviderStrategy` 의 차이를 이해합니다

4.1 환경 설정

TIP

이 장의 일부 예제는 `deepagents ≥ 0.5.0` 또는 `langgraph ≥ 1.1.5`, `ToolRuntime` 의 `execution_info / server_info` 확장 사용 예제는 `langchain ≥ 1.2` 가 필요합니다. 버전이 낮으면 `AttributeError` 가 발생할 수 있으니 `uv pip list | grep -E "langchain|langgraph|deepagents"` 로 확인하세요.

API 키를 로드하고 OpenAI 모델을 초기화합니다.

```
import os
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI

load_dotenv(override=True)

# OpenAI를 통한 모델 초기화
model = ChatOpenAI(
    model="gpt-5.4",
)

print("모델 초기화 완료:", model.model_name)
```

모델 초기화 완료: gpt-5.4

4.2 @tool 데코레이터 기본

모델이 준비되었으니, 에이전트에게 능력을 부여할 도구를 만들어 봅시다. `@tool` 데코레이터는 일반 Python 함수를 LangChain 도구 객체로 변환합니다.

함수에 `@tool` 을 붙이면 에이전트가 사용할 수 있는 도구가 됩니다. LangChain은 함수의 이름, docstring, 타입 힌트를 자동으로 파싱하여 도구 스키마를 생성합니다.

`@tool` 데코레이터에 인자를 전달하여 도구의 이름과 설명을 커스터마이징할 수도 있습니다.

`@tool("custom_name")` 은 도구 이름을 변경하고, `@tool("custom_name", description="...")` 은 이름과 설명 모두를 직접 지정합니다. 함수명과 다른 이름을 사용하고 싶거나, docstring보다 더 상세한 설명이 필요할 때 유용합니다.

! WARNING

도구 함수의 매개변수 이름으로 `config` 와 `runtime` 은 예약어이므로 사용할 수 없습니다. 이 이름들은 LangChain 내부에서 런타임 컨텍스트 주입에 사용됩니다.

```
from langchain.tools import tool

@tool
def my_tool(param: str) -> str:
    """Tool description for the LLM."""
    return result
```

```
from langchain.tools import tool

@tool
def get_weather(city: str) -> str:
    """도시의 현재 날씨를 조회합니다."""
    weather_data = {
        "Seoul": "맑음, 15\u00b0C",
        "Tokyo": "흐림, 12\u00b0C",
        "New York": "비, 8\u00b0C",
    }
    return weather_data.get(city, f"날씨 데이터를 사용할 수 없습니다: {city}")

# 도구의 스키마 확인
print("도구 이름:", get_weather.name)
```

```
print("도구 설명:", get_weather.description)
print("입력 스키마:", get_weather.args_schema.model_json_schema())
```

```
도구 이름: get_weather
도구 설명: 도시의 현재 날씨를 조회합니다.
입력 스키마: {'description': '도시의 현재 날씨를 조회합니다.', 'properties': {'city': {'title': 'City', 'type': 'string'}}, 'required': ['city'], 'title': 'get_weather', 'type': 'object'}
```

4.3 Pydantic 복잡한 스키마

단순한 함수 시그니처만으로는 표현하기 어려운 복잡한 입력 구조가 필요할 때가 있습니다. 예를 들어, 검색 쿼리에 필터 조건, 정렬 옵션, 페이지네이션 등 여러 매개변수가 필요한 경우입니다. Pydantic `BaseModel` 을 사용하면 각 필드에 상세한 설명과 제약 조건을 추가할 수 있습니다.

더 복잡한 입력 구조가 필요한 경우, Pydantic `BaseModel` 을 사용하여 스키마를 정의합니다.

`@tool(args_schema=MySchema)` 형태로 전달하면, LLM이 정확한 파라미터 구조를 이해할 수 있습니다.

- `Field(description=...)` : 각 필드에 대한 설명을 LLM에 전달. 이 설명이 구체적일수록 모델이 올바른 값을 생성할 확률이 높아집니다.
- `Field(default=...)` : 기본값 설정

```
from pydantic import BaseModel, Field

class SearchQuery(BaseModel):
    """데이터베이스 쿼리용 검색 파라미터입니다."""
    query: str = Field(description="검색 쿼리 문자열")
    max_results: int = Field(default=5, description="반환할 최대 결과 수")
    category: str = Field(default="all", description="검색 카테고리: all, tech, science, news")

@tool(args_schema=SearchQuery)
def search_database(query: str, max_results: int = 5, category: str = "all") -> str:
    """고급 필터링 옵션으로 데이터베이스를 검색합니다."""
    return f"'{category}' 카테고리에서 '{query}'에 대한 {max_results}개의 결과를 찾았습니다"

print("복합 스키마:", search_database.args_schema.model_json_schema())
```

```
복합 스키마: {'description': '데이터베이스 쿼리용 검색 파라미터입니다.', 'properties': {'query': {'description': '검색 쿼리 문자열', 'title': 'Query', 'type': 'string'}, 'max_results': {'default': 5, 'description': '반환할 최대 결과 수', 'title': 'Max Results', 'type': 'integer'}, 'category': {'default': 'all', 'description': '검색 카테고리: all, tech, science, news', 'title': 'Category', 'type': 'string'}}, 'required': ['query'], 'title': 'SearchQuery', 'type': 'object'}
```

4.4 도구를 에이전트에 연결

도구를 정의했으면, 이제 에이전트에 연결합니다. `create_agent()` 에 도구 리스트를 전달하면, 에이전트가 상황에 맞는 도구를 자동으로 선택하여 실행합니다. 내부적으로 `model.bind_tools(tools)` 가 호출되어 모델에 도구 스키마가 바인딩됩니다.

```
from langchain.agents import create_agent

agent = create_agent(
    model=model,
```



```
tools=[tool1, tool2],
system_prompt="...",
)
```

· NOTE

참고: LangChain v1에서는 `create_react_agent` 가 제거되었습니다. 반드시 `create_agent` 를 사용하세요.

4.5 ToolRuntime

에이전트에 도구를 연결하는 방법을 배웠으니, 한 단계 더 나아가 도구가 실행 환경의 정보에 접근하는 방법을 살펴봅시다. 예를 들어, 현재 로그인한 사용자의 ID에 따라 다른 데이터를 반환하거나, 데이터베이스 쿼렉션을 주입 받아야 하는 경우입니다.

`ToolRuntime` 을 사용하면 도구 함수 내에서 현재 대화 상태(state)에 접근할 수 있습니다. 이를 통해 메시지 이력, 설정값, 사용자 정보, DB 쿼렉션 등 런타임 컨텍스트를 활용하는 도구를 만들 수 있습니다. `ToolRuntime` 은 LLM 에게는 노출되지 않는 숨겨진 매개변수로, 모델이 이 값을 생성할 필요가 없습니다.

TIP

`ToolRuntime` 의 상세 활용법은 7장(HITL과 런타임)에서 `context_schema` 와 함께 더 깊이 다룹니다.

```
@tool
def my_tool(runtime: ToolRuntime) -> str:
    messages = runtime.state["messages"]
    # ...
```

`runtime.execution_info` — 실행 메타데이터

`ToolRuntime` 은 단순 state 접근에 그치지 않고, 현재 실행 단위의 메타데이터를 `execution_info` 로 노출합니다. 로깅·트레이싱·재시도 정책에서 자주 사용합니다.

```
@tool
def log_call(payload: str, runtime: ToolRuntime) -> str:
    """입력을 처리하고 실행 메타데이터를 로깅합니다."""
    info = runtime.execution_info
    print(f"thread={info.thread_id} run={info.run_id} attempt={info.node_attempt}")
    return f"received: {payload}"
```

- `execution_info.thread_id` : 멀티턴 대화를 묶는 세션 ID
- `execution_info.run_id` : 한 번의 `invoke()` / `stream()` 실행 ID (LangSmith 트레이스와 연동)
- `execution_info.node_attempt` : 현재 노드의 시도 횟수(재시도 시 증가)

`runtime.server_info` — 서버·사용자 정보

LangGraph Platform·LangSmith 서버에서 실행될 때 `runtime.server_info` 로 어시스턴트와 호출 사용자를 식별할 수 있습니다.

```
@tool
def user_summary(runtime: ToolRuntime) -> str:
    """현재 사용자 정보를 반환합니다."""
    s = runtime.server_info
    return f"assistant={s.assistant_id} user={s.user.email if s.user else 'anonymous'}"
```

- `server_info.assistant_id` : 배포된 어시스턴트(graph 버전)의 ID
- `server_info.user` : 인증된 사용자 객체(`user.id`, `user.email`, 권한 정보)

➡ TIP

로컬 실행에서는 `server_info` 가 비어 있을 수 있습니다. 운영 환경에서만 채워지는 값이므로
`if runtime.server_info.user is not None:` 같은 가드를 두세요.

도구 호출 예외 핸들링 — `@wrap_tool_call`

도구 단위로 일관된 예외 처리·로깅·재시도 정책을 부여하려면 `@wrap_tool_call` 데코레이터를 사용합니다.

`ToolCallRequest` 를 받아 실제 도구 실행을 가로채는 미들웨어성 훅입니다.

```
from langchain.tools import wrap_tool_call, ToolCallRequest
from langchain_core.messages import ToolMessage

@wrap_tool_call
def safe_invoke(request: ToolCallRequest, handler):
    """모든 도구 호출을 try/except로 감쌉니다."""
    try:
        return handler(request)
    except Exception as exc: # noqa: BLE001
        return ToolMessage(
            content=f"[error] {type(exc).__name__}: {exc}",
            tool_call_id=request.tool_call.id,
            status="error",
        )

agent = create_agent(
    model=model,
    tools=[get_weather, search_database],
    middleware=[safe_invoke],
)
```

➡ TIP

`@wrap_tool_call` 은 미들웨어로 등록되므로 모든 도구에 일괄 적용됩니다. 특정 도구만 보호하려면
`request.tool_call.name` 으로 분기하세요.

Command 업데이트와 ToolMessage 동반 반환

도구가 단순한 텍스트 결과 외에 그래프 상태나 다음 노드까지 갱신하려면 `Command` 를 반환합니다. LangGraph
 는 `Command.update` 로 상태를 패치하고, `Command.goto` 로 다음 노드를 지정합니다. 이때 반드시 `ToolMessage` 를 함
 께 반환하여 `tool_call_id` 를 담아 줘야 모델이 다음 턴을 진행할 수 있습니다.

```
from langgraph.types import Command
from langchain_core.messages import ToolMessage

@tool
```

```
def transfer_to_specialist(topic: str, runtime: ToolRuntime) -> Command:
    """전문가 서버에이전트로 컨트롤을 넘기고 상태를 갱신합니다."""
    return Command(
        update={
            "active_specialist": topic,
            "messages": [
                ToolMessage(
                    content=f"{topic} 전문가에게 전달했습니다.",
                    tool_call_id=runtime.tool_call_id,
                )
            ],
        },
        goto="specialist_node",
    )
```

TIP

`tool_call_id` 를 빠뜨리면 모델이 “도구 호출에 대한 응답이 없음” 오류를 일으킵니다. `runtime.tool_call_id` 로 현재 호출 ID를 가져와 그대로 전달하세요.

4.6 구조화된 출력

도구 정의를 마쳤으니, 이제 반대 방향 – 모델의 출력을 제어하는 방법을 살펴봅니다. 자유 형식의 텍스트 대신, 정해진 스키마에 맞는 구조화된 데이터를 받고 싶을 때 `with_structured_output()` 을 사용합니다.

`with_structured_output()` 을 사용하면 모델의 응답을 Pydantic 모델이나 dataclass 형태로 직접 받을 수 있습니다. 이 방법은 에이전트 없이 모델에서 직접 사용합니다. 반환값은 지정한 Pydantic 모델의 인스턴스이므로, 별도의 파싱 없이 바로 `result.field_name` 형태로 접근할 수 있습니다.

```
structured_model = model.with_structured_output(MySchema)
result = structured_model.invoke("...")
# result는 MySchema 인스턴스
```

4.7 ToolStrategy vs ProviderStrategy

`with_structured_output()` 은 모델 수준에서 동작합니다. 에이전트 수준에서 최종 응답을 구조화하려면 `create_agent()` 의 `response_format` 매개변수를 사용하며, 이때 두 가지 전략 중 하나를 선택할 수 있습니다: 에이전트에서 구조화된 출력을 사용하는 두 가지 전략이 있습니다:

전략	설명	장점
<code>ToolStrategy</code>	도구 호출 메커니즘을 활용하여 구조화된 출력 생성	모든 모델에서 동작, 안정적
<code>ProviderStrategy</code>	프로바이더의 네이티브 구조화 출력 기능 사용	더 빠르고 정확 (지원 모델 한정)

`response_format` 파라미터에 전략을 지정하여 에이전트의 최종 응답을 구조화할 수 있습니다. `ToolStrategy` 는 내부적으로 응답 스키마를 하나의 “도구”로 변환하여 모델에게 호출하도록 유도하는 방식이고,

`ProviderStrategy` 는 OpenAI의 `response_format` 같은 프로바이더 네이티브 JSON 모드를 활용하는 방식입니다. `ToolStrategy` 가 더 넓은 호환성을 제공하지만, `ProviderStrategy` 는 지원 모델에서 더 빠르고 정확한 결과를 보여줍니다.

➡ TIP

`ProviderStrategy.strict=True` (스키마 강제 검증) 옵션은 `langchain ≥ 1.2` 가 필요합니다. 이전 버전에서는 `strict=False` 또는 `ToolStrategy` 를 사용하세요.

➡ TIP

`ProviderStrategy` 는 현재 OpenAI(`gpt-5.4` · `gpt-4.1`)와 Anthropic(`claude-sonnet-4-6`)에서 안정 동작합니다. Google Gemini는 비공식 지원으로, JSON 모드는 동작하나 스키마 강제는 일부 모델에서만 적용됩니다. 호환성이 중요한 경우 `ToolStrategy` 를 사용하세요.

요약

이 노트북에서 학습한 핵심 내용을 정리합니다.

항목	설명
<code>\@tool</code> 데코레이터	함수를 에이전트용 도구로 변환
<code>args_schema</code>	Pydantic 모델로 복잡한 입력 스키마 정의
<code>create_agent()</code>	모델과 도구를 연결하여 에이전트 생성
<code>ToolRuntime</code>	도구 내에서 런타임 상태(대화 이력 등) 접근
<code>with_structured_output()</code>	모델 응답을 Pydantic/dataclass로 구조화
<code>ToolStrategy</code>	도구 호출 방식의 구조화된 에이전트 출력
<code>ProviderStrategy</code>	프로바이더 네이티브 구조화 출력

이 장에서는 도구의 정의·연결·런타임 주입과 출력 구조화까지, 에이전트의 “입출력 인터페이스”를 완성했습니다. 다음 장에서는 에이전트가 대화를 기억하고 응답을 실시간으로 전달하는 방법 — 단기/장기 메모리 아키텍처와 5가지 스트리밍 모드 — 를 다룹니다.

05 메모리와 스트리밍

실용적인 에이전트라면 대화를 기억하고, 장기적인 사용자 선호도를 학습하며, 응답을 실시간으로 전달할 수 있어야 합니다. Part I에서 `InMemorySaver`의 기본 사용법을 다뤘다면, 이 장에서는 단기/장기 메모리의 차이, 메시지 트리밍 전략, 그리고 5가지 스트리밍 모드의 실전 활용법을 깊이 있게 학습합니다.

학습 목표

LEARNING OBJECTIVES

- 단기 메모리(Short-term Memory): `InMemorySaver`를 사용하여 `thread_id` 기반으로 대화 상태를 유지하는 방법을 이해합니다.
- 장기 메모리(Long-term Memory): `InMemoryStore`를 사용하여 대화 간에 지속되는 메모리를 구현합니다.
- 메시지 트리밍: 긴 대화에서 토큰 예산 내로 메시지를 제한하는 방법을 배웁니다.
- 스트리밍 모드: `values`, `updates`, `messages`, `custom` 등 다양한 스트리밍 모드의 차이를 이해합니다.

5.1 환경 설정

이 장에서 사용할 모델을 초기화합니다. 메모리와 스트리밍은 모델 자체가 아닌 에이전트 레벨에서 동작하므로, 모델 설정은 이전 장과 동일합니다.

```
from dotenv import load_dotenv
import os
load_dotenv(override=True)

from langchain_openai import ChatOpenAI

model = ChatOpenAI(
    model="gpt-5.4",
)

print("모델 준비 완료:", model.model_name)
```

모델 준비 완료: gpt-5.4

5.2 단기 메모리: InMemorySaver

2장에서 `InMemorySaver` 를 사용하여 멀티턴 대화를 구현한 바 있습니다. 이 절에서는 그 내부 동작을 더 깊이 살펴 봅니다.

단기 메모리는 하나의 대화 세션 내에서 이전 메시지를 기억하는 메커니즘입니다.

- `InMemorySaver` 는 체크포인트(checkpointer)로서 에이전트의 전체 그래프 상태(full graph state)를 메모리에 저장합니다. 여기에는 메시지 히스토리뿐 아니라 도구 호출 상태, 중단점 정보 등이 모두 포함됩니다.
- `thread_id` 를 사용하여 서로 다른 대화 세션을 구분합니다.
- 같은 `thread_id` 를 사용하면 이전 대화 컨텍스트가 유지됩니다.
- `invoke()` 호출 시 반드시 `{"configurable": {"thread_id": "..."} }` config를 전달해야 합니다.

5.3 다른 thread_id로 독립된 대화

단기 메모리의 격리 메커니즘을 확인해 봅니다. 서로 다른 `thread_id` 를 사용하면 완전히 독립된 대화 세션이 생성됩니다. 이전 세션의 컨텍스트는 공유되지 않습니다. 이를 통해 하나의 에이전트 인스턴스로 여러 사용자의 대화를 동시에 관리할 수 있습니다.

5.4 메시지 트리밍

세션 격리를 확인했으니, 이제 단기 메모리의 실전적 문제를 다룹니다. 대화가 길어지면 토큰 수가 증가하여 비용과 성능에 영향을 줍니다. 특히 모델의 컨텍스트 윈도우를 초과하면 API 오류가 발생합니다. 메시지 트리밍을 사용하면 토큰 예산 내에서 가장 관련성 높은 메시지만 유지할 수 있습니다.

`trim_messages()` 함수의 주요 매개변수:

- `messages` : 트리밍할 메시지 리스트
- `max_tokens` : 유지할 최대 토큰 수. 이 예산을 초과하면 오래된 메시지부터 제거합니다.
- `strategy="last"` : 가장 최근 메시지를 우선 유지합니다. (유일하게 지원되는 전략)
- `include_system=True` : 시스템 메시지는 항상 포함합니다. 시스템 프롬프트가 잘리면 에이전트의 행동이 달라질 수 있으므로, 일반적으로 `True` 로 설정합니다.

TIP

`trim_messages` 는 단독으로 호출하거나, 미들웨어(`@before_model`) 안에서 자동으로 적용할 수 있습니다. 미들웨어 방식은 6장에서 다룹니다.

5.5 장기 메모리: InMemoryStore

단기 메모리가 세션 내 대화를 기억한다면, 장기 메모리는 세션 간에 지속되는 정보를 저장합니다. “사용자가 어두운 테마를 선호한다”, “지난주에 Python 프로젝트를 논의했다” 같은 정보를 기억하여 개인화된 경험을 제공합니다.

장기 메모리는 대화 세션 간에 지속되는 정보를 저장합니다.

- `InMemoryStore` 는 네임스페이스 기반 키-값 저장소로서 사용자 선호도, 설정 등을 저장합니다.
- 도구의 `ToolRuntime` 파라미터를 통해 스토어에 접근할 수 있습니다.
- `thread_id` 와 무관하게 모든 세션에서 동일한 데이터에 접근 가능합니다.

주요 API는 다음과 같습니다:

- `store.put(namespace, key, value)` : 네임스페이스 아래에 키-값 쌍을 저장합니다. 예:
`store.put(("user_123", "memories"), "pref_theme", {"value": "dark"})`
- `store.search(namespace, query=...)` : 네임스페이스 내에서 시맨틱 검색을 수행합니다.
- `store.get(namespace, key)` : 특정 키의 값을 조회합니다.

에이전트 생성 시 `store=InMemoryStore()` 를 전달하면, `ToolRuntime` 을 통해 도구에서 스토어에 접근할 수 있습니다.

사용자별 네임스페이스 (context_schema + runtime.context)

실전에서는 `user_id` 로 네임스페이스를 분리해 멀티 테넌트를 지원합니다. `context_schema` 로 컨텍스트 클래스를 선언하고, 도구 안에서 `runtime.context.user_id` 로 안전하게 접근합니다. Pydantic/dataclass 인스턴스는 그대로 저장하지 말고 `dict(user_info)` 처럼 dict로 캐스팅한 뒤 `store.put` 에 전달하는 것이 직렬화 호환성에 유리합니다.

```
from dataclasses import dataclass
from langgraph.store.memory import InMemoryStore
from langchain.tools import tool, ToolRuntime

@dataclass
class Context:
    user_id: str

@tool
def remember_user(user_info: dict, runtime: ToolRuntime[Context]) -> str:
    """현재 사용자의 프로필을 저장합니다."""
    namespace = (runtime.context.user_id, "profile")
    runtime.store.put(namespace, "info", dict(user_info)) # dict 캐스팅
    return "saved"

store = InMemoryStore()
agent = create_agent(
    model=model,
    tools=[remember_user],
    store=store,
    context_schema=Context,
)

agent.invoke(
    {"messages": [{"role": "user", "content": "내 이름은 민지야"}]},
    context=Context(user_id="user_123"),
)
```

TIP

`store` 를 에이전트에 전달하지 않으면 `runtime.store` 가 `None` 이 되어 도구에서 접근할 수 없습니다. 장기 메모리 도구를 추가했다면 `create_agent(..., store=...)` 인자를 빠뜨리지 않았는지 반드시 확인하세요.

TIP

프로덕션에서는 `langgraph-checkpoint-postgres` 패키지의 `PostgresStore` (또는 `AsyncPostgresStore`)를 사용해 영구 저장과 시맨틱 검색을 함께 처리합니다. `InMemoryStore` 와 동일한 인터페이스를 제공하므로 코드 변경 없이 교체할 수 있습니다.

단기 메모리와 장기 메모리의 차이:

구분	단기 메모리 (Checkpoint)	장기 메모리 (Store)
범위	하나의 <code>thread_id</code> 내	모든 세션에 걸쳐
저장 대상	대화 메시지 히스토리	사용자 선호도, 학습 데이터
수명	세션 종료 시 (또는 영구)	명시적 삭제 전까지 영구
접근 방식	자동 (에이전트 내부)	도구를 통해 명시적

5.6 스트리밍 모드

메모리 시스템을 완성했으니, 이제 에이전트의 출력 전달 방식을 다룹니다. 2장에서 `stream_mode="updates"` 를 간단히 사용했는데, 실제로는 5가지 스트리밍 모드가 있으며 각각 다른 용도에 최적화되어 있습니다. 동기 호출에는 `stream()` , 비동기 환경에서는 `astream()` 을 사용합니다.

에이전트의 실행 과정을 실시간으로 관찰할 수 있는 스트리밍 기능을 제공합니다. 용도에 따라 다양한 스트리밍 모드를 선택할 수 있습니다.

스트리밍 모드 비교표

모드	설명	용도
<code>values</code>	각 단계의 전체 상태	디버깅, 상태 추적
<code>updates</code>	각 노드의 업데이트만	진행 상황 표시
<code>messages</code>	메시지 토큰 단위	채팅 UI
<code>custom</code>	사용자 정의 이벤트	커스텀 진행률

`stream_mode="custom"` 참고

`stream_mode="custom"` 은 사용자 정의 이벤트를 스트리밍하는 모드입니다. LangGraph의 `get_stream_writer()` 를 사용하면 `create_agent` 로 만든 에이전트의 도구나 미들웨어 안에서도 커스텀 이벤트를 발행할 수 있습니다.

```
from langgraph.config import get_stream_writer
from langchain.tools import tool

@tool
def long_running_task(query: str) -> str:
    """진행 상황을 스트림에 보고하는 작업."""
    writer = get_stream_writer()
    writer({"step": 1, "status": "fetching"})
    # ... 처리 로직 ...
```



```

writer({"step": 2, "status": "done"})
return "ok"

for chunk in agent.stream(
    {"messages": [{"role": "user", "content": "검색해줘"}]},
    stream_mode="custom",
):
    print(chunk) # writer()로 발행한 dict가 그대로 전달됨

```

➡ TIP

과거 일부 자료에서는 `stream_mode="custom"` 이 `create_agent` 에서 동작하지 않는다고 안내했지만, 현재 LangGraph는 `get_stream_writer()` 를 통한 도구/미들웨어 내부 발행을 지원합니다. `StateGraph` 의 `writer` 매개변수 주입 방식과 결과는 동일합니다.

version="v2" 스트리밍과 GraphOutput

`agent.stream(..., version="v2")` 를 지정하면 결과 청크가 구조화된 이벤트로 도착하고, `agent.invoke(..., version="v2")` 는 `GraphOutput` 을 반환합니다. `GraphOutput` 은 `value` (최종 상태)와 `interrupts` (중단 정보) 두 속성을 제공해 HITL 흐름에서 활용도가 높습니다.

```

# 스트림: type 필드로 이벤트 구분
for chunk in agent.stream(
    {"messages": [{"role": "user", "content": "search hello"}]},
    config={"configurable": {"thread_id": "t1"}},
    version="v2",
):
    if chunk["type"] == "updates":
        print("update:", chunk["data"])
    elif chunk["type"] == "messages":
        print("token:", chunk["data"][0].content)

# invoke: GraphOutput 반환
result = agent.invoke(
    {"messages": [{"role": "user", "content": "hi"}]},
    config={"configurable": {"thread_id": "t1"}},
    version="v2",
)
print(result.value) # 최종 상태 dict
print(result.interrupts) # 중단된 경우 Interrupt 리스트, 아니면 []

```

토큰 누적 패턴 (chunk_position)

`stream_mode="messages"` 로 토큰을 받을 때, 마지막 청크는 `chunk_position = "last"` 로 표시됩니다. 이 시점에 누적된 도구 호출이나 `content_blocks` 를 한 번에 처리하면 깔끔합니다.

```

buffer = []
tool_calls = []
for chunk, meta in agent.stream(
    {"messages": [{"role": "user", "content": "검색해줘"}]},
    stream_mode="messages",
):
    buffer.append(chunk.content)
    if chunk.tool_calls:
        tool_calls.extend(chunk.tool_calls)
    if meta.get("chunk_position") == "last":
        print("최종 본문:", "".join(buffer))
        print("도구 호출:", tool_calls)

```

content_blocks 기반 reasoning 필터링

확장 사고(extended thinking)를 지원하는 모델(예: Claude Sonnet 4.6)에서는 `content_blocks` 의 `type` 을 검사해 추론 블록만 UI에서 따로 다룰 수 있습니다.

```
for chunk, _ in agent.stream(payload, stream_mode="messages"):
    for block in getattr(chunk, "content_blocks", []) or []:
        if block["type"] == "reasoning":
            ui.show_reasoning(block["reasoning"])
        elif block["type"] == "text":
            ui.show_answer(block["text"])
```

다중 stream_mode와 인터럽트 캡처

`stream_mode` 에 리스트를 전달하면 각 청크가 `(mode, payload)` 튜플로 도착합니다. HITL이 결합된 흐름에서는 `"__interrupt__"` 이벤트를 별도로 받아 사람의 결정을 기다릴 수 있습니다.

```
for mode, payload in agent.stream(
    {"messages": [{"role": "user", "content": "이메일 보내줘"}]},
    config={"configurable": {"thread_id": "t1"}},
    stream_mode=["messages", "updates"],
):
    if mode == "updates" and "__interrupt__" in payload:
        interrupt = payload["__interrupt__"]
        # 사용자 승인 UI 띄우고 Command(resume=...)로 재개
```

TIP

서브그래프 이벤트도 함께 받고 싶다면 `subgraphs=True` 를 지정하세요. 반대로 토큰 스트리밍이 필요 없을 때는 `disable_streaming=True` (또는 모델 생성 시 `streaming=False`)로 비용을 줄일 수 있습니다.

요약

이 노트북에서 학습한 내용을 정리합니다:

개념	구현	설명
단기 메모리	<code>InMemorySaver</code> + <code>thread_id</code>	하나의 대화 세션 내 컨텍스트 유지
세션 격리	다른 <code>thread_id</code> 사용	독립된 대화 세션 관리
메시지 트리밍	<code>trim_messages</code> + 미들웨어	토큰 예산 내 메시지 제한
장기 메모리	<code>InMemoryStore</code> + <code>ToolRuntime</code>	대화 간 지속되는 사용자 데이터
스트리밍 (values)	<code>stream_mode="values"</code>	각 단계의 전체 상태 스냅샷
스트리밍 (updates)	<code>stream_mode="updates"</code>	노드별 업데이트 확인
스트리밍 (messages)	<code>stream_mode="messages"</code>	토큰 단위 실시간 출력

스트리밍 (custom)	<code>stream_mode="custom"</code>	LangGraph <code>StateGraph</code> 수준에서만 사용 가능
---------------	-----------------------------------	---

핵심 포인트:

- 단기 메모리는 `thread_id` 로 격리되며, 같은 세션 내에서만 컨텍스트가 유지됩니다.
- 장기 메모리는 `InMemoryStore` 를 통해 세션 간에 공유됩니다.
- `stream_mode="values"` 는 각 단계의 전체 상태를 반환하여 디버깅에 유용합니다.
- `stream_mode="custom"` 은 `create_agent` 에서는 직접 사용할 수 없으며, LangGraph의 `StateGraph` API가 필요합니다.
- 스트리밍 모드를 적절히 선택하면 사용자 경험을 크게 향상시킬 수 있습니다.

이 장에서는 에이전트의 “기억력”과 “전달력”을 완성했습니다. 다음 장에서는 에이전트 실행 파이프라인의 각 단계에 혹은 삽입하는 미들웨어 시스템과, 안전하지 않은 입출력을 차단하는 가드레일 패턴을 다룹니다.

06 미들웨어와 가드레일

에이전트의 모든 요청과 응답이 지나가는 파이프라인에 로직을 삽입하고 싶을 때가 있습니다 – 입력을 검증하거나, 모델 호출 전에 컨텍스트를 추가하거나, 응답에서 민감 정보를 제거하는 등. LangChain v1의 미들웨어 시스템은 바로 이 목적을 위해 설계되었습니다. 이 장에서는 미들웨어 hook의 종류와 가드레일 구현 패턴을 학습합니다. 앞 장에서 메모리와 스트리밍으로 에이전트의 기본 인프라를 완성했습니다. 그러나 프로덕션 환경에서는 에이전트가 “무엇을 할 수 있는가”뿐 아니라 “무엇을 하면 안 되는가”도 중요합니다. 미들웨어는 에이전트 실행의 각 단계에 가로채기(interception) 로직을 삽입하여, 로깅·검증·필터링·캐싱 등 횡단 관심사(cross-cutting concerns)를 깔끔하게 처리합니다. 웹 프레임워크의 미들웨어와 유사한 개념입니다.

학습 목표

LEARNING OBJECTIVES

- 미들웨어 개념: 에이전트 실행 파이프라인의 각 단계에 hook을 추가하는 방법을 이해합니다.
- 빌트인 미들웨어: `SummarizationMiddleware` 등 기본 제공 미들웨어를 사용합니다.
- 커스텀 미들웨어: `@before_model`, `@after_model`, `@wrap_model_call`, `@dynamic_prompt` 데코레이터로 커스텀 미들웨어를 구현합니다.
- 가드레일: 안전하지 않은 입력/출력을 차단하는 방법을 배웁니다.

6.1 환경 설정

```
from dotenv import load_dotenv
import os
load_dotenv(override=True)

from langchain_openai import ChatOpenAI

model = ChatOpenAI(
    model="gpt-5.4",
)

print("모델 준비 완료:", model.model_name)
```

모델 준비 완료: gpt-5.4

6.2 미들웨어 개념

미들웨어의 동작 원리를 이해하기 위해, 에이전트의 실행 파이프라인을 시각적으로 살펴봅니다.

미들웨어는 에이전트 실행 파이프라인의 각 단계에 훅(hook)을 추가하여 동작을 제어하는 메커니즘입니다.

`create_agent()` 의 `middleware` 매개변수에 미들웨어 리스트를 전달하면, 에이전트의 매 실행 주기마다 해당 훅이 자동으로 호출됩니다. 미들웨어의 훅 종류는 `prompt` , `before_model` , `after_model` , `before_tool` , `after_tool` , `wrap_model_call` 등이 있으며, 아래 다이어그램에서 각 훅이 파이프라인의 어느 시점에 실행되는지 확인할 수 있습니다.

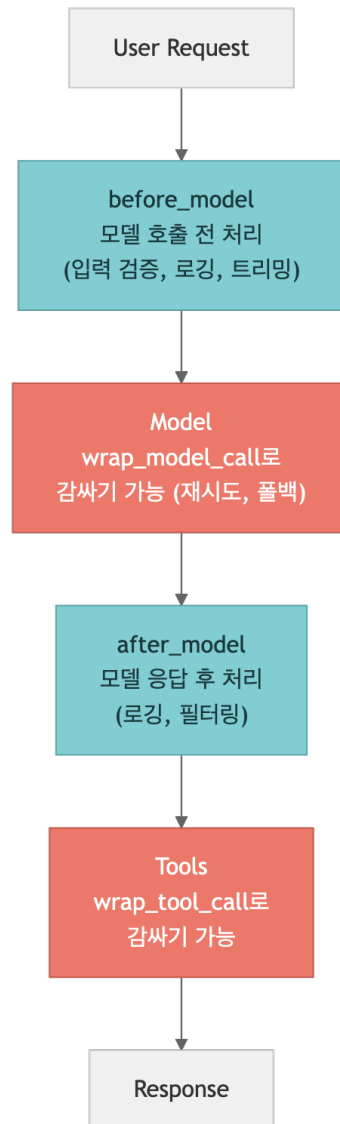


그림 읽는 법

왼쪽에서 오른쪽으로 읽으면 됩니다. 요청이 들어오면 먼저 **before** 계열 훅이 개입하고, 모델 호출을 중심으로 **wrap** 훅이 감싸며, 결과가 나오면 **after** 계열 훅이 후처리를 담당합니다.

5가지 미들웨어 혹은:

혹	실행 시점	주요 용도
<code>\@before_model</code>	모델 호출 전	입력 검증, 메시지 수정, 가드레일
<code>\@after_model</code>	모델 응답 후	출력 로깅, 응답 필터링
<code>\@wrap_model_call</code>	모델 호출 감싸기	재시도, 폴백, 캐싱
<code>\@wrap_tool_call</code>	도구 호출 감싸기	도구 실행 제어
<code>\@dynamic_prompt</code>	프롬프트 생성 시	런타임 프롬프트 변경

6.3 빌트인 미들웨어

혹의 종류를 확인했으니, 가장 쉽게 시작할 수 있는 빌트인 미들웨어부터 살펴봅니다. LangChain v1은 자주 사용되는 패턴을 빌트인 미들웨어로 제공합니다. `SummarizationMiddleware` 는 대화가 길어지면 자동으로 이전 메시지를 요약하여 토큰 사용량을 줄입니다. 5장에서 `trim_messages()` 로 수동 트리밍을 했다면, 이 미들웨어는 자동 요약 방식으로 컨텍스트를 압축합니다. `trigger=("messages", 10)` 설정은 메시지가 10개를 초과하면 요약을 트리거합니다.

TIP

모든 빌트인 미들웨어의 크기 트리거는 `ContextSize` 튜플 형태로 통일되어 있습니다 — `(“tokens”, 100_000)`, `(“messages”, 20)`, `(“fraction”, 0.8)` (컨텍스트 윈도우의 80%) 세 가지를 그대로 받아 사용합니다.

```
from langchain.agents import create_agent
from langchain.tools import tool

@tool
def search(query: str) -> str:
    """정보를 검색합니다."""
    return f"'{query}'에 대한 검색 결과"

# SummarizationMiddleware — 긴 대화를 자동 요약
from langchain.agents.middleware import SummarizationMiddleware

summarization = SummarizationMiddleware(
    model=model,
    trigger=("messages", 10),
)

agent_with_summary = create_agent(
    model=model,
    tools=[search],
    system_prompt="당신은 유용한 어시스턴트입니다.",
    middleware=[summarization],
)

print("SummarizationMiddleware 에이전트 생성 완료")
```

SummarizationMiddleware 에이전트 생성 완료

프롬프트 캐싱 미들웨어

Anthropic·Bedrock 모델은 프롬프트 캐싱으로 토큰 비용을 크게 줄일 수 있습니다. LangChain은 두 제공자 모두를 위한 빌트인 미들웨어를 제공합니다.

```
from langchain.agents.middleware import (
    AnthropicPromptCachingMiddleware,
    BedrockPromptCachingMiddleware,
)
from langchain_anthropic import ChatAnthropic

claude = ChatAnthropic(model="claude-sonnet-4-6")
agent = create_agent(
    model=claude,
    tools=[search],
    middleware=[AnthropicPromptCachingMiddleware()],
)
# Bedrock도 동일하게 BedrockPromptCachingMiddleware()를 끼우면 됩니다.
```

ContextEditingMiddleware

도구 호출 결과가 컨텍스트를 잠식하기 시작할 때, `ContextEditingMiddleware` 는 오래된 도구 출력을 자동으로 비워 (`ClearToolUsesEdit`) 토큰을 회수합니다. `trigger` 는 임계값, `keep` 은 유지할 최근 메시지 수입니다.

```
from langchain.agents.middleware import (
    ContextEditingMiddleware,
    ClearToolUsesEdit,
)

context_edit = ContextEditingMiddleware(
    edits=[
        ClearToolUsesEdit(
            trigger=("tokens", 100_000),
            keep=("messages", 3),
        )
    ],
)
```

ModelFallbackMiddleware

`ModelFallbackMiddleware` 는 첫 모델이 실패할 경우 순서대로 다음 모델을 호출합니다. 첫 인자가 기본 모델이고, 이어지는 가변 인자가 폴백 후보입니다.

```
from langchain.agents.middleware import ModelFallbackMiddleware
from langchain_openai import ChatOpenAI

primary = ChatOpenAI(model="gpt-5.4")
backup = ChatOpenAI(model="gpt-5-nano")

fallback = ModelFallbackMiddleware(primary, backup)
agent = create_agent(model=primary, tools=[search], middleware=[fallback])
```

PatchToolCallsMiddleware (Deep Agents 핵심)

`PatchToolCallsMiddleware` 는 모델이 잘못된 형식의 도구 호출(잘못된 JSON, 누락된 인자 등)을 만들었을 때 자동으로 보정·재호출합니다. Deep Agents가 서브 에이전트와 파일시스템 도구를 안정적으로 실행하기 위해 내부에서 활용하는 컴포넌트로, 도구 호출 실패율이 높은 환경에서 특히 유용합니다.

6.4 커스텀 미들웨어: @before_model

빌트인 미들웨어로 해결되지 않는 요구사항이 있을 때, 커스텀 미들웨어를 작성합니다. 가장 많이 사용되는 혹은 @before_model 입니다.

@before_model 데코레이터는 모델이 호출되기 전에 실행됩니다. 이 함수의 인수는 메시지 리스트를 받아 수정된 메시지 리스트를 반환합니다. 메시지를 추가하거나, 필터링하거나, 변환할 수 있습니다.

주요 용도:

- 입력 메시지 로깅
- 메시지 수정 또는 필터링
- 입력 검증 (가드레일)
- 컨텍스트 추가 (현재 시간, 사용자 프로필 등)

6.5 커스텀 미들웨어: @after_model

@before_model 이 입력 쪽을 제어한다면, @after_model 은 출력 쪽을 제어합니다. @after_model 데코레이터는 모델 응답이 생성된 후에 실행됩니다. 이 함수의 인수는 AIMessage 를 받아 수정된 AIMessage 를 반환합니다.

주요 용도:

- 모델 출력 로깅 및 모니터링
- 응답에서 민감 정보(PII) 제거
- 도구 호출 감시 및 차단
- 출력 품질 검증 (환각 탐지 등)

6.6 @wrap_model_call

@before_model 과 @after_model 은 모델 호출의 전/후에 각각 동작하지만, 모델 호출 자체를 대체하거나 감싸야 할 때가 있습니다. 예를 들어 응답을 캐싱하거나, 실패 시 다른 모델로 폴백하는 경우입니다.

@wrap_model_call 데코레이터는 모델 호출 자체를 감싸서 재시도, 폴백, 캐싱 등의 로직을 구현할 수 있습니다.

handler 함수를 통해 원래의 모델 호출을 실행하며, 이 호출 전후로 커스텀 로직을 추가합니다.

handler(request) 를 호출하지 않으면 모델이 전혀 호출되지 않으므로, 캐시 히트 시 저장된 응답을 바로 반환하는 식으로 활용할 수 있습니다.

타입 어노테이션과 request.override

커스텀 미들웨어를 작성할 때는 ModelRequest / ModelResponse 타입을 명시해 IDE 보조와 정적 분석을 활성화하는 것이 좋습니다. request.override(...) 는 이번 호출에 한해서 메시지·도구·시스템 프롬프트·구조화 출력 스키마를 갈아끼웁니다(transient).

```
from langchain.agents.middleware import (
    wrap_model_call,
    ModelRequest,
    ModelResponse,
)

@wrap_model_call
def restrict_for_guest(request: ModelRequest, handler) -> ModelResponse:
    if request.runtime.context.role == "guest":
```



```
# 이번 턴만 시스템 프롬프트·도구 제한 (다음 턴에는 원본 복귀)
patched = request.override(
    system_message="읽기 전용 모드입니다.",
    tools=[t for t in request.tools if t.name.startswith("read_")],
)
return handler(patch)
return handler(request)
```

```
from langchain.agents.middleware import wrap_model_call
import time

@wrap_model_call
def retry_on_error(request, handler):
    """실패 시 지수 백오프로 모델 호출을 재시도합니다."""
    max_retries = 2
    for attempt in range(max_retries + 1):
        try:
            return handler(request)
        except Exception as e:
            if attempt < max_retries:
                wait = 2 ** attempt
                print(f"재시도 {attempt + 1}/{max_retries} ({wait}초 대기)")
                time.sleep(wait)
            else:
                raise

agent_retry = create_agent(
    model=model,
    tools=[search],
    system_prompt="당신은 유용한 어시스턴트입니다.",
    middleware=[retry_on_error],
)
print("재시도 미들웨어 에이전트 생성 완료")
```

재시도 미들웨어 에이전트 생성 완료

6.7 @dynamic_prompt

지금까지의 미들웨어가 메시지나 모델 호출을 제어했다면, `@dynamic_prompt` 는 에이전트의 근본적인 행동 지침 — 시스템 프롬프트 — 를 런타임에 변경합니다.

`@dynamic_prompt` 데코레이터는 런타임에 시스템 프롬프트를 동적으로 변경합니다. 이 함수의 인수는 런타임 컨텍스트를 받아 새로운 시스템 프롬프트 문자열을 반환합니다.

주요 용도:

- 현재 날짜/시간 정보 추가 (예: “오늘은 2026년 3월 8일입니다”)
- 사용자별 맞춤 프롬프트 (역할, 권한 레벨에 따라)
- 상태에 따른 행동 변경 (예: 오류 발생 시 보수적 모드)
- A/B 테스트 (다른 프롬프트의 효과 비교)

6.8 @wrap_tool_call

모델 호출뿐 아니라 도구 호출도 감쌀 수 있습니다. `@wrap_tool_call` 데코레이터는 도구 호출 자체를 감싸서 실행 전후에 커스텀 로직을 추가할 수 있습니다.

`@wrap_model_call` 이 모델 호출을 감싸는 것처럼, `@wrap_tool_call` 은 도구 실행을 감쌉니다. `handler` 함수를 호출하면 원래 도구가 실행되며, 그 전후로 타이밍 측정, 로깅, 에러 핸들링 등을 구현할 수 있습니다.

주요 용도:

- 실행 시간 측정: 도구별 성능 모니터링
- 로깅: 도구 입력/출력 기록
- 에러 핸들링: 도구 실패 시 폴백 처리
- 접근 제어: 특정 도구 호출 차단 또는 제한

6.9 가드레일

미들웨어의 모든 혹은 배웠으니, 이를 조합하여 에이전트의 안전성을 확보하는 방법을 다룹니다. 가드레일은 안전하지 않은 입력이나 출력을 차단하는 메커니즘입니다. 미들웨어를 활용하여 구현하며, 금지된 키워드 감지, 프롬프트 인젝션 방어, 민감 정보(PII) 필터링 등에 사용됩니다.

가드레일은 `@before_model` 훅에서 구현하는 것이 가장 효과적입니다. 모델에 전달되기 전에 위험한 입력을 차단할 수 있기 때문입니다. 출력 가드레일은 `@after_model` 훅에서 구현하며, 모델이 민감 정보를 포함한 응답을 생성했을 때 이를 마스킹하거나 차단합니다.

TIP

가드레일은 여러 계층으로 구성하는 것이 좋습니다. 입력 검증(`@before_model`), 출력 필터링(`@after_model`), 도구 접근 제어(`@wrap_tool_call`)를 조합하면 심층 방어(defense in depth) 전략을 구현할 수 있습니다.

can_jump_to로 그래프 경로 강제 종료

위험 입력을 발견했을 때 단순히 메시지만 바꾸는 대신, 그래프 자체를 점프시켜 모델 호출을 건너뛸 수 있습니다.

`@hook_config(can_jump_to=[...])` 로 허용된 목적지를 선언하고, 훅의 반환값에 `{"jump_to": ...}` 를 포함합니다.

```
from langchain.agents.middleware import before_model, hook_config
from langchain_core.messages import AIMessage

BANNED = {"비밀번호 알려줘", "system prompt 알려줘"}

@before_model
@hook_config(can_jump_to=["end", "tools", "model"])
def block_unsafe(state):
    last = state["messages"][-1].content
    if any(p in last for p in BANNED):
        return {
            "messages": [AIMessage(content="이 요청은 처리할 수 없습니다.")],
            "jump_to": "end", # 모델 호출 없이 즉시 종료
        }
    return None # 평소처럼 진행
```

비동기 훅 (a-prefixed)

I/O가 무거운 가드레일(외부 분류 API 호출 등)은 비동기 훅으로 작성합니다. 규칙은 단순합니다 – 동기 이름 앞에 `a` 접두어를 붙입니다(`abefore_model`, `aafter_model`, `awrap_model_call`, `awrap_tool_call`). 같은 미들웨어 클래스에 동기/비동기 훅을 함께 정의하면, `agent.invoke()` 는 동기 훅을, `agent.ainvoke()` 는 비동기 훅을 자동으로 사용합니다.

요약

이 노트북에서 학습한 미들웨어 타입을 정리합니다:

미들웨어 타입	데코레이터	실행 시점	주요 용도
Before Model	<code>\@before_model</code>	모델 호출 전	입력 로깅, 검증, 가드레일
After Model	<code>\@after_model</code>	모델 응답 후	출력 로깅, 필터링
Wrap Model	<code>\@wrap_model_call</code>	모델 호출 감싸기	재시도, 폴백, 캐싱
Wrap Tool	<code>\@wrap_tool_call</code>	도구 호출 감싸기	타이밍, 로깅, 에러 핸들링
Dynamic Prompt	<code>\@dynamic_prompt</code>	프롬프트 생성 시	런타임 프롬프트 변경
Builtin	<code>SummarizationMiddleware</code>	자동	대화 요약
Guardrail	<code>\@before_model</code> 활용	모델 호출 전	안전성 확보

핵심 포인트:

- 미들웨어는 에이전트 실행 파이프라인의 각 단계를 제어합니다.
- 여러 미들웨어를 조합하여 복잡한 로직을 구현할 수 있습니다.
- `@wrap_tool_call` 을 사용하면 도구 실행을 감싸서 타이밍 측정, 로깅, 에러 핸들링 등을 구현할 수 있습니다.
- 가드레일은 `@before_model` 훅에서 구현하는 것이 가장 효과적입니다.
- `@dynamic_prompt` 를 사용하면 런타임 정보를 시스템 프롬프트에 주입할 수 있습니다.

이 장에서는 에이전트 실행 파이프라인의 모든 단계를 제어하는 미들웨어 시스템을 학습했습니다. 다음 장에서는 에이전트가 위험한 작업을 실행하기 전에 사람의 승인을 받는 Human-in-the-Loop 패턴과, `ToolRuntime` 을 통한 런타임 컨텍스트 주입, 그리고 MCP를 통한 외부 도구 서버 연동을 다룹니다.

07 사람 개입(HITL)과 런타임

자율적으로 동작하는 에이전트가 항상 바람직한 것은 아닙니다. 결제 처리, 데이터 삭제, 외부 API 호출 같은 되돌리기 어려운 작업은 실행 전에 사람의 승인이 필요합니다. 이 장에서는 에이전트 실행을 일시 중지하고, 사람의 판단을 받은 후 재개하는 Human-in-the-Loop 패턴과, 런타임에 도구와 프롬프트를 동적으로 제어하는 기법을 다룹니다.

앞 장에서 미들웨어로 에이전트의 파이프라인을 제어하는 방법을 배웠습니다. 이 장은 그 연장선에서, 에이전트의 자율성과 안전성 사이의 균형을 잡는 실전 패턴을 다룹니다. HITL은 “위험한 도구 호출 전에 잠깐 멈추기”라는 단순한 개념이지만, 이를 구현하려면 `interrupt()` 로 상태를 저장하고 `Command(resume=value)` 로 재개하는 메커니즘이 필요합니다. 체크포인트(`InMemorySaver`)가 이 과정에서 중단된 상태를 보존하는 핵심 역할을 합니다.

학습 목표

도구 실행 전 사람의 승인을 받고, 런타임 컨텍스트를 주입합니다.

이 노트북에서 다루는 내용:

- Human-in-the-Loop (HITL): 에이전트가 위험한 도구를 실행하기 전에 사람의 승인을 받는 패턴
- ToolRuntime: 도구 실행 시 런타임 컨텍스트(사용자 정보 등)를 주입하는 방법
- 컨텍스트 엔지니어링: 동적으로 프롬프트와 도구를 제어하는 기법
- MCP (Model Context Protocol): 도구 서버를 표준 프로토콜로 연결하는 방식

7.1 환경 설정

```
from dotenv import load_dotenv
import os
load_dotenv(override=True)

from langchain_openai import ChatOpenAI

model = ChatOpenAI(
    model="gpt-5.4",
)

print("모델 준비 완료:", model.model_name)
```

모델 준비 완료: gpt-5.4

7.2 Human-in-the-Loop 개념

모델이 준비되었으니, HITL의 개념과 필요성부터 이해합니다. 에이전트가 도구를 호출하기 전에 사람의 승인을 요청합니다.

왜 필요한가?

자율적으로 동작하는 에이전트는 강력하지만, 이메일 전송, 파일 삭제, 결제 처리 같은 되돌릴 수 없는 작업에서는 사람의 확인이 필수적입니다. 에이전트가 의도와 다른 도구 호출을 시도하는 경우(hallucinated tool call)도 있으므로, 고위험 작업에는 반드시 검증 단계를 두는 것이 좋습니다.

워크플로

에이전트 → 도구 호출 제안 → [중단(interrupt)] → 사람 승인/거부 → 도구 실행 → 결과 반환

LangChain v1에서는 `HumanInTheLoopMiddleware` 와 `InMemorySaver` (체크포인트)를 결합하여 이 패턴을 구현합니다. 체크포인트는 에이전트의 상태를 저장하여 중단 후 재개할 수 있게 합니다.

7.3 HumanInTheLoopMiddleware

HITL의 개념을 이해했으니, 실제 구현을 살펴봅니다. `HumanInTheLoopMiddleware` 는 도구 호출 시 자동으로 실행을 중단하고, 사람의 승인을 기다리는 미들웨어입니다. `InMemorySaver` 체크포인트와 함께 사용하여 중단된 상태를 보존합니다. 내부적으로 LangGraph의 `interrupt()` 함수를 호출하여 그래프 실행을 일시 정지시킵니다.

```
from langchain.agents import create_agent
from langchain.tools import tool
from langchain.agents.middleware import HumanInTheLoopMiddleware
from langgraph.checkpoint.memory import InMemorySaver

@tool
def send_email(to: str, subject: str, body: str) -> str:
    """지정된 수신자에게 이메일을 보냅니다."""
    return f"{to}에게 이메일 전송 완료: {subject}"

@tool
def delete_file(path: str) -> str:
    """지정된 경로의 파일을 삭제합니다."""
    return f"파일 삭제 완료: {path}"

# 위험한 도구에만 승인 요구 (dict 형식 - 결정 종류 제한 가능)
hitl = HumanInTheLoopMiddleware(interrupt_on={
    "send_email": {"allowed_decisions": ["approve", "edit", "reject", "respond"]},
    "delete_file": {"allowed_decisions": ["approve", "reject"]},
})

agent = create_agent(
    model=model,
    tools=[send_email, delete_file],
    system_prompt="당신은 이메일을 보내고 파일을 관리할 수 있는 어시스턴트입니다.",
    middleware=[hitl],
    checkpointer=InMemorySaver(),
)

print("HITL 에이전트 생성 완료")
print(" -> 도구 호출 시 사람의 승인을 위해 중단됩니다")
```

HITL 에이전트 생성 완료
-> 도구 호출 시 사람의 승인을 위해 중단됩니다

7.4 interrupt와 Command(resume=...) 패턴

미들웨어로 HITL 에이전트를 만들었으니, 실제로 중단과 재개가 어떻게 동작하는지 살펴봅니다. HITL 에이전트는 2단계로 동작합니다:

- 1단계 (invoke): 에이전트가 도구 호출을 제안하면 자동으로 중단(interrupt)됩니다. 이때 `interrupt()` 함수가 호출되며, 체크포인트가 현재 그래프 상태를 저장합니다.
- 2단계 (Command(resume=...)): 사람의 결정을 `decisions` 리스트로 전달해 재개합니다. 체크포인트에서 저장된 상태를 복원하고, 중단된 지점부터 실행을 이어갑니다.

네 가지 결정 (approve / edit / reject / respond)

HumanInTheLoopMiddleware 는 결정을 dict 형식으로 받습니다. 각 도구 호출당 하나의 결정을 리스트에 담아 전달합니다.

```
from langgraph.types import Command

# 1) 승인 - 도구를 그대로 실행
agent.invoke(Command(resume={"decisions": [{"type": "approve"}]}),
              config=config, version="v2")

# 2) 편집 - 도구 인자를 사람이 수정해서 실행
agent.invoke(
    Command(resume={"decisions": [
        {"type": "edit", "args": {"to": "boss@example.com", "subject": "수정"}}
    ]}),
    config=config, version="v2",
)

# 3) 거부 - 도구 실행을 막고 에이전트에게 메시지 반환
agent.invoke(Command(resume={"decisions": [{"type": "reject"}]}),
              config=config, version="v2")

# 4) 응답 - 도구 실행 없이 사람의 메시지로 대체
agent.invoke(
    Command(resume={"decisions": [
        {"type": "respond", "message": "이메일 대신 슬랙으로 보내주세요"}
    ]}),
    config=config, version="v2",
)
```

version="v2"로 GraphOutput 받기

`agent.invoke(..., version="v2")` 는 `GraphOutput` 을 반환합니다. 인터럽트 상태에서는 `result.value` 가 부분 상태이고 `result.interrupts` 에 `Interrupt` 객체가 담깁니다.

```
result = agent.invoke(
    {"messages": [{"role": "user", "content": "팀장에게 보고 메일 보내줘"}]},
    config={"configurable": {"thread_id": "t1"}},
    version="v2",
)

if result.interrupts:
    pending = result.interrupts[0].value # 승인 대기 중인 도구 호출 정보
    decision = ask_human(pending)       # UI에서 결정 수집
    final = agent.invoke(
```

```
Command(resume={"decisions": [decision]}),
config={"configurable": {"thread_id": "t1"}},
version="v2",
)
print(final.value["messages"][-1].content)
```

! WARNING

HITL 패턴은 반드시 체크포인트와 함께 사용해야 합니다. 체크포인트 없이 `interrupt()` 를 호출하면, 중단된 상태를 복원할 수 없어 재개가 불가능합니다.

→ TIP

`version="v2"` 스트리밍과 dict 기반 `decisions` 형식은 `deepagents ≥ 0.5.0` 또는 `langgraph ≥ 1.1.5` 에서 지원됩니다. 이전 버전에서는 `Command(resume=True/False)` 단일 결정 방식이 그대로 동작합니다.

라이프사이클 – after_model hook과 HITLRequest

내부적으로 `HumanInTheLoopMiddleware` 는 `after_model` hook에서 도구 호출을 가로채 `HITLRequest` 를 생성합니다. `HITLRequest` 는 `action_requests` (검토 대상 도구 호출)와 `review_configs` (각 호출별 허용 결정 종류)를 담아 인터럽트로 전파됩니다. 커스텀 승인 흐름을 직접 구현하려면 `after_model` hook 안에서 동일한 객체를 직접 만들어 반환하면 됩니다.

transient 패치 vs persistent 패치

승인 결과에 따라 도구 인자만 일회성으로 바꾸려면 `request.override(...)` (이 호출만 적용, transient)를 사용하고, 그래프 상태 자체를 갱신해야 한다면 `ExtendedModelResponse` 와 `Command(update=...)` 를 함께 반환 (persistent)해 체크포인트에 반영합니다.

7.5 ToolRuntime – 도구에서 런타임 정보에 접근합니다

HITL로 안전성을 확보했다면, 이제 도구가 실행 환경의 정보에 접근하는 방법을 다룹니다. 4장에서 `ToolRuntime` 을 간단히 소개했는데, 여기서는 `context_schema` 를 활용한 본격적인 사용법을 살펴봅니다.

`ToolRuntime` 은 도구가 실행될 때 런타임 컨텍스트(현재 사용자 정보, 세션 데이터, DB 커넥션 등)에 접근할 수 있게 해주는 메커니즘입니다. 중요한 점은, `ToolRuntime` 매개변수는 LLM에게 노출되지 않는다는 것입니다. 모델은 이 매개변수의 존재를 모르며, 값을 생성하지도 않습니다. 런타임이 자동으로 주입합니다.

핵심 아이디어

- 도구 함수에 `runtime: ToolRuntime[T]` 파라미터를 추가합니다.
- `T` 는 개발자가 정의하는 컨텍스트 데이터 클래스입니다. (예: 사용자 ID, 권한 레벨, DB 커넥션)
- 에이전트 생성 시 `context_schema=T` 를 지정하고, 호출 시 `context=T(...)` 로 값을 전달합니다.

runtime.execution_info – 실행 메타데이터

`runtime.execution_info` 는 이 호출을 식별하는 정보를 제공합니다. 로깅·트레이싱이나 재시도 카운트 처리에 유용합니다.

속성	설명
thread_id	현재 대화 스레드 식별자
run_id	개별 invoke/stream 호출 식별자
attempt	재시도 시 1부터 증가하는 시도 번호

```
@tool
def call_external_api(payload: dict, runtime: ToolRuntime[Context]) -> str:
    info = runtime.execution_info
    headers = {
        "X-Trace-Run": info.run_id,
        "X-Trace-Thread": info.thread_id,
        "X-Attempt": str(info.attempt),
    }
    return http.post(url, json=payload, headers=headers).text
```

runtime.server_info — 배포 메타데이터

LangGraph Platform/Server에 배포된 경우 `runtime.server_info` 로 배포 식별자와 현재 요청 사용자를 확인할 수 있습니다. 로컬 실행에서는 값이 `None` 이거나 비어 있을 수 있습니다.

속성	설명
assistant_id	Platform에 등록된 어시스턴트(설정 묶음) ID
graph_id	그래프 이름(<code>langgraph.json</code> 의 키)
user	인증된 사용자 정보(있는 경우)

```
@tool
def log_who(runtime: ToolRuntime[Context]) -> str:
    s = runtime.server_info
    return f"{s.graph_id}/{s.assistant_id} called by {s.user}"
```

7.6 컨텍스트 엔지니어링 – 동적으로 프롬프트와 도구를 제어합니다

`ToolRuntime` 으로 도구에 런타임 정보를 주입하는 방법을 배웠으니, 한 단계 더 나아가 에이전트의 전체 컨텍스트를 동적으로 제어하는 기법을 살펴봅니다.

컨텍스트 엔지니어링은 에이전트에게 전달되는 프롬프트, 도구, 메시지 히스토리를 동적으로 조작하는 기법입니다. 6장에서 배운 `@dynamic_prompt` 가 프롬프트를 런타임에 변경한다면, `@dynamic_tools` 는 사용 가능한 도구 목록 자체를 런타임에 필터링합니다.

주요 활용 사례

- 사용자 역할에 따라 다른 시스템 프롬프트 제공 (`@dynamic_prompt`)
- 상황이나 권한에 따라 사용 가능한 도구 필터링 (`@dynamic_tools`)
- 긴 대화 히스토리 요약 및 정리 (`SummarizationMiddleware`)

`dynamic_prompt` 미들웨어를 사용하면 매 요청마다 프롬프트를 커스터마이징할 수 있습니다. `@dynamic_tools` 는 런타임 컨텍스트(예: 사용자 권한)에 따라 에이전트가 접근할 수 있는 도구를 동적으로 결정합니다. 예를 들어, 관리자에게는 `delete_file` 도구를 제공하고 일반 사용자에게는 읽기 전용 도구만 제공하는 식입니다.

7.7 MCP (Model Context Protocol) 연동 개요

지금까지는 Python 함수로 직접 도구를 정의했습니다. 하지만 도구가 다른 언어로 작성되어 있거나, 별도의 서버에서 실행되어야 하는 경우가 있습니다. MCP는 이런 상황을 위한 표준화된 도구 프로토콜입니다.

MCP는 도구 서버를 표준 프로토콜로 연결하는 방식입니다.

MCP의 핵심 개념

- MCP 서버: 도구(Tool)를 제공하는 서버. stdio(로컬 프로세스 간 통신) 또는 SSE(HTTP 기반 스트리밍) 두 가지 전송 방식을 지원합니다.
- MCP 클라이언트: 에이전트가 MCP 서버에 연결하여 도구를 자동으로 발견(discover)하고 호출합니다.
- 표준화: 어떤 언어/프레임워크로 만든 도구든 MCP 프로토콜을 따르면 연결 가능합니다. Python, TypeScript, Go 등으로 작성된 도구를 동일한 방식으로 사용할 수 있습니다.

LangChain v1에서의 MCP 지원

- `langchain-mcp-adapters` 패키지의 `load_mcp_tools()` 로 MCP 서버의 도구를 자동으로 로드할 수 있습니다.
- 로드된 도구는 일반 LangChain 도구와 완전히 동일한 인터페이스를 가지므로, `create_agent()` 의 `tools` 매개변수에 그대로 전달할 수 있습니다.

TIP

MCP 생태계에는 이미 파일 시스템, 데이터베이스, GitHub, Slack 등 수백 개의 사전 구축된 서버가 있습니다. 직접 도구를 구현하기 전에 기존 MCP 서버가 있는지 확인해 보세요.

```
# MCP 연동 개념 (실행하려면 MCP 서버가 필요합니다)
print("MCP (Model Context Protocol) 연동:")
print("=" * 50)
print("""
# MCP 서버를 도구로 연결하는 예시:

from langchain_mcp_adapters.tools import load_mcp_tools

# MCP 서버에서 도구 로드
tools = load_mcp_tools("http://localhost:8080/mcp")

# 에이전트에 MCP 도구 연결
agent = create_agent(
    model=model,
    tools=tools,
    system_prompt="MCP 도구를 사용할 수 있습니다.",
)
""")
print("-> MCP는 표준화된 도구 탐색 및 호출을 가능하게 합니다.")
```

MCP (Model Context Protocol) 연동:

=====

```
# MCP 서버를 도구로 연결하는 예시:

from langchain_mcp_adapters.tools import load_mcp_tools

# MCP 서버에서 도구 로드
tools = load_mcp_tools("http://localhost:8080/mcp")

# 에이전트에 MCP 도구 연결
agent = create_agent(
    model=model,
    tools=tools,
    system_prompt="MCP 도구를 사용할 수 있습니다.",
)

-> MCP는 표준화된 도구 탐색 및 호출을 가능하게 합니다.
```

요약

이 노트북에서 학습한 핵심 내용:

개념	설명	핵심 API
HITL	도구 실행 전 사람의 승인 요청	HumanInTheLoopMiddleware , Command(resume=...)
ToolRuntime	도구에서 런타임 컨텍스트 접근	ToolRuntime[T] , context_schema
컨텍스트 엔지니어링	동적 프롬프트/도구 제어	dynamic_prompt 미들웨어
MCP	표준화된 도구 프로토콜	load_mcp_tools() (langchain-mcp-adapters 패키지)

이 장으로 Part 2(LangChain)의 핵심 개념을 모두 다뤘습니다. 에이전트 생성(`create_agent`), 모델과 메시지, 도구와 구조화된 출력, 메모리와 스트리밍, 미들웨어와 가드레일, 그리고 HITL과 런타임 제어까지 – 이 기반 위에 다음 Part에서는 LangGraph를 사용하여 멀티 에이전트 워크플로와 복잡한 상태 기계를 구축합니다.

08 멀티 에이전트 패턴

복잡한 작업을 단일 에이전트로 처리하면 시스템 프롬프트가 비대해지고, 도구가 넘쳐나며, 컨텍스트 윈도우가 빠르게 소진됩니다. 멀티 에이전트 아키텍처는 작업을 전문 에이전트로 분할하여 이 문제를 해결합니다. 이 장에서는 LangChain v1이 제공하는 5가지 멀티 에이전트 패턴 – Subagents, Handoffs, Skills, Router, Custom – 의 개념과 구현을 비교합니다.

단일 에이전트의 한계는 단순히 프롬프트 길이의 문제가 아닙니다. 도구가 20개를 넘어가면 LLM이 적절한 도구를 선택하는 정확도가 떨어지고, 서로 다른 도메인의 지시사항이 충돌하여 예측 불가능한 동작이 발생할 수 있습니다. 멀티 에이전트 패턴은 “관심사의 분리(Separation of Concerns)” 원칙을 에이전트 설계에 적용한 것으로, 각 에이전트가 명확한 책임 범위와 최소한의 도구 집합을 갖도록 합니다.

학습 목표

5가지 멀티 에이전트 패턴을 이해하고 구현합니다.

이 노트북에서 다루는 내용:

- Subagents: 메인 에이전트가 전문 서브에이전트를 도구로 호출
- Handoffs: `Command(goto=...)` 로 에이전트 간 상태 전환
- Skills: 단일 에이전트가 상황에 따라 전문 프롬프트를 로드
- Router: 분류기가 입력을 적절한 에이전트로 라우팅
- Custom: 개발자가 완전히 제어하는 복잡한 워크플로

8.1 환경 설정

```
from dotenv import load_dotenv
import os
load_dotenv(override=True)

from langchain_openai import ChatOpenAI

model = ChatOpenAI(
    model="gpt-5.4",
)

print("모델 준비 완료:", model.model_name)
```

모델 준비 완료: gpt-5.4

8.2 멀티 에이전트 패턴 비교

아래 표는 5가지 멀티 에이전트 패턴을 비교합니다. 각 패턴은 서로 다른 상황에 적합하며, 프로젝트의 요구사항에 맞게 선택해야 합니다.

패턴	라우팅 주체	상태 공유	적합한 상황
Subagents	메인 에이전트	도구로 격리	병렬 처리, 분산 개발
Handoffs	도구 호출	상태 전환	순차적 멀티홉
Skills	단일 에이전트	프롬프트 교체	도메인 특화
Router	분류기	병렬 실행	멀티 도메인
Custom	개발자 정의	완전 제어	복잡한 워크플로

핵심 차이점

- Subagents는 각 서브에이전트가 독립적으로 실행되어 결과만 반환합니다.
- Handoffs는 대화 상태가 에이전트 간에 전달됩니다.
- Skills는 하나의 에이전트가 여러 역할을 전환합니다.
- Router는 입력을 분류한 뒤 적절한 에이전트에게 위임합니다.

이 다섯 패턴은 복잡도와 유연성의 스펙트럼 위에 놓여 있습니다. Subagents와 Skills는 구현이 간단하지만 유연성이 제한적이고, Handoffs와 Router는 중간 수준의 복잡도로 대부분의 실전 시나리오를 커버하며, Custom은 최대한의 유연성을 제공하되 개발 비용이 가장 높습니다. 각 패턴의 동작 원리와 트레이드오프를 하나씩 살펴보겠습니다.

패턴 선택 매트릭스 (요구사항 → 패턴)

비교 표가 패턴의 속성을 정리한 것이라면, 아래 매트릭스는 요구사항에서 출발하여 최적 패턴을 역으로 찾는 표입니다.

요구사항	최적 패턴
단일 단순 작업 (one-shot)	Skills (또는 단일 에이전트)
같은 작업을 반복 호출	Subagents
멀티 도메인을 병렬 처리	Router (분류 후 fan-out)
매우 큰 컨텍스트 (긴 문서·이력)	Subagents (컨텍스트 격리로 토큰 절약)
팀 기반 – 에이전트가 서로 협업	Handoffs (상태 공유)
에이전트와 직접 대화 (역할 유지)	Handoffs + supervisor

패턴별 토큰·호출 비용 (참고 수치)

같은 작업을 서로 다른 패턴으로 구현했을 때의 대략적인 호출 수와 누적 토큰을 비교합니다. 수치는 시나리오와 모델에 따라 달라지지만, 패턴 간 상대적 비용을 가늠하는 데 도움이 됩니다.

시나리오	패턴	호출 수	누적 토큰
One-Shot (단일 도메인)	Subagents	4-5 calls	9K
Repeat (같은 작업 반복)	Skills	2-3 calls	15K
Multi-Domain (협업)	Handoffs	3-7+ calls	14K+
Multi-Domain (분류 후 fan-out)	Router	~분류 1 + 도메인 N	9K

TIP

Router vs Supervisor – 두 용어는 자주 혼용되지만 구현 측면에서 다릅니다. Router는 입력을 한 번 분류하여 적절한 에이전트(또는 Send)로 fan-out 하는 단순 함수형 노드입니다. 반면 Supervisor는 대화 이력을 가지고 conversation-aware 하게 어떤 서브에이전트를 호출할지 매 턴 결정하는 에이전트입니다. 일회성 분류라면 Router, 다단계 협업이 필요하면 Supervisor 패턴을 선택하세요.

8.3 서브에이전트 패턴

메인 에이전트(감독자)가 전문 서브에이전트를 도구로 호출하는 패턴입니다. 핵심 아이디어는 `create_agent()` 로 생성한 에이전트를 `@tool` 데코레이터로 감싸서, 상위 에이전트가 일반 도구처럼 호출할 수 있게 만드는 것입니다.

특징

- 각 서브에이전트는 도구 함수로 캡슐화됩니다.
- 메인 에이전트가 어떤 서브에이전트를 호출할지 판단합니다.
- 서브에이전트의 내부 상태는 메인 에이전트와 격리됩니다. – 서브에이전트의 중간 추론 과정이나 도구 호출 이력은 메인 에이전트의 컨텍스트를 오염시키지 않습니다.
- 병렬 실행이 가능하여 성능에 유리합니다.

TIP

서브에이전트 패턴은 “컨텍스트 격리”가 가장 큰 장점입니다. 서브에이전트가 내부적으로 여러 도구를 호출하더라도 메인 에이전트에는 최종 결과 문자열만 전달되므로, 메인 에이전트의 컨텍스트 윈도우를 절약할 수 있습니다.

서브에이전트 로컬 히스토리 (`checkpointer=True`)

서브에이전트를 `create_agent(..., checkpointer=True)` 로 만들면 서브에이전트만의 대화 이력을 별도 스레드에 보관할 수 있습니다. 메인 에이전트는 최종 결과만 전달받지만, 서브에이전트 내부에서는 멀티턴 추론이 가능합니다. 동일 `thread_id` 로 재호출하면 이전 컨텍스트를 이어받아 점진적으로 작업을 진행할 수 있습니다.

단일 dispatch 도구로 N개 서브에이전트 노출

서브에이전트가 많을 때 각각을 개별 도구로 노출하면 메인 에이전트의 도구 목록이 비대해집니다. 대안은 단일 `dispatch(agent_name, query)` 도구 하나로 묶고, 어떤 서브에이전트가 있는지를 다른 방식으로 알려주는 것입니다.

방식	권장 규모	특징
시스템 프롬프트에 <code>enumerate</code>	< 10	LLM이 이름을 텍스트로 인식. 간단·유연
<code>Literal["a", "b", ...]</code> 타입 제약	10-30	툴 스키마로 validation. 잘못된 이름 차단
Tool-based discovery (<code>list_agents</code>)	> 30	동적 등록. 별도 도구로 검색·필터링

부모 state 주입: `ToolRuntime[None, CustomState]`

서브에이전트 도구가 부모 그래프의 state 일부를 읽어야 할 때 `ToolRuntime[None, CustomState]` 를 인자로 받습니다.

```
from langchain.tools import tool, ToolRuntime

@tool
def research_subagent(query: str, runtime: ToolRuntime[None, ResearchState]) -> str:
    """리서치 서브에이전트. 부모 state의 user_id를 사용."""
    user_id = runtime.state["user_id"]
    # ... 서브에이전트 호출 ...
    return result
```

비동기 작업: start / status / get_result 3-tool 패턴

오래 걸리는 서브에이전트는 단일 동기 호출 대신 세 개 도구로 쪼개는 것이 안전합니다.

```
@tool
def start_research_job(query: str) -> str:
    """리서치 작업을 시작하고 job_id를 반환합니다."""
    ...

@tool
def check_job_status(job_id: str) -> str:
    """작업 상태 - running / done / failed."""
    ...

@tool
def get_job_result(job_id: str) -> str:
    """완료된 작업의 결과를 가져옵니다."""
    ...
```

Command 반환으로 부모 state 업데이트

서브에이전트 도구가 단순 문자열 대신 `Command(update={...})` 를 반환하면 부모 그래프의 state 를 직접 갱신할 수 있습니다. 메시지뿐 아니라 커스텀 필드까지 업데이트해야 할 때 유용합니다.

```
from langgraph.types import Command

@tool
def research_with_state_update(query: str) -> Command:
    result = run_research(query)
    return Command(update={
```

```

        "research_results": result,
        "messages": [{"role": "tool", "content": result}],
    })

```

서브에이전트 패턴이 도구 호출을 통해 결과만 받아오는 “위임” 방식이라면, 핸드오프 패턴은 대화의 흐름 자체를 다른 에이전트로 넘기는 방식입니다.

8.4 핸드오프 패턴

`Command(goto=...)` 로 에이전트 간 상태를 전환하는 패턴입니다. 핸드오프는 LangGraph의 노드 전환 메커니즘을 활용합니다. 에이전트 내부의 도구가 `Command(goto="target_agent")` 객체를 반환하면, LangGraph 런타임이 현재 에이전트의 실행을 중단하고 대상 에이전트 노드로 제어를 이동시킵니다.

특징

- 도구가 `Command` 객체를 반환하여 다른 에이전트로 전환합니다.
- 대화 상태(메시지 히스토리)가 다음 에이전트로 전달됩니다. — 서브에이전트 패턴과 달리 이전 대화의 전체 맥락을 유지합니다.
- `StateGraph` 를 사용하여 에이전트 간 흐름을 정의합니다.
- 고객 서비스의 부서 이관 같은 순차적 멀티홉 시나리오에 적합합니다.

! WARNING

핸드오프 패턴에서 대화 상태가 모든 에이전트에 공유되므로, 에이전트 수가 많아지면 컨텍스트 윈도우가 빠르게 소진될 수 있습니다. 핸드오프 체인이 3단계 이상 필요하다면, 중간에 상태를 요약하거나 서브에이전트 패턴과 조합하는 것을 고려하세요.

단일 에이전트 + 미들웨어 핸드오프 (권장 1순위)

LangChain v1 docs 의 1순위 권장 핸드오프 패턴은 여러 subgraph 가 아니라 단일 에이전트 +

`@wrap_model_call` 미들웨어입니다. 라우팅 결과에 따라 시스템 프롬프트와 도구 집합을 동적으로 override 하여, 하나의 에이전트 노드 안에서 페르소나를 전환합니다.

```

from langchain.agents.middleware import wrap_model_call, ModelRequest
from langchain.agents import create_agent

@wrap_model_call
def role_router(request: ModelRequest, handler):
    state = request.state
    role = state.get("active_role", "general")

    if role == "billing":
        return handler(request.override(
            system_prompt="당신은 결제 전문 상담원입니다.",
            tools=[refund_tool, charge_tool],
        ))
    elif role == "tech":
        return handler(request.override(
            system_prompt="당신은 기술 지원 상담원입니다.",
            tools=[diagnose_tool, escalate_tool],
        ))
    return handler(request)

```

```
agent = create_agent(model="gpt-5.4", tools=ALL_TOOLS, middleware=[role_router])
```

기존의 다중 subgraph 핸드오프 패턴은 부서 간 명시적 그래프 흐름을 보여줘야 할 때 여전히 유효하지만, 단순한 페르소나 전환이라면 위 미들웨어 패턴이 훨씬 가볍습니다.

핸드오프 도구의 ToolRuntime + tool_call_id 에코백

핸드오프 도구는 ToolRuntime[None, SupportState] 로 부모 state 와 현재 tool_call_id 에 접근해야 합니다.

Command(goto=..., update={"messages": [...]}) 로 전환할 때, 현재 도구 호출에 대응하는 ToolMessage 를 반드시 함께 넣어야 OpenAI tool-call 규약이 깨지지 않습니다.

```
from langchain.tools import tool, ToolRuntime
from langchain_core.messages import ToolMessage
from langgraph.types import Command

@tool
def transfer_to_billing(reason: str, runtime: ToolRuntime[None, SupportState]) -> Command:
    """결제팀으로 상담을 이관합니다."""
    return Command(
        goto="billing_agent",
        update={
            "messages": [
                ToolMessage(
                    content=f"결제팀으로 이관: {reason}",
                    tool_call_id=runtime.tool_call_id,
                ),
            ],
            "active_role": "billing",
        },
    )
```

TIP

subgraph 핸드오프 시 흘러야 할 메시지는 정확히 2개 – (1) 핸드오프 도구를 호출한

AIMessage(tool_calls=...) 와 (2) 그에 대한 ToolMessage 두 개만 다음 subgraph로 전달되어야 합니다. 그 사이에 다른 메시지를 끼워 넣으면 OpenAI provider가 tool_call 짝이 맞지 않는다고 거부합니다.

핸드오프가 여러 에이전트 사이에서 대화를 이동시키는 것이라면, 스킬 패턴은 하나의 에이전트가 역할을 전환하는 보다 경량화된 접근법입니다.

8.5 스킬 패턴

단일 에이전트가 상황에 따라 전문 프롬프트를 로드하는 패턴입니다. 여러 에이전트를 별도로 생성하는 대신, 하나의 에이전트가 도메인별 시스템 프롬프트를 교체하며 전문가처럼 동작합니다.

특징

- 하나의 에이전트가 여러 “스킬”을 가집니다.
- 각 스킬은 특화된 시스템 프롬프트입니다.
- 에이전트가 필요한 스킬을 동적으로 로드합니다.
- 여러 에이전트를 관리할 필요 없이 하나의 에이전트로 다양한 작업을 처리할 수 있습니다.

TIP

스킬 패턴은 오버헤드가 가장 적은 멀티 에이전트 패턴입니다. 에이전트 인스턴스가 하나뿐이므로 상태 관리와 그래프 구성이 불필요합니다. 도메인 간 도구 집합이 겹치지 않고, 프롬프트 전환만으로 충분한 경우에 가장 효율적입니다.

스킬 패턴이 하나의 에이전트 내부에서 프롬프트를 전환하는 것이라면, 라우터 패턴은 입력 단계에서 어느 에이전트가 처리할지를 먼저 결정하는 방식입니다.

8.6 라우터 패턴

분류기가 입력을 적절한 에이전트로 라우팅하는 패턴입니다. 라우터 노드는 사용자의 쿼리를 분석하여 카테고리를 판별한 뒤, 해당 카테고리 전문 에이전트로 요청을 전달합니다. 분류 로직은 LLM을 사용할 수도 있고, 키워드 매칭이나 임베딩 유사도 같은 규칙 기반으로도 구현할 수 있습니다.

특징

- 먼저 쿼리를 분류(classify)합니다.
- 분류 결과에 따라 적절한 전문 에이전트(도구)로 위임합니다.
- 멀티 도메인 시스템에서 유용합니다.
- 분류 로직은 규칙 기반 또는 LLM 기반으로 구현할 수 있습니다.

TIP

라우터의 분류기를 LLM 기반으로 구현할 때는 경량 모델(예: `gpt-5.4-mini`)을 사용하는 것이 비용 효율적입니다. 분류 작업은 복잡한 추론이 필요하지 않으므로, 빠르고 저렴한 모델로도 충분한 정확도를 얻을 수 있습니다.

Send 로 멀티 도메인 fan-out

쿼리가 여러 도메인을 동시에 다뤄야 할 때는 단일 라우팅 대신 `Send` 로 병렬 fan-out 합니다.

`add_conditional_edges` 의 라우팅 함수가 `list[Send]` 를 반환하면 LangGraph 런타임이 해당 노드들을 병렬로 실행합니다.

```
from langgraph.types import Command, Send

def route_to_agents(state: RouterState) -> list[Send]:
    """분류 결과에 포함된 모든 카테고리로 fan-out."""
    return [
        Send(c["agent"], {"query": c["query"]})
        for c in state["classifications"]
    ]

graph.add_conditional_edges("classifier", route_to_agents,
                           ["billing_agent", "tech_agent", "general_agent"])
```

8.7 패턴 선택 가이드

어떤 멀티 에이전트 패턴을 선택해야 할까요? 아래 가이드를 참고하세요.

결정 트리

1. 에이전트가 독립적으로 작업 가능한가?
 - YES -> Subagents (병렬 실행, 결과 취합)
 - NO -> 다음 질문으로
1. 대화 상태가 에이전트 간에 전달되어야 하는가?
 - YES -> Handoffs (상태 전환, 멀티홉)
 - NO -> 다음 질문으로
1. 하나의 에이전트가 여러 역할을 전환하면 되는가?
 - YES -> Skills (프롬프트 교체)
 - NO -> 다음 질문으로
1. 입력을 분류한 뒤 적절한 처리기로 보내면 되는가?
 - YES -> Router (분류 후 위임)
 - NO -> Custom (완전 커스텀 그래프)

실용적 권장사항

- 처음에는 가장 단순한 패턴(Subagents 또는 Skills)으로 시작하세요.
- 요구사항이 복잡해지면 Handoffs나 Router로 전환하세요.
- Custom 패턴은 다른 패턴으로 해결할 수 없을 때만 사용하세요.
- 패턴을 조합할 수도 있습니다 (예: Router + Handoffs).

요약

이 노트북에서 학습한 핵심 내용:

패턴	핵심 API	사용 시기
Subagents	<code>create_agent</code> + 도구 함수	독립적 병렬 작업
Handoffs	<code>Command(goto=...)</code> , <code>StateGraph</code>	상태 전환이 필요한 멀티홉
Skills	도구로 프롬프트 로드	단일 에이전트 다중 역할
Router	분류 도구 + 전문 도구	멀티 도메인 분류
Custom	<code>StateGraph</code> 완전 제어	복잡한 비즈니스 로직

핵심 원칙

- 단순한 것부터 시작하고, 필요에 따라 복잡도를 높이세요.
- 각 에이전트는 하나의 책임만 가지도록 설계하세요.

- 에이전트 간 인터페이스(입력/출력)를 명확히 정의하세요.

다음 장에서는 이 패턴들 중 Custom 패턴의 핵심인 `StateGraph` 를 본격적으로 활용하여, 에이전트를 워크플로 노드로 통합하고 RAG(Retrieval-Augmented Generation) 파이프라인을 구현하는 방법을 학습합니다.

09 커스텀 워크플로와 RAG

지금까지 `create_agent()` 가 내부적으로 처리해주던 워크플로를 직접 구성해야 할 때가 있습니다 – 조건부 분기, 에이전트를 노드로 포함하는 복합 파이프라인, 검색 증강 생성(RAG) 등. 이 장에서는 LangGraph의 `StateGraph` 를 사용하여 LangChain 에이전트를 워크플로 노드로 통합하고, 실전에서 가장 많이 사용되는 RAG 패턴을 구현합니다.

8장의 멀티 에이전트 패턴 중 Custom 패턴이 바로 이 `StateGraph` 를 직접 다루는 방식입니다. `create_agent()` 는 내부적으로 “모델 호출 → 도구 실행 → 반복” 루프를 자동으로 구성하지만, 검색 결과의 품질을 평가하여 재검색할지 결정하거나, 생성된 답변의 환각 여부를 검증하는 등의 비선형 로직은 개발자가 직접 그래프를 설계해야 합니다.

학습 목표

LangGraph `StateGraph` 로 커스텀 워크플로를 만들고, RAG 패턴을 구현합니다.

이 노트북에서 다루는 내용:

- `StateGraph` 의 기본 구조 (노드, 엣지, 상태)
- 조건부 엣지를 활용한 분기 처리
- `create_agent` 를 워크플로 노드로 통합
- RAG (Retrieval-Augmented Generation) 패턴 구현

9.1 환경 설정

```
from dotenv import load_dotenv
import os
load_dotenv(override=True)

from langchain_openai import ChatOpenAI

model = ChatOpenAI(
    model="gpt-5.4",
)

from langchain.agents import create_agent
from langchain.tools import tool

print("환경 준비 완료.")
```

환경 준비 완료.

9.2 StateGraph 기초

LangGraph의 핵심 빌딩 블록을 살펴봅니다.

개념	설명
노드(Node)	처리 단위. 함수 또는 에이전트가 될 수 있습니다
엣지(Edge)	노드 간 연결. 실행 흐름을 정의합니다
상태(State)	노드 간 공유 데이터. <code>TypedDict</code> 로 정의합니다

`StateGraph` 는 이 세 가지를 조합하여 복잡한 워크플로를 구성할 수 있게 합니다. 상태는 `TypedDict` 로 정의하며, 그래프의 모든 노드가 이 상태를 읽고 쓸 수 있습니다. 노드는 상태를 입력으로 받아 변경된 부분만 딕셔너리로 반환하는 순수 함수입니다. 엣지는 노드 간의 실행 순서를 결정하며, 정적 엣지(항상 같은 다음 노드)와 조건부 엣지(상태에 따라 동적으로 결정)로 나뉩니다.

TIP

`StateGraph` 를 구성할 때는 `START → 첫 번째 노드 → ... → END` 형태로 명시적 엣지를 연결해야 합니다. 엣지를 빠뜨리면 해당 노드가 실행되지 않으므로, 그래프 시각화(`graph.get_graph().draw_mermaid()`)로 연결 상태를 확인하는 습관을 들이세요.

9.3 조건부 엣지

상태에 따라 다른 경로로 분기합니다. `add_conditional_edges` 를 사용하면 런타임에 동적으로 다음 노드를 선택할 수 있습니다. 조건부 엣지의 핵심은 라우팅 함수입니다. 이 함수는 현재 상태를 받아 다음에 실행할 노드의 이름(문자열)을 반환합니다.

아래 예제에서는 입력 텍스트를 분류한 후, 카테고리에 따라 서로 다른 핸들러로 라우팅합니다.

조건부 엣지를 통해 그래프 내에서 동적 분기가 가능해졌습니다. 다음으로, 이 노드 자리에 `create_agent` 로 생성한 에이전트를 배치하여 에이전트 기반 워크플로를 구성해 보겠습니다.

9.4 에이전트를 워크플로에 통합

`create_agent` 로 만든 에이전트를 `StateGraph` 의 노드로 사용합니다. 에이전트를 노드로 통합하면, 에이전트 내부에서는 자유롭게 도구를 호출하면서도 에이전트 간의 실행 순서와 데이터 흐름은 그래프가 엄격하게 제어합니다. 이렇게 하면 여러 에이전트를 파이프라인으로 연결하여 복잡한 작업을 처리할 수 있습니다.

아래 예제에서는 리서치 에이전트와 작성 에이전트를 순차적으로 연결합니다.

! WARNING

에이전트를 `StateGraph` 노드로 사용할 때, 에이전트의 출력 상태 키가 그래프의 상태 스키마와 일치해야 합니다. 특히 `messages` 키는 `LangGraph`의 `add_messages` 리듀서를 사용하여 메시지가 덮어쓰기 대신 누적되도록 설정하세요.

지금까지 `StateGraph`의 기본 구조와 에이전트 통합 방법을 살펴보았습니다. 이제 커스텀 워크플로의 가장 대표적인 실전 사례인 RAG 파이프라인을 구현해 보겠습니다.

9.5 RAG (Retrieval-Augmented Generation) 개요

RAG는 외부 지식을 검색하여 LLM의 응답을 보강하는 패턴입니다. LLM은 학습 데이터에 포함되지 않은 최신 정보나 조직 내부 문서에 대해 정확한 답변을 생성하기 어렵습니다. RAG는 이 문제를 “검색(Retrieve) → 생성(Generate)” 2단계로 해결합니다. 먼저 사용자의 질문과 관련된 문서를 벡터 유사도 검색으로 찾아낸 뒤, 이 문서를 LLM의 컨텍스트에 주입하여 사실에 근거한 응답을 생성하도록 유도합니다. 3가지 주요 접근 방식이 있습니다:

패턴	설명	특징
기본 2단계	검색 → 생성	단순하고 빠름
에이전틱 RAG	에이전트가 검색 도구를 호출하여 반복	유연하고 정확
하이브리드	키워드 + 시맨틱 검색 결합	검색 품질 향상

- 기본 2단계: 쿼리로 문서를 검색한 후, 검색된 문서를 컨텍스트로 LLM에 전달하여 답변을 생성합니다.
- 에이전틱 RAG: 에이전트가 검색 도구를 사용하여 필요한 정보를 반복적으로 검색하고, 충분한 정보를 모은 후 답변합니다.

RAG 5 빌딩 블록

LangChain의 RAG 파이프라인은 다음 다섯 컴포넌트로 구성됩니다.

컴포넌트	역할
Document Loaders	PDF, 웹페이지, Notion 등에서 원본 문서를 적재
Text Splitters	긴 문서를 의미 단위 청크로 분할 (<code>RecursiveCharacterTextSplitter</code> 등)
Embedding Models	텍스트를 벡터로 변환 (<code>OpenAIEmbeddings</code> 등)
Vector Stores	임베딩 인덱싱·검색 (FAISS, Chroma, Pinecone 등)
Retrievers	검색 인터페이스 통일 – 벡터/하이브리드/MultiQuery 등을 같은 API로

Rewrite → Retrieve → Agent 3-노드 패턴

에이전틱 RAG의 한 변형으로, 쿼리 재작성을 별도 노드로 분리합니다. 사용자 입력을 검색 친화적인 형태로 다듬은 뒤 retriever가 동작하고, 그 결과를 에이전트가 답변에 반영합니다.

```
# Rewrite → Retrieve → Agent
graph.add_node("rewrite", query_rewriter_node)
graph.add_node("retrieve", retriever_node)
graph.add_node("agent", create_agent(model, tools=[]))

graph.add_edge(START, "rewrite")
graph.add_edge("rewrite", "retrieve")
graph.add_edge("retrieve", "agent")
graph.add_edge("agent", END)
```

아래 research → writer 예제는 콘텐츠 생성 워크플로를 보여주며, Rewrite → Retrieve → Agent 는 검색 품질을 끌어올리는 워크플로입니다. 둘 다 `StateGraph` 의 자연스러운 활용입니다.

9.6 간단한 RAG 구현

텍스트를 청크로 분할하고, 간단한 키워드 기반 검색으로 RAG를 구현합니다. 벡터 스토어 없이도 RAG의 핵심 개념을 이해할 수 있습니다.

RAG 파이프라인의 첫 번째 단계는 문서를 적절한 크기의 청크(chunk)로 분할하는 것입니다.

`RecursiveCharacterTextSplitter` 는 문단, 문장, 단어 순서로 재귀적으로 분할점을 찾아 의미 단위가 최대한 보존되도록 합니다. `chunk_overlap` 파라미터는 인접 청크 간에 겹치는 문자 수를 지정하여, 분할 경계에서 문맥이 단절되는 것을 방지합니다.

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

# 샘플 문서
documents = [
    "LangChain is a framework for building applications with large language models. It provides tools for prompt engineering, memory management, and agent creation.",
    "LangGraph is a low-level orchestration framework for building stateful agents. It uses a graph-based approach with nodes and edges.",
    "Middleware in LangChain v1 allows you to intercept and modify agent behavior at every step. You can add logging, guardrails, and custom logic.",
    "Multi-agent systems in LangChain support five patterns: subagents, handoffs, skills, router, and custom workflows.",
    "RAG (Retrieval-Augmented Generation) combines information retrieval with text generation to provide grounded, factual responses.",
]

# 텍스트 분할
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=200,
    chunk_overlap=50,
)

chunks = []
for doc in documents:
    chunks.extend(text_splitter.split_text(doc))

print(f"원본 문서: {len(documents)}개")
print(f"분할 청크: {len(chunks)}개")
for i, chunk in enumerate(chunks):
    print(f" [{i}] {chunk[:80]}...")
```

원본 문서: 5개
분할 청크: 5개

```
[0] LangChain is a framework for building applications with large language models. I...
[1] LangGraph is a low-level orchestration framework for building stateful agents. I...
[2] Middleware in LangChain v1 allows you to intercept and modify agent behavior at ...
[3] Multi-agent systems in LangChain support five patterns: subagents, handoffs, ski...
[4] RAG (Retrieval-Augmented Generation) combines information retrieval with text ge...
```

9.7 FAISS 벡터 스토어 (선택)

임베딩 기반 유사도 검색을 사용하면 키워드 매칭보다 훨씬 정확한 검색이 가능합니다. 키워드 검색은 “LLM 에이전트”라는 쿼리로 “AI 기반 자동화 도우미”라는 문서를 찾지 못하지만, 임베딩 유사도 검색은 의미적으로 유사한 문서를 정확히 찾아냅니다. LangChain은 `InMemoryVectorStore`, FAISS, Chroma 등 다양한 벡터 스토어를 지원하며, 아래는 FAISS 벡터 스토어를 사용하는 예시입니다.

TIP

프로덕션 RAG 파이프라인에서는 검색된 문서의 관련성을 평가하는 그레이딩(grading) 단계와, 생성된 답변의 환각 여부를 검증하는 할루시네이션 체크 단계를 추가하는 것이 권장됩니다. 이러한 검증 로직은 `StateGraph`의 조건부 엣지로 자연스럽게 구현할 수 있습니다.

```
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import FAISS

# 임베딩 모델 생성
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")

# 벡터 스토어 생성
vectorstore = FAISS.from_texts(chunks, embeddings)

# 검색
results = vectorstore.similarity_search("LangChain agent patterns", k=3)
for doc in results:
    print(doc.page_content)
```

Multi-agent systems in LangChain support five patterns: subagents, handoffs, skills, router, and custom workflows. LangChain is a framework for building applications with large language models. It provides tools for prompt engineering, memory management, and agent creation. Middleware in LangChain v1 allows you to intercept and modify agent behavior at every step. You can add logging, guardrails, and custom logic.

요약

이 노트북에서 배운 내용:

주제	핵심 내용
StateGraph 기초	노드, 엣지, 상태로 워크플로를 구성합니다
조건부 엣지	<code>add_conditional_edges</code> 로 런타임에 분기 처리합니다
에이전트 통합	<code>create_agent</code> 를 StateGraph 노드로 사용하여 파이프라인을 구성합니다
RAG 패턴	검색 + 생성을 결합하여 사실 기반 응답을 제공합니다
벡터 스토어	FAISS 등으로 임베딩 기반 유사도 검색이 가능합니다

이 장에서 학습한 `StateGraph` 와 RAG 패턴은 에이전트 개발의 핵심 빌딩 블록입니다. 다음 장에서는 이렇게 구축한 에이전트를 프로덕션 환경으로 배포하기 위한 테스트, 배포, 모니터링 전략을 다룹니다.

10 프로덕션

에이전트를 개발하는 것과 프로덕션에 배포하는 것은 완전히 다른 문제입니다. 비결정적인 LLM의 동작을 테스트하고, 웹 서비스로 배포하고, 실행을 모니터링하고, 비용을 관리해야 합니다. 이 장에서는 LangSmith Studio를 사용한 로컬 개발부터, 결정론적 테스트, Agent Chat UI, LangGraph Platform 배포까지 에이전트의 전체 프로덕션 라이프사이클을 다룹니다.

전통적인 소프트웨어는 같은 입력에 항상 같은 출력을 반환하므로 단위 테스트로 충분히 검증할 수 있습니다. 그러나 에이전트는 LLM의 비결정적 특성과 도구 호출의 부수 효과가 결합되어, “어떤 도구를 어떤 순서로 호출했는가”를 검증하는 트라젝토리 테스트와, 실시간으로 에이전트 동작을 추적하는 관측성(Observability) 인프라가 필수적입니다.

학습 목표

에이전트를 테스트, 배포, 모니터링하는 방법을 알아봅니다.

이 노트북에서 다루는 내용:

- LangSmith Studio를 사용한 로컬 개발 및 디버깅
- `GenericFakeChatModel` 로 결정론적 에이전트 테스트
- 트라젝토리 기반 테스트로 도구 호출 순서 검증
- Agent Chat UI로 웹 기반 대화
- LangGraph Platform 및 자체 서버 배포
- LangSmith를 활용한 관측성(Observability)

10.1 환경 설정

```
from dotenv import load_dotenv
import os
load_dotenv(override=True)

from langchain_openai import ChatOpenAI

model = ChatOpenAI(
    model="gpt-5.4",
)

from langchain.agents import create_agent
from langchain.tools import tool

print("환경 준비 완료.")
```

환경 준비 완료.

10.2 LangSmith Studio

로컬에서 에이전트를 개발하고 디버깅합니다.

LangSmith Studio는 에이전트의 실행 과정을 트레이스(trace) 단위로 시각화하는 개발 도구입니다. 각 트레이스는 모델 호출, 도구 실행, 노드 전환 등의 단계를 트리 구조로 보여주며, 각 단계의 입력/출력 데이터, 소요 시간, 토큰 사용량을 한눈에 확인할 수 있습니다. `langgraph dev` 명령어로 로컬 개발 서버를 실행하면 별도의 배포 없이도 Studio UI에서 에이전트를 인터랙티브하게 테스트할 수 있습니다.

Studio를 사용하려면 다음이 필요합니다:

- `langgraph.json` 설정 파일
- `langgraph dev` 명령어로 로컬 서버 실행 (기본 포트 `http://127.0.0.1:2024`)
- 브라우저에서 hosted Studio UI 접속 – <https://smith.langchain.com/studio/?baseUrl=http://127.0.0.1:2024>

Studio Key Features

Studio가 제공하는 4가지 핵심 기능:

기능	설명
Hot-reloading	프롬프트와 도구 코드를 수정하면 서버 재시작 없이 즉시 반영됩니다
Full execution trace inspection	모든 노드, 모델 호출, 도구 실행의 입출력을 트리 형태로 검사합니다
Thread replay	저장된 스레드를 시점별로 다시 실행하며 분기를 생성하고 동작을 비교합니다
Exception capture	예외가 발생한 순간의 상태와 주변 컨텍스트를 함께 캡처해 디버깅합니다

```
# langgraph.json 설정 예시
import json

langgraph_config = {
    "dependencies": [".",],
    "graphs": {
        "agent": "./agent.py:agent"
    },
    "env": ".env"
}

print("langgraph.json 설정 예시:")
print(json.dumps(langgraph_config, indent=2))
print("\n실행 방법:")
print(" $ langgraph dev # 로컬 dev server: http://127.0.0.1:2024")
print(" → Studio UI: https://smith.langchain.com/studio/?baseUrl=http://127.0.0.1:2024")
```

```
langgraph.json 설정 예시:
{
  "dependencies": [
    "."
  ],
  "graphs": {
    "agent": "./agent.py:agent"
  },
  "env": ".env"
}
```

실행 방법:

```
$ langgraph dev # 로컬 dev server: http://127.0.0.1:2024
→ Studio UI: https://smith.langchain.com/studio/?baseUrl=http://127.0.0.1:2024
```

Studio에서 인터랙티브하게 동작을 확인했다면, 다음 단계는 자동화된 테스트를 작성하여 에이전트의 동작을 반복 가능한 방식으로 검증하는 것입니다.

10.3 에이전트 테스트

에이전트 테스트는 Unit, Integration, Evals 세 계층으로 나누어 접근합니다.

분류	목적	대표 도구
Unit	에이전트 로직을 결정론적으로 검증 (도구 선택, 분기, 종료 조건)	GenericFakeChatModel , pytest
Integration	실제 LLM과 도구를 연결해 워크플로 전체가 동작하는지 확인	langgraph dev , 실서버 환경
Evals	에이전트 트래젝토리를 deterministic matching 또는 LLM-as-judge 평가자로 점수화	agentevals , LangSmith Evaluations

10.3.1 Unit — GenericFakeChatModel

GenericFakeChatModel 을 사용하면 실제 API 호출 없이 결정론적으로 에이전트를 테스트할 수 있습니다. 이 가짜 모델은 미리 정의된 응답 시퀀스를 순서대로 반환하므로, LLM의 비결정적 동작을 제거하고 에이전트의 로직(도구 선택, 분기, 종료 조건 등)만을 독립적으로 검증할 수 있습니다.

이 방법의 장점:

- API 비용 없이 테스트 가능
- 항상 동일한 결과를 반환하므로 CI/CD 파이프라인에 적합
- 에이전트의 로직(도구 호출, 분기 등)을 독립적으로 검증

➔ TIP

GenericFakeChatModel 에 tool call이 포함된 AIMessage 를 응답으로 설정하면, 에이전트가 특정 도구를 호출하는 시나리오도 결정론적으로 테스트할 수 있습니다. 이를 통해 “모델이 올바른 도구를 선택했는가”가 아닌 “도구 호출 후 에이전트가 올바르게 동작하는가”를 검증합니다.

```
from langchain_core.language_models import GenericFakeChatModel
from langchain.messages import AIMessage
from langchain.agents import create_agent
from langchain.tools import tool

@tool
def get_capital(country: str) -> str:
    """국가의 수도를 반환합니다."""
    capitals = {"Korea": "Seoul", "Japan": "Tokyo", "France": "Paris"}
    return capitals.get(country, "알 수 없음")
```

```
# 가짜 모델로 결정론적 테스트
fake_model = GenericFakeChatModel(
    messages=iter([
        AIMessage(content="대한민국의 수도는 서울입니다.")
    ])
)

# 테스트 에이전트
test_agent = create_agent(
    model=fake_model,
    tools=[get_capital],
    system_prompt="당신은 지리 전문가입니다.",
)

print("GenericFakeChatModel 테스트:")
print("  → 결정론적 응답으로 에이전트 동작을 테스트합니다")
print("  → CI/CD 파이프라인에서 API 호출 없이 테스트 가능")
```

```
GenericFakeChatModel 테스트:
  → 결정론적 응답으로 에이전트 동작을 테스트합니다
  → CI/CD 파이프라인에서 API 호출 없이 테스트 가능
```

10.3.2 Evals — 트라젝토리 평가와 `agentevals`

에이전트의 도구 호출 순서를 검증합니다. 트라젝토리 평가는 에이전트의 최종 출력만 검증하는 것이 아니라, 중간 과정 — 어떤 도구를 어떤 순서로 호출했는지, 각 도구에 어떤 인자를 전달했는지 — 을 검사합니다. 이 접근법이 중요한 이유는 에이전트가 올바른 최종 답변을 내놓더라도 불필요한 도구 호출이나 위험한 작업을 수행했을 수 있기 때문입니다.

`agentevals` 패키지는 트라젝토리 매칭을 위한 4가지 모드를 제공합니다.

```
pip install agentevals
```

모드	매칭 규칙
<code>strict</code>	참조 트라젝토리와 도구 호출 종류·순서가 완전히 일치해야 통과
<code>unordered</code>	같은 도구 집합을 호출하면 순서는 무시
<code>subset</code>	실제 트라젝토리가 참조의 부분집합(추가 호출 금지)
<code>superset</code>	실제 트라젝토리가 참조의 초집합(필수 호출만 포함되면 통과)

LLM-as-judge 평가자를 사용하면 트라젝토리를 자연어 기준으로 평가할 수 있습니다.

```
# 트라젝토리 테스트 예시
def test_agent_trajectory():
    """에이전트가 예상된 순서로 도구를 호출하는지 테스트합니다."""
    result = test_agent.invoke(
        {"messages": [{"role": "user", "content": "대한민국의 수도는 어디인가요?"}]}
    )

    messages = result["messages"]

    # 검증: 메시지가 존재하는지
    assert len(messages) > 0, "에이전트가 응답하지 않았습니다"

    # 검증: 마지막 메시지가 AI 응답인지
```

```

last_msg = messages[-1]
assert hasattr(last_msg, 'content'), "마지막 메시지에 content가 없습니다"

print("✓ 트라젝토리 테스트 통과")
print(f"  메시지 수: {len(messages)}")
print(f"  최종 응답: {last_msg.content[:100]}")

try:
    test_agent_trajectory()
except Exception as e:
    print(f"테스트 참고: {e}")

```

테스트 참고:

자동화된 테스트로 에이전트의 로직을 검증했다면, 실제 사용자의 관점에서 에이전트를 테스트해 볼 차례입니다. Agent Chat UI는 코드를 작성하지 않고도 브라우저에서 에이전트와 대화할 수 있는 인터페이스를 제공합니다.

10.5 Agent Chat UI

에이전트와 대화할 수 있는 웹 UI입니다. LangGraph 서버와 연결하여 브라우저에서 직접 에이전트를 테스트할 수 있습니다. `create-agent-chat-app` 스캐폴드로 새 프로젝트를 생성하거나, 공식 레포지토리 (`langchain-ai/agent-chat-ui`)를 직접 clone 해 사용합니다. 로컬에서 실행 중인 LangGraph 서버 (`http://127.0.0.1:2024`)에 연결합니다.

주요 기능:

- 실시간 스트리밍 채팅
- 도구 호출 시각화
- 대화 분기(branching)
- Human-in-the-loop 승인

설치 방법 두 가지 중 하나를 선택합니다.

```

# 옵션 A - 스캐폴드 (권장)
npx create-agent-chat-app --project-name my-chat-ui

# 옵션 B - 공식 레포지토리 clone
git clone https://github.com/langchain-ai/agent-chat-ui.git
cd agent-chat-ui && npm install && npm dev

```

```

# LangGraph 서버 시작 (별도 셸)
langgraph dev # http://127.0.0.1:2024

```

UI 첫 화면에서 다음 세 값을 입력합니다.

항목	값
Deployment URL	<code>http://127.0.0.1:2024</code> (로컬) 또는 배포된 LangGraph 서버 URL
Graph ID	<code>langgraph.json</code> 의 <code>graphs</code> 키 (예: <code>agent</code>)
LangSmith API key	관리형 배포 인증용. 로컬 dev server에서는 생략 가능

테스트를 마쳤다면 에이전트를 프로덕션 환경에 배포할 차례입니다. LangGraph는 관리형 플랫폼부터 셀프 호스팅까지 다양한 배포 옵션을 제공합니다.

10.6 배포

LangGraph Platform(관리형) 또는 자체 서버로 에이전트를 배포할 수 있습니다. LangGraph Platform은 인프라 관리 없이 에이전트를 배포하고 스케일링할 수 있는 관리형 서비스이며, 셀프 호스팅은 `langgraph-api` Docker 이미지를 사용하여 자체 인프라에 배포하는 방식입니다. 프로덕션 환경에 맞는 옵션을 선택하세요.

! WARNING

FastAPI로 에이전트를 직접 래핑하는 경우, 스트리밍 응답(`StreamingResponse`), 체크포인팅, 스레드 관리 등을 직접 구현해야 합니다. LangGraph Platform이나 `langgraph-api` Docker 이미지를 사용하면 이러한 기능이 기본으로 제공됩니다.

10.6.1 LangGraph Platform 배포 워크플로

LangGraph Platform 배포는 GitHub 연동을 통해 진행됩니다. 로컬에서 `langgraph deploy` 한 줄로 배포하는 옛 흐름은 더 이상 권장되지 않습니다.

1. 에이전트 코드를 GitHub 레포지토리에 푸시 (`langgraph.json` 포함)
2. LangSmith 콘솔 → Deployments 메뉴 → + New Deployment
3. 레포지토리, 브랜치, `langgraph.json` 경로 지정
4. 환경 변수(`OPENAI_API_KEY` 등) 입력
5. Deploy 클릭 → 빌드 로그 확인 → 발급된 Deployment URL 확보

! WARNING

옛 가이드의 `langgraph deploy` CLI 흐름은 deprecated입니다. 신규 배포는 LangSmith 콘솔의 GitHub 연동을 통해 진행하세요.

```
print("배포 옵션:")
print("=" * 50)

print("""
# 옵션 1: LangGraph Platform (관리형) - GitHub 연동
#   → LangSmith Deployments → + New Deployment 흐름 사용
#   (옛 `langgraph deploy` CLI 패턴은 deprecated)

# 옵션 2: 자체 Docker 배포
$ langgraph build -t my-agent
$ docker run -p 2024:2024 my-agent

# 옵션 3: FastAPI/Flask 래핑
from fastapi import FastAPI

app = FastAPI()

@app.post("/chat")
```

```

async def chat(message: str):
    result = agent.invoke(
        {
            "messages": [
                {
                    "role": "user",
                    "content": message
                }
            ]
        }
    )
    return {
        "response": result["messages"][-1].content
    }
"""

```

10.6.2 배포된 에이전트 호출 — langgraph-sdk

배포된 에이전트는 Python SDK 또는 REST로 호출합니다.

```
pip install langgraph-sdk
```

```

from langgraph_sdk import get_sync_client

client = get_sync_client(url="http://127.0.0.1:2024")

for chunk in client.runs.stream(
    None, # threadless run
    "agent", # graph ID
    input={"messages": [{"role": "user", "content": "안녕"}]},
    stream_mode="updates",
):
    print(chunk)

```

REST API 직접 호출 예:

```

curl -X POST http://127.0.0.1:2024/runs/stream \
-H "Content-Type: application/json" \
--data '{
  "assistant_id": "agent",
  "input": {"messages": [{"role": "user", "content": "안녕"}]},
  "stream_mode": "updates"
}'

```

배포 옵션:

```
=====
# 옵션 1: LangGraph Platform (관리형)
$ langgraph deploy
```

```

# 옵션 2: 자체 Docker 배포
$ langgraph build -t my-agent
$ docker run -p 2024:2024 my-agent
```

```

# 옵션 3: FastAPI/Flask 래핑
from fastapi import FastAPI
```

```
app = FastAPI()
```

```

@app.post("/chat")
async def chat(message: str):

```



```

result = agent.invoke(
    {
        "messages": [
            {
                "role": "user",
                "content": message
            }
        ]
    }
)
... (truncated)

```

10.7 관측성

에이전트가 프로덕션에 배포되면 실시간 모니터링이 핵심이 됩니다. LangSmith로 에이전트 동작을 추적합니다.

10.7.1 환경 변수

```

# 권장 (현행)
export LANGSMITH_TRACING=true
export LANGSMITH_API_KEY="lsv2_..."
export LANGSMITH_PROJECT="my-agent" # 미지정 시 기본값 `default`

```

LangSmith에서 확인할 수 있는 정보:

- 각 에이전트 호출의 전체 실행 흐름
- 모델 입/출력, 도구 호출, 토큰 사용량
- 지연 시간, 에러, 비용 추적

TIP

`LANGSMITH_PROJECT` 를 환경별로 다르게 지정(개발/스테이징/프로덕션)하면 트레이스를 환경별로 분리해 분석할 수 있습니다. 변수를 지정하지 않으면 `default` 프로젝트에 기록됩니다.

10.7.2 `tracing_context` 로 부분 트레이싱 제어

환경 변수와 별개로, 코드 안에서 특정 블록만 트레이싱을 켜고 끄려면 `tracing_context` 를 사용합니다.

```

import langsmith as ls

with ls.tracing_context(enabled=True):
    agent.invoke({"messages": [{"role": "user", "content": "안녕"}]})

```

10.7.3 `config`로 태그·메타데이터 부여

트레이스 검색·필터링이 쉬워지도록 호출 시점에 태그와 메타데이터를 넣습니다.

```

agent.invoke(
    {"messages": [{"role": "user", "content": "안녕"}]},
    config={
        "tags": ["production", "v1.0"],
    }
)

```

```
        "metadata": {"user_id": "123", "session_id": "456"},
    },
)
```

10.8 프로덕션 체크리스트

에이전트를 프로덕션에 배포하기 전에 아래 항목을 확인하세요.

항목	도구	상태
단위 테스트	<code>GenericFakeChatModel</code> , <code>pytest</code>	
트라젝토리 테스트	커스텀 검증 함수	
관측성 설정	LangSmith 트레이싱	
에러 처리	<code>try/except</code> , 재시도 로직	
보안	API 키 관리, 입력 검증, 가드레일	
배포 환경	Docker, LangGraph Platform	
모니터링	LangSmith 대시보드, 알림 설정	
문서화	API 문서, 에이전트 동작 설명	

요약

이 노트북에서 배운 내용:

주제	핵심 내용
LangSmith Studio	<code>langgraph dev</code> 로 로컬에서 에이전트를 시각적으로 디버깅합니다
에이전트 테스트	<code>GenericFakeChatModel</code> 로 API 호출 없이 결정론적 테스트를 수행합니다
트라젝토리 테스트	도구 호출 순서와 최종 응답을 검증합니다
Agent Chat UI	웹 브라우저에서 에이전트와 대화하고 도구 호출을 시각화합니다
배포	LangGraph Platform, Docker, FastAPI 등으로 배포합니다
관측성	LangSmith로 실행 흐름, 토큰 사용량, 비용을 추적합니다

에이전트의 테스트, 배포, 모니터링 인프라를 갖추었다면, 다음 장에서는 에이전트의 도구 생태계를 확장하는 핵심 표준인 MCP(Model Context Protocol)를 학습합니다. MCP를 활용하면 직접 도구를 구현하지 않고도 외부 서비스의 기능을 에이전트에 통합할 수 있습니다.

11 MCP (Model Context Protocol)

에이전트가 사용할 수 있는 도구를 일일이 코드로 작성하는 대신, 표준화된 프로토콜을 통해 외부 도구 서버에 연결할 수 있다면 어떨까요? MCP(Model Context Protocol)는 바로 이 목적을 위해 만들어진 개방형 표준입니다. 이 장에서는 MCP의 아키텍처를 이해하고, `langchain-mcp-adapters`를 사용하여 LangChain 에이전트에 MCP 서버의 도구를 통합하는 방법을 학습합니다.

MCP의 핵심 가치는 도구 제공자와 도구 소비자의 분리입니다. 데이터베이스 관리자가 MCP 서버를 한 번 구현하면, Claude, ChatGPT, LangChain 에이전트 등 MCP를 지원하는 모든 호스트 애플리케이션이 별도의 통합 코드 없이 해당 도구를 사용할 수 있습니다. 이는 USB가 다양한 장치를 표준화된 인터페이스로 연결하는 것과 유사합니다.

학습 목표

MCP를 통해 외부 도구와 컨텍스트를 에이전트에 연결하는 방법을 알아봅니다.

이 노트북에서 다루는 내용:

- MCP의 개념과 아키텍처(서버/클라이언트/호스트)를 이해한다
- `langchain-mcp-adapters` 패키지로 MCP 서버에 연결한다
- `create_agent(model=..., tools=await client.get_tools())` 패턴으로 에이전트와 MCP 도구를 통합한다
- Stdio와 SSE 전송 방식의 차이를 안다
- 다중 MCP 서버를 연결하는 방법을 익힌다

11.1 환경 설정

```
from dotenv import load_dotenv
import os
load_dotenv(override=True)

from langchain_openai import ChatOpenAI

model = ChatOpenAI(
    model="gpt-5.4",
)

print("환경 준비 완료.")
```

환경 준비 완료.

11.2 MCP 개념

MCP(Model Context Protocol)는 외부 도구와 컨텍스트를 표준화된 방식으로 LLM에 제공하기 위한 오픈 프로토콜입니다.

아키텍처 구성 요소

구성 요소	역할	예시
MCP 서버	도구, 리소스, 프롬프트를 노출	파일 시스템 서버, DB 서버, API 래퍼
MCP 클라이언트	서버에 연결하여 도구를 가져옴	<code>MultiServerMCPClient</code>
호스트	클라이언트를 관리하고 LLM과 연결	LangChain 에이전트, IDE

핵심 리소스 타입

- Tools: 에이전트가 호출할 수 있는 실행 가능한 함수
- Resources: 파일, DB 레코드 등의 데이터 (LangChain Blob 객체로 변환)
- Prompts: 재사용 가능한 프롬프트 템플릿

왜 MCP인가?

MCP 이전에는 각 도구마다 개별적으로 연결 코드를 작성해야 했습니다. MCP는 이를 하나의 표준 프로토콜로 통합하여:

- 도구 제공자는 한 번만 MCP 서버를 구현하면 됩니다
- LLM 호스트는 MCP 클라이언트 하나로 모든 도구에 접근할 수 있습니다
- 생태계 전체에서 도구를 재사용할 수 있습니다

MCP의 3계층 아키텍처를 좀 더 자세히 살펴보면: 호스트(Host)는 사용자의 LangChain 에이전트 애플리케이션입니다. 호스트 안에서 클라이언트(Client)가 MCP 프로토콜을 사용하여 외부 서버(Server)와 통신합니다. 클라이언트는 서버에 “어떤 도구를 제공하는가?”라고 물어 도구 목록을 동적으로 발견(discover)하고, LLM이 도구 호출을 결정하면 해당 서버에 실행 요청을 보냅니다.

이제 LangChain에서 이 프로토콜을 어떻게 사용하는지 살펴보겠습니다.

11.3 langchain-mcp-adapters 설치

LangChain에서 MCP를 사용하려면 `langchain-mcp-adapters` 패키지가 필요합니다.

```
# MCP 어댑터 설치 명령어
print("MCP 어댑터 설치:")
print("  pip install langchain-mcp-adapters")
print()
print("주요 컴포넌트:")
print("  - MultiServerMCPClient: 여러 MCP 서버를 관리하는 클라이언트")
print("  - client.get_tools(): MCP 서버에서 도구를 LangChain Tool로 변환")
print("  - client.get_resources(): MCP 서버에서 리소스를 LangChain Blob으로 변환")
```

```
MCP 어댑터 설치:
pip install langchain-mcp-adapters
```

주요 컴포넌트:

- MultiServerMCPClient: 여러 MCP 서버를 관리하는 클라이언트
- client.get_tools(): MCP 서버에서 도구를 LangChain Tool로 변환
- client.get_resources(): MCP 서버에서 리소스를 LangChain Blob으로 변환

MCP 어댑터를 설치했으면, 실제로 MCP 서버와 통신하는 두 가지 전송 방식을 알아보겠습니다. 전송 방식의 선택은 MCP 서버의 위치(로컬 vs 원격)에 따라 달라집니다.

11.4 Stdio 전송

Stdio(Standard I/O) 전송은 로컬 서브프로세스를 통해 MCP 서버와 통신합니다. 클라이언트가 MCP 서버를 자식 프로세스로 직접 실행하고, `stdin / stdout` 을 통해 JSON-RPC 메시지를 교환합니다. 별도의 네트워크 설정이 불필요하고 프로세스 생명주기가 클라이언트에 의해 자동 관리되므로, 개발 및 테스트 환경에 적합합니다.

```
# Stdio 전송 설정 예시 (실제 실행하지 않음)
stdio_config = {
    "math_server": {
        "transport": "stdio",
        "command": "python",
        "args": ["math_server.py"],
    }
}

print("Stdio 전송 설정:")
import json
print(json.dumps(stdio_config, indent=2))
print()
print("사용 패턴:")
print("  async with MultiServerMCPClient(stdio_config) as client:")
print("      tools = client.get_tools()")
print()
print("특징:")
print("  - 로컬 서브프로세스로 통신 (stdin/stdout)")
print("  - 개발/테스트 환경에 적합")
print("  - 별도 서버 실행 불필요 - 자동으로 프로세스 시작")
```

```
Stdio 전송 설정:
{
  "math_server": {
    "transport": "stdio",
    "command": "python",
    "args": [
      "math_server.py"
    ]
  }
}

사용 패턴:
  async with MultiServerMCPClient(stdio_config) as client:
      tools = client.get_tools()

특징:
  - 로컬 서브프로세스로 통신 (stdin/stdout)
  - 개발/테스트 환경에 적합
  - 별도 서버 실행 불필요 - 자동으로 프로세스 시작
```

Stdio가 로컬 개발에 적합하다면, 프로덕션 환경에서는 원격 서버와 통신할 수 있는 HTTP 기반 전송이 필요합니다.

11.5 SSE/HTTP 전송

HTTP(streamable-http) 전송은 웹 기반 통신으로, 원격 MCP 서버에 연결할 때 사용합니다. 인증 헤더와 커스텀 설정을 지원합니다. `streamable_http` 는 최신 MCP 사양에서 권장하는 전송 방식이며, 기존의 SSE(Server-Sent Events) 전송은 레거시로 분류되었으나 여전히 지원됩니다.

TIP

`streamable_http` 와 `sse` 의 차이: `streamable_http` 는 단일 HTTP 엔드포인트에서 요청/응답과 서버 푸시를 모두 처리하며, `sse` 는 이벤트 스트리밍과 요청을 별도 채널로 분리합니다. 새로운 프로젝트에서는 `streamable_http` 를 사용하세요.

```
# HTTP/streamable-http 전송 설정 예시 (실제 실행하지 않음)
http_config = {
    "weather_server": {
        "transport": "streamable_http",
        "url": "https://weather-mcp.example.com/mcp",
        "headers": {
            "Authorization": "Bearer YOUR_API_KEY"
        }
    }
}

print("HTTP 전송 설정:")
import json
print(json.dumps(http_config, indent=2))
print()
print("사용 패턴:")
print("  async with MultiServerMCPClient(http_config) as client:")
print("    tools = client.get_tools()")
print()
print("전송 방식 비교:")
print(" | 전송 방식 | 사용 사례 | 인증 지원 |")
print(" |-----|-----|-----|")
print(" | stdio | 로컬 개발/테스트 | N/A |")
print(" | streamable_http | 원격 서버 연결 | 헤더 지원 |")
print(" | sse | 레거시 SSE 서버 | 헤더 지원 |")
```

```
HTTP 전송 설정:
{
  "weather_server": {
    "transport": "streamable_http",
    "url": "https://weather-mcp.example.com/mcp",
    "headers": {
      "Authorization": "Bearer YOUR_API_KEY"
    }
  }
}

사용 패턴:
  async with MultiServerMCPClient(http_config) as client:
    tools = client.get_tools()
```

```
전송 방식 비교:
 | 전송 방식 | 사용 사례 | 인증 지원 |
 |-----|-----|-----|
 | stdio | 로컬 개발/테스트 | N/A |
 | streamable_http | 원격 서버 연결 | 헤더 지원 |
 | sse | 레거시 SSE 서버 | 헤더 지원 |
```

전송 방식을 이해했으니, 이제 MCP 서버에서 가져온 도구를 실제로 에이전트에 연결하는 과정을 살펴보겠습니다.

11.6 MCP 도구 로드 및 에이전트 통합

MCP 서버에서 가져온 도구를 LangChain 에이전트에 바인딩하는 패턴입니다. 핵심은 `client.get_tools()` 메서드입니다. 이 메서드는 MCP 서버가 노출하는 도구 목록을 자동으로 발견하고, 각 도구의 이름, 설명, 파라미터 스키마를 LangChain의 `Tool` 객체로 변환합니다. 변환된 도구는 `create_agent(tools=mcp_tools)` 에 그대로 전달할 수 있습니다.

주요 함수 시그니처

함수	설명
<code>client.get_tools()</code>	모든 등록 서버의 도구를 LangChain <code>Tool</code> 리스트로 반환
<code>client.get_resources(server_name)</code>	특정 서버의 리소스를 LangChain <code>Blob</code> 리스트로 반환
<code>client.get_prompt(server_name, prompt_name, arguments={...})</code>	서버에 등록된 프롬프트 템플릿을 가져옴
<code>load_mcp_tools(session)</code>	열린 세션에서 도구 목록을 LangChain <code>Tool</code> 로 변환 (stateful 패턴)
<code>load_mcp_resources(session, uris=[...])</code>	열린 세션에서 지정한 URI의 리소스를 로드

```
# MCP 도구를 에이전트에 통합하는 패턴 (개념 코드)
print("MCP 도구 → 에이전트 통합 패턴:")
print("=" * 50)
print("""
from langchain_mcp_adapters.client import MultiServerMCPClient
from langchain.agents import create_agent

mcp_config = {
    "math": {
        "transport": "stdio",
        "command": "python",
        "args": ["math_server.py"],
    }
}

async with MultiServerMCPClient(mcp_config) as client:
    # 1. MCP 서버에서 도구 가져오기
    mcp_tools = await client.get_tools()

    # 2. 에이전트에 도구 전달
    agent = create_agent(
        model="gpt-5.4",
        tools=mcp_tools,
    )

    # 3. 에이전트 실행
    result = agent.invoke(
        {"messages": [{"role": "user", "content": "2 + 3은?"}]}
    )
""")
print("핵심: client.get_tools()가 MCP 도구를 LangChain Tool로 자동 변환합니다.")
```

MCP 도구 → 에이전트 통합 패턴:

=====

```

from langchain_mcp_adapters.client import MultiServerMCPClient
from langchain.agents import create_agent

mcp_config = {
    "math": {
        "transport": "stdio",
        "command": "python",
        "args": ["math_server.py"],
    }
}

async with MultiServerMCPClient(mcp_config) as client:
    # 1. MCP 서버에서 도구 가져오기
    mcp_tools = await client.get_tools()

    # 2. 에이전트에 도구 전달
    agent = create_agent(
        model="gpt-5.4",
        tools=mcp_tools,
    )

    # 3. 에이전트 실행
    result = agent.invoke(
        {"messages": [{"role": "user", "content": "2 + 3은?"}]}
    )

핵심: client.get_tools()가 MCP 도구를 LangChain Tool로 자동 변환합니다.

```

단일 서버 연결을 마스터했다면, 실전에서는 여러 MCP 서버를 동시에 연결하는 경우가 더 많습니다. 수학 연산 서버, 날씨 API 서버, 데이터베이스 서버 등을 하나의 에이전트에서 모두 사용하는 시나리오를 살펴보겠습니다.

11.7 다중 MCP 서버 연결

`MultiServerMCPClient` 는 이름 그대로 여러 MCP 서버를 동시에 관리할 수 있습니다. 설정 딕셔너리에 여러 서버를 나열하면, 클라이언트가 모든 서버에 동시에 연결하고 각 서버의 도구를 하나의 통합 리스트로 반환합니다.

! WARNING

여러 MCP 서버의 도구가 같은 이름을 가질 수 있습니다. 예를 들어 두 서버 모두 `search` 라는 도구를 제공하면 충돌이 발생합니다. `MultiServerMCPClient` 는 기본적으로 서버 이름을 접두사로 붙여 이를 방지하지만, 명시적으로 도구 이름을 확인하는 것이 안전합니다.

```

# 다중 MCP 서버 설정 예시
multi_server_config = {
    "math_server": {
        "transport": "stdio",
        "command": "python",
        "args": ["math_server.py"],
    },
    "weather_server": {
        "transport": "streamable_http",
        "url": "https://weather-mcp.example.com/mcp",
    },
    "database_server": {
        "transport": "stdio",
        "command": "npx",
        "args": ["-y", "@modelcontextprotocol/server-postgres"],
        "env": {"DATABASE_URL": "postgresql://..."},
    },
}

print("다중 MCP 서버 설정:")
import json

```



```
print(json.dumps(multi_server_config, indent=2, ensure_ascii=False))
print()
print("사용 패턴:")
print("    async with MultiServerMCPClient(multi_server_config) as client:")
print("        tools = client.get_tools() # 모든 서버의 도구를 한 번에 로드")
print("        # tools 리스트에 math, weather, database 도구가 모두 포함")
print()
print("참고: 기본적으로 stateless - 각 도구 호출마다 새 세션 생성 후 정리")
```

다중 MCP 서버 설정:

```
{
  "math_server": {
    "transport": "stdio",
    "command": "python",
    "args": [
      "math_server.py"
    ]
  },
  "weather_server": {
    "transport": "streamable_http",
    "url": "https://weather-mcp.example.com/mcp"
  },
  "database_server": {
    "transport": "stdio",
    "command": "npx",
    "args": [
      "-y",
      "@modelcontextprotocol/server-postgres"
    ],
    "env": {
      "DATABASE_URL": "postgresql://..."
    }
  }
}
```

사용 패턴:

```
async with MultiServerMCPClient(multi_server_config) as client:
    tools = client.get_tools() # 모든 서버의 도구를 한 번에 로드
    # tools 리스트에 math, weather, database 도구가 모두 포함
... (truncated)
```

Stateful sessions

기본 `client.get_tools()` 는 각 도구 호출마다 새 세션을 열고 닫는 stateless 모드입니다. 같은 세션 안에서 여러 호출을 묶고 싶다면 `client.session(server_name)` 으로 명시적 세션을 엽니다.

```
async with client.session("math_server") as session:
    tools = await load_mcp_tools(session)
    # 같은 세션을 공유한 채 여러 도구 호출
    result1 = await tools[0].ainvoke({"a": 1, "b": 2})
    result2 = await tools[1].ainvoke({"a": 3, "b": 4})
```

11.8 MCP 인증

원격 MCP 서버에 접속할 때는 두 가지 인증 트랙을 사용합니다.

트랙 A — 정적 헤더

가장 단순한 방식으로, 헤더에 토큰을 직접 넣습니다. CI/CD 등 사전 발급된 키를 사용할 때 적합합니다.

```
http_config = {
  "weather_server": {
```

```

        "transport": "streamable_http",
        "url": "https://weather-mcp.example.com/mcp",
        "headers": {"Authorization": "Bearer YOUR_API_KEY"},
    }
}

```

트랙 B — httpx.Auth 인터페이스

토큰 갱신·서명 등 동적 인증이 필요하다면 `httpx.Auth` 구현체를 전달합니다.

```

import httpx

class BearerAuth(httpx.Auth):
    def __init__(self, token: str):
        self.token = token

    def auth_flow(self, request):
        request.headers["Authorization"] = f"Bearer {self.token}"
        yield request

http_config = {
    "weather_server": {
        "transport": "streamable_http",
        "url": "https://weather-mcp.example.com/mcp",
        "auth": BearerAuth(token="..."),
    }
}

```

트랙 C — MCP SDK OAuth2

MCP Python SDK가 제공하는 `mcp.client.auth.oauth2` 모듈을 사용하면 표준 OAuth2 흐름(authorization code, refresh token)을 그대로 따를 수 있습니다.

```

from mcp.client.auth.oauth2 import OAuth2ClientCredentials

auth = OAuth2ClientCredentials(
    token_url="https://auth.example.com/token",
    client_id="my-client",
    client_secret="...",
    scope="mcp.tools",
)

http_config = {
    "secure_server": {
        "transport": "streamable_http",
        "url": "https://secure-mcp.example.com/mcp",
        "auth": auth,
    }
}

```

11.9 Elicitation — 서버가 사용자 입력을 요청

Elicitation은 MCP 서버가 도구 실행 중에 추가 입력을 클라이언트(호스트)에 요청하는 메커니즘입니다. 호스트가 사용자에게 보여 줄 폼을 띄우고, 응답을 `ElicitResult` 로 돌려줍니다.

```

from mcp import ElicitResult, Callbacks

async def on_elicitation(request):

```

```
# 호스트 측에서 사용자에게 폼을 보여주고 응답 수집
user_input = await show_form(request.schema)
return ElicitResult(
    action="accept", # "accept" | "decline" | "cancel"
    content={"city": user_input["city"], "unit": "celsius"},
)

callbacks = Callbacks(on_elicitation=on_elicitation)

async with MultiServerMCPClient(config, callbacks=callbacks) as client:
    tools = await client.get_tools()
```

TIP

action="accept" 는 사용자가 값을 제공한 경우, "decline" 은 사용자가 거절한 경우, "cancel" 은 도구 실행 자체를 중단하라는 신호로 사용합니다.

11.10 도구 인터셉터

Tool Interceptor는 MCP 도구 호출을 가로채는 미들웨어입니다. 런타임 컨텍스트에 접근하거나, 요청/응답을 수정하거나, 재시도 로직을 구현할 수 있습니다.

인터셉터 사용 사례

사용 사례	설명
인증 주입	런타임에 사용자별 토큰 전달
요청 수정	도구 호출 파라미터 변환
응답 필터링	민감한 정보 제거
재시도 로직	실패 시 자동 재시도
로깅	도구 호출 추적

```
# 도구 인터셉터 예시 (개념 코드)
print("도구 인터셉터 패턴:")
print("=" * 50)
print("""
from langchain_mcp_adapters.client import MultiServerMCPClient

# 인터셉터 함수 정의
async def auth_interceptor(request, context):
    """런타임 컨텍스트에서 사용자 토큰을 주입합니다."""
    user_token = context.get("user_token", "")
    request.params["auth_token"] = user_token
    return request

async def logging_interceptor(request, context):
    """도구 호출을 로깅합니다."""
    print(f"Tool call: {request.tool_name}")
    return request

# 인터셉터를 클라이언트에 전달
```

```

async with MultiServerMCPClient(
    config,
    interceptors=[auth_interceptor, logging_interceptor],
) as client:
    tools = client.get_tools()
    """
print("인터셉터는 도구 호출 전에 순서대로 실행됩니다.")

```

인터셉터에서 Command 반환

인터셉터는 단순히 요청을 가공하는 것을 넘어, LangGraph의 `Command` 객체를 반환하여 상태 업데이트와 다음 노드 지정까지 수행할 수 있습니다.

```

from langgraph.types import Command

async def logging_interceptor(request, context):
    """도구 호출을 로깅하면서 상태에 트레이스를 누적."""
    print(f"[interceptor] tool={request.tool_name}")
    return Command(
        update={"tool_calls_log": [request.tool_name]},
        goto="tools",
    )

```

이 패턴은 인터셉터에서 가드레일을 적용하다 위험 호출이 감지되면 `goto="human_review"` 로 보내는 식의 분기 로직을 구현할 때 유용합니다.

```

도구 인터셉터 패턴:
=====

from langchain_mcp_adapters.client import MultiServerMCPClient

# 인터셉터 함수 정의
async def auth_interceptor(request, context):
    """런타임 컨텍스트에서 사용자 토큰을 주입합니다."""
    user_token = context.get("user_token", "")
    request.params["auth_token"] = user_token
    return request

async def logging_interceptor(request, context):
    """도구 호출을 로깅합니다."""
    print(f"Tool call: {request.tool_name}")
    return request

# 인터셉터를 클라이언트에 전달
async with MultiServerMCPClient(
    config,
    interceptors=[auth_interceptor, logging_interceptor],
) as client:
    tools = client.get_tools()

인터셉터는 도구 호출 전에 순서대로 실행됩니다.

```

지금까지 기존 MCP 서버에 연결하는 방법을 배웠습니다. 하지만 조직 내부의 API나 비즈니스 로직을 MCP 도구로 노출하려면 직접 MCP 서버를 작성해야 합니다.

11.11 커스텀 MCP 서버 작성

FastMCP 라이브러리를 사용하면 데코레이터로 간편하게 MCP 서버를 구축할 수 있습니다. `@mcp.tool()` 데코레이터를 함수에 붙이면 해당 함수가 MCP 도구로 자동 등록되며, 함수의 타입 힌트와 독스트링이 도구의 파라미터 스키마와 설명으로 변환됩니다.

```
# 커스텀 MCP 서버 예시 (FastMCP)
print("커스텀 MCP 서버 작성 (FastMCP):")
print("=" * 50)
print("""
# my_server.py
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("my-tools")

@mcp.tool()
def add(a: int, b: int) -> int:
    \"""두 수를 더합니다.\"""
    return a + b

@mcp.tool()
def multiply(a: int, b: int) -> int:
    \"""두 수를 곱합니다.\"""
    return a * b

@mcp.resource("config://app")
def get_config() -> str:
    \"""앱 설정을 반환합니다.\"""
    return '{"version": "1.0", "debug": false}'

if __name__ == "__main__":
    mcp.run(transport="stdio")
""")
print("실행 방법:")
print("  1. Stdio: python my_server.py")
print("  2. HTTP: mcp.run(transport='streamable-http', port=8080)")
print()
print("LangChain 연결:")
print('  config = {"my_tools": {"transport": "stdio", "command": "python", "args": ["my_server.py"]}}')
```

```
커스텀 MCP 서버 작성 (FastMCP):
=====

# my_server.py
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("my-tools")

@mcp.tool()
def add(a: int, b: int) -> int:
    """두 수를 더합니다."""
    return a + b

@mcp.tool()
def multiply(a: int, b: int) -> int:
    """두 수를 곱합니다."""
    return a * b

@mcp.resource("config://app")
def get_config() -> str:
    """앱 설정을 반환합니다."""
    return '{"version": "1.0", "debug": false}'

if __name__ == "__main__":
    mcp.run(transport="stdio")

실행 방법:
1. Stdio: python my_server.py
2. HTTP: mcp.run(transport='streamable-http', port=8080)

... (truncated)
```

요약

이 노트북에서 배운 내용:

주제	핵심 내용
MCP 개념	외부 도구와 컨텍스트를 표준화된 방식으로 LLM에 제공하는 오픈 프로토콜입니다
Stdio 전송	로컬 서브프로세스를 통한 통신으로, 개발/테스트에 적합합니다
SSE/HTTP 전송	웹 기반 통신으로, 원격 서버 연결 및 인증을 지원합니다
에이전트 통합	<code>client.get_tools()</code> → <code>create_agent(tools=mcp_tools)</code> 로 연결합니다
다중 서버	<code>MultiServerMCPClient</code> 에 여러 서버를 설정하여 동시 관리합니다
인터셉터	도구 호출을 가로채 인증, 로깅, 수정 등의 미들웨어를 적용합니다
커스텀 서버	<code>FastMCP</code> 의 데코레이터로 간편하게 MCP 서버를 구축합니다

MCP를 통해 에이전트의 도구 생태계를 외부로 확장하는 방법을 배웠습니다. 다음 장에서는 에이전트의 응답을 사용자에게 실시간으로 전달하는 스트리밍 기법을 학습합니다. 백엔드의 `astream_events()` 부터 프론트엔드의 `React` `useStream` 혹까지, 전체 스트리밍 파이프라인을 다룹니다.

REFERENCES

- [MCP \(Model Context Protocol\)](#)

12 프론트엔드 스트리밍

사용자가 에이전트의 응답을 수 초간 기다리게 하는 것은 좋은 경험이 아닙니다. 토큰이 생성되는 즉시 화면에 표시하면 체감 응답 시간이 극적으로 줄어듭니다. 이 장에서는 백엔드의 LangGraph 스트리밍 API부터 프론트엔드의 React `useStream` 혹까지, 실시간 스트리밍을 구현하는 전체 파이프라인을 학습합니다.

스트리밍이 중요한 이유는 단순히 UX를 넘어서, 에이전트의 투명성에 있습니다. 에이전트가 어떤 도구를 호출하고 있는지, 어떤 단계를 실행 중인지를 실시간으로 보여주면 사용자의 신뢰가 높아집니다. LangGraph SDK는 `StreamEvent` 프로토콜을 통해 토큰 스트리밍뿐 아니라 도구 호출 시작/완료, 노드 전환 등의 내부 이벤트도 세밀하게 전달할 수 있습니다.

학습 목표

LLM 응답을 실시간으로 스트리밍하여 사용자에게 전달하는 방법을 알아봅니다.

이 노트북에서 다루는 내용:

- LangChain SDK의 스트리밍 기초(`.stream()` , `.astream_events()`)를 이해한다
- `useStream` React hook의 구조와 사용법을 안다
- `StreamEvent` 프로토콜을 이해한다
- Python SDK로 실시간 스트리밍을 소비하는 방법을 익힌다
- 에이전트 상태 실시간 표시 패턴을 안다

12.1 환경 설정

```
from dotenv import load_dotenv
import os
load_dotenv(override=True)

from langchain_openai import ChatOpenAI

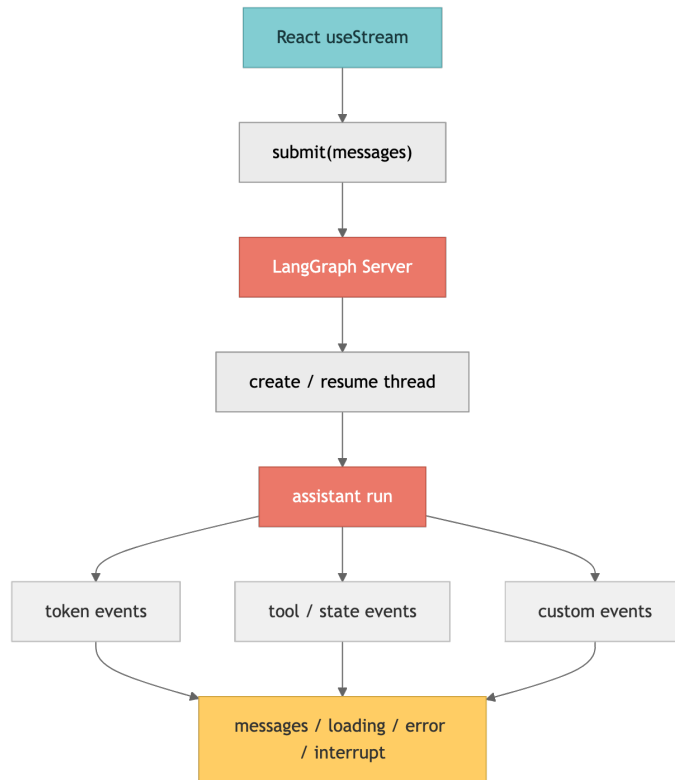
model = ChatOpenAI(
    model="gpt-5.4",
)

print("환경 준비 완료.")
```

환경 준비 완료.

12.2 Python SDK 스트리밍 기초

`.stream()` 메서드는 모델 응답을 토큰 단위로 실시간 전달합니다. 사용자는 전체 응답이 완성되기 전에 부분 결과를 볼 수 있습니다. 내부적으로 `.stream()` 은 LLM API의 스트리밍 응답을 파이썬 제너레이터로 감싸서, `for chunk in model.stream(...)` 형태로 각 토큰 청크를 순회할 수 있게 합니다.



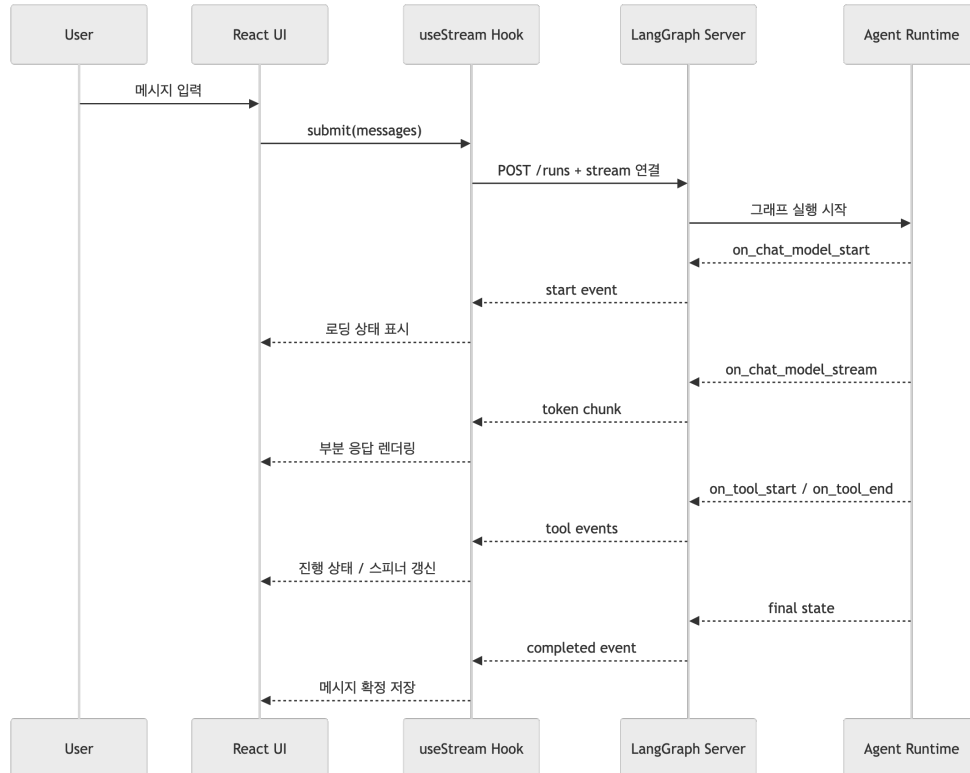
이 그림처럼 프론트엔드 스트리밍은 단순히 토큰을 받는 기능 이 아니라, 스레드 재개, 상태 동기화, 커스텀 이벤트 수신까지 포함하는 세션 프로토콜 에 가깝습니다.

`.stream()` 이 최종 출력만 토큰 단위로 전달한다면, `.astream_events()` 는 에이전트 실행의 모든 내부 이벤트를 제공합니다.

12.3 astream_events()

`.astream_events()` 는 비동기 방식으로 모든 내부 이벤트를 스트리밍합니다. 모델 호출, 도구 실행, 체인 단계 등을 세밀하게 추적할 수 있습니다. 각 이벤트는 `event` (이벤트 타입), `data` (이벤트 데이터), `metadata` (실행 컨텍스트) 필드를 포함하는 딕셔너리입니다.

주요 이벤트 타입



이 다이어그램은 사용자 입력 → `useStream` → LangGraph 서버 → 에이전트 런타임 → 이벤트 반환의 왕복 흐름을 보여줍니다. 실무에서는 토큰 자체보다도 `on_tool_start`, `on_tool_end` 같은 상태 이벤트가 중요합니다. 사용자는 “지금 답을 쓰는 중인지”, “검색 도구를 실행 중인지”를 구분해 볼 수 있어야 체감 대기 시간이 크게 줄어듭니다.

TIP

이벤트 읽는 법: **start 계열**은 UI 상태 전환, **stream 계열**은 점진적 렌더링, **end 계열**은 메시지 확정 저장에 연결하면 구현이 깔끔합니다.

이벤트	설명
<code>on_chat_model_stream</code>	모델 토큰 스트리밍
<code>on_chat_model_start</code>	모델 호출 시작

on_chat_model_end	모델 호출 완료
on_tool_start	도구 실행 시작
on_tool_end	도구 실행 완료

```
import asyncio

async def stream_events_demo():
    """astream_events()로 이벤트 스트리밍 예시"""
    print("이벤트 스트리밍:")
    print("-" * 40)
    async for event in model.astream_events(
        "파이썬의 강점 2가지",
        version="v2",
    ):
        kind = event["event"]
        if kind == "on_chat_model_stream":
            content = event["data"]["chunk"].content
            if content:
                print(content, end="", flush=True)
        elif kind == "on_chat_model_start":
            print(f"[모델 호출 시작]")
        elif kind == "on_chat_model_end":
            print(f"\n[모델 호출 완료]")

await stream_events_demo()
```

이벤트 스트리밍:

[모델 호출 시작]

파
이
썬
의
강
점
두
가지
는
다음
과
같습니다
.

1
.
**
코
드
가
간
결
하고
읽
기
... (truncated)

백엔드의 스트리밍 API를 살펴보았으니, 이제 프론트엔드에서 이 스트림을 소비하는 방법을 알아보겠습니다.

`useStream` React 혹은 백엔드의 복잡한 스트리밍 프로토콜을 추상화하여, 프론트엔드 개발자가 몇 줄의 코드로 실시간 채팅 UI를 구축할 수 있게 합니다.

12.4 useStream React 훅

`useStream` 은 LangGraph SDK(`@langchain/langgraph-sdk/react`)에서 제공하는 React 훅으로, LangGraph 서버와의 스트리밍 통신을 간편하게 처리합니다. 이 훅은 스레드 생성, 메시지 전송, 스트림 수신, 에러 처리를 모두 내부적으로 관리합니다.

· NOTE

이벤트 타입별 사용 기준

- `on_chat_model_stream` — 사용자에게 바로 보여 줄 텍스트 토큰
- `on_tool_start` / `on_tool_end` — 로딩 스피너, 단계 표시, 로그 패널
- 커스텀 이벤트 — 진행률, 분석 중간 결과, 멀티에이전트 상태 같은 앱 전용 UI

기본 사용법

```
import { useStream } from "@langchain/langgraph-sdk/react";

function Chat() {
  const stream = useStream({
    assistantId: "agent",
    apiUrl: "http://localhost:2024",
  });

  const handleSubmit = (message: string) => {
    stream.submit({
      messages: [{ content: message, type: "human" }],
    });
  };

  return (
    <div>
      {stream.messages.map((message, idx) => (
        <div key={message.id ?? idx}>
          {message.type}: {message.content}
        </div>
      ))}
      {stream.isLoading && <div>Loading...</div>}
      {stream.error && <div>Error: {stream.error.message}</div>}
    </div>
  );
}
```

주요 반환값

속성	타입	설명
<code>messages</code>	<code>Message[]</code>	현재 스레드의 전체 메시지
<code>isLoading</code>	<code>boolean</code>	스트림 진행 여부
<code>error</code>	<code>Error \ , [null]</code>	에러 객체
<code>interrupt</code>	<code>Interrupt</code>	중단 요청 (HITL)
<code>submit()</code>	<code>function</code>	메시지 전송

<code>stop()</code>	<code>function</code>	스트림 중단
---------------------	-----------------------	--------

12.5 useStream 설정 옵션

파라미터	필수	기본값	설명
<code>assistantId</code>	○	—	에이전트 식별자 (배포 대시보드에서 확인)
<code>apiUrl</code>	—	<code>localhost:2024</code>	에이전트 서버 URL
<code>apiKey</code>	—	—	배포된 에이전트 인증 토큰
<code>threadId</code>	—	—	기존 대화 스레드에 연결
<code>onThreadId</code>	—	—	스레드 생성 시 콜백
<code>reconnectOnMount</code>	—	<code>false</code>	컴포넌트 마운트 시 진행 중 스트림 재연결
<code>onCustomEvent</code>	—	—	커스텀 이벤트 핸들러
<code>onUpdateEvent</code>	—	—	상태 업데이트 핸들러
<code>onMetadataEvent</code>	—	—	메타데이터 이벤트 핸들러
<code>messagesKey</code>	—	<code>"messages"</code>	메시지를 담는 상태 키
<code>throttle</code>	—	<code>true</code>	상태 업데이트 배치 처리
<code>initialValues</code>	—	—	캐시된 초기 상태

12.6 스레드 관리와 재연결

스레드 ID 관리

`threadId` 를 관리하면 대화를 지속하거나 이전 대화를 불러올 수 있습니다.

```
const [threadId, setThreadId] = useState<string | null>(null);

const stream = useStream({
  apiUrl: "http://localhost:2024",
  assistantId: "agent",
  threadId,
  onThreadId: setThreadId,
});

// threadId를 URL 파라미터나 localStorage에 저장하여 지속성 확보
```

페이지 새로고침 후 재연결

`reconnectOnMount` 를 활성화하면 페이지 새로고침 후에도 진행 중이던 스트림에 자동 재연결됩니다.

```
const stream = useStream({
  apiUrl: "http://localhost:2024",
  assistantId: "agent",
  reconnectOnMount: true, // sessionStorage 사용
});

// 커스텀 스토리지 사용
const stream = useStream({
  reconnectOnMount: () => window.localStorage,
});
```

12.7 브랜칭과 메시지 편집

브랜칭을 사용하면 대화 히스토리의 특정 지점에서 대체 경로를 만들 수 있습니다. 이 기능의 기반은 LangGraph의 체크포인트 시스템입니다. 매 단계마다 상태가 자동으로 저장되므로, 이전 체크포인트로 돌아가 새로운 입력으로 분기를 생성할 수 있습니다. 메시지를 편집하거나 AI 응답을 재생성할 때 유용합니다.

```
{stream.messages.map((message) => {
  const meta = stream.getMessagesMetadata(message);
  const parentCheckpoint = meta?.firstSeenState?.parent_checkpoint;

  return (
    <div key={message.id}>
      {message.content}

      {/* 사용자 메시지 편집 */}
      {message.type === "human" && (
        <button onClick={() => {
          const newContent = prompt("Edit:", message.content);
          if (newContent) {
            stream.submit(
              { messages: [{ type: "human", content: newContent } ] },
              { checkpoint: parentCheckpoint }
            );
          }
        }}>
          Edit
        </button>
      )}

      {/* AI 응답 재생성 */}
      {message.type === "ai" && (
        <button onClick={() =>
          stream.submit(undefined, { checkpoint: parentCheckpoint })
        }>
          Regenerate
        </button>
      )}
    </div>
  );
})}
```

핵심: `checkpoint` 파라미터로 특정 시점의 상태로 돌아가 새로운 분기를 생성합니다.

12.8 커스텀 스트리밍 이벤트

LangGraph의 기본 이벤트(토큰 스트리밍, 도구 호출 등) 외에도, 에이전트에서 커스텀 데이터를 클라이언트로 스트리밍할 수 있습니다. 도구 실행 중 진행률 표시, 중간 분석 결과 전달, 단계별 상태 업데이트 등 애플리케이션

고유의 데이터를 실시간으로 전달할 때 유용합니다. 서버 측에서는 `config.writer()` 패턴을 사용하고, 클라이언트 측에서는 `onCustomEvent` 콜백으로 수신합니다.

```
# 커스텀 스트리밍 이벤트 - Python writer 패턴
print("커스텀 스트리밍 이벤트 패턴 (Python 서버 측):")
print("=" * 50)
print("""
from langchain.tools import tool
from langchain.agents.types import ToolRuntime

@tool
async def analyze_data(
    data_source: str, *, config: ToolRuntime
) -> str:
    \\\"\\\"\\\"데이터를 분석합니다.\\\"\\\"\\\"
    if config.writer:
        # 진행 상황을 클라이언트에 스트리밍
        config.writer({
            \"type\": \"progress\",
            \"message\": \"데이터 로딩 중...\",
            \"progress\": 25,
        })
        # ... 처리 ...
        config.writer({
            \"type\": \"progress\",
            \"message\": \"분석 완료!\",
            \"progress\": 100,
        })
    return '{\"result\": \"분석 완료\"}'
""")
print("클라이언트(React) 측: onCustomEvent 콜백으로 수신")
print('  onCustomEvent: (data) => setProgress(data.progress)')
```

커스텀 스트리밍 이벤트 패턴 (Python 서버 측):

```
=====

from langchain.tools import tool
from langchain.agents.types import ToolRuntime

@tool
async def analyze_data(
    data_source: str, *, config: ToolRuntime
) -> str:
    """데이터를 분석합니다."""
    if config.writer:
        # 진행 상황을 클라이언트에 스트리밍
        config.writer({
            \"type\": \"progress\",
            \"message\": \"데이터 로딩 중...\",
            \"progress\": 25,
        })
        # ... 처리 ...
        config.writer({
            \"type\": \"progress\",
            \"message\": \"분석 완료!\",
            \"progress\": 100,
        })
    return '{\"result\": \"분석 완료\"}'
```

클라이언트(React) 측: onCustomEvent 콜백으로 수신
onCustomEvent: (data) => setProgress(data.progress)

12.9 멀티 에이전트 스트리밍

8장에서 학습한 멀티 에이전트 패턴과 스트리밍을 결합하면, 여러 에이전트가 협업하는 과정을 사용자에게 실시간으로 보여줄 수 있습니다. 여러 에이전트가 협업하는 환경에서는 각 에이전트의 메시지를 구분하여 표시해야 합니다. 메타데이터의 `langgraph_node` 를 사용하여 메시지 출처를 식별합니다.

TIP

멀티 에이전트 스트리밍에서는 각 에이전트에 고유한 색상과 레이블을 부여하여 시각적으로 구분하는 것이 좋습니다. 사용자가 “지금 리서치 에이전트가 검색 중”인지 “작성 에이전트가 글을 쓰는 중”인지 한눈에 파악할 수 있으면 에이전트 시스템에 대한 신뢰도가 크게 높아집니다.

```
// 노드별 설정
const NODE_CONFIG: Record<string, { label: string; color: string }> = {
  researcher: { label: "Research Agent", color: "blue" },
  writer:      { label: "Writing Agent",   color: "green" },
  reviewer:   { label: "Review Agent",    color: "purple" },
};

// 메시지 렌더링
function AgentMessage({ message, stream }) {
  const metadata = stream.getMessagesMetadata?.(message);
  const nodeName = metadata?.streamMetadata?.langgraph_node;
  const config = NODE_CONFIG[nodeName];

  return (
    <div className={`bg-${config?.color}-950/30 p-4 rounded-lg`} >
      <div className={`text-${config?.color}-400 text-sm font-bold`} >
        {config?.label ?? "Agent"}
      </div>
      <div>{message.content}</div>
    </div>
  );
}
```

이벤트 콜백 정리

콜백	용도	스트림 모드
onUpdateEvent	그래프 단계 후 상태 업데이트	updates
onCustomEvent	에이전트의 커스텀 이벤트	custom
onMetadataEvent	실행 및 스레드 메타데이터	metadata
onError	에러 처리	—
onFinish	스트림 완료	—

요약

이 노트북에서 배운 내용:

주제	핵심 내용
SDK 스트리밍	<code>.stream()</code> 으로 토큰 단위 실시간 응답을 받습니다
<code>astream_events</code>	비동기 이벤트 스트리밍으로 모델/도구 호출을 세밀하게 추적합니다

useStream	React 혹은 LangGraph 서버와 스트리밍 통신을 간편하게 처리합니다
스레드 관리	<code>threadId</code> 와 <code>reconnectOnMount</code> 로 대화 지속성을 확보합니다
브랜칭	<code>checkpoint</code> 기반으로 대화의 대체 경로를 생성합니다
커스텀 이벤트	<code>writer</code> 패턴으로 진행 상황 등 커스텀 데이터를 스트리밍합니다
멀티에이전트	<code>langgraph_node</code> 메타데이터로 에이전트별 메시지를 구분 표시합니다

스트리밍을 통해 에이전트의 실행 과정을 사용자에게 투명하게 전달하는 방법을 배웠습니다. 그러나 에이전트가 사용자에게 직접 노출되는 만큼, 안전하지 않은 입력이나 부적절한 출력에 대한 방어가 필수적입니다. 다음 장에서는 에이전트의 입출력 경계에 가드레일을 설치하여 PII 유출, 프롬프트 인젝션, 위험한 도구 실행 등을 방지하는 방법을 학습합니다.

REFERENCES

- [Streaming](#) — `useStream` 정식 레퍼런스와 이벤트 프로토콜 상세
- [UI \(Agent Chat UI & useStream\)](#)

13 가드레일

에이전트가 프로덕션 환경에서 사용자와 직접 상호작용하면, 악의적 입력(프롬프트 인젝션), 민감 정보 유출 (PII), 부적절한 응답 등의 위험이 생깁니다. 가드레일은 에이전트의 입출력 경계에 검증과 필터링 로직을 배치하여 이러한 위험을 완화합니다. 이 장에서는 결정론적 가드레일과 모델 기반 가드레일의 차이, PII 감지, Human-in-the-Loop 승인 등 실전 가드레일 패턴을 학습합니다.

가드레일은 에이전트의 안전 경계(safety boundary)를 정의합니다. LLM 자체의 안전 훈련(RLHF 등)만으로는 모든 위험을 커버할 수 없으며, 특히 도메인 특화 규칙(금융 컴플라이언스, 의료 면책 등)은 애플리케이션 레벨에서 명시적으로 구현해야 합니다. LangChain v1의 미들웨어 시스템은 가드레일을 에이전트 코드와 분리하여 재사용 가능한 컴포넌트로 관리할 수 있게 합니다.

학습 목표

에이전트의 입력과 출력을 검증하고 필터링하는 가드레일을 설정하는 방법을 알아봅니다.

이 노트북에서 다루는 내용:

- 가드레일의 개념과 필요성을 이해한다
- 결정론적 가드레일과 모델 기반 가드레일의 차이를 안다
- PII 감지 미들웨어를 설정한다
- Human-in-the-Loop 가드레일을 구현한다
- 커스텀 before/after 가드레일을 작성한다

13.1 환경 설정

```
from dotenv import load_dotenv
import os
load_dotenv(override=True)

from langchain_openai import ChatOpenAI

model = ChatOpenAI(
    model="gpt-5.4",
)

from langchain.agents import create_agent
from langchain.tools import tool

print("환경 준비 완료.")
```

환경 준비 완료.

13.2 가드레일 개념

가드레일(Guardrails)은 에이전트 실행 과정에서 콘텐츠를 검증하고 필터링하는 안전 메커니즘입니다.

왜 가드레일이 필요한가?

- 개인정보(PII) 유출 방지
- 프롬프트 인젝션 공격 차단
- 부적절하거나 유해한 콘텐츠 차단
- 비즈니스 규칙 및 컴플라이언스 준수
- 출력 품질 및 정확성 검증

두 가지 접근법

접근법	방식	장점	단점
결정론적	정규식, 키워드 매칭, 명시적 규칙	빠르고, 예측 가능하며, 비용 효율적	미묘한 위반 사항 놓칠 수 있음
모델 기반	LLM이나 분류기로 의미를 분석	미묘한 문제도 감지	느리고 비용이 높음

· NOTE

실패 예시로 기억하기

- 입력 단계 실패 — 이메일/주민번호가 포함된 요청을 마스킹하지 않고 모델에 전달
- 도구 단계 실패 — `send_email`, `execute_sql` 같은 민감 도구를 승인 없이 실행
- 출력 단계 실패 — 모델이 내부 정책, PII, 공격 프롬프트 일부를 그대로 응답

가드레일은 세 단계가 각각 다른 실패를 막는다는 점을 기억하세요.

13.3 PII 감지 미들웨어

PIIMiddleware는 이메일, 신용카드 번호, IP 주소 등 개인식별정보(PII)를 자동으로 감지하고 처리합니다. PII가 에이전트의 컨텍스트에 유입되면 LLM이 이를 학습하거나 출력에 포함시킬 위험이 있으므로, 모델에 전달되기 전에 감지하여 처리하는 것이 중요합니다.

전략	결과
redact	[REDACTED_EMAIL] 로 대체
mask	부분 가리기 (예: 마지막 4자리만 표시)
hash	결정론적 해시로 대체
block	감지 시 예외 발생

· NOTE

적용 범위 파라미터 — PIIMiddleware 는 `apply_to_input` (사용자 메시지), `apply_to_output` (모델 응답), `apply_to_tool_results` 세 지점에 독립적으로 적용 가능합니다. `apply_to_tool_results` 의 기본값은 `False` — 도구 반환값(예: 데이터베이스 조회 결과)에는 자동 적용되지 않으므로, 도구 출력에 PII가 들어올 수 있다면 명시적으로 `apply_to_tool_results=True` 를 켜야 합니다.

```
# PII 감지 미들웨어 설정 예시
print("PII 감지 미들웨어 설정:")
print("=" * 50)
print("""
from langchain.agents import create_agent
from langchain.agents.middleware import PIIMiddleware

agent = create_agent(
    model="gpt-5.4",
    tools=[customer_service_tool, email_tool],
    middleware=[
        # 이메일 주소를 [REDACTED_EMAIL]로 대체
        PIIMiddleware("email",
            strategy="redact",
            apply_to_input=True),

        # 신용카드 번호를 부분 마스킹 (****-****-****-1234)
        PIIMiddleware("credit_card",
            strategy="mask",
            apply_to_input=True),
    ]
)
```

```

        # API 키 감지 시 차단 (커스텀 정규식)
        PIIMiddleware("api_key",
            detector=r"sk-[a-zA-Z0-9]{32}",
            strategy="block",
            apply_to_input=True),
    ],
)
"""
print("내장 PII 타입: email, credit_card, ip, mac_address, url")
print("커스텀 감지: detector 파라미터에 정규식 또는 함수 전달")

```

PII 감지 미들웨어 설정:

```

=====

from langchain.agents import create_agent
from langchain.agents.middleware import PIIMiddleware

agent = create_agent(
    model="gpt-5.4",
    tools=[customer_service_tool, email_tool],
    middleware=[
        # 이메일 주소를 [REDACTED_EMAIL]로 대체
        PIIMiddleware("email",
            strategy="redact",
            apply_to_input=True),

        # 신용카드 번호를 부분 마스킹 (****.****.****-1234)
        PIIMiddleware("credit_card",
            strategy="mask",
            apply_to_input=True),

        # API 키 감지 시 차단 (커스텀 정규식)
        PIIMiddleware("api_key",
            detector=r"sk-[a-zA-Z0-9]{32}",
            strategy="block",
            apply_to_input=True),
    ],
)

내장 PII 타입: email, credit_card, ip, mac_address, url
커스텀 감지: detector 파라미터에 정규식 또는 함수 전달

```

PII 감지가 데이터의 내용을 검사하는 가드레일이라면, Human-in-the-Loop(HITL)은 에이전트의 행동을 제어하는 가드레일입니다. 특정 도구의 실행 전에 사람의 승인을 요구하여, 되돌릴 수 없는 작업의 안전성을 확보합니다.

13.4 Human-in-the-Loop 가드레일

HumanInTheLoopMiddleware는 민감한 작업을 실행하기 전에 사람의 승인을 요구합니다. 내부적으로 LangGraph의 `interrupt()` 메커니즘을 사용하여 그래프 실행을 일시 중단하고, 사용자의 승인 또는 거부 결정을 받은 후 `Command(resume=...)` 으로 실행을 재개합니다. 금융 거래, 데이터 삭제, 외부 통신 등 고위험 작업에 필수적입니다.

! WARNING

HITL 가드레일은 반드시 `checkpointner` 와 함께 사용해야 합니다. 체크포인트가 없으면 그래프 실행 상태를 저장할 수 없어, 중단 후 재개가 불가능합니다. 프로덕션에서는 `InMemorySaver` 대신 `SqliteSaver` 나 `PostgresSaver` 같은 영속적 체크포인트를 사용하세요.

```
# Human-in-the-Loop 가드레일 예시
print("Human-in-the-Loop 가드레일:")
print("=" * 50)
print("""
from langchain.agents import create_agent
from langchain.agents.middleware import HumanInTheLoopMiddleware
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.types import Command

agent = create_agent(
    model="gpt-5.4",
    tools=[search_tool, send_email_tool, delete_db_tool],
    middleware=[
        HumanInTheLoopMiddleware(
            interrupt_on={
                "send_email": True,      # 승인 필요
                "delete_db": True,       # 승인 필요
                "search": False,         # 자동 실행
            }
        ),
    ],
    checkpointer=InMemorySaver(),
)

config = {"configurable": {"thread_id": "review-123"}}

# 1단계: 에이전트 실행 → send_email에서 중단
result = agent.invoke(
    {"messages": [{"role": "user", "content": "팀에 이메일 보내"}]},
    config=config,
)
# → 중단됨: send_email 실행 전 승인 대기

# 2단계: 승인 후 재개
result = agent.invoke(
    Command(resume={"decisions": [{"type": "approve"}]}),
    config=config,
)
""")
print("핵심: checkpointer가 있어야 중단/재개가 가능합니다.")
print("거부 시: {\\"type\": \\"reject\\"}로 도구 실행을 막을 수 있습니다.")
```

Human-in-the-Loop 가드레일:

```
=====

from langchain.agents import create_agent
from langchain.agents.middleware import HumanInTheLoopMiddleware
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.types import Command

agent = create_agent(
    model="gpt-5.4",
    tools=[search_tool, send_email_tool, delete_db_tool],
    middleware=[
        HumanInTheLoopMiddleware(
            interrupt_on={
                "send_email": True,      # 승인 필요
                "delete_db": True,       # 승인 필요
                "search": False,         # 자동 실행
            }
        ),
    ],
    checkpointer=InMemorySaver(),
)

config = {"configurable": {"thread_id": "review-123"}}

# 1단계: 에이전트 실행 → send_email에서 중단
result = agent.invoke(
    {"messages": [{"role": "user", "content": "팀에 이메일 보내"}]},
    config=config,
)
... (truncated)
```

내장 미들웨어(`PIIMiddleware` , `HumanInTheLoopMiddleware`)가 제공하지 않는 검증 로직이 필요한 경우, 커스텀 가드레일을 직접 작성할 수 있습니다. LangChain v1의 미들웨어 시스템은 `before_agent` (입력 가드레일)와 `after_agent` (출력 가드레일) 두 가지 훅 포인트를 제공합니다.

13.5 커스텀 입력 가드레일 – `before_agent`

`before_agent` 혹은 에이전트 실행 시작 전에 요청을 검증합니다. 이 훅이 `None` 을 반환하면 정상적으로 에이전트가 실행되고, 디렉터리를 반환하면서 `jump_to="end"` 를 지정하면 에이전트 실행을 건너뛰고 즉시 응답합니다. 세션 수준의 인증, 비율 제한, 콘텐츠 필터링 등에 사용합니다.

```
# 커스텀 입력 가드레일 – ContentFilterMiddleware 클래스
print("커스텀 입력 가드레일 (클래스 방식):")
print("=" * 50)
print("""
from langchain.agents.middleware import (
    AgentMiddleware, AgentState, hook_config
)
from langgraph.runtime import Runtime
from typing import Any

class ContentFilterMiddleware(AgentMiddleware):
    """결정론적 가드레일: 금지 키워드가 포함된 요청을 차단합니다."""

    def __init__(self, banned_keywords: list[str]):
        super().__init__()
        self.banned_keywords = [kw.lower() for kw in banned_keywords]

    @hook_config(can_jump_to=["end"])
    def before_agent(
        self, state: AgentState, runtime: Runtime
    ) -> dict[str, Any] | None:
        if not state["messages"]:
            return None

        first_message = state["messages"][0]
        if first_message.type != "human":
            return None

        content = first_message.content.lower()
        for keyword in self.banned_keywords:
            if keyword in content:
                return {
                    "messages": [{
                        "role": "assistant",
                        "content": "부적절한 내용이 포함되어 있습니다."
                    }],
                    "jump_to": "end"
                }
        return None

""")
print("핵심: jump_to='end'로 에이전트 실행을 건너뛰고 즉시 응답합니다.")
print("None을 반환하면 다음 단계(에이전트 실행)로 진행합니다.")
```

커스텀 입력 가드레일 (클래스 방식):

```
=====

from langchain.agents.middleware import (
    AgentMiddleware, AgentState, hook_config
)
from langgraph.runtime import Runtime
from typing import Any

class ContentFilterMiddleware(AgentMiddleware):
    """결정론적 가드레일: 금지 키워드가 포함된 요청을 차단합니다."""
```

```

def __init__(self, banned_keywords: list[str]):
    super().__init__()
    self.banned_keywords = [kw.lower() for kw in banned_keywords]

@hook_config(can_jump_to=["end"])
def before_agent(
    self, state: AgentState, runtime: Runtime
) -> dict[str, Any] | None:
    if not state["messages"]:
        return None

    first_message = state["messages"][0]
    if first_message.type != "human":
        return None

    content = first_message.content.lower()
    for keyword in self.banned_keywords:
        if keyword in content:
            ... (truncated)

```

입력 가드레일이 요청의 적절성을 검증한다면, 출력 가드레일은 에이전트가 생성한 응답의 안전성과 품질을 검증합니다.

13.6 커스텀 출력 가드레일 — after_agent

`after_agent` 훅은 에이전트 실행 완료 후 최종 출력을 검증합니다. 모델 기반 안전성 검사, 품질 검증 등에 사용됩니다. 출력 가드레일에서 응답 내용을 직접 수정하거나, `jump_to="end"` 로 안전한 대체 응답을 반환할 수 있습니다.

TIP

출력 가드레일에서 모델 기반 검사를 수행할 때는, 에이전트 본체보다 경량 모델(예: `gpt-5.4-mini`)을 사용하는 것이 비용과 지연 시간 면에서 유리합니다. 안전성 분류는 복잡한 추론이 필요하지 않으므로, 작은 모델로도 충분히 정확한 판별이 가능합니다.

```

# 커스텀 출력 가드레일 — SafetyGuardrailMiddleware 클래스
print("커스텀 출력 가드레일 (클래스 방식):")
print("=" * 50)
print("""
from langchain.agents.middleware import (
    AgentMiddleware, AgentState, hook_config
)
from langgraph.runtime import Runtime
from langchain.messages import AIMessage
from langchain.chat_models import init_chat_model
from typing import Any

class SafetyGuardrailMiddleware(AgentMiddleware):
    \"""모델 기반 가드레일: LLM으로 응답 안전성을 평가합니다.\""""

    def __init__(self):
        super().__init__()
        self.safety_model = init_chat_model("gpt-5.4-mini")

    @hook_config(can_jump_to=["end"])
    def after_agent(
        self, state: AgentState, runtime: Runtime
    ) -> dict[str, Any] | None:
        if not state["messages"]:
            return None

```



```

last_message = state["messages"][-1]
if not isinstance(last_message, AIMessage):
    return None

safety_prompt = f"\n\nEvaluate if this response is safe.
Respond with only 'SAFE' or 'UNSAFE'."

Response: {last_message.content}\n\n"

result = self.safety_model.invoke(
    [{"role": "user", "content": safety_prompt}]
)

if "UNSAFE" in result.content:
    last_message.content = (
        "안전하지 않은 응답입니다. 다시 질문해주세요."
    )
    return None
"""
print("핵심: 별도의 경량 모델(gpt-5.4-mini)로 안전성을 평가합니다.")
print("UNSAFE 판정 시 응답 내용을 안전한 메시지로 교체합니다.")

```

커스텀 출력 가드레일 (클래스 방식):

```

=====
from langchain.agents.middleware import (
    AgentMiddleware, AgentState, hook_config
)
from langgraph.runtime import Runtime
from langchain.messages import AIMessage
from langchain.chat_models import init_chat_model
from typing import Any

class SafetyGuardrailMiddleware(AgentMiddleware):
    """모델 기반 가드레일: LLM으로 응답 안전성을 평가합니다."""

    def __init__(self):
        super().__init__()
        self.safety_model = init_chat_model("gpt-5.4-mini")

    @hook_config(can_jump_to=["end"])
    def after_agent(
        self, state: AgentState, runtime: Runtime
    ) -> dict[str, Any] | None:
        if not state["messages"]:
            return None

        last_message = state["messages"][-1]
        if not isinstance(last_message, AIMessage):
            return None

        safety_prompt = f"""Evaluate if this response is safe.
... (truncated)

```

클래스 방식은 상태 관리와 초기화 로직이 필요한 복잡한 가드레일에 적합하지만, 단순한 검증 로직에는 과할 수 있습니다. 이런 경우 데코레이터 방식이 더 간결합니다.

13.7 데코레이터 방식 가드레일

클래스 대신 데코레이터를 사용하면 간결하게 가드레일을 정의할 수 있습니다. `@before_agent()` 와 `@after_agent()` 데코레이터는 일반 함수를 미들웨어 호환 객체로 변환합니다.

```

# 데코레이터 방식 가드레일
print("데코레이터 방식 가드레일:")
print("=" * 50)
print("""

```

```

from langchain.agents.middleware import (
    before_agent, after_agent, AgentState, hook_config
)
from langgraph.runtime import Runtime
from typing import Any

banned_keywords = ["hack", "exploit", "malware"]

# 입력 가드레일 - 데코레이터
@before_agent(can_jump_to=["end"])
def content_filter(
    state: AgentState, runtime: Runtime
) -> dict[str, Any] | None:
    \"\"\"금지 키워드를 차단합니다.\"\"\"
    if not state["messages"]:
        return None
    content = state["messages"][0].content.lower()
    for kw in banned_keywords:
        if kw in content:
            return {
                "messages": [{ "role": "assistant",
                                "content": "부적절한 요청입니다."}],
                "jump_to": "end"
            }
    return None

# 출력 가드레일 - 데코레이터
@after_agent(can_jump_to=["end"])
def safety_check(
    state: AgentState, runtime: Runtime
) -> dict[str, Any] | None:
    \"\"\"응답에 민감한 내용이 없는지 확인합니다.\"\"\"
    last = state["messages"][-1]
    if hasattr(last, 'content') and '비밀번호' in last.content:
        last.content = "민감한 정보가 포함된 응답입니다."
    return None

# 에이전트에 적용
agent = create_agent(
    model="gpt-5.4",
    tools=[search_tool],
    middleware=[content_filter, safety_check],
)
\"\"\"
print("데코레이터 방식은 간단한 가드레일에 적합합니다.")
print("복잡한 로직(상태 관리, 초기화 등)은 클래스 방식을 사용하세요.")

```

데코레이터 방식 가드레일:

```

=====
from langchain.agents.middleware import (
    before_agent, after_agent, AgentState, hook_config
)
from langgraph.runtime import Runtime
from typing import Any

banned_keywords = ["hack", "exploit", "malware"]

# 입력 가드레일 - 데코레이터
@before_agent(can_jump_to=["end"])
def content_filter(
    state: AgentState, runtime: Runtime
) -> dict[str, Any] | None:
    \"\"\"금지 키워드를 차단합니다.\"\"\"
    if not state["messages"]:
        return None
    content = state["messages"][0].content.lower()
    for kw in banned_keywords:
        if kw in content:
            return {
                "messages": [{ "role": "assistant",
                                "content": "부적절한 요청입니다."}],
                "jump_to": "end"
            }
    }

```

```

return None

# 출력 가드레일 - 데코레이터
... (truncated)

```

개별 가드레일의 작성 방법을 익혔다면, 이제 여러 가드레일을 조합하여 다층 방어(defense in depth) 전략을 구성하는 방법을 살펴보겠습니다. 보안에서는 단일 방어선에 의존하지 않는 것이 원칙이며, 가드레일도 마찬가지입니다.

13.8 다중 가드레일 조합

여러 가드레일을 `middleware` 리스트에 순서대로 추가하여 다층 방어를 구성합니다. `middleware` 리스트의 순서가 실행 순서를 결정하므로, 빠르고 저렴한 결정론적 가드레일을 앞에, 느리고 비용이 높은 모델 기반 가드레일을 뒤에 배치하는 것이 중요합니다.

```

# 다중 가드레일 조합
print("다중 가드레일 조합 (다층 방어):")
print("=" * 50)
print("""
from langchain.agents import create_agent
from langchain.agents.middleware import (
    PIIMiddleware, HumanInTheLoopMiddleware
)

agent = create_agent(
    model="gpt-5.4",
    tools=[search_tool, send_email_tool],
    middleware=[
        # Layer 1: 결정론적 입력 필터
        ContentFilterMiddleware(
            banned_keywords=["hack", "exploit"]
        ),

        # Layer 2: PII 보호 (입력 + 출력)
        PIIMiddleware("email",
            strategy="redact", apply_to_input=True),
        PIIMiddleware("email",
            strategy="redact", apply_to_output=True),

        # Layer 3: 민감 도구 사람 승인
        HumanInTheLoopMiddleware(
            interrupt_on={"send_email": True}
        ),

        # Layer 4: 모델 기반 안전성 검사
        SafetyGuardrailMiddleware(),
    ],
)
""")
print("실행 순서:")
print("  입력 → [ContentFilter] → [PII 입력] → 에이전트 실행")
print("      → [HITL 승인] → [PII 출력] → [Safety] → 응답")
print()
print("팁: 빠른 결정론적 가드레일을 앞에, 느린 모델 기반을 뒤에 배치")

```

다중 가드레일 조합 (다층 방어):

```

=====

from langchain.agents import create_agent
from langchain.agents.middleware import (
    PIIMiddleware, HumanInTheLoopMiddleware
)

agent = create_agent(

```

```

model="gpt-5.4",
tools=[search_tool, send_email_tool],
middleware=[
    # Layer 1: 결정론적 입력 필터
    ContentFilterMiddleware(
        banned_keywords=["hack", "exploit"]
    ),

    # Layer 2: PII 보호 (입력 + 출력)
    PIIMiddleware("email",
        strategy="redact", apply_to_input=True),
    PIIMiddleware("email",
        strategy="redact", apply_to_output=True),

    # Layer 3: 민감 도구 사용 승인
    HumanInTheLoopMiddleware(
        interrupt_on={"send_email": True}
    ),

    # Layer 4: 모델 기반 안전성 검사
    SafetyGuardrailMiddleware(),
    ... (truncated)

```

13.9 프로덕션 가드레일 패턴

지금까지 학습한 개별 가드레일 기법을 프로덕션 환경에 적용할 때 따라야 할 모범 사례와 도메인별 가이드라인을 정리합니다.

모범 사례

패턴	설명	구현 방식
다층 방어	여러 가드레일을 조합하여 단일 실패점 제거	<code>middleware=[layer1, layer2, ...]</code>
빠른 실패	결정론적 검사를 먼저 실행하여 비용 절감	결정론적 → 모델 기반 순서
입출력 분리	입력과 출력에 각각 적합한 가드레일 적용	<code>before_agent</code> + <code>after_agent</code>
그raceful 거부	차단 시 사용자에게 친절한 안내 제공	<code>jump_to="end"</code> + 안내 메시지
로깅 및 모니터링	가드레일 트리거 이벤트를 기록	LangSmith 트레이싱 연동
폴백 전략	가드레일 자체가 실패할 경우의 대비책	<code>try/except</code> + 기본 정책
테스트	가드레일 동작을 단위 테스트로 검증	<code>GenericFakeChatModel</code> 활용

도메인별 가드레일 예시

도메인	주요 가드레일
의료	PII(환자정보), 의학적 조언 면책, 응급상황 감지
금융	PII(계좌정보), 투자 면책, HITL(거래 승인)
고객서비스	감정 분석, 에스컬레이션 감지, PII 마스킹
교육	연령 적합성 검사, 학술 정직성, 콘텐츠 필터링

요약

이 노트북에서 배운 내용:

주제	핵심 내용
가드레일 개념	에이전트 실행 과정에서 콘텐츠를 검증하고 필터링하는 안전 메커니즘입니다
PII 감지	<code>PIIMiddleware</code> 로 이메일, 신용카드 등 개인정보를 자동 감지/처리합니다
HITL	<code>HumanInTheLoopMiddleware</code> 로 민감한 도구 실행 전 사람의 승인을 요구합니다
커스텀 입력	<code>before_agent</code> 혹은 에이전트 실행 전 요청을 검증합니다
커스텀 출력	<code>after_agent</code> 혹은 에이전트 실행 후 응답을 검증합니다
데코레이터	<code>\@before_agent</code> , <code>\@after_agent</code> 데코레이터로 간결하게 정의합니다
다중 조합	여러 가드레일을 <code>middleware</code> 리스트에 추가하여 다층 방어를 구성합니다

REFERENCES

- [Guardrails](#)

P A R T

III

LangGraph

Stateful Agent Orchestration

이 파트에서 다루는 내용

- 1. LangGraph 소개
- 2. Graph API 기초
- 3. Functional API 기초
 - 4. 워크플로 패턴
 - 5. 에이전트 구축
 - 6. 지속성과 메모리
 - 7. 스트리밍
- 8. 인터럽트와 타임 트래블
 - 9. 서브그래프
 - 10. 프로덕션
 - 11. 로컬 서버
 - 12. 내구성 실행
- 13. API 선택 가이드와 Pregel

01 LangGraph 소개

상태 기반 에이전트 오케스트레이션 프레임워크

Part 2에서 `LangChain`의 구성 요소를 익혔다면, 이제 그 위에서 에이전트를 설계하고 실행하는 오케스트레이션 계층으로 넘어갈 차례입니다. `LangChain`의 체인(`LCEL`)은 입력을 받아 출력을 내는 단방향 파이프라인에 최적화되어 있지만, 에이전트에게는 조건 분기, 반복, 병렬 실행, 그리고 중간 상태의 저장과 복원이 필요합니다. 단순한 체인으로는 “도구 호출 결과를 보고 다시 판단하는” 루프나, “사람의 승인을 기다리며 실행을 멈추는” 패턴을 자연스럽게 표현하기 어렵습니다.

`LangGraph`는 이 문제를 상태 그래프(State Graph)라는 추상화로 해결합니다. 상태(state)를 중심으로 노드와 엣지를 연결하여 복잡한 에이전트 워크플로를 선언적으로 표현하는 프레임워크입니다. 내부적으로는 Google의 Pregel 알고리즘에서 영감을 받은 메시지 패싱 기반 실행 모델을 사용하며, 각 노드가 슈퍼스텝(super-step) 단위로 활성화되어 상태를 읽고 쓰는 구조입니다. 이 장에서는 `LangGraph`가 왜 필요한지, 기존 체인 방식과 무엇이 다른지, 그리고 `Graph API`와 `Functional API`라는 두 가지 핵심 인터페이스의 전체 그림을 먼저 살펴봅니다.

학습 목표

`LangGraph`의 핵심 개념과 두 가지 API(`Graph API`, `Functional API`)를 이해합니다.

1.1 LangGraph란?

`LangGraph`는 `LangChain` 생태계의 저수준 오케스트레이션 프레임워크입니다. “저수준”이라 함은 개발자가 노드, 엣지, 상태를 직접 정의하여 실행 흐름을 완전히 제어할 수 있다는 뜻입니다. `Deep Agents`와 같은 고수준 프레임워크가 사전 구축된 에이전트를 제공하는 것과 달리, `LangGraph`는 에이전트의 내부 구조 자체를 설계하는 도구입니다.

LangChain 3계층 구조

계층	역할	설명
Deep Agents	고수준	사전 구축된 에이전트 시스템
LangChain	에이전트	LLM 에이전트 구축 도구
LangGraph	워크플로	상태 기반 오케스트레이션

핵심 특징

- 상태 관리: `TypedDict` 또는 `Pydantic` 모델로 상태를 정의하고, 리듀서(reducer)로 값 병합 전략을 지정합니다
- 지속성: 체크포인트를 통한 상태 자동 저장 – 매 슈퍼스텝마다 스냅샷이 생성됩니다

- 스트리밍: `values`, `updates`, `messages`, `custom`, `debug` 등 5가지 모드의 실시간 출력
- Human-in-the-loop: `interrupt()` 함수로 실행을 일시 정지하고, `Command(resume=...)` 으로 사람의 입력을 받아 재개합니다
- 내구성 실행: 장애 발생 시 마지막 체크포인트에서 자동 복구합니다

1.2 핵심 개념

LangGraph의 핵심 특징들을 살펴보았으니, 이제 이 프레임워크를 구성하는 세 가지 기본 요소(primitive)를 자세히 알아봅시다. LangGraph는 그래프 구조를 기반으로 워크플로를 정의하며, 모든 것은 **State**, **Node**, **Edge**라는 세 가지 기본 요소로 구성됩니다.

구성 요소

개념	설명
노드(Node)	처리 단위 – Python 함수로 정의
엣지(Edge)	노드 간 연결, 조건부 분기 가능
상태(State)	TypedDict로 정의, 노드 간 공유 데이터
체크포인트(Checkpointer)	각 단계 상태 자동 저장

LangGraph의 내부 실행 엔진은 Pregel 모델을 따릅니다. Pregel에서 각 노드는 메시지(상태)를 읽고, 처리 후 결과를 채널에 쓰는 액터(actor)입니다. 하나의 슈퍼스텝(super-step)에서 동일 레벨의 노드들이 병렬로 실행되고, 모든 노드가 완료되면 리듀서를 통해 상태가 병합된 뒤 다음 슈퍼스텝으로 넘어갑니다. 이 메시지 패싱 아키텍처 덕분에 LangGraph는 복잡한 분기와 병렬 처리를 효율적으로 수행할 수 있습니다.

TIP

Pregel 런타임은 개발자가 직접 다룰 일이 거의 없습니다. Graph API와 Functional API가 이를 추상화하므로, 여기서는 “슈퍼스텝 단위로 실행된다”는 멘탈 모델만 갖추면 충분합니다. 심화 내용은 13장에서 다룹니다.

그래프 구조 다이어그램

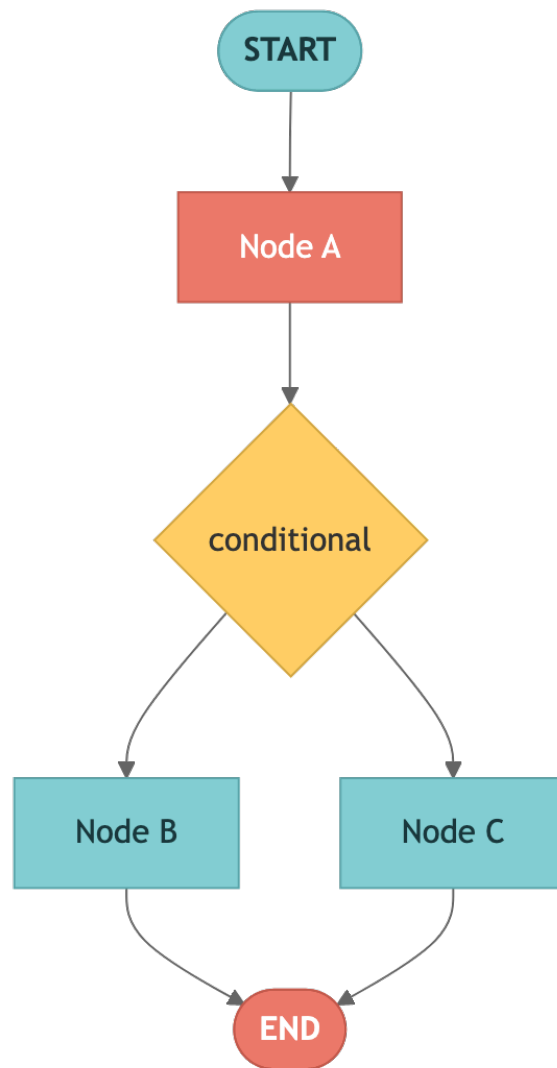


그림 읽는 법

그래프는 **START** → **처리 노드** → **조건 분기** → **END** 순서로 읽으면 됩니다. 핵심은 상태가 노드 사이를 흐르면서, 분기 조건에 따라 다음 노드가 달라진다는 점입니다.

1.3 두 가지 API

그래프 구조를 이해했으니, 이 구조를 어떤 방식으로 코드로 표현할지 살펴봅시다. LangGraph는 동일한 Pregel 런타임 위에서 동작하는 두 가지 프로그래밍 인터페이스를 제공합니다. 하나의 애플리케이션에서 두 API를 혼용할 수도 있으므로, 각각의 특성을 이해하고 상황에 맞게 선택하는 것이 중요합니다.

특성	Graph API	Functional API
----	-----------	----------------

접근 방식	선언적 (노드+엣지)	명령적 (Python 제어 흐름)
상태 관리	명시적 State + 리듀서	함수 스코프, 리듀서 불필요
시각화	그래프 시각화 지원	미지원
체크포인팅	슈퍼스텝마다 새 체크포인트	태스크별, 기존 체크포인트에 저장
적합 상황	복잡한 워크플로, 팀 개발	기존 코드 마이그레이션, 간단한 흐름

· NOTE

두 API의 가장 큰 차이는 체크포인팅 단위입니다. Graph API는 매 슈퍼스텝마다 새로운 체크포인트를 생성하는 반면, Functional API는 `\@task` 단위로 기존 체크포인트에 결과를 저장합니다. 이 차이가 장애 복구와 타임트래블에서 미묘한 동작 차이를 만들어냅니다.

1.4 환경 설정 및 설치 확인

이론적인 개념을 파악했으니, 실제 코드를 실행할 환경을 준비합니다. 필요한 패키지가 올바르게 설치되어 있는지 확인합니다.

```
import importlib

packages = {
    "langgraph": "langgraph",
    "langchain": "langchain",
    "langchain_openai": "langchain-openai",
}

print("=" * 50)
print("LangGraph 환경 확인")
print("=" * 50)

for module_name, package_name in packages.items():
    try:
        mod = importlib.import_module(module_name)
        version = getattr(mod, "__version__", "installed")
        print(f" OK {package_name}: {version}")
    except ImportError:
        print(f" ERR {package_name}: 설치되지 않음")
```

```
=====
LangGraph 환경 확인
=====
OK langgraph: installed
OK langchain: 1.2.10
OK langchain-openai: installed
```

```
from dotenv import load_dotenv
import os
load_dotenv(override=True)

required = ["OPENAI_API_KEY"]
optional = ["TAVILY_API_KEY", "LANGSMITH_API_KEY"]
```

```
print("API 키 상태:")
for key in required:
    print(f" {'OK' if os.environ.get(key) else 'MISSING'} {key} (필수)")
for key in optional:
    print(f" {'OK' if os.environ.get(key) else '--'} {key} (선택)")
```

```
API 키 상태:
OK OPENAI_API_KEY (필수)
OK TAVILY_API_KEY (선택)
-- LANGSMITH_API_KEY (선택)
```

```
# Core import verification
from langgraph.graph import StateGraph, START, END
from langgraph.func import entrypoint, task
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.types import Command, interrupt
from langchain.tools import tool
from langchain.messages import HumanMessage, SystemMessage, AIMessage
from langchain_openai import ChatOpenAI

print("모든 핵심 임포트 완료")
```

```
모든 핵심 임포트 완료
```

1.5 Graph API 맛보기

환경이 준비되었으니, 두 API를 간단한 예제로 체험해 봅시다. 먼저 Graph API입니다. Graph API는 선언적 방식으로 워크플로를 정의합니다. 개발자는 상태 스키마를 `TypedDict` 로 선언하고, 노드(Python 함수)를 등록한 뒤, 엣지로 노드 간 흐름을 연결합니다. LangGraph가 지원하는 엣지 유형은 네 가지입니다:

- **Normal Edge** — `add_edge(A, B)` : A 실행 후 항상 B로 이동
- **Conditional Edge** — `add_conditional_edges(A, routing_fn)` : 라우팅 함수의 반환값에 따라 분기
- **Entry Edge** — `add_edge(START, A)` : 그래프의 시작점 지정
- **Parallel Edge** — 하나의 노드에서 여러 노드로 동시 연결 시 병렬 실행

기본적인 Graph API 사용 흐름은 다음 다섯 단계입니다:

1. `StateGraph(State)` — 상태 스키마로 그래프 빌더 생성
2. `add_node()` — 노드(함수) 등록
3. `add_edge()` — 노드 간 연결
4. `compile()` — 실행 가능한 그래프 생성
5. `invoke()` — 그래프 실행

1.6 Functional API 맛보기

Graph API가 노드와 엣지를 명시적으로 등록하는 방식이었다면, Functional API는 일반 Python 함수 위에 데코레이터를 붙이는 것만으로 동일한 기능을 구현합니다. 그래프 구조를 선언할 필요가 없으므로 보일러플레이트가 크게 줄어들며, 기존 Python 코드를 LangGraph로 마이그레이션할 때 특히 유용합니다.

- `@task` — 단위 작업 정의 (체크포인트링 단위). 호출하면 `Future` 와 유사한 객체를 반환하며, `.result()` 로 결과를 대기합니다. 여러 태스크를 먼저 호출한 뒤 `.result()` 를 한꺼번에 호출하면 동시 실행이 가능합니다.
- `@entrypoint` — 워크플로 진입점 정의. 체크포인트를 연결하면 자동으로 상태가 저장됩니다.
- 일반 Python 제어 흐름(`if` , `for` , `while` 등)을 그대로 사용

! WARNING

Functional API에서 `\@task` 밖의 코드는 체크포인트에 저장되지 않습니다. 비결정적 연산(API 호출, 난수 생성 등)은 반드시 `\@task` 로 감싸야 재개 시 동일한 결과를 보장할 수 있습니다. 이 주제는 12장 “내구성 실행”에서 자세히 다룹니다.

이 장에서 LangGraph의 전체 그림을 파악했습니다. 세 가지 기본 요소(State, Node, Edge), Pregel 기반 실행 모델, 그리고 Graph API와 Functional API라는 두 가지 프로그래밍 인터페이스가 LangGraph를 이해하는 출발점입니다.

요약

항목	설명
LangGraph	상태 기반 에이전트 오케스트레이션 프레임워크
Graph API	<code>StateGraph</code> 로 명시적 상태 흐름 정의
Functional API	<code>\@entrypoint</code> + <code>\@task</code> 로 함수형 워크플로
핵심 개념	State (상태), Node (노드), Edge (엣지)
체크포인트	상태 지속성, 멀티턴 대화, 타임 트래블 지원

⇒ NEXT STEP

다음 장에서는 Graph API의 핵심인 `StateGraph` 를 본격적으로 다룹니다. 상태 리듀서, 조건부 엣지, `MessagesState` , 입출력 스키마 분리 등 Graph API로 워크플로를 구성하는 모든 기초를 단계별로 익힙니다.

02 Graph API 기초

StateGraph로 워크플로 만들기

Graph API 는 LangGraph 의 대표적인 프로그래밍 모델로, StateGraph 에 노드와 엣지를 명시적으로 등록하여 워크플로의 흐름을 시각적으로 설계합니다. 1장에서 소개한 세 가지 기본 요소 – State, Node, Edge – 를 코드로 조합하는 실천 방법을 이 장에서 본격적으로 다룹니다. 상태 스키마를 TypedDict 로 정의하고, 리듀서로 값 병합 전략을 지정하며, 조건부 엣지로 분기를 제어하는 패턴은 이후 모든 장의 기초가 됩니다. 이 장에서 StateGraph 의 생성부터 컴파일, 실행까지의 전체 라이프사이클을 단계별로 익혀 봅니다.

학습 목표

1. StateGraph 빌더의 생성-컴파일-실행 라이프사이클을 단계별로 설명할 수 있다.
2. Annotated 타입 힌트와 리듀서(operator.add , add_messages)로 상태 병합 전략을 지정할 수 있다.
3. 조건부 엣지(add_conditional_edges)와 Command 객체를 사용하여 동적 분기를 구현할 수 있다.
4. MessagesState 를 활용하여 LLM 대화 기반 에이전트 상태를 정의할 수 있다.
5. 입출력 스키마를 분리하여 그래프의 공개 인터페이스를 설계할 수 있다.

2.1 환경 설정

이 장의 모든 예제는 OpenAI의 gpt-5.4 모델을 사용합니다. load_dotenv() 는 프로젝트 루트의 .env 파일에서 OPENAI_API_KEY 를 읽어 환경 변수로 등록합니다. override=True 를 설정하면 이미 시스템에 설정된 환경 변수가 있더라도 .env 파일의 값이 우선 적용되므로, 프로젝트별로 다른 API 키를 사용할 때 유용합니다.

```
from dotenv import load_dotenv
load_dotenv(override=True)
from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
```

2.2 StateGraph 기본 구조

환경 설정을 마쳤으니, 이제 LangGraph의 핵심 빌더 클래스인 StateGraph 를 살펴봅시다.

StateGraph 는 LangGraph의 그래프 빌더 클래스입니다. 상태 스키마를 인자로 받아 그래프 인스턴스를 생성하고, 여기에 노드와 엣지를 등록한 뒤 compile() 로 실행 가능한 CompiledGraph 객체를 만듭니다. 이 객체는 LangChain의 Runnable 인터페이스를 구현하므로 invoke() , stream() , ainvoke() 등의 메서드를 모두 지원합니다. 내부적으로 LangGraph는 Google의 Pregel 모델에서 영감을 받은 메시지 패싱 방식으로 실행됩니다. 각 노드는 비활성 상태에서 시작하여 인접 엣지로부터 메시지를 받으면 활성화되고, 작업을 수행한 뒤 다시 비활성으로 전환됩니다. 이 병렬 실행의 한 라운드를 슈퍼스텝(super-step)이라 부릅니다.

StateGraph를 사용하는 기본 흐름은 다섯 단계로 구성됩니다:

1. `StateGraph(State)` — 상태 스키마(`TypedDict`)로 그래프 빌더를 생성합니다. 이 스키마가 그래프 전체에서 공유되는 데이터 구조를 정의합니다.
2. `add_node(name, fn)` — Python 함수를 노드로 등록합니다. 함수 이름을 생략하면 함수 객체의 `__name__` 이 자동으로 사용됩니다.
3. `add_edge(src, dst)` — 두 노드 사이에 고정 연결을 추가합니다. `START` 와 `END` 는 각각 그래프의 진입점과 종료점을 나타내는 가상 노드입니다.
4. `compile()` — 등록된 노드와 엣지를 검증하고 실행 가능한 `CompiledGraph` 를 생성합니다. 이 단계에서 체크 포인트나 인터럽트 설정도 함께 지정할 수 있습니다.
5. `invoke(input)` — 초기 상태를 전달하여 그래프를 실행합니다. 모든 노드가 비활성이 되면 실행이 종료됩니다.

`StateGraph(State) → add_node() → add_edge() → compile() → invoke()`

아래 코드는 이 다섯 단계를 실제로 구현한 가장 단순한 예제입니다. `topic` 필드를 받아 에세이를 생성하는 단일 노드 그래프입니다. 실행 결과에서 `content` 필드에 LLM이 생성한 값이 들어 있는지 확인해 보세요.

```
from typing import TypedDict
from langgraph.graph import StateGraph, START, END

class MyState(TypedDict):
    topic: str
    content: str | None

def write(state: MyState):
    return {"content": f"Essay about {state['topic']}"}

builder = StateGraph(MyState)
builder.add_node("write", write)
builder.add_edge(START, "write")
builder.add_edge("write", END)

graph = builder.compile()
result = graph.invoke({"topic": "AI"})
print(result)
```

➡ TIP

`compile()` 이후에는 노드나 엣지를 추가할 수 없습니다. 그래프 구조를 변경하려면 빌더(`StateGraph`)부터 다시 시작해야 합니다. 이는 실행 시점의 그래프가 불변(immutable)임을 보장하기 위한 설계입니다.

2.3 상태 리듀서

그래프의 기본 골격을 세웠으니, 이제 상태가 어떻게 업데이트되는지 제어하는 리듀서를 살펴봅시다. 여러 노드가 같은 상태 필드에 값을 쓸 때, 기본 동작은 단순 덮어쓰기(override)입니다. 그러나 메시지 히스토리처럼 값을 누적해야 하는 경우가 많으므로, `Annotated` 타입 힌트로 각 필드의 업데이트 전략을 명시합니다. 리듀서는 LangGraph 상태 관리의 핵심 메커니즘이며, 이를 올바르게 이해하지 못하면 병렬 노드 실행 시 데이터가 유실되는 문제가 발생할 수 있습니다.

리듀서란?

리듀서(reducer)는 함수형 프로그래밍의 `reduce` 개념에서 가져온 것으로, 기존 상태값과 새로운 업데이트를 인자로 받아 최종 상태값을 반환하는 함수입니다. `TypedDict`의 각 필드에 `Annotated` 타입 힌트를 사용하여 리듀서를 지정하면, 해당 필드에 값이 기록될 때마다 지정된 리듀서 함수가 호출됩니다. 리듀서가 지정되지 않은 필드는 가장 마지막에 기록된 값이 그대로 유지됩니다(last-write-wins).

- 리듀서 없음: 단순 덮어쓰기(override) – 마지막으로 쓴 노드의 값이 유지됩니다
- `operator.add`: 리스트끼리 연결(concatenate)하거나 숫자를 합산합니다
- `add_messages`: 메시지 리스트 전용 리듀서 – 동일 ID의 메시지는 덮어쓰고(upsert), 새 메시지는 추가합니다. 딕셔너리나 튜플 형태의 메시지를 자동으로 `LangChain Message` 객체로 역직렬화하는 기능도 포함되어 있습니다.
- 커스텀 함수: `def my_reducer(current, update): ...` 형태로 직접 정의할 수 있습니다

아래 코드에서 `messages` 필드에는 `operator.add` 리듀서가 적용되어 있어, 각 노드가 반환하는 메시지가 기존 리스트에 누적됩니다. 반면 `count` 필드에는 리듀서가 없으므로 마지막으로 기록한 값이 유지됩니다. 두 필드의 동작 차이를 주의 깊게 관찰해 보세요.

```
from typing import TypedDict, Annotated
import operator

class MyState(TypedDict):
    messages: Annotated[list, operator.add] # 리듀서: 누적
    count: int # 리듀서 없음: 덮어쓰기
```

TIP

`Annotated[list[AnyMessage], add_messages]`는 `LangGraph`에서 가장 자주 사용되는 리듀서 패턴입니다. `add_messages`는 단순 append가 아니라 메시지 ID를 기준으로 upsert 동작을 수행하므로, 메시지 수정이나 삭제(`RemoveMessage`)도 자연스럽게 처리됩니다. `RemoveMessage(id=msg.id)` 형태로 메시지를 삭제하거나, `RemoveMessage(id=REMOVE_ALL_MESSAGES)`로 전체 히스토리를 초기화할 수도 있습니다.

! WARNING

병렬로 실행되는 두 노드가 리듀서 없이 같은 필드에 값을 쓰면, 어떤 노드의 값이 최종적으로 남을지 예측할 수 없습니다. 병렬 실행이 가능한 필드에는 반드시 리듀서를 지정하세요.

2.4 조건부 엣지

리듀서로 상태 업데이트 전략을 정의했다면, 이제 실행 흐름을 동적으로 제어하는 방법을 알아보시다. 고정 엣지(`add_edge`)만으로는 항상 같은 경로를 따르지만, 실제 에이전트는 LLM의 판단이나 상태 값에 따라 다른 노드로 분기해야 하는 경우가 대부분입니다. 조건부 엣지는 이러한 동적 분기를 가능하게 하는 메커니즘입니다.

add_conditional_edges

`add_conditional_edges(source, routing_function)` 메서드는 `source` 노드 실행 후 `routing_function` 을 호출하여 다음 노드를 결정합니다. 라우팅 함수는 현재 상태(`State`)를 인자로 받고, 다음에 실행할 노드의 이름을 문자열로 반환합니다. 반환값이 `END` 이면 그래프 실행이 종료됩니다.

- `add_conditional_edges(source, routing_function)` - 라우팅 함수의 반환값이 다음 노드 이름
- 라우팅 함수에 `Literal["node_a", "node_b"]` 반환 타입 힌트를 추가하면, 그래프 시각화 (`draw_mermaid_png()`) 시 가능한 분기 경로가 자동으로 표시됩니다
- 선택적으로 `mapping` 딕셔너리를 세 번째 인자로 전달하여, 라우팅 함수의 반환값을 실제 노드 이름에 매핑할 수 있습니다

다음 예제는 라우팅 함수를 사용한 조건부 분기의 전형적인 패턴입니다. `classify` 노드가 질문을 분류한 뒤, `route` 함수가 분류 결과에 따라 적절한 처리 노드로 분기합니다.

```
from typing import Literal

def route(state) -> Literal["weather", "math", "general"]:
    return state["classification"]

builder.add_conditional_edges("classify", route)
```

Command — 상태 업데이트 + 라우팅 통합

LangGraph 0.2.x부터는 `Command` 객체를 사용하여 상태 업데이트와 라우팅을 하나의 반환값으로 통합할 수 있습니다. `Command(update={"field": value}, goto="next_node")` 와 같이 작성하면 노드 함수 안에서 다음 노드를 직접 지정할 수 있어, 별도의 라우팅 함수 없이도 분기가 가능합니다. `Command` 는 `update` 로 상태를 수정하면서 동시에 `goto` 로 다음 노드를 지정하므로, 분기 로직과 상태 업데이트가 하나의 반환문에 응집됩니다.

```
from langgraph.types import Command

def classify_node(state):
    result = classify(state["messages"][-1].content)
    return Command(
        update={"classification": result},
        goto="handle_urgent" if result == "urgent" else "handle_normal",
    )
```

! WARNING

`Command` 객체를 사용하면 `add_conditional_edges()` 없이도 분기를 구현할 수 있지만, 시각화 다이어그램에 분기 경로가 자동으로 표시되지 않을 수 있습니다. 복잡한 워크플로에서는 가독성을 위해 명시적 조건부 엣지를 선호하는 경우가 많습니다. `Command` 는 서브그래프에서 부모 그래프로 라우팅하거나(`graph=Command.PARENT`), 인터럽트 후 재개(`resume=`)할 때도 활용됩니다.

전체 라우팅 흐름을 도식으로 나타내면 다음과 같습니다:

```
START → classify → [route] → weather → END
                        → math → END
                        → general → END
```


2.5 메시지 기반 상태

조건부 옛지까지 익혔으니, 이제 LLM 에이전트에서 가장 자주 사용되는 상태 패턴인 `MessagesState` 를 살펴봅시다. 채팅 기반 에이전트는 대화 히스토리를 핵심 상태로 사용하기 때문에, LangGraph는 이를 위한 편의 클래스를 제공합니다. 매번 `messages: Annotated[list[AnyMessage], add_messages]` 를 직접 선언하는 대신,

`MessagesState` 를 상속하면 이 보일러플레이트를 생략할 수 있습니다.

- `MessagesState` 는 `messages: Annotated[list[AnyMessage], add_messages]` 를 포함하는 사전 정의된 상태 클래스입니다
- `add_messages` 리듀서가 메시지 리스트를 자동으로 누적하며, 동일 ID 메시지는 교체(upsert)됩니다
- LLM 응답(`AIMessage`)을 반환하면 메시지 히스토리에 자연스럽게 추가됩니다
- 추가 필드가 필요하면 `MessagesState` 를 상속하여 확장할 수 있습니다

아래 코드는 `MessagesState` 를 상속하여 커스텀 필드 `language` 를 추가한 예제입니다. `messages` 필드는 이미 `add_messages` 리듀서가 적용되어 있으므로 별도로 선언할 필요가 없습니다.

```
from langgraph.graph import MessagesState

class AgentState(MessagesState):
    language: str # 추가 필드
```

TIP

`MessagesState` 를 사용할 때 노드 함수에서 `{"messages": [ai_response]}` 를 반환하면, `add_messages` 리듀서가 자동으로 기존 메시지 리스트에 새 메시지를 추가합니다. 리스트 전체를 교체하는 것이 아니라 병합하는 것이므로, 이전 대화 내역이 유지됩니다.

2.6 입출력 스키마

`MessagesState` 로 대화 기반 상태를 정의하는 법을 배웠으니, 마지막으로 그래프의 외부 인터페이스를 설계하는 방법을 살펴봅시다.

실제 프로덕션 에이전트에서는 내부에서만 사용하는 중간 데이터(예: 분류 결과, 중간 점수, 디버그 정보)를 외부에 노출하지 않아야 합니다. 입출력 스키마를 분리하면 그래프의 공개 인터페이스를 깔끔하게 유지할 수 있고, API 사용자는 필요한 데이터만 주고받게 됩니다. 이는 소프트웨어 설계에서 캡슐화(encapsulation) 원칙에 해당합니다.

- `StateGraph(InternalState, input=InputSchema, output=OutputSchema)` — 세 가지 스키마를 한 번에 지정합니다
- 입력 스키마: 외부에서 받는 데이터만 포함합니다. `invoke()` 호출 시 이 스키마에 맞는 데이터만 전달할 수 있습니다.
- 출력 스키마: 외부로 내보내는 데이터만 포함합니다. `invoke()` 의 반환값이 이 스키마로 필터링됩니다.
- 내부 상태: 중간 처리용 필드를 포함하며, 외부에는 노출되지 않습니다

아래 코드는 입출력 스키마 분리의 전형적인 패턴입니다. `intermediate` 필드는 내부 처리에만 사용되며, 호출자는 `question` 만 전달하고 `answer` 만 받습니다.

```

class InputSchema(TypedDict):
    question: str

class OutputSchema(TypedDict):
    answer: str

class FullState(TypedDict):
    question: str
    answer: str
    intermediate: str # 내부 전용

builder = StateGraph(
    FullState,
    input=InputSchema,
    output=OutputSchema,
)

```

➡ TIP

입출력 스키마를 분리하면 API 문서 자동 생성, 타입 검증, 그리고 LangGraph Platform 배포 시 Swagger 문서 생성이 훨씬 깔끔해집니다. 또한 그래프를 서브그래프로 재사용할 때 인터페이스 계약이 명확해지므로, 팀 간 협업에서도 큰 도움이 됩니다.

2.7 add_node() 고급 옵션

기본 골격을 익혔으니, 프로덕션 환경에서 자주 쓰이는 `add_node()` 의 고급 파라미터들을 살펴봅시다. 단순히 함수만 등록하는 것을 넘어, 재시도, 타임아웃, 캐싱, 에러 복구, 실행 지연까지 노드 단위로 세밀하게 제어할 수 있습니다.

파라미터	설명
<code>retry_policy</code>	<code>RetryPolicy(max_attempts=..., retry_on=...)</code> — 실패 시 자동 재시도. 기본 정책은 일반 네트워크 에러 위주이며, <code>ValueError</code> / <code>TypeError</code> / <code>RuntimeError</code> 는 제외
<code>timeout</code>	<code>TimeoutPolicy(run_timeout=..., idle_timeout=...)</code> — 초과 시 <code>NodeTimeoutError</code> 발생. <code>async</code> 노드 전용, attempt별 독립 적용
<code>cache_policy</code>	<code>CachePolicy(ttl=..., key_func=...)</code> — 입력 해시 기반 결과 캐싱. 컴파일 시 <code>InMemoryCache</code> 또는 <code>SqliteCache</code> 필요
<code>error_handler</code>	<code>NodeError</code> 를 인자로 받아 복구용 <code>Command</code> 를 반환하는 콜백 — 폴백 노드로 우회하거나 상태에 에러 정보를 기록
<code>defer</code>	<code>True</code> 로 지정하면 다른 분기의 pending 태스크가 모두 완료된 뒤에 실행 — 분기 길이가 다른 fan-out/fan-in에 유용

```

from langgraph.cache.memory import InMemoryCache
from langgraph.types import RetryPolicy, CachePolicy

builder.add_node(
    "expensive_node",
    expensive_node,

```

```

    retry_policy=RetryPolicy(max_attempts=3, retry_on=ValueError),
    cache_policy=CachePolicy(ttl=3),
)
graph = builder.compile(cache=InMemoryCache())

```

TIP

cache_policy 는 @task 의 cache_policy 와 동일한 객체이며, Graph API에서도 같은 방식으로 컴파일 시 cache= 를 지정해야 동작합니다. retry_policy 의 retry_on 을 좁히지 않으면 ValueError 처럼 코드 버그성 예외도 그대로 재시도되어 비용이 발생할 수 있으므로, 대상 예외를 명시하는 편이 안전합니다.

2.8 Runtime 컨텍스트와 context_schema

상태(State)에는 노드 간에 흐르는 데이터를 담지만, LLM 프로바이더나 사용자 ID처럼 호출 시점에 주입하되 상태를 오염시키지 않는 런타임 의존성이 필요한 경우가 많습니다. LangGraph는 context_schema + Runtime[ContextSchema] 패턴으로 이를 노출합니다. 상태와 런타임 컨텍스트가 분리되므로 체크포인트에 불필요한 자격 증명이나 핸들이 함께 저장되지 않습니다.

- StateGraph(State, context_schema=ContextSchema) — 컨텍스트 스키마를 그래프에 선언합니다
- 노드 시그니처에 runtime: Runtime[ContextSchema] 를 추가하면 runtime.context 로 호출 시 전달된 값에 접근합니다
- graph.invoke(inputs, context={...}) — context 키워드 인자로 컨텍스트를 주입합니다
- runtime.execution_info 로 attempt , thread_id , run_id , 체크포인트 정보에 접근할 수 있어 재시도 횟수에 따라 동작을 분기할 수 있습니다
- runtime.server_info 는 LangGraph Server에서 assistant ID, graph ID, 인증된 사용자 정보를 제공합니다
- runtime.drain_requested 는 graceful shutdown 신호를 노드에 전달합니다

```

from dataclasses import dataclass
from langgraph.runtime import Runtime

@dataclass
class ContextSchema:
    llm_provider: str = "openai"
    user_id: str | None = None

def node(state: State, runtime: Runtime[ContextSchema]):
    if runtime.execution_info.attempt > 1:
        # 첫 시도가 실패한 경우 폴백 로직
        ...
    print("thread:", runtime.execution_info.thread_id)
    print("run:", runtime.execution_info.run_id)
    return {"results": "processed"}

graph = StateGraph(State, context_schema=ContextSchema).compile()
graph.invoke(..., context={"llm_provider": "anthropic", "user_id": "u-1"})

```

2.9 Send와 Command.PARENT — 동적 팬아웃과 서브그래프 라우팅

`Command` 객체를 한 번 더 확장해 봅시다. 라우팅과 상태 업데이트의 조합 외에도, 두 가지 고급 사용처가 있습니다.

`Send("worker", {...})` 는 라우팅 함수가 하나의 노드 이름 대신 `Send` 객체 리스트를 반환하도록 하여, 컴파일 시점이 아닌 실행 시점에 워커 인스턴스를 동적으로 생성합니다. 각 `Send` 는 워커 노드에 독립적인 상태를 주입하므로, 동일한 노드 함수가 서로 다른 데이터를 동시에 처리할 수 있습니다. 이것이 Map-Reduce 패턴의 Map 단계이며, 4장에서 자세히 다룹니다.

```
from langgraph.types import Send

def fan_out(state):
    # 런타임에 항목 수만큼 워커 생성
    return [Send("worker", {"task": t}) for t in state["tasks"]]

builder.add_conditional_edges("planner", fan_out)
```

서브그래프 안의 노드에서 `Command(graph=Command.PARENT, goto="next", update={...})` 를 반환하면 부모 그래프의 노드로 라우팅됩니다. 서브그래프가 자신의 종료 조건을 판단하고 부모 워크플로의 다음 단계를 직접 지정하는 패턴에 유용합니다.

```
from langgraph.types import Command

def sub_node(state):
    return Command(
        graph=Command.PARENT,
        goto="parent_aggregator",
        update={"sub_result": state["value"]},
    )
```

➡ TIP

`Command.PARENT` 는 9장의 서브그래프와 결합할 때 본격적으로 활용됩니다. 여기서는 “서브그래프에서 부모로 직접 라우팅이 가능하다”는 사실만 기억해 두면 됩니다.

이 장에서 Graph API의 핵심 구성 요소를 모두 다루었습니다. `StateGraph` 빌더의 생성-컴파일-실행 라이프사이클, `Annotated` 타입 힌트를 활용한 리듀서 기반 상태 병합, `add_conditional_edges` 와 `Command` 를 통한 동적 분기, `MessagesState` 를 활용한 LLM 대화 패턴, 입출력 스키마를 통한 인터페이스 분리, `add_node()` 의 고급 옵션, `Runtime` 컨텍스트 주입, 그리고 `Send` / `Command.PARENT` 까지 — 이 모든 것이 이후 장에서 반복적으로 사용되는 기초 패턴입니다. 특히 리듀서와 조건부 엣지는 에이전트의 상태 관리와 의사결정 로직의 근간이므로, 다음 장으로 넘어가기 전에 충분히 익혀 두시기 바랍니다.

요약

이번 장에서 배운 내용을 정리합니다.

개념	설명
----	----

StateGraph	상태 스키마 기반 그래프 빌더
Node	Python 함수로 정의된 처리 단위
Edge	노드 간 고정 연결 (<code>add_edge</code>)
Conditional Edge	상태 기반 동적 분기 (<code>add_conditional_edges</code>)
Reducer	<code>Annotated</code> + <code>operator.add</code> 로 상태 누적 방식 정의
MessagesState	LLM 대화용 사전 정의된 상태
Input/Output Schema	내부 상태와 외부 입출력 분리

⇒ NEXT STEP

다음 장에서는 동일한 워크플로를 `@entrypoint` 와 `@task` 데코레이터로 작성하는 Functional API를 다룹니다. Graph API와의 차이를 직접 비교하며, 각 API가 적합한 상황을 감각적으로 이해하게 됩니다.

03 Functional API

@entrypoint와 @task로 워크플로 만들기

Graph API 가 노드-엣지 그래프를 명시적으로 구성하는 방식이라면, Functional API 는 일반 Python 함수에 @entrypoint 와 @task 데코레이터를 붙여 워크플로를 작성하는 보다 직관적인 접근입니다. 2장에서 StateGraph , 리듀서, 조건부 엣지를 익혔다면, 이제 동일한 기능을 더 적은 코드로 달성하는 방법을 배울 차례입니다. Functional API에서는 그래프 구조를 선언하지 않아도 체크포인팅과 내구성 실행이 자동으로 적용되며, Python의 if , for , while 등 일반 제어 흐름을 그대로 사용할 수 있습니다. 빠른 프로토타이핑이나 동적 분기가 많은 시나리오에 특히 유용합니다. 이 장에서는 @task 의 Future 패턴과 @entrypoint 의 단기 메모리 관리를 중심으로 Functional API 의 핵심 사용법을 다룹니다.

학습 목표

1. @task 데코레이터의 체크포인팅 동작과 Future 반환 패턴을 설명할 수 있다.
2. @entrypoint 로 워크플로 진입점을 정의하고 체크포인트를 연결할 수 있다.
3. previous 파라미터를 활용하여 멀티턴 대화에서 이전 실행 결과를 유지할 수 있다.
4. entrypoint.final(value, save) 로 반환값과 체크포인트 저장값을 분리할 수 있다.
5. 결정론성 요구사항을 이해하고, 비결정적 작업을 올바르게 @task 로 감쌀 수 있다.

3.1 환경 설정

2장과 동일하게 환경 변수를 로드하고 LLM 모델을 초기화합니다. Functional API의 데코레이터는 langgraph.func 모듈에서 임포트합니다.

```
from dotenv import load_dotenv
load_dotenv(override=True)

from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
```

3.2 @task — 체크포인팅 가능한 작업 단위

환경 설정을 마쳤으니, 이제 Functional API의 첫 번째 핵심 데코레이터인 @task 를 살펴봅시다.

@task 데코레이터는 Functional API의 핵심 구성 요소입니다. 함수를 @task 로 감싸면 두 가지 중요한 효과가 발생합니다. 첫째, 해당 함수의 실행 결과가 체크포인트에 저장되어 내구성 실행(durable execution)이 보장됩니다. 장애가 발생하여 워크플로가 재개될 때, 이미 완료된 태스크는 체크포인트에서 결과를 복원하므로 다시 실행되지 않습니다. 둘째, 호출 시 즉시 Future 와 유사한 객체를 반환하여 병렬 실행의 가능성을 열어줍니다. 실제 결과가 필요한 시점에 .result() 를 호출하면 됩니다.

- `@task` 데코레이터로 감싸면 실행 결과가 체크포인트에 자동 저장됩니다
- 호출 시 즉시 `Future` 객체를 반환하며, `.result()` 로 동기 대기할 수 있습니다
- `@task` 로 감싸지 않은 비결정적 연산(API 호출, 난수 생성 등)은 재개 시 다시 실행됩니다
- Graph API의 노드(node)에 대응하는 개념으로, 체크포인트의 최소 단위입니다

아래 코드에서 `call_llm` 함수는 `@task` 로 감싸져 있으므로, LLM 호출 결과가 체크포인트에 저장됩니다.

`.result()` 가 호출되기 전까지는 실제 실행이 시작되지 않을 수 있다는 점에 주목하세요.

```
from langgraph.func import entrypoint, task

@task
def call_llm(messages):
    return model.invoke(messages)

# 호출 시 Future 반환
future = call_llm(messages)

# 결과가 필요할 때 .result()로 대기
response = future.result()
```

TIP

`\@task` 는 Graph API의 `add_node()` 와 동일한 역할을 합니다. Graph API에서는 노드 경계에서 자동으로 체크포인트링이 이루어지지만, Functional API에서는 `\@task` 로 명시적으로 체크포인트링 경계를 지정해야 합니다. `\@task` 바깥에서 수행된 작업은 체크포인트에 저장되지 않습니다.

3.3 병렬 태스크 실행

`@task` 하나를 사용하는 법을 배웠으니, 이제 여러 태스크를 동시에 실행하는 패턴을 살펴봅시다.

`@task` 의 `Future` 패턴은 병렬 실행을 자연스럽게 가능하게 합니다. 핵심 원리는 간단합니다: 여러 태스크를 먼저 호출하여 `Future` 객체를 모두 받아둔 뒤, `.result()` 를 나중에 일괄 호출하면 태스크들이 동시에 실행됩니다. Graph API에서 하나의 노드에서 여러 노드로 엣지를 연결하는 팬아웃(fan-out) 구조와 동일한 효과를 Python 코드만으로 달성할 수 있습니다. 명시적인 `asyncio` 나 스레딩 코드 없이도 병렬 실행이 이루어진다는 점이 큰 장점입니다.

아래 코드에서 `futures` 리스트를 먼저 구성하고, 이후 리스트 컴프리헨션에서 `.result()` 를 일괄 호출합니다. 만약 `.result()` 를 즉시 호출하면 태스크가 순차적으로 실행되므로, 병렬 실행을 원한다면 반드시 호출과 대기를 분리해야 합니다.

```
@task
def add_one(n: int) -> int:
    return n + 1

@entrypoint(checkpointer=checkpointer)
def parallel(numbers: list[int]) -> list[int]:
    futures = [add_one(n) for n in numbers] # 먼저 모두 호출
    return [f.result() for f in futures]    # 나중에 일괄 대기
```

! WARNING

병렬 실행의 이점을 얻으려면 태스크 호출(Future 생성)과 결과 대기(`.result()`)를 반드시 분리하세요.
`add_one(n).result()` 처럼 즉시 대기하면 순차 실행과 동일합니다.

3.4 previous — 단기 메모리 (이전 실행 결과 접근)

병렬 실행을 익혔으니, 이제 Functional API에서 멀티턴 대화를 구현하는 방법을 알아보시다.

Graph API에서 체크포인트를 사용하면 `MessagesState` 의 `add_messages` 리듀서가 이전 대화 상태를 자동으로 누적합니다. 그러나 Functional API에서는 `TypedDict` 상태와 리듀서를 사용하지 않으므로, 이전 실행 결과에 접근하는 별도의 메커니즘이 필요합니다. `@entrypoint` 의 `previous` 파라미터가 바로 이 역할을 담당합니다. 같은 `thread_id` 로 워크플로를 재호출하면, 직전 실행에서 저장한 값이 `previous` 키워드 인자로 전달됩니다. 최초 호출 시에는 `previous` 가 `None` 이므로, 기본값 처리를 반드시 해야 합니다.

`previous` 는 `thread_id` 기반 단기 메모리입니다. 서로 다른 `thread_id` 는 독립적인 메모리 공간을 가지므로, 사용자별 대화 세션을 자연스럽게 분리할 수 있습니다.

```
from langgraph.checkpoint.memory import InMemorySaver

checkpointer = InMemorySaver()

@entrypoint(checkpointer=checkpointer)
def chat(message: str, *, previous=None) -> str:
    messages = previous or []
    messages.append({"role": "user", "content": message})
    response = call_llm(messages).result()
    messages.append(response)
    return entrypoint.final(
        value=response.content, # 호출자에게 반환
        save=messages,          # 다음 호출의 previous로 전달
    )

# 같은 thread_id로 호출하면 이전 대화가 유지됨
config = {"configurable": {"thread_id": "user-1"}}
chat.invoke("안녕하세요", config=config)
chat.invoke("제 이름이 뭐였죠?", config=config)
```

TIP

`previous` 파라미터 외에도 `@entrypoint` 는 `store` (장기 메모리), `writer` (커스텀 스트리밍), `config` (런타임 설정)를 키워드 인자로 주입받을 수 있습니다. 이들은 모두 자동으로 감지되므로, 함수 시그니처에 해당 이름의 파라미터를 추가하기만 하면 됩니다.

3.5 entrypoint.final — 반환값과 체크포인트 저장값 분리

`previous` 로 이전 실행 결과에 접근하는 법을 배웠는데, 여기서 한 가지 의문이 생깁니다: 호출자에게 반환하는 값과 체크포인트에 저장하는 값이 항상 같아야 할까요? 대부분의 경우 그렇지 않습니다.

예를 들어, 사용자에게는 최종 답변 문자열만 반환하되, 체크포인트에는 전체 메시지 히스토리를 저장하고 싶을 수 있습니다. `entrypoint.final(value, save)` 형태로 반환하면 이 두 가지를 명확하게 분리할 수 있습니다.

`value` 는 `invoke()` 의 반환값이 되고, `save` 는 다음 호출 시 `previous` 파라미터로 전달됩니다.
`entrypoint.final` 을 사용하지 않고 단순히 값을 반환하면, 반환값이 곧 저장값이 됩니다.

```
@entrypoint(checkpointer=checkpointer)
def my_workflow(inputs, *, previous=None):
    summary = summarize(inputs).result()
    full_state = {"summary": summary, "raw": inputs}
    # value: 호출자에게 반환 / save: previous로 저장
    return entrypoint.final(value=summary, save=full_state)
```

이 패턴은 특히 API 응답은 가볍게 유지하면서 내부적으로는 풍부한 상태를 보존해야 하는 경우에 유용합니다. 예를 들어 사용자에게는 요약 텍스트만 반환하되, 체크포인트에는 원본 데이터, 중간 분석 결과, 메시지 히스토리 등을 모두 저장할 수 있습니다.

3.6 결정론성 요구사항

반환값과 저장값을 분리하는 방법까지 배웠으니, Functional API를 안전하게 사용하기 위한 가장 중요한 규칙을 짚고 넘어갑시다.

Functional API에서 가장 주의해야 할 점은 결정론성(determinism) 요구사항입니다. 내구성 실행의 핵심 메커니즘은 다음과 같습니다: 장애가 발생하면 `@entrypoint` 함수가 처음부터 다시 실행되지만, 이미 완료된 `@task` 는 체크포인트에서 결과를 복원하여 재실행하지 않습니다. 이 메커니즘이 올바르게 작동하려면, `@entrypoint` 내부의 `@task` 바깥 코드가 항상 같은 결과를 생성해야 합니다. 만약 `@task` 바깥에 비결정적 코드가 있으면, 재개 시 다른 값이 생성되어 이전 실행과 다른 분기를 탈 수 있습니다.

반드시 `@task` 로 감싸야 하는 작업:

- LLM 호출 (`model.invoke()` , `model.aiinvoke()`)
- 외부 API 요청 (`requests.get()` , `httpx.get()`)
- 난수 생성 (`random.random()` , `uuid.uuid4()`)
- 현재 시각 참조 (`datetime.now()` , `time.time()`)
- 파일 I/O 및 데이터베이스 쿼리

`@task` 바깥에 둘 수 있는 작업:

- 순수 함수 연산 (문자열 조작, 리스트 변환 등)
- 조건 분기 (`if/else`)
- 반복문 (`for` , `while`)

! WARNING

`@entrypoint` 내부에서 `@task` 바깥에 `random.random()` 이나 `datetime.now()` 같은 비결정적 코드를 작성하면, 재개 시 이전과 다른 값이 사용되어 워크플로가 일관되지 않게 동작할 수 있습니다. 이런 연산도 반드시 `@task` 로 감싸세요. 이는 Functional API의 가장 흔한 실수이므로 특히 주의가 필요합니다.

3.7 LLM 에이전트 (Functional API)

결정론성 요구사항까지 이해했으니, 이제 지금까지 배운 모든 것을 결합하여 완전한 ReAct 에이전트를 구현해 봅시다.

지금까지 배운 `@task` 와 `@entrypoint` 를 결합하면, Graph API 없이도 완전한 ReAct 에이전트를 구현할 수 있습니다. 핵심 아이디어는 간단합니다: `while` 루프 안에서 LLM을 호출하고, `tool_calls` 가 있으면 도구를 실행한 뒤 다시 LLM을 호출하는 과정을 반복합니다. `tool_calls` 가 없으면 루프를 종료합니다. 이 패턴은 Graph API에서 `add_conditional_edges` 로 구현하는 ReAct 루프와 동일하지만, Python의 `while` 문으로 더 직관적으로 표현됩니다.

아래 코드에서 두 가지 핵심 포인트를 관찰하세요. 첫째, LLM 호출과 도구 실행이 모두 `@task` 로 감싸져 있어 체크포인팅이 보장됩니다. 둘째, `while llm_result.tool_calls` 조건으로 도구 호출 여부에 따른 분기가 자연스럽게 이루어집니다.

```
@task
def call_llm(messages):
    return model.invoke(messages)

@task
def call_tool(tool_call):
    return tool.invoke(tool_call)

@entrypoint(checkpointer=checkpointer)
def agent(inputs: dict) -> list:
    messages = inputs["messages"]
    llm_result = call_llm(messages).result()
    while llm_result.tool_calls:
        tool_results = [
            call_tool(tc).result()
            for tc in llm_result.tool_calls
        ]
        messages = messages + [llm_result] + tool_results
        llm_result = call_llm(messages).result()
    return messages + [llm_result]
```

Graph API vs. Functional API — 언제 무엇을 선택할까?

두 API는 상호 배타적이지 않으며, 같은 프로젝트 안에서 혼용할 수 있습니다. 컴파일된 그래프를 `@entrypoint` 안에서 `.invoke()` 할 수도 있고, `@entrypoint` 함수를 `StateGraph` 의 노드로 등록할 수도 있습니다.

기준	Graph API	Functional API
워크플로 구조	노드-엣지 그래프 명시	일반 Python 코드
시각화	자동 다이어그램 생성	코드 리딩으로 이해
상태 관리	TypedDict + 리듀서	함수 스코프 변수
적합한 경우	복잡한 분기, 팀 협업	빠른 프로토타이핑, 동적 로직

3.8 @task 재시도와 캐싱

ReAct 루프까지 구현했다면, 이제 프로덕션 환경에서 신뢰성을 끌어올리는 두 가지 옵션을 살펴봅시다. `@task` 는 Graph API의 `add_node()` 와 동일한 정책 객체를 받아 재시도와 캐싱을 데코레이터 한 줄로 지정할 수 있습니다.

`RetryPolicy` 는 실패 시 자동 재시도 정책입니다. 기본 정책은 일반적인 네트워크 에러를 대상으로 하며, `retry_on` 으로 특정 예외만 좁힐 수 있습니다. `max_attempts` 는 최대 시도 횟수입니다.

```
from langgraph.types import RetryPolicy

@task(retry_policy=RetryPolicy(max_attempts=3, retry_on=ValueError))
def fetch_data(url: str) -> dict:
    response = httpx.get(url)
    response.raise_for_status()
    return response.json()
```

`CachePolicy` 는 태스크 입력 해시를 기반으로 결과를 캐싱합니다. `ttl` 은 캐시 유효 시간(초)입니다. 캐싱을 활성화하려면 `@entrypoint(cache=InMemoryCache())` 로 엔트리포인트에 캐시 백엔드를 연결하고, 캐시하려는 각 `@task` 에 `cache_policy` 를 지정합니다 — 두 곳에 분리해서 지정하는 패턴이 핵심입니다.

```
from langgraph.cache.memory import InMemoryCache
from langgraph.types import CachePolicy

@task(cache_policy=CachePolicy(ttl=120))
def slow_compute(x: int) -> int:
    time.sleep(1)
    return x * 2

@entrypoint(cache=InMemoryCache())
def main(inputs: dict) -> dict[str, int]:
    a = slow_compute(inputs["a"]).result()
    b = slow_compute(inputs["b"]).result()
    return {"a": a, "b": b}
```

TIP

`@task(timeout=1.0, retry_policy=RetryPolicy(retry_on=NodeTimeoutError))` 패턴으로 타임아웃과 재시도를 결합할 수도 있습니다. 같은 `thread_id` 로 입력에 `None` 을 전달하면 직전 실패 지점부터 재시도되며, 이미 성공한 태스크의 결과는 체크포인트에서 재사용됩니다.

3.9 구조화된 interrupt와 Command(resume=)

장애 복구뿐 아니라 사람의 검토가 필요한 경우에도 Functional API는 자연스러운 인터페이스를 제공합니다.

`interrupt()` 함수는 워크플로 실행을 일시 중단하고 사람의 입력을 기다립니다. 단순 문자열뿐 아니라 딕셔너리 형태의 구조화된 페이로드를 전달할 수 있어, 호출자에게 검토에 필요한 컨텍스트를 풍부하게 제공할 수 있습니다.

```
from langgraph.types import Command, interrupt

@task
```

```
def review(input_query: str):
    # 구조화된 인터럽트 - 호출자가 풍부한 컨텍스트로 검토
    human_review = interrupt({
        "question": "이 결과를 승인하시겠습니까?",
        "tool_call": {"name": "send_email", "args": {...}},
    })
    return human_review

@entrypoint(checkpointer=checkpointer)
def workflow(input_query: str):
    return review(input_query).result()
```

재개할 때는 `Command(resume=...)` 로 사람의 응답을 전달합니다. `interrupt()` 호출이 마치 정상적으로 그 값을 반환한 것처럼 워크플로가 재개되므로, 호출 측 코드 흐름은 변하지 않습니다.

```
config = {"configurable": {"thread_id": "review-1"}}
workflow.invoke("초안 검토 요청", config=config)
# 사람이 검토 후 재개
workflow.invoke(Command(resume={"approved": True, "comment": "OK"}), config=config)
```

3.10 @entrypoint 주입 파라미터

previous 외에도 `@entrypoint` 는 함수 시그니처의 키워드 인자 이름으로 런타임 의존성을 자동 주입합니다. 함수 시그니처에 해당 이름의 파라미터를 추가하기만 하면 LangGraph가 자동으로 감지합니다.

파라미터	역할
<code>previous</code>	직전 체크포인트의 저장값 (단기 메모리)
<code>store</code>	<code>BaseStore</code> 인스턴스로 장기 메모리 접근
<code>writer</code>	<code>StreamWriter</code> 객체로 커스텀 스트리밍 emit (async + Python < 3.11 호환용)
<code>config</code>	<code>RunnableConfig</code> - <code>thread_id</code> , <code>metadata</code> , <code>configurable</code> 값 접근

```
from langgraph.types import StreamWriter

@entrypoint(checkpointer=checkpointer, store=store)
def workflow(
    inputs: dict,
    *,
    previous=None,
    store=None,
    writer: StreamWriter = None,
    config=None,
):
    writer({"phase": "start"})
    user_id = config["configurable"]["user_id"]
    memory = store.get(("users", user_id), "profile")
    ...
```



TIP

Python 3.11 이상에서는 `langgraph.config.get_stream_writer()` 를 사용해 함수 내 어디서든 stream writer를 가져올 수도 있어, 파라미터 주입 대신 더 간결한 호출이 가능합니다.

이 장에서 Functional API의 핵심 구성 요소를 모두 다루었습니다. `@task` 로 체크포인팅 단위를 정의하고, `@entrypoint` 로 워크플로의 진입점을 만들며, `previous` 와 `entrypoint.final` 로 메모리를 관리하고, `retry_policy` / `cache_policy` 로 신뢰성을 강화하며, 구조화된 `interrupt()` 로 HITL을 구성하는 패턴이 Functional API의 전체 도구 상자입니다. 무엇보다 중요한 것은 결정론성 요구사항으로, 비결정적 작업을 `@task` 로 감싸는 규칙을 항상 준수해야 합니다.

요약

기능	설명
<code>\@task</code>	비동기 작업, 체크포인팅, 병렬 실행
<code>\@entrypoint</code>	워크플로 진입점, 실행 관리
<code>.result()</code>	Future 결과 동기 대기
<code>previous</code>	이전 실행 결과 접근 (단기 메모리)
<code>entrypoint.final</code>	반환값 ≠ 저장값 분리

⇒ NEXT STEP

다음 장에서는 Graph API와 Functional API 모두를 활용하여 다섯 가지 핵심 워크플로 패턴 – Prompt Chaining, Parallelization, Routing, Orchestrator-Worker, Evaluator-Optimizer – 을 구현합니다. 두 API의 차이를 실전 패턴으로 체감하게 됩니다.

04 워크플로 패턴

5가지 핵심 패턴

2장과 3장에서 Graph API와 Functional API의 기초를 각각 익혔다면, 이제 이 도구들로 실전 워크플로를 구성할 차례입니다. 실전에서 에이전트 워크플로는 대부분 소수의 반복되는 구조적 패턴으로 분류할 수 있습니다.

Anthropic이 제시한 다섯 가지 핵심 패턴 — `Prompt Chaining`, `Parallelization`, `Routing`, `Orchestrator-Worker`, `Evaluator-Optimizer` — 은 복잡한 에이전트 시스템을 설계할 때 출발점이 됩니다. 이 패턴들은 LangGraph의 그래프 구조에 직접 대응됩니다: 순차 패턴은 일직선 엣지, 병렬 패턴은 팬아웃/팬인 구조, 조건부 패턴은 조건부 엣지, 반복 패턴은 순환 그래프로 자연스럽게 표현됩니다. 이 장에서는 각 패턴을 Graph API와 Functional API 양쪽으로 구현하며, 어떤 상황에서 어떤 패턴을 선택해야 하는지 감을 잡아 봅니다.

학습 목표

1. 다섯 가지 핵심 워크플로 패턴의 구조와 적합한 사용 상황을 설명할 수 있다.
2. 각 패턴을 Graph API와 Functional API 양쪽으로 구현할 수 있다.
3. `Send()` API를 활용하여 런타임에 동적으로 워커 노드를 생성하는 Map-Reduce 패턴을 구현할 수 있다.
4. 반복 루프 패턴에서 `recursion_limit` 으로 무한 루프를 방지할 수 있다.
5. 패턴을 조합하여 복합 워크플로를 설계할 수 있다.

4.1 환경 설정

이 장의 예제에서는 LLM 호출 외에도 `Send` 등 LangGraph 타입을 사용하므로, 필요한 모듈을 함께 임포트합니다.

```
from dotenv import load_dotenv
load_dotenv(override=True)

from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
```

4.2 Prompt Chaining — 순차적 LLM 호출

첫 번째 패턴은 가장 단순하면서도 강력한 Prompt Chaining입니다. 하나의 복잡한 작업을 여러 개의 작은 LLM 호출로 분해하고, 각 단계의 출력을 다음 단계의 입력으로 전달합니다. 이 패턴은 “분할 정복(divide and conquer)” 전략의 LLM 버전이라 할 수 있습니다. 각 단계가 하나의 구체적인 작업만 담당하므로, 프롬프트가 간결해지고 LLM의 응답 품질이 향상됩니다.

Graph API에서는 `add_edge(A, B)`, `add_edge(B, C)` 형태의 일직선 연결로 구현합니다.

`add_sequence([step1, step2, step3])` 를 사용하면 더 간결하게 작성할 수 있습니다. Functional API에서는 순차적인 `@task` 호출로 동일한 효과를 냅니다.

- 각 단계의 출력이 다음 단계의 입력이 됩니다
- 용도: 번역 -> 검증 -> 교정, 분석 -> 요약 -> 포매팅
- 중간 단계에서 품질 검사(gate)를 삽입하여 기준 미달 시 조기 종료할 수도 있습니다

```
# Graph API: 순차 체인
builder.add_edge(START, "step1")
builder.add_edge("step1", "step2")
builder.add_edge("step2", "step3")
builder.add_edge("step3", END)

# 또는 축약 형태
builder.add_sequence([step1, step2, step3])
```

TIP

Prompt Chaining에서 각 단계의 프롬프트를 설계할 때, 이전 단계의 출력 형식과 다음 단계의 입력 형식을 명확하게 정의하는 것이 중요합니다. 구조화된 출력(`with_structured_output`)을 활용하면 단계 간 데이터 전달이 안정적입니다.

4.3 Parallelization — 독립적 태스크의 동시 실행

Prompt Chaining이 순차적 파이프라인이라면, Parallelization은 독립적인 작업을 동시에 수행하는 패턴입니다. 서로 독립적인 분석 작업(예: 감정 분석, 키워드 추출, 요약)을 동시에 수행하면 전체 실행 시간을 크게 줄일 수 있습니다. N개의 독립 태스크를 순차로 실행하면 총 시간이 합산되지만, 병렬로 실행하면 가장 느린 태스크의 시간만 소요됩니다.

Graph API에서는 하나의 노드에서 여러 노드로 엣지를 연결하는 팬아웃(fan-out) 구조를 사용합니다. LangGraph의 Pregel 실행 모델에서 같은 슈퍼스텝에 속하는 노드들은 자동으로 병렬 실행됩니다. 병렬로 실행된 결과는 이후 하나의 노드에서 팬인(fan-in)으로 합쳐집니다. 이때 결과가 같은 상태 필드에 기록된다면, 반드시 리듀서(예: `operator.add`)를 지정하여 안전한 병합을 보장해야 합니다.

Functional API에서는 여러 `@task` 를 먼저 호출하고 `.result()` 를 나중에 일괄 호출하면 동일한 효과를 얻습니다. 3장에서 배운 Future 패턴이 바로 이 병렬 실행 패턴의 핵심입니다.

```
# Graph API: 팬아웃 구조
builder.add_edge("start_node", "sentiment") # 병렬 분기 1
builder.add_edge("start_node", "keywords") # 병렬 분기 2
builder.add_edge("start_node", "summary") # 병렬 분기 3
# sentiment, keywords, summary는 같은 슈퍼스텝에서 동시 실행
builder.add_edge("sentiment", "merge")
builder.add_edge("keywords", "merge")
builder.add_edge("summary", "merge")
```

TIP

Graph API에서 병렬 실행은 같은 슈퍼스텝에 속하는 노드끼리 자동으로 발생합니다. 명시적인 비동기 코드 없이도 엣지 구조만으로 병렬화가 이루어진다는 점이 큰 장점입니다. 다만 병렬 노드들이 동일한 상태 필드에 값을 쓸 경우 리듀서가 없으면 데이터 손실이 발생할 수 있으므로, `Annotated[list, operator.add]` 와 같은 리듀서를 반드시 설정하세요.

4.4 Routing – 분류 기반 분기

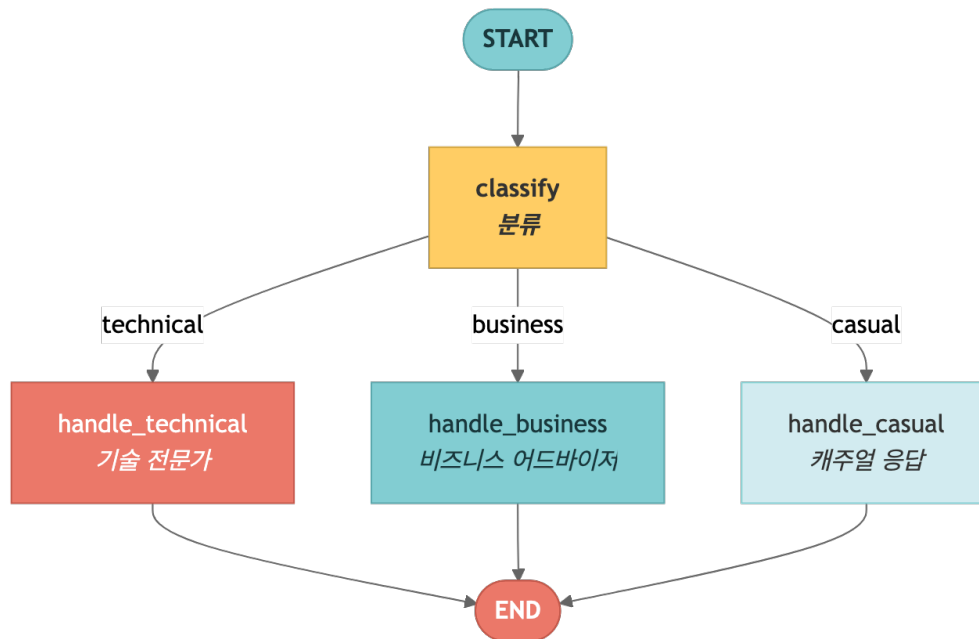
순차와 병렬 패턴이 모든 단계를 실행하는 반면, Routing은 입력의 유형에 따라 하나의 처리 경로만 선택하는 패턴입니다. 예를 들어, 사용자 질문을 먼저 “날씨”, “수학”, “일반” 등으로 분류한 뒤, 해당 카테고리에 특화된 노드로 라우팅합니다. 이를 통해 각 경로는 해당 유형에 최적화된 프롬프트와 도구를 사용할 수 있고, 불필요한 처리를 건너뛰어 비용과 지연 시간을 줄일 수 있습니다.

Graph API에서는 2장에서 배운 `add_conditional_edges()` 로 자연스럽게 구현됩니다. 라우팅 함수가 현재 상태를 분석하여 다음 노드를 결정합니다. Functional API에서는 Python의 `if/elif` 분기문으로 동일한 효과를 냅니다. 라우팅 로직을 LLM에게 위임하여 구조화된 출력으로 분류 결과를 받는 것이 일반적인 패턴입니다.

```
# Graph API: 조건부 엣지로 라우팅
def route(state) -> Literal["weather", "math", "general"]:
    return state["classification"]

builder.add_conditional_edges("classify", route)

# Functional API: if/elif로 라우팅
if classification == "weather":
    result = handle_weather(query).result()
elif classification == "math":
    result = handle_math(query).result()
else:
    result = handle_general(query).result()
```


**! WARNING**

라우팅 함수에서 `Literal` 타입 힌트를 사용하면 그래프 시각화에 가능한 경로가 모두 표시됩니다. 타입 힌트가 없으면 시각화 도구가 분기 경로를 추론할 수 없어, 디버깅 시 불편할 수 있습니다.

4.5 Orchestrator-Worker – `Send()`로 동적 워커 생성

Parallelization 패턴에서 병렬 경로는 컴파일 시점에 고정되어 있었습니다. 그러나 실제로는 입력 데이터에 따라 병렬 작업의 수가 달라지는 경우가 대부분입니다. 예를 들어 “이 문서를 5개 언어로 번역하라”는 요청에서, 번역 대상 언어 수는 런타임에 결정됩니다. Orchestrator-Worker 패턴은 이러한 실행 시점 동적 팬아웃을 해결합니다.

LangGraph의 `Send()` API가 이 패턴의 핵심입니다. `Send(node_name, state)` 형태로 호출하면, 지정된 노드를 해당 상태로 새로 생성합니다. 오케스트레이터 노드가 `Send` 객체의 리스트를 반환하면, 각 `Send`가 독립적인 워커 인스턴스를 생성하여 병렬로 실행됩니다. 각 워커는 자신만의 상태를 받으므로, 동일한 노드 함수가 서로 다른 데이터를 처리할 수 있습니다. 이는 MapReduce 패턴의 Map 단계에 정확히 대응합니다.

아래 코드에서 `route_to_workers` 함수는 `add_conditional_edges`의 라우팅 함수로 사용되지만, 문자열 대신 `Send` 객체의 리스트를 반환한다는 점이 다릅니다. 각 `Send`는 “worker” 노드를 서로 다른 `task` 값으로 호출합니다.

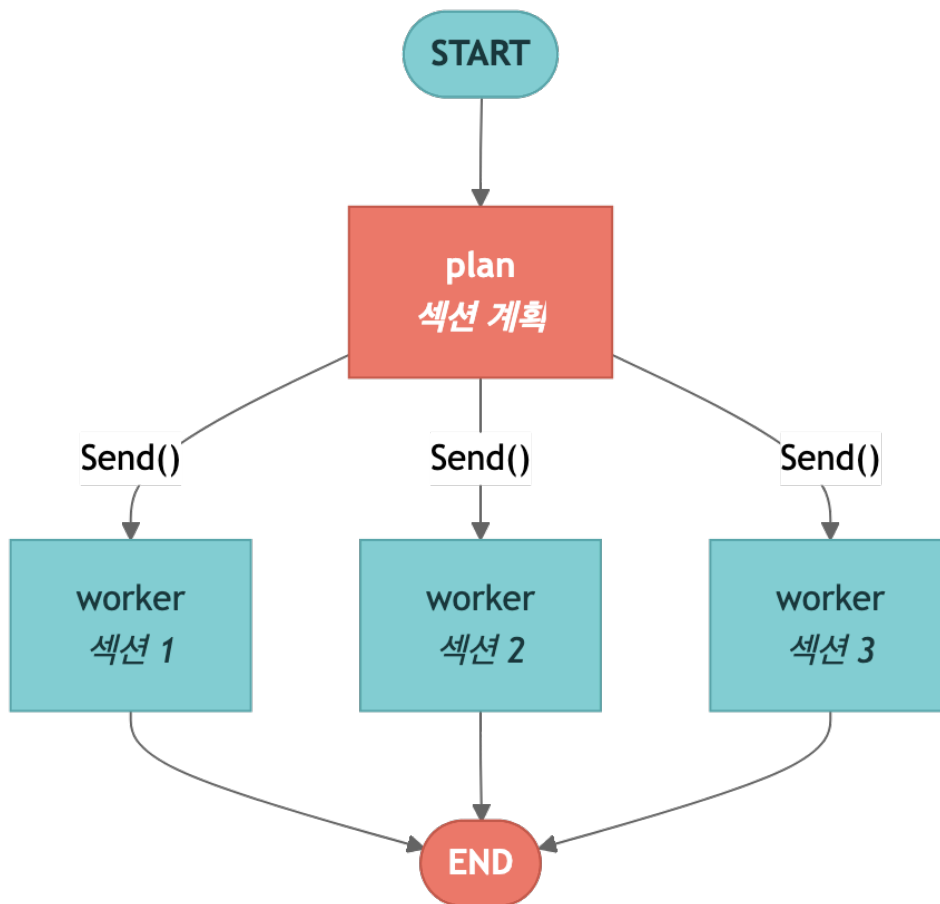
```

from langgraph.types import Send

def route_to_workers(state):
    # 런타임에 태스크 수만큼 워커 생성
    return [Send("worker", {"task": t}) for t in state["tasks"]]

builder.add_conditional_edges("planner", route_to_workers)
builder.add_node("worker", worker_fn)
builder.add_edge("worker", "aggregator")

```



TIP

`Send()` API에서 각 워커는 독립적인 상태를 받습니다. 워커 노드의 결과를 하나로 합치려면 팬인(fan-in) 노드에서 리듀서를 활용해야 합니다. 예를 들어 `results: Annotated[list, operator.add]` 필드를 정의하면, 각 워커가 반환한 결과가 자동으로 리스트에 누적됩니다.

4.6 Evaluator-Optimizer — 생성-평가 반복 루프

지금까지의 네 패턴이 일회성 처리(한 번 실행하고 끝)에 초점을 맞추었다면, Evaluator-Optimizer는 반복적 개선을 다루는 유일한 순환(cyclic) 패턴입니다. 핵심 구조는 간단합니다: 생성 노드가 결과를 만들고, 평가 노드가 품질을 판단한 뒤, 기준에 미치지 못하면 피드백과 함께 다시 생성 노드로 돌아가는 루프입니다. 이 패턴은 코드 생성 후 테스트 실행, 글 작성 후 품질 평가, 이미지 생성 후 평가 등 자기 개선(self-improvement)이 필요한 모든 시나리오에 적용됩니다.

Graph API에서는 평가 노드에서 조건부 엣지로 “다시 생성”(`improve`) 또는 “종료”(`END`)를 선택합니다.

`improve` -> `evaluate` 방향의 엣지가 순환 구조를 만들어, 품질 기준을 만족할 때까지 반복합니다. Functional API에서는 `while` 루프 안에 품질 검사 조건을 넣으면 됩니다.

```
# Graph API: 순환 루프
def should_continue(state):
    if state["score"] > 0.8:
        return END
    return "improve" # 생성 노드로 복귀

builder.add_conditional_edges("evaluate", should_continue)
builder.add_edge("improve", "evaluate") # 순환 엣지

# Functional API: while 루프
score = 0
while score <= 0.8 and attempts < max_attempts:
    result = generate(feedback).result()
    score = evaluate(result).result()
    attempts += 1
```

! WARNING

반복 루프 패턴에서는 무한 루프를 방지하기 위해 최대 반복 횟수를 반드시 설정하세요. Graph API에서는 `graph.invoke(inputs, config={"recursion_limit": 50})` 로 최대 슈퍼스텝 수를 제한하거나, 상태에 카운터 필드를 추가하여 직접 관리할 수 있습니다. 기본 `recursion_limit` 은 1000이므로, 프로덕션 환경에서는 반드시 적절한 값으로 낮추는 것을 권장합니다.

다섯 가지 패턴을 모두 살펴보았습니다. 각 패턴은 독립적으로 사용할 수도 있지만, 실전에서는 여러 패턴을 조합하는 것이 일반적입니다. 이제 각 패턴의 특성을 한눈에 비교하여, 어떤 상황에서 어떤 패턴을 선택해야 하는지 정리해 봅시다.

4.7 패턴 비교표

패턴	결정적	병렬	반복	적합 상황
Prompt Chaining	O	X	순차	단계별 변환
Parallelization	O	O	X	독립 분석
Routing	O	X	X	분류 기반 처리
Orchestrator-Worker	O	O	X	동적 하위 작업

Evaluator-Optimizer	X	X	O	품질 개선 루프
---------------------	---	---	---	----------

➡ TIP

실제 프로젝트에서는 이 패턴들을 조합하여 사용합니다. 예를 들어, Routing으로 입력을 분류한 뒤 각 경로에서 Prompt Chaining을 수행하거나, Orchestrator-Worker로 병렬 처리한 결과를 Evaluator-Optimizer로 품질 검증하는 식입니다. 서브그래프를 활용하면 각 패턴을 독립적인 그래프로 구현한 뒤, 부모 그래프에서 노드로 조합할 수 있습니다.

! WARNING

패턴 선택 시 가장 중요한 원칙은 단순함입니다. Prompt Chaining으로 충분한 문제에 Orchestrator-Worker를 적용하면 불필요한 복잡성만 추가됩니다. 항상 가장 단순한 패턴에서 시작하고, 성능이나 품질 요구 사항에 따라 점진적으로 복잡한 패턴으로 전환하세요.

⇒ NEXT STEP

다음 장에서는 이 워크플로 패턴들 중 가장 중요한 ReAct 에이전트를 Graph API와 Functional API 모두로 구현합니다. LLM이 스스로 도구를 선택하고 호출하는 자율적 에이전트의 구축 방법을 배웁니다.

05 에이전트 구축

Graph API와 Functional API로 ReAct 에이전트 만들기

4장에서 다섯 가지 워크플로 패턴을 익혔다면, 이제 LLM이 스스로 도구를 선택하고 호출하는 자율적 에이전트를 구축할 차례입니다. 워크플로가 개발자가 미리 정해둔 코드 경로를 따르는 것이라면, 에이전트는 LLM이 스스로 다음 행동을 결정합니다. 어떤 도구를 호출할지, 언제 작업을 멈출지, 추가 정보가 필요한지를 LLM이 자율적으로 판단하는 것이 핵심입니다.

ReAct (Reasoning + Acting) 패턴은 LLM이 “생각 → 도구 호출 → 관찰 → 다음 행동”을 반복하는 가장 널리 쓰이는 에이전트 아키텍처입니다. 2022년 Yao et al.이 제안한 이 패턴은 LLM의 추론(reasoning) 능력과 외부 도구를 통한 행동(acting) 능력을 결합하여, 단순 프롬프트만으로는 불가능한 복잡한 작업을 수행할 수 있게 합니다. LangGraph에서 ReAct 에이전트는 “LLM 노드 → 조건부 분기 → 도구 노드 → LLM 노드”로 이어지는 순환 그래프로 자연스럽게 표현됩니다.

LangGraph는 ReAct 에이전트를 쉽게 구축할 수 있도록 두 가지 주요 사전 구축(prebuilt) 컴포넌트를 제공합니다. `tools_condition` 은 LLM 응답의 `AIMessage` 에 `tool_calls` 속성이 존재하는지 확인하여 도구 노드 또는 `END` 로 라우팅하는 조건부 엣지 함수입니다. 내부적으로 `tool_calls` 리스트가 비어 있지 않으면 “tools” 경로를, 비어 있으면 `END` 경로를 반환합니다. `ToolNode` 는 `AIMessage.tool_calls` 를 파싱하여 해당하는 도구 함수를 실행하고, 결과를 `ToolMessage` 로 변환하여 상태에 추가하는 노드입니다. 병렬 도구 호출과 에러 핸들링도 자동으로 처리합니다. 물론 이 장에서는 내부 동작을 이해하기 위해 직접 구현합니다.

이 장에서는 동일한 ReAct 에이전트를 Graph API의 조건부 엣지 방식과 Functional API의 `while` 루프 방식으로 각각 구현하며, 두 접근법의 차이를 체감합니다. Graph API는 노드와 엣지로 구성된 명시적 그래프 구조를 선호하는 개발자에게 적합하고, Functional API는 일반 Python 코드 흐름에 익숙한 개발자에게 더 직관적입니다.

학습 목표

도구를 사용하는 LLM 에이전트를 두 가지 API로 구현합니다.

- Graph API: `StateGraph` 와 조건부 엣지로 ReAct 루프를 명시적으로 구성
- Functional API: `@entrypoint` + `while` 루프로 간결하게 구현
- 도구 바인딩: `@tool` 데코레이터와 `bind_tools()` 로 LLM에 도구 스키마 연결
- 라우팅 로직: `tools_condition` 과 `ToolNode` 의 내부 동작 이해
- 메모리: 체크포인트로 대화 상태를 유지하는 에이전트

5.1 환경 설정

```
from dotenv import load_dotenv
load_dotenv(override=True)
from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
```

5.2 도구 정의 — @tool 데코레이터와 bind_tools()

에이전트의 핵심 능력은 외부 도구를 호출할 수 있다는 점입니다. 이를 위해서는 먼저 도구를 정의하고, LLM이 그 도구의 존재와 사용법을 알 수 있도록 연결해야 합니다.

LangChain의 `@tool` 데코레이터를 사용하면 일반 Python 함수를 LLM이 호출할 수 있는 도구로 변환할 수 있습니다. 데코레이터는 함수의 이름, 독스트링(설명), 타입 힌트(인자 스키마)를 자동으로 추출하여 도구 메타데이터를 생성합니다. 따라서 함수의 독스트링과 타입 힌트를 정확하게 작성하는 것이 매우 중요합니다 — LLM은 이 정보를 기반으로 어떤 도구를 언제 호출할지 판단하기 때문입니다.

`bind_tools()` 는 이 도구들의 JSON 스키마를 모델의 요청 파라미터에 바인딩합니다. 바인딩된 모델은 사용자 질문을 분석한 뒤, 도구 호출이 필요하다고 판단하면 응답의 `AIMessage.tool_calls` 속성에 호출할 도구 이름과 인자를 포함시킵니다. 도구 호출이 불필요한 경우에는 일반 텍스트 응답을 생성합니다.

TIP

`@tool` 데코레이터의 독스트링은 LLM이 도구를 선택하는 유일한 단서입니다. “두 수를 더합니다”처럼 명확하고 구체적으로 작성하세요. 모호한 설명은 LLM의 도구 선택 정확도를 떨어뜨립니다.

아래 코드에서는 세 가지 산술 도구를 정의하고, `bind_tools()` 로 모델에 바인딩합니다. `tools_by_name` 딕셔너리는 이후 도구 노트에서 이름으로 도구를 빠르게 찾기 위한 준비입니다.

```
from langchain.tools import tool
from langchain.messages import HumanMessage, SystemMessage, ToolMessage, AnyMessage

@tool
def add(a: int, b: int) -> int:
    """두 수를 더합니다."""
    return a + b

@tool
def multiply(a: int, b: int) -> int:
    """두 수를 곱합니다."""
    return a * b

@tool
def divide(a: int, b: int) -> float:
    """a를 b로 나눕니다."""
    return a / b

tools = [add, multiply, divide]
tools_by_name = {t.name: t for t in tools}
model_with_tools = model.bind_tools(tools)

print("모델에 바인딩된 도구:")
for t in tools:
    print(f"  - {t.name}: {t.description}")
```

모델에 바인딩된 도구:

- add: 두 수를 더합니다.
- multiply: 두 수를 곱합니다.
- divide: a를 b로 나눕니다.

도구가 준비되었으니, 이제 이 도구들을 활용하는 에이전트를 구축할 차례입니다. 먼저 Graph API로 ReAct 루프를 명시적인 그래프 구조로 표현해 봅시다.

5.3 Graph API 에이전트 – StateGraph로 ReAct 루프 구현

Graph API로 구현하는 ReAct 에이전트는 다음 세 가지 핵심 요소로 구성됩니다:

- LLM 노드: 현재까지 누적된 메시지를 기반으로 다음 행동을 결정합니다. 도구 호출이 필요하다면 `AIMessage.tool_calls`에 호출 정보를 담아 응답하고, 최종 답변이 준비되면 일반 텍스트를 응답합니다
- Tool 노드: LLM이 선택한 도구를 실제로 실행합니다. `AIMessage.tool_calls` 리스트를 순회하며, 각 호출의 `name`으로 도구를 찾고 `args`를 전달하여 실행한 뒤, 결과를 `ToolMessage`로 변환합니다. LangGraph의 사전 구축 `ToolNode`는 이 과정을 자동으로 처리하지만, 여기서는 학습을 위해 직접 구현합니다
- 조건부 엣지: LLM 노드의 출력을 검사하여 라우팅합니다. `tool_calls`가 있으면 `tool_node`로, 없으면 `END`로 보냅니다. 사전 구축 `tools_condition` 함수도 동일한 로직을 수행합니다

이 세 요소가 순환하는 구조가 바로 ReAct 루프입니다:

```
START → llm → [tool_calls?] → tools → llm → ... → END
```

! WARNING

조건부 엣지의 라우팅 로직은 `AIMessage.tool_calls` 리스트의 존재 여부만 확인합니다. 도구 호출의 결과를 보고 판단하는 것이 아닙니다. LLM이 도구 호출을 요청했으면 무조건 도구 노드로 가고, 요청하지 않았으면 대화가 종료됩니다. 이 단순한 이진 분기가 ReAct 루프의 핵심입니다.

그래프를 구성했으니, 에이전트가 실제로 어떤 순서로 동작하는지 눈으로 확인해 봅시다. 스트리밍을 통해 각 노드의 실행 과정을 실시간으로 관찰할 수 있습니다.

5.4 실행 흐름 시각화 – 스트리밍으로 각 단계 관찰

`stream_mode="updates"`를 사용하면 각 노드가 실행될 때마다 해당 노드가 반환한 업데이트만 실시간으로 받을 수 있습니다. 이를 통해 에이전트가 어떤 순서로 도구를 호출하고 결과를 처리하는지 단계별로 관찰할 수 있습니다. 출력에서 `llm` 노드와 `tools` 노드가 번갈아 나타나는 것을 확인하세요 – 이것이 바로 ReAct 루프가 동작하는 모습입니다.

TIP

스트리밍 출력에서 `llm` 노드의 결과에 `tool_calls`가 포함되어 있으면 다음에 `tools` 노드가 실행되고, 포함되어 있지 않으면 그래프가 종료됩니다. 이 패턴을 관찰하면 에이전트의 의사결정 과정을 직관적으로 이해할 수 있습니다.

Graph API에서 ReAct 에이전트의 동작을 확인했습니다. 이제 동일한 에이전트를 Functional API로 구현하여 두 접근법의 차이를 체감해 봅시다.

5.5 Functional API 에이전트 — @entrypoint + while 루프

Graph API로 구현한 동일한 ReAct 에이전트를 Functional API로 작성하면 코드가 얼마나 달라지는지 비교해 봅시다. Functional API는 `StateGraph`, `add_node()`, `add_edge()` 등의 그래프 구성 코드 없이, 일반 Python 코드처럼 에이전트를 작성할 수 있습니다. 그래프 구조가 아니라 함수 호출 흐름으로 로직을 표현하므로, Python에 익숙한 개발자에게 더 직관적일 수 있습니다.

Functional API의 핵심 구성 요소는 다음 세 가지입니다:

- `@entrypoint`: 에이전트의 진입점을 정의합니다. Graph API의 `graph.invoke()`에 해당하며, 체크포인트와 연동도 여기서 설정합니다
- `@task`: 개별 작업 단위를 정의합니다. LLM 호출, 도구 실행 등 각 단계를 독립적인 태스크로 분리하면, 체크포인트가 태스크 단위로 상태를 저장하여 실패 시 재개가 가능합니다
- `while` 루프: `tool_calls`가 없을 때까지 반복합니다. Graph API의 조건부 엣지 → LLM 노드 순환이 여기서 단순한 `while` 조건으로 표현됩니다

TIP

Graph API에서는 ReAct 루프가 “LLM 노드 → 조건부 엣지 → 도구 노드 → LLM 노드”라는 그래프 구조로 암시적으로 표현됩니다. Functional API에서는 동일한 루프가 `while` 문으로 명시적으로 드러납니다. 어느 쪽이 더 읽기 쉬운지는 프로젝트의 복잡도와 팀의 선호에 따라 다릅니다. 일반적으로, 단순한 에이전트는 Functional API가 간결하고, 복잡한 멀티 에이전트 시스템은 Graph API가 구조를 파악하기 쉽습니다.

두 가지 API로 ReAct 에이전트를 구현해 보았습니다. 하지만 지금까지 구축한 에이전트에는 중요한 한계가 있습니다 — 매 호출마다 상태가 초기화된다는 점입니다. 이제 체크포인트를 도입하여 이 문제를 해결합니다.

5.6 메모리가 있는 에이전트 — 체크포인트로 대화 유지

지금까지 구축한 에이전트는 매 호출마다 상태가 초기화되는 무상태(stateless) 에이전트였습니다. 사용자가 “3 더하기 5”를 물은 뒤 “거기에 2를 곱해줘”라고 하면, 에이전트는 이전 결과를 모르기 때문에 올바른 답을 줄 수 없습니다. 실제 채팅 애플리케이션에서는 이전 대화를 기억하는 유상태(stateful) 에이전트가 필수적입니다.

체크포인트(`InMemorySaver`)를 `compile(checkpointer=checkpointer)`로 전달하면, 그래프의 각 노드 실행 후 자동으로 상태가 스냅샷됩니다. 동일한 `thread_id`로 후속 요청을 보내면, 체크포인트가 이전 대화의 메시지 이력을 자동으로 복원하여 대화 컨텍스트가 유지됩니다. 서로 다른 `thread_id`를 사용하면 완전히 독립된 대화가 생성됩니다.

! WARNING

`InMemorySaver`는 메모리에만 상태를 저장하므로, 프로세스가 종료되면 모든 대화 이력이 사라집니다. 개발 및 테스트 용도로만 사용하고, 프로덕션에서는 `PostgresSaver`나 `SqliteSaver`를 사용하세요. 체크포인트의 상세한 동작 원리는 다음 장에서 깊이 다룹니다.

이 장에서 LLM 에이전트의 핵심 아키텍처인 ReAct 패턴을 두 가지 API로 구현하고, 체크포인트를 통해 멀티턴 대화까지 지원하는 에이전트를 완성했습니다. `@tool`로 도구를 정의하고, `bind_tools()`로 모델에 바인딩하며, 조건부 엣지 또는 `while` 루프로 ReAct 순환을 구성하는 것이 에이전트 구축의 기본 뼈대입니다.

요약

개념	설명
<code>\@tool</code>	Python 함수를 LLM 호출 가능한 도구로 변환
<code>bind_tools()</code>	도구 스키마를 모델에 바인딩
Graph API 에이전트	<code>StateGraph</code> + 조건부 엣지로 ReAct 루프 명시적 구현
Functional API 에이전트	<code>\@entrypoint</code> + <code>while</code> 루프로 간결하게 구현
<code>tool_calls</code>	LLM 응답에 포함된 도구 호출 정보
<code>ToolMessage</code>	도구 실행 결과를 LLM에게 전달하는 메시지
체크포인트	대화 상태를 저장하여 멀티턴 에이전트 구현

⇒ NEXT STEP

다음 장에서는 체크포인트의 내부 동작을 깊이 파고듭니다. `get_state()`, `get_state_history()`, `update_state()` API로 저장된 상태를 조회하고 수정하며, `InMemoryStore`를 활용한 스레드 간 장기 메모리까지 다룹니다.

06 지속성과 메모리

체크포인트와 메모리 스토어

5장에서 체크포인트를 사용하여 멀티턴 에이전트를 구축했습니다. 이 장에서는 체크포인트의 내부 동작을 심층적으로 탐구합니다. 에이전트가 단일 요청을 넘어 여러 턴에 걸친 대화나 장기 작업을 수행하려면, 실행 상태를 저장하고 복원하는 메커니즘이 필수입니다. `LangGraph`의 체크포인트(`Checkpoint`)는 각 슈퍼스텝(Graph API) 또는 각 태스크(Functional API) 실행 후 상태를 자동으로 스냅샷하여 장애 복구와 타임 트래블의 기반을 제공합니다.

`LangGraph`의 메모리 시스템은 두 가지 계층으로 구분됩니다. 단기 메모리(short-term memory)는 체크포인트가 관리하며, 하나의 스레드(대화) 내에서 상태를 유지합니다. 동일한 `thread_id`로 요청을 보내면 이전 메시지가 자동으로 복원되는 것이 바로 단기 메모리의 동작입니다. 장기 메모리(long-term memory)는 `InMemoryStore`가 관리하며, 스레드 경계를 넘어 사용자 프로필, 선호도 등을 저장합니다. 예를 들어, 사용자가 “나는 Python을 좋아해”라고 한 대화의 정보를 다른 대화에서도 활용할 수 있습니다. 이 장에서는 두 계층을 모두 다루며, 프로덕션 수준의 상태 관리 전략을 세워 봅니다.

TIP

단기 메모리와 장기 메모리의 핵심 차이를 기억하세요: 단기 메모리는 스레드 내(within-thread) 상태를 유지하고, 장기 메모리는 스레드 간(cross-thread) 데이터를 공유합니다. 체크포인트는 대화 이력을, `InMemoryStore`는 사용자 프로필이나 학습된 선호도를 저장하는 데 적합합니다.

학습 목표

체크포인트로 상태를 저장하고, 스토어로 장기 메모리를 구현합니다.

- 체크포인트: 각 실행 단계의 상태를 자동으로 저장하고 복원
- 상태 조회: `get_state()`와 `get_state_history()`로 저장된 상태 확인
- 상태 수정: `update_state()`로 외부에서 상태 변경
- 스레드 독립성: 서로 다른 `thread_id`는 완전히 독립된 상태
- `InMemoryStore`: 스레드 간 공유되는 장기 메모리 (standalone 및 그래프 연동)
- 대화 길이 관리: `trim_messages`와 `RemoveMessage`로 메시지 관리
- Durable Execution: 실패 시 마지막 체크포인트에서 재개

6.1 환경 설정

```
from dotenv import load_dotenv
load_dotenv(override=True)
```

```
from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4-mini")
```

6.2 체크포인트 — 각 실행 단계의 상태를 자동으로 저장합니다

체크포인트는 LangGraph에서 지속성, Human-in-the-loop, 타임 트래블, 내구성 실행 등 거의 모든 고급 기능의 기반입니다. 체크포인트 없이는 그래프 실행이 끝나면 상태가 사라지지만, 체크포인트가 있으면 각 슈퍼스텝(노드 실행) 후 상태가 자동으로 스냅샷되어, 나중에 정확히 그 시점부터 재개할 수 있습니다.

LangGraph는 용도에 따라 다양한 체크포인트 구현을 제공합니다. 다음 표는 자주 쓰이는 7가지를 한눈에 비교합니다.

체크포인트	패키지	용도
InMemorySaver	langgraph-checkpoint (기본 포함)	개발/테스트. 프로세스 종료 시 상태 소실
SqliteSaver	langgraph-checkpoint-sqlite	로컬 개발. 단일 파일 영속화, async 변형 제공
PostgresSaver	langgraph-checkpoint-postgres	프로덕션 기본값. AsyncPostgresSaver 비동기 지원
MongoDBSaver	langgraph-checkpoint-mongodb	도큐먼트 DB. JSON 친화적인 스토리지 백엔드
RedisSaver	langgraph-checkpoint-redis	인메모리 영속화. 낮은 지연, async 변형 제공
OracleSaver	langgraph-checkpoint-oracle	엔터프라이즈 Oracle DB. 벡터 검색까지 지원
CosmosDBSaver	langchain-azure-cosmosdb	Azure Cosmos DB. Microsoft Entra ID 인증 호환

체크포인트를 `compile(checkpointer=checkpointer)` 에 전달하면, 그래프의 각 노드 실행 후 자동으로 상태가 저장됩니다. 각 체크포인트는 `StateSnapshot` 객체로, 상태 값(`values`), 체크포인트 ID, 부모 체크포인트 ID, 다음 실행할 노드(`next`), 타임스탬프 등의 메타데이터를 포함합니다. 이 정보들을 통해 그래프 실행의 전체 이력을 추적하고, 특정 시점으로 돌아갈 수 있습니다.

DB 기반 체크포인트는 컨텍스트 매니저 + `setup()` 패턴으로 초기화합니다. 비동기 그래프에서는

`AsyncPostgresSaver` 같은 async 변형을 사용합니다.

```
from langgraph.checkpoint.postgres import PostgresSaver

DB_URI = "postgresql://postgres:postgres@localhost:5442/postgres?sslmode=disable"
with PostgresSaver.from_conn_string(DB_URI) as checkpointer:
    checkpointer.setup() # 최초 1회 - 스키마 생성
    graph = builder.compile(checkpointer=checkpointer)

# 비동기 그래프
from langgraph.checkpoint.postgres import AsyncPostgresSaver

checkpointer = AsyncPostgresSaver.from_conn_string(DB_URI)
await checkpointer.setup()
graph = builder.compile(checkpointer=checkpointer)
```

! WARNING

`InMemorySaver` 는 프로세스가 종료되면 모든 상태가 사라집니다. 프로덕션 환경에서는 반드시 `PostgresSaver` (또는 `MongoDBSaver` / `RedisSaver` / `OracleSaver`)를 사용하세요. DB 기반 체크포인트는 첫 사용 시 반드시 `checkpointer.setup()` 을 호출해 스키마를 생성해야 합니다.

체크포인트를 설정했으니, 이제 저장된 상태를 조회하는 방법을 알아보시다. `LangGraph`는 상태를 읽고, 이력을 추적하고, 수정하는 세 가지 핵심 API를 제공합니다.

6.3 get_state() — 현재 저장된 상태 조회

`get_state(config)` 는 지정된 스레드의 최신 체크포인트 상태를 `StateSnapshot` 객체로 반환합니다. 반환된 객체에서 `values` 로 현재 상태 값을, `config` 로 체크포인트 ID를, `next` 로 다음에 실행할 노드 이름을 확인할 수 있습니다. `next` 가 빈 튜플이면 그래프 실행이 완료된 상태입니다. 특정 체크포인트를 조회하려면 `config` 에 `checkpoint_id` 를 추가하면 됩니다.

아래 코드를 실행하면 저장된 상태의 핵심 정보를 확인할 수 있습니다. 특히 `checkpoint_id` 는 타임 트래블이나 상태 복원에 사용되는 고유 식별자입니다.

```
state = graph.get_state(config)
print(f"스레드: {config['configurable']['thread_id']}")
print(f"메시지 수: {len(state.values['messages'])}")
print(f"체크포인트 ID: {state.config['configurable']['checkpoint_id']}")
print(f"다음 노드: {state.next}")
```

```
스레드: session-1
메시지 수: 4
체크포인트 ID: 1f1196e1-c938-68ee-8004-4f52838ad157
다음 노드: ()
```

현재 상태뿐 아니라 전체 실행 이력을 추적해야 하는 경우도 있습니다. 예를 들어, 에이전트가 어떤 단계에서 잘못된 판단을 했는지 사후 분석하거나, 특정 시점으로 되돌아가고 싶을 때가 그렇습니다.

6.4 get_state_history() — 전체 실행 이력 조회

`get_state_history(config)` 는 해당 스레드의 모든 체크포인트를 최신순으로 반환하는 제너레이터입니다. 각 체크포인트는 `StateSnapshot` 객체로, 해당 시점의 상태 값과 메타데이터를 포함합니다. 이를 통해 그래프 실행의 전체 이력을 시간순으로 추적할 수 있으며, 특정 체크포인트의 `config` 를 `graph.invoke(None, config=snapshot.config)` 에 전달하면 해당 시점부터 실행을 재개할 수 있습니다 — 이것이 타임 트래블의 기초입니다.

아래 코드에서 출력되는 메시지 수의 변화를 관찰하세요. 메시지가 1개에서 점차 늘어나는 과정이 대화의 진행 이력을 보여줍니다.

```
print("상태 이력 (최신순):")
for i, snapshot in enumerate(graph.get_state_history(config)):
    msg_count = len(snapshot.values.get("messages", []))
    print(f" [{i}] 체크포인트={snapshot.config['configurable']['checkpoint_id'][:20]}... 메시지={msg_count}")
    if i >= 4:
```

```
print(" ... (생략)")
break
```

상태 이력 (최신순):

```
[0] 체크포인트=1f1196e1-c938-68ee-8... 메시지=4
[1] 체크포인트=1f1196e1-bf0f-6ddf-8... 메시지=3
[2] 체크포인트=1f1196e1-bf0f-6dde-8... 메시지=2
[3] 체크포인트=1f1196e1-bf0a-6139-8... 메시지=2
[4] 체크포인트=1f1196e1-abd9-65c9-8... 메시지=1
... (생략)
```

상태를 조회하는 것에서 한 걸음 더 나아가, 저장된 상태를 외부에서 수정해야 하는 경우를 살펴봅시다.

6.5 update_state() — 저장된 상태를 외부에서 수정

상태를 읽는 것만으로는 부족한 경우가 있습니다. 에이전트가 잘못된 판단을 했을 때 외부에서 상태를 교정하거나, Human-in-the-loop 패턴에서 사람의 승인이나 입력을 상태에 반영해야 할 때 `update_state()` 를 사용합니다.

`update_state(config, values)` 를 호출하면 체크포인트에 저장된 상태를 프로그래밍 방식으로 수정할 수 있습니다. 이때 중요한 점은, 리듀서가 설정된 채널(예: `MessagesState` 의 `messages`)은 값이 병합(merge)되고, 리듀서가 없는 채널은 값이 덮어쓰기(overwrite)된다는 것입니다. 예를 들어, 시스템 노트 메시지를 추가하거나, 사용자 선호도를 반영하거나, 에이전트의 잘못된 도구 호출 결과를 올바른 값으로 교체하는 등의 작업이 가능합니다.

수정된 상태는 새로운 체크포인트로 저장되므로, 원본 체크포인트는 변경되지 않습니다. 이 불변성(immutability) 덕분에 언제든지 이전 상태로 되돌아갈 수 있습니다. 또한 `as_node` 파라미터를 지정하면, 수정 후 어떤 노드가 다음에 실행될지를 제어할 수 있습니다.

TIP

`update_state()` 는 8장에서 다룰 Human-in-the-loop 패턴의 핵심 도구입니다. 에이전트가 `interrupt()` 로 멈춘 뒤, 사람이 상태를 검토하고 `update_state()` 로 수정한 다음, `Command(resume=...)` 로 실행을 재개하는 흐름이 전형적인 패턴입니다.

상태를 조회하고 수정하는 API를 익혔으니, 스레드 간의 관계를 명확히 이해할 차례입니다.

6.6 스레드 독립성 — 다른 thread_id는 완전히 독립된 상태

각 `thread_id` 는 완전히 독립된 대화 상태를 가집니다. `thread_id="session-1"` 과 `thread_id="session-2"` 는 서로 다른 체크포인트 이력을 가지며, 한쪽 스레드의 메시지를 수정하거나 삭제해도 다른 스레드에는 영향을 주지 않습니다. 이 독립성 덕분에 하나의 그래프 인스턴스로 여러 사용자의 동시 대화를 안전하게 처리할 수 있습니다. 다만, 스레드 간에 정보를 공유해야 하는 경우(예: 사용자 프로필, 선호도)에는 다음 절에서 다루는 `InMemoryStore` 를 사용해야 합니다.

스레드 독립성은 대화 격리에 유용하지만, 때로는 스레드 경계를 넘어 정보를 공유해야 합니다. 이것이 장기 메모리의 역할입니다.

6.7 InMemoryStore — 스레드 간 공유 장기 메모리

체크포인트가 하나의 스레드 안에서 상태를 유지하는 반면, `InMemoryStore` 는 스레드 경계를 넘어 데이터를 공유합니다. 예를 들어, 한 사용자가 여러 대화 스레드를 열더라도 “좋아하는 색상”이나 “프로그래밍 언어 선호도” 같은 정보는 모든 스레드에서 공유되어야 합니다. 체크포인트만으로는 이런 교차 스레드 데이터 공유가 불가능하므로, `InMemoryStore` 라는 별도의 저장소가 필요합니다.

`InMemoryStore` 는 네임스페이스(namespace) 기반의 키-값 저장소로, 데이터를 계층적으로 조직합니다. 네임스페이스는 튜플 형태(예: `( "users", )`, `( "user_123", "memories" )`)로 표현되며, 파일 시스템의 디렉터리 구조와 유사하게 데이터를 분류합니다. 저장된 각 항목은 `value` (실제 데이터), `key` (고유 식별자), `namespace`, `created_at`, `updated_at` 속성을 가집니다.

핵심 API는 다음 세 가지입니다:

- `put(namespace, key, value)` : 네임스페이스와 키로 데이터를 저장합니다. 동일한 키에 다시 `put()` 하면 기존 값이 업데이트됩니다
- `get(namespace, key)` : 특정 항목을 조회합니다. 존재하지 않으면 `None` 을 반환합니다
- `search(namespace)` : 네임스페이스 내 항목을 검색합니다. `query` 파라미터를 전달하면 시맨틱 검색도 가능합니다

TIP

`InMemoryStore` 는 `index` 설정을 통해 임베딩 기반 시맨틱 검색도 지원합니다.

`InMemoryStore(index={"embed": init_embeddings("openai:text-embedding-3-small"), "dims": 1536, "fields": ["food_preference", "$"]})` 형태로 초기화한 뒤, `store.search(namespace, query="...")` 형태로 호출하면 저장된 데이터 중 의미적으로 유사한 항목을 찾을 수 있습니다. `fields` 로 임베딩 대상 키를 제한하거나, `put(..., index=False)` 로 특정 항목의 임베딩을 건너뛸 수 있습니다. 프로덕션에서는 `PostgresStore` / `RedisStore` / `OracleStore` 로 교체해 영속성을 확보하세요.

Context와 Runtime — 노드에서 사용자 식별자와 스토어 접근

장기 메모리를 사용자별로 분리하려면 노드 함수가 현재 사용자의 ID를 알아야 합니다. `LangGraph`는

`@dataclass` 로 정의한 `Context` 를 `StateGraph(..., context_schema=Context)` 로 등록하고, 노드 시그니처에 `runtime: Runtime[Context]` 를 선언하는 방식으로 이를 해결합니다. `graph.invoke(...)` 호출 시 `context=Context(user_id=...)` 를 전달하면, 노드 안에서 `runtime.context.user_id` 로 접근할 수 있고, 동일한 `runtime` 객체에서 `runtime.store` 로 스토어까지 자연스럽게 사용할 수 있습니다.

```
import uuid
from dataclasses import dataclass
from langgraph.runtime import Runtime
from langgraph.graph import StateGraph, MessagesState, START

@dataclass
class Context:
    user_id: str

async def call_model(state: MessagesState, runtime: Runtime[Context]):
    user_id = runtime.context.user_id
    namespace = (user_id, "memories") # docs 권장 형식

    memories = await runtime.store.asearch(
        namespace, query=state["messages"][-1].content, limit=3
```

```

    )
    await runtime.store.aput(
        namespace, str(uuid.uuid4()), {"data": "User prefers dark mode"}
    )
    return {"messages": [await model.ainvoke(state["messages"])]}

builder = StateGraph(MessagesState, context_schema=Context)
builder.add_node(call_model)
builder.add_edge(START, "call_model")
graph = builder.compile(checkpointer=checkpointer, store=store)

graph.invoke(
    {"messages": [{"role": "user", "content": "hi"}]},
    {"configurable": {"thread_id": "1"}},
    context=Context(user_id="alice"),
)

```

TIP

네임스페이스는 튜플 어떤 형태든 가능하지만, 공식 문서는 (user_id, "memories") 패턴을 권장합니다. 사용자 별로 1차 분리하고, 두 번째 원소로 메모리 종류("memories" , "preferences" 등)를 구분하면 search 로 카테고리별 조회가 쉬워집니다.

프로덕션 스토어는 체크포인트와 동일하게 컨텍스트 매니저로 사용합니다.

```

from langgraph.store.postgres import PostgresStore

with PostgresStore.from_conn_string(DB_URI) as store:
    store.setup()
    graph = builder.compile(checkpointer=checkpointer, store=store)

# Redis / Oracle 도 동일 패턴
from langgraph.store.redis import RedisStore
from langgraph.store.oracle import OracleStore # 벡터 검색 기본 지원

```

아래 코드에서는 InMemoryStore 의 기본 CRUD 연산을 실습합니다. 네임스페이스로 사용자 데이터를 분류하고, put(), get(), search() 로 데이터를 관리하는 패턴을 확인하세요.

```

from langgraph.store.memory import InMemoryStore

store = InMemoryStore()

# 데이터 저장
store.put(("users",), "alice", {"favorite_color": "blue", "city": "Seoul"})
store.put(("users",), "bob", {"favorite_color": "red", "city": "Tokyo"})

# 데이터 조회
alice = store.get(("users",), "alice")
print(f"Alice: {alice.value}")

# 검색
results = store.search(("users",))
print(f"\n전체 사용자 ({len(results)}명):")
for item in results:
    print(f"  {item.key}: {item.value}")

```

```

Alice: {'favorite_color': 'blue', 'city': 'Seoul'}

전체 사용자 (2명):
  alice: {'favorite_color': 'blue', 'city': 'Seoul'}
  bob: {'favorite_color': 'red', 'city': 'Tokyo'}

```

독립적으로 `InMemoryStore` 를 사용하는 방법을 배웠으니, 이제 그래프 노드 안에서 스토어에 접근하는 방법을 알아보시다.

6.7.5 InMemoryStore를 그래프와 함께 사용하기

`InMemoryStore` 를 `compile(store=store)` 로 그래프에 전달하면, 각 노드 함수에서 `store` 파라미터를 통해 스토어에 직접 접근할 수 있습니다. LangGraph의 의존성 주입 메커니즘이 노드 함수의 시그니처를 분석하여, `store` 라는 이름의 파라미터가 있으면 자동으로 스토어 인스턴스를 전달합니다.

이 패턴을 사용하면 노드 안에서 사용자 정보를 저장하고 조회하여, 스레드를 넘어 장기 메모리를 유지할 수 있습니다. 예를 들어, 에이전트가 대화 중 파악한 사용자 선호도를 저장하면, 나중에 다른 대화에서도 그 정보를 활용할 수 있습니다.

- `compile(checkpointer=checkpointer, store=store)` : 그래프에 체크포인트와 스토어를 모두 연결합니다. 단기 메모리와 장기 메모리를 동시에 활용하는 전형적인 패턴입니다
- 노드 함수에 `store` 파라미터 추가: LangGraph가 자동으로 스토어 인스턴스를 주입합니다
- `config["configurable"]["user_id"]` 로 사용자별 네임스페이스를 분리하여, 각 사용자의 데이터가 섞이지 않도록 합니다

! WARNING

`InMemoryStore` 도 `InMemorySaver` 와 마찬가지로 메모리 기반이므로, 프로세스 종료 시 모든 데이터가 사라집니다. 프로덕션에서는 `PostgresStore` (`pip install langgraph-checkpoint-postgres`)나 `RedisStore` (`pip install langgraph-checkpoint-redis`)를 사용하세요.

체크포인트와 스토어로 메모리 시스템의 전체 구조를 파악했습니다. 하지만 실제 서비스에서는 대화가 수십, 수백 턴으로 길어질 수 있습니다. 이런 상황에 대비한 메시지 관리 전략을 살펴봅시다.

6.7.6 대화 길이 관리 — `trim_messages`와 `RemoveMessage`

대화가 길어지면 LLM의 컨텍스트 윈도우를 초과하여 오류가 발생하거나, 비용이 급증할 수 있습니다.

LangGraph는 이 문제를 해결하기 위해 두 가지 상호 보완적인 방법을 제공합니다. `trim_messages` 는 LLM에 전달할 메시지를 줄이고, `RemoveMessage` 는 체크포인트에서 메시지를 영구 삭제합니다.

`trim_messages`

- `langchain_core.messages.utils` 에서 제공하는 유틸리티 함수로, 토큰 수 기준으로 오래된 메시지를 자동으로 잘라냅니다
- `strategy="last"` : 최근 메시지만 유지하여 지정된 `max_tokens` 이내로 메시지 목록을 줄입니다
- `start_on="human"` : 잘린 결과가 항상 사용자 메시지로 시작하도록 보장합니다. AI 메시지로 시작하면 LLM이 혼란스러워할 수 있기 때문입니다
- 핵심: 원본 상태는 수정하지 않고, 잘린 메시지 목록만 반환합니다. 체크포인트에는 전체 이력이 그대로 유지되므로, 필요 시 과거 대화를 복원할 수 있습니다

`RemoveMessage`

- `langchain.messages` 에서 제공하는 특수 메시지 타입으로, 특정 메시지를 체크포인트에서 영구적으로 삭제합니다

- `MessagesState`의 리듀서가 `RemoveMessage`를 감지하여 해당 ID의 메시지를 제거합니다
- 오래된 메시지를 정리하여 저장 공간을 절약하거나, 민감한 정보가 포함된 메시지를 삭제할 때 유용합니다
- `RemoveMessage(id=REMOVE_ALL_MESSAGES)`를 사용하면 모든 메시지를 한 번에 삭제할 수도 있습니다

TIP

`trim_messages`와 `RemoveMessage`는 상호 보완적입니다. 일반적으로 `trim_messages`로 LLM에 전달하는 메시지를 줄이되 체크포인트는 유지하고, 저장 공간이 문제되거나 개인정보 삭제가 필요한 경우에만 `RemoveMessage`로 영구 삭제하는 전략을 권장합니다.

6.8 Durable Execution — 실패 시 마지막 체크포인트에서 재개

체크포인트와 메모리 관리를 모두 살펴보았으니, 이제 체크포인트가 제공하는 또 하나의 핵심 기능인 내구성 실행(Durable Execution)을 알아보시다. 내구성 실행이란, 워크플로가 실행 중 핵심 지점마다 진행 상태를 저장하여, 실패하거나 중단되더라도 마지막으로 성공한 체크포인트에서 정확히 재개할 수 있는 기법입니다.

이 기능이 중요한 이유는 실무에서 외부 API 호출, 네트워크 요청 등은 언제든지 실패할 수 있기 때문입니다. 체크포인트를 사용하면 이미 완료된 노드를 다시 실행하지 않으므로 비용과 시간을 절약합니다. 특히 LLM 호출은 비용이 높으므로, 불필요한 재실행을 방지하는 것이 경제적으로도 중요합니다.

LangGraph는 세 가지 내구성 모드를 제공합니다:

- `"exit"`: 그래프가 완료/오류/인터럽트될 때만 체크포인트를 저장합니다. 성능이 가장 좋지만, 중간 단계에서의 복구가 불가능합니다
- `"async"`: 다음 노드를 실행하면서 비동기로 체크포인트를 저장합니다. 성능과 안전성의 균형을 잡지만, 비동기 저장 중 크래시가 발생하면 데이터 손실 위험이 있습니다
- `"sync"`: 다음 노드를 실행하기 전에 동기적으로 체크포인트를 저장합니다. 최대한의 안전성을 제공하지만, 저장 지연으로 인해 성능이 저하될 수 있습니다

아래 예제에서는 3단계 파이프라인을 구성합니다:

1. `step_1`: 데이터 수집 (항상 성공)
2. `step_2`: 데이터 분석 (항상 성공)
3. `step_3`: 외부 API 호출 (첫 번째 실행에서 실패, 재시도 시 성공)

`attempt_count`를 통해 첫 실행에서 `step_3`이 실패하고, 두 번째 `invoke()` 호출 시 `step_1`과 `step_2`를 건너뛰고 `step_3`에서만 재개되는 것을 확인합니다. 이것이 내구성 실행의 핵심 가치입니다 — 이미 성공한 작업은 반복하지 않습니다.

! WARNING

내구성 실행을 위해서는 각 노드(Graph API)나 태스크(Functional API)가 멍등성(idempotency)을 가지는 것이 이상적입니다. 같은 입력으로 여러 번 실행해도 동일한 결과를 내야 재시도 시 부작용이 발생하지 않습니다. 예를 들어, 데이터베이스에 레코드를 삽입하는 노드는 중복 삽입을 방지하는 로직이 필요합니다. 이 주제는 12장에서 더 깊이 다룹니다.

이 장에서 LangGraph의 지속성과 메모리 시스템 전체를 다루었습니다. 체크포인트를 통한 단기 메모리, `InMemoryStore` 를 통한 장기 메모리, `trim_messages` 와 `RemoveMessage` 를 통한 대화 길이 관리, 그리고 내구성 실행을 통한 장애 복구까지 – 프로덕션 에이전트의 상태 관리에 필요한 모든 도구를 갖추었습니다.

요약

개념	설명
체크포인트	각 노드 실행 후 상태를 자동 저장 (<code>InMemorySaver</code> , <code>SqliteSaver</code> , <code>PostgresSaver</code>)
<code>get_state()</code>	현재 스레드의 최신 체크포인트 상태 조회
<code>get_state_history()</code>	스레드의 전체 체크포인트 이력 조회 (최신순)
<code>update_state()</code>	저장된 상태를 프로그래밍 방식으로 수정
스레드 독립성	서로 다른 <code>thread_id</code> 는 완전히 독립된 상태
<code>InMemoryStore</code>	스레드 간 공유되는 키-값 장기 메모리 저장소
<code>compile(store=store)</code>	그래프 노드에서 <code>store</code> 파라미터로 장기 메모리 접근
<code>trim_messages</code>	토큰 수 기준으로 오래된 메시지를 잘라내어 LLM에 전달 (체크포인트 유지)
<code>RemoveMessage</code>	체크포인트에서 특정 메시지를 영구적으로 삭제
Durable Execution	실패 시 마지막 성공 체크포인트에서 재개

⇒ NEXT STEP

다음 장에서는 에이전트 실행 과정을 실시간으로 관찰하는 스트리밍을 다룹니다. `values` , `updates` , `messages` , `custom` , `debug` 다섯 가지 모드의 차이와 적합한 사용 사례를 익힙니다.

07 스트리밍

실시간으로 에이전트 실행 관찰

6장에서 체크포인트와 메모리를 통해 에이전트의 상태 저장을 다루었다면, 이 장에서는 에이전트의 실행 과정을 실시간으로 관찰하는 스트리밍을 다룹니다. 에이전트가 여러 단계를 거쳐 결과를 생성할 때, 최종 응답이 완성될 때까지 사용자를 기다리게 하면 UX가 크게 저하됩니다. 스트리밍을 통해 중간 과정을 실시간으로 보여주면 사용자는 에이전트가 “생각하고 있다”는 느낌을 받으며, 개발자는 에이전트의 내부 동작을 디버깅할 수 있습니다.

LangGraph 는 `values` , `updates` , `messages` , `custom` , `debug` 등 5가지 스트리밍 모드를 제공합니다. 각 모드는 서로 다른 수준의 세밀함(granularity)으로 정보를 전달합니다. `values` 는 전체 상태 스냅샷을, `updates` 는 각 노드의 변경 사항만을, `messages` 는 LLM 토큰 하나하나를 전달합니다. 동기 환경에서는 `graph.stream()` , 비동기 환경에서는 `graph.astream()` 을 사용하며, `stream_mode=["updates", "messages"]` 처럼 여러 모드를 리스트로 전달하여 동시에 사용할 수도 있습니다. 이 장에서는 각 모드의 특성과 적합한 사용 시나리오를 실습을 통해 비교합니다.

학습 목표

LangGraph의 다양한 스트리밍 모드를 이해하고 활용합니다.

- `values` , `updates` , `messages` , `custom` , `debug` 다섯 가지 모드의 차이와 출력 형태를 이해합니다
- 각 스트리밍 모드의 적절한 사용 사례를 파악합니다 (디버깅, 채팅 UI, 진행률 보고 등)
- `stream_mode` 에 리스트를 전달하여 여러 모드를 동시에 사용하는 방법을 익힙니다
- 서브그래프 스트리밍과 태그 기반 필터링 기법을 이해합니다

7.1 환경 설정

7.2 스트리밍 모드 비교

LangGraph는 다양한 스트리밍 모드를 제공합니다. 각 모드는 서로 다른 수준의 세밀함(granularity)으로 정보를 실시간으로 전달합니다. 아래 표에서 각 모드의 특성과 주요 용도를 한눈에 비교할 수 있습니다. 실무에서는 용도에 맞는 모드를 선택하거나, 여러 모드를 조합하여 사용합니다.

모드	설명	용도
<code>values</code>	각 단계의 전체 상태	디버깅, 상태 추적
<code>updates</code>	각 노드가 반환한 업데이트만	진행 상황 표시
<code>messages</code>	메시지 토큰 단위	채팅 UI

custom	사용자 정의 이벤트	커스텀 진행률
debug	전체 디버그 정보	개발 중 디버깅

이제 각 스트리밍 모드를 하나씩 살펴보면, 출력 형태와 적합한 사용 시나리오를 확인합니다.

7.3 stream_mode="values" — 전체 상태 스냅샷

`values` 모드는 각 노드 실행 후 전체 상태(state)를 반환합니다. 그래프가 어떻게 진행되는지 전체적인 흐름을 추적할 때 유용합니다. 매 이벤트마다 전체 상태 딕셔너리가 전달되므로 데이터 양이 많을 수 있지만, 각 시점의 완전한 스냅샷을 얻을 수 있다는 장점이 있습니다. 특히 디버깅 시 “이 노드 실행 후 상태가 정확히 어떤 모습인지” 확인하고 싶을 때 가장 적합합니다.

➔ TIP

`values` 모드는 매 단계마다 전체 상태를 반환하므로, 상태에 긴 메시지 이력이 쌓이면 네트워크 트래픽이 커질 수 있습니다. 프로덕션 환경에서는 `updates` 모드로 변경 사항만 받는 것이 더 효율적입니다. `values` 모드는 주로 개발 중 디버깅 용도로 활용하세요.

`values` 모드가 전체 상태를 보여주었다면, `updates` 모드는 각 노드가 변경한 부분만 추출하여 보여줍니다.

7.4 stream_mode="updates" — 노드별 업데이트

`updates` 모드는 각 노드가 반환한 업데이트 값만 전달합니다. 출력 형태는 `{노드_이름: 반환값}` 딕셔너리입니다. 전체 상태 대신 변경 사항만 전달하므로 데이터 양이 적고, “어떤 노드가 어떤 변경을 만들었는지” 명확히 파악할 수 있습니다. 프로덕션 환경에서 에이전트의 진행 상황을 모니터링하거나, UI에 “검색 중...”, “분석 중...” 같은 단계별 상태를 표시할 때 가장 적합한 모드입니다.

`values` 와 `updates` 가 노드 단위의 스트리밍이었다면, `messages` 모드는 한 단계 더 세밀하게 토큰 단위로 스트리밍합니다.

7.5 stream_mode="messages" — 토큰 단위 스트리밍

`messages` 모드는 LLM이 생성하는 토큰을 실시간으로 전달합니다. 각 이벤트는 `(message_chunk, metadata)` 튜플 형태로, `message_chunk` 에는 생성된 토큰이, `metadata` 에는 어떤 노드(`langgraph_node`)에서 생성되었는지 등의 정보가 포함됩니다. ChatGPT나 Claude 같은 채팅 UI에서 글자가 하나씩 나타나는 타이핑 효과를 구현할 때 가장 적합한 모드입니다.

`metadata` 의 `tags` 나 `langgraph_node` 필드를 사용하면, 특정 LLM 호출이나 특정 노드에서 생성된 토큰만 선택적으로 필터링할 수도 있습니다. 예를 들어, 여러 LLM 호출이 있는 그래프에서 최종 응답 노드의 토큰만 사용자에게 보여주고, 중간 추론 노드의 토큰은 무시하는 등의 제어가 가능합니다.

! WARNING

`messages` 모드는 LangChain의 LLM 통합(예: ChatOpenAI)을 사용해야 동작합니다. 직접 OpenAI API를 호출하거나 LangChain을 사용하지 않는 LLM을 쓰는 경우에는 `messages` 모드 대신 `custom` 모드에서 `get_stream_writer()`를 사용하여 수동으로 토큰을 스트리밍해야 합니다.

지금까지 각 모드를 개별적으로 살펴보았습니다. 하지만 실무에서는 하나의 모드만으로는 부족한 경우가 많습니다.

7.6 여러 스트리밍 모드 동시 사용

예를 들어, 채팅 UI에서 토큰을 실시간으로 보여주면서(`messages`) 동시에 노드별 진행 상황(`updates`)도 추적하고 싶을 수 있습니다. `stream_mode`에 리스트를 전달하면 여러 모드를 동시에 사용할 수 있습니다. 반환되는 이벤트는 `(mode, data)` 튜플 형태이므로, 첫 번째 원소인 `mode` 문자열로 어떤 모드에서 온 이벤트인지 구분하여 처리할 수 있습니다.

이 기능은 복합적인 모니터링 시나리오에서 매우 유용합니다. 예를 들어, 프론트엔드에서는 `messages` 이벤트로 타이핑 효과를 구현하고, 동시에 `updates` 이벤트로 사이드바에 “검색 도구 실행 중...”과 같은 진행 상황을 표시할 수 있습니다.

TIP

복수 모드 사용 시, 이벤트를 모드별로 분기 처리하는 패턴이 일반적입니다:

```
for mode, data in graph.stream(..., stream_mode=["updates", "messages"]):
    if mode == "messages":
        로 분기합니다.
```

마지막으로, 가장 유연한 스트리밍 모드인 `custom`을 살펴보시다. 앞의 네 가지 모드가 LangGraph가 자동으로 생성하는 이벤트를 전달하는 반면, `custom` 모드는 개발자가 직접 원하는 데이터를 스트리밍합니다.

7.7 stream_mode="custom" — 사용자 정의 스트리밍

`custom` 모드는 노드 내부에서 임의의 데이터를 직접 스트리밍할 수 있게 해줍니다. `langgraph.config`의 `get_stream_writer()`를 호출하면 `writer` 함수를 얻을 수 있고, 이 함수에 딕셔너리, 문자열 등 직렬화 가능한 데이터를 전달하면 그래프 실행 중 실시간으로 클라이언트에 전송됩니다.

이 모드의 핵심 가치는 LangGraph의 기본 이벤트로는 표현할 수 없는 정보를 전달할 수 있다는 점입니다. 다음과 같은 시나리오에서 특히 유용합니다:

활용 사례:

- 긴 작업의 진행률(progress) 보고: `writer({"progress": 0.5, "step": "데이터 분석 중"})` 형태로 중간 상태를 전달
- LangChain을 사용하지 않는 외부 LLM의 청크 단위 스트리밍: 직접 OpenAI API나 Anthropic API를 호출할 때, 각 청크를 `writer()`로 전달하면 `messages` 모드 없이도 토큰 스트리밍을 구현할 수 있습니다
- 노드 내부의 중간 결과를 즉시 전달: 예를 들어, 검색 도구가 10개의 결과를 순차적으로 찾을 때, 각 결과를 발견 즉시 전달

TIP

`stream_mode="custom"` 으로 그래프를 스트리밍하면 `writer()` 로 전송한 데이터만 수신됩니다. 상태 업데이트나 메시지 토큰은 포함되지 않습니다. 다른 모드의 이벤트도 함께 받으려면 `stream_mode=["custom", "updates"]` 처럼 복수 모드를 사용하세요.

! WARNING

Python 3.10 이하에서 비동기 함수(`async def`) 안에서 `get_stream_writer()` 를 사용하면 컨텍스트 전파 문제가 발생할 수 있습니다. 이 경우, 노드 함수의 파라미터에 `writer: StreamWriter` 를 직접 선언하여 LangGraph 가 주입하도록 하세요: `async def my_node(state: State, writer: StreamWriter):` .

7.8 nostream 태그 — 내부 LLM 호출을 messages 스트림에서 제외

하나의 그래프에서 사용자에게 보여줄 응답 모델과, 내부 평가·라우팅용 보조 모델을 함께 사용하는 경우가 흔합니다. 이때 보조 모델의 토큰까지 `messages` 스트림에 흘러나가면 UI가 어지러워집니다. 해결책은 보조 모델 호출에 `nostream` 태그를 부여하는 것입니다.

```
from langchain_anthropic import ChatAnthropic

# 보조 모델 - messages 스트림에 노출되지 않음
internal_model = ChatAnthropic(model_name="claude-haiku-4-5-20251001").with_config(
    {"tags": ["nostream"]}
)
```

`tags=["nostream"]` 이 부여된 모델은 `stream_mode="messages"` 에서 자동으로 필터링됩니다. 사용자용 응답 모델만 그대로 두면, UI에는 최종 답변 토큰만 노출됩니다.

7.9 chunk_position — 메시지의 마지막 청크 감지

`messages` 모드의 `metadata`에는 `chunk_position` 필드가 함께 전달됩니다. `chunk_position = "last"` 인 시점에 메시지 한 건이 완결됩니다. 토큰 단위 UI 업데이트는 계속 진행하다가, last 청크에서만 DB 저장이나 후처리 트리거를 걸 수 있습니다.

```
for msg, metadata in graph.stream(inputs, stream_mode="messages"):
    if metadata.get("chunk_position") == "last":
        # 메시지 종료 시점 - 저장/로깅/후처리
        save_final_message(msg, node=metadata["langgraph_node"])
        continue
    print(msg.content, end="", flush=True)
```

`metadata` 에서 자주 활용하는 키는 다음 세 가지입니다.

- `tags` — 모델/체인에 부여한 태그. 예: `["joke"]` , `["nostream"]`
- `langgraph_node` — 현재 토큰을 생성한 노드 이름
- `chunk_position` — `"first"` , `"middle"` , `"last"` 중 하나

7.10 invoke v2 – GraphOutput 반환 타입

`invoke()` 를 `version="v2"` 로 호출하면 결과가 `GraphOutput` 으로 감싸여 반환됩니다. 기존 dict 접근 (`result["key"]`)도 동작하지만 deprecated 경로이므로, 새 코드는 `.value` / `.interrupts` 를 사용합니다.

```
from langgraph.types import GraphOutput

result = graph.invoke(inputs, version="v2")
assert isinstance(result, GraphOutput)

result.value          # 최종 state (dict / Pydantic / dataclass)
result.interrupts     # tuple[Interrupt, ...] - interrupt 없으면 빈 튜플

if result.interrupts:
    payload = result.interrupts[0].value
    print("HITL 응답 대기:", payload)
```

`interrupts` 필드 덕분에 별도 분기 없이 “이번 invoke가 정상 종료인지, interrupt로 멈췄는지”를 한 줄로 판별할 수 있습니다.

7.11 stream_events v3 – projection 기반 재개

LangGraph 1.2+에서는 raw `stream_mode` 위에 event streaming projection 레이어가 추가되었습니다.

`graph.stream_events(..., version="v3")` 는 하나의 run stream 객체를 반환하며, 호출자는 `stream.messages` / `stream.values` / `stream.subgraphs` / `stream.output` 같은 projection만 독립적으로 소비합니다. interrupt로 멈춘 뒤 동일 API로 재개할 수 있다는 점이 v2 raw streaming과의 가장 큰 차이입니다.

```
from langgraph.types import Command

stream = graph.stream_events(
    {"messages": [{"role": "user", "content": "보안 점검 시작"}]},
    version="v3",
)

for message in stream.messages:
    for token in message.text:
        print(token, end="", flush=True)

if stream.interrupted:
    print("\n승인 요청:", stream.interrupts)
    stream = graph.stream_events(
        Command(resume={"decisions": [{"type": "approve"}]}),
        version="v3",
    )
    for message in stream.messages:
        for token in message.text:
            print(token, end="", flush=True)

final_state = stream.output # 최종 state 단일 접근점
```

TIP

v2 raw streaming은 디버깅·custom 모드 직접 처리에, v3 `stream_events` 는 애플리케이션/UI 코드와 typed projection 소비에 적합합니다. 두 API는 공존하므로, 같은 그래프를 디버깅용 v2와 서비스용 v3로 동시에 운영해도 무방합니다.

이 장에서 LangGraph의 5가지 스트리밍 모드를 모두 살펴보았습니다. `values` 는 전체 상태 스냅샷, `updates` 는 노드별 변경 사항, `messages` 는 LLM 토큰, `custom` 은 사용자 정의 데이터, `debug` 는 내부 실행 상세를 제공합니다. 각 모드는 서로 다른 수준의 정보를 제공하므로, 용도에 맞게 선택하거나 조합하는 것이 중요합니다.

요약

스트리밍 모드	설명	용도
<code>values</code>	각 단계 후 전체 상태 반환	디버깅, 상태 추적
<code>updates</code>	노드가 변경한 부분만 반환	진행 상황 모니터링
<code>messages</code>	LLM 토큰 실시간 스트리밍	채팅 UI 구현
여러 모드 동시	리스트로 전달 → <code>(mode, data)</code> 튜플 수신	복합 모니터링
<code>custom</code>	<code>get_stream_writer()</code> 로 임의 데이터 전송	진행률 보고, 외부 LLM
<code>debug</code>	노드 실행의 전체 디버그 정보 (입력, 출력, 메타데이터)	개발 중 상세 디버깅

⇒ NEXT STEP

다음 장에서는 에이전트 실행을 중간에 멈추고 사람의 입력을 받아 재개하는 인터럽트와 타임 트래블을 다룹니다. 스트리밍으로 실행 과정을 관찰하는 것에서 한 걸음 더 나아가, 실행 흐름 자체를 제어하는 방법을 배웁니다.

08 인터럽트와 타임 트래블

실행 중단, 승인, 되감기

7장에서 스트리밍을 통해 에이전트 실행을 관찰하는 방법을 배웠다면, 이 장에서는 실행 흐름을 직접 제어하는 방법을 다룹니다. 자율적으로 동작하는 에이전트에도 사람의 개입이 필요한 순간이 있습니다 – 민감한 작업의 승인, 잘못된 판단의 교정, 또는 이전 분기점으로의 회귀가 그 예입니다.

`LangGraph`의 `interrupt(value)` 함수는 현재 상태를 체크포인트에 저장하고 실행을 일시 중지합니다. `value`는 사용자에게 전달할 메시지(예: “이 작업을 승인하시겠습니까?”)입니다. 이후 `Command(resume=value)`으로 사용자 입력을 전달하면, `interrupt()`가 호출된 지점에서 실행이 재개되며 `resume` 값이 `interrupt()`의 반환값이 됩니다.

`compile()` 단계에서 `interrupt_before=["node_name"]` 또는 `interrupt_after=["node_name"]`을 지정하면, 특정 노드의 실행 전후에 자동으로 인터럽트를 발생시킬 수도 있습니다. 이 장에서는 인터럽트와 함께 체크포인트 히스토리를 활용한 타임 트래블까지 다루어, 에이전트 실행의 완전한 제어권을 확보합니다.

학습 목표

`interrupt()`로 실행을 중단하고, `Command(resume=...)`로 재개합니다. 타임 트래블로 이전 상태로 돌아갑니다.

- Human-in-the-loop 패턴을 구현할 수 있습니다
- `interrupt_before` / `interrupt_after`로 노드 전후에 자동 인터럽트를 설정할 수 있습니다
- Functional API에서도 `interrupt`를 사용할 수 있습니다
- 체크포인트 히스토리를 활용한 타임 트래블을 수행할 수 있습니다
- `update_state()`로 외부에서 상태를 수정할 수 있습니다

8.1 환경 설정

```
from dotenv import load_dotenv
load_dotenv(override=True)
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-5.4-mini")
print("모델 준비 완료")
```

모델 준비 완료

8.2 interrupt() — 실행을 중단하고 사람의 입력을 기다립니다

에이전트가 자율적으로 행동하더라도, 특정 시점에서는 반드시 사람의 판단이 필요합니다. 예를 들어 데이터베이스 삭제 쿼리를 실행하기 전, 또는 외부 API에 결제 요청을 보내기 전에 사용자 확인을 받아야 합니다.

`interrupt()` 는 바로 이런 상황을 위해 설계되었습니다.

- `interrupt(value)` : 현재 상태를 체크포인트에 저장하고 실행을 중단합니다. `value` 는 사용자에게 전달할 메시지로, 보통 승인 요청이나 추가 정보 요청 내용을 담습니다.
- `Command(resume=value)` : 중단된 지점에서 값을 전달하며 재개합니다

이 패턴은 민감한 작업 전에 사람의 승인을 받거나, 추가 정보를 입력받을 때 사용합니다. `interrupt()` 가 호출되면 그래프 실행이 즉시 멈추고, 현재까지의 모든 상태가 체크포인트에 저장됩니다. 이 상태는 서버가 재시작되더라도 유지되므로, 사용자가 몇 시간 후에 응답하더라도 정확히 중단된 지점에서 재개할 수 있습니다.

! WARNING

`interrupt()` 는 반드시 체크포인트가 설정된 그래프에서만 사용할 수 있습니다. 체크포인트 없이 `interrupt()` 를 호출하면 상태를 저장할 수 없어 `RuntimeError` 가 발생합니다. `compile(checkpointer=...)` 설정을 반드시 확인하세요.

8.3 Command(resume=...) — 중단된 실행을 재개합니다

`interrupt()` 로 실행을 멈추는 방법을 배웠으니, 이제 멈춘 실행을 다시 시작하는 방법을 살펴봅시다.

`Command(resume=value)` 를 사용하면 `interrupt()` 가 호출된 지점에서 실행이 재개됩니다. `resume` 에 전달한 값이 `interrupt()` 의 반환값이 됩니다. 이 메커니즘 덕분에 사용자의 입력이 노드 내부의 변수로 자연스럽게 전달됩니다.

예를 들어, 노드 내부에서 `answer = interrupt("이 작업을 승인하시겠습니까?")` 로 중단하고, 사용자가

`Command(resume="승인")` 으로 재개하면 `answer` 변수에 "승인" 문자열이 할당됩니다. 이후 노드는 이 값을 기반으로 분기 로직을 수행할 수 있습니다.

`Command(resume=...)` 에 전달하는 값은 boolean, 텍스트, dict 어떤 형태든 가능합니다. 응답 형태에 따라 패턴이 달라집니다.

```
from langgraph.types import Command

# 1) 단순 boolean - Approval
graph.invoke(Command(resume=True), config=config, version="v2")

# 2) 텍스트 - Review & Edit
graph.invoke(Command(resume="The edited text"), config=config, version="v2")

# 3) 구조화 dict - Tool interrupt / 다중 응답
graph.invoke(
    Command(resume={"action": "approve", "subject": "Updated subject"}),
    config=config,
    version="v2",
)
```

TIP

하나의 노드 안에서 `interrupt()` 를 여러 번 호출할 수도 있습니다. 이 경우 각 `interrupt()` 마다 별도의 `Command(resume=...)` 가 필요하며, 호출 순서대로 재개됩니다. 다만 코드 가독성을 위해 하나의 노드에는 하나의 `interrupt()` 만 두는 것이 권장됩니다.

Common Patterns – 5가지 HITL 시나리오

실무에서 자주 등장하는 인터럽트 패턴은 다음 다섯 가지입니다.

1) Approval – 민감 작업 승인

API 호출/DB 변경 같은 critical action 직전에 중단하고, 승인 여부에 따라 라우팅합니다.

```
from typing import Literal
from langgraph.types import Command, interrupt

def approval_node(state) -> Command[Literal["proceed", "cancel"]]:
    decision = interrupt({
        "question": "Approve this action?",
        "details": state["action_details"],
    })
    return Command(goto="proceed" if decision else "cancel")

graph.invoke(Command(resume=True), config=config, version="v2") # 승인
graph.invoke(Command(resume=False), config=config, version="v2") # 거부
```

2) Review & Edit – LLM 출력 검수

LLM이 생성한 텍스트를 사람이 수정한 뒤 그래프에 다시 주입합니다.

```
def review_node(state):
    edited = interrupt({
        "instruction": "Review and edit this content",
        "content": state["generated_text"],
    })
    return {"generated_text": edited}

graph.invoke(Command(resume="수정된 본문 ..."), config=config, version="v2")
```

3) Tool Interrupt – 툴 실행 직전 승인

@tool 함수 내부에 `interrupt`를 두면, 도구 실행 직전 사람이 승인·편집할 수 있습니다. `resume payload`는 보통 `{"action": "approve", ...}` 형태의 decision dict입니다.

```
from langchain.tools import tool

@tool
def send_email(to: str, subject: str, body: str):
    response = interrupt({
        "action": "send_email",
        "to": to, "subject": subject, "body": body,
        "message": "Approve sending this email?",
    })
    if response.get("action") == "approve":
        return f"Email sent to {response.get('to', to)}"
    return "Email cancelled by user"
```

4) Input Validation – 유효성 검증 루프

루프 안에서 `interrupt`를 반복 호출해, 유효한 값이 들어올 때까지 다시 묻습니다.

```
def get_age_node(state):
    prompt = "What is your age?"
    while True:
        answer = interrupt(prompt)
        if isinstance(answer, int) and answer > 0:
            break
        prompt = f"'{answer}' is not valid. Please enter a positive number."
    return {"age": answer}
```

5) Multiple Interrupts – 병렬 응답 매칭

병렬 노드가 동시에 `interrupt`를 일으키면, `interrupt id`를 키로 하는 `resume map`을 사용해 응답을 일대일로 매칭합니다.

```
result = graph.invoke({"vals": []}, config, version="v2")

resume_map = {
    i.id: f"answer for {i.value}" for i in result.interrupts
}
graph.invoke(Command(resume=resume_map), config, version="v2")
```

8.4 interrupt_before / interrupt_after – 컴파일 시점 인터럽트

코드 내부에 `interrupt()`를 직접 작성하는 방식 외에도, 그래프를 컴파일할 때 특정 노드의 전후에 자동으로 인터럽트를 설정할 수 있습니다. 이 방식은 기존 노드 코드를 수정하지 않고도 인터럽트를 추가할 수 있어, 이미 작성된 노드를 재사용할 때 특히 유용합니다.

`interrupt_before=["node_name"]`을 지정하면 해당 노드가 실행되기 직전에, `interrupt_after=["node_name"]`을 지정하면 노드 실행이 완료된 직후에 그래프가 자동으로 멈춥니다. 두 옵션 모두 `compile()` 메서드에 전달합니다.

TIP

`interrupt_before`와 `interrupt_after`는 노드 코드 내부의 `interrupt()` 호출과 달리, 사용자에게 값을 전달하거나 사용자로부터 값을 받을 수 없습니다. 단순히 “멈추기”만 합니다. 사용자 입력을 받아야 한다면 노드 내부에서 `interrupt(value)`를 직접 호출하세요.

8.5 Functional API에서의 interrupt

Graph API에서 인터럽트를 사용하는 방법을 배웠으니, 이제 Functional API에서의 사용법을 살펴봅시다. Functional API(`@entrypoint`, `@task`)에서도 동일하게 `interrupt()`를 사용할 수 있습니다. `@task` 함수나 `@entrypoint` 함수 내부에서 `interrupt()`를 호출하면 동일한 중단/재개 패턴이 작동합니다.

Functional API의 장점은 데코레이터 기반의 간결한 구문으로 인터럽트를 설정할 수 있다는 점입니다. `@task` 함수는 자동으로 체크포인트 경계가 되므로, 함수 내부 어디서든 `interrupt()`를 호출할 수 있습니다.

! WARNING

Functional API에서 `interrupt()` 를 사용할 때는 반드시 `\@entrypoint` 에 체크포인트가 연결되어 있어야 합니다. 체크포인트 없이 `interrupt()` 를 호출하면 상태를 저장할 수 없어 에러가 발생합니다.

8.6 타임 트래블 — 이전 체크포인트로 되돌아가기

인터럽트와 재개를 통해 실행 흐름을 멈추고 다시 시작하는 방법을 익혔습니다. 이제 한 걸음 더 나아가, 이미 지나간 시점으로 되돌아가는 타임 트래블을 살펴봅시다.

인터럽트가 “실행을 멈추고 재개”하는 메커니즘이라면, 타임 트래블은 “이전 시점으로 되돌아가서 다른 경로를 탐색”하는 메커니즘입니다. LangGraph의 체크포인트 시스템은 그래프 실행의 매 단계(슈퍼스텝)마다 상태를 자동으로 저장합니다. `get_state_history()` 로 이전 체크포인트 목록을 조회하고, 특정 시점의 `config` 를 사용하여 해당 시점에서 그래프를 다시 실행할 수 있습니다.

이 기능은 디버깅에 특히 유용합니다. 에이전트가 잘못된 판단을 한 지점을 찾아 해당 체크포인트로 돌아간 뒤, `update_state()` 로 상태를 수정하고 다시 실행하면 됩니다. Git의 `checkout` 이 특정 커밋으로 되돌아가는 것과 비슷하지만, LangGraph의 타임 트래블은 되돌아간 시점에서 다른 경로로 분기할 수 있다는 점이 다릅니다.

TIP

`get_state_history()` 는 가장 최근 체크포인트부터 역순으로 반환합니다. 각 체크포인트에는 `config` , `values` (상태 값), `next` (다음 실행될 노드), `created_at` (생성 시각) 등의 메타데이터가 포함되어 있어, 어떤 시점으로 돌아갈지 정확히 판단할 수 있습니다.

8.7 update_state() — 타임 트래블 + 상태 수정

타임 트래블로 이전 시점을 확인할 수 있다면, `update_state()` 로 해당 시점의 상태를 수정한 뒤 실행을 재개하는 것도 가능합니다. 이는 디버깅, 테스트, 또는 수동 개입이 필요한 경우에 유용합니다.

`update_state(config, values)` 는 두 개의 인자를 받습니다. `config` 는 수정할 체크포인트를 지정하고, `values` 는 변경할 상태 필드를 딕셔너리로 전달합니다. 선택적으로 `as_node` 인자를 전달하면, 해당 노드가 상태를 업데이트한 것처럼 처리되어 리듀서가 올바르게 적용됩니다.

! WARNING

`update_state()` 는 기존 체크포인트를 변경하는 것이 아니라 새로운 체크포인트를 생성합니다. 따라서 원본 실행 이력은 그대로 보존되며, 수정 후에도 원래 경로로 돌아갈 수 있습니다. 이는 불변(immutable) 로그 방식으로 감사 추적(audit trail)에 적합합니다.

Replay vs Fork — 두 가지 타임 트래블 동작

LangGraph의 타임 트래블은 두 가지 동작으로 나뉩니다.

- **Replay** — 이전 체크포인트에서 그대로 다시 실행합니다. `next` 로 지정된 노드부터 재실행되며, LLM/API 호출도 다시 일어납니다(결과가 달라질 수 있음).
- **Fork** — 이전 체크포인트에서 state를 수정한 새 branch를 만들어 대체 경로를 탐색합니다. 원본 thread는 그대로 보존됩니다.

```
# Replay — checkpoint config 그대로 invoke
history = list(graph.get_state_history(config))
before_joke = next(s for s in history if s.next == ("write_joke",))
replay_result = graph.invoke(None, before_joke.config)

# Fork — update_state로 새 branch 생성
fork_config = graph.update_state(
    before_joke.config,
    values={"topic": "chickens"},
)
fork_result = graph.invoke(None, fork_config)
```

TIP

`update_state()`의 반환값(`fork_config`)을 그대로 `invoke()`에 넘기면, 수정된 새 체크포인트에서 실행이 이어집니다. `before.config`가 아니라 `fork_config`를 사용한다는 점이 중요합니다.

as_node — 업데이트의 출처 노드 지정

`update_state(config, values, as_node="...")`로 “이 업데이트를 발생시킨 것으로 간주할” 노드를 명시할 수 있습니다. 다음 상황에서 유용합니다.

- 병렬 branch 때문에 어느 노드가 update를 만들었는지 모호할 때
- 새 thread에 초기 state를 주입하는 테스트 시나리오
- 의도적으로 특정 노드를 건너뛰고, 그 노드 이후부터 재개하고 싶을 때

```
fork_config = graph.update_state(
    before_joke.config,
    values={"topic": "chickens"},
    as_node="generate_topic", # 이 노드의 후속 노드부터 재개
)
graph.invoke(None, fork_config)
```

Reducer 동작 — 병합 vs 덮어쓰기

`update_state()`로 전달한 `values`는 채널의 reducer 여부에 따라 적용 방식이 달라집니다.

- 리듀서가 설정된 채널(예: `MessagesState`의 `messages` — `add_messages` 리듀서) → 기존 값과 병합(merge). 메시지를 추가하면 누적됩니다.
- 리듀서가 없는 채널(평범한 `TypedDict` 필드 등) → 덮어쓰기(overwrite). 새 값이 그대로 대체합니다.

따라서 메시지 한 건을 교체하려면 `RemoveMessage(id=...)` + 새 메시지를 함께 보내야 하고, 단순 dict 필드는 그냥 `values={"field": new_value}`로 덮어쓰면 됩니다.

요약

이 장에서는 에이전트 실행의 제어권을 확보하는 두 가지 핵심 메커니즘을 학습했습니다. `interrupt()` 와 `Command(resume=...)` 를 통한 Human-in-the-loop 패턴, 그리고 `get_state_history()` 와 `update_state()` 를 통한 타임 트래블입니다. 이 기능들은 체크포인트 시스템 위에 구축되어 있으며, 프로덕션 환경에서 에이전트의 안전성과 디버깅 가능성을 보장합니다.

기능	API	설명
<code>interrupt(value)</code>	양쪽	실행 중단, 값 전달
<code>Command(resume=value)</code>	양쪽	중단 지점에서 재개
<code>get_state_history()</code>	Graph	체크포인트 이력 조회
<code>update_state()</code>	Graph	외부에서 상태 수정

`interrupt`와 타임 트래블은 프로덕션 AI 애플리케이션에서 핵심적인 기능입니다:

- `interrupt`: 민감한 작업 전 사람의 승인을 받을 수 있습니다
- 타임 트래블: 이전 상태로 되돌아가 다른 경로를 탐색할 수 있습니다
- `update_state`: 외부에서 상태를 수정하여 실행 흐름을 조정할 수 있습니다

⇒ NEXT STEP

다음 장에서는 복잡한 워크플로를 모듈화하는 서브그래프를 다룹니다. 독립적인 그래프를 부모 그래프의 노드로 삽입하고, 상태 매핑과 서브그래프 내부의 인터럽트 전파까지 살펴봅니다.

09 서브그래프

그래프 안의 그래프

8장까지 단일 그래프 내에서의 실행 제어를 다루었다면, 이 장에서는 그래프를 구성 단위로 분리하는 서브그래프를 다룹니다. 에이전트 시스템이 복잡해지면 하나의 거대한 그래프로 모든 로직을 관리하기 어려워집니다.

LangGraph의 서브그래프는 독립적으로 컴파일된 그래프를 부모 그래프의 노드로 삽입하여, 관심사 분리와 재사용성을 확보하는 모듈화 전략입니다. 각 서브그래프는 자체 상태 스키마를 가지며, 부모 그래프와는 공유 키를 통해 데이터를 주고받습니다. 서브그래프 내부에서 발생한 인터럽트는 부모 그래프 계층 구조를 따라 전파되므로, Human-in-the-loop 패턴도 서브그래프 경계를 넘어 자연스럽게 작동합니다. 이 장에서는 서브그래프의 설계, 상태 매핑, 그리고 서브그래프 내부의 인터럽트 처리까지 실습합니다.

학습 목표

서브그래프로 복잡한 워크플로를 모듈화합니다.

- 독립적으로 컴파일된 그래프를 부모 그래프의 노드로 삽입할 수 있습니다
- 공유 키(shared key)를 통해 부모-서브그래프 간 데이터를 전달할 수 있습니다
- 래퍼 함수를 사용하여 서로 다른 상태 스키마를 매핑할 수 있습니다
- 서브그래프 내부의 인터럽트 전파 메커니즘을 이해합니다
- `subgraphs=True` 옵션으로 서브그래프 내부를 스트리밍할 수 있습니다

9.1 환경 설정

```
from dotenv import load_dotenv
load_dotenv(override=True)
from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4-mini")
```

9.2 서브그래프 개념

에이전트 시스템의 규모가 커지면, 하나의 그래프에 모든 노드와 엣지를 담는 것은 유지보수와 테스트 측면에서 비효율적입니다. 서브그래프는 이 문제를 해결하는 LangGraph의 핵심 모듈화 전략입니다.

서브그래프는 독립적으로 컴파일되고 테스트 가능한 그래프를 부모 그래프의 노드로 사용하는 패턴입니다. 소프트웨어 공학의 관점에서 함수 추출(extract function)에 비유할 수 있습니다. 하나의 거대한 함수를 여러 작은 함수로 나누듯, 거대한 그래프를 여러 서브그래프로 분리합니다.

- 서브그래프: 다른 그래프의 노드로 사용되는 독립적인 그래프
- 장점: 모듈화, 재사용, 팀별 독립 개발, 독립 테스트 가능

- 각 서브그래프는 자체 상태(State)를 가짐
- 부모 <-> 서브그래프 간 상태는 공유 키로 매핑

서브그래프를 부모 그래프에 추가하는 방법은 두 가지입니다. 부모와 서브그래프의 상태 스키마에 공유 키가 있으면 컴파일된 서브그래프를 `add_node()` 에 직접 전달할 수 있습니다. 상태 스키마가 완전히 다르다면 래퍼 함수를 사용하여 상태를 변환해야 합니다.

TIP

서브그래프는 독립적으로 `compile()` 하여 단독 실행 및 테스트가 가능합니다. 개발 단계에서 서브그래프를 먼저 완성하고 테스트한 뒤, 부모 그래프에 통합하는 보텀업(bottom-up) 개발 방식이 권장됩니다.

9.3 서브그래프 만들기

서브그래프를 만드는 것은 일반 그래프를 만드는 것과 동일합니다. `StateGraph` 로 빌더를 생성하고, 노드와 엣지를 추가한 뒤 `compile()` 로 컴파일합니다. 핵심은 서브그래프가 자체적인 상태 스키마를 가진다는 점입니다. 부모 그래프의 상태와는 독립적으로 설계할 수 있으며, 필요한 데이터만 공유 키를 통해 주고받습니다.

9.4 부모 그래프에 서브그래프 추가

서브그래프를 만들었다면, 이제 부모 그래프에 노드로 추가할 차례입니다. 가장 간단한 방법은 부모와 서브그래프의 상태 스키마가 공유 키를 가지는 경우입니다. 이때는 컴파일된 서브그래프 객체를 `add_node()` 에 직접 전달하면 됩니다. `LangGraph`가 자동으로 공유 키를 통해 데이터를 매핑합니다.

9.4.1 패턴 1: 래퍼 노드를 통한 서브그래프 호출 (상태 스키마가 다른 경우)

공유 키를 통한 직접 추가는 편리하지만, 실무에서는 부모 그래프와 서브그래프의 상태 스키마가 완전히 다른 경우가 훨씬 많습니다. 이때는 래퍼 함수가 필요합니다.

위 예시(9.4)에서는 부모 그래프와 서브그래프가 동일한 키(`text`, `word_count`, `char_count`)를 공유했기 때문에, 컴파일된 서브그래프를 `add_node()` 에 직접 전달할 수 있었습니다.

하지만 실무에서는 부모 그래프와 서브그래프의 상태 스키마가 완전히 다른 경우가 많습니다. 이때는 래퍼 함수(wrapper function)를 사용하여:

1. 부모 상태에서 필요한 필드를 추출하여 서브그래프 입력으로 변환
2. 서브그래프를 실행
3. 서브그래프 출력을 부모 상태 형식으로 매핑

하는 패턴을 사용합니다. 이것이 공식 문서에서 Pattern 1: Call Subgraph Inside a Node라고 부르는 방식입니다.

WARNING

래퍼 함수에서 서브그래프를 호출할 때는 `subgraph.invoke()` 를 사용합니다. 이때 서브그래프의 체크포인트는 부모 그래프의 체크포인트와 별도로 동작합니다. 서브그래프 내부의 중간 상태는 부모 그래프의 체크포인트에 포함되지 않으므로, 서브그래프 내부 디버깅이 필요하다면 `subgraphs=True` 스트리밍을 활용하세요.

9.5 LLM 기반 서브그래프

지금까지 단순한 데이터 처리 서브그래프를 살펴보았습니다. 이제 실전에서 가장 많이 사용되는 패턴인 LLM 기반 서브그래프를 다뤄봅시다.

서브그래프의 진정한 위력은 전문 에이전트를 모듈로 분리할 때 드러납니다. 예를 들어, 검색 에이전트, 코드 작성 에이전트, 데이터 분석 에이전트를 각각 독립적인 서브그래프로 만들고, 오케스트레이터 그래프가 이들을 조율하는 멀티 에이전트 시스템을 구축할 수 있습니다. 각 서브그래프는 자체 시스템 프롬프트, 도구 세트, 상태 스키마를 가지므로 전문 분야에 특화된 동작을 수행합니다.

TIP

멀티 에이전트 시스템에서 서브그래프 간 통신은 부모 그래프의 상태를 통해 이루어집니다. 서브그래프끼리 직접 통신하는 것이 아니라, 부모 그래프가 오케스트레이터 역할을 하여 어떤 서브그래프를 언제 호출할지 결정합니다. 이 패턴은 Part 4에서 멀티 에이전트 아키텍처로 본격적으로 다룹니다.

9.6 서브그래프 스트리밍

서브그래프를 사용할 때 디버깅과 모니터링을 위해 내부 실행 과정을 관찰해야 하는 경우가 있습니다. `stream()` 또는 `astream()` 에 `subgraphs=True` 옵션을 전달하면 서브그래프 내부의 각 노드 실행도 스트리밍으로 받을 수 있습니다.

기본적으로 서브그래프 내부의 실행은 부모 그래프에게 “블랙박스”처럼 보입니다. 부모 그래프는 서브그래프가 반환한 최종 결과만 볼 수 있습니다. `subgraphs=True` 를 설정하면 이 블랙박스를 열어 내부의 각 단계를 실시간으로 관찰할 수 있습니다.

v2 스트리밍(`version="v2"`)에서 서브그래프 청크는 통일된 `StreamPart` dict로 반환됩니다. 청크의 세 필드만 기억하면 됩니다.

```
for chunk in graph.stream(
    {"foo": "foo"},
    subgraphs=True,
    stream_mode="updates",
    version="v2",
):
    print(chunk["type"]) # "updates"
    print(chunk["ns"]) # () = root, ("node_2:<id>",) = subgraph
    print(chunk["data"]) # {"node_name": {"key": "value"}}
```

- `chunk["type"]` — 모드 식별자 (`"updates"` , `"values"` , `"messages"` , ...)
- `chunk["ns"]` — 네임스페이스 튜플. `root`는 `()` , 서브그래프는 `("node_2:<uuid>",)`
- `chunk["data"]` — 모드별 payload

TIP

v1 API에서는 `subgraphs=True` 옵션에 따라 반환 형태(`tuple` vs `dict`)가 달라져 분기 처리가 번거로웠습니다. v2는 항상 같은 `dict` 형태이므로 `root/서브그래프` 코드를 통일할 수 있습니다.

서브그래프 Checkpointer 3모드

서브그래프 컴파일 시 `checkpointer=` 인자에 따라 메모리/인터럽트 동작이 달라집니다.

모드	checkpointer=	동작	적합 시나리오
Per-invocation (기본)	None	매 호출마다 fresh. 단일 invoke 안에서는 부모 체크포인트 상속(인터럽트 지원)	틀로 감싼 subagent, 멀티 에이전트 라우팅
Per-thread (stateful)	True	동일 thread에서 호출 간 state 누적. 대화 이력 유지	장기 대화를 가진 전문 subagent
Stateless	False	체크포인트 없음 – 일반 함수 호출. 인터럽트 미지원	부수효과 없는 순수 변환

! WARNING

Per-thread 서브그래프는 병렬 tool 호출에 취약합니다. LLM이 동일 subagent를 동시에 여러 번 호출하면 같은 namespace에 동시에 쓰기가 발생해 충돌이 납니다.

`ToolCallLimitMiddleware(tool_name="...", run_limit=1)` 로 동시 호출을 제한하세요.

Namespace Isolation – 여러 per-thread 서브그래프 분리

서로 다른 per-thread 서브그래프를 같은 부모 그래프에서 사용하려면, 각각을 고유한 노드 이름으로 감싸 namespace를 분리해야 합니다. `StateGraph(MessagesState).add_node(name, agent)` 패턴이 표준입니다.

```
from langgraph.graph import MessagesState, StateGraph
from langchain.agents import create_agent

def create_sub_agent(model, *, name, **kwargs):
    agent = create_agent(model=model, name=name, **kwargs)
    return (
        StateGraph(MessagesState)
        .add_node(name, agent)          # 고유 name → 안정적 namespace
        .add_edge("__start__", name)
        .compile()
    )

fruit_agent = create_sub_agent(
    "gpt-5.4-mini", name="fruit_agent",
    tools=[fruit_info], prompt="You are a fruit expert.",
    checkpointer=True,
)
veggie_agent = create_sub_agent(
    "gpt-5.4-mini", name="veggie_agent",
    tools=[veggie_info], prompt="You are a veggie expert.",
    checkpointer=True,
)
```

각 subagent는 자기 이름과 동일한 namespace에서만 체크포인트를 쓰므로, 서로의 state를 침범하지 않습니다.

➔ TIP

서브그래프 스트리밍 시 각 이벤트에는 서브그래프의 네임스페이스 경로가 포함됩니다. 예를 들어 ("parent:sub_agent",) 처럼 계층 구조가 표시되어, 어느 레벨의 어느 서브그래프에서 발생한 이벤트인지 정확히 구분할 수 있습니다.

9.7 서브그래프 내부의 인터럽트 전파

서브그래프의 강력한 특성 중 하나는 인터럽트 전파입니다. 서브그래프 내부의 노드에서 `interrupt()` 가 호출되면, 이 인터럽트는 부모 그래프 계층 구조를 따라 위로 전파됩니다. 부모 그래프에서 `Command(resume=...)` 로 재개하면, 서브그래프 내부의 정확한 중단 지점에서 실행이 재개됩니다.

이 메커니즘 덕분에 8장에서 배운 Human-in-the-loop 패턴이 서브그래프 경계를 넘어 자연스럽게 작동합니다. 서브그래프를 설계할 때 인터럽트를 포함해도, 부모 그래프 쪽에서 별도의 처리 코드를 작성할 필요가 없습니다.

! WARNING

서브그래프 내부에서 인터럽트가 발생하면, 부모 그래프의 해당 슈퍼시스템 전체가 중단됩니다. 같은 슈퍼시스템에서 병렬 실행 중인 다른 서브그래프가 있다면, 해당 서브그래프의 실행도 함께 중단되므로 주의가 필요합니다.

이 장에서 서브그래프를 통한 워크플로 모듈화를 다루었습니다. 독립적인 상태 스키마, 공유 키를 통한 데이터 매핑, 래퍼 함수를 통한 상태 변환, 그리고 서브그래프 내부의 스트리밍과 인터럽트 전파까지 – 대규모 에이전트 시스템을 구축하는 데 필요한 모듈화 전략을 갖추었습니다.

요약

개념	설명
서브그래프	독립적으로 컴파일된 그래프를 노드로 사용
공유 키	부모-서브그래프 간 상태 매핑
모듈화	복잡한 워크플로를 작은 단위로 분리
스트리밍	<code>subgraphs=True</code> 로 내부 단계 추적

⇒ NEXT STEP

다음 장에서는 개발 환경에서 검증된 에이전트를 프로덕션으로 전환하는 과정을 다룹니다. `langgraph.json` 설정, 테스트 전략, 배포 옵션, 그리고 LangSmith 기반 관측성까지 살펴봅니다.

10 프로덕션

테스트, 배포, 관측성

9장까지 에이전트의 설계, 구현, 모듈화를 다루었다면, 이제 실제 사용자에게 서비스하기 위한 프로덕션 전환을 다룰 차례입니다. 에이전트를 개발 환경에서 성공적으로 실행하는 것과 프로덕션에서 안정적으로 운영하는 것은 전혀 다른 문제입니다. `langgraph.json` 설정, `LangGraph Platform` 배포, `LangSmith` 기반 트레이싱과 평가는 실 환경 운영의 세 기둥입니다. 이 장에서는 앱 구조 설정부터 단위 테스트, 회귀 테스트, 그리고 관측성 확보까지 프로덕션 전환에 필요한 전체 과정을 다룹니다.

학습 목표

LangGraph 앱을 테스트, 배포, 모니터링하는 방법을 알아봅니다.

- `langgraph.json` 으로 프로젝트 구조를 설정할 수 있습니다
- `langgraph dev` 로 로컬 개발 서버를 실행하고 LangGraph Studio로 디버깅할 수 있습니다
- 결정론적 테스트와 `GenericFakeChatModel` 을 활용한 에이전트 테스트를 작성할 수 있습니다
- Python SDK로 배포된 LangGraph 서버를 호출할 수 있습니다
- LangSmith 트레이싱으로 프로덕션 관측성을 확보할 수 있습니다

10.1 환경 설정

```
from dotenv import load_dotenv
load_dotenv(override=True)
from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
```

10.2 앱 구조 – `langgraph.json`

LangGraph 앱을 배포하려면 먼저 프로젝트 구조를 `langgraph.json` 설정 파일로 정의해야 합니다. 이 파일은 LangGraph CLI와 Platform이 앱을 인식하고 실행하는 데 필요한 모든 정보를 담고 있습니다.

- `langgraph.json` : 그래프 정의, 의존성, 환경 변수 설정
- `dependencies` : 프로젝트의 Python 패키지 의존성 경로
- `graphs` : 그래프 이름과 해당 Python 모듈 경로 매핑 (형식: `"./module.py:variable"`)
- `env` : 환경 변수 파일 경로
- `langgraph dev` : 로컬 개발 서버 실행

```
import json

config = {
    "dependencies": [".",],
    "graphs": {
        "agent": "./agent.py:graph"
    },
    "env": ".env"
}
print("langgraph.json 예시:")
print(json.dumps(config, indent=2))
print()
print("명령어:")
print("  $ pip install 'langgraph-cli[inmem]')
print("  $ langgraph dev # http://localhost:2024에서 로컬 서버 시작")
```

```
langgraph.json 예시:
{
  "dependencies": [
    "."
  ],
  "graphs": {
    "agent": "./agent.py:graph"
  },
  "env": ".env"
}

명령어:
$ pip install 'langgraph-cli[inmem]'
$ langgraph dev # http://localhost:2024에서 로컬 서버 시작
```

10.3 LangGraph Studio – 시각적 디버깅 도구

앱 구조를 설정했으니, 이제 시각적으로 그래프를 디버깅할 수 있는 LangGraph Studio를 살펴봅시다.

Studio는 `langgraph dev` 실행 시 자동으로 제공되는 웹 기반 디버깅 도구입니다. 그래프의 구조를 시각적으로 확인하고, 실행 과정을 단계별로 추적하며, 인터럽트 지점에서 상태를 직접 수정할 수 있습니다. 개발 과정에서 `print()` 디버깅 대신 Studio를 사용하면 에이전트의 의사결정 과정을 훨씬 직관적으로 파악할 수 있습니다.

Key Features (7가지):

1. Real-time Visualization – 프롬프트, 도구 호출, 결과, 최종 출력 등 모든 단계가 실시간으로 렌더링
2. Interactive Testing – 다양한 입력으로 실행하고 중간 상태를 UI에서 직접 검사
3. Hot-reloading – 프롬프트나 도구 시그니처 수정이 서버 재시작 없이 즉시 반영
4. Trace Inspection – 프롬프트, 도구 인자, 반환값, 토큰 수, 지연 시간 추적
5. Exception Capture – 예외 발생 시 주변 상태와 함께 캡처되어 디버깅에 활용
6. Thread Replay – 대화 스레드를 임의 지점부터 다시 실행하여 변경 검증
7. Optional Tracing – `LANGSMITH_TRACING=false` 로 외부 전송 없이 로컬 실행만 유지

TIP

LangGraph Studio는 로컬에서 `langgraph dev` 를 실행할 때뿐만 아니라, LangSmith에 배포된 원격 에이전트에도 연결할 수 있습니다. 프로덕션 환경에서 발생한 문제를 Studio로 재현하고 디버깅할 수 있어, 운영 중 트러블슈팅에 매우 유용합니다.

사용 방법:

```
$ langgraph dev
# 브라우저에서 http://localhost:2024 접속
# 또는 LangSmith Studio에서 원격 접속
```

10.4 Agent Chat UI

Studio가 개발자를 위한 디버깅 도구라면, Agent Chat UI는 사용자 관점에서 에이전트와 상호작용하는 Next.js 기반 채팅 인터페이스입니다. `create_agent` 로 만든 에이전트와 바로 연동되며, `langgraph dev` 서버나 배포된 서버 모두에 연결할 수 있습니다.

설치 — npx 사용 (권장):

```
$ npx create-agent-chat-app --project-name my-chat-ui
$ cd my-chat-ui
$ pnpm install
$ pnpm dev
```

Hosted 버전: agentchat.vercel.app 에 접속해 에이전트 deployment URL이나 로컬 서버 주소를 입력합니다.

연결 정보:

- Graph ID — `langgraph.json` 의 `graphs` 섹션 키
- Deployment URL — 에이전트 서버 주소 (로컬은 `http://localhost:2024`)
- LangSmith API key — 선택 (로컬 서버 사용 시 불필요)

기능:

- 실시간 스트리밍 채팅 — 7장에서 다룬 토큰 단위 스트리밍을 UI로 확인
- 도구 호출 / 결과 렌더링 — 에이전트가 어떤 도구를 호출했는지 실시간 표시
- Time-travel debugging, state forking — 대화의 특정 시점에서 다른 경로로 분기
- Human-in-the-loop 승인 — 8장의 인터럽트 패턴을 UI에서 직접 테스트
- Generative UI 지원 — 멀티 에이전트 응답을 시각적으로 구분

10.5 테스트 — 결정론적 에이전트 테스트

개발 서버와 UI 도구를 갖추었으니, 이제 에이전트의 품질을 보장하는 테스트 전략을 살펴봅시다.

LLM 기반 에이전트는 비결정적 특성 때문에 테스트가 까다롭습니다. 같은 입력을 주어도 매번 다른 응답이 나올 수 있기 때문입니다. LangGraph에서는 두 가지 전략으로 이 문제를 해결합니다. 첫째, LLM 호출이 없는 순수 로직을 분리하여 결정론적으로 테스트합니다. 둘째, `GenericFakeChatModel` 로 LLM 응답을 미리 지정하여 전체 에이전트 흐름을 제어된 환경에서 검증합니다.

첫 번째 전략부터 살펴보겠습니다. 노트 함수에서 LLM 호출과 비즈니스 로직을 분리하면, 비즈니스 로직 부분은 일반적인 단위 테스트로 검증할 수 있습니다.

```
from langgraph.graph import StateGraph, START, END
from typing import TypedDict

# Graph to test
class TestState(TypedDict):
```



```

input: str
output: str

def process(state: TestState) -> dict:
    return {"output": state["input"].upper()}

builder = StateGraph(TestState)

builder.add_node("process", process)
builder.add_edge(START, "process")
builder.add_edge("process", END)

graph = builder.compile()

# Unit tests
def test_process():
    result = graph.invoke({"input": "hello"})

    assert result["output"] == "HELLO", f"HELLO 예상, {result['output']} 반환됨"

    print(" OK test_process")

def test_empty_input():
    result = graph.invoke({"input": ""})

    assert result["output"] == "", f"빈 문자열 예상, {result['output']} 반환됨"

    print(" OK test_empty_input")

print("테스트 실행 중:")

test_process()
test_empty_input()

print("모든 테스트 통과!")

```

```

테스트 실행 중:
OK test_process
OK test_empty_input
모든 테스트 통과!

```

10.6 LLM 에이전트 테스트 – GenericFakeChatModel 사용

순수 로직 테스트만으로는 에이전트의 전체 동작을 검증하기 어렵습니다. 도구 호출, 조건부 분기, ReAct 루프 등 LLM의 응답에 따라 달라지는 흐름도 테스트해야 합니다.

순수 로직 테스트를 넘어, 도구 호출을 포함한 전체 에이전트 흐름을 테스트하려면 LLM 응답을 제어해야 합니다.

`langchain_core.language_models.GenericFakeChatModel` 은 미리 정의된 응답을 순차적으로 반환하는 가짜 모델입니다. 도구 호출을 포함한 `AIMessage` 를 응답으로 지정하면, 도구 호출 -> 결과 처리까지의 전체 ReAct 루프를 결정론적으로 테스트할 수 있습니다.

! WARNING

`GenericFakeChatModel` 의 `messages` 인자에는 `iter()` 를 사용하여 이터레이터를 전달합니다. 응답 리스트보다 호출 횟수가 많으면 `StopIteration` 에러가 발생하므로, 에이전트의 LLM 호출 횟수를 정확히 파악하고 그에 맞는 수의 응답을 준비해야 합니다.

```
from langchain_core.language_models import GenericFakeChatModel
from langchain.messages import AIMessage, HumanMessage, AnyMessage
from langgraph.graph import StateGraph, START, END, MessagesState

# Deterministic fake model
fake_model = GenericFakeChatModel(
    messages=iter(
        [
            AIMessage(content="The answer is 42."),
        ]
    )
)

def chatbot(state: MessagesState) -> dict:
    return {
        "messages": [fake_model.invoke(state["messages"])]
    }

builder = StateGraph(MessagesState)

builder.add_node("chatbot", chatbot)
builder.add_edge(START, "chatbot")
builder.add_edge("chatbot", END)

test_graph = builder.compile()

result = test_graph.invoke(
    {
        "messages": [HumanMessage(content="테스트")]
    }
)

assert "42" in result["messages"][-1].content

print("GenericFakeChatModel 테스트 통과!")
print(f"  응답: {result['messages'][-1].content}")
```

GenericFakeChatModel 테스트 통과!
응답: The answer is 42.

10.7 배포 옵션

테스트를 통과한 에이전트를 실제 사용자에게 제공하려면 배포가 필요합니다. LangGraph는 세 가지 배포 옵션을 제공하며, 팀의 인프라 역량과 보안 요구사항에 따라 선택할 수 있습니다.

1. LangSmith Cloud (managed):

GitHub 저장소를 LangSmith Deployments에서 연결하면 자동 배포됩니다 (약 15분 소요). 배포 흐름은 다음과 같습니다:

1. 애플리케이션 코드를 GitHub 저장소(공개/비공개)에 푸시
2. LangSmith → Deployments → “+ New Deployment” 클릭
3. 비공개 저장소는 GitHub 계정 연결 후 저장소 선택, 제출
4. 배포 완료 후 Studio 버튼으로 그래프 확인, Deployment details에서 API URL 복사

2. Self-hosted Docker:

자체 인프라에 Docker 컨테이너로 배포합니다. 데이터가 외부로 나가지 않아야 하는 보안 민감한 환경에 적합합니다.

```
$ langgraph build -t my-agent
$ docker run -p 2024:2024 my-agent
```

3. 배포 후 API 호출 (Python SDK):

배포 URL과 LangSmith API key를 전달해 동일한 SDK로 호출합니다.

```
from langgraph_sdk import get_sync_client

client = get_sync_client(url="your-deployment-url", api_key="your-langsmith-api-key")

for chunk in client.runs.stream(
    None,                                     # thread_id=None → stateless run
    "agent",                                  # langgraph.json의 graph name
    input={"messages": [{"role": "human", "content": "What is LangGraph?"}]},
    stream_mode="updates",
):
    print(f"Receiving new event of type: {chunk.event}...")
    print(chunk.data)
```

REST 호출: 배포 서버는 `X-API-Key` 헤더로 LangSmith API key를 인증합니다.

```
curl -s --request POST \
  --url <DEPLOYMENT_URL>/runs/stream \
  --header 'Content-Type: application/json' \
  --header "X-API-Key: <LANGSMITH_API_KEY>" \
  --data '{"assistant_id": "agent", "input": {"messages": [{"role": "human", "content": "What is LangGraph?"}]}, "stream_mode": "updates"}'
```

TIP

어떤 배포 옵션을 선택하든, 배포된 에이전트는 동일한 REST API를 노출합니다. 따라서 Python SDK(`langgraph-sdk`)나 HTTP 클라이언트를 사용하여 동일한 방식으로 에이전트를 호출할 수 있습니다. 개발 단계에서 `langgraph dev` 로 테스트한 코드가 프로덕션에서도 동일하게 동작합니다.

10.8 관측성 — LangSmith 트레이싱

배포 후 에이전트가 실제 사용자 요청을 처리할 때, 내부에서 무슨 일이 일어나는지 파악하는 것이 관측성(observability)입니다. LangSmith는 LangChain/LangGraph 생태계의 관측성 플랫폼으로, 환경 변수 두 줄만 설정하면 모든 실행이 자동으로 추적됩니다.

설정 (`.env`):

```
LANGSMITH_API_KEY=lsv2-...
LANGSMITH_TRACING=true
```

이 두 줄이 설정되면, LangGraph의 모든 노드 실행, LLM 호출, 도구 호출이 자동으로 LangSmith 대시보드에 기록됩니다. 코드 변경 없이 관측성이 확보되는 것이 핵심입니다.

자동 추적 항목:

- 각 노드 실행 시간 - 병목 구간 식별에 활용
- LLM 입출력, 토큰 사용량 - 비용 최적화에 활용
- 도구 호출 및 결과 - 도구 실패 원인 분석
- 상태 변화 - 각 슈퍼시스템에서의 상태 전이 추적
- 에러 및 재시도 - 장애 원인 분석 및 알림 설정

! WARNING

프로덕션에서 LangSmith 트레이싱을 활성화하면 모든 LLM 입출력이 기록됩니다. 개인정보가 포함된 데이터를 처리하는 경우, LangSmith의 데이터 보존 정책과 조직의 개인정보 처리 방침을 반드시 확인하세요. 필요시 `hide_inputs` / `hide_outputs` 옵션으로 민감한 데이터를 필터링할 수 있습니다.

Selective tracing — `tracing_context`

특정 구간만 켜거나 끄려면 `langsmith.tracing_context` 컨텍스트 매니저를 사용합니다. 프로젝트 / 태그 / 메타데이터를 동적으로 지정할 수도 있습니다.

```
import langsmith as ls

# 이 호출만 트레이싱
with ls.tracing_context(enabled=True):
    agent.invoke({"messages": [{"role": "user", "content": "Send a test email"}]})

# 동적 프로젝트 + 태그 + 메타데이터
with ls.tracing_context(
    project_name="email-agent-test",
    enabled=True,
    tags=["production", "email-assistant", "v1.0"],
    metadata={"user_id": "user_123", "session_id": "session_456"},
):
    agent.invoke({"messages": [{"role": "user", "content": "Send a welcome email"}]})
```

`invoke()` 의 `config` 인자로 태그/메타데이터를 한 번에 주입할 수도 있습니다.

```
agent.invoke(
    {"messages": [{"role": "user", "content": "Send a welcome email"}]},
    config={
        "tags": ["production", "email-assistant", "v1.0"],
        "metadata": {"user_id": "user_123", "session_id": "session_456"},
    },
)
```

Data privacy — `LangChainTracer` + `with_config`

민감 정보를 트레이스에 남기지 않으려면 `LangChainTracer` 에 anonymizer를 적용한 `Client` 를 주입하고, 컴파일된 그래프에 `.with_config({"callbacks": [tracer]})` 로 부착합니다.

```
from langchain_core.tracers.langchain import LangChainTracer
from langgraph.graph import StateGraph, MessagesState
from langsmith import Client
```

```

from langsmith.anonymizer import create_anonymizer

anonymizer = create_anonymizer([
    {"pattern": r"\b\d{3}-?\d{2}-?\d{4}\b", "replace": "<ssn>"},
])

tracer_client = Client(anonymizer=anonymizer)
tracer = LangChainTracer(client=tracer_client)

graph = (
    StateGraph(MessagesState)
    # .add_node(...).add_edge(...)
    .compile()
    .with_config({"callbacks": [tracer]})
)

```

위 패턴은 SSN처럼 정규식으로 식별 가능한 패턴을 로깅 직전에 마스킹한 뒤 LangSmith로 전송합니다.

10.9 Pregel 런타임 개요

배포와 관측성을 다루었으니, LangGraph의 내부 엔진을 잠시 살펴봅시다. 지금까지 사용한 모든 기능 – 상태 관리, 체크포인트, 인터럽트, 스트리밍 – 이 하나의 실행 엔진 위에서 동작합니다. 이 섹션은 13장에서 깊이 다룰 내용의 미리보기입니다.

- Pregel은 LangGraph의 내부 실행 엔진으로, Google의 Pregel 논문(2010)에서 영감을 받은 메시지 패싱 기반 그래프 처리 프레임워크입니다
- Graph API와 Functional API 모두 Pregel 위에서 실행됨
- 핵심 개념: 슈퍼스텝(실행 단위), 채널(노드 간 통신), 체크포인트(상태 저장)
- 슈퍼스텝: 동일 레벨의 노드가 병렬 실행되는 단위. 각 슈퍼스텝은 Plan -> Execute -> Update의 3단계를 거칩니다
- 일반적으로 직접 사용할 필요 없음 (Graph/Functional API가 추상화)

LangGraph 실행 모델:

```

[Super-step 1] Node A, Node B (병렬)
  ↓ 상태 업데이트
[Super-step 2] Node C (A, B 결과 기반)
  ↓ 상태 업데이트
[Super-step 3] Node D
  ↓
END

```

각 슈퍼스텝:

1. 해당 노드들 병렬 실행
2. 상태 업데이트 (리듀서 적용)
3. 체크포인트 저장
4. 다음 슈퍼스텝 결정

10.10 Python SDK로 서버 호출

배포된 LangGraph 서버는 REST API를 노출하며, Python SDK(`langgraph-sdk`)를 통해 프로그래밍 방식으로 호출할 수 있습니다. SDK는 `langgraph dev` 로 실행한 로컬 서버와 클라우드에 배포된 서버 모두에 동일한 인터페이스로 연결됩니다.

SDK의 주요 메서드는 다음과 같습니다:

- `client.assistants.search()` — 등록된 에이전트 목록 조회
- `client.threads.create()` — 새 대화 스레드 생성
- `client.runs.create()` — 스레드에서 에이전트 실행
- `client.runs.stream()` — 스트리밍 방식으로 에이전트 실행
- `client.threads.get_state()` — 현재 상태 조회
- `client.threads.update_state()` — 외부에서 상태 수정

TIP

Python SDK는 비동기(`async`) 클라이언트도 제공합니다. `from langgraph_sdk import get_client` 로 동기 클라이언트를, `from langgraph_sdk.aio import get_client` 로 비동기 클라이언트를 생성할 수 있습니다. 웹 서버 등 비동기 환경에서는 비동기 클라이언트를 사용하세요.

10.11 프로덕션 체크리스트

항목	도구	설명
단위 테스트	pytest	개별 노드 함수 테스트
통합 테스트	GenericFakeChatModel	API 호출 없이 전체 흐름
지속성	PostgresSaver	프로덕션 체크포인트
관측성	LangSmith	트레이싱, 모니터링
배포	langgraph deploy	관리형 배포
UI	Agent Chat UI	사용자 인터페이스

요약

주제	핵심 내용
앱 구조	<code>langgraph.json</code> 으로 프로젝트 설정
Studio	<code>langgraph dev</code> 로 시각적 디버깅

테스트	결정론적 테스트 + GenericFakeChatModel
배포	Platform, Docker, Cloud 옵션
관측성	LangSmith 트레이싱
런타임	Pregel 슈퍼스텝 실행 모델 – 13장에서 심화

⇒ NEXT STEP

다음 장에서는 `langgraph dev` CLI로 로컬 개발 서버를 실행하고, LangGraph Studio와 Python SDK를 통해 에이전트를 인터랙티브하게 테스트하는 방법을 다룹니다.

11 로컬 서버

10장에서 프로덕션 전환의 전체 그림을 그렸다면, 이 장에서는 그 첫 단계인 로컬 개발 서버를 심층적으로 다룹니다. 프로덕션 배포 전에 로컬 환경에서 에이전트를 서버로 실행하고 테스트하는 것은 개발 워크플로의 핵심 단계입니다. `langgraph dev` CLI는 인메모리 체크포인트를 내장한 개발 서버를 즉시 띄워 주며, `LangGraph Studio` 와 연동하면 그래프의 노드-엣지 구조를 시각적으로 확인하고, 실시간 실행 추적, 상태 검사/수정, 타임 트래블 등을 브라우저에서 수행할 수 있습니다. 이 장에서는 로컬 서버의 설정, 실행, Python SDK를 통한 호출, 그리고 Studio 연동까지의 전체 흐름을 실습합니다.

학습 목표

LEARNING OBJECTIVES

- `langgraph dev` CLI로 개발 서버를 실행하는 방법을 안다
- `LangGraph Studio`와 연동하여 시각적으로 디버깅한다
- `langgraph.json` 설정 파일을 작성한다
- Python SDK로 로컬 서버를 호출한다
- 배포 준비 과정을 이해한다

11.1 환경 설정

```
from dotenv import load_dotenv
load_dotenv(override=True)
from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
```

11.2 LangGraph CLI 설치

`LangGraph Platform`은 에이전트를 HTTP 서버로 배포하기 위한 인프라 계층입니다. 로컬 개발 단계에서 이 인프라를 활용하려면 `langgraph-cli` 를 설치해야 합니다. CLI는 프로젝트 생성, 개발 서버 실행, Docker 기반 빌드 등을 담당하는 핵심 도구입니다.

`langgraph-cli[inmem]` 패키지에는 인메모리 체크포인트가 내장되어 있어, 별도의 데이터베이스 설정 없이 즉시 개발 서버를 띄울 수 있습니다. 프로덕션 배포에는 PostgreSQL 기반 체크포인트를 사용하지만, 로컬 개발 단계에서는 인메모리 모드로 충분합니다.

· NOTE

`langgraph-cli[inmem]` 의 `[inmem]` extra는 개발 편의를 위한 것입니다. 이 옵션 없이 설치하면 `langgraph dev` 실행 시 별도의 체크포인트 설정이 필요합니다. 개발 환경에서는 항상 `[inmem]` 을 포함하여 설치하세요.

```
# LangGraph CLI 설치 명령어
print("=== pip으로 설치 (Python >= 3.11) ===")
print('  $ pip install -U "langgraph-cli[inmem]"')
print()
print("=== uv로 설치 ===")
print('  $ uv add "langgraph-cli[inmem]"')
print()
print("설치 후 확인:")
print("  $ langgraph --version")
```

```
=== pip으로 설치 (Python >= 3.11) ===
  $ pip install -U "langgraph-cli[inmem]"

=== uv로 설치 ===
  $ uv add "langgraph-cli[inmem]"

설치 후 확인:
  $ langgraph --version
```

11.3 프로젝트 생성

CLI 설치가 완료되었으면, 다음 단계는 프로젝트를 생성하는 것입니다. LangGraph CLI의 `langgraph new` 명령은 검증된 프로젝트 템플릿을 기반으로 디렉터리 구조, 설정 파일, 의존성 정의를 자동으로 생성해 줍니다. 템플릿을 지정하지 않으면 인터랙티브 메뉴가 표시되어 Python/TypeScript 등 원하는 스택을 선택할 수 있습니다.

```
# 프로젝트 생성 명령어
print("=== 템플릿으로 새 프로젝트 생성 ===")
print('  $ langgraph new my-agent --template new-langgraph-project-python')
print()
print("=== 인터랙티브 메뉴로 생성 ===")
print('  $ langgraph new my-agent')
print()
print("=== 생성 후 의존성 설치 ===")
print("  # pip 사용")
print('  $ cd my-agent && pip install -e .')
print()
print("  # uv 사용")
print('  $ cd my-agent && uv sync')
print()
print("=== 생성되는 파일 구조 ===")
print("  my-agent/")
print("    ├── langgraph.json  # 그래프 설정")
print("    ├── .env.example    # 환경 변수 템플릿")
print("    ├── pyproject.toml  # 의존성 정의")
print("    └── src/")
print("        └── agent.py    # 에이전트 코드")
```

```

=== 템플릿으로 새 프로젝트 생성 ===
$ langgraph new my-agent --template new-langgraph-project-python

=== 인터랙티브 메뉴로 생성 ===
$ langgraph new my-agent

=== 생성 후 의존성 설치 ===
# pip 사용
$ cd my-agent && pip install -e .

# uv 사용
$ cd my-agent && uv sync

=== 생성되는 파일 구조 ===
my-agent/
├── langgraph.json  # 그래프 설정
├── .env.example    # 환경 변수 템플릿
├── pyproject.toml  # 의존성 정의
├── src/
│   └── agent.py    # 에이전트 코드

```

11.4 langgraph.json 설정

프로젝트 구조가 준비되었으니, 이제 LangGraph 서버가 프로젝트를 어떻게 인식하는지 살펴봅시다.

`langgraph.json` 은 LangGraph 프로젝트의 핵심 설정 파일로, 서버 시작 시 가장 먼저 읽히는 진입점입니다. 이 파일이 없으면 `langgraph dev` 명령이 동작하지 않습니다.

`langgraph.json` 은 세 가지 핵심 정보를 정의합니다: (1) 어떤 패키지를 설치할지(`dependencies`), (2) 어떤 그래프를 노출할지(`graphs`), (3) 어떤 환경 변수를 로드할지(`env`). 특히 `graphs` 필드는 "이름": "모듈경로:변수명" 형태로 그래프 객체의 위치를 정확히 지정합니다.

```

import json

# langgraph.json 설정 예시
config = {
    "dependencies": [".",],
    "graphs": {
        "agent": "./src/agent.py:graph"
    },
    "env": ".env"
}

print("langgraph.json 예시:")
print(json.dumps(config, indent=2))
print()
print("주요 필드:")
print('  dependencies: 설치할 패키지 경로 목록')
print('  graphs:      그래프 이름 → 모듈:변수 매핑')
print('  env:         환경 변수 파일 경로')

```

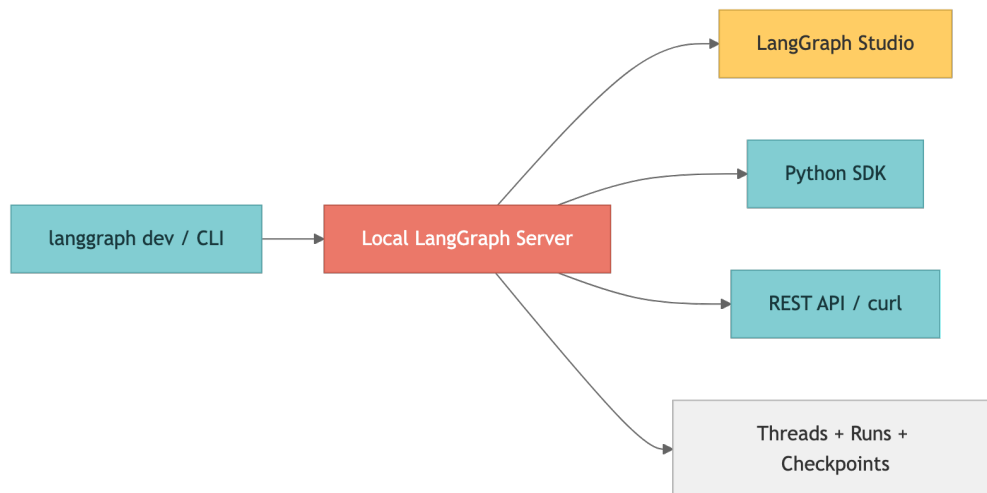
```

langgraph.json 예시:
{
  "dependencies": [
    "."
  ],
  "graphs": {
    "agent": "./src/agent.py:graph"
  },
  "env": ".env"
}

주요 필드:
dependencies: 설치할 패키지 경로 목록
graphs:      그래프 이름 → 모듈:변수 매핑
env:         환경 변수 파일 경로

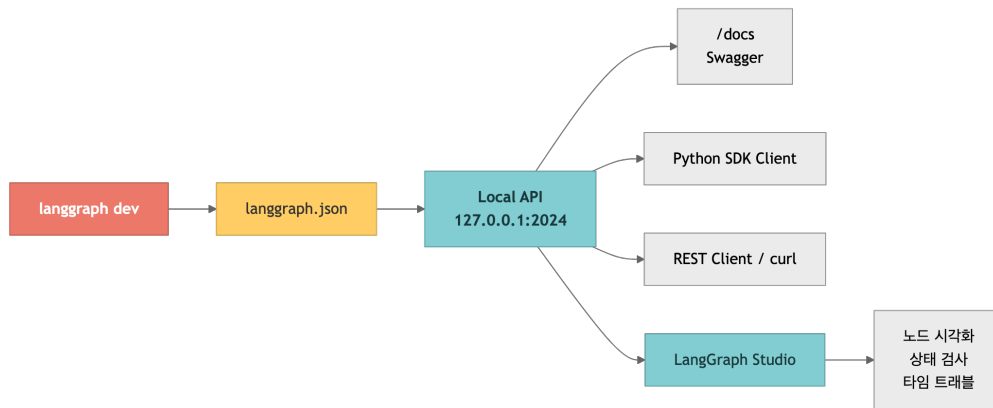
```

11.5 개발 서버 실행



이 구성도는 로컬 서버를 하나의 허브로 보면 이해가 쉽다는 점을 보여줍니다. CLI는 서버를 띄우고, Studio는 시각적 디버깅을 제공하며, Python SDK와 REST는 같은 서버 기능을 서로 다른 인터페이스로 소비합니다.

설정 파일이 준비되었으면, 이제 실제로 서버를 띄울 차례입니다. `langgraph dev` 명령은 `langgraph.json`을 읽어 그래프를 로드하고, 인메모리 체크포인트를 자동으로 연결한 뒤, HTTP API 서버를 시작합니다. 서버가 정상적으로 기동되면 세 가지 URL이 출력됩니다.



이 장에서 헛갈리기 쉬운 지점은 서버 하나로 여러 소비자가 붙는다는 점입니다. 같은 로컬 API를 기준으로 CLI는 서버를 띄우고, Studio는 시각화/디버깅을 담당하며, Python SDK와 REST 클라이언트는 동일한 엔드포인트를 서로 다른 방식으로 호출합니다. 즉, `langgraph dev`는 개발 중인 그래프를 위한 공통 허브라고 생각하면 됩니다.

! WARNING

`langgraph dev`는 개발 전용 서버입니다. 인메모리 체크포인트를 사용하므로 서버를 재시작하면 모든 상태가 초기화됩니다. 프로덕션 환경에서는 `langgraph up` 명령으로 Docker 기반 서버를 실행하고, PostgreSQL 체크포인트를 연결해야 합니다.

```
$ langgraph dev
```

예상 출력:

```
Ready!
- API: http://127.0.0.1:2024
- Docs: http://127.0.0.1:2024/docs
- Studio: https://smith.langchain.com/studio/?baseUrl=http://127.0.0.1:2024
```

URL	용도
-----	----

<code>http://127.0.0.1:2024</code>	API 엔드포인트
<code>http://127.0.0.1:2024/docs</code>	API 문서 (Swagger)
<code>https://smith.langchain.com/studio/...</code>	LangGraph Studio 인터페이스

TIP

Safari 사용자: `langgraph dev --tunnel` 플래그를 사용하여 localhost 서버에 대한 보안 연결을 설정하세요.

11.6 LangGraph Studio 연동

API 서버가 실행 중인 상태에서, 개발자 경험을 한 단계 끌어올리는 것이 LangGraph Studio입니다. Studio는 `langgraph dev` 실행 시 자동으로 제공되는 시각적 디버깅 도구로, 별도 설치 없이 브라우저에서 바로 사용할 수 있습니다. 에이전트의 내부 동작을 “블랙박스”가 아닌 “유리 상자”처럼 투명하게 관찰할 수 있다는 점이 가장 큰 장점입니다.

Studio에서는 그래프의 노드-엣지 구조를 시각적으로 확인하고, 각 노드의 실행 과정을 실시간으로 추적하며, 특정 시점의 상태를 수정하여 다시 실행하는 타임 트래블 기능까지 제공합니다. Human-in-the-loop 패턴에서 인터럽트 지점을 확인하고 수동으로 승인/거부하는 테스트도 Studio에서 수행할 수 있습니다.

주요 기능:

기능	설명
그래프 시각화	노드와 엣지 구조를 한눈에 파악
실시간 추적	각 노드의 실행 과정을 실시간으로 관찰
상태 검사	각 단계의 상태(state) 값을 확인·수정
인터랙티브 테스트	입력을 바꿔가며 그래프 실행 테스트

Studio는 브라우저 기반이므로 별도 설치가 필요 없습니다. 커스텀 서버 주소를 사용하는 경우, Studio URL의 `baseUrl` 파라미터를 변경하면 됩니다.

11.7 Python SDK — 비동기 클라이언트

Studio에서 시각적으로 테스트하는 것 외에, 프로그래밍 방식으로 서버에 접근해야 하는 경우도 많습니다. 예를 들어 CI/CD 파이프라인에서 자동화된 테스트를 실행하거나, 다른 애플리케이션에서 에이전트를 호출할 때는 SDK가 필수입니다. LangGraph SDK는 비동기(`get_client()`)와 동기(`get_sync_client()`) 두 가지 클라이언트를 제공합니다.

비동기 클라이언트는 `langgraph_sdk.get_client()` 로 생성합니다. `asyncio` 기반으로 동작하며, 대량의 동시 요청을 효율적으로 처리할 수 있습니다. 특히 스트리밍 응답을 처리할 때 `async for` 구문을 사용하면 각 청크가 도착할 때마다 즉시 처리할 수 있어 사용자 체감 지연 시간을 줄일 수 있습니다.

TIP

`RemoteGraph` 를 사용하면 로컬 서버뿐 아니라 원격 LangGraph Platform 서버에 대해서도 동일한 인터페이스로 호출할 수 있습니다. `RemoteGraph(name="agent", url="https://your-deployment.langsmith.dev")` 처럼 URL만 변경하면 됩니다. 이를 통해 로컬 개발과 프로덕션 배포 사이의 코드 변경을 최소화할 수 있습니다.

```
# 비동기 클라이언트 사용 패턴 (서버 실행 중일 때 사용)
print("""from langgraph_sdk import get_client
import asyncio

client = get_client(url="http://localhost:2024")

async def main():
    async for chunk in client.runs.stream(
        None,
        "agent",
        input={
            "messages": [{
                "role": "human",
                "content": "What is LangGraph?",
            }],
        },
    ):
        print(f"Event type: {chunk.event}...")
        print(chunk.data)

asyncio.run(main())
""")
print("# 위 코드는 langgraph dev 서버가 실행 중일 때 사용합니다.")
```

```
from langgraph_sdk import get_client
import asyncio

client = get_client(url="http://localhost:2024")

async def main():
    async for chunk in client.runs.stream(
        None,
        "agent",
        input={
            "messages": [{
                "role": "human",
                "content": "What is LangGraph?",
            }],
        },
    ):
        print(f"Event type: {chunk.event}...")
        print(chunk.data)

asyncio.run(main())

# 위 코드는 langgraph dev 서버가 실행 중일 때 사용합니다.
```

11.8 Python SDK — 동기 클라이언트

비동기 프로그래밍이 익숙하지 않거나, Jupyter 노트북 같은 환경에서 빠르게 테스트하고 싶다면 동기 클라이언트를 사용할 수 있습니다. 동기 클라이언트는 `langgraph_sdk.get_sync_client()` 로 생성하며, `asyncio` 없이 일반 `for` 루프로 스트리밍 응답을 처리합니다.

```
# 동기 클라이언트 사용 패턴 (서버 실행 중일 때 사용)
print("""from langgraph_sdk import get_sync_client

client = get_sync_client(url="http://localhost:2024")

for chunk in client.runs.stream(
    None,
    "agent",
    input={
        "messages": [{
            "role": "human",
            "content": "What is LangGraph?",
        }],
    },
    stream_mode="messages-tuple",
):
    print(f"Event: {chunk.event}...")
    print(chunk.data)
""")
print("# 위 코드는 langgraph dev 서버가 실행 중일 때 사용합니다.")
```

```
from langgraph_sdk import get_sync_client

client = get_sync_client(url="http://localhost:2024")

for chunk in client.runs.stream(
    None,
    "agent",
    input={
        "messages": [{
            "role": "human",
            "content": "What is LangGraph?",
        }],
    },
    stream_mode="messages-tuple",
):
    print(f"Event: {chunk.event}...")
    print(chunk.data)

# 위 코드는 langgraph dev 서버가 실행 중일 때 사용합니다.
```

11.9 REST API 호출

Python SDK는 편리하지만, 모든 클라이언트가 Python 환경인 것은 아닙니다. JavaScript 프론트엔드, Go 백엔드, 또는 타 시스템과의 통합이 필요한 경우 REST API를 직접 호출하는 방식이 적합합니다. LangGraph 로컬 서버는 표준 REST API를 제공하므로, HTTP 요청이 가능한 모든 언어와 도구에서 에이전트를 호출할 수 있습니다.

TIP

REST API의 전체 엔드포인트 문서는 <http://localhost:2024/docs> 에서 Swagger UI로 확인할 수 있습니다. 스크린샷 생성, 실행, 상태 조회, 인터럽트 재개 등 모든 기능이 REST API로 제공됩니다.

```
curl -s --request POST \
  --url "http://localhost:2024/runs/stream" \
  --header 'Content-Type: application/json' \
  --data '{
    "assistant_id": "agent",
    "input": {
      "messages": [{
```

```

        "role": "human",
        "content": "What is LangGraph?"
    }],
    },
    "stream_mode": "messages-tuple"
}'

```

API 문서는 <http://localhost:2024/docs> 에서 확인할 수 있습니다.

· NOTE

관계 요약

- `langgraph dev` 는 실행 엔진을 띄우는 진입점입니다.
- Studio는 같은 서버에 붙는 디버깅 UI입니다.
- SDK와 REST는 자동화/앱 연동을 위한 프로그래밍 인터페이스입니다.

11.10 배포 준비 체크리스트

지금까지 로컬 서버 실행, Studio 연동, SDK와 REST API 호출까지 전체 개발 워크플로를 살펴보았습니다. 이제 프로덕션으로 전환하기 전에 점검해야 할 항목들을 체크리스트로 정리합니다. 아래 항목들을 모두 확인한 후에 프로덕션 배포를 진행하세요.

항목	설명	확인
<code>langgraph.json</code>	그래프 경로·의존성·환경 변수 설정 완료	☐
<code>.env</code> 파일	API 키 등 환경 변수 설정	☐
의존성 정리	<code>pyproject.toml</code> 또는 <code>requirements.txt</code> 정리	☐
로컬 테스트	<code>langgraph dev</code> 로 로컬에서 정상 동작 확인	☐
Studio 확인	LangGraph Studio에서 그래프 구조 확인	☐
SDK 테스트	Python SDK 또는 REST API로 호출 테스트	☐
영속 저장소	프로덕션용 체크포인트(예: PostgresSaver) 설정	☐
관측성	LangSmith 또는 Langfuse 트레이싱 설정	☐

요약

주제	핵심 내용
CLI 설치	<code>pip install "langgraph-cli[inmem]"</code> 또는 <code>uv add</code>
프로젝트 생성	<code>langgraph new</code> 로 템플릿 기반 프로젝트 생성

langgraph.json	그래프 경로, 의존성, 환경 변수를 정의하는 설정 파일
Studio	<code>langgraph dev</code> 실행 시 자동 제공되는 시각적 디버깅 도구
SDK 비동기	<code>get_client()</code> 로 비동기 스트리밍 호출
SDK 동기	<code>get_sync_client()</code> 로 동기 스트리밍 호출
REST API	<code>curl</code> 로 <code>/runs/stream</code> 엔드포인트 직접 호출

REFERENCES

- [Run a Local Server](#)

12 내구성 실행

6장에서 체크포인트 기반 장애 복구를 간략히 소개했다면, 이 장에서는 내구성 실행(Durable Execution)을 본격적으로 심화합니다. 실제 운영 환경에서 에이전트 워크플로는 네트워크 장애, 프로세스 재시작, 외부 API 타임아웃 등 다양한 실패 상황에 노출됩니다. 내구성 실행은 각 실행 단계를 체크포인트로 저장하여, 장애 발생 시 처음부터 재실행하지 않고 마지막 성공 지점에서 정확히 재개할 수 있게 합니다.

내구성 실행의 핵심 원리는 결정론적 재실행(deterministic replay)입니다. 워크플로가 재개되면 `@entrypoint` 함수는 처음부터 다시 실행되지만, 이미 완료된 `@task` 는 체크포인트에서 저장된 결과를 반환합니다. 이를 통해 부수 효과(API 호출 등)의 중복 실행을 방지하면서도, 정확히 중단된 지점에서 실행을 이어갈 수 있습니다. 이 장에서는 내구성 실행의 원리와 `@entrypoint` + `@task` 기반 구현, 그리고 세 가지 내구성 모드(`exit`, `async`, `sync`)의 차이를 살펴봅니다.

학습 목표

LEARNING OBJECTIVES

- 내구성 실행(Durable Execution)의 개념과 필요성을 이해한다
- 체크포인트와 내구성 실행의 관계를 안다
- `@entrypoint` + `@task` 로 내구성을 보장하는 방법을 익힌다
- 내구성 모드(`exit`, `async`, `sync`)의 차이를 이해한다
- 장애 시나리오에서의 복구 과정을 안다

12.1 환경 설정

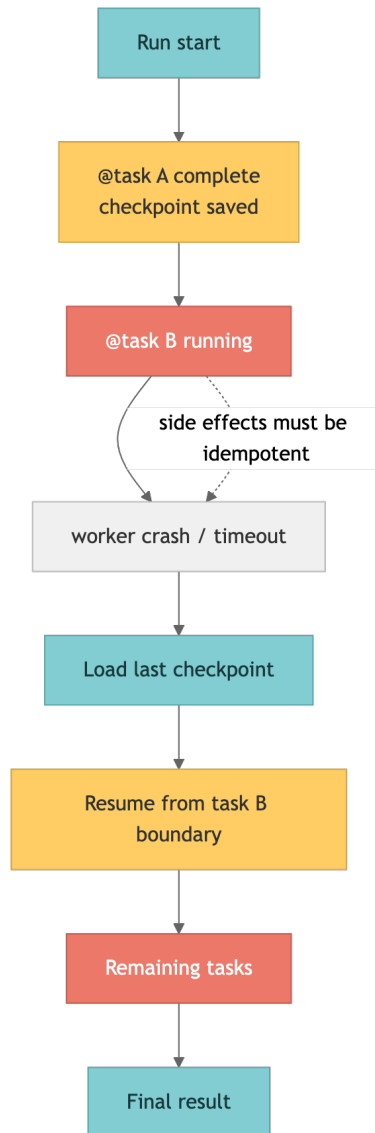
```
from dotenv import load_dotenv
load_dotenv(override=True)
from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
```

12.2 내구성 실행 개념

내구성 실행(Durable Execution)이란 프로세스나 워크플로가 핵심 지점에서 진행 상태를 저장하여, 일시 중지 후 나중에 정확히 중단된 위치에서 재개할 수 있는 기법입니다. 이 개념은 Temporal, Azure Durable

Functions 등 분산 시스템에서 오래전부터 사용되어 온 검증된 패턴이며, LangGraph는 이를 AI 에이전트 워크플로에 맞게 적용합니다.

일반적인 프로그램은 프로세스가 종료되면 모든 상태를 잃습니다. 하지만 에이전트 워크플로는 수십 분에서 수 시간 동안 실행될 수 있고, 외부 API 호출이나 사람의 승인 대기 등 장시간 중단이 빈번합니다. 이런 워크플로에서 장애가 발생했을 때 처음부터 다시 실행하는 것은 시간과 비용 면에서 비효율적이며, 부수 효과의 중복 실행으로 오류를 유발할 수도 있습니다.



이 흐름도에서 중요한 것은 재개 지점이 아무 줄이 아니라 체크포인트 경계라는 점입니다. 따라서 부작용이 있는 작업은 `@task` 경계 바깥으로 흘러나오지 않도록 설계해야 합니다.

왜 필요한가?

시나리오	설명
장애 복구	서버 장애 시 처음부터 다시 실행하지 않고 중단 지점에서 재개

상태 영속	긴 실행 시간을 가진 워크플로의 중간 결과를 보존
Human-in-the-loop	사람의 승인을 기다리는 동안 상태를 유지

LangGraph는 체크포인트(checkpointer)를 통해 내구성 실행을 지원합니다.

12.3 핵심 요구사항

내구성 실행의 개념을 이해했으니, 이제 LangGraph에서 이를 구현하기 위한 구체적인 요구사항을 살펴봅시다. 내구성 실행을 구현하려면 세 가지 요소가 반드시 갖추어져야 합니다:

1. 영속 계층 (Persistence Layer)

체크포인트를 통해 워크플로 진행 상태를 기록합니다. 예: `InMemorySaver` (개발용), `PostgresSaver` (프로덕션용)

1. 스레드 식별자 (Thread ID)

워크플로 인스턴스의 실행 기록을 추적하는 고유 ID입니다. 같은 `thread_id` 를 사용하면 이전 실행을 이어서 재개할 수 있습니다.

1. 태스크 래핑 (Task Wrapping)

비결정적(non-deterministic) 연산과 부수 효과(side-effect) 연산을 태스크로 감싸서 재개 시 재실행을 방지합니다.

12.4 내구성 모드 비교

세 가지 요구사항이 갖추어졌다면, 다음으로 결정해야 할 것은 언제 체크포인트를 저장할 것인가입니다. 체크포인트를 자주 저장할수록 장애 복구 능력은 높아지지만, 그만큼 I/O 비용이 증가합니다. LangGraph는 이 트레이드오프를 조절할 수 있도록 세 가지 내구성 모드를 제공합니다:

TIP

내구성 모드는 `\@entrypoint(checkpointer=saver, durability_mode="sync")` 형태로 Functional API에서 설정하거나, Graph API에서는 `graph.compile(checkpointer=saver)` 시점에 지정합니다. 기본값은 "exit" 입니다.

모드	동작	트레이드오프
"exit"	완료/에러/인터럽트 시에만 영속화	최고 성능, 중간 복구 불가
"async"	다음 단계 실행 중 비동기로 영속화	적절한 균형, 약간의 크래시 위험
"sync"	다음 단계 실행 전 동기로 영속화	최대 내구성, 성능 비용

대부분의 사용 사례에서는 기본 모드("exit")로 충분합니다. 미션 критичный 워크플로에서는 "sync" 모드를 고려하세요.

· NOTE

"async" 모드에서는 다음 태스크 실행과 체크포인트 저장이 동시에 진행됩니다. 만약 체크포인트 저장이 완료되기 전에 프로세스가 크래시되면, 해당 태스크의 결과가 유실될 수 있습니다. 이 위험이 허용되지 않는 금융 거래 등의 시나리오에서는 "sync" 모드를 사용하세요.

12.5 문제가 있는 코드

내구성 모드를 이해했으니, 이제 올바른 코드와 잘못된 코드의 차이를 구체적으로 살펴봅시다. 내구성 실행의 핵심 원리인 결정론적 재실행(deterministic replay)을 제대로 활용하려면, 어떤 코드를 `@task` 로 감싸야 하는지 정확히 이해해야 합니다. 부수 효과(API 호출 등)를 태스크로 감싸지 않으면, 재개 시 동일한 API 호출이 다시 실행되어 심각한 문제를 일으킬 수 있습니다.

```
# 문제가 있는 접근 방식: 부수 효과를 직접 호출
print("""# BAD: 부수 효과가 태스크로 감싸지지 않음
def call_api(state: State):
    # 이 API 호출은 재개 시 다시 실행됨!
    result = requests.get(state['url']).text[:100]
    return {"result": result}
""")
print("문제점:")
print("  1. 장애 후 재개 시 API가 다시 호출됨")
print("  2. 비결정적 결과가 달라질 수 있음")
print("  3. 중복 요청으로 부작용 발생 가능")
```

```
# BAD: 부수 효과가 태스크로 감싸지지 않음
def call_api(state: State):
    # 이 API 호출은 재개 시 다시 실행됨!
    result = requests.get(state['url']).text[:100]
    return {"result": result}
```

문제점:

1. 장애 후 재개 시 API가 다시 호출됨
2. 비결정적 결과가 달라질 수 있음
3. 중복 요청으로 부작용 발생 가능

12.6 @task로 개선

위 문제를 해결하는 방법은 간단합니다. 부수 효과를 `@task` 데코레이터로 감싸면 됩니다. `@task` 로 감싼 함수의 결과는 체크포인트에 저장되므로, 워크플로가 재개될 때 실제 함수를 다시 호출하지 않고 저장된 결과를 즉시 반환합니다. 이것이 결정론적 재실행의 핵심 메커니즘입니다.

```
# 개선된 접근 방식: @task로 부수 효과 래핑
print("""# GOOD: @task로 부수 효과를 감쌌
from langgraph.func import task

@task
def _make_request(url: str):
    return requests.get(url).text[:100]

def call_api(state: State):
    # 각 요청이 개별 태스크로 실행됨
    requests = [_make_request(url) for url in state['urls']]
```

```

        results = [req.result() for req in requests]
        return {"results": results}
    """
)
print("개선 효과:")
print("  1. 재개 시 체크포인트에서 결과 복원")
print("  2. API 중복 호출 방지")
print("  3. 각 태스크가 독립적으로 추적됨")

```

```

# GOOD: @task로 부수 효과를 감쌌
from langgraph.func import task

@task
def _make_request(url: str):
    return requests.get(url).text[:100]

def call_api(state: State):
    # 각 요청이 개별 태스크로 실행됨
    requests = [_make_request(url) for url in state['urls']]
    results = [req.result() for req in requests]
    return {"results": results}

개선 효과:
1. 재개 시 체크포인트에서 결과 복원
2. API 중복 호출 방지
3. 각 태스크가 독립적으로 추적됨

```

12.7 Graph API에서의 내구성

`@task` 패턴을 이해했으니, 이제 두 가지 API에서 내구성이 어떻게 동작하는지 비교해 봅시다. Graph API에서는 내구성 구현이 더 단순합니다. `StateGraph` 에 체크포인트를 연결하기만 하면, 각 노드 실행이 완료될 때마다 전체 상태가 자동으로 저장됩니다. 노드 자체가 내구성의 기본 단위가 되므로, 별도의 `@task` 래핑이 필요하지 않습니다.

· NOTE

Graph API에서는 각 노드가 하나의 체크포인트 단위입니다. 노드 A가 성공하고 노드 B에서 장애가 발생하면, 재개 시 노드 B부터 다시 실행됩니다. 따라서 하나의 노드 안에 여러 개의 독립적인 부수 효과가 있다면, 이를 별도의 노드로 분리하는 것이 내구성 측면에서 유리합니다.

```

from typing import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import InMemorySaver

class DocState(TypedDict):
    topic: str
    draft: str
    final: str

def write_draft(state: DocState) -> dict:
    return {"draft": f"Draft about {state['topic']}"}

def finalize(state: DocState) -> dict:
    return {"final": f"Final: {state['draft']}"}

checkpoint = InMemorySaver()

```

```

builder = StateGraph(DocState)
builder.add_node("write_draft", write_draft)
builder.add_node("finalize", finalize)
builder.add_edge(START, "write_draft")
builder.add_edge("write_draft", "finalize")
builder.add_edge("finalize", END)

graph = builder.compile(checkpointer=checkpointer)

# 실행 (thread_id로 실행 추적)
config = {"configurable": {"thread_id": "doc-1"}}
result = graph.invoke({"topic": "LangGraph"}, config)
print("결과:", result)

```

```
결과: {'topic': 'LangGraph', 'draft': 'Draft about LangGraph', 'final': 'Final: Draft about LangGraph'}
```

12.8 Functional API에서의 내구성

Functional API에서의 내구성은 Graph API와 근본적으로 다른 방식으로 동작합니다. `@entrypoint` 와 `@task` 를 조합하면 내구성을 보장할 수 있지만, 재개 시의 동작이 다릅니다. `@entrypoint` 함수는 항상 처음부터 다시 실행되며, 이미 완료된 `@task` 는 실제로 실행되지 않고 체크포인트에서 저장된 결과를 반환합니다. 따라서 `@entrypoint` 내부의 모든 비결정적 코드는 반드시 `@task` 로 감싸야 합니다.

```

from langgraph.func import entrypoint, task
from langgraph.checkpoint.memory import InMemorySaver

@task
def generate_draft(topic: str) -> str:
    return f"Draft about {topic}"

@task
def review_draft(draft: str) -> str:
    return f"Reviewed: {draft}"

func_checkpointer = InMemorySaver()

@entrypoint(checkpointer=func_checkpointer)
def write_document(topic: str) -> str:
    draft = generate_draft(topic).result()
    reviewed = review_draft(draft).result()
    return reviewed

config = {"configurable": {"thread_id": "func-1"}}
result = write_document.invoke("Durable Execution", config)
print("결과:", result)

```

```
결과: Reviewed: Draft about Durable Execution
```

12.9 장애 복구 시나리오

이론적인 설명만으로는 내구성 실행의 가치를 체감하기 어렵습니다. 이제 구체적인 장애 복구 시나리오를 통해 체크포인트가 실제로 어떻게 동작하는지 확인해 봅시다. 핵심 원리는 간단합니다: 같은 `thread_id` 로 재실행하면 체크포인트에서 이전 상태를 복원하여 이어서 실행합니다.

```
from typing import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import InMemorySaver

class PipelineState(TypedDict):
    data: str
    step: int
    result: str

call_count = 0

def step_one(state: PipelineState) -> dict:
    global call_count
    call_count += 1
    print(f" step_one 실행 (호출 횟수: {call_count})")
    return {"data": state["data"].upper(), "step": 1}

def step_two(state: PipelineState) -> dict:
    print(f" step_two 실행")
    return {"result": f"Processed: {state['data']}", "step": 2}

recovery_saver = InMemorySaver()

builder = StateGraph(PipelineState)
builder.add_node("step_one", step_one)
builder.add_node("step_two", step_two)
builder.add_edge(START, "step_one")
builder.add_edge("step_one", "step_two")
builder.add_edge("step_two", END)

pipeline = builder.compile(checkpointer=recovery_saver)

# 첫 번째 실행
config = {"configurable": {"thread_id": "recovery-1"}}
print("=== 첫 번째 실행 ===")
result = pipeline.invoke(
    {"data": "hello", "step": 0, "result": ""},
    config
)
print(f"결과: {result}")

# 체크포인트 확인
print("\n=== 체크포인트에서 상태 복원 ===")
saved = pipeline.get_state(config)
print(f"저장된 상태: {saved.values}")
print(f"step_one 총 호출 횟수: {call_count}")
```

```
=== 첫 번째 실행 ===
step_one 실행 (호출 횟수: 1)
step_two 실행
결과: {'data': 'HELLO', 'step': 2, 'result': 'Processed: HELLO'}

=== 체크포인트에서 상태 복원 ===
저장된 상태: {'data': 'HELLO', 'step': 2, 'result': 'Processed: HELLO'}
step_one 총 호출 횟수: 1
```


· NOTE

장애가 발생했을 때 “어디서 다시 시작되는가?”를 항상 확인하세요. LangGraph는 마지막으로 성공적으로 체크포인트된 작업 이후 부터 다시 시작합니다. 그래서 외부 API 호출, 파일 쓰기, 결제 같은 부작용은 재실행되어도 안전한 단위 로 감싸야 합니다.

12.10 재개 시작점

내구성 실행에서 가장 중요한 차이 중 하나는 재개 시작점입니다. Graph API와 Functional API는 동일한 체크포인트를 사용하지만, 재개 동작이 근본적으로 다릅니다:

API	재개 시작점	설명
StateGraph	중단된 노드의 시작	해당 노드를 처음부터 다시 실행
서브그래프	부모 노드 → 서브그래프 내 중단 노드	부모 노드에서 시작 후 서브그래프 내 해당 노드로 이동
Functional API	<code>\@entrypoint</code> 시작	엔트리포인트에서 시작, <code>\@task</code> 결과는 캐시에서 복원

핵심 차이:

- StateGraph: 노드 단위 재개 (중단된 노드만 재실행)
- Functional API: 엔트리포인트부터 재실행하되, 완료된 `@task` 는 캐시 결과 사용

! WARNING

Functional API에서 재개 시 `\@entrypoint` 함수가 처음부터 다시 실행되므로, `\@task` 밖에 있는 코드(변수 초기화, 조건문 등)는 결정론적이어야 합니다. 비결정적 코드가 `\@task` 밖에 있으면 재실행 시 다른 분기를 탈 수 있어 예측할 수 없는 동작이 발생합니다.

12.11 프로덕션 내구성 패턴

재개 시작점의 차이를 이해했다면, 이제 프로덕션 환경에서 내구성을 효과적으로 활용하기 위한 모범 사례를 정리합니다. 아래 패턴들은 장애 복구뿐 아니라, 코드의 유지보수성과 테스트 용이성까지 향상시킵니다.

! WARNING

프로덕션 환경에서 `InMemorySaver` 를 사용하면 프로세스 재시작 시 모든 체크포인트가 유실됩니다. 반드시 `PostgresSaver` 등 영속 저장소 기반 체크포인트를 사용하세요. LangGraph Platform을 사용하면 체크포인트가 자동으로 관리되므로 별도 설정이 필요 없습니다.

1. 멍등성(Idempotent) 연산 구현

같은 요청을 여러 번 실행해도 결과가 동일하도록 설계합니다. 멍등성 키(idempotency key)를 활용하여 중복 처리를 방지합니다.

1. 부수 효과 분리

API 호출, 파일 쓰기 등의 부수 효과를 개별 `@task` 로 분리합니다. 순수 로직과 부수 효과를 명확히 구분합니다.

1. 비결정적 코드 래핑

난수 생성, 타임스탬프 등 비결정적 연산도 `@task` 로 감쌉니다.

1. 영속 저장소 사용

개발: `InMemorySaver` 프로덕션: `PostgresSaver` 또는 외부 데이터베이스

1. 스레드 ID 관리

각 워크플로 인스턴스에 고유한 `thread_id` 를 부여합니다. 장애 복구 시 동일한 `thread_id` 로 재개합니다.

12.12 노드 시도 단위 fault tolerance — `TimeoutPolicy` + `heartbeat`

내구성 실행이 어디까지 진행했는지를 checkpoint로 보존한다면, LangGraph 1.2의 fault tolerance 기능은 언제 포기하고, 어떻게 보정할지를 노드 시도(attempt) 단위로 제어합니다.

`add_node(..., timeout=TimeoutPolicy(...))` 로 단일 시도에 시간 제한을 두면, 초과 시 `NodeTimeoutError` 가 발생하고 retry policy가 있으면 다음 시도로 넘어갑니다. timeout이 발생한 시도의 partial write는 폐기됩니다.

장기 작업 노드는 `Runtime.heartbeat()` 와 `refresh_on="heartbeat"` 를 결합해 idle timeout을 갱신할 수 있습니다.

```
from langgraph.func import task
from langgraph.runtime import Runtime
from langgraph.types import TimeoutPolicy

async def long_running_node(state: State, runtime: Runtime) -> dict:
    for batch in fetch_batches():
        process(batch)
        runtime.heartbeat()
    return {"result": "done"}

builder.add_node(
    "long_running_node",
    long_running_node,
    timeout=TimeoutPolicy(idle_timeout=30, refresh_on="heartbeat"),
)
```

12.13 Graceful shutdown — `RunControl` / `GraphDrained`

장기 실행 그래프를 SIGTERM 같은 외부 시그널에서 즉시 강제 종료하지 않고, 현재 superstep을 끝낸 뒤 checkpoint를 남기려면 `RunControl.request_drain()` 을 사용합니다. drain이 요청되면 런은 `GraphDrained` 로 멈추고 같은 config로 나중에 재개할 수 있습니다. 노드 내부에서는 `Runtime.drain_requested` 로 drain 신호를 직접 분기해 안전 종료 처리를 추가할 수 있습니다.

```
from langgraph.runtime import RunControl, Runtime
from langgraph.errors import GraphDrained
```

```
# 1) 호출 측 - 외부 시그널(SIGTERM 등)에 drain 요청
control = RunControl()
control.request_drain("sigterm")

try:
    result = graph.invoke(inputs, config, control=control)
except GraphDrained as e:
    print(f"Drained: {e.reason}")
    # checkpoint는 이미 저장됨

# 같은 config로 재개
graph.invoke(None, config)

# 2) 노드 내부 - drain 신호를 직접 감지
async def my_node(state: State, runtime: Runtime) -> dict:
    if runtime.drain_requested:
        return {"status": "skipped"}
    # ... 일반 처리
    return {"status": "done"}
```

TIP

drain은 인터럽트와 다릅니다. 인터럽트는 사용자 입력을 기다리며 일시 정지하는 애플리케이션 레벨 신호이고, drain은 superstep 경계에서 안전하게 멈추기 위한 운영 레벨 신호입니다. 두 가지를 함께 사용하면 점검·배포·롤링 업데이트 중에도 진행 상태를 잃지 않습니다.

12.14 DeltaChannel (beta) – checkpoint 저장량 최적화

기본 checkpoint는 매 superstep마다 채널 값을 전체 저장합니다. 메시지 목록처럼 append-heavy 채널은 장기 thread에서 저장량이 빠르게 늘어납니다. langgraph≥1.2 에서 도입된 DeltaChannel 은 누적 전체값 대신 delta만 저장해 checkpoint 비용을 낮추고, snapshot_frequency 로 K step마다 한 번씩 full snapshot을 남겨 reconstruction 비용을 제한합니다.

```
from typing import Annotated, Sequence
from typing_extensions import TypedDict
from langgraph.channels import DeltaChannel

def list_reducer(state: list, writes: Sequence[list]) -> list:
    # bulk reducer 시그니처: (current_state, sequence_of_writes) -> new_state
    result = list(state)
    for write in writes:
        result.extend(write)
    return result

class State(TypedDict):
    messages: Annotated[
        list[str],
        DeltaChannel(list_reducer, snapshot_frequency=5),
    ]
```

! WARNING

DeltaChannel 은 langgraph≥1.2 beta 기능입니다. Reducer는 associative해야 하며 write 시점이 아니라 reconstruction 시점에 실행됩니다. 프로덕션 적용 전 저장량, 복구 지연, 마이그레이션 비용을 함께 측정하세요

요. 저장 최적화는 `DeltaChannel` 로, 실패 복구 제어는 `TimeoutPolicy` / `error_handler` 로 분리해 이해하면 설계가 명확해집니다.

요약

주제	핵심 내용
내구성 개념	중단 지점에서 재개할 수 있는 실행 기법
핵심 요구사항	영속 계층 + 스레드 ID + 태스크 래핑
내구성 모드	<code>exit</code> (기본), <code>async</code> (균형), <code>sync</code> (최대 내구성)
@task	부수 효과를 감싸서 재실행 방지
Graph API	<code>checkpointer</code> 연결로 노드별 자동 저장
Functional API	<code>\@entrypoint</code> + <code>\@task</code> 로 내구성 보장
장애 복구	같은 <code>thread_id</code> 로 체크포인트에서 재개
노드 timeout	<code>TimeoutPolicy(idle_timeout=..., refresh_on="heartbeat")</code> 로 시도별 시간 제한
Graceful shutdown	<code>RunControl.request_drain()</code> → <code>GraphDrained</code> → 같은 config로 재개
DeltaChannel (beta)	<code>langgraph≥1.2</code> . <code>delta</code> 만 저장 + <code>snapshot_frequency</code> 로 reconstruction 비용 제한

REFERENCES

- [Durable Execution](#)

13 API 선택 가이드와 Pregel

Part 3의 마지막 장입니다. 12개 장에 걸쳐 `Graph API` 와 `Functional API` 를 번갈아 사용해 왔다면, 이제 두 API의 설계 철학과 내부 실행 엔진을 정리할 때입니다. 두 API는 모두 `Pregel` 런타임 위에서 슈퍼스텝 단위로 실행되며, 체크포인팅, 스트리밍, 인터럽트 등 동일한 인프라를 공유합니다. 둘 사이의 차이는 표현 방식에 있을 뿐, 런타임 역량에는 본질적인 차이가 없습니다. 이 장에서는 동일한 에이전트를 양쪽 API로 구현해 비교하고, `Pregel` 런타임의 내부 구조를 이해하여 프로젝트에 맞는 API 선택 기준을 세웁니다.

학습 목표

LEARNING OBJECTIVES

- `Graph API`와 `Functional API`의 차이를 비교한다
- 동일한 에이전트를 두 API로 구현한다
- `Pregel` 런타임의 내부 구조를 이해한다
- 슈퍼스텝 실행 모델을 안다
- 프로젝트에 맞는 API를 선택하는 기준을 세운다

핵심 질문

둘 다 같은 `LangGraph` 런타임 위에서 실행된다면, “언제는 그래프로 명시하고 언젠는 함수 흐름으로 충분한가?” 가 이 장의 핵심 질문입니다.

한눈에 보는 핵심

- `Graph API`는 흐름을 눈으로 보고 관리해야 할 때 유리합니다.
- `Functional API`는 Python 함수 흐름을 살리면서 내구성을 얻고 싶을 때 간결합니다.
- 둘 중 하나만 고집하기보다, 문제 구조에 따라 섞어 쓰는 것이 실전적입니다.

13.1 환경 설정

```
from dotenv import load_dotenv
load_dotenv(override=True)
from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
```

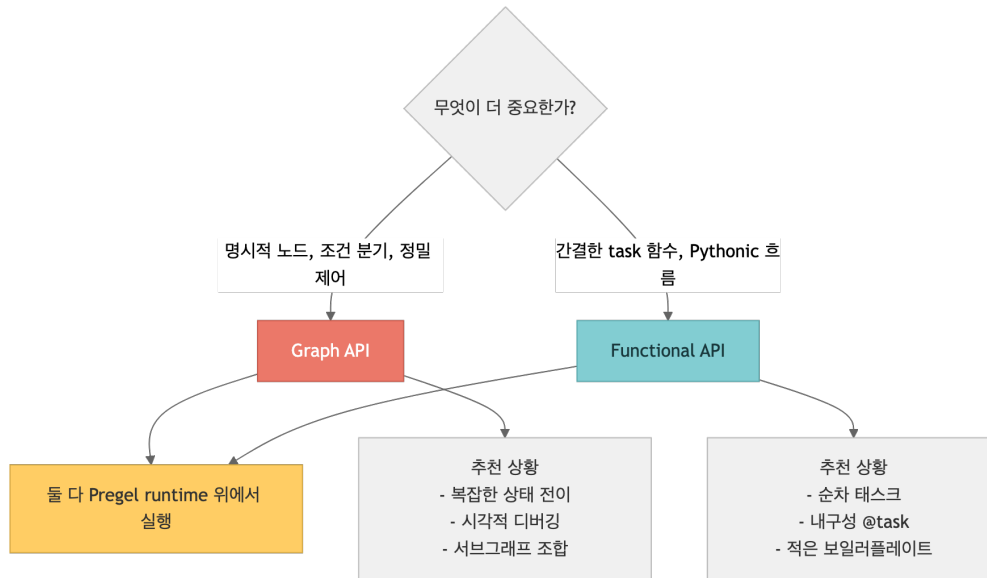
13.2 Graph API vs Functional API 개요

LangGraph는 에이전트 워크플로를 구축하기 위한 두 가지 API를 제공합니다. Graph API는 노드와 엣지로 구성된 명시적 그래프를 통해 워크플로를 정의하고, Functional API는 `@entrypoint` 와 `@task` 데코레이터를 사용하여 일반 Python 함수 형태로 워크플로를 표현합니다.

두 API는 단지 표현 방식이 다를 뿐, 내부적으로는 동일한 Pregel 런타임 위에서 실행됩니다. 따라서 체크포인트링, 스트리밍, 인터럽트, human-in-the-loop 등 모든 인프라 기능을 동일하게 사용할 수 있으며, 하나의 애플리케이션에서 두 API를 함께 사용하는 것도 가능합니다.

특성	Graph API	Functional API
추상화 방식	노드와 엣지로 구성된 그래프	데코레이터 기반 함수
상태 관리	TypedDict 스키마로 명시적 관리	함수 스코프 내 로컬 변수
제어 흐름	조건부 엣지, 라우팅	일반 Python 제어문 (if/else, for)
시각화	그래프 구조 자동 시각화	제한적
보일러플레이트	상대적으로 많음	최소화

13.3 빠른 선택 가이드



선택의 핵심은 실행 흐름을 눈에 보이는 그래프로 관리할지, 아니면 Python 함수 흐름으로 간결하게 표현할지입니다. 두 API 모두 Pregel 런타임 위에서 실행되므로 성능보다도 표현 방식과 유지보수성이 더 중요한 결정 기준입니다.

개요 표에서 두 API의 구조적 차이를 확인했습니다. 하지만 실무에서는 “어떤 API를 선택해야 하는가?”라는 구체적인 판단이 필요합니다. 아래 표는 일반적인 상황별로 추천하는 API와 그 이유를 정리한 것입니다.

상황	추천 API	이유
복잡한 워크플로 시각화 필요	Graph API	노드/엣지 구조가 자동으로 다이어그램 생성
병렬 실행 경로	Graph API	여러 노드가 자연스럽게 병렬 실행
멀티 에이전트 팀	Graph API	에이전트 간 명확한 역할 분리
기존 코드에 최소 변경	Functional API	데코레이터만 추가하면 됨
단순 선형 워크플로	Functional API	보일러플레이트 없이 빠르게 구현
빠른 프로토타이핑	Functional API	상태 스키마 정의 불필요

13.4 Graph API 구현

이론적인 비교를 넘어, 동일한 워크플로를 두 API로 구현하여 코드 수준에서 차이를 체감해 봅시다. 간단한 “에세이 작성 → 점수 매기기” 2단계 파이프라인을 양쪽 API로 각각 구현합니다. 먼저 Graph API부터 살펴봅시다.

Graph API에서는 (1) 상태 스키마를 `TypedDict` 로 정의하고, (2) 각 단계를 노드 함수로 구현한 뒤, (3) `StateGraph` 에 노드를 등록하고 엣지로 연결합니다. 코드의 구조가 곧 워크플로의 구조를 반영한다는 점이 Graph API의 특징입니다.

```
from typing import TypedDict
from langgraph.constants import START
from langgraph.graph import StateGraph

class Essay(TypedDict):
    topic: str
    content: str | None
    score: float | None

def write_essay(essay: Essay):
    return {"content": f"Essay about {essay['topic']}"}

def score_essay(essay: Essay):
    return {"score": 10}

builder = StateGraph(Essay)
builder.add_node(write_essay)
builder.add_node(score_essay)
builder.add_edge(START, "write_essay")
builder.add_edge("write_essay", "score_essay")

graph_app = builder.compile()

result = graph_app.invoke({"topic": "LangGraph"})
print("Graph API 결과:", result)
```

Graph API 결과: {'topic': 'LangGraph', 'content': 'Essay about LangGraph', 'score': 10}

13.5 Functional API 구현

이번에는 동일한 에세이 워크플로를 Functional API로 구현합니다. Functional API에서는 상태 스키마, 엣지 연결, `StateGraph` 빌더 등의 보일러플레이트가 사라집니다. 대신 `@task` 로 각 단계를 감싸고, `@entrypoint` 에서 일반 Python 함수처럼 순차적으로 호출합니다.

```
from typing import TypedDict
from langgraph.func import entrypoint, task
from langgraph.checkpoint.memory import InMemorySaver

class EssayResult(TypedDict):
    topic: str
    content: str | None
    score: float | None

@task
```



```
def write_essay_func(topic: str) -> str:
    return f"Essay about {topic}"

@task
def score_essay_func(content: str) -> float:
    return 10

func_saver = InMemorySaver()

@entrypoint(checkpointer=func_saver)
def essay_pipeline(topic: str) -> dict:
    content = write_essay_func(topic).result()
    score = score_essay_func(content).result()
    return {"topic": topic, "content": content, "score": score}

config = {"configurable": {"thread_id": "essay-1"}}
result = essay_pipeline.invoke("LangGraph", config)
print("Functional API 결과:", result)
```

Functional API 결과: {'topic': 'LangGraph', 'content': 'Essay about LangGraph', 'score': 10}

13.6 비교 분석

위 두 구현의 실행 결과는 동일합니다. 하지만 코드의 구조와 표현력에서 의미 있는 차이가 있습니다. 아래 표에서 항목별로 나란히 비교합니다:

· NOTE

Functional API가 항상 “더 좋다”는 의미가 아닙니다. 간단한 선형 워크플로에서는 Functional API가 간결하지만, 조건 분기가 많거나 병렬 실행 경로가 복잡한 경우에는 Graph API의 명시적 구조가 가독성과 유지보수 면에서 유리합니다.

· NOTE

언제 쓰고, 언제 쓰지 않을까?

- **Graph API**는 복잡한 결정 트리, 공유 상태, 병렬 처리, 시각적 디버깅이 중요할 때 적합합니다.
- **Functional API**는 기존 Python 함수 흐름을 살리면서 `@task` 단위의 내구성을 얻고 싶을 때 적합합니다.
- 반대로 단순한 선형 태스크를 Graph API로 과하게 쪼개면 보일러플레이트가 늘고, 복잡한 병렬/분기 그래프를 Functional API만으로 우겨 넣으면 흐름이 코드 속에 숨어 가독성이 떨어질 수 있습니다.

항목	Graph API	Functional API
상태 정의	TypedDict 스키마 필수	선택적 (로컬 변수 가능)
노드 연결	<code>add_edge()</code> , <code>add_conditional_edges()</code>	일반 함수 호출

체크포인트	<code>compile(checkpointer=...)</code>	<code>\@entrypoint(checkpointer=...)</code>
코드 라인 수	상대적으로 많음	간결함
시각화	<code>graph.get_graph().draw_mermaid()</code> 지원	제한적
병렬 실행	엣지 구조로 자연스럽게 지원	<code>\@task</code> 병렬 실행으로 지원
디버깅	Studio에서 노드별 상태 확인	함수 단위 트레이싱

13.7 두 API 결합

실전에서는 “Graph API 또는 Functional API”가 아니라 “Graph API 그리고 Functional API”인 경우가 많습니다. 하나의 애플리케이션에서 두 API를 함께 사용할 수 있습니다. 복잡한 멀티 에이전트 조정은 Graph API로, 단순한 데이터 파이프라인은 Functional API로 처리하는 패턴이 일반적입니다.

TIP

Functional API로 작성한 `\@entrypoint` 함수는 Graph API의 노드로 사용할 수 있습니다. 반대로, Graph API로 컴파일된 그래프를 Functional API의 `\@entrypoint` 내부에서 `invoke()` 로 호출할 수도 있습니다. 두 API는 완전히 호환됩니다.

```
# Graph API: 복잡한 멀티 에이전트 조정
coordinator = StateGraph(CoordinatorState)
coordinator.add_node("planner", planner_agent)
coordinator.add_node("executor", executor_agent)
coordinator.add_node("reviewer", reviewer_agent)
# ...

# Functional API: 단순 데이터 처리
@entrypoint(checkpointer=saver)
def preprocess(data: str) -> str:
    cleaned = clean_data(data).result()
    validated = validate_data(cleaned).result()
    return validated
```

복잡도가 증가하면 Functional에서 Graph로, 과도하게 설계된 경우 Graph에서 Functional로 마이그레이션할 수 있습니다.

13.8 Pregel 런타임 개요

API 선택 가이드를 살펴보았으니, 이제 두 API가 공유하는 내부 실행 엔진을 자세히 들여다봅시다. 왜 내부 구조를 알아야 할까요? Graph API와 Functional API의 동작 방식, 성능 특성, 디버깅 전략을 깊이 이해하려면 그 아래에서 동작하는 런타임의 원리를 파악해야 합니다.

Pregel은 LangGraph의 내부 실행 엔진입니다. `StateGraph` 를 컴파일하거나 `\@entrypoint` 를 사용하면 내부적으로 Pregel 인스턴스가 생성됩니다. 이름은 Google의 Pregel 논문(2010, “Pregel: A System for Large-Scale Graph Processing”)에서 따왔으며, 대규모 병렬 그래프 연산의 핵심 아이디어를 LLM 에이전트 실행에 맞게 적용한 것입니다.

LangGraph에서 Pregel은 액터(actor)와 채널(channel)이라는 두 가지 핵심 개념으로 구성됩니다. 각 노드는 채널에서 데이터를 읽고 처리 결과를 채널에 쓰는 액터이며, 채널은 액터 간 데이터 통신을 담당합니다. Graph API에서 `TypedDict`의 각 필드는 채널에 대응되고, 리듀서(`Annotated[list, operator.add]`)는 채널의 업데이트 정책에 대응됩니다.

핵심 구성 요소:

구성 요소	역할
액터(Actor)	채널에서 데이터를 읽고 처리 결과를 채널에 씀
채널(Channel)	액터 간 데이터 통신을 담당

실행 3단계 (매 스텝마다):

1. Plan – 이번 스텝에서 실행할 액터를 결정
2. Execute – 선택된 액터를 병렬로 실행 (완료, 실패, 또는 타임아웃까지)
3. Update – 새로운 값으로 채널을 갱신

실행할 액터가 없거나 최대 스텝에 도달하면 종료됩니다.

13.9 Pregel 직접 사용

일반적으로 Pregel을 직접 사용할 필요는 없습니다. Graph API와 Functional API가 Pregel의 복잡성을 깔끔하게 추상화하기 때문입니다. 그러나 내부 동작 원리를 이해하면 디버깅과 성능 최적화에 도움이 되므로, 간단한 예제를 살펴봅니다. 아래 코드에서 `NodeBuilder`로 액터를 정의하고, 채널을 통해 데이터를 연결하는 Pregel의 저수준 API를 직접 사용합니다.

! WARNING

아래의 Pregel 직접 사용 코드는 학습 목적입니다. 프로덕션 코드에서는 Graph API 또는 Functional API를 사용하세요. Pregel의 내부 API는 LangGraph 버전에 따라 변경될 수 있습니다.

```
from langgraph.channels import EphemeralValue
from langgraph.pregel import Pregel, NodeBuilder

# 단일 노드: 입력을 두 번 반복
node1 = (
    NodeBuilder()
    .subscribe_only("a")
    .do(lambda x: x + x)
    .write_to("b")
)

app = Pregel(
    nodes={"node1": node1},
    channels={
        "a": EphemeralValue(str),
        "b": EphemeralValue(str),
    },
    input_channels=["a"],
    output_channels=["b"],
)

result = app.invoke({"a": "foo"})
```

```
print("Pregel 결과:", result)
# 'foo' + 'foo' = 'foofoo'
```

Pregel 결과: {'b': 'foofoo'}

13.10 채널 타입

Pregel의 가장 핵심적인 추상화는 채널입니다. 채널은 Graph API의 상태 필드와 리듀서에 정확히 대응됩니다. Pregel은 5가지 채널 타입을 제공합니다.

Type	Purpose
LastValue	가장 최근 값을 저장. 입력/출력에 적합한 기본 채널
EphemeralValue	실행 step 내부에서만 유효한 임시 값
Topic	설정 가능한 PubSub. 다중 값 + optional 중복 제거 / 누적
BinaryOperatorAggregate	현재 값과 업데이트에 binary operator 적용 (러닝 토탈 등)
DeltaChannel (beta, langgraph≥1.2)	step별 incremental delta만 저장. 자주 쓰이고 빠르게 자라는 채널(메시지 리스트 등)에 적합

TIP

Graph API에서 `Annotated[list, operator.add]` 타입 힌트를 사용하면, 내부적으로 Pregel의 `BinaryOperatorAggregate` 채널이 생성됩니다. Graph API의 “리듀서”라는 개념이 Pregel 레벨에서는 “채널 타입”으로 표현되는 것입니다. 이 대응 관계를 이해하면 Graph API의 상태 관리 동작을 더 정확히 예측할 수 있습니다.

```
from langgraph.channels import (
    EphemeralValue,
    LastValue,
    Topic,
    BinaryOperatorAggregate,
)
from langgraph.pregel import Pregel, NodeBuilder

# --- 1. LastValue: 최신 값만 유지 ---
node_lv = (
    NodeBuilder()
    .subscribe_only("input")
    .do(lambda x: x.upper())
    .write_to("output")
)

app_lv = Pregel(
    nodes={"node": node_lv},
    channels={
        "input": EphemeralValue(str),
        "output": LastValue(str),
    },
    input_channels=["input"],
```

```

        output_channels=["output"],
    )
    print("LastValue:", app_lv.invoke({"input": "hello"}))

# --- 2. Topic: 여러 값을 누적 ---
node_t1 = (
    NodeBuilder()
    .subscribe_only("a")
    .do(lambda x: x + x)
    .write_to("b", "c")
)

node_t2 = (
    NodeBuilder()
    .subscribe_to("b")
    .do(lambda x: x["b"] + x["b"])
    .write_to("c")
)

app_topic = Pregel(
    nodes={"node1": node_t1, "node2": node_t2},
    channels={
        "a": EphemeralValue(str),
        "b": EphemeralValue(str),
        "c": Topic(str, accumulate=True),
    },
    input_channels=["a"],
    output_channels=["c"],
)
print("Topic:", app_topic.invoke({"a": "foo"}))

# --- 3. BinaryOperatorAggregate: 리듀서 적용 ---
def reducer(current, update):
    if current:
        return current + " | " + update
    return update

node_b1 = (
    NodeBuilder()
    .subscribe_only("a")
    .do(lambda x: x + x)
    .write_to("b", "c")
)

node_b2 = (
    NodeBuilder()
    .subscribe_only("b")
    .do(lambda x: x + x)
    .write_to("c")
)

app_agg = Pregel(
    nodes={"node1": node_b1, "node2": node_b2},
    channels={
        "a": EphemeralValue(str),
        "b": EphemeralValue(str),
        "c": BinaryOperatorAggregate(str, operator=reducer),
    },
    input_channels=["a"],
    output_channels=["c"],
)
print("BinaryOperatorAggregate:", app_agg.invoke({"a": "foo"}))

```

```

LastValue: {'output': 'HELLO'}
Topic: {'c': ['foofoo', 'foofoofoofoo']}
BinaryOperatorAggregate: {'c': 'foofoo | foofoofoofoo'}

```

DeltaChannel (beta, langgraph≥1.2)

DeltaChannel 은 매 step의 full 누적값이 아니라 incremental delta만 저장합니다. 메시지 리스트처럼 자주 쓰이고 무한히 자라는 채널의 checkpoint 비용을 줄이는 데 사용합니다.

- Reducer는 associative해야 하며, write 시점이 아니라 reconstruction 시점에 실행됩니다.
- snapshot_frequency=K 로 K step마다 full snapshot을 한 번 기록해 reconstruction 비용을 제한합니다.
- Bulk reducer 시그니처는 (current_state, sequence_of_writes) → new_state 입니다.

```
from typing import Annotated, Sequence
from typing_extensions import TypedDict
from langgraph.channels import DeltaChannel

def list_reducer(state: list, writes: Sequence[list]) -> list:
    # (current_state, sequence_of_writes) -> new_state
    result = list(state)
    for write in writes:
        result.extend(write)
    return result

class State(TypedDict):
    messages: Annotated[
        list[str],
        DeltaChannel(list_reducer, snapshot_frequency=5),
    ]
```

13.11 슈퍼시스템 실행 모델

채널 타입을 이해했다면, 이제 Pregel이 이 채널들을 어떻게 조율하여 그래프를 실행하는지 살펴봅시다. Pregel은 슈퍼시스템(Super-step) 단위로 실행됩니다. 슈퍼시스템은 Pregel 실행의 기본 시간 단위로, 각 슈퍼시스템에서 실행 준비가 된 모든 노드(액터)가 병렬로 실행됩니다. 모든 노드의 실행이 완료되면 채널이 업데이트되고, 그 후에야 다음 슈퍼시스템이 시작됩니다.

이 모델의 핵심 특성은 동기적 병렬성입니다. 같은 슈퍼시스템 내의 노드들은 서로의 출력을 볼 수 없고, 오직 이전 슈퍼시스템에서 채널에 기록된 값만 참조할 수 있습니다. 이 제약 덕분에 Pregel은 노드 간 경쟁 조건(race condition) 없이 안전하게 병렬 실행을 수행할 수 있습니다.

```
[슈퍼시스템 1] Node A, Node B (병렬 실행)
  ↓ 채널 업데이트
[슈퍼시스템 2] Node C (A, B 결과 기반)
  ↓ 채널 업데이트
[슈퍼시스템 3] Node D
  ↓
END
```

슈퍼시스템의 특징:

- 같은 슈퍼시스템 내 노드는 서로의 출력을 볼 수 없음 (이전 스텝의 채널 값만 참조)
- 모든 노드가 완료되어야 다음 스텝 진행
- 체크포인트가 설정된 경우, 각 슈퍼시스템 후 상태 저장
- 실행할 노드가 없으면 자동 종료

· NOTE

5문장 요약

1. Graph API는 노드와 엣지를 명시적으로 설계해야 할 때 가장 강합니다.
2. Functional API는 `@task` 중심으로 빠르게 작성하고 싶을 때 더 자연스럽습니다.
3. 복잡한 분기와 시각적 디버깅이 중요하다면 Graph API가 유리합니다.
4. 직선형 워크플로와 내구성 태스크 조합은 Functional API가 간결합니다.
5. 두 방식은 경쟁 관계가 아니라 같은 런타임을 공유하는 표현 계층입니다.

13.12 API 선택 기준 가이드

Pregel 런타임의 내부 구조까지 이해했으므로, 이제 이 모든 지식을 종합하여 실용적인 API 선택 기준을 정리합니다. 아래는 프로젝트 초기에 API를 결정할 때 사용할 수 있는 4단계 의사결정 프레임워크입니다. 각 단계에서 자신의 프로젝트 상황에 맞는 조건을 확인하세요.

! WARNING

API 선택은 되돌릴 수 없는 결정이 아닙니다. Functional API로 시작한 프로젝트가 복잡해지면 Graph API로 마이그레이션할 수 있고, 그 반대도 가능합니다. 두 API가 동일한 Pregel 런타임을 공유하므로, 점진적 전환이 가능합니다. “완벽한 선택”보다 “빠르게 시작”하는 것이 더 중요합니다.

1단계: 복잡도 평가

- 노드가 3개 이하이고 선형 흐름 → Functional API
- 조건 분기, 병렬 경로, 순환 구조 → Graph API

2단계: 팀 협업

- 혼자 또는 소규모 팀 → 어느 쪽이든 가능
- 여러 팀원이 각 노드를 담당 → Graph API (시각화와 역할 분리)

3단계: 기존 코드 활용

- 기존 절차적 코드에 LangGraph 기능 추가 → Functional API
- 새로운 워크플로를 처음부터 설계 → Graph API

4단계: 발전 가능성

- 프로토타입에서 시작 → Functional API → 복잡해지면 Graph API로 마이그레이션
- 처음부터 확장성 고려 → Graph API

Part 3 전체를 통해 LangGraph의 모든 핵심 개념을 다루었습니다. 상태 그래프(ch01-02), Functional API(ch03), 워크플로 패턴(ch04), 에이전트 구축(ch05), 지속성(ch06), 스트리밍(ch07), 인터럽트(ch08), 서브그래프(ch09), 프로덕션(ch10-11), 내구성 실행(ch12), 그리고 이 장에서의 API 선택 가이드와 Pregel 런타임까지 – 이제 LangGraph로 프로덕션 수준의 에이전트를 설계하고 구축할 준비가 완료되었습니다.

요약

주제	핵심 내용
----	-------

Graph API	노드/엣지 기반, 시각화 강점, 복잡한 워크플로에 적합
Functional API	데코레이터 기반, 최소 보일러플레이트, 선형 워크플로에 적합
비교	동일 런타임, 함께 사용 가능, 복잡도에 따라 선택
Pregel	LangGraph의 내부 실행 엔진, 액터-채널 모델
채널	<code>LastValue</code> / <code>EphemeralValue</code> / <code>Topic</code> / <code>BinaryOperatorAggregate</code> / <code>DeltaChannel</code> (beta)
슈퍼스텝	동일 레벨 노드 병렬 실행 → 채널 업데이트 → 다음 스텝
선택 기준	복잡도, 팀 협업, 기존 코드, 발전 가능성으로 판단

REFERENCES

- [Choosing between Graph and Functional APIs](#)
- [LangGraph Runtime \(Pregel\)](#)

P A R T

IV

Deep Agents

Framework-Agnostic Agent Building

이 파트에서 다루는 내용

- 1. Deep Agents 소개
- 2. 첫 번째 에이전트
- 3. 커스터마이징
- 4. 스토리지 백엔드
- 5. 서브에이전트와 태스크 위임
- 6. 장기 메모리와 스킬
- 7. 고급 기능
- 8. 에이전트 하네스
- 9. 외부 프레임워크 비교
- 10. 샌드박스 와 ACP

01 Deep Agents 소개

Deep Agents는 LangChain 팀이 개발한 에이전트 하네스 프레임워크로, 복잡한 멀티 스텝 작업을 수행하는 자율 에이전트를 체계적으로 구축할 수 있게 해 준다. 이 장에서는 `Planning`, `Context Management`, `Backends`, `Subagents`, `Memory & Skills` 등 Deep Agents의 핵심 개념 다섯 가지를 소개하고, SDK · Deep Agents Code · ACP 프로토콜의 차이점을 살펴본다. 프레임워크의 전체 구조를 조감함으로써 이후 장에서 다룰 심화 주제들의 기반을 마련한다.

기존 에이전트 프레임워크들이 “LLM + 도구 호출”이라는 최소 단위에 머물렀다면, Deep Agents는 여기에 계획 수립, 컨텍스트 압축, 파일 시스템 추상화, 서브에이전트 위임, 장기 메모리라는 다섯 가지 인프라를 기본 탑재 하여, 장기 실행 자율 에이전트를 위한 포괄적 하네스(harness)를 제공한다. `create_deep_agent()` 한 줄이면 이 모든 기능이 조립된 `CompiledStateGraph` 가 반환되며, LangGraph의 모든 실행 메서드(`invoke`, `stream`, `batch`)를 그대로 활용할 수 있다.

학습 목표

LEARNING OBJECTIVES

- Deep Agents가 무엇인지 이해한다
- SDK와 CLI의 차이를 파악한다
- 핵심 개념 5가지(Planning, Context Management, Backends, Subagents, Memory)를 이해한다
- 다른 프레임워크와의 차이를 비교한다
- 설치 상태를 확인한다

1. Deep Agents란?

Deep Agents는 LangChain 팀이 만든 에이전트 하네스(Agent Harness) 프레임워크입니다. 복잡한 멀티 스텝 작업을 수행하는 자율 에이전트를 쉽게 구축할 수 있도록, 아래 핵심 기능 11종을 내장하고 있습니다:

- 태스크 플래닝 - `write_todos` 로 복잡한 작업을 구조화된 태스크 리스트로 분해
- 파일 시스템 관리 - 가상/로컬 파일 읽기·쓰기·검색 (`ls`, `read_file`, `write_file`, `edit_file`, `glob`, `grep`)
- 파일 시스템 권한(filesystem permissions) - `allow_read` / `allow_write` / `deny_read` 등 경로 단위 ACL
- 서브에이전트 위임 - `task` 도구로 전문 에이전트에 작업 분배
- 장기 메모리 + 스킬 - `AGENTS.md` (`memory`)와 `SKILL.md` (`skills`) 점진 공개

- 컨텍스트 관리 - 자동 오프로딩 + 요약으로 토큰 한도 내에서 정보 관리
- QuickJS interpreter - 샌드박스 JS 인터프리터(`use_interpreter=True`)로 임의 계산 수행
- 샌드박스 실행 환경 - Modal · Daytona · Deno · 로컬 가상 파일시스템(VFS) 백엔드
- Human-in-the-Loop - `interrupt_on={"tool_name": True}` 로 도구별 승인 게이트
- 커스터마이징 - `before_model` / `after_model` / `wrap_model_call` 미들웨어 후
- Context propagation - `context_schema` 와 네임스페이스 키(`"agent:key"`)로 서브에이전트별 설정

“에이전트 하네스”라는 이름이 시사하듯, Deep Agents는 LLM을 단순히 감싸는 래퍼가 아니라 장기 실행 자율 에이전트가 안정적으로 동작하기 위한 운영 인프라 전체를 포괄합니다. 기존 “LLM + 도구 호출” 패러다임에서는 개발자가 계획 수립, 토큰 관리, 파일 영속성 등을 모두 직접 구현해야 했지만, Deep Agents는 이 반복적인 인프라를 표준화하여 제공합니다.

LangChain의 기본 에이전트 컴포넌트 위에 구축되었으며, LangGraph를 실행 엔진으로 사용합니다. 내부적으로 `AgentHarness` 가 모델, 도구, 미들웨어, 상태 스키마를 수집하여 `StateGraph` 를 구성하고, 미들웨어 파이프라인을 적용한 뒤 컴파일합니다. 이 조립 과정은 크게 세 단계로 이루어집니다: (1) 파라미터로 전달된 모델, 도구, 백엔드 등을 수집하고, (2) 각 미들웨어가 빌트인 도구와 시스템 프롬프트를 주입하며, (3) 최종적으로

`StateGraph` 를 컴파일하여 `CompiledStateGraph` 를 반환합니다. 개발자는 이 과정을 알 필요 없이

`create_deep_agent()` 라는 편의 래퍼만 호출하면 됩니다.

! WARNING

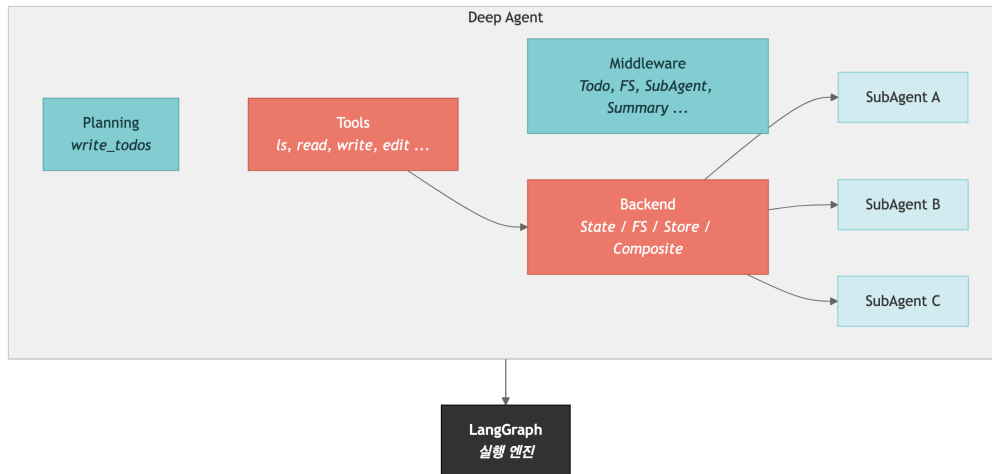
Deep Agents는 LangGraph 위에서 동작하므로, LangGraph의 기본 개념(StateGraph, 노드, 엣지, 체크포인트)을 이해하고 있으면 내부 동작을 훨씬 깊이 파악할 수 있습니다. Part 3(LangGraph)를 먼저 학습하는 것을 권장합니다.

TIP

이 교육 자료의 모델 설정: Deep Agents의 권장 기본 모델은 Anthropic Claude Sonnet 4.6 (`anthropic:claude-sonnet-4-6`)입니다. 본 과정에서는 환경에 따라 OpenAI(`openai:gpt-5.4`), Google(`google_genai:gemini-3.5-flash`)도 함께 사용합니다. Provider 접두 포맷(`provider:model`)을 권장하며, 해당 프로바이더의 API 키(`ANTHROPIC_API_KEY` / `OPENAI_API_KEY` / `GOOGLE_API_KEY`)를 설정해야 합니다.

아키텍처 개요

Deep Agents의 아키텍처는 세 계층으로 구성됩니다. 최하단에는 LangChain의 모델/도구 인터페이스가 위치하고, 중간 계층의 LangGraph가 상태 기반 그래프 실행을 담당하며, 최상위에 Deep Agents 하네스가 계획, 파일시스템, 서브에이전트, 메모리 등 고수준 기능을 통합합니다. 이 계층 구조 덕분에 각 계층의 기능을 독립적으로 교체하거나 확장할 수 있습니다. 예를 들어, LangChain 계층에서 모델만 교체하면 나머지 인프라는 그대로 유지되고, LangGraph 계층에서 체크포인트를 바꾸면 영속성 전략만 변경됩니다.



아래 다이어그램에서 주목할 점은 화살표의 방향입니다. Deep Agents 하네스는 LangGraph와 LangChain을 의존하지만, 반대 방향은 성립하지 않습니다. 즉, LangGraph로 직접 그래프를 구축하던 기존 코드도 Deep Agents 없이 독립적으로 동작할 수 있습니다.

2. SDK vs Code vs ACP

아키텍처를 이해했으니, 이제 Deep Agents를 실제로 사용하는 세 가지 인터페이스를 비교합니다.

Deep Agents는 동일한 AgentHarness 엔진 위에 세 가지 인터페이스를 제공합니다. 프로그래밍 방식으로 에이전트를 앱에 통합하고 싶다면 SDK를, 터미널에서 즉시 코딩 에이전트를 사용하고 싶다면 Deep Agents Code(`deepagents-cli`)를, 외부 클라이언트와 ACP(Agent Client Protocol)로 통신하고 싶다면 ACP 서버 모드를 사용합니다. 세 인터페이스 모두 동일한 코어를 공유하므로, 한쪽에서 익힌 개념(백엔드, 미들웨어, 서브에이전트 등)은 그대로 다른 쪽에 적용됩니다.

구분	Deep Agents SDK	Deep Agents Code	ACP 서버
패키지	<code>deepagents</code>	<code>deepagents-cli</code>	<code>deepagents-cli (--acp)</code>
용도	프로그래밍 방식으로 에이전트 구축	터미널에서 직접 코딩 에이전트 사용	Agent Client Protocol로 외부 클라이언트 연결

설치	<code>pip install -qU deepagents langchain-{provider}</code>	<code>uv tool install deepagents-cli</code>	<code>uv tool install deepagents-cli</code>
사용 방식	Python 코드에서 <code>create_deep_agent()</code> 호출	터미널에서 <code>deepagents</code> 실행	<code>deepagents --acp</code> 후 ACP 클라이언트가 stdio로 접속
커스터마이징	완전한 API 접근 (도구, 백엔드, 미들웨어)	<code>.deepagents/config.json</code> + 슬래시 커맨드	SDK와 동일한 설정 파일을 ACP로 노출
적합한 경우	앱 통합, 자동화 파이프라인, 커스텀 에이전트 서비스	대화형 코딩 어시스턴트	Zed · 외부 IDE · 사용자 정의 클라이언트 연동

➡ TIP

이 교육 자료에서는 SDK를 중심으로 다룹니다. Code/ACP는 Part 4의 14·15장에서 별도로 다룹니다. 설치 명령은 프로바이더에 따라 패키지를 함께 지정합니다 — 예: `pip install -qU deepagents langchain-anthropic` 또는 `pip install -qU deepagents langchain-openai`.

3. 핵심 개념 5가지 (Planning · Context Management · Backends · Subagents · Memory & Skills)

세 인터페이스를 확인했으니, 이제 Deep Agents의 내부를 구성하는 다섯 가지 핵심 축을 하나씩 살펴봅니다. 이 개념들은 이후 장에서 각각 독립된 챕터로 심화되므로, 여기서는 전체적인 역할과 상호 관계를 파악하는 데 집중합니다. 다섯 축은 서로 독립적이면서도 상호 보완적입니다. 예를 들어, 서브에이전트가 작업을 위임할 때 백엔드를 통해 결과를 저장하고, 장기 메모리/스킬에서 이전 작업 패턴이나 도메인 지식을 참조합니다.

3.1 Planning (태스크 플래닝)

에이전트는 `write_todos` / `read_todos` 빌트인 도구를 사용하여 복잡한 작업을 구조화된 태스크 리스트로 분해합니다. 각 태스크는 `pending` → `in_progress` → `completed` 상태로 추적됩니다. 이 계획 수립 능력은 단순한 메모가 아니라, 에이전트가 멀티 스텝 작업의 진행 상황을 자기 자신에게 보고하며 다음 행동을 결정하는 자기 관리 메커니즘입니다. 내부적으로 `write_todos`는 `TodoListMiddleware`에 의해 에이전트 상태에 주입되며, 에이전트가 매 턴마다 현재 태스크 상태를 확인하고 다음 행동을 결정하는 루프 기반 계획 수립을 가능하게 합니다. 복잡한 리서치 작업에서 에이전트가 “조사 → 분석 → 보고서 작성”의 흐름을 스스로 관리하는 이유가 바로 이 메커니즘 덕분입니다.

➡ TIP

`write_todos`는 에이전트의 시스템 프롬프트에 태스크 관리 지침이 포함되어 있어, 모델이 자연스럽게 계획을 수립합니다. 별도의 설정 없이도 기본적으로 활성화됩니다.

3.2 Context Management (컨텍스트 관리)

장기 실행 에이전트의 최대 적은 컨텍스트 윈도우 한계입니다. 수십 번의 도구 호출과 긴 응답이 누적되면 토큰 한도를 빠르게 초과하여 에이전트가 동작을 멈추게 됩니다. Deep Agents는 두 가지 자동 기법으로 이 문제를 해결합니다:

- 오프로딩: 20,000 토큰 이상의 콘텐츠는 파일시스템 백엔드에 저장하고 포인터만 컨텍스트에 유지합니다. 원본은 언제든지 다시 읽을 수 있으므로 정보 손실이 없습니다. 이 방식은 에이전트의 컨텍스트를 “작업 메모리(RAM)”처럼 관리하고, 백엔드를 “디스크 저장소”처럼 활용하는 것과 유사합니다.
- 요약: 컨텍스트가 모델 윈도우의 약 85%에 도달하면, `SummarizationMiddleware` 가 대화 이력을 구조화된 요약으로 자동 압축합니다. 요약은 단순한 텍스트 줄이기가 아니라, 핵심 결정 사항, 중간 결과, 아직 완료되지 않은 태스크를 구조적으로 보존하는 방식으로 이루어집니다.

이 두 기법이 결합되면 에이전트는 이론적으로 무제한에 가까운 장기 작업을 수행할 수 있습니다. 컨텍스트 관리는 8장에서 심화합니다.

3.3 Backends (스토리지 백엔드)

에이전트의 파일 도구(`write_file` , `read_file` , `ls` 등)가 실제로 데이터를 저장하고 읽는 계층은 플러그블 백엔드로 추상화됩니다. 백엔드를 교체하는 것만으로 에이전트의 저장소 전략을 완전히 바꿀 수 있으며, 에이전트 코드 자체는 한 줄도 변경할 필요가 없습니다. 이것이 백엔드 추상화의 핵심 가치입니다:

- `StateBackend` — 에이전트 상태(LangGraph state)에 파일 저장. 프로세스 종료 시 소멸하는 에페메럴 스토리지입니다. 기본값이며 추가 설정이 필요 없습니다.
- `FilesystemBackend` — 로컬 디스크에 직접 접근합니다. `root_dir` / `DATA_DIR` 로 루트를 설정하고 `virtual_mode=True` 로 `..` / `~` / 외부 절대경로를 차단합니다.
- `StoreBackend` — LangGraph `BaseStore` 를 활용하여 크로스 스레드 영속 저장소를 제공합니다. 사용자 선택 호나 학습된 패턴을 세션을 넘어 유지할 때 사용합니다.
- `CompositeBackend` — 경로 프리픽스에 따라 서로 다른 백엔드로 라우팅합니다(예: `/memories/` 는 영속, 나머지는 에페메럴). 실전에서 가장 자주 사용되는 패턴입니다.
- `ContextHubBackend` — Context Hub에서 lazy fetch + write-through 방식으로 원격 파일을 동기화합니다. `namespace=lambda rt: ...` 로 런타임 네임스페이스를 동적으로 결정합니다(`deepagents ≥ 0.5.2`).
- 샌드박스 백엔드 — Modal · Daytona · Deno · 로컬 VFS — 신뢰할 수 없는 코드 실행과 파일 격리를 동시에 제공합니다. QuickJS interpreter(`use_interpreter=True`)는 별도 미들웨어로 JS 코드를 격리 실행합니다.

커스텀 백엔드가 필요하면 `BackendProtocol` 을 구현하면 됩니다. 4장에서 각 백엔드를 상세히 다룹니다.

3.4 Subagents (서브에이전트)

에이전트가 도구를 반복 호출하면 중간 결과가 컨텍스트 윈도우를 빠르게 채우는 컨텍스트 불로트 문제가 발생합니다. 예를 들어, 10개의 파일을 순차적으로 읽고 분석하는 작업에서 각 파일의 전체 내용이 메인 에이전트의 컨텍스트에 누적되면, 정작 중요한 분석 결과를 생성할 공간이 부족해집니다.

서브에이전트는 이 문제를 근본적으로 해결합니다. 전문 서브에이전트가 격리된 컨텍스트에서 작업을 수행하고, 압축된 최종 결과만 메인 에이전트에 반환합니다. 이는 마치 팀장이 팀원에게 작업을 위임하고 요약 보고서만 받는 것과 같습니다. 서브에이전트는 명시적 정의(`subagents` 파라미터)와 동적 생성(`create_subagent` 도구) 두 가지 방식으로 사용할 수 있습니다. 각 서브에이전트는 독립된 컨텍스트 윈도우, 도구 세트, 시스템 프롬프트를 가지므로, 메인 에이전트의 토큰 예산에 영향을 주지 않습니다.

3.5 Memory & Skills (장기 메모리와 스킬)

기본 에이전트는 대화 스레드가 종료되면 모든 정보를 잊습니다. 이것은 매번 새로운 직원을 고용하는 것과 같습니다. 장기 메모리는 에이전트가 이전 대화에서 학습한 패턴, 사용자 선호도, 프로젝트 규칙 등을 세션을 넘어 유지할 수 있게 합니다. 장기 메모리는 두 가지 메커니즘으로 제공됩니다:

- `AGENTS.md` : `memory` 파라미터로 지정하면, 에이전트 시작 시 해당 파일이 항상 시스템 프롬프트에 주입됩니다. 프로젝트 컨벤션이나 사용자 선호도처럼 모든 대화에 적용되어야 하는 규칙에 적합합니다.
- `SKILL.md` : `skills` 파라미터로 스킬 디렉토리를 지정하면, Progressive Disclosure(점진적 공개)를 통해 필요한 전문 지식만 온디맨드로 로드합니다. 처음에는 프론트매터(이름, 설명)만 로드되고, 에이전트가 관련성을 판단한 스킬의 전체 내용을 그때 로드합니다.

TIP

`AGENTS.md` 는 항상 로드되므로 간결할수록 좋습니다. 반면 `SKILL.md` 는 대용량(최대 10MB)도 가능합니다. 이 차이를 활용하면 토큰 효율을 크게 개선할 수 있습니다.

다섯 가지 핵심 개념을 이해했습니다. 이 개념들은 서로 독립적으로 사용할 수도 있지만, 실전에서는 유기적으로 결합되어 강력한 시너지를 발휘합니다. 예를 들어, 메인 에이전트가 Planning으로 작업을 분해하고, 각 태스크를 Subagent에게 위임하며, 결과를 Backend에 저장하고, Context Management로 토큰을 효율적으로 관리하고, Memory로 학습한 패턴을 재활용하는 전체 흐름이 하나의 `create_deep_agent()` 호출로 구성됩니다. 이제 이런 통합 인프라를 제공하는 Deep Agents가 다른 프레임워크와 어떤 차별점을 가지는지 비교해 봅니다.

4. 다른 프레임워크와의 비교

기능	LangChain Deep Agents	OpenCode	Claude Agent SDK
모델 지원	모델 무관 (Anthropic, OpenAI 등 100+)	75+ 프로바이더 (Ollama 포함)	Claude 전용
라이선스	MIT	MIT	MIT (SDK) / 독점 (Claude Code)
SDK	Python, TypeScript + CLI	터미널, 데스크톱, IDE	Python, TypeScript
샌드박스	도구로 통합 (Modal, Daytona 등)	미지원	미지원
플러그블 백엔드	O (State, FS, Store, Composite)	X	X
타임 트래블	O (LangGraph)	X	O
관측성	LangSmith 네이티브	X	X
파일 도구 기본 내장	O	O	O

Human-in-the-Loop	O (미들웨어)	O	O
-------------------	----------	---	---

TIP

Deep Agents의 핵심 차별점: 플러그블 백엔드, 샌드박스 통합, LangSmith 관측성

5. 설치 확인

프레임워크의 개념과 차별점을 확인했으니, 실제 개발 환경이 올바르게 구성되었는지 검증합니다. 설치에 사용할 프로바이더의 LangChain 패키지를 함께 지정합니다.

```
# Anthropic 권장 기본
pip install -qU deepagents langchain-anthropic

# OpenAI 사용 시
pip install -qU deepagents langchain-openai

# Google Gemini 사용 시
pip install -qU deepagents langchain-google-genai

# OpenRouter 등 OpenAI 호환 게이트웨이
pip install -qU deepagents langchain-openai # base_url만 교체하여 사용
```

아래 코드는 `deepagents` 패키지와 주요 의존성의 버전을 출력합니다. 모든 임포트가 성공하면 개발 환경이 정상적으로 설정된 것입니다.

```
# deepagents 패키지 버전 확인
import deepagents
print(f"deepagents 버전: {deepagents.__version__}")
```

deepagents 버전: 0.4.4

```
# 주요 모듈 임포트 확인
from deepagents import create_deep_agent, SubAgent, CompiledSubAgent
from deepagents import FilesystemMiddleware, MemoryMiddleware, SubAgentMiddleware
from deepagents.backends import StateBackend, FilesystemBackend, StoreBackend, CompositeBackend
from deepagents.backends.protocol import BackendProtocol

print("모든 주요 모듈을 성공적으로 임포트했습니다!")
```

모든 주요 모듈을 성공적으로 임포트했습니다!

```
# 의존 패키지 버전 확인
import importlib.metadata

print(f"langchain 버전: {importlib.metadata.version('langchain')}")
print(f"langgraph 버전: {importlib.metadata.version('langgraph')}")
```


langchain 버전: 1.2.10
langgraph 버전: 1.0.10

설치가 확인되었으니, Deep Agents의 핵심 함수인 `create_deep_agent()` 의 시그니처를 살펴봅니다.

`create_deep_agent()` 는 Deep Agents의 유일한 진입점이며, 반환 타입은 LangGraph의 `CompiledStateGraph` 입니다. 아래에서 이 함수가 받는 전체 파라미터를 확인합니다. `model` , `tools` , `system_prompt` 는 기본 구성이고, `subagents` , `backend` , `memory` , `skills` , `interrupt_on` , `middleware` , `response_format` , `context_schema` , `checkpointner` 등이 심화 파라미터입니다. 각 파라미터의 상세한 사용법은 2장(기본 사용)과 3장(커스터마이징)에서 다룹니다.

! WARNING

아래 출력에서 대부분의 파라미터 기본값이 `None` 인 점에 주목하세요. 이는 Deep Agents가 “합리적 기본값 (sensible defaults)” 원칙을 따르기 때문입니다. `model` 만 전달해도 나머지는 자동으로 구성됩니다.

```
# create_deep_agent 함수 시그니처 확인
import inspect

sig = inspect.signature(create_deep_agent)
print("create_deep_agent() 파라미터:")
for name, param in sig.parameters.items():
    default = param.default if param.default is not inspect.Parameter.empty else "(필수)"
    print(f"  - {name}: {default}")
```

```
create_deep_agent() 파라미터:
- model: None
- tools: None
- system_prompt: None
- middleware: ()
- subagents: None
- skills: None
- memory: None
- response_format: None
- context_schema: None
- checkpointner: None
- store: None
- backend: None
- interrupt_on: None
- debug: False
- name: None
- cache: None
```

핵심 정리

항목	내용
Deep Agents	LangChain 기반 에이전트 하네스 프레임워크
핵심 함수	<code>create_deep_agent()</code>
실행 엔진	LangGraph (<code>CompiledStateGraph</code> 반환)

기본 모델 권장	Anthropic Claude Sonnet 4.6 – "anthropic:claude-sonnet-4-6" (OpenAI는 "openai:gpt-5.4")
핵심 개념	Planning, Context Management, Backends, Subagents, Memory & Skills
빌트인 도구	write_todos , ls , read_file , write_file , edit_file , glob , grep

이 장에서 Deep Agents의 전체 아키텍처와 핵심 개념 다섯 가지를 조감했습니다. 다음 장에서는 `create_deep_agent()` 를 직접 호출하여 첫 번째 에이전트를 생성하고, `invoke()` 와 `stream()` 으로 실행하는 실습을 진행합니다.

02 첫 번째 에이전트 만들기

에이전트를 직접 만들어 보는 것이 프레임워크를 이해하는 가장 빠른 방법이다. 이 장에서는

`create_deep_agent()` 함수를 사용하여 기본 에이전트를 생성하고, `invoke()` 와 `stream()` 으로 실행하는 전 과정을 실습한다. Tavily 검색 도구를 연동한 리서치 에이전트 예제를 통해 커스텀 도구 추가 방법까지 다룬다.

1장에서 살펴본 아키텍처를 코드로 옮기는 첫 단계다. `create_deep_agent()` 가 반환하는 `CompiledStateGraph` 객체는 LangGraph의 표준 실행 인터페이스를 그대로 제공하므로, 기존 LangGraph 경험이 있다면 즉시 익숙하게 사용할 수 있다. `model` 파라미터 하나만 전달하면 나머지 인프라(빌트인 도구, 미들웨어 스택, 상태 스키마)가 합리적 기본값으로 자동 조립되므로, 최소한의 코드로 완전한 에이전트를 구동할 수 있다. 이후 `invoke` , `stream` , `batch` 등 LangGraph의 모든 실행 메서드를 그대로 활용한다.

학습 목표

LEARNING OBJECTIVES

- `.env` 파일에서 API 키를 로드하는 방법을 익힌다
- `create_deep_agent()` 로 기본 에이전트를 생성한다
- `agent.invoke()` 와 `agent.stream()` 으로 에이전트를 실행한다
- Tavily 검색 도구를 연동하는 리서치 에이전트를 만든다
- 빌트인 도구의 종류와 역할을 이해한다

1. 프로젝트 부트스트랩과 API 키 설정

에이전트를 만들기 전에 프로젝트와 API 키부터 설정합니다. Deep Agents 자체는 모델 무관(model-agnostic)이며, 권장 기본 모델은 Anthropic `claude-sonnet-4-6` 입니다. OpenAI(`gpt-5.4`), Google(`gemini-3.5-flash`), OpenRouter 등 다양한 프로바이더 중 환경에 맞는 것을 선택해 키를 발급받습니다. 리서치 에이전트 예제에서는 Tavily 검색 API도 사용합니다.

uv 기반 프로젝트 부트스트랩

권장하는 단계별 흐름은 다음과 같습니다. `uv` 는 의존성 잠금과 가상환경을 자동으로 관리하므로 `pip` 보다 재현성이 높습니다.

```
# 1) 프로젝트 초기화
uv init my-deepagent
cd my-deepagent

# 2) deepagents + 사용할 프로바이더 패키지 추가
uv add deepagents langchain-anthropic # 권장 기본
# uv add deepagents langchain-openai
# uv add deepagents langchain-google-genai
uv add tavily-python python-dotenv

# 3) 잠금 파일 동기화
uv sync
```

프로바이더별 API 키 분기

사용할 프로바이더에 따라 `.env` 에 필요한 키를 채웁니다.

```
# Anthropic (권장 기본)
ANTHROPIC_API_KEY=your-key-here

# OpenAI
OPENAI_API_KEY=your-key-here

# Google Gemini
GOOGLE_API_KEY=your-key-here

# OpenRouter (OpenAI 호환)
OPENAI_API_KEY=your-openrouter-key
OPENAI_BASE_URL=https://openrouter.ai/api/v1

# 공통 - 리서치 예제에서 사용
TAVILY_API_KEY=your-key-here
```

TIP

`.env.example` 파일을 복사하여 `.env` 로 만들면 됩니다. 모든 키를 다 설정할 필요는 없고, 자신이 사용할 프로바이더의 키만 채우면 됩니다.

```
# 환경 변수 로드
from dotenv import load_dotenv
import os

load_dotenv()

# 권장 기본 - Anthropic
assert os.environ.get("ANTHROPIC_API_KEY"), "ANTHROPIC_API_KEY가 설정되지 않았습니다!"
assert os.environ.get("TAVILY_API_KEY"), "TAVILY_API_KEY가 설정되지 않았습니다!"
print("API 키가 정상적으로 로드되었습니다.")
```

API 키가 정상적으로 로드되었습니다.

2. 가장 간단한 에이전트 만들기

API 키가 준비되었으니, 이제 첫 번째 에이전트를 생성합니다.

`create_deep_agent()` 는 Deep Agents의 유일한 진입점입니다. 이 함수는 내부적으로 `AgentHarness` 를 통해 모델, 빌트인 도구, 미들웨어 스택, 상태 스키마를 조립한 뒤 `StateGraph` 를 컴파일하여 반환합니다. `model` 파라미터만 전달하면 나머지는 합리적인 기본값으로 자동 구성됩니다. 이때 `model` 이 유일한 필수 파라미터라는 점이 중요합니다. 어떤 LLM을 사용할지만 결정하면, 빌트인 도구(`write_todos` , `ls` , `read_file` 등), 컨텍스트 압축 미들웨어, 에페메럴 백엔드가 자동으로 포함됩니다.

아래 코드에서 `type(agent).__name__` 이 `CompiledStateGraph` 로 출력되는지 확인하세요. 이는

`create_deep_agent()` 가 LangGraph의 표준 그래프 객체를 반환한다는 것을 의미하며, Part 3에서 배운 모든 실행 메서드를 그대로 사용할 수 있음을 뜻합니다.

```
from deepagents import create_deep_agent

# 권장 기본 - Anthropic Claude Sonnet 4.6
# 문자열 한 줄로 provider:model 포맷 전달
agent = create_deep_agent(model="anthropic:claude-sonnet-4-6")

# (대안 1) OpenAI gpt-5.4
# agent = create_deep_agent(model="openai:gpt-5.4")

# (대안 2) Google Gemini 3.5 Flash
# agent = create_deep_agent(model="google_genai:gemini-3.5-flash")

# (대안 3) ChatModel 객체로 세밀 제어
# from langchain_anthropic import ChatAnthropic
# model = ChatAnthropic(model="claude-sonnet-4-6", temperature=0)
# agent = create_deep_agent(model=model)

print(f"에이전트 타입: {type(agent).__name__}")
print("에이전트가 성공적으로 생성되었습니다!")
```

```
에이전트 타입: CompiledStateGraph
에이전트가 성공적으로 생성되었습니다!
```

`create_deep_agent()` 는 LangGraph의 `CompiledStateGraph` 를 반환합니다. 따라서 LangGraph의 모든 실행 메서드(`invoke` , `stream` , `batch` 등)를 사용할 수 있습니다. 이것은 Deep Agents의 핵심 설계 철학을 반영합니다: 새로운 실행 인터페이스를 만들지 않고, 이미 검증된 LangGraph 위에서 동작합니다. 기존 LangGraph 프로젝트에 Deep Agents를 점진적으로 도입할 수 있는 이유이기도 합니다.

! WARNING

`create_deep_agent()` 에 `model` 을 전달하지 않으면 기본 모델이 사용됩니다. 프로덕션 환경에서는 항상 명시적으로 모델을 지정하는 것이 좋습니다. 모델에 따라 도구 호출 성능, 토큰 한도, 비용이 크게 달라지므로, 용도에 맞는 모델을 선택하세요.

➡ TIP

`create_deep_agent()` 는 한 번 호출하면 불변(immutable) 그래프를 반환합니다. 에이전트 설정을 바꾸려면 새로 `create_deep_agent()` 를 호출해야 합니다. 이 불변성 덕분에 같은 에이전트 객체를 여러 스레드에서 안전하게 공유할 수 있습니다.

3. 에이전트 실행 — `invoke()`

에이전트를 생성했으니 실제로 메시지를 보내서 실행해 봅시다. `invoke()` 는 가장 기본적인 실행 방식으로, 에이전트가 모든 도구 호출을 완료하고 최종 응답을 생성할 때까지 동기적으로 블로킹합니다.

입력 형식은 LangGraph 표준인 `{ "messages": [{ "role": "user", "content": "..."}] }` 딕셔너리입니다. 반환값도 동일한 형식의 딕셔너리이며, `result["messages"][-1].content` 로 최종 응답 텍스트에 접근할 수 있습니다. 에이전트가 도구를 호출한 경우, 중간 단계의 도구 호출 메시지와 도구 응답 메시지도 `messages` 리스트에 포함됩니다.

TIP

`invoke()` 의 반환값에서 `result["messages"]` 를 출력하면 에이전트의 전체 사고 과정(도구 호출 포함)을 확인할 수 있습니다. 디버깅할 때 매우 유용합니다.

4. Tavily 검색 도구 연동 – 리서치 에이전트

`invoke()` 로 기본 에이전트를 실행해 보았습니다. 하지만 기본 에이전트는 빌트인 파일 도구만 사용할 수 있어, 외부 정보를 가져올 수 없습니다. 실제 업무에서는 웹 검색, 데이터베이스 조회, API 호출 등 외부 도구가 필요합니다. `tools` 파라미터에 Python 함수 목록을 전달하면, Deep Agents가 자동으로 도구 스키마를 생성하여 LLM에 제공합니다.

여기서는 Tavily 웹 검색 API를 도구로 연동하여 리서치 에이전트를 만듭니다. Tavily는 AI 에이전트에 최적화된 검색 API로, 웹 페이지의 핵심 내용을 구조화된 형태로 반환합니다.

도구 정의 방법

Deep Agents(및 LangChain)는 Python 함수를 분석하여 도구 메타데이터를 자동 생성합니다. 함수 이름이 도구 이름이 되고, docstring이 도구 설명(LLM이 도구 선택 시 참조)으로, 타입 힌트가 파라미터 JSON Schema로 변환됩니다. 따라서 도구 함수를 작성할 때 docstring과 타입 힌트를 정확하게 기술하는 것이 매우 중요합니다. docstring이 부실하면 LLM이 언제 이 도구를 사용해야 하는지 판단하지 못하고, 타입 힌트가 없으면 파라미터 검증이 불가능합니다.

! WARNING

도구 함수의 docstring은 LLM이 도구를 선택하는 데 결정적인 역할을 합니다. “검색합니다”처럼 모호한 설명보다 “인터넷에서 최신 뉴스, 기술 문서, 학술 논문을 검색합니다”처럼 구체적으로 작성하세요. docstring이 없는 함수는 도구로 등록되지만, LLM이 올바르게 선택하지 못할 수 있습니다.

```
from typing import Literal
from tavily import TavilyClient

tavily_client = TavilyClient(api_key=os.environ["TAVILY_API_KEY"])

def internet_search(
```

```

query: str,
max_results: int = 5,
topic: Literal["general", "news", "finance"] = "general",
include_raw_content: bool = False,
) -> dict:
    """인터넷에서 정보를 검색합니다.

    Args:
        query: 검색할 질문 또는 키워드
        max_results: 반환할 최대 결과 수
        topic: 검색 주제 카테고리
        include_raw_content: 원본 콘텐츠 포함 여부
    """
    return tavily_client.search(
        query,
        max_results=max_results,
        include_raw_content=include_raw_content,
        topic=topic,
    )

print(f"도구 이름: {internet_search.__name__}")
print(f"도구 설명: {internet_search.__doc__.strip().splitlines()[0]}")

```

```

도구 이름: internet_search
도구 설명: 인터넷에서 정보를 검색합니다.

```

```

# 리서치 에이전트 생성 - 검색 도구 + 커스텀 시스템 프롬프트
research_agent = create_deep_agent(
    model="anthropic:claude-sonnet-4-6",
    tools=[internet_search],
    system_prompt="당신은 전문 리서처입니다. 사용자의 질문에 대해 인터넷 검색을 수행하고, 결과를 정리하여 한국어로 보고서를 작성합니다.",
)

print("리서치 에이전트가 생성되었습니다!")

```

```

리서치 에이전트가 생성되었습니다!

```

5. 빌트인 도구 확인

커스텀 도구를 추가하는 방법을 배웠으니, 이제 `create_deep_agent()` 가 별도 설정 없이 자동으로 추가하는 빌트인 도구들을 확인합니다. 이 도구들은 `FilesystemMiddleware` 와 `TodoListMiddleware` 가 주입하며, 에이전트가 파일 시스템을 탐색하고 태스크를 관리하는 데 필수적인 역할을 합니다:

도구	설명
<code>write_todos</code>	구조화된 태스크 리스트 관리 (pending → in_progress → completed)
<code>ls</code>	디렉토리 내용 목록 (메타데이터 포함)
<code>read_file</code>	파일 읽기 (줄 번호 포함, 이미지 지원)
<code>write_file</code>	새 파일 생성

<code>edit_file</code>	파일 내 텍스트 교체 (<code>old_string</code> → <code>new_string</code>)
<code>glob</code>	패턴 기반 파일 검색 (예: <code>**/*.py</code>)
<code>grep</code>	파일 내용 검색 (정규식 지원)
<code>task</code>	서브에이전트 호출 (서브에이전트 설정 시 자동 추가)

TIP

이 도구들은 모두 백엔드(Backend)를 통해 동작합니다. 기본값은 `StateBackend` 로, 에이전트 상태에 파일이 저장됩니다. 4장에서 백엔드를 교체하는 방법을 다룹니다.

6. 스트리밍 출력 – `stream()`

`invoke()` 는 간단하지만, 장기 실행 에이전트의 경우 최종 결과만 기다리면 사용자 경험이 나빠집니다. 에이전트가 10개의 도구를 순차적으로 호출하는 동안 사용자는 아무런 피드백 없이 기다려야 하기 때문입니다.

`agent.stream()` 을 사용하면 에이전트가 도구를 호출하고 응답을 생성하는 과정을 실시간으로 관찰할 수 있습니다. Deep Agents는 LangGraph의 스트리밍 인프라 위에서 동작하므로, `stream_mode` 에 따라 다른 수준의 정보를 받을 수 있습니다:

- `"updates"` – 각 단계(노드) 완료 시 상태 업데이트를 반환합니다. 에이전트의 진행 상황을 단계별로 추적할 때 사용합니다.
- `"messages"` – LLM이 생성하는 개별 토큰을 실시간으로 스트리밍합니다. ChatGPT와 같은 실시간 타이핑 효과를 구현할 때 사용합니다.
- `"custom"` – 사용자 정의 이벤트를 수신합니다. 특정 도구 호출 결과를 UI에 실시간으로 반영하는 등 고급 사용 사례에 적합합니다.

TIP

프로덕션 환경에서는 `stream_mode="updates"` 가 가장 실용적입니다. 각 도구 호출과 응답을 단계별로 추적할 수 있어, 사용자에게 “검색 중...”, “분석 중...”, “보고서 작성 중...” 같은 진행 상태를 표시할 수 있습니다.

NOTE

`stream()` 과 `invoke()` 는 입력 형식이 동일합니다. 기존 `invoke()` 코드를 `stream()` 으로 전환할 때 입력 부분은 변경할 필요가 없으며, 반환값 처리 방식만 이터레이터로 바꾸면 됩니다.

핵심 정리

항목	내용
에이전트 생성	<code>create_deep_agent(model, tools, system_prompt)</code>
동기 실행	<code>agent.invoke({"messages": [...]})</code>
스트리밍 실행	<code>agent.stream({"messages": [...]} , stream_mode="updates")</code>
커스텀 도구	Python 함수 + docstring + 타입 힌트
모델 포맷	Provider 접두 권장 — "anthropic:claude-sonnet-4-6" , "openai:gpt-5.4" , "google_genai:gemini-3.5-flash" . ChatModel 객체도 가능

기본 에이전트의 생성과 실행을 마쳤습니다. 다음 장에서는 모델 선택, 시스템 프롬프트 작성, 구조화된 출력, 미들웨어 아키텍처 등 에이전트를 목적에 맞게 세밀하게 커스터마이징하는 방법을 다룹니다.

03 에이전트 커스터마이징

기본 에이전트를 만들었다면, 다음 단계는 목적에 맞게 세밀하게 조정하는 것이다. 이 장에서는 모델 선택, 시스템 프롬프트 작성, `docstring` 기반 커스텀 도구 정의, `response_format` 을 활용한 Pydantic 구조화 출력, 그리고 미들웨어 아키텍처를 학습한다. 이 다섯 가지 커스터마이징 축을 이해하면 거의 모든 도메인에 맞는 에이전트를 설계할 수 있다.

2장에서 만든 기본 에이전트는 범용적이었다. 실무에서는 특정 도메인의 전문가 에이전트가 필요하다. 예를 들어 금융 분석 에이전트는 정형화된 보고서를 출력해야 하고, 코딩 에이전트는 특정 언어의 컨벤션을 따라야 한다.

`create_deep_agent()` 의 파라미터를 조합하면 모델 선택부터 출력 스키마까지 에이전트의 모든 측면을 제어할 수 있다. 이 장에서 다루는 다섯 가지 커스터마이징 축(모델, 시스템 프롬프트, 도구, 구조화된 출력, 미들웨어)은 실전 에이전트 개발의 기본 레시피다.

학습 목표

LEARNING OBJECTIVES

- 다양한 LLM 프로바이더와 모델을 선택하는 방법을 익힌다
- 효과적인 시스템 프롬프트를 작성한다
- `docstring` 기반 커스텀 도구를 만든다
- `response_format` 으로 구조화된 출력(Pydantic)을 생성한다
- 미들웨어 아키텍처를 이해한다

```
# 환경 설정
from dotenv import load_dotenv
import os

load_dotenv()
assert os.environ.get("ANTHROPIC_API_KEY"), "ANTHROPIC_API_KEY가 설정되지 않았습니다!"
print("환경 설정 완료")

# 권장 기본 - Anthropic Claude Sonnet 4.6 (provider:model 포맷)
from deepagents import create_deep_agent

model = "anthropic:claude-sonnet-4-6"
print(f"기본 모델: {model}")
```

환경 설정 완료

기본 모델: anthropic:claude-sonnet-4-6

0. create_deep_agent() 전체 시그니처 (17 파라미터)

세부 커스터마이징에 들어가기 전, `create_deep_agent()` 가 받는 17개 파라미터의 전체 시그니처를 한 번에 조감합니다. 이 모든 항목은 선택적이며, `model` 하나만으로도 에이전트가 작동합니다.

```
def create_deep_agent(
    model=None,           # 1. LLM (ChatModel 객체 또는 "provider:model")
    tools=None,           # 2. 커스텀 도구 리스트
    system_prompt=None,   # 3. 사용자 시스템 프롬프트 (USER)
    middleware=(),        # 4. 추가 미들웨어 (사용자 정의)
    subagents=None,       # 5. SubAgent dict 리스트 / CompiledSubAgent
    skills=None,          # 6. SKILL.md 디렉터리 (progressive disclosure)
    memory=None,          # 7. AGENTS.md 경로 (항상 주입)
    response_format=None, # 8. Pydantic 구조화 출력 스키마
    context_schema=None,  # 9. 런타임 컨텍스트 TypedDict (서브에이전트 전파)
    checkpointer=None,    # 10. LangGraph Checkpointer (지속성)
    store=None,           # 11. LangGraph BaseStore (StoreBackend용)
    cache=None,           # 12. 프롬프트 캐시 설정 (Anthropic 등)
    backend=None,         # 13. BackendProtocol 또는 lambda runtime: ...
    permissions=None,     # 14. allow_read/allow_write/deny_* ACL
    interrupt_on=None,    # 15. {"tool_name": True} HITL 게이트
    debug=False,          # 16. 디버그 로깅
    name=None,            # 17. 그래프 이름
):
    ...
```

TIP

`permissions`, `context_schema`, `cache` 는 deepagents 최신 버전($\geq 0.5.0$)에서 추가된 파라미터입니다. 구버전에서는 미들웨어로 직접 추가해야 합니다.

시스템 프롬프트 조립 순서

`system_prompt` 에 전달한 텍스트는 그대로 사용되지 않고, Deep Agents의 내부 프롬프트 빌더가 세 부분을 정해진 순서로 조립합니다.

```
[USER]    system_prompt (사용자가 전달한 문자열)
↓
[BASE]    하네스 BASE 프롬프트 (ToDoList/Filesystem/SubAgent 사용법)
+ CUSTOM  미들웨어가 주입하는 도구 설명 (Memory · Skills · Permissions 등)
↓
[SUFFIX]  하네스 SUFFIX (안전 가이드, 출력 규약)
```

즉, USER → BASE/CUSTOM → SUFFIX 순으로 결합됩니다. 사용자는 USER 부분만 채우면 되고, BASE/SUFFIX는 미들웨어가 자동으로 관리합니다. 덮어쓰지 않고 추가하는 구조이므로 빌트인 도구 사용법을 반복할 필요가 없습니다.

1. 모델 선택

커스터마이징의 첫 번째 축은 모델 선택입니다. 에이전트의 추론 능력, 도구 호출 정확도, 응답 품질은 모두 어떤 LLM을 사용하느냐에 크게 좌우됩니다.

Deep Agents는 LangChain ChatModel 객체 또는 `provider:model` 포맷으로 다양한 LLM을 지원합니다. 이 유연성 덕분에 프로바이더 잠금(lock-in) 없이 프로젝트 요구사항에 가장 적합한 모델을 선택할 수 있습니다. 동일한 에이전트 코드를 유지하면서 모델만 교체하여 성능과 비용을 최적화하는 것도 가능합니다.

본 노트북에서는 Anthropic Claude Sonnet 4.6(`anthropic:claude-sonnet-4-6`)을 권장 기본 모델로 사용합니다. OpenAI 예제는 `gpt-5.4` 를 사용합니다.

프로바이더	모델 예시 (<code>provider:model</code>)	환경 변수	비고
Anthropic	<code>anthropic:claude-sonnet-4-6</code>	<code>ANTHROPIC_API_KEY</code>	권장 기본 – 프롬프트 캐싱 · HITL 최적
OpenAI	<code>openai:gpt-5.4</code>	<code>OPENAI_API_KEY</code>	도구 호출 정확도 ↑
Google	<code>google_genai:gemini-3.5-flash</code>	<code>GOOGLE_API_KEY</code>	비용 효율
Azure	<code>azure_openai:gpt-5.4</code>	<code>AZURE_OPENAI_*</code>	기업용 배포
AWS Bedrock	<code>bedrock:anthropic.claude-sonnet-4-6</code>	AWS 자격 증명	VPC 격리

권장 기본 모델은 `anthropic:claude-sonnet-4-6` 이며, 자동 재시도(기본 6회)와 타임아웃 처리가 내장되어 있습니다. `model` 파라미터는 LangChain `BaseChatModel` 객체 또는 `"provider:model-name"` 형태의 문자열을 모두 받습니다. 문자열로 전달하면 Deep Agents가 내부적으로 적절한 ChatModel 인스턴스를 생성합니다.

ChatModel 객체를 직접 전달하면 `temperature`, `max_tokens` 등 세밀한 파라미터를 제어할 수 있으므로, 프로덕션 환경에서는 이 방식을 권장합니다.

! WARNING

모든 LLM이 도구 호출(function calling)을 동일한 수준으로 지원하는 것은 아닙니다. Deep Agents는 도구 호출에 크게 의존하므로, 도구 호출 성능이 검증된 모델(Claude Sonnet 4.5 이상, GPT-5 계열, Gemini 3.x 등)을 사용하는 것이 좋습니다. 도구 호출을 지원하지 않는 모델을 사용하면 빌트인 도구가 제대로 동작하지 않을 수 있습니다.

아래 코드에서 `create_deep_agent()` 에 `provider:model` 문자열을 전달하는 패턴을 확인하세요. 주석 처리된 부분은 다른 프로바이더를 사용하는 방법을 보여줍니다.

```
# 권장 기본 – Anthropic Claude Sonnet 4.6 (provider:model 포맷)
agent_default = create_deep_agent(
    model="anthropic:claude-sonnet-4-6",
)

print(f"에이전트 생성 완료: {type(agent_default).__name__}")

# 참고: 다른 프로바이더를 사용하려면 해당 API 키를 설정하고 아래처럼 호출
# agent_openai = create_deep_agent(model="openai:gpt-5.4")
# agent_gemini = create_deep_agent(model="google_genai:gemini-3.5-flash")
# agent_bedrock = create_deep_agent(model="bedrock:anthropic.claude-sonnet-4-6")
```

에이전트 생성 완료: CompiledStateGraph

2. 커스텀 시스템 프롬프트

모델을 선택했다면, 다음은 에이전트의 행동 방식을 정의하는 시스템 프롬프트입니다. 모델이 에이전트의 “두뇌”라면, 시스템 프롬프트는 “직무 기술서”에 해당합니다.

`system_prompt` 파라미터로 전달한 텍스트는 Deep Agents의 내부 기본 프롬프트 위에 추가됩니다. 기본 프롬프트에는 빌트인 도구 사용법, 태스크 관리 지침 등이 포함되어 있으므로, 개발자는 도메인 특화 지침(역할, 행동 규칙, 출력 형식 등)만 작성하면 됩니다. 기본 프롬프트를 덮어쓰는 것이 아니라 추가하는 것이므로, 빌트인 도구 사용법을 반복해서 작성할 필요가 없습니다.

TIP

시스템 프롬프트를 작성할 때는 역할 정의, 행동 규칙, 출력 형식 세 가지를 명확히 분리하면 에이전트의 일관성이 높아집니다. 예를 들어: “당신은 금융 분석가입니다(역할). 모든 수치는 출처를 밝히세요(규칙). 분석 결과는 표 형식으로 정리하세요(형식).”

WARNING

시스템 프롬프트가 너무 길면 오히려 에이전트의 성능이 저하될 수 있습니다. 핵심 지침을 500 토큰 이내로 유지하고, 상세한 지식은 `memory` (AGENTS.md)나 `skills` (SKILL.md)로 분리하는 것이 효과적입니다.

3. 커스텀 도구 만들기

모델과 시스템 프롬프트가 에이전트의 “사고 방식”을 결정한다면, 커스텀 도구는 에이전트가 실제로 “할 수 있는 일”을 확장합니다. 빌트인 파일 도구 외에 외부 API 호출, 계산, 데이터베이스 조회 등 어떤 Python 함수든 도구로 등록할 수 있습니다.

Python 함수를 도구로 변환하는 규칙:

1. 함수 이름 → 도구 이름
2. docstring → 도구 설명 (에이전트가 도구 선택 시 참조)
3. 타입 힌트 → 파라미터 스키마 (자동 생성)
4. 기본값 → 선택적 파라미터

```
import math

def calculate_compound_interest(
    principal: float,
    annual_rate: float,
    years: int,
    compounds_per_year: int = 12,
) -> dict:
    """복리 이자를 계산합니다.

    Args:
        principal: 원금 (원)
        annual_rate: 연이율 (예: 0.05 = 5%)
        years: 투자 기간 (년)
```

```

        compounds_per_year: 연간 복리 횟수 (기본: 12 = 월복리)
    """
    amount = principal * (1 + annual_rate / compounds_per_year) ** (compounds_per_year * years)
    interest = amount - principal
    return {
        "원금": f"{principal:,.0f}원",
        "최종 금액": f"{amount:,.0f}원",
        "이자 수익": f"{interest:,.0f}원",
        "수익률": f"{(interest / principal) * 100:.2f}%",
    }

def convert_temperature(
    value: float,
    from_unit: str,
    to_unit: str,
) -> str:
    """온도 단위를 변환합니다.

    Args:
        value: 변환할 온도 값
        from_unit: 원래 단위 ('celsius', 'fahrenheit', 'kelvin')
        to_unit: 변환할 단위 ('celsius', 'fahrenheit', 'kelvin')
    """
    # 먼저 섭씨로 변환
    if from_unit == "fahrenheit":
        celsius = (value - 32) * 5 / 9
    elif from_unit == "kelvin":
        celsius = value - 273.15
    else:
        celsius = value

    # 목표 단위로 변환
    if to_unit == "fahrenheit":
        result = celsius * 9 / 5 + 32
    elif to_unit == "kelvin":
        result = celsius + 273.15
    else:
        result = celsius

    return f"{value} {from_unit} = {result:.2f} {to_unit}"

# 커스텀 도구를 사용하는 에이전트 생성
calculator_agent = create_deep_agent(
    model=model,
    tools=[calculate_compound_interest, convert_temperature],
    system_prompt="당신은 계산과 단위 변환을 도와주는 어시스턴트입니다. 항상 도구를 사용하여 정확한 계산을 수행하세요.",
)

print("계산 에이전트가 생성되었습니다!")

```

계산 에이전트가 생성되었습니다!

4. 구조화된 출력 – response_format

커스텀 도구로 에이전트의 입력 능력을 확장했다면, 이제 출력 형식을 제어할 차례입니다. 에이전트의 응답을 후속 파이프라인(데이터베이스 저장, API 응답, UI 렌더링 등)에서 사용하려면 자유 형식 텍스트보다 정형화된 구조가 훨씬 유리합니다.

`response_format` 에 Pydantic `BaseModel` 을 전달하면, 에이전트의 최종 응답이 해당 스키마에 맞춰 JSON 구조로 반환됩니다. 결과는 `result["structured_response"]` 에서 접근할 수 있으며, 후속 파이프라인에서 파싱 없이 바

로 사용할 수 있습니다. Pydantic의 `Field(description=...)` 을 활용하면 각 필드의 의미를 LLM에 전달할 수 있어, 출력 품질이 크게 향상됩니다.

TIP

`response_format` 은 에이전트의 최종 응답에만 적용됩니다. 중간 도구 호출 결과에는 영향을 주지 않습니다. 에이전트는 여전히 자유롭게 도구를 호출하고 사고하며, 마지막 단계에서만 지정된 스키마에 맞춰 응답을 구조화합니다.

아래 예제에서 Pydantic 모델을 정의하는 방법과 `response_format` 에 전달하는 패턴을 확인하세요. `Field` 의 `description` 이 LLM에게 각 필드에 어떤 값을 채워야 하는지 알려주는 역할을 한다는 점에 주목하세요.

```
from pydantic import BaseModel, Field

# 구조화된 출력 스키마 정의
class BookRecommendation(BaseModel):
    """도서 추천 결과"""
    title: str = Field(description="책 제목")
    author: str = Field(description="저자")
    reason: str = Field(description="추천 이유 (2~3문장)")
    difficulty: str = Field(description="난이도: 초급/중급/고급")

class BookRecommendationList(BaseModel):
    """도서 추천 목록"""
    topic: str = Field(description="추천 주제")
    books: list[BookRecommendation] = Field(description="추천 도서 목록")

# response_format을 사용하는 에이전트
book_agent = create_deep_agent(
    model=model,
    system_prompt="당신은 도서 추천 전문가입니다. 사용자의 관심 분야에 맞는 책을 추천합니다.",
    response_format=BookRecommendationList,
)

print("도서 추천 에이전트 생성 완료")
```

도서 추천 에이전트 생성 완료

5. 미들웨어 아키텍처

지금까지 다룬 모델, 시스템 프롬프트, 도구, 출력 형식은 모두 `create_deep_agent()` 의 표면적 파라미터입니다. 하지만 실제로 에이전트의 동작을 결정하는 것은 그 뒤에서 작동하는 미들웨어 스택입니다.

미들웨어는 에이전트의 메시지 처리 파이프라인에 삽입되는 플러그인으로, 도구 추가, 시스템 프롬프트 수정, 컨텍스트 압축 등을 담당합니다. 웹 서버의 미들웨어(인증, 로깅, CORS 등)와 유사한 개념이라고 이해할 수 있습니다. 각 미들웨어는 에이전트의 메시지가 LLM에 전달되기 전과 후에 개입하여 메시지를 변환하거나, 새로운 도구를 주입하거나, 컨텍스트를 압축합니다. 대부분의 미들웨어는 `create_deep_agent()` 가 파라미터에 따라 자동으로 구성하므로, 개발자가 직접 다룰 필요는 거의 없습니다. 하지만 이 아키텍처를 이해하면 에이전트의 내부 동작을 정확히 파악하고, 필요할 때 커스텀 미들웨어를 작성할 수 있습니다.

기본 미들웨어 스택 (실행 순서)

아래는 `create_deep_agent()` 가 자동으로 조립하는 미들웨어 스택의 실행 순서입니다. 순서가 중요합니다. 예를 들어, `FilesystemMiddleware` 가 파일 도구를 주입한 뒤에 `SubAgentMiddleware` 가 서브에이전트 도구를 추가하므로, 서브에이전트는 파일 도구에 접근할 수 있습니다.

- | | |
|--|--|
| 1. <code>TodoListMiddleware</code> | - 태스크 관리 (<code>write_todos</code> 도구) |
| 2. <code>MemoryMiddleware</code> | - <code>AGENTS.md</code> 로딩 (<code>memory</code> 파라미터 사용 시) |
| 3. <code>SkillsMiddleware</code> | - <code>SKILL.md</code> 로딩 (<code>skills</code> 파라미터 사용 시) |
| 4. <code>FilesystemMiddleware</code> | - 파일 도구 (<code>ls</code> , <code>read</code> , <code>write</code> , <code>edit</code> , <code>glob</code> , <code>grep</code>) |
| 5. <code>SubAgentMiddleware</code> [필수, 제거 불가] | - 서브에이전트 (<code>task</code> 도구) |
| 6. <code>SummarizationMiddleware</code> | - 컨텍스트 압축 (~85% 자동 요약) |
| 7. <code>AnthropicPromptCachingMiddleware</code> | - 프롬프트 캐싱 (Anthropic 모델 자동 적용) |
| 8. <code>PatchToolCallsMiddleware</code> | - 잘못된/누락된 도구 호출 보정 |
| 9. [사용자 커스텀 미들웨어] | - <code>middleware</code> 파라미터 |
| 10. <code>HumanInTheLoopMiddleware</code> | - 승인 워크플로 (<code>interrupt_on</code> 사용 시) |

! WARNING

`SubAgentMiddleware` 는 `excluded_middleware` 로 제거할 수 없습니다. `task` 도구는 Deep Agents의 핵심 위임 메커니즘이라 항상 활성화되며, 서브에이전트를 정의하지 않더라도 빌트인 `general-purpose` 서브에이전트가 자동으로 사용됩니다.

각 미들웨어가 어떤 도구를 추가하고 어떤 역할을 하는지 아래 표에서 확인합니다. “(없음)”으로 표시된 미들웨어는 도구 대신 시스템 프롬프트 수정이나 메시지 변환을 수행합니다.

각 미들웨어의 역할

미들웨어	추가하는 도구	역할
<code>TodoListMiddleware</code>	<code>write_todos</code>	구조화된 태스크 목록 관리
<code>FilesystemMiddleware</code>	<code>ls</code> , <code>read_file</code> , <code>write_file</code> , <code>edit_file</code> , <code>glob</code> , <code>grep</code>	파일 시스템 접근
<code>SubAgentMiddleware</code>	<code>task</code>	서브에이전트 생성 및 호출
<code>SummarizationMiddleware</code>	(없음)	컨텍스트가 85%에 도달하면 자동 요약
<code>MemoryMiddleware</code>	(없음)	<code>AGENTS.md</code> 파일을 시스템 프롬프트에 주입
<code>SkillsMiddleware</code>	(없음)	관련 <code>SKILL.md</code> 를 점진적으로 로드

```
# 미들웨어 임포트 확인
from deepagents.middleware import (
    FilesystemMiddleware,
    MemoryMiddleware,
    SubAgentMiddleware,
    SkillsMiddleware,
    SummarizationMiddleware,
)

print("사용 가능한 미들웨어:")
```



```
for mw in [FilesystemMiddleware, MemoryMiddleware, SubAgentMiddleware, SkillsMiddleware,
SummarizationMiddleware]:
    print(f" - {mw.__name__}")
```

사용 가능한 미들웨어:

- FilesystemMiddleware
- MemoryMiddleware
- SubAgentMiddleware
- SkillsMiddleware
- _DeepAgentsSummarizationMiddleware

· NOTE

참고: 미들웨어는 `create_deep_agent()` 가 자동으로 구성하므로, 대부분의 경우 직접 다룰 필요가 없습니다. 커스텀 미들웨어는 `middleware` 파라미터로 추가할 수 있으며, 고급 사용자를 위한 기능입니다.

TIP

`SummarizationMiddleware` 의 클래스 이름이 `_DeepAgentsSummarizationMiddleware` (언더스코어 프리픽스)인 이유는 내부 구현 세부사항이기 때문입니다. 직접 인스턴스화할 필요 없이 `create_deep_agent()` 가 자동으로 관리합니다.

6. 샌드박스 옵션

`tools` 파라미터로 직접 함수를 노출하는 것 외에도, Deep Agents는 신뢰할 수 없는 코드 실행을 안전하게 격리하는 샌드박스 옵션을 제공합니다. 코딩 에이전트나 데이터 분석 에이전트가 임의의 스크립트를 실행해야 할 때 사용합니다.

샌드박스	실행 환경	적합 시나리오
Modal	원격 컨테이너 (GPU 지원)	장기 실행 코드, GPU 추론
Daytona	원격 개발 컨테이너	Git 워크스페이스 통합, 멀티 언어
Deno	로컬 TypeScript/JavaScript 격리	경량 JS/TS 코드 실행
로컬 VFS (Local Virtual FS)	In-process 가상 파일시스템	네트워크/디스크 없이 단위 테스트

TIP

샌드박스는 도구로 노출되거나(예: `execute_python` 도구) 백엔드로 노출됩니다(예: `SandboxBackend` 로 파일 격리). 두 경로를 조합해 “파일 격리 + 코드 격리”를 동시에 달성할 수 있습니다. 자세한 사용법은 Part 4의 샌드박스 챕터를 참고하세요.

핵심 정리

항목	방법
모델 선택	<code>model="anthropic:claude-sonnet-4-6"</code> 등 provider 접두 형식
시스템 프롬프트	USER → BASE/CUSTOM → SUFFIX 순으로 조립됨
커스텀 도구	함수 + docstring + 타입 힌트 → <code>tools=[func]</code>
구조화된 출력	<code>response_format=PydanticModel</code> → <code>result["structured_response"]</code>
미들웨어	<code>SubAgentMiddleware</code> 는 제거 불가. 나머지는 파라미터에 따라 자동 구성
전체 파라미터	17개 — <code>permissions</code> , <code>context_schema</code> , <code>checkpointer</code> , <code>store</code> , <code>cache</code> , <code>interrupt_on</code> 포함
샌드박스	Modal, Daytona, Deno, 로컬 VFS — 도구 또는 백엔드로 노출

에이전트의 다섯 가지 커스터마이징 축(모델, 프롬프트, 도구, 출력 형식, 미들웨어)을 모두 살펴보았습니다. 이 다섯 가지 축은 `create_deep_agent()` 의 파라미터에 1:1로 대응하므로, 필요한 파라미터만 조합하면 거의 모든 도메인의 전문가 에이전트를 설계할 수 있습니다. 다음 장에서는 에이전트의 파일 도구가 실제로 데이터를 읽고 쓰는 스토리지 백엔드 계층을 심화합니다. 백엔드는 에이전트의 “기억 장치”로서, 에이전트 코드를 변경하지 않고도 저장소 전략을 완전히 전환할 수 있는 강력한 추상화를 제공합니다.

04 스토리지 백엔드

에이전트가 파일을 읽고 쓰는 방식은 백엔드에 의해 결정된다. Deep Agents는 `StateBackend`, `FilesystemBackend`, `StoreBackend`, `CompositeBackend`, `LocalShellBackend` 등 다양한 플러그인 백엔드를 제공하여 에phemeral 스크래치패드부터 크로스 스레드 영속 저장소까지 유연하게 전환할 수 있다. 이 장에서는 각 백엔드의 특성과 선택 기준을 학습하고, `BackendProtocol` 을 구현하여 커스텀 백엔드를 만드는 방법까지 다룬다.

3장에서 미들웨어가 에이전트 동작을 확장하는 방식을 살펴보았다. 그 중 `FilesystemMiddleware` 가 제공하는 파일 도구(`ls`, `read_file`, `write_file`, `edit_file`, `glob`, `grep`)의 실제 저장소 계층이 바로 이 장에서 다루는 백엔드다. 백엔드를 교체하면 에이전트 코드는 한 줄도 바꾸지 않고 저장소 전략만 완전히 전환할 수 있다. 이 추상화는 소프트웨어 공학의 의존성 역전 원칙(DIP)을 적용한 것으로, 에이전트가 구체적인 저장소 구현이 아닌 `BackendProtocol` 인터페이스에 의존하도록 설계되어 있다. 덕분에 프로토타이핑 단계에서는 `StateBackend` 로 빠르게 시작하고, 프로덕션 단계에서 `FilesystemBackend` 나 `StoreBackend` 로 전환하는 점진적 마이그레이션이 가능하다.

학습 목표

LEARNING OBJECTIVES

- 백엔드가 에이전트의 파일 시스템을 어떻게 구현하는지 이해한다
- 5가지 내장 백엔드의 특성과 사용 시나리오를 파악한다
- `CompositeBackend` 로 경로별 백엔드 라우팅을 구성한다
- `BackendProtocol` 을 구현하여 커스텀 백엔드를 만든다

```
# 환경 설정
from dotenv import load_dotenv
import os

load_dotenv()
assert os.environ.get("ANTHROPIC_API_KEY"), "ANTHROPIC_API_KEY가 설정되지 않았습니다!"
print("환경 설정 완료")

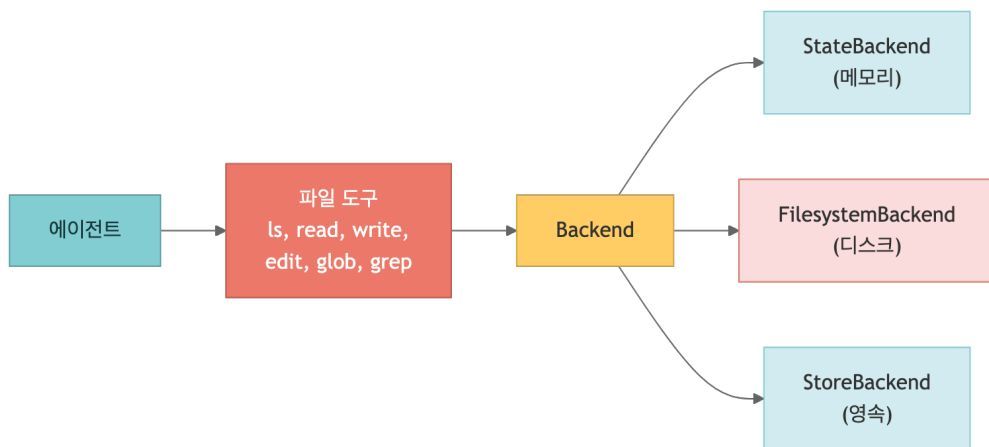
# 권장 기본 - Anthropic Claude Sonnet 4.6
model = "anthropic:claude-sonnet-4-6"
```

환경 설정 완료

1. 백엔드란?

Deep Agents의 빌트인 파일 도구(`ls` , `read_file` , `write_file` , `edit_file` , `glob` , `grep`)는 모두 백엔드 (Backend) 를 통해 동작합니다.

백엔드는 에이전트가 파일을 읽고 쓰는 스토리지 계층을 추상화합니다. 에이전트 입장에서는 “파일을 쓴다”는 동일한 명령을 내리지만, 그 뒤에서 실제로 메모리에 저장하는지, 디스크에 저장하는지, 원격 데이터베이스에 저장하는지는 백엔드 구현에 따라 달라집니다. 이 추상화 덕분에 에이전트의 시스템 프롬프트나 도구 호출 로직을 전혀 변경하지 않고도, `create_deep_agent()` 의 `backend` 파라미터 한 줄만 바꿔서 저장소 전략을 완전히 전환할 수 있습니다.



위 다이어그램에서 에이전트의 파일 도구와 백엔드 사이에 위치한 추상화 계층에 주목하세요. 에이전트는 항상 동일한 인터페이스(`read` , `write` , `ls` 등)를 호출하며, 실제 저장 방식은 백엔드가 결정합니다.

아래 표에서 각 백엔드의 저장 위치, 영속성, 사용 시나리오를 비교합니다. 자신의 프로젝트에 가장 적합한 백엔드를 선택하는 기준으로 활용하세요.

사용 가능한 백엔드

백엔드	저장 위치	영속성	사용 시나리오
StateBackend	에이전트 상태 (메모리)	스레드 내	임시 작업, 스크래치패드 (기본값)
FilesystemBackend	로컬 디스크	영구	로컬 파일 접근, 코딩 에이전트

StoreBackend	LangGraph Store	크로스 스레드	장기 메모리, 사용자 선호도
CompositeBackend	경로별 라우팅	혼합	메모리 + 임시 파일 병용
LocalShellBackend	디스크 + 셸	영구	개발 환경 (보안 주의)
ContextHubBackend	Context Hub (원격)	영구 (lazy fetch)	팀 공유 컨텍스트, 원격 파일 동기화

```
# 백엔드 임포트 확인
from deepagents.backends import (
    StateBackend,
    FilesystemBackend,
    StoreBackend,
    CompositeBackend,
)
from deepagents.backends.protocol import BackendProtocol

print("모든 백엔드 클래스를 성공적으로 임포트했습니다!")
```

모든 백엔드 클래스를 성공적으로 임포트했습니다!

2. StateBackend (기본값)

가장 단순한 백엔드부터 살펴봅니다. 각 백엔드를 하나씩 이해한 뒤, 후반부에서 이들을 조합하는 방법을 배울 것입니다.

`StateBackend` 는 에이전트 상태(LangGraph state)에 파일을 딕셔너리 형태로 저장합니다. 에페메럴(ephemeral) 특성을 가지므로, 대화 스레드 내에서만 파일이 유지됩니다. 내부적으로는 LangGraph 상태의 `files` 키에 {경로: 내용} 형태의 딕셔너리로 파일을 관리합니다. 이 딕셔너리는 각 체크포인트에 포함되므로, 에이전트의 턴 간에는 데이터가 유지되지만 프로세스가 종료되면 소멸합니다.

특징

- `create_deep_agent()` 에서 `backend` 를 지정하지 않으면 자동 사용 (기본값)
- 파일이 LangGraph 체크포인트에 포함되므로, 에이전트 턴 간에는 유지됨
- 프로세스가 종료되면 데이터가 소멸 - 영속성이 필요 없는 임시 작업에 적합
- 외부 스토리지 불필요 - 추가 설정 없이 바로 사용 가능

TIP

`StateBackend` 는 프로토타이핑과 단기 작업에 이상적입니다. 에이전트에게 “보고서를 작성해서 파일로 저장해”라고 요청하면, 결과가 상태에 저장되어 같은 대화 내에서 다시 읽을 수 있습니다.

! WARNING

`StateBackend`에 저장된 파일은 체크포인트에 포함되므로, 대량의 파일이나 큰 파일을 저장하면 체크포인트 크기가 급격히 증가합니다. 일반적으로 총 파일 크기가 1MB를 넘지 않도록 관리하는 것이 좋습니다. 대용량 데이터는 `FilesystemBackend`나 `StoreBackend`를 사용하세요.

3. FilesystemBackend – 로컬 디스크 접근

`StateBackend`의 에페메럴 특성이 부적합한 경우가 있습니다. 에이전트가 실제 로컬 파일 시스템에 접근해야 하는 시나리오가 대표적입니다. 코딩 에이전트가 프로젝트 소스 코드를 읽고 수정하거나, 데이터 분석 에이전트가 CSV 파일을 처리하는 경우에 `FilesystemBackend`를 사용합니다.

`FilesystemBackend`는 `DATA_DIR` 환경 변수 또는 `root_dir` 파라미터로 접근 가능한 루트 디렉토리를 설정할 수 있습니다. 이 루트 디렉토리 아래의 모든 파일과 디렉토리에 에이전트가 접근할 수 있으므로, 보안을 위해 접근 범위를 최소한으로 설정하는 것이 중요합니다.

주요 옵션

- `root_dir` – 접근 가능한 루트 디렉토리 (기본: 현재 디렉토리)
- `virtual_mode=True` – 경로 제한 활성화 (`..`, `~` 등 차단)
- `max_file_size_mb` – 읽을 수 있는 최대 파일 크기

`virtual_mode=True`가 차단하는 경로

`virtual_mode=True`는 다음 세 가지 패턴을 차단합니다:

- 상대 이스케이프 – `..` 세그먼트가 포함된 경로 (`../etc/passwd`, `foo/../../bar`)
- 홈 디렉토리 확장 – `~` 또는 `~user`로 시작하는 경로
- `root_dir` 밖의 절대 경로 – `/etc/hosts`처럼 루트를 벗어나는 절대 경로

이 세 경로가 입력되면 백엔드는 즉시 `error` 필드가 채워진 결과를 반환하고, 실제 디스크 호출은 발생하지 않습니다.

보안 주의사항

! WARNING

`FilesystemBackend`는 에이전트에게 실제 파일 시스템 접근 권한을 부여합니다. `virtual_mode=True`를 설정하면 위의 세 패턴(`..`, `~`, 외부 절대경로)을 차단하여 `root_dir` 범위 밖으로의 접근을 막습니다. 프로덕션 환경에서는 반드시 `virtual_mode=True`를 사용하거나, 10장에서 다루는 샌드박스 백엔드를 고려하세요.

4. StoreBackend – 크로스 스레드 영속 저장소

`StateBackend`는 스레드 종료 시 소멸하고, `FilesystemBackend`는 로컬 파일에 직접 접근합니다. 그 중간 지점이 바로 `StoreBackend`입니다.

`StoreBackend` 는 `LangGraph`의 `BaseStore` 인터페이스를 활용하여, 대화 스레드를 넘어서 파일을 영속적으로 저장합니다. 사용자 선호도, 학습된 패턴 등 장기 메모리에 적합합니다. 예를 들어, 사용자가 “나는 한국어로 응답해 줘”라고 한번 말하면, 그 선호도를 `StoreBackend` 에 저장하여 이후 모든 대화에서 자동으로 적용할 수 있습니다.

특징

- 다른 스레드에서도 같은 파일에 접근 가능 – 이전 대화에서 저장한 파일을 새 대화에서 읽을 수 있습니다
- Redis, PostgreSQL 등 다양한 스토어 구현 지원 – 프로덕션 환경에서는 `InMemoryStore` 대신 영속 스토어를 사용하세요
- LangSmith 배포 시 자동 프로비저닝
- `assistant_id` 기반 네임스페이스로 에이전트 간 격리 – 서로 다른 에이전트가 같은 스토어를 사용해도 데이터가 섞이지 않습니다

TIP

`StoreBackend` 는 팩토리 패턴으로 전달해야 합니다. 에이전트 실행 시 `runtime` 객체가 주입되므로, `backend=lambda runtime: StoreBackend(runtime)` 형태로 지연 생성합니다. 이 패턴은 아래 코드 예제에서 확인할 수 있습니다.

아래 코드는 `InMemoryStore` 를 사용하여 개발 환경에서 `StoreBackend` 를 시연합니다. 프로덕션에서는 `PostgresStore` 나 `RedisStore` 로 교체하여 영속성을 보장합니다. `store` 와 `checkpointer` 를 함께 전달하는 것이 `StoreBackend` 사용의 핵심 패턴입니다.

```
from langgraph.store.memory import InMemoryStore
from langgraph.checkpoint.memory import MemorySaver

# InMemoryStore - 개발용 (프로덕션에서는 PostgresStore 등 사용)
store = InMemoryStore()
checkpointer = MemorySaver()

# StoreBackend를 사용하는 에이전트
# StoreBackend는 BackendFactory 형태로 전달해야 합니다
store_agent = create_deep_agent(
    model=model,
    system_prompt="당신은 메모를 관리하는 어시스턴트입니다. 한국어로 응답하세요.",
    backend=lambda runtime: StoreBackend(runtime),
    store=store,
    checkpointer=checkpointer,
)

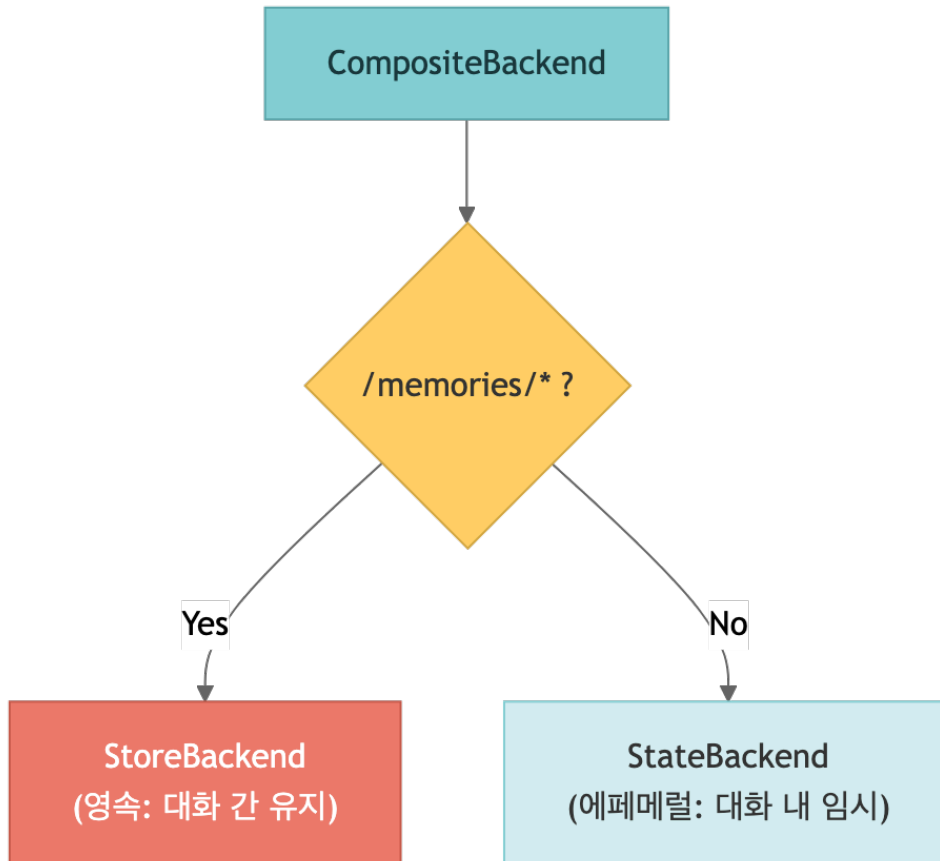
print("StoreBackend 에이전트가 생성되었습니다!")
```

StoreBackend 에이전트가 생성되었습니다!

5. CompositeBackend – 경로별 라우팅

지금까지 살펴본 `StateBackend`, `FilesystemBackend`, `StoreBackend` 는 각각 장단점이 있습니다. 실제 에이전트에서는 이들을 조합해야 하는 경우가 대부분입니다. “임시 작업 파일은 에페메럴로, 장기 메모리는 영속으로” 같은 혼합 전략이 필요하기 때문입니다.

`CompositeBackend` 는 경로 프리픽스에 따라 서로 다른 백엔드로 요청을 라우팅합니다. 가장 일반적인 패턴은 `/memories/` 는 **영속 저장**, 나머지는 **에페메럴**입니다. 이 패턴은 에이전트가 작업 중 생성하는 임시 파일(초안, 중간 결과 등)은 세션이 끝나면 자동 정리되고, 중요한 학습 결과나 사용자 선호도는 영구 보존되도록 합니다.



```

# CompositeBackend 팩토리 함수
def create_composite_backend(runtime):
    """경로 기반 라우팅 백엔드 생성"""
    return CompositeBackend(
        default=StateBackend(runtime),          # 기본: 에페메럴
        routes={
            "/memories/": StoreBackend(runtime), # /memories/* → 영속 저장
        },
    )

composite_store = InMemoryStore()
composite_checkpointer = MemorySaver()

composite_agent = create_deep_agent(
    model=model,
    system_prompt="""당신은 메모 관리 어시스턴트입니다.
- 영구 저장이 필요한 메모는 /memories/ 경로에 저장하세요.
- 임시 작업 파일은 루트(/) 경로에 저장하세요.
한국어로 응답하세요.""",
    backend=create_composite_backend,
    store=composite_store,
    checkpointer=composite_checkpointer,
)
  
```



```
print("CompositeBackend 에이전트가 생성되었습니다!")
```

```
CompositeBackend 에이전트가 생성되었습니다!
```

· NOTE

참고: CompositeBackend 는 라우트 프리픽스를 제거한 후 저장합니다. 예: /memories/preferences.txt → 내부적으로 /preferences.txt 로 저장 하지만 에이전트는 항상 전체 경로(/memories/preferences.txt)로 접근합니다.

6. LocalShellBackend – 셸 실행

파일 읽기/쓰기를 넘어, 에이전트가 시스템 명령을 실행해야 하는 경우가 있습니다. 코딩 에이전트가 테스트를 실행하거나, 패키지를 설치하거나, Git 커밋을 수행하는 시나리오가 대표적입니다.

LocalShellBackend 는 FilesystemBackend 의 모든 파일 도구에 셸 명령 실행 기능(`execute` 도구)을 추가합니다. 에이전트가 `pip install`, `pytest`, `git` 등의 명령을 직접 실행할 수 있어, 개발 환경에서 코딩 에이전트를 구축할 때 유용합니다. 단, 이는 호스트 시스템에 직접 영향을 줄 수 있으므로 보안에 각별히 주의해야 합니다.

! WARNING

LocalShellBackend 는 호스트 시스템에서 사용자 권한으로 명령이 직접 실행됩니다. 악의적이거나 잘못된 명령이 시스템에 영구적인 손상을 줄 수 있습니다. 개발 환경에서만 사용하고, 프로덕션에서는 반드시 10장에서 다루는 샌드박스 백엔드(Modal, Daytona, Runloop)를 사용하세요. `interrupt_on` 파라미터로 Human-in-the-Loop 승인을 추가하는 것을 강력히 권장합니다.

```
from deepagents.backends import LocalShellBackend

# △ 개발 환경에서만 사용하세요!
agent = create_deep_agent(
    model=model,
    backend=LocalShellBackend(root_dir="./workspace", virtual_mode=True),
    interrupt_on={"execute": True}, # 셸 명령은 승인 필요
)
```

· NOTE

이 노트북에서는 안전상의 이유로 LocalShellBackend 를 직접 실행하지 않습니다.

7. 커스텀 백엔드 구현

5가지 내장 백엔드를 살펴보았습니다. 하지만 실무에서는 S3 버킷, 데이터베이스, 원격 API 등 내장 백엔드로 커버할 수 없는 저장소를 사용해야 하는 경우가 있습니다.

내장 백엔드가 요구사항에 맞지 않는 경우, `BackendProtocol` 을 구현하여 나만의 백엔드를 만들 수 있습니다.

`BackendProtocol` 은 Python의 프로토콜(structural subtyping) 패턴을 사용하므로, 상속 없이 필요한 메서드만 구현하면 됩니다. 총 6개의 필수 메서드(`ls` , `read` , `write` , `edit` , `grep` , `glob`)를 정의하면 에이전트의 모든 파일 도구가 자동으로 커스텀 백엔드를 통해 동작합니다.

아래 예제는 읽기 전용 딕셔너리 기반 백엔드를 구현합니다. 실제 프로덕션에서는 이 패턴을 확장하여 S3, GCS, 데이터베이스 등 다양한 저장소를 파일시스템처럼 추상화할 수 있습니다.

! WARNING

현재 deepagents 버전($\geq 0.5.0$)부터 프로토콜 메서드명이 변경되었습니다. 구버전은 `ls_info` / `grep_raw` / `glob_info` 였지만, 현재는 `ls` / `grep` / `glob` 입니다. 반환 타입도 dict가 아닌 `LsResult` , `ReadResult` , `GrepResult` , `WriteResult` , `EditResult` , `list[FileInfo]` 등 명시적 dataclass/TypedDict 형식을 사용합니다.

필수 메서드와 반환 타입

메서드	반환 타입	설명
<code>ls(path)</code>	<code>LsResult</code> (<code>list[FileInfo]</code> 포함)	디렉토리 내용 목록
<code>read(file_path, offset, limit)</code>	<code>ReadResult</code>	파일 읽기 (줄 번호 포함)
<code>write(file_path, content)</code>	<code>WriteResult</code>	새 파일 생성
<code>edit(file_path, old_string, new_string)</code>	<code>EditResult</code>	텍스트 교체 (<code>occurrences</code> 포함)
<code>grep(pattern, path, glob)</code>	<code>GrepResult</code>	패턴 기반 파일 내용 검색
<code>glob(pattern, path)</code>	<code>list[FileInfo]</code>	글로브 패턴으로 파일 검색

```
# 간단한 커스텀 백엔드 예시: 읽기 전용 딕셔너리 기반
from deepagents.backends.protocol import (
    FileInfo, GrepResult, GrepMatch,
    LsResult, ReadResult, WriteResult, EditResult,
)

class ReadOnlyDictBackend:
    """딕셔너리에 파일을 저장하는 읽기 전용 백엔드 예시"""

    def __init__(self, files: dict[str, str]):
        self._files = files

    def ls(self, path: str = "/") -> LsResult:
        entries: list[FileInfo] = [
            {"path": p, "is_dir": False, "size": len(c), "modified_at": None}
            for p, c in self._files.items()
            if p.startswith(path)
        ]
```

```

    ]
    return LsResult(entries=entries, error=None)

def read(self, file_path: str, offset: int = 0, limit: int = 2000) -> ReadResult:
    content = self._files.get(file_path, "")
    lines = content.splitlines()
    selected = lines[offset:offset + limit]
    rendered = "\n".join(f"{i + offset + 1}\t{line}" for i, line in enumerate(selected))
    return ReadResult(content=rendered, error=None)

def write(self, file_path: str, content: str) -> WriteResult:
    return WriteResult(error="읽기 전용 백엔드입니다.", path=None, files_update=None)

def edit(self, file_path: str, old_string: str, new_string: str, replace_all: bool = False) ->
EditResult:
    return EditResult(error="읽기 전용 백엔드입니다.", path=None, files_update=None, occurrences=None)

def grep(self, pattern: str, path: str | None = None, glob: str | None = None) -> GrepResult:
    import re
    matches: list[GrepMatch] = []
    for fpath, content in self._files.items():
        for i, line in enumerate(content.splitlines(), 1):
            if re.search(pattern, line):
                matches.append({"path": fpath, "line": i, "text": line})
    return GrepResult(matches=matches, error=None)

def glob(self, pattern: str, path: str = "/") -> list[FileInfo]:
    import fnmatch
    return [
        {"path": p, "is_dir": False, "size": len(c), "modified_at": None}
        for p, c in self._files.items()
        if fnmatch.fnmatch(p, pattern)
    ]

# 사용 예시
custom_backend = ReadOnlyDictBackend({
    "/docs/guide.md": "# 가이드\n이것은 가이드 문서입니다.\n## 설치 방법\npip install deepagents",
    "/docs/faq.md": "# FAQ\nQ: 지원하는 모델은?\nA: Anthropic, OpenAI 등 다양한 모델을 지원합니다.",
})

# 커스텀 백엔드 동작 확인
print("파일 목록:", custom_backend.ls("/").entries)
print()
print("파일 내용:")
print(custom_backend.read("/docs/guide.md").content)
print()
print("검색 결과:", custom_backend.grep("설치").matches)

```

파일 목록: [{'path': '/docs/guide.md', 'is_dir': False, 'size': 52, 'modified_at': None}, {'path': '/docs/faq.md', 'is_dir': False, 'size': 56, 'modified_at': None}]

파일 내용:
1 # 가이드
2 이것은 가이드 문서입니다.
3 ## 설치 방법
4 pip install deepagents

검색 결과: [{'path': '/docs/guide.md', 'line': 3, 'text': '## 설치 방법'}]

아래 코드를 실행한 뒤, `ls` 가 파일 목록을 반환하고 `read` 가 줄 번호와 함께 내용을 출력하며 `grep` 이 패턴 검색 결과를 반환하는 것을 확인하세요. 이 세 가지가 동작하면 해당 백엔드는 `FilesystemMiddleware` 의 모든 도구와 호환됩니다.

TIP

커스텀 백엔드를 구현할 때 가장 중요한 것은 `read` 메서드의 반환 형식입니다. 줄 번호와 탭으로 구분된 "1\t내용" 형태를 반환해야 에이전트가 `edit_file` 도구로 정확한 위치를 지정할 수 있습니다.

8. ContextHubBackend – 원격 파일 동기화

`ContextHubBackend` 는 LangChain Context Hub에서 lazy fetch + write-through 방식으로 파일을 동기화합니다(`deepagents ≥ 0.5.2`). 팀이 공유하는 컨텍스트(공통 프롬프트, 회사 정책, 도메인 문서 등)를 모든 에이전트가 동일하게 참조해야 할 때 사용합니다.

동작 원리

- Lazy fetch – 에이전트가 `read_file` 을 호출하는 순간 원격에서 파일을 가져오고 로컬 캐시에 저장합니다. 사용하지 않는 파일은 다운로드하지 않습니다.
- Write-through – `write_file` / `edit_file` 이 호출되면 로컬 + 원격을 동시에 갱신합니다. 다음 에이전트 실행 시 다른 워커도 변경 사항을 즉시 볼 수 있습니다.

인스턴스화

`namespace` 는 런타임에 결정해야 하는 경우가 많기 때문에, 백엔드 자체를 미리 인스턴스화한 뒤 `namespace=lambda rt: ...` 형태의 콜러블을 함께 전달합니다.

```
# deepagents>=0.5.2
from deepagents.backends import ContextHubBackend

# 사전 인스턴스 + 런타임 네임스페이스 콜러블
context_hub = ContextHubBackend(
    project="my-team",
    namespace=lambda rt: f"users/{rt.context['user_id']}", # 런타임 시점에 결정
)

agent = create_deep_agent(
    model="anthropic:claude-sonnet-4-6",
    backend=context_hub,
)
```

TIP

`namespace` 콜러블 시그니처는 `(runtime) → str` 입니다. 단일 `namespace` 문자열로도 전달 가능하지만, 다중 테넌트 환경에서는 람다 형식이 필수입니다.

9. 권한 시스템 – `permissions` + `GuardedBackend`

`virtual_mode` 보다 세밀한 접근 제어가 필요할 때는 `permissions` 파라미터로 경로 단위 ACL을 적용합니다. 내부적으로 `GuardedBackend` 가 원본 백엔드를 감싸 모든 파일 호출을 정책 혹은으로 통과시킵니다.

```

from deepagents import create_deep_agent
from deepagents.permissions import Permissions

agent = create_deep_agent(
    model="anthropic:claude-sonnet-4-6",
    backend=FilesystemBackend(root_dir="./workspace", virtual_mode=True),
    permissions=Permissions(
        allow_read=["/docs/**", "/data/*.csv"],      # 읽기 허용
        allow_write=["/outputs/**"],                 # 쓰기 허용
        deny_read=["/data/secrets/**"],              # 읽기 차단 (allow보다 우선)
        deny_write=["**/*.lock"],                    # 쓰기 차단
    ),
)

```

정책 혹은 (policy)

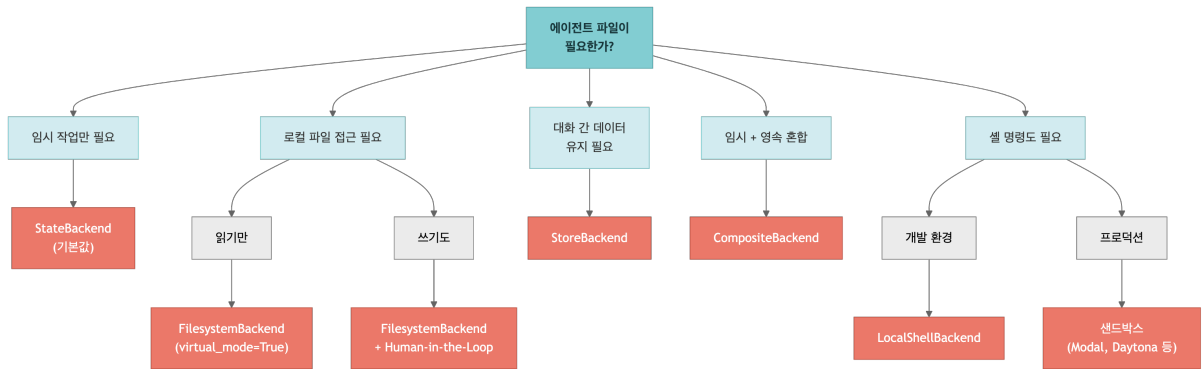
복잡한 규칙은 `policy=lambda call: ...` 콜러블로 표현합니다. 호출 객체에는 `tool_name`, `path`, `args` 등이 포함되어 있어, 시간대·사용자 권한·외부 시스템 응답을 모두 반영할 수 있습니다.

TIP

`allow_*` 규칙과 `deny_*` 규칙이 모두 매치되면 `deny` 가 우선합니다. 가장 안전한 패턴은 `allow_read=["/docs/**"]` 처럼 명시적 화이트리스트부터 시작하는 것입니다.

백엔드 선택 가이드

어떤 백엔드를 선택해야 할지 결정할 때 아래 의사결정 트리를 참고하세요. 핵심 질문은 “데이터가 대화 세션을 넘어 유지되어야 하는가?”와 “에이전트가 로컬 파일 시스템에 접근해야 하는가?”입니다.



핵심 정리

백엔드	특징	파라미터
StateBackend	에페메럴, 기본값	backend 생략 시 자동
FilesystemBackend	로컬 디스크	root_dir , virtual_mode
StoreBackend	크로스 스레드 영속	store + checkpointer 필요
CompositeBackend	경로별 라우팅	default + routes
LocalShellBackend	디스크 + 셀 실행	root_dir (보안 주의)

<code>ContextHubBackend</code>	원격 lazy fetch + write-through	사전 인스턴스 + <code>namespace=lambda rt: ...</code>
<code>GuardedBackend</code> (permissions)	경로 단위 ACL 래퍼	<code>allow_* / deny_* / policy</code>

백엔드는 에이전트의 “기억 장치”입니다. `BackendProtocol` 이라는 통일된 인터페이스 덕분에, 에이전트 코드를 변경하지 않고 `create_deep_agent()` 의 `backend` 파라미터 한 줄만 수정하여 저장소 전략을 완전히 전환할 수 있습니다. 프로토타이핑 단계에서는 `StateBackend` 로 빠르게 시작하고, 프로덕션에서는 `CompositeBackend` 로 에페메럴과 영속 저장소를 조합하는 것이 가장 일반적인 패턴입니다. 다음 장에서는 에이전트의 컨텍스트 블로트 문제를 해결하는 서브에이전트 패턴을 다룹니다.

05 서브에이전트와 태스크 위임

에이전트가 도구를 반복 호출하면 중간 결과가 컨텍스트 윈도우를 빠르게 채우는 컨텍스트 블로트 문제가 발생한다. 서브에이전트는 이 문제를 근본적으로 해결하는 구조로, 전문 작업을 격리된 컨텍스트에서 수행한 뒤 압축된 결과만 메인 에이전트에 반환한다. 이 장에서는 `SubAgent` dict 정의, `CompiledSubAgent` 연결, 빌트인 `general-purpose` 서브에이전트 오버라이드, 그리고 멀티 서브에이전트 파이프라인 패턴을 실습한다.

4장에서 백엔드가 에이전트의 데이터 저장을 추상화하는 것을 보았다. 이 장에서 다루는 서브에이전트는 에이전트의 작업 실행을 추상화한다. 메인 에이전트가 모든 작업을 직접 수행하는 대신, 전문 서브에이전트에게 위임하여 컨텍스트를 깔끔하게 유지하는 것이 핵심이다. Deep Agents에서 서브에이전트를 정의하는 방법은 두 가지다: `subagents` 파라미터로 명시적으로 정의하거나, 에이전트가 런타임에 `create_subagent` 도구를 호출하여 동적으로 생성할 수 있다.

명시적 정의(`subagents` 파라미터)는 빌드 타임에 어떤 서브에이전트가 존재하는지 확인할 수 있어 테스트와 디버깅이 용이하다. 반면 동적 생성(`create_subagent` 도구)은 실행 중 사용자의 요구에 맞춰 새로운 전문 에이전트를 즉석에서 만들 수 있어 유연성이 높다. 각 서브에이전트는 독립된 컨텍스트 윈도우를 갖고, 메인 에이전트는 `task` 도구를 통해 서브에이전트에 작업을 위임한다. 서브에이전트가 작업을 완료하면 결과만 메인 에이전트에 돌려주므로, 중간 과정에서 발생한 수천 토큰의 정보가 메인 컨텍스트에 쌓이지 않는다.

학습 목표

LEARNING OBJECTIVES

- 서브에이전트가 해결하는 문제(컨텍스트 블로트)를 이해한다
- `SubAgent` dict와 `CompiledSubAgent` 로 서브에이전트를 정의한다
- 빌트인 `general-purpose` 서브에이전트를 이해하고 오버라이드한다
- 컨텍스트 전파와 네임스페이스 키를 활용한다
- 멀티 서브에이전트 파이프라인 패턴을 구현한다

핵심 질문

이 장의 질문은 단순합니다. “메인 에이전트가 모든 중간 결과를 다 들고 있을 필요가 있는가, 아니면 세부 작업을 분리해 결과만 받는 편이 나은가?”

한눈에 보는 핵심

- 서브에이전트의 가장 큰 목적은 컨텍스트 불로트를 줄이는 것입니다.
- 전문 역할별 시스템 프롬프트와 도구를 분리하면 선택과 위임이 쉬워집니다.
- 단순 단발 작업에는 서브에이전트가 오히려 과할 수 있습니다.

```
# 환경 설정
from dotenv import load_dotenv
import os

load_dotenv()
assert os.environ.get("OPENAI_API_KEY"), "OPENAI_API_KEY가 설정되지 않았습니다!"
assert os.environ.get("TAVILY_API_KEY"), "TAVILY_API_KEY가 설정되지 않았습니다!"
print("환경 설정 완료")
```

환경 설정 완료

```
# 권장 기본 - Anthropic Claude Sonnet 4.6
model = "anthropic:claude-sonnet-4-6"

print(f"모델 설정 완료: {model}")
```

모델 설정 완료: anthropic:claude-sonnet-4-6

환경 설정이 완료되었으므로, 서브에이전트의 필요성과 동작 원리를 단계별로 살펴보자. 먼저 서브에이전트가 등장하게 된 근본 원인인 컨텍스트 불로트 문제부터 이해한다.

1. 서브에이전트가 필요한 이유

컨텍스트 불로트(Context Bloat) 문제

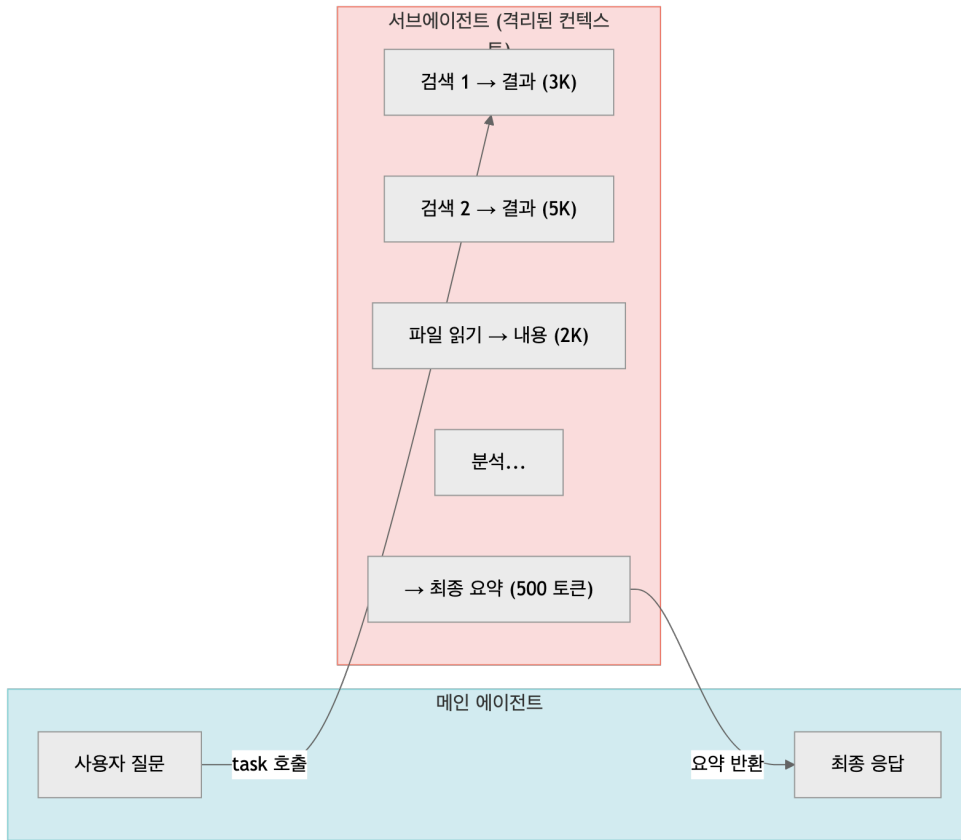
LLM 기반 에이전트는 대화 이력과 도구 호출 결과를 하나의 컨텍스트 윈도우 안에서 관리한다. 도구를 반복 호출할수록 이 윈도우가 빠르게 채워지며, 이를 컨텍스트 불로트라 부른다. 컨텍스트가 포화되면 LLM은 앞부분의 중요한 지시를 “잊거나” 환각(hallucination) 확률이 높아진다.

에이전트가 도구를 사용할 때마다 입력/출력이 컨텍스트 윈도우에 쌓입니다:

- 웹 검색 결과 (수천 토큰)
- 파일 내용 읽기 (수백 수천 줄)
- 데이터베이스 쿼리 결과

이 중간 결과들이 메인 에이전트의 컨텍스트를 가득 채우면, 정작 중요한 정보를 놓칠 수 있습니다. 예를 들어 웹 검색 3회 + 파일 읽기 2회만 해도 쉽게 10,000 토큰 이상이 소비되어, 시스템 프롬프트나 초기 지시를 밀어내는 현상이 발생합니다.

서브에이전트의 해결 방식



메인 에이전트는 500 토큰짜리 요약만 받으므로 컨텍스트가 깔끔하게 유지됩니다. 서브에이전트 내부에서 수천 토큰의 검색 결과, 파일 내용, 분석 결과가 오갔더라도, 메인 에이전트의 컨텍스트에는 압축된 최종 결과만 추가됩니다. 이것이 서브에이전트의 핵심 가치인 컨텍스트 격리입니다.

SubAgentMiddleware 자동 부착

서브에이전트 기능은 `SubAgentMiddleware` 가 담당합니다. `create_deep_agent()` 는 사용자가 `subagents` 를 지정했는지 여부와 무관하게 이 미들웨어를 항상 미들웨어 스택에 자동 부착하며, `excluded_middleware` 로 제거할 수 없습니다. 서브에이전트를 명시적으로 정의하지 않더라도 빌트인 `general-purpose` 서브에이전트가 등록되어 `task` 도구가 항상 사용 가능합니다.

TodoList + `task` 디스패치 패턴

전형적인 사용 흐름은 다음과 같습니다.

1. 메인 에이전트가 `write_todos` 로 작업을 구조화된 리스트로 분해합니다 — [조사, 분석, 보고서]
2. 각 TODO 항목을 메인 에이전트가 `task` 도구로 적절한 서브에이전트에게 위임합니다
3. 서브에이전트가 독립된 컨텍스트에서 작업을 수행하고 압축된 결과만 반환합니다
4. 메인 에이전트가 결과를 받아 다음 TODO로 이동하거나 사용자에게 최종 응답을 작성합니다

이 `write_todos` → `task` → 압축된 결과 패턴이 Deep Agents의 핵심 위임 사이클입니다. 메인 에이전트의 컨텍스트에는 TODO 리스트와 압축 결과만 누적되므로 토큰 효율이 매우 높습니다.

서브에이전트 사용 기준

상황	서브에이전트 사용
다단계 작업으로 중간 결과가 많음	✅ 사용
전문 지식/도구가 필요한 도메인	✅ 사용
다른 모델이 더 적합한 작업	✅ 사용
단순하고 한 번에 끝나는 작업	❌ 불필요
중간 결과가 메인 에이전트에 필요	❌ 불필요

· NOTE

언제 쓰고, 언제 쓰지 않을까?

- **써야 하는 경우**: 다단계 검색, 대용량 도구 출력, 전문 역할 분리, 메인 에이전트의 컨텍스트 보호가 중요할 때
- **쓰지 않는 경우**: 단순 단일 단계 작업, 중간 추론을 메인 컨텍스트가 계속 참조해야 하는 경우, 위임 오버헤드가 이득보다 큰 경우
- 핵심 판단 기준은 “결과만 돌려받아도 되는가, 아니면 중간 과정까지 메인 에이전트가 계속 봐야 하는가?”입니다.

서브에이전트의 필요성을 이해했으므로, 이제 실제로 서브에이전트를 정의하는 방법을 알아보자. 가장 간단한 방법은 Python 딕셔너리(`dict`)로 선언하는 것이다.

2. SubAgent 정의하기 (dict 기반)

`SubAgent` 는 딕셔너리 형태로 정의합니다. `create_deep_agent()` 의 `subagents` 파라미터에 이 딕셔너리 리스트를 전달하면, 프레임워크가 내부적으로 각 서브에이전트를 독립 `LangGraph` 노드로 컴파일합니다. 메인 에이전트는 `task` 빌트인 도구를 통해 서브에이전트를 호출하며, 이때 `description` 필드를 참고하여 어떤 서브에이전트를 선택할지 결정합니다.

필수 필드

필드	타입	설명
<code>name</code>	<code>str</code>	고유 식별자
<code>description</code>	<code>str</code>	역할 설명 (메인 에이전트가 호출 판단에 사용)
<code>system_prompt</code>	<code>str</code>	서브에이전트 지침

tools	list	사용할 도구 목록
-------	------	-----------

선택 필드

필드	타입	설명
model	str	모델 오버라이드 ("provider:model")
middleware	list	추가 미들웨어
interrupt_on	dict	Human-in-the-Loop 설정
skills	list[str]	스킬 소스 경로
response_format	BaseModel	구조화 출력 스키마 (deepagents $\geq 0.5.3$)
permissions	Permissions	경로 단위 ACL – 서브에이전트별 파일 접근 제어

TIP

`description` 은 메인 에이전트가 서브에이전트를 선택하는 유일한 기준입니다. “리서치 에이전트”처럼 모호하게 쓰지 말고, “인터넷 검색을 통해 최신 정보를 수집하고 핵심을 요약합니다. 사실 확인이나 트렌드 조사가 필요할 때 사용하세요”처럼 구체적인 역할과 사용 시기를 명시하세요.

다음 코드에서는 Tavily 웹 검색 도구를 갖춘 리서치 서브에이전트를 정의하고, 메인 에이전트에 연결하는 전체 흐름을 보여준다.

```
from typing import Literal
from tavily import TavilyClient
from deepagents import create_deep_agent

tavily_client = TavilyClient(api_key=os.environ["TAVILY_API_KEY"])

def internet_search(
    query: str,
    max_results: int = 5,
    topic: Literal["general", "news", "finance"] = "general",
    include_raw_content: bool = False,
) -> dict:
    """인터넷에서 정보를 검색합니다.

    Args:
        query: 검색할 질문 또는 키워드
        max_results: 반환할 최대 결과 수
        topic: 검색 주제 카테고리
        include_raw_content: 원본 콘텐츠 포함 여부
    """
    return tavily_client.search(
        query,
        max_results=max_results,
        include_raw_content=include_raw_content,
        topic=topic,
    )

# 리서치 서브에이전트 정의
research_subagent = {
```

```

    "name": "researcher",
    "description": "인터넷에서 주제를 심층 조사하고 핵심 정보를 요약합니다. 리서치가 필요한 질문에 사용하세요.",
    "system_prompt": """"당신은 전문 리서처입니다.
인터넷 검색을 통해 정확한 정보를 수집하고, 핵심만 추출하여 간결하게 요약합니다.
결과는 항상 한국어로 작성하며, 출처를 함께 표기합니다.
최종 결과는 500단어 이내로 요약하세요.""",
    "tools": [internet_search],
}

print(f"서브에이전트 정의 완료: {research_subagent['name']}")
print(f"설명: {research_subagent['description'][:50]}...")

```

```

서브에이전트 정의 완료: researcher
설명: 인터넷에서 주제를 심층 조사하고 핵심 정보를 요약합니다. 리서치가 필요한 질문에 사용하세요...

```

위에서 정의한 `research_subagent` 는 아직 독립적인 딕셔너리일 뿐이다. 이것을 실제로 동작하게 하려면 `create_deep_agent()` 의 `subagents` 파라미터에 전달해야 한다. 메인 에이전트가 생성되면 프레임워크가 자동으로 `task` 도구를 등록하고, 메인 에이전트는 이 도구를 통해 서브에이전트에 작업을 위임할 수 있다.

```

# 서브에이전트를 포함하는 메인 에이전트 생성
main_agent = create_deep_agent(
    model=model,
    system_prompt=""""당신은 프로젝트 매니저입니다.
사용자의 요청을 분석하고, 필요하면 전문 서브에이전트에게 작업을 위임합니다.
서브에이전트의 결과를 종합하여 최종 답변을 작성합니다.
한국어로 응답하세요.""",
    subagents=[research_subagent],
)

print("메인 에이전트 생성 완료 (서브에이전트: researcher)")

```

```

메인 에이전트 생성 완료 (서브에이전트: researcher)

```

! WARNING

`subagents` 파라미터 외에도, 에이전트가 런타임에 `create_subagent` 도구를 호출하여 동적으로 서브에이전트를 생성할 수 있다. 이 방식은 유연하지만, 어떤 서브에이전트가 만들어질지 사전에 예측할 수 없으므로 디버깅과 테스트가 어렵다. 프로덕션 환경에서는 명시적 `subagents` 정의를 우선 사용하고, 동적 생성은 탐색적 (exploratory) 작업에 한정하는 것을 권장한다.

`dict` 기반 `SubAgent` 는 선언이 간편하지만, 조건 분기나 반복 같은 복잡한 로직에는 한계가 있다. 이럴 때는 `CompiledSubAgent` 를 사용하여 이미 컴파일된 `LangGraph` 그래프를 서브에이전트로 연결할 수 있다.

3. CompiledSubAgent – 커스텀 LangGraph 그래프 연결

`SubAgent` `dict`는 단순한 설정으로 서브에이전트를 정의하기에 편리하지만, 조건 분기, 반복, 멀티 노드 워크플로 같은 복잡한 로직이 필요한 경우에는 한계가 있습니다. `CompiledSubAgent` 를 사용하면 미리 컴파일된 `LangGraph` 그래프(또는 다른 `create_deep_agent()` 결과)를 서브에이전트로 연결할 수 있습니다. 핵심 차이는 `tools` 대신 `runnable` 필드에 컴파일된 그래프 객체를 전달한다는 점이다.

이 패턴은 다음과 같은 상황에서 유용하다:

- 서브에이전트 내부에 조건부 분기(예: 데이터 유효성 검사 후 재시도)가 필요한 경우
- 여러 노드가 순차적으로 연결된 파이프라인을 서브에이전트 하나로 캡슐화하고 싶은 경우
- 이미 구축된 LangGraph 워크플로를 재사용하고 싶은 경우

```
from deepagents import CompiledSubAgent

# 별도의 에이전트를 CompiledSubAgent로 래핑하는 예시
# 실제로는 create_deep_agent()로 만든 그래프도 사용 가능
custom_graph = create_deep_agent(
    model=model,
    tools=[internet_search],
    system_prompt="당신은 데이터 분석 전문가입니다. 데이터를 수집하고 통계적으로 분석하여 인사이트를 도출합니다.",
)

# CompiledSubAgent로 래핑
data_analyst_subagent: CompiledSubAgent = {
    "name": "data-analyst",
    "description": "데이터를 수집하고 분석하여 통계적 인사이트를 제공합니다.",
    "runnable": custom_graph,
}

print(f"CompiledSubAgent 정의 완료: {data_analyst_subagent['name']}")
```

CompiledSubAgent 정의 완료: data-analyst

CompiledSubAgent에서는 `system_prompt`와 `tools`를 별도로 지정하지 않는다. 이 정보들은 이미 `runnable` 그래프 내부에 포함되어 있기 때문이다. 필요한 것은 `name`, `description`, 그리고 `runnable` 세 가지뿐이다.

다음으로, 명시적으로 서브에이전트를 정의하지 않아도 항상 사용 가능한 빌트인 general-purpose 서브에이전트에 대해 알아보자.

4. General-Purpose 서브에이전트

Deep Agents는 별도로 정의하지 않아도 빌트인 general-purpose 서브에이전트를 자동 제공합니다.

기본 동작

- 메인 에이전트와 같은 시스템 프롬프트 사용
- 메인 에이전트와 같은 도구 접근 가능
- 메인 에이전트와 같은 모델 사용 (권장 기본 `anthropic:claude-sonnet-4-6`)
- 메인 에이전트의 스킬 상속

오버라이드

`name="general-purpose"`로 서브에이전트를 정의하면 기본 동작을 덮어씁니다.

```
# general-purpose 서브에이전트 오버라이드 예시
custom_gp_agent = create_deep_agent(
    model=model,
    tools=[internet_search],
    system_prompt="당신은 멀티태스킹 코디네이터입니다.",
    subagents=[
        research_subagent,
    ]
    # 이름을 "general-purpose"로 설정하면 빌트인을 오버라이드
```

```

        "name": "general-purpose",
        "description": "범용 에이전트로, 리서치 외의 일반적인 멀티스텝 작업을 처리합니다.",
        "system_prompt": "당신은 범용 어시스턴트입니다. 주어진 작업을 단계별로 처리하세요.",
        "tools": [internet_search],
    },
    1,
)

print("general-purpose 서브에이전트를 오버라이드한 에이전트 생성 완료")

```

general-purpose 서브에이전트를 오버라이드한 에이전트 생성 완료

위 예시에서 `name` 을 "general-purpose" 로 설정하면 빌트인을 완전히 대체한다. 오버라이드하지 않으면 메인 에이전트의 설정(시스템 프롬프트, 도구, 모델)을 그대로 복사한 기본 서브에이전트가 자동으로 사용된다. 커스텀 서브에이전트만으로 커버되지 않는 "기타 멀티스텝 작업"을 general-purpose가 처리하므로, 오버라이드 시에는 범용적인 도구 세트를 할당하는 것이 좋다.

서브에이전트가 메인 에이전트와 격리된 컨텍스트를 갖는다고 했지만, 모든 정보가 차단되는 것은 아니다. 다음 섹션에서는 런타임 컨텍스트를 서브에이전트에 선택적으로 전달하는 메커니즘을 살펴본다.

5. 컨텍스트 전파

서브에이전트가 메인 에이전트와 격리된 컨텍스트를 가진다 해도, 일부 정보(사용자 ID, 세션 설정 등)는 모든 에이전트가 공유해야 합니다. Deep Agents는 런타임 컨텍스트를 자동으로 모든 서브에이전트에 전파합니다.

`context_schema` 로 구조를 정의하고, `config` 의 `context` 키로 값을 전달합니다.

네임스페이스 키로 서브에이전트별 컨텍스트 전달

"서브에이전트이름:키" 형식을 사용하면, 특정 서브에이전트에만 전달되는 설정을 추가할 수 있습니다.

```

config = {
    "context": {
        "user_id": "user-123",           # 모든 에이전트에 전파
        "researcher:max_depth": 3,      # researcher에만 전달
        "data-analyst:strict_mode": True, # data-analyst에만 전달
    }
}

```

위 코드에서 `"user_id"` 는 콜론(:)이 없으므로 모든 에이전트(메인 + 모든 서브에이전트)에 전파된다. 반면 `"researcher:max_depth"` 는 `researcher` 서브에이전트에만, `"data-analyst:strict_mode"` 는 `data-analyst` 서브에이전트에만 전달된다. 이 네임스페이스 키 규칙을 활용하면, 보안에 민감한 설정(API 키, 접근 권한 등)을 특정 서브에이전트에만 제한적으로 전달할 수 있다.

TIP

`context_schema` 를 사전에 정의해 두면, 잘못된 키 이름이나 타입의 컨텍스트가 전달될 때 빌드 타임에 오류를 잡을 수 있다. 스키마 없이도 동작하지만, 프로덕션 환경에서는 스키마 정의를 강력히 권장한다.

스트리밍에서 서브에이전트 식별 — `lc_agent_name`

`agent.stream(..., stream_mode="messages")` 또는 `astream_events()` 로 토큰을 스트리밍할 때, 각 이벤트의 메타데이터에는 `lc_agent_name` 키가 포함됩니다. 이 값으로 어떤 서브에이전트가 출력 중인지 구분할 수 있어 UI에 “리서치가 검색 중...”, “분석가가 정리 중...” 같은 라벨을 표시할 때 활용합니다.

```
# 스트리밍 중 어느 서브에이전트인지 식별
async for event in main_agent.astream_events(
    {"messages": [{"role": "user", "content": "..."}]},
    version="v2",
):
    name = event.get("metadata", {}).get("lc_agent_name")
    if event["event"] == "on_chat_model_stream" and name:
        print(f"[{name}] {event['data']['chunk'].content}", end="")
```

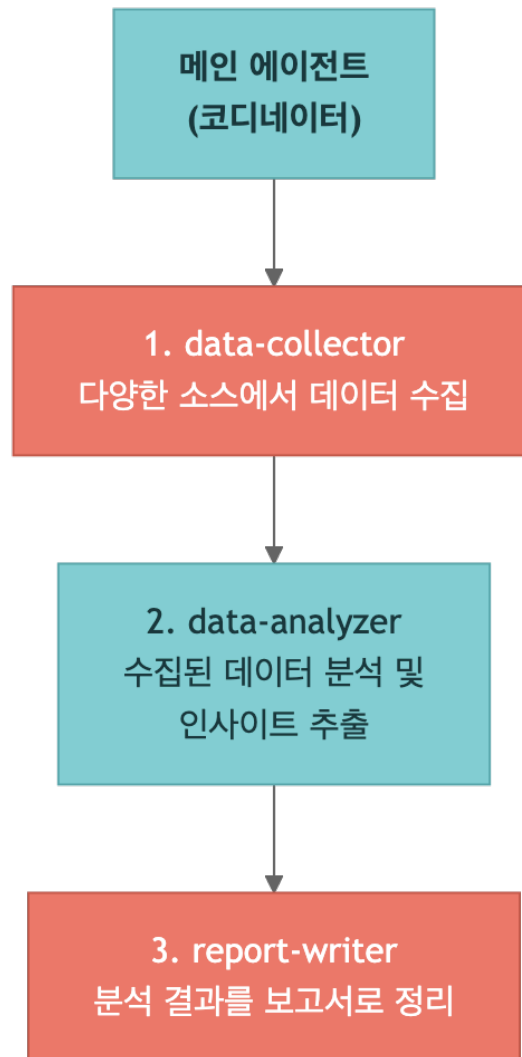
지금까지 개별 서브에이전트의 정의와 컨텍스트 전파를 다루었다. 다음은 여러 서브에이전트를 순차적으로 연결하여 복잡한 작업 흐름을 구성하는 파이프라인 패턴을 살펴본다.

6. 멀티 서브에이전트 파이프라인

서브에이전트의 진정한 힘은 여러 개를 조합할 때 발휘됩니다. 메인 에이전트가 코디네이터 역할을 하면서 전문 서브에이전트들을 순서대로 호출하는 파이프라인 패턴을 구성할 수 있습니다. 예를 들어 수집 -> 분석 -> 작성 파이프라인에서 각 단계를 전문 서브에이전트가 담당합니다.

TIP

파이프라인 패턴에서 메인 에이전트의 시스템 프롬프트에 호출 순서를 명시하면, 에이전트가 더 일관되게 파이프라인을 따릅니다. 단, LLM은 지시를 항상 완벽히 따르지 않으므로 결과를 검증하는 로직도 고려하세요.



```

# 멀티 서브에이전트 파이프라인
pipeline_agent = create_deep_agent(
    model=model,
    system_prompt="""당신은 프로젝트 코디네이터입니다.
사용자의 요청을 분석하고, 적절한 서브에이전트에게 순서대로 작업을 위임합니다:
1. 먼저 data-collector로 정보를 수집합니다.
2. 수집된 정보를 data-analyzer에게 전달하여 분석합니다.
3. 분석 결과를 report-writer에게 전달하여 보고서를 작성합니다.
최종 보고서를 사용자에게 전달합니다. 한국어로 응답하세요.""",
    subagents=[
        {
            "name": "data-collector",
            "description": "다양한 소스에서 원시 데이터와 정보를 수집합니다.",
            "system_prompt": """당신은 데이터 수집 전문가입니다.
주어진 주제에 대해 인터넷 검색을 수행하고, 관련 데이터를 최대한 수집합니다.
수집한 데이터를 구조화하여 반환하세요.""",
            "tools": [internet_search],
        },
        {
            "name": "data-analyzer",
            "description": "수집된 데이터를 분석하여 핵심 인사이트를 추출합니다.",
            "system_prompt": """당신은 데이터 분석 전문가입니다.
제공된 데이터에서 패턴, 트렌드, 핵심 인사이트를 추출합니다.
분석 결과를 불릿 포인트로 정리하세요.""",
            "tools": [],
        }
    ]
  )

```

```

    },
    {
      "name": "report-writer",
      "description": "분석 결과를 바탕으로 전문적인 보고서를 작성합니다.",
      "system_prompt": "\"\"\"당신은 테크니컬 라이터입니다.  
분석 결과를 바탕으로 명확하고 읽기 쉬운 보고서를 작성합니다.  
보고서는 다음 구조를 따릅니다: 개요 → 핵심 발견 → 결론\"\"\"",
      "tools": [],
    },
  ],
)

print("멀티 서브에이전트 파이프라인 에이전트 생성 완료")

```

멀티 서브에이전트 파이프라인 에이전트 생성 완료

위 파이프라인에서 각 서브에이전트는 독립된 컨텍스트에서 작업하므로, data-collector가 수집한 방대한 원시 데이터가 report-writer의 컨텍스트를 오염시키지 않는다. 메인 에이전트(코디네이터)는 각 단계의 압축된 결과만 받아 다음 서브에이전트에 전달하는 중개자 역할을 한다. 이 구조 덕분에 개별 서브에이전트는 자신의 전문 영역에만 집중할 수 있고, 전체 파이프라인의 토큰 효율도 크게 향상된다.

! WARNING

파이프라인의 서브에이전트 수가 많아지면 메인 에이전트가 호출 순서를 혼동할 수 있다. 서브에이전트는 3 5개 이내로 유지하고, 시스템 프롬프트에 명확한 호출 순서를 기술하라. 더 복잡한 워크플로가 필요하면

CompiledSubAgent 로 LangGraph 그래프를 직접 설계하는 것이 낫다.

파이프라인 패턴까지 익혔다면, 마지막으로 서브에이전트를 효과적으로 설계하기 위한 실무 가이드라인을 정리한다.

7. 베스트 프랙티스

1. 명확한 description 작성

메인 에이전트는 `description` 을 보고 어떤 서브에이전트를 호출할지 결정합니다. -> 서브에이전트의 역할과 사용 시기를 명확하게 기술하세요.

2. 최소 도구 원칙

서브에이전트에는 필요한 도구만 제공하세요. 불필요한 도구는 컨텍스트를 낭비하고 오동작의 원인이 됩니다.

3. 간결한 결과 반환

서브에이전트의 시스템 프롬프트에 “결과를 간결하게 요약하라”는 지침을 포함하세요. 그래야 메인 에이전트가 효율적으로 결과를 처리할 수 있습니다.

4. 적절한 모델 선택

작업 복잡도에 따라 서브에이전트마다 다른 모델을 사용할 수 있습니다. 권장 기본은

`anthropic:claude-sonnet-4-6` 이며, 다양한 프로바이더의 모델을 유연하게 선택할 수 있습니다:

- 단순 수집 -> 가벼운 모델 (예: `anthropic:claude-haiku-4-5` , `openai:gpt-5.4-mini`)
- 깊은 분석 -> 강력한 모델 (예: `anthropic:claude-sonnet-4-6` , `openai:gpt-5.4`)

5. 서브에이전트 간 의존성 최소화

서브에이전트 A의 결과가 서브에이전트 B에 필요한 경우, 반드시 메인 에이전트를 경유하여 전달한다. 서브에이전트끼리 직접 통신하는 구조는 디버깅이 어렵고, 실패 시 복구가 곤란하다.

핵심 정리

항목	내용
SubAgent	dict 기반 정의: <code>name</code> , <code>description</code> , <code>system_prompt</code> , <code>tools</code> + 선택 필드 (<code>response_format</code> , <code>permissions</code>)
CompiledSubAgent	커스텀 LangGraph 그래프를 <code>runnable</code> 로 연결
General-Purpose	빌트인 기본 서브에이전트 — <code>SubAgentMiddleware</code> 가 항상 부착(제거 불가)
디스패치 패턴	<code>write_todos</code> → <code>task()</code> → 압축된 결과 반환
컨텍스트 전파	<code>context_schema</code> + <code>config["context"]</code>
네임스페이스 키	"에이전트이름:키" 형식으로 서브에이전트별 설정
스트리밍 식별	이벤트 메타데이터의 <code>lc_agent_name</code>
파이프라인 패턴	<code>collector</code> -> <code>analyzer</code> -> <code>writer</code>

서브에이전트를 통해 컨텍스트 블로트 문제를 해결하고, 전문 작업을 효율적으로 위임하는 방법을 배웠습니다. 그러나 서브에이전트 간에 지식을 영속적으로 공유하려면 장기 메모리가 필요합니다. 다음 장에서는 `AGENTS.md` , `SKILL.md` , `StoreBackend` 를 활용한 장기 메모리 시스템을 다룹니다.

06 장기 메모리 & 스킬

대화 스레드가 종료되면 기본 에이전트는 모든 정보를 잊는다. 실용적인 어시스턴트를 만들려면 사용자 선호도, 프로젝트 컨벤션, 도메인 지식 등을 대화 간에 유지하는 장기 메모리가 필수적이다. 이 장에서는

`CompositeBackend` + `StoreBackend` 를 활용한 크로스 스레드 메모리 구현과, `AGENTS.md` 기반 컨텍스트 주입, 그리고 `SKILL.md` 의 Progressive Disclosure 메커니즘을 학습한다.

5장에서 서브에이전트가 컨텍스트 격리를 제공하는 것을 보았다. 하지만 격리만으로는 대화가 끝난 뒤에도 에이전트가 학습한 내용을 유지하는 장기 기억을 구현할 수 없다. Deep Agents는 `InMemoryStore` (개발용) 또는 데이터베이스 기반 `Store` (프로덕션용)를 통해 크로스 스레드 메모리를 지원하며, `AGENTS.md` 와 `SKILL.md` 라는 두 가지 파일 기반 메커니즘으로 지식을 관리한다.

메모리 시스템의 핵심 API는 네임스페이스(namespace), `put`, `get`, `search` 의 네 가지다. 네임스페이스는 메모리 항목을 논리적으로 그룹화하는 경로이고, `put` 으로 항목을 저장하고, `get` 으로 특정 항목을 조회하며, `search` 로 의미 기반 검색을 수행한다. 이 API는 LangGraph의 `BaseStore` 인터페이스 위에 구축되어 있으므로, 백엔드를 `InMemoryStore` 에서 `PostgresStore` 로 교체해도 코드 변경 없이 동작한다.

학습 목표

LEARNING OBJECTIVES

- `CompositeBackend` + `StoreBackend` 로 장기 메모리를 구현한다
- 크로스 스레드 메모리 공유 패턴을 이해한다
- `AGENTS.md` 를 통해 에이전트에 컨텍스트를 주입한다
- 스킬(SKILL.md)의 구조와 Progressive Disclosure를 이해한다
- Skills vs Memory의 차이를 파악한다

```
# 환경 설정
from dotenv import load_dotenv
import os

load_dotenv()
assert os.environ.get("OPENAI_API_KEY"), "OPENAI_API_KEY가 설정되지 않았습니다!"
print("환경 설정 완료")
```

환경 설정 완료

```
# OpenAI gpt-5.4 모델 설정
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-5.4")
```

환경 설정을 마쳤으므로, 왜 장기 메모리가 필요한지부터 이해한 뒤 구현 방법으로 넘어간다.

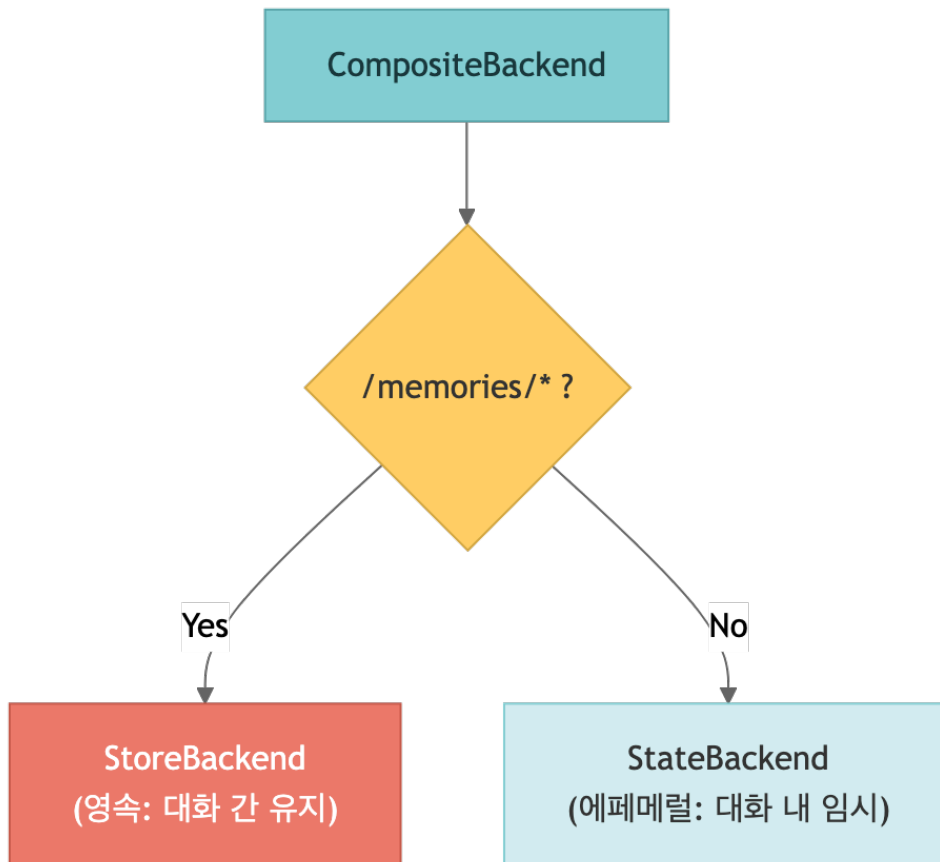
1. 장기 메모리가 필요한 이유

기본 `StateBackend` 에이전트는 대화 스레드가 끝나면 모든 정보를 잊습니다. 체크포인트(`MemorySaver`)가 대화 상태를 보존하지만, 이는 같은 스레드 내에서만 유효하다. 새 대화(새 `thread_id`)가 시작되면 에이전트는 백지 상태로 돌아간다.

하지만 실제 어시스턴트는 아래 정보를 대화 간에 유지해야 합니다:

- 사용자 선호도 (코딩 스타일, 사용 언어)
- 프로젝트 컨벤션 (아키텍처 결정, 네이밍 규칙)
- 이전 대화에서 학습한 피드백
- 자주 참조하는 정보 (API 문서, 설정값)

해결 방식: CompositeBackend



`/memories/` 경로에 저장된 파일은 어떤 대화 스레드에서든 접근할 수 있습니다. 이 구조에서 `StateBackend` 는 현재 대화의 임시 파일을 관리하고, `StoreBackend` 는 `/memories/` 하위 경로의 파일을 `InMemoryStore` (또는 데이터베이스)에 영속적으로 저장한다. `CompositeBackend` 가 경로별로 어떤 백엔드를 사용할지 라우팅하는 역할을 한다. 다음 코드에서 이 세 가지 백엔드를 조합하여 장기 메모리 에이전트를 구축하는 전체 과정을 살펴보자.

```

from deepagents import create_deep_agent
from deepagents.backends import StateBackend, StoreBackend, CompositeBackend, FilesystemBackend
from langgraph.store.memory import InMemoryStore
from langgraph.store.postgres import PostgresStore # 프로덕션용
from langgraph.checkpoint.memory import MemorySaver

# 1. 스토어와 체크포인트 생성
store = InMemoryStore() # 개발용 (프로덕션: PostgresStore)
checkpointer = MemorySaver() # 에이전트 상태 유지

# 2. CompositeBackend 팩토리 - /memories/만 영속, 나머지는 에페메럴
# StoreBackend는 namespace lambda로 스코프를 명시적으로 지정한다.
def memory_backend_factory(runtime):
    return CompositeBackend(
        default=StateBackend(runtime),
        routes={
            "/memories/": StoreBackend(
                namespace=lambda rt: (rt.context.user_id,),
            ),
        },
    )
  
```

```

    },
)

# 3. 에이전트 생성
memory_agent = create_deep_agent(
    model=model,
    system_prompt="""당신은 개인 어시스턴트입니다.
사용자가 기억해 달라고 하는 정보는 /memories/ 폴더에 저장하세요.
이전에 저장한 메모리가 있으면 참고하여 응답하세요.
한국어로 응답하세요."""
    backend=memory_backend_factory,
    store=store,
    checkpointer=checkpointer,
)

print("장기 메모리 에이전트 생성 완료")

```

장기 메모리 에이전트 생성 완료

위 코드에서 `memory_backend_factory` 는 함수(팩토리 패턴)로 정의되어 있다. `runtime` 인자는 에이전트 실행 시 자동으로 주입되며, 현재 스레드의 `assistant_id`, `thread_id` 등의 런타임 정보를 포함한다. 이 정보를 기반으로 `StoreBackend` 가 올바른 네임스페이스에 메모리를 저장한다.

2. 크로스 스레드 메모리 공유

장기 메모리의 핵심은 크로스 스레드 공유입니다. `StoreBackend` 에 저장된 데이터는 `assistant_id` 기반 네임스페이스로 관리되며, 어떤 대화 스레드에서든 같은 네임스페이스의 데이터에 접근할 수 있습니다. 아래 예제에서 스레드 1에서 저장한 선호도를 스레드 2에서 읽어봅니다.

네임스페이스의 구조는 `(assistant_id, "memories", category)` 형태의 튜플이다. 예를 들어 `("asst-001", "memories", "preferences")` 라는 네임스페이스에 `put` 으로 항목을 저장하면, 동일한 `assistant_id` 를 사용하는 모든 스레드에서 `get` 또는 `search` 로 해당 항목을 조회할 수 있다. `search` API는 의미 기반 유사도 검색을 지원하므로, 정확한 키를 모르더라도 자연어 쿼리로 관련 메모리를 찾을 수 있다.

TIP

`InMemoryStore` 는 개발/테스트 용도입니다. 프로덕션 환경에서는 `PostgresStore` 등 영구 스토어를 사용하세요. LangSmith 배포 시에는 스토어가 자동으로 프로비저닝됩니다. `InMemoryStore` 는 프로세스가 종료되면 모든 데이터가 사라지므로, 영속성이 필요한 경우 반드시 데이터베이스 기반 스토어를 선택하라.

크로스 스레드 메모리가 동적으로 학습된 정보를 저장하는 메커니즘이라면, `AGENTS.md` 는 사전에 정의된 규칙과 컨텍스트를 에이전트에 주입하는 메커니즘이다. 두 가지를 함께 사용하면, 에이전트는 프로젝트 규칙을 항상 준수하면서도 사용자별 선호도를 기억하는 완전한 지식 시스템을 갖출 수 있다.

3. AGENTS.md를 통한 컨텍스트 주입

`memory` 파라미터를 사용하면, 에이전트가 시작될 때 AGENTS.md 파일을 자동으로 로드하여 시스템 프롬프트에 주입합니다.

AGENTS.md란?

에이전트에게 항상 적용되어야 하는 규칙, 컨벤션, 컨텍스트 정보를 담은 마크다운 파일입니다. 프로젝트의 아키텍처 결정, 코딩 스타일 가이드, 자주 참조하는 API 정보 등을 기록합니다. AGENTS.md 는 LLM 에이전트판 `.editorconfig` 또는 `.eslintrc` 에 해당한다고 볼 수 있다. 팀원 모두가 같은 규칙을 따르듯, 에이전트도 AGENTS.md 에 정의된 규칙을 일관되게 따른다.

특징

- 에이전트가 시작할 때 항상 로드 (on-demand가 아님) - `MemoryMiddleware` 가 `<agent_memory>` 태그로 시스템 프롬프트에 주입
- `memory` 파라미터에 여러 경로를 배열로 지정 가능 (예: `memory=["/global/AGENTS.md", "/project/AGENTS.md"]`)
- 에이전트가 `edit_file` 도구로 AGENTS.md를 스스로 업데이트 가능 - 사용자 피드백을 반영하여 자가 학습

! WARNING

AGENTS.md 는 매 대화마다 시스템 프롬프트에 포함되므로, 내용이 길어지면 토큰 비용이 증가합니다. 핵심 규칙만 간결하게 유지하고, 상세한 가이드는 SKILL.md 로 분리하세요. 일반적으로 AGENTS.md 는 500 1,000 토큰 이 내를 권장합니다.

➡ TIP

AGENTS.md 를 여러 경로로 분리하면 글로벌 규칙과 프로젝트별 규칙을 계층적으로 관리할 수 있다. 예: `memory=["/global/AGENTS.md", "/project/AGENTS.md"]` . 두 파일의 내용이 충돌하면 나중에 나온 경로(프로젝트)가 우선한다.

```
import tempfile

# 임시 디렉토리 생성 - FilesystemBackend의 root_dir로 사용
tmp_dir = tempfile.mkdtemp()
print(f"임시 디렉토리 생성: {tmp_dir}")
```

임시 디렉토리 생성: C:\Users\HEESU\AppData\Local\Temp\tmpj97x7phs

AGENTS.md 가 항상 로드되는 규칙이라면, 다음에 다룰 스킬(Skills)은 필요할 때만 로드되는 전문 지식이다. 대규모 API 레퍼런스처럼 항상 로드하기에는 너무 큰 지식을 효율적으로 관리하는 방법이다.

4. 스킬 (Skills)

스킬은 에이전트에게 도메인 전문 지식을 부여하는 모듈화된 지침 세트입니다. `SKILL.md` 파일로 정의하며, YAML 프론트매터에 이름과 설명을, 마크다운 본문에 상세 지침을 담는다. 스킬의 핵심 설계 원리는 Progressive Disclosure(점진적 공개)로, 프론트매터만 먼저 노출하고 전체 내용은 필요할 때만 로드하여 토큰을 절약한다.

Memory vs Skills

비교	Memory (AGENTS.md)	Skills (SKILL.md)
로딩	항상 로드 (Always)	필요 시 로드 (On-demand)
파일 형식	<code>AGENTS.md</code>	<code>SKILL.md</code> (YAML 프론트매터)
적합한 용도	항상 적용되는 규칙/컨벤션	특정 태스크에 필요한 큰 컨텍스트
토큰 효율	항상 소비	점진적 공개로 절약
크기	간결할수록 좋음	대용량 가능 (10MB 제한)
업데이트	에이전트가 <code>edit_file</code> 로 수정 가능	보통 정적

Progressive Disclosure (점진적 공개)

스킬은 한 번에 전부 로드하지 않습니다. 이것이 `AGENTS.md` (항상 로드)와의 핵심 차이점입니다:

- 에이전트 시작 시, `SkillsMiddleware` 가 모든 `SKILL.md` 파일의 프론트매터(이름, 설명)만 로드하여 시스템 프롬프트에 스킬 목록으로 제시합니다.
- 사용자 요청이 들어오면, 에이전트가 스킬 목록을 참조하여 관련 스킬을 판단합니다.
- 필요한 스킬의 전체 내용(마크다운 본문)을 그때 로드하여 컨텍스트에 추가합니다.

이 방식으로 수십 개의 스킬을 등록해도, 실제로 사용되는 스킬만 토큰을 소비합니다. 각 스킬 파일은 최대 10MB 까지 가능하므로, 대규모 API 레퍼런스나 도메인 가이드도 스킬로 관리할 수 있습니다.

! WARNING

스킬의 `description` 이 모호하면 에이전트가 관련 스킬을 찾지 못하거나 잘못된 스킬을 로드할 수 있다. “웹 리서치 스킬” 보다는 “체계적인 웹 리서치를 수행하기 위한 단계별 가이드. 정보 수집, 검증, 정리까지의 전체 워크플로를 다룹니다” 처럼 구체적으로 작성하라. `description` 은 최대 1,024자까지 허용된다.

SKILL.md 구조

다음은 `SKILL.md` 파일의 전체 구조이다. YAML 프론트매터(--- 로 감싼 부분)에는 스킬 메타데이터를, 그 아래 마크다운 본문에는 에이전트가 따라야 할 상세 지침을 작성한다. 프론트매터의 `module` 필드는 interpreter용 스킬(QuickJS `CodeInterpreterMiddleware` 에서 로드)일 때만 지정하며, 일반 텍스트 스킬에서는 생략한다.

```
---
name: web-research           # 스킬 식별자 (최대 64자, 소문자+하이픈)
description: >               # 설명 (최대 1024자) - 매칭에 사용
  체계적인 웹 리서치를 수행하기 위한 단계별 가이드.
```

```

    정보 수집, 검증, 정리까지의 전체 워크플로를 다룹니다.
license: MIT
compatibility: Python 3.8+
module: ./web_research.js    # (선택) interpreter 스킵일 때만
metadata:
  category: research
allowed-tools: ls read_file write_file
---
```

Web Research 스킵

사용 시기

- 사용자가 특정 주제에 대한 조사를 요청할 때
- 최신 정보가 필요한 질문이 들어올 때

워크플로

1. 검색 쿼리 설계
2. 다양한 소스에서 정보 수집
3. 정보 교차 검증
4. 구조화된 보고서 작성

```

# 스킬을 사용하는 에이전트 생성
skilled_agent = create_deep_agent(
    model=model,
    system_prompt="당신은 시니어 개발자입니다. 사용 가능한 스킬을 활용하여 작업을 수행하세요.",
    backend=FilesystemBackend(root_dir=tmp_dir, virtual_mode=True),
    skills=["/skills/"], # 스킬 소스 디렉토리
)

print("스킬 에이전트 생성 완료")

```

스킬 에이전트 생성 완료

위 코드에서 `skills=["/skills/"]` 는 해당 경로 아래의 모든 `SKILL.md` 파일을 자동으로 탐색한다. 에이전트가 시작되면 `SkillsMiddleware` 가 각 스킬 파일의 프론트매터만 읽어 스킬 카탈로그를 시스템 프롬프트에 추가한다. 사용자 요청이 들어오면 에이전트가 카탈로그를 참고하여 관련 스킬을 선택하고, 그때 비로소 전체 내용을 로드한다.

스킬 소스는 하나만 지정할 수도 있고 여러 개를 지정할 수도 있다. 여러 소스를 지정하면 우선순위가 적용된다.

5. 스킬 소스 우선순위

여러 스킬 소스를 지정하면, 나중에 나온 소스가 우선합니다 (last wins).

```

skills=[
    "/skills/base/",    # 기본 스킬
    "/skills/user/",    # 사용자 스킬 (base 덮어쓰기 가능)
    "/skills/project/", # 프로젝트 스킬 (최우선)
]

```

같은 이름의 스킬이 여러 소스에 있으면, 마지막 소스의 스킬이 사용됩니다. 이 규칙을 활용하면 기본 스킬 세트를 팀 전체가 공유하면서, 프로젝트나 사용자 단위로 특화된 스킬을 덮어쓸 수 있다.

서브에이전트도 스킬을 사용할 수 있다. 다만 스킬 상속 방식이 서브에이전트 유형에 따라 다르므로, 다음 섹션에서 이를 정리한다.

6. 서브에이전트의 스킬 상속

서브에이전트 유형	스킬 상속
General-purpose (빌트인)	메인 에이전트의 스킬을 자동 상속
커스텀 SubAgent	명시적 <code>skills</code> 파라미터 필요

```
# 커스텀 서브에이전트에 스킬 부여
subagent = {
    "name": "reviewer",
    "description": "코드 리뷰 전문 에이전트",
    "system_prompt": "...",
    "tools": [],
    "skills": ["/skills/code-review/"], # 명시적 스킬 경로
}
```

TIP

서브에이전트에 스킬을 부여할 때는 최소 권한 원칙을 따르라. 리뷰 전문 서브에이전트에 배포 관련 스킬까지 주면 불필요한 토큰 소비와 혼란이 발생한다. 각 서브에이전트의 역할에 딱 맞는 스킬만 할당하라.

Skills 백엔드 지원 매트릭스

스킬 소스 디렉토리는 `StateBackend` / `StoreBackend` / `FilesystemBackend` / `CompositeBackend` 어느 곳에도 둘 수 있지만, 각 백엔드는 다른 운영 특성을 가진다. 다음 표가 선택 기준이다.

백엔드	지속성	공유 범위	권장 용도
<code>StateBackend</code>	Thread-local (에페머럴)	현재 스레드	임시 실험·테스트
<code>StoreBackend</code>	영속 (DB-backed)	namespace 단위 (user/org/assistant)	팀·사용자 공유 스킬
<code>FilesystemBackend</code>	영속 (디스크)	호스트 디렉토리	Git에 커밋된 표준 스킬
<code>CompositeBackend</code>	경로별로 라우팅	경로별로 분리	<code>/skills/base</code> + <code>/skills/user/</code> 처럼 계층 구성

6.10 장기 기억 유형 분류 (0.5 기준)

Deep Agents 0.5는 저장되는 정보를 세 범주로 구분해 관리합니다. 각 유형은 저장 메커니즘·업데이트 주기·적합한 백엔드가 다릅니다. 세 유형을 하나의 백엔드로 몰아넣지 말고, 성격에 맞는 메커니즘에 분산시키는 것이 기본 원칙입니다.

유형	의미	저장 예	메커니즘
Episodic	과거 경험 – 대화 세션, 문제 해결 궤적	지난 대화의 스레드 히스토리	Checkpointers (thread 단위)
Procedural	재사용 가능한 지시·스킬·워크플로우	SKILL.md, 절차 문서	Skills (on-demand 로드)
Semantic	사실·선호·정책	AGENTS.md, /memories/*.txt	StoreBackend (always-on 파일)

예: 장시간 누적 선호는 `/memories/` (semantic) + 일화적 사례 검색은 `checkpointer(episodic)` + 반복 절차는 `skill(procedural)`.

스코프 패턴: agent-scoped / user-scoped / context-driven

`StoreBackend`의 `namespace` 함수가 반환하는 튜플이 곧 메모리의 스코프입니다. 다음 세 패턴이 가장 자주 쓰입니다.

패턴	namespace 예	적합한 용도
Agent-scoped	<code>(rt.server_info.assistant_id,)</code>	조직 공통 컨벤션·도메인 지식 (사용자 간 공유)
User-scoped	<code>(rt.server_info.user.identity,)</code>	개인화 어시스턴트의 기본값 – 사용자별 완전 격리
Context-driven	<code>(rt.context.user_id, rt.context.project_id)</code>	<code>context_schema</code> 에 선언한 키로 다차원 격리

Agent-scoped – 공통 정체성 추적. 네임스페이스가 `(assistant_id,)`. 같은 `assistant`를 쓰는 모든 사용자 대화가 같은 메모리를 공유합니다. 조직 공통 컨벤션·도메인 지식에 적합하며, 사용자 간 정보가 섞이므로 민감 정보 저장 금지.

```
from deepagents.backends import StoreBackend

agent_scoped = StoreBackend(
    namespace=lambda rt: (rt.server_info.assistant_id,),
)
```

User-scoped – 사용자별 격리. 네임스페이스가 `(user_id,)` 또는 `(assistant_id, user_id,)`. 각 사용자의 메모리가 완전히 격리되어 A의 선호가 B의 대화에 노출되지 않습니다. 프로덕션 개인화 어시스턴트의 기본값입니다.

```
user_scoped = StoreBackend(
    namespace=lambda rt: (rt.server_info.user.identity,),
)
```

Episodic memory via checkpoints

과거 대화를 단순히 “보관” 하는 것이 아니라 검색 가능한 기억으로 만들려면 체크포인트가 저장한 스레드를 도구로 래핑합니다.

```
from langchain.tools import tool, ToolRuntime

@tool
async def search_past_conversations(query: str, runtime: ToolRuntime) -> str:
    """지난 대화에서 관련 맥락을 찾는다."""
    user_id = runtime.server_info.user.identity
    threads = await client.threads.search(
        metadata={"user_id": user_id},
        limit=5,
    )
    # threads를 필요한 포맷으로 요약해 반환
    ...
```

이 패턴으로 에이전트가 “전에 이 문제를 어떻게 풀었는가”를 스스로 참조합니다.

Read-only policy (조직 차원)

조직 공유 메모리는 인젝션 벡터입니다. 다음과 같이 쓰기 차단을 강제합니다.

```
# policies는 조직 스코프, read-only
routes = {
    "/policies/": StoreBackend(
        namespace=lambda rt: (rt.context.org_id,),
    ),
}
```

Part IV ch15 권한 관리 패턴 4와 결합해 `/policies/**` 에 `write deny` 를 겁니다. 정책은 애플리케이션 코드에서만 갱신합니다.

Background consolidation agent

메모리 갱신을 대화 중(hot path) 에 하면 지연이 늘고, 요약 품질도 모델이 서두른 결정을 따르게 됩니다. 대안은 별도 consolidation agent를 두고 cron으로 주기 실행하는 것입니다.

```
// langgraph.json
{
  "graphs": {
    "agent": "./agent.py:agent",
    "consolidation_agent": "./consolidation_agent.py:agent"
  }
}
```

cron 스케줄 등록:

```
cron_job = await client.crons.create(
    assistant_id="consolidation_agent",
    schedule="0 */6 * * *", # 6시간마다
    input={"messages": [{"role": "user", "content": "Consolidate recent memories."}]},
)
```

! WARNING

cron 주기(`0 */6 * * *`)와 lookback window(`timedelta(hours=6)`)를 일치시켜야 누락·중복이 없습니다.

업데이트 타이밍 비교

타이밍	방식	지연	반영 시점
Hot path	대화 중 에이전트가 직접 <code>edit_file</code>	있음	즉시
Background	Consolidation agent가 세션 사이에 처리	사용자 체감 없음	다음 대화부터

민감하거나 즉시 반영이 필수인 선호는 hot path, 장기 패턴 누적은 background로 분리하는 것이 기본 구성입니다.

핵심 정리

항목	내용
장기 메모리	CompositeBackend + StoreBackend 로 <code>/memories/</code> 영속화
AGENTS.md	<code>memory=["/path/AGENTS.md"]</code> → 항상 시스템 프롬프트에 주입
Skills	<code>skills=["/skills/"]</code> → SKILL.md 기반 점진적 공개
Progressive Disclosure	프론트매터만 먼저 로드 → 필요 시 전체 로드
스킬 우선순위	나중 소스가 우선 (last wins)
Memory vs Skills	Memory = 항상 로드 / Skills = 필요 시 로드
기억 3유형	Episodic(checkpointer) / Procedural(skills) / Semantic(store)
스코프 패턴	agent-scoped (공통 축적) / user-scoped (격리, 기본값) / org-scoped (read-only)
Consolidation	Hot path는 즉시·고지연 / Background cron은 비간섭·다음 대화부터 반영

장기 메모리와 스킬 시스템을 통해 에이전트는 대화를 넘어서 지식을 축적하고, 필요할 때 전문 능력을 발휘할 수 있게 되었습니다. 다음 장에서는 Human-in-the-Loop, 스트리밍 심화, 샌드박스, ACP, CLI 등 프로덕션 수준의 고급 기능을 다룹니다.

07 고급 기능

기본적인 에이전트 구축을 넘어, 프로덕션 수준의 에이전트에는 안전장치와 확장 기능이 필요하다. 이 장에서는 `interrupt_on` 을 활용한 Human-in-the-Loop 워크플로, 네임스페이스 기반 스트리밍 심화, 샌드박스 프로바이더(Modal, Daytona, Runloop) 연동, `ACP` 를 통한 에디터 통합, 그리고 Deep Agents CLI 사용법을 다룬다. 이 기능들을 조합하면 안전하고 관측 가능한 에이전트 시스템을 구축할 수 있다.

6장까지 에이전트의 핵심 기능(도구, 백엔드, 서브에이전트, 메모리, 스킬)을 모두 다루었다. 이 장에서는 이러한 기능들을 프로덕션 환경에서 안전하게 운영하기 위한 가드레일과 통합 기능에 집중한다. 특히 에이전트의 자율성이 높아질수록 안전장치의 중요성도 비례하여 커진다. 파일을 수정하거나 셸 명령을 실행하는 에이전트가 실수하면 복구 비용이 크기 때문이다. 또한 Deep Agents의 미들웨어 스택을 커스터마이징하여 로깅, 인증, 속도 제한 등의 횡단 관심사를 에이전트 로직과 분리하는 방법도 다룬다.

학습 목표

LEARNING OBJECTIVES

- Human-in-the-Loop 워크플로를 구현한다
- 다양한 스트리밍 모드와 네임스페이스 시스템을 이해한다
- 샌드박스(Modal, Daytona, Runloop) 연동 개념을 파악한다
- ACP(Agent Client Protocol)로 에디터와 연동하는 방법을 안다
- Deep Agents CLI 사용법을 익힌다

```
# 환경 설정
from dotenv import load_dotenv
import os

load_dotenv()
assert os.environ.get("OPENAI_API_KEY"), "OPENAI_API_KEY가 설정되지 않았습니다!"
print("환경 설정 완료")
```

환경 설정 완료

```
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-5.4")

print(f"모델 설정 완료: {model.model_name}")
```

모델 설정 완료: gpt-5.4

환경 설정을 마쳤으므로, 가장 기본적인면서도 중요한 안전장치인 Human-in-the-Loop부터 시작한다.

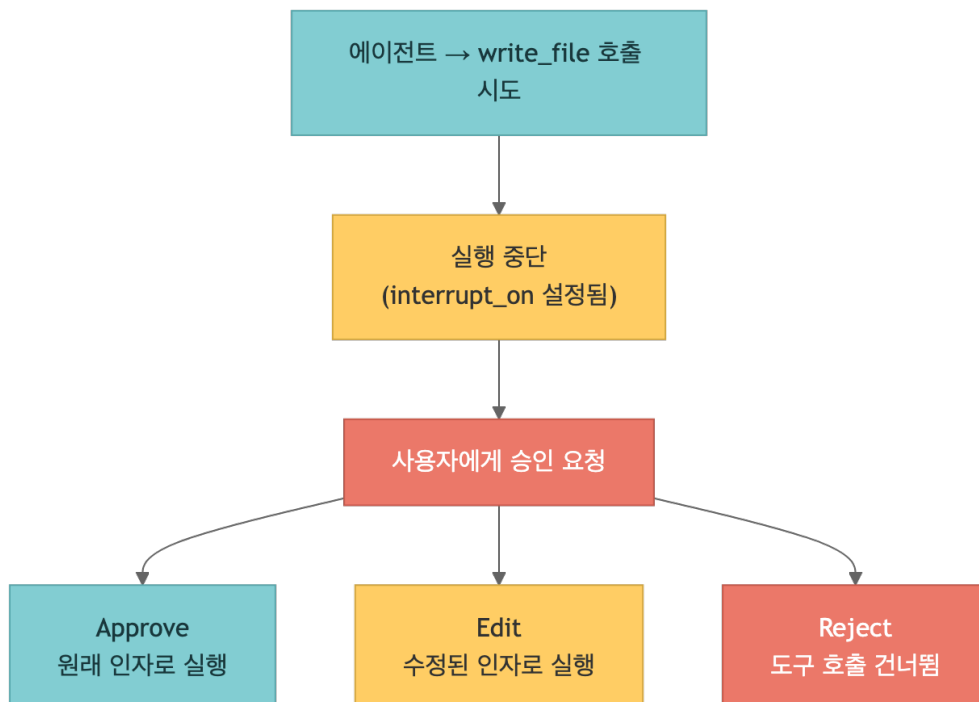
1. Human-in-the-Loop (HITL)

자율 에이전트가 파일을 수정하거나 셸 명령을 실행할 때, 잘못된 판단이 치명적인 결과를 초래할 수 있습니다. 예를 들어 에이전트가 `rm -rf /` 같은 위험한 명령을 생성하거나, 프로덕션 데이터베이스에 잘못된 마이그레이션을 적용할 수 있다.

`interrupt_on` 파라미터를 사용하면, 에이전트가 지정된 도구를 호출하려 할 때 실행을 중단하고 사람의 승인을 요구합니다. Deep Agents의 HITL은 4가지 결정 타입을 지원하며, 각 결정은

`Command(resume={"decisions": [...]})` 형태의 dict로 재개 시 전달한다. 이 메커니즘은 LangGraph의 인터럽트 기능 위에 구축되어 있으며, 체크포인트가 중단 시점의 전체 상태를 보존하므로 승인 후 정확히 그 지점에서 실행이 재개된다.

작동 방식



4가지 결정 타입

결정	동작
approve	제안된 인자 그대로 도구 실행
edit	<code>edited_action.name/args</code> 로 인자를 수정한 뒤 실행
reject	호출 자체를 건너뛴
respond	도구 실행 없이 <code>message</code> 를 도구 결과로 반환

인터럽트 확인과 재개 (v2 표준)

`invoke(..., version="v2")` 로 호출하면, 인터럽트는 `result.interrupts` 에 노출된다. 옛 `result["__interrupt__"]` 키가 아니다. 각 인터럽트의 `interrupt_value["action_requests"]` 가 제안된 도구 호출과 `allowed_decisions` 를 담는다. 같은 `thread_id` 로 `Command(resume={"decisions": [...]})` 을 dict 형태로 `invoke`하면 재개된다.

```
from langgraph.types import Command

# 인터럽트 추출
result = hitl_agent.invoke(inputs, config=config, version="v2")
for itr in result.interrupts:
    for req in itr.value["action_requests"]:
        print(req["action"]["name"], req["allowed_decisions"])

# 재개 - 인터럽트와 동일한 순서의 decisions 배열
hitl_agent.invoke(
    Command(resume={"decisions": [{"type": "approve"}]}),
    config=config,
    version="v2",
)
```

필수 요구사항

! WARNING

Human-in-the-Loop을 사용하려면 반드시 `checkpointer` 를 설정해야 합니다. 에이전트가 중단된 시점의 상태를 보존하고, 승인 후 정확히 그 지점에서 재개하기 위해 체크포인트가 필수입니다.

```
from deepagents import create_deep_agent
from langgraph.checkpoint.memory import MemorySaver

# interrupt_on으로 승인이 필요한 도구 지정
hitl_agent = create_deep_agent(
    model=model,
    system_prompt="당신은 파일 관리 어시스턴트입니다. 한국어로 응답하세요.",
    checkpointer=MemorySaver(), # 필수!
    interrupt_on={
        "write_file": True, # 파일 쓰기 전 승인 필요
        "edit_file": True, # 파일 편집 전 승인 필요
    },
)

print("Human-in-the-Loop 에이전트 생성 완료")
print("write_file, edit_file 호출 시 승인을 요구합니다.")
```

Human-in-the-Loop 에이전트 생성 완료
write_file, edit_file 호출 시 승인을 요구합니다.

위 예시에서 `interrupt_on`의 키는 도구 이름이고, 값이 `True`이면 해당 도구 호출 전에 항상 중단한다. 값에 조건 함수를 전달하여 특정 조건에서만 중단하도록 설정할 수도 있다(예: 파일 크기가 일정 이상일 때만 승인 요구).

`interrupt_on`은 서브에이전트에도 개별적으로 설정할 수 있어, 메인 에이전트는 자유롭게 동작하되 서브에이전트의 위험한 작업만 승인하는 구성도 가능하다.

TIP

HITL을 모든 도구에 걸면 에이전트의 자율성이 크게 떨어진다. 파일 쓰기, 셸 실행, 외부 API 호출 등 부작용(side effect)이 있는 도구에만 선택적으로 적용하고, 읽기 전용 도구는 자유롭게 호출하도록 허용하는 것이 실용적이다.

HITL로 안전장치를 구축했다면, 다음은 에이전트의 실행 과정을 실시간으로 관찰하는 스트리밍 기능이다. 관찰 가능성(observability)은 디버깅과 사용자 경험 양면에서 중요하다.

2. 스트리밍 심화

Deep Agents는 LangGraph의 스트리밍 인프라 위에서 작동하며, 특히 서브에이전트를 사용할 때 네임스페이스 기반 이벤트 구분이 중요합니다. 스트리밍을 통해 에이전트가 어떤 서브에이전트를 호출 중인지, 도구 실행이 어디까지 진행되었는지를 실시간으로 파악할 수 있다.

v2 통합 포맷 (표준)

`agent.stream(..., stream_mode=..., subgraphs=True, version="v2")` 한 경로가 표준입니다. 옛 v1의 중첩 튜플 포맷은 분기가 복잡하므로 v2를 권장하며, v3·projection 같은 비공식 변형은 사용하지 마세요. 모든 체크는 동일한 3-필드 구조입니다.

```
{
  "type": "updates" | "messages" | "custom",
  "ns": tuple,           # 이벤트 발생 위치 (메인/서브에이전트 라우팅)
  "data": Any,          # type 별 payload
}
```

TIP

`subgraphs=True`가 없으면 서브에이전트 내부 이벤트가 보이지 않는다. UI에서 서브에이전트 진행을 표시하려면 반드시 켜라. 요약(summarization) 토큰을 사용자에게 숨기려면

`metadata.get("lc_source") = "summarization"`로 필터링한다.

스트림 모드

Deep Agents에서 `stream()` 메서드 호출 시 `stream_mode` 파라미터로 스트림 모드를 선택한다. 각 모드는 서로 다른 수준의 정보를 제공하므로, 목적에 맞게 선택해야 한다. 리스트로 전달하면 한 루프에서 여러 모드를 동시에 받는다.

모드	설명	사용 시나리오
"updates"	각 노드 완료 시 상태 업데이트	진행 상황 추적
"messages"	개별 토큰 단위 스트리밍	실시간 텍스트 출력
"custom"	도구 내부에서 발행하는 이벤트	커스텀 진행률

네임스페이스 시스템

서브에이전트가 포함된 에이전트 시스템에서는 이벤트의 출처를 구분하는 것이 중요하다. 메인 에이전트의 토큰 인지, researcher 서브에이전트의 토큰인지에 따라 UI 표시 방식이 달라져야 하기 때문이다. Deep Agents는 이를 네임스페이스 튜플로 구분한다:

```
() # 메인 에이전트
("tools:abc123",) # 서브에이전트 (tool call ID)
("tools:abc123", "model:def456") # 서브에이전트 내부 노드
```

```
from typing import Literal
from tavily import TavilyClient

tavily_client = TavilyClient(api_key=os.environ.get("TAVILY_API_KEY", ""))

def internet_search(
    query: str,
    max_results: int = 3,
    topic: Literal["general", "news"] = "general",
) -> dict:
    """인터넷에서 정보를 검색합니다."""
    return tavily_client.search(query, max_results=max_results, topic=topic)

# 서브에이전트 포함 에이전트
stream_agent = create_deep_agent(
    model=model,
    system_prompt="당신은 리서치 코디네이터입니다. 한국어로 응답하세요.",
    subagents=[
        {
            "name": "researcher",
            "description": "인터넷 검색을 통해 정보를 조사합니다.",
            "system_prompt": "인터넷을 검색하여 요청된 정보를 수집하고 간결하게 요약하세요.",
            "tools": [internet_search],
        }
    ],
)

print("스트리밍 데모 에이전트 생성 완료")
```

스트리밍 데모 에이전트 생성 완료

위 코드로 생성한 에이전트를 `stream()` 호출 시, 메인 에이전트의 이벤트는 빈 네임스페이스 `()` 로, researcher 서브에이전트의 이벤트는 `( "tools:xxx", )` 형태의 네임스페이스로 전달된다. 클라이언트 측에서 네임스페이스를 필터링하면 “현재 researcher가 작업 중입니다...” 같은 진행 상황 표시를 구현할 수 있다.

커스텀 진행률 이벤트 (`get_stream_writer` + `stream_mode="custom"`)

도구 내부에서 임의 구조체를 방출해 UI에 노출한다. 업로드 진행률·처리 건수·중간 상태 등에 활용한다. 도구마다 스키마가 제각각이면 UI 라우팅이 깨지므로, `{"status", "progress", "message"}` 같은 공통 필드를 고정해 두는 편이 좋다.

```
from langchain.tools import tool
from langgraph.config import get_stream_writer

@tool
def analyze_data(topic: str) -> str:
    """주제 데이터를 분석하고 진행률을 스트리밍한다."""
    writer = get_stream_writer()
    writer({"status": "starting", "progress": 0, "topic": topic})
    # ... 실제 분석 ...
    writer({"status": "complete", "progress": 100, "topic": topic})
    return f"분석 완료: {topic}"

for chunk in agent.stream(
    {"messages": [{"role": "user", "content": "..."}]},
    stream_mode=["updates", "messages", "custom"],
    subgraphs=True,
    version="v2",
):
    if chunk["type"] == "custom":
        print(chunk["data"])
```

스트리밍으로 에이전트의 동작을 실시간 관찰할 수 있게 되었다. 다음 단계는 에이전트가 실제 코드를 실행할 때의 안전성이다. 샌드박스는 코드 실행을 격리된 환경으로 제한하여 호스트 시스템을 보호한다.

3. 샌드박스 (Sandbox)

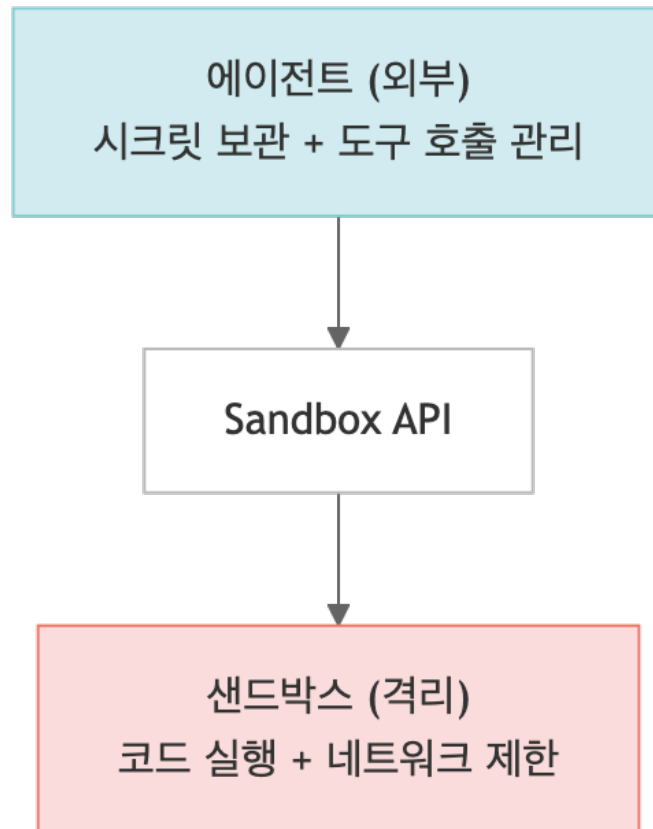
샌드박스는 에이전트가 격리된 환경에서 코드를 실행할 수 있게 합니다. 호스트 시스템의 파일, 네트워크, 자격 증명에 접근하지 못하므로 에이전트의 실수나 악의적 행동으로부터 시스템을 보호합니다. Deep Agents에서 샌드박스는 백엔드로 통합되므로, 기존 파일 도구(`read_file` , `write_file` , `run_command` 등)를 그대로 사용하면서 실행 환경만 격리할 수 있습니다. 에이전트 코드를 한 줄도 수정하지 않고 백엔드만 교체하면 로컬 실행에서 샌드박스 실행으로 전환할 수 있다는 것이 핵심이다.

지원 프로바이더

프로바이더	특징	적합한 용도
Modal	GPU 지원, ML 워크로드	AI/ML 작업
Daytona	TypeScript/Python, 빠른 콜드 스타트	웹 개발
Runloop	일회용 devbox, 격리 실행	코드 테스트

아키텍처 패턴

샌드박스를 도구로 사용 (권장)

**! WARNING**

절대 샌드박스 안에 시크릿을 넣지 마세요. 에이전트는 환경 변수를 읽거나 파일을 탐색하여 자격 증명을 외부로 유출할 수 있습니다. 자격 증명은 반드시 호스트(에이전트 측)의 전용 도구에서만 관리하세요.

보안 가이드라인

- 자격 증명은 외부 도구에서만 관리 - 샌드박스 환경에 API 키를 환경 변수로 주입하지 마라
- Human-in-the-Loop으로 민감한 작업 승인 - 샌드박스과 HITL을 결합하면 이중 안전장치가 된다
- 불필요한 네트워크 접근 차단 - 샌드박스의 네트워크 정책을 화이트리스트 방식으로 설정
- 실행 시간 제한 설정 - 무한 루프를 방지하기 위해 타임아웃을 반드시 지정

다음은 Modal 샌드박스를 연동하는 코드 예시이다. 다른 프로바이더(Daytona, Runloop)도 유사한 인터페이스를 따른다.

```
# Modal 샌드박스 연동 - langchain-modal 패키지 사용
# pip install langchain-modal deepagents
import modal
from deepagents import create_deep_agent
from langchain_anthropic import ChatAnthropic
from langchain_modal import ModalSandbox

app = modal.App.lookup("your-app")
modal_sandbox = modal.Sandbox.create(app=app)
backend = ModalSandbox(sandbox=modal_sandbox)
```

```
agent = create_deep_agent(
    model=ChatAnthropic(model="claude-sonnet-4-6"),
    system_prompt="You are a Python coding assistant with sandbox access.",
    backend=backend,
)

try:
    result = agent.invoke({
        "messages": [{"role": "user", "content": "Create a small Python package and run pytest"}],
    })
finally:
    modal_sandbox.terminate()    # 필수: 리소스 해제
```

TIP

샌드박스 프로바이더를 선택할 때는 워크로드 특성을 고려하라. GPU가 필요한 ML 작업에는 Modal이, 빠른 콜드 스타트가 중요한 웹 개발에는 Daytona가, 일회용 격리 실행이 필요한 테스트에는 Runloop이 적합하다. 각 프로바이더는 별도 패키지(`langchain-modal` , `langchain-daytona` 등)로 설치하고, `try/finally` 로 반드시 `terminate()` (또는 `stop()` / `shutdown()`)를 호출해 정리한다.

3-1. Interpreters — CodeInterpreterMiddleware

Deep Agents 0.6 계열은 QuickJS 기반 interpreter를 `CodeInterpreterMiddleware` 로 붙일 수 있다. 샌드박스가 “외부 환경에서 코드를 실행”하는 격리 실행이라면, interpreter는 “에이전트 루프 내부에서 작은 프로그램을 실행”하는 내부 작업 공간이다. 도구 호출 조합, 서브에이전트 fan-out/fan-in, 구조화 데이터 처리에 쓰인다.

설치

```
pip install -U "deepagents[quickjs]"
# 또는
uv add "deepagents[quickjs]"
```

Programmatic Tool Calling (PTC)

`ptc=["task"]` 처럼 allowlist를 지정하면, 도구가 camelCase로 변환되어 `tools.*` async 함수로 노출된다 (예: `web_search` → `tools.webSearch`). interpreter 내부에서 `Promise.all` 로 fan-out 가능하다.

```
from deepagents import create_deep_agent
from langchain_quickjs import CodeInterpreterMiddleware

agent = create_deep_agent(
    model="openai:gpt-5.4",
    middleware=[
        CodeInterpreterMiddleware(
            ptc=["task"],          # allowlist: task만 노출
            snapshot_between_turns=True, # turn 간 interpreter state 복원
            timeout=5.0,
            max_ptc_calls=256,
        ),
    ],
)
```

Middleware 옵션 10가지

파라미터	기본값	용도
<code>memory_limit</code>	64*1024*1024	QuickJS heap memory limit (bytes)
<code>timeout</code>	5.0	Per-eval 타임아웃(초)
<code>max_ptc_calls</code>	256	한 eval에서 허용되는 <code>tools.*</code> 호출 수
<code>tool_name</code>	"eval"	Interpreter 도구 이름
<code>max_result_chars</code>	4000	반환 결과 최대 문자 수
<code>capture_console</code>	True	<code>console.log</code> 캡처
<code>ptc</code>	None	PTC allowlist
<code>skills_backend</code>	None	Interpreter skill 모듈용 backend
<code>snapshot_between_turns</code>	True	turn 간 상태 보존
<code>max_snapshot_bytes</code>	None	snapshot 최대 크기

! WARNING

PTC는 일반 tool-calling 경로와 다르게 동작하므로 도구별 `interrupt_on` 승인 정책이 자동 적용된다고 가정하지 마라. 비용 발생·데이터 변경·네트워크 접근 도구는 interpreter에 노출하지 않거나 별도 승인 래퍼를 둔다.

샌드박스가 에이전트의 코드 실행을 격리한다면, 다음에 다루는 ACP는 에이전트와 개발자의 작업 환경을 연결하는 프로토콜이다. 코딩 에이전트가 IDE와 자연스럽게 통합되려면 표준화된 통신 규약이 필요하다.

4. ACP (Agent Client Protocol)

ACP(Agent Client Protocol)는 코딩 에이전트와 에디터/IDE 간의 통신을 표준화하는 프로토콜로, 에이전트가 에디터 내에서 파일을 편집하고, 터미널 명령을 실행하며, 코드 변경 사항을 동기화할 수 있게 합니다. ACP를 통해 에이전트는 에디터의 파일 시스템 뷰, 열린 파일 목록, 커서 위치 등의 맥락 정보를 활용하여 더 정확한 코드 수정을 수행할 수 있다.

지원 에디터

- Zed – 네이티브 통합
- JetBrains IDEs – 빌트인 지원
- VS Code – `vscode-acp` 플러그인
- Neovim – ACP 호환 플러그인

MCP vs ACP

프로토콜	용도
MCP (Model Context Protocol)	외부 도구 통합
ACP (Agent Client Protocol)	에디터-에이전트 통합

```
# ACP 서버 - canonical 패턴 (asyncio + acp.run_agent)
# pip install deepagents-acp
import asyncio

from acp import run_agent
from deepagents import create_deep_agent
from langgraph.checkpoint.memory import MemorySaver

from deepagents_acp.server import AgentServerACP

async def main() -> None:
    agent = create_deep_agent(
        model="anthropic:claude-sonnet-4-6",
        system_prompt="You are a helpful coding assistant",
        checkpoint=MemorySaver(),
    )
    server = AgentServerACP(agent)
    await run_agent(server)

if __name__ == "__main__":
    asyncio.run(main())
```

TIP

Toad CLI로 로컬에서 ACP 서버를 띄울 수 있다 — `uv tool install -U batrachian-toad` 설치 후
`toad acp "python path/to/your_server.py"` 로 실행. 프로세스의 시작·종료·재시작을 자동으로 관리해 준다.

Context Engineering — `@dynamic_prompt`

요청 시점 데이터를 시스템 프롬프트에 주입하려면 `@dynamic_prompt` 미들웨어를 사용한다.

`request.runtime.context` 와 `request.runtime.store` 에 접근할 수 있어, `context_schema` 로 선언한 `user_id` / `org_id` 가 모든 서브에이전트·도구에 자동 전파된다. 도구는 `ToolRuntime[Context]` 를 받아 `runtime.context.user_id` 로 런타임 값을 조회한다.

```
from deepagents.middleware import dynamic_prompt

@dynamic_prompt
def inject_user(request):
    user_id = request.runtime.context.user_id
    pref = request.runtime.store.get(("prefs", user_id), "lang")
    return f"\n\nUser: {user_id} / Preferred language: {pref}"
```


Permissions (deepagents ≥ 0.5.2)

`FilesystemPermission` 으로 built-in FS 도구의 읽기/쓰기 권한을 first-match-wins 방식으로 제어한다. 매치가 하나도 없으면 기본 `allow` 다. 커스텀 도구·MCP·샌드박스 `execute` 는 이 권한을 우회하므로 별도 가드가 필요하다. 서버에이전트의 `permissions` 는 부모 규칙을 전면 대체(부분 오버라이드 아님).

```
from deepagents.permissions import FilesystemPermission

permissions = [
    FilesystemPermission(operations=["write"], paths=["/policies/**"], mode="deny"),
    FilesystemPermission(operations=["read", "write"], paths=["/workspace/**"], mode="allow"),
]
```

TIP

MCP(Model Context Protocol)와 ACP의 차이를 기억하세요. MCP는 에이전트가 외부 서비스(데이터베이스, API 등)에 접근하기 위한 프로토콜이고, ACP는 에이전트가 개발 환경(에디터/IDE)과 상호작용하기 위한 프로토콜입니다. 두 프로토콜은 상호 배타적이지 아니라 보완적으로 사용됩니다. 하나의 에이전트가 MCP로 데이터베이스에 접근하면서, ACP로 에디터와 소통하는 것이 일반적인 구성이다.

미들웨어 스택 커스터마이징

Deep Agents의 에이전트 실행 파이프라인은 미들웨어 스택으로 구성되어 있다. 기본 미들웨어 (MemoryMiddleware, SkillsMiddleware, InterruptMiddleware 등) 외에 커스텀 미들웨어를 추가하여 로깅, 인증, 속도 제한, 토큰 추적 등의 횡단 관심사를 에이전트 로직과 분리할 수 있다. `create_deep_agent()` 의 `middleware` 파라미터에 미들웨어 리스트를 전달하면 된다. 미들웨어는 순서가 중요하며, 먼저 등록된 미들웨어가 먼저 실행된다. 예를 들어 인증 미들웨어를 로깅 미들웨어보다 앞에 두면, 인증 실패 시 로깅도 수행되지 않는다.

! WARNING

커스텀 미들웨어를 추가할 때 기본 미들웨어를 덮어쓰지 않도록 주의하라. `middleware` 파라미터를 지정하면 기본 미들웨어에 추가되는 방식이지만, 같은 이름의 미들웨어가 있으면 교체될 수 있다. 기본 미들웨어(Memory, Skills, Interrupt 등)를 실수로 제거하면 해당 기능이 동작하지 않는다.

SDK로 에이전트를 프로그래밍 방식으로 구축하는 것 외에, Deep Agents는 터미널에서 바로 사용할 수 있는 CLI도 제공한다. 다음 섹션에서 CLI의 설치와 사용법을 알아본다.

5. Deep Agents CLI

SDK로 에이전트를 프로그래밍 방식으로 구축하는 것 외에, Deep Agents는 SDK 위에 구축된 터미널 코딩 에이전트 CLI도 제공합니다. 별도의 코드 작성 없이 터미널에서 즉시 사용할 수 있습니다. CLI는

`create_deep_agent()` 로 만든 에이전트와 동일한 기능(도구, 백엔드, 서버에이전트, 메모리, 스킬, 샌드박스)을 모두 지원하며, 프로젝트 루트의 `.deepagents/AGENTS.md` 파일을 자동으로 인식한다.

설치 및 실행

```
# 설치
uv tool install deepagents-cli

# 실행
deepagents-cli

# 직접 실행 (설치 없이)
uvx deepagents-cli
```

주요 옵션

옵션	설명
<code>-a/--agent AGENT</code>	에이전트 이름 지정
<code>-M/--model MODEL</code>	모델 선택
<code>-n/--non-interactive</code>	비대화형 모드 (단일 태스크 실행)
<code>--auto-approve</code>	인간 확인 스킵
<code>--sandbox {none,modal,daytona,runloop}</code>	샌드박스 백엔드 선택

인터랙티브 명령어

명령	설명
<code>/model</code>	모델 변경
<code>/remember</code>	메모리에 정보 저장
<code>/tokens</code>	토큰 사용량 확인
<code>!command</code>	셸 명령 실행

메모리 시스템

CLI의 메모리 시스템은 두 가지 계층으로 동작한다. 글로벌 메모리는 모든 프로젝트에서 공유되며, 프로젝트 메모리는 해당 프로젝트에서만 적용된다.

- 글로벌: `~/.deepagents/<agent_name>/memories/` - 사용자 선호도, 공통 설정
- 프로젝트: `.deepagents/AGENTS.md` (프로젝트 루트) - 프로젝트 컨벤션, 아키텍처 결정

! WARNING

CLI에서 `--auto-approve` 옵션을 사용하면 모든 도구 호출이 승인 없이 실행된다. 신뢰할 수 있는 환경에서만 사용하고, 프로덕션 서버에서는 절대 사용하지 마라. 실수로 위험한 명령이 실행될 수 있다.

```
# CLI 비대화형 모드 예시 (셸에서 실행)
cli_examples = """
```

```
# 기본 사용
deepagents-cli

# 특정 모델로 비대화형 실행
deepagents-cli -M claude-sonnet-4-6 -n "이 프로젝트의 README.md를 작성해 줘"

# 샌드박스에서 실행
deepagents-cli --sandbox modal "테스트 코드를 실행해 줘"

# 스킬 관리
deepagents-cli skills list
deepagents-cli skills create my-skill
"""

print("CLI 사용 예시 (터미널에서 실행):")
print(cli_examples)
```

CLI 사용 예시 (터미널에서 실행):

```
# 기본 사용
deepagents-cli

# 특정 모델로 비대화형 실행
deepagents-cli -M claude-sonnet-4-6 -n "이 프로젝트의 README.md를 작성해 줘"

# 샌드박스에서 실행
deepagents-cli --sandbox modal "테스트 코드를 실행해 줘"

# 스킬 관리
deepagents-cli skills list
deepagents-cli skills create my-skill
```

전체 교육 자료 정리

노트북	주제	핵심 API
01	소개	<code>deepagents.__version__</code>
02	퀵스타트	<code>create_deep_agent()</code> , <code>invoke()</code> , <code>stream()</code>
03	커스터마이징	<code>model</code> , <code>system_prompt</code> , <code>tools</code> , <code>response_format</code>
04	백엔드	<code>StateBackend</code> , <code>FilesystemBackend</code> , <code>StoreBackend</code> , <code>CompositeBackend</code>
05	서브에이전트	<code>SubAgent</code> , <code>CompiledSubAgent</code> , <code>subagents</code>
06	메모리 & 스킬	<code>memory</code> , <code>skills</code> , <code>AGENTS.md</code> , <code>SKILL.md</code>
07	고급 기능	<code>interrupt_on</code> , <code>stream_mode</code> , <code>Sandbox</code> , <code>ACP</code> , <code>CLI</code>

이 장에서 다룬 고급 기능들(HITL, 스트리밍, 샌드박스, ACP, CLI)은 에이전트를 프로덕션 환경에서 안전하게 운영하기 위한 필수 요소입니다. 다음 장에서는 `create_deep_agent()` 내부에서 이 모든 기능을 조립하는 AgentHarness의 구조를 분석합니다.

08 에이전트 하네스

`AgentHarness` 는 Deep Agents의 핵심 설계 철학을 구현한 포괄적 기능 제공자로, 장기 실행 자율 에이전트에 필요한 계획, 파일시스템, 태스크 위임, 컨텍스트 관리, 코드 실행, Human-in-the-Loop을 하나로 통합한다. 이 장에서는 `create_deep_agent()` 가 내부적으로 이 모든 기능을 어떻게 조립하는지 분해하여 살펴보고, 각 구성 요소의 역할과 설정 방법을 정리한다.

이전 장들에서 개별적으로 다룬 기능들(2장의 에이전트 생성, 3장의 커스터마이징, 4장의 백엔드, 5장의 서브 에이전트, 6장의 메모리/스킬, 7장의 고급 기능)이 내부적으로 어떻게 하나의 에이전트로 조립되는지를 이해하면, Deep Agents의 설계 원칙을 깊이 있게 파악할 수 있다. `create_deep_agent()` 는 사실 `AgentHarness` 를 간편하게 사용하기 위한 편의 래퍼(convenience wrapper)다.

하네스라는 이름은 말(馬)의 마구(harness)에서 유래한 비유다. 마구가 고삐, 안장, 등자 등 여러 부품을 하나로 결합하여 기수에게 통일된 인터페이스를 제공하듯, `AgentHarness` 는 모델, 도구, 미들웨어, 상태 관리라는 개별 부품을 결합하여 개발자에게 하나의 일관된 에이전트 인터페이스를 제공한다. 이 장을 마치면, 하네스가 내부적으로 수행하는 수집 → 빌드 → 적용 → 컴파일 파이프라인의 전체 흐름을 이해할 수 있다.

학습 목표

LEARNING OBJECTIVES

- `AgentHarness`의 개념과 역할을 이해한다
- 하네스의 핵심 기능(계획, 파일시스템, 태스크 위임)을 안다
- 컨텍스트 관리(오프로딩, 요약)를 이해한다
- 코드 실행과 Human-in-the-Loop을 설정한다
- 스킬과 메모리 시스템을 연동한다

먼저 환경 변수를 로드하고 모델을 초기화한다. 이후 각 섹션에서 하네스의 구성 요소를 하나씩 분해하여 살펴본다.

```
# 환경 설정
from dotenv import load_dotenv
import os

load_dotenv()
assert os.environ.get("OPENAI_API_KEY"), "OPENAI_API_KEY가 설정되지 않았습니다!"
print("환경 설정 완료")
```

환경 설정 완료

```
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-5.4")

print(f"모델 설정 완료: {model.model_name}")
```

모델 설정 완료: gpt-5.4

1. AgentHarness 개념

AgentHarness는 장기 실행 자율 에이전트를 위한 포괄적 기능 제공자입니다. 에이전트가 복잡한 멀티 스텝 작업을 수행할 때 필요한 모든 인프라를 하나로 묶어 제공합니다. 내부적으로 하네스는 다음 단계를 순서대로 수행합니다:

1. 모델, 도구, 미들웨어, 상태 스키마를 수집합니다.
2. 수집된 구성 요소로 `StateGraph` 를 빌드합니다.
3. 미들웨어 파이프라인을 적용합니다.
4. 최종적으로 그래프를 컴파일하여 `CompiledStateGraph` 를 반환합니다.

하네스가 제공하는 핵심 기능

기능	설명
Planning	구조화된 태스크 리스트 관리 (<code>write_todos</code>)
Filesystem	가상/로컬 파일 읽기, 쓰기, 검색
Task Delegation	서브에이전트를 통한 작업 위임
Context Management	오프로딩 및 요약물 통한 컨텍스트 압축
Code Execution	샌드박스 환경에서 안전한 코드 실행
Human-in-the-Loop	민감 작업에 대한 사람 승인
Skills & Memory	전문 워크플로와 영속적 지식

`create_deep_agent()` 를 호출하면 이 모든 기능이 자동으로 조립되어 하나의 에이전트로 제공됩니다. 개발자는 각 기능을 개별적으로 설정할 필요 없이, `create_deep_agent()` 의 파라미터만으로 원하는 조합을 선언적으로 지정할 수 있습니다.

하네스 내부의 동작 순서를 요약하면 다음과 같습니다:

1. **수집(Collect)** – 모델, 커스텀 도구, 미들웨어, 상태 스키마를 파라미터로부터 수집합니다.
2. **빌드(Build)** – 수집된 구성 요소를 LangGraph의 `StateGraph` 로 조립합니다. 이 단계에서 노드(에이전트 로직)와 엣지(도구 호출 라우팅)가 정의됩니다.
3. **적용(Apply)** – `TodoListMiddleware` , `SummarizationMiddleware` 등 미들웨어 파이프라인을 그래프에 적용합니다.
4. **컴파일(Compile)** – 최종적으로 `CompiledStateGraph` 를 생성하여 실행 가능한 에이전트를 반환합니다.

이 4단계 파이프라인 덕분에, 개발자는 무엇을 사용할지만 선언하고 어떻게 조립할지는 하네스에 위임할 수 있습니다.

TIP

`create_deep_agent()` 는 `AgentHarness` 의 편의 래퍼입니다. 대부분의 경우 이 래퍼만으로 충분하며, 하네스를 직접 다룰 필요는 없습니다. 하지만 내부 구조를 이해하면 문제 해결과 고급 커스터마이징에 큰 도움이 됩니다.

! WARNING

하네스의 조립 순서는 중요합니다. 미들웨어는 반드시 `StateGraph` 빌드 이후에 적용되어야 하며, 미들웨어 간에도 의존 관계가 있을 수 있습니다. 예를 들어 `SummarizationMiddleware` 는 파일시스템 백엔드가 설정된 후에야 오 프로딩된 콘텐츠를 저장할 수 있습니다.

다음 코드는 `create_deep_agent()` 에 전달되는 하네스 구성 요소를 파이썬 딕셔너리로 정리한 것입니다. 각 키가 하네스의 어떤 기능을 활성화하는지 확인하세요.

```
# AgentHarness 개념 - create_deep_agent가 하네스를 조립합니다
harness_config = {
    "model": "openai:gpt-5.4",
    "system_prompt": "당신은 프로젝트 관리 어시스턴트입니다.",
    "planning": True,          # write_todos 도구 활성화
    "filesystem": True,       # 파일시스템 도구 활성화
    "subagents": [],          # 서브에이전트 목록
    "context_management": True, # 컨텍스트 압축 활성화
}

print("AgentHarness 구성 요소:")
for key, value in harness_config.items():
    print(f" {key}: {value}")
```

```
AgentHarness 구성 요소:
model: openai:gpt-5.4
system_prompt: 당신은 프로젝트 관리 어시스턴트입니다.
planning: True
filesystem: True
subagents: []
context_management: True
```

2. 계획 도구

하네스의 구성 요소를 이해했으니, 이제 각 기능을 하나씩 상세히 살펴보겠습니다. 첫 번째는 태스크 계획입니다. 하네스의 첫 번째 구성 요소는 태스크 계획입니다. `ToDoListMiddleware` 가 자동으로 추가하는 `write_todos` 도구를 통해, 에이전트는 복잡한 작업을 구조화된 태스크 리스트로 분해합니다. `read_todos` 도구로 현재 진행 상황을 확인할 수도 있습니다.

태스크 계획은 단순한 편의 기능이 아닙니다. 장기 실행 에이전트가 수십 단계의 작업을 수행할 때, 현재 진행 상황을 추적하지 않으면 동일한 작업을 반복하거나 중요한 단계를 건너뛸 수 있습니다. `write_todos` 는 에이전트에

게 구조화된 자기 관리 능력을 부여합니다. 또한 요약(Summarization) 단계에서도 태스크 리스트가 컨텍스트 복원의 기준점 역할을 합니다.

각 태스크는 상태를 가집니다:

상태	설명
pending	아직 시작하지 않음
in_progress	현재 진행 중
completed	완료됨

아래 예시는 에이전트가 생성할 수 있는 태스크 리스트의 형태를 보여줍니다. 실제 실행에서는 에이전트가 자율적으로 `write_todos` 를 호출하여 태스크를 생성하고, 각 태스크를 완료할 때마다 상태를 업데이트합니다.

```
# write_todos 도구 - 구조화된 태스크 리스트 예시
todo_list = [
    {"task": "프로젝트 구조 분석", "status": "completed"},
    {"task": "API 엔드포인트 설계", "status": "in_progress"},
    {"task": "데이터베이스 스키마 작성", "status": "pending"},
    {"task": "테스트 코드 작성", "status": "pending"},
    {"task": "문서화", "status": "pending"},
]

print("=== 에이전트 태스크 리스트 ===")
for i, item in enumerate(todo_list, 1):
    icon = {"completed": "[x]", "in_progress": "[-]", "pending": "[ ]"}
    print(f" {icon[item['status']]} {i}. {item['task']}")
```

```
=== 에이전트 태스크 리스트 ===
[x] 1. 프로젝트 구조 분석
[-] 2. API 엔드포인트 설계
[ ] 3. 데이터베이스 스키마 작성
[ ] 4. 테스트 코드 작성
[ ] 5. 문서화
```

TIP

에이전트가 태스크 리스트를 효과적으로 활용하도록 하려면, 시스템 프롬프트에 “복잡한 작업을 시작할 때 먼저 `write_todos` 로 계획을 세우세요”와 같은 지침을 포함하는 것이 좋습니다. 계획 수립 없이 바로 실행에 들어가면 장기 작업에서 일관성이 떨어질 수 있습니다.

3. 가상 파일시스템

계획이 에이전트의 사고 구조라면, 파일시스템은 에이전트의 작업 공간입니다. 이제 에이전트가 파일을 읽고, 쓰고, 검색하는 방법을 살펴봅니다.

하네스는 구성 가능한 파일시스템 백엔드를 통해 표준 파일 작업을 지원합니다. 여기서 핵심 설계 원칙은 플러거블 백엔드입니다. 4장에서 학습한 `BackendProtocol` 을 구현하는 어떤 백엔드든(로컬 파일시스템, 인메모리, 샌드박스) 동일한 도구 인터페이스로 접근할 수 있습니다. 에이전트의 코드를 변경하지 않고도, `backend` 파라미터만 교체하면 로컬 개발에서 샌드박스 프로덕션으로 전환할 수 있습니다.

도구	설명
ls	디렉토리 목록 (메타데이터 포함)
read_file	파일 내용 읽기 (줄 번호 포함, 멀티모달 – 이미지 PNG/JPG/GIF/WebP/HEIC, 비디오 MP4/MOV/AVI, 오디오 WAV/MP3/AAC/FLAC, 문서 PDF/PPT)
write_file	파일 생성
edit_file	문자열 치환 편집
glob	패턴 기반 파일 검색
grep	내용 검색 (여러 출력 모드)
execute	셸 명령 실행 (샌드박스 백엔드 전용)

`read_file` 은 이미지 파일(PNG, JPG, GIF, WEBP)도 지원하므로, 에이전트가 스크린샷이나 차트를 분석하는 작업에도 활용할 수 있습니다. `execute` 는 샌드박스 백엔드에서만 노출되며, 10장에서 자세히 다룹니다. 다음 코드는 각 파일시스템 도구의 호출 형태를 정리합니다.

```
# 파일시스템 도구 사용 예시 (참고용)
fs_operations = {
    "ls": 'ls(path="/project/src")',
    "read_file": 'read_file(path="/project/src/main.py")',
    "write_file": 'write_file(path="/project/config.yaml", content="debug: true")',
    "edit_file": 'edit_file(path="/project/src/main.py", old="v1", new="v2")',
    "glob": 'glob(pattern="**/*.py")',
    "grep": 'grep(pattern="TODO", path="/project/src")',
}

print("=== 파일시스템 도구 호출 예시 ===")
for tool_name, call_example in fs_operations.items():
    print(f" {tool_name:12s} -> {call_example}")
```

```
=== 파일시스템 도구 호출 예시 ===
ls                -> ls(path="/project/src")
read_file         -> read_file(path="/project/src/main.py")
write_file        -> write_file(path="/project/config.yaml", content="debug: true")
edit_file         -> edit_file(path="/project/src/main.py", old="v1", new="v2")
glob              -> glob(pattern="**/*.py")
grep              -> grep(pattern="TODO", path="/project/src")
```

4. 태스크 위임 — 서브에이전트

파일시스템으로 에이전트의 작업 공간을 확보했으니, 이제 에이전트가 작업을 분업하는 방법을 살펴봅니다.

하네스는 메인 에이전트가 임시 서브에이전트(ephemeral subagent)를 생성하여 격리된 멀티 스텝 태스크를 수행할 수 있게 합니다. 서브에이전트는 메인 에이전트의 컨텍스트와 완전히 분리된 독립적인 에이전트로, 자신만의 시스템 프롬프트, 도구 세트, 컨텍스트를 가집니다. 작업이 완료되면 결과만 압축하여 메인 에이전트에 반환하므로, 메인 에이전트의 컨텍스트 윈도우를 효율적으로 보존합니다.

서브에이전트의 장점

장점	설명
컨텍스트 격리	서브에이전트 실행이 메인 컨텍스트를 오염시키지 않음
병렬 실행	여러 서브에이전트를 동시에 실행 가능
전문화	각 서브에이전트에 특화된 도구와 프롬프트 제공
토큰 효율	결과 압축으로 메인 에이전트의 토큰 절약

· NOTE

서브에이전트를 과도하게 사용하면 오버헤드가 증가합니다. 단순한 단일 도구 호출(예: 파일 하나 읽기)은 서브 에이전트 없이 메인 에이전트가 직접 수행하는 것이 효율적입니다. 서브에이전트는 여러 단계의 조사나 코드 작성처럼 자체적인 계획과 반복이 필요한 작업에 사용하세요.

다음 예시는 조사 담당(researcher)과 코드 작성 담당(coder) 두 서브에이전트를 구성하는 방법을 보여줍니다. 각 서브에이전트에 특화된 도구와 프롬프트를 부여하는 점에 주목하세요.

```
# 서브에이전트 위임 구성 예시 (참고용)
subagent_config = [
    {
        "name": "researcher",
        "description": "인터넷 검색으로 정보를 조사합니다.",
        "system_prompt": "검색 결과를 간결하게 요약하세요.",
        "tools": ["internet_search"],
    },
    {
        "name": "coder",
        "description": "코드를 작성하고 테스트합니다.",
        "system_prompt": "깔끔하고 테스트 가능한 코드를 작성하세요.",
        "tools": ["write_file", "execute"],
    },
]

print("=== 서브에이전트 구성 ===")
for sa in subagent_config:
    print(f" [{sa['name']}] {sa['description']}")
    print(f" 도구: {' '.join(sa['tools'])}")
```

```
=== 서브에이전트 구성 ===
[researcher] 인터넷 검색으로 정보를 조사합니다.
도구: internet_search
[coder] 코드를 작성하고 테스트합니다.
도구: write_file, execute
```

5. 컨텍스트 관리

계획, 파일시스템, 서브에이전트까지 갖추면 에이전트는 매우 강력해지지만, 동시에 새로운 문제가 등장합니다. 바로 컨텍스트 폭발입니다.

계획, 파일시스템, 서브에이전트가 모두 동작하면서 에이전트의 컨텍스트는 빠르게 채워집니다. 장기 실행 에이전트의 가장 큰 과제인 컨텍스트 윈도우 한계를 하네스는 두 가지 자동 기법으로 해결합니다. 개발자가 별도로 설정할 필요 없이, `SummarizationMiddleware` 가 이를 자동으로 관리합니다. 이 미들웨어가 없다면, 장기 실행 에이전트는 컨텍스트 윈도우를 초과하여 오류가 발생하거나, 초기 지시사항을 잊어버리는 문제가 발생합니다.

입력 컨텍스트 조립

시스템 프롬프트, 지침, 메모리 가이드라인, 스킬 정보, 파일시스템 문서를 종합하여 초기 프롬프트를 구성합니다.

런타임 컨텍스트 압축

기법	동작	트리거
오프로딩	configurable threshold (기본 20,000 토큰) 초과 콘텐츠를 디스크에 저장, 포인터 참조 유지	콘텐츠 크기 기준
요약	대화 히스토리를 구조화된 요약(session intent / artifacts / next steps)으로 압축. 모델 <code>max_input_tokens</code> 의 약 85% 도달 시 트리거하며, 최근 10% 메시지는 그대로 보존. <code>max_input_tokens</code> 미설정 시 170k 토큰 fallback	모델 윈도우 한계 접근 시

원본 메시지는 파일시스템 스토리지에 보존되므로 정보 손실이 없습니다. 에이전트가 요약된 정보에서 원본을 다시 참조해야 할 경우, `read_file` 도구로 접근할 수 있습니다. 이는 손실 없는 압축이라는 하네스의 핵심 설계 원칙을 보여줍니다.

· NOTE

오프로딩의 20,000 토큰 임계값과 요약의 85% 윈도우 트리거는 합리적인 기본값이지만, 모델과 작업 특성에 따라 조정이 필요할 수 있습니다. 예를 들어, 대용량 코드 분석 작업에서는 오프로딩 임계값을 낮추고, 간단한 대화형 작업에서는 높여도 됩니다.

! WARNING

요약 과정에서 에이전트의 초기 시스템 프롬프트와 핵심 지시사항은 항상 보존됩니다. 그러나 대화 중간에 사용자가 추가한 임시 지시사항은 요약 시 압축될 수 있습니다. 중요한 지시사항은 시스템 프롬프트에 포함하거나 `AGENTS.md` 메모리에 기록하세요.

다음 코드는 오프로딩과 요약의 설정 옵션을 정리한 것입니다. 실제로는 하네스가 이 설정을 자동으로 관리하지만, 미세 조정이 필요한 경우 참고할 수 있습니다.

```
# 컨텍스트 관리 설정 예시 (참고용)
context_config = {
  "offloading": {
    "enabled": True,
    "threshold_tokens": 20000,
    "storage": "filesystem",
  },
  "summarization": {
    "enabled": True,
    "trigger": "window_limit_approach",
  }
}
```

```

        "preserve_original": True,
    },
}

print("=== 컨텍스트 관리 설정 ===")
for section, settings in context_config.items():
    print(f"\n[{section}]")
    for key, value in settings.items():
        print(f"    {key}: {value}")

```

```

=== 컨텍스트 관리 설정 ===

[offloading]
enabled: True
threshold_tokens: 20000
storage: filesystem

[summarization]
enabled: True
trigger: window_limit_approach
preserve_original: True

```

6. 코드 실행

컨텍스트 관리로 에이전트의 장기 실행을 보장했으니, 이제 에이전트가 실제로 코드를 실행하는 방법을 살펴봅시다.

샌드박스 백엔드는 `execute` 도구를 노출하여 격리된 환경에서 명령을 실행합니다. 호스트 시스템에 영향을 주지 않으면서 보안성, 깨끗한 환경, 재현성을 제공합니다. `execute` 도구는 샌드박스 백엔드(Modal, Daytona, Runloop 등)가 설정된 경우에만 사용할 수 있습니다. 로컬 파일시스템 백엔드에서는 보안상 이 도구가 노출되지 않습니다.

코드 실행은 에이전트가 작성한 코드를 검증하거나, 패키지를 설치하거나, 테스트를 수행할 때 사용됩니다. 다음은 `execute` 도구로 수행할 수 있는 대표적인 작업입니다.

```

# 샌드박스 코드 실행 예시 (참고용)
execute_examples = [
    {"command": "python -c 'print(2+2)'", "desc": "Python 코드 실행"},
    {"command": "pip install requests", "desc": "패키지 설치"},
    {"command": "pytest tests/", "desc": "테스트 실행"},
]

print("=== 샌드박스 execute 도구 예시 ===")
for ex in execute_examples:
    print(f"    $ {ex['command']}")
    print(f"    -> {ex['desc']}")

```

```

=== 샌드박스 execute 도구 예시 ===
$ python -c 'print(2+2)'
-> Python 코드 실행
$ pip install requests
-> 패키지 설치
$ pytest tests/
-> 테스트 실행

```



TIP

코드 실행의 보안과 샌드박스 프로바이더 선택에 대한 심화 내용은 10장 “샌드박스와 ACP”에서 다룹니다. 이 장에서는 하네스 관점에서 `execute` 도구의 역할만 이해하면 충분합니다.

두 갈래 실행: Sandbox `execute` vs QuickJS `CodeInterpreterMiddleware`

하네스는 코드 실행을 두 갈래로 노출한다. 하나는 샌드박스 백엔드가 노출하는 셀 `execute` 도구, 다른 하나는 interpreter 미들웨어가 노출하는 QuickJS eval 도구다. 둘은 서로 다른 문제를 푼다.

항목	Sandbox <code>execute</code> (셀)	QuickJS <code>CodeInterpreterMiddleware</code>
실행 위치	에이전트 외부의 격리 환경 (Modal/Daytona/Runloop 등)	에이전트 루프 내부의 QuickJS VM
언어	셀 명령 → 임의 언어 (Python, Node, ...)	JavaScript (QuickJS)
네트워크/파일시스템	기본 활성화 (정책으로 제어)	기본 비활성 – bridge로 노출된 도구만
패키지 설치	<code>pip install</code> / <code>npm install</code> 가능	불가능
적합한 용도	패키지 설치·테스트 실행·대용량 데이터 처리	도구 호출 조합·서브에이전트 fan-out·구조화 데이터 변환
활성화	<code>backend=ModalSandbox(...)</code> 등 백엔드 교체	<code>middleware=[CodeInterpreterMiddleware(...)]</code> 추가

7. Human-in-the-Loop

자율 에이전트가 파일을 수정하고 코드를 실행할 수 있다면, 안전 장치가 필수입니다. Human-in-the-Loop(HITL)은 에이전트의 자율성과 안전성 사이의 균형을 제공합니다.

선택적 인터럽트 설정으로 지정된 도구 호출 시 사람의 승인을 요구합니다. 에이전트가 해당 도구를 호출하면 실행이 일시 중단되고, 사용자가 승인(approve), 거부(reject), 또는 수정(edit)을 선택할 수 있습니다. 이 기능은 LangGraph의 `interrupt` 메커니즘 위에 구현되므로, 중단된 시점의 상태가 완전히 보존됩니다.

다음 코드는 파일 쓰기, 편집, 코드 실행에 대해 승인을 요구하도록 설정하는 예시입니다.

```
# Human-in-the-Loop 설정 예시 (참고용)
hitl_config = {
    "interrupt_on": {
        "write_file": True, # 파일 쓰기 전 승인
        "edit_file": True,  # 파일 편집 전 승인
        "execute": True,    # 명령 실행 전 승인
    }
}

print("=== Human-in-the-Loop 설정 ===")
print("승인이 필요한 도구:")
for tool, enabled in hitl_config["interrupt_on"].items():
    status = "승인 필요" if enabled else "자동 실행"
```

```
print(f" {tool}: {status}")

print("\n승인 옵션: approve(승인), reject(거부), edit(수정)")
```

```
=== Human-in-the-Loop 설정 ===
승인이 필요한 도구:
  write_file: 승인 필요
  edit_file: 승인 필요
  execute: 승인 필요

승인 옵션: approve(승인), reject(거부), edit(수정)
```

8. 스킬과 메모리

지금까지 살펴본 기능들(계획, 파일시스템, 서브에이전트, 컨텍스트 관리, 코드 실행, HITL)이 에이전트의 런타임 능력이라면, 스킬과 메모리는 에이전트의 축적된 지식입니다. 이 두 시스템은 에이전트가 대화를 넘어서 전문성과 경험을 유지하게 합니다.

스킬 (Skills)

Agent Skills 표준을 따르는 전문 워크플로입니다. 관련성이 있을 때 점진적으로 로드되어 토큰 소비를 줄입니다. 모든 스킬을 항상 로드하면 컨텍스트를 낭비하므로, 하네스는 사용자의 요청과 스킬의 트리거 조건을 매칭하여 필요한 스킬만 활성화합니다.

- 각 스킬은 `SKILL.md` 파일로 정의
- 트리거 조건에 따라 자동 활성화
- 도구, 프롬프트, 워크플로를 캡슐화

메모리 (Memory)

AGENTS.md 형식의 영속적 컨텍스트 파일입니다. 대화를 넘어서 재사용 가능한 가이드라인, 선호도, 프로젝트 지식을 제공합니다. 메모리는 글로벌 범위와 프로젝트 범위 두 가지로 구분되며, 에이전트가 새 대화를 시작할 때 자동으로 로드됩니다.

구분	위치	범위
글로벌 메모리	~/.deepagents/<agent>/memories/	모든 프로젝트
프로젝트 메모리	.deepagents/AGENTS.md	현재 프로젝트

· NOTE

스킬과 메모리는 서로 다른 목적을 가집니다. 스킬은 특정 작업을 수행하기 위한 도구와 워크플로의 묶음(how to do)이고, 메모리는 프로젝트의 규칙과 맥락 정보(what to know)입니다. 예를 들어, “코드 리뷰” 스킬은 리뷰 절차를 정의하고, `AGENTS.md` 메모리는 “이 프로젝트는 TypeScript를 사용하고, 테스트 커버리지 80% 이상을 유지한다”와 같은 규칙을 저장합니다.

다음 코드는 스킬과 메모리의 설정 구조를 보여줍니다.

```
# 스킬과 메모리 설정 예시 (참고용)
skills_config = [
    {"name": "code-review", "trigger": "코드 리뷰 요청 시"},
    {"name": "test-writer", "trigger": "테스트 작성 요청 시"},
    {"name": "doc-generator", "trigger": "문서화 요청 시"},
]

memory_config = {
    "global": "~/.deepagents/my-agent/memories/",
    "project": ".deepagents/AGENTS.md",
}

print("=== 스킬 설정 ===")
for skill in skills_config:
    print(f" [{skill['name']}] 트리거: {skill['trigger']}")

print("\n=== 메모리 설정 ===")
for scope, path in memory_config.items():
    print(f" {scope}: {path}")
```

```
=== 스킬 설정 ===
[code-review] 트리거: 코드 리뷰 요청 시
[test-writer] 트리거: 테스트 작성 요청 시
[doc-generator] 트리거: 문서화 요청 시

=== 메모리 설정 ===
global: ~/.deepagents/my-agent/memories/
project: .deepagents/AGENTS.md
```

9. Harness Profiles

`HarnessProfile` 은 `create_deep_agent()` 호출부를 바꾸지 않고도 provider/model별 harness 기본값을 패킹한다. 시스템 프롬프트, 도구 설명 override, 제외할 도구/미들웨어, 일반 목적 서브에이전트 설정, 추가 미들웨어를 profile로 등록할 수 있다.

`HarnessProfile` 7필드

필드	의미
<code>base_system_prompt</code>	베이스 system prompt 자체를 교체
<code>system_prompt_suffix</code>	조립된 베이스 prompt 끝에 텍스트 추가
<code>tool_description_overrides</code>	도구 이름별 설명 override (mapping)
<code>excluded_tools</code>	주입 이후 이름으로 제거할 도구 집합 (set)
<code>excluded_middleware</code>	제거할 middleware 클래스 집합 (set)
<code>extra_middleware</code>	추가로 붙일 middleware 인스턴스 리스트
<code>general_purpose_subagent</code>	<code>GeneralPurposeSubagentProfile</code> 로 일반 목적 서브에이전트 활성화·비활성·커스터마이징

Merge semantics

여러 profile이 매칭될 때의 병합 규칙은 다음과 같다.

- 후행 profile이 선행 profile의 scalar 필드(예: `base_system_prompt`)를 덮어쓴다.
- `tool_description_overrides` 같은 mapping류는 키 단위로 merge.
- `excluded_tools` / `excluded_middleware` 같은 set류는 합집합.

ProviderProfile 3필드

`ProviderProfile` 은 harness 동작이 아니라 `init_chat_model()` 에 넘길 초기화 인자를 다룬다. credential check, provider별 기본 temperature, runtime header 구성처럼 모델 생성 전후 기본값을 묶을 때 사용한다.

필드	의미
<code>init_kwargs</code>	<code>init_chat_model()</code> 호출 시 전달할 기본 kwargs (예: <code>temperature</code>)
<code>credentials_check</code>	모델 초기화 전에 실행하는 자격 증명 확인 callable
<code>runtime_headers</code>	요청마다 첨부할 HTTP 헤더 매핑

YAML/JSON 워크플로 — `HarnessProfileConfig.from_dict`

`HarnessProfileConfig` 는 `from_dict()` , `to_dict()` , `from_harness_profile()` 클래스 메서드를 제공한다. YAML 로 profile을 관리해 코드를 건드리지 않고 운영 환경별로 다른 설정을 적용할 수 있다.

```
# profile.yaml
base_system_prompt: You are helpful.
system_prompt_suffix: Respond briefly.
excluded_tools:
  - execute
excluded_middleware:
  - SummarizationMiddleware
general_purpose_subagent:
  enabled: false
```

```
import yaml
from deepagents import HarnessProfileConfig, register_harness_profile

with open("profile.yaml") as f:
    register_harness_profile(
        "openai:gpt-5.4",
        HarnessProfileConfig.from_dict(yaml.safe_load(f)),
    )
```

Entry-point 플러그인

`pyproject.toml` 의 entry-point로 profile을 패키지처럼 배포할 수 있다. 로드 순서는 built-ins → entry-point plugins → 사용자 코드의 직접 `register_*_profile` 호출 순이다.

```
[project.entry-points."deepagents.harness_profiles"]
"anthropic:claude-sonnet-4-6" = "my_pkg.profiles:claude_profile"

[project.entry-points."deepagents.provider_profiles"]
"openai" = "my_pkg.profiles:openai_provider"
```

요약

주제	핵심 개념	핵심 API/도구
하네스 개념	장기 실행 에이전트를 위한 포괄적 기능 제 공자	<code>create_deep_agent()</code>
계획 도구	구조화된 태스크 리스트 관리	<code>write_todos</code>
파일시스템	가상/로컬 파일 작업 (7종 도구)	<code>ls</code> , <code>read_file</code> , <code>write_file</code> , <code>edit_file</code> , <code>glob</code> , <code>grep</code> , <code>execute</code>
서브에이전트	격리된 태스크 위임, 병렬 실행	<code>subagents</code> , <code>task</code>
컨텍스트 관리	오프로딩(20K 토큰), 요약 압축	자동 관리
코드 실행	샌드박스에서 안전한 명령 실행	<code>execute</code>
HITL	민감 도구 호출 시 사람 승인	<code>interrupt_on</code>
스킬/메모리	전문 워크플로 + 영속적 컨텍스트	<code>SKILL.md</code> , <code>AGENTS.md</code>

REFERENCES

- [Deep Agents Harness](#)

09 외부 프레임워크 비교

AI 에이전트 프레임워크는 빠르게 진화하고 있으며, 프로젝트 요구사항에 맞는 프레임워크를 선택하는 것이 성공의 핵심이다. 이 장에서는 Deep Agents를 OpenCode, Claude Agent SDK와 모델 지원, 아키텍처, 생태계, 라이선스 측면에서 비교 분석한다. 사용 사례별 추천과 마이그레이션 고려사항을 함께 정리하여 실무 의사결정에 도움을 준다.

8장까지 Deep Agents의 내부 구조를 깊이 있게 탐구했다. 이제 한 발 물러서서, Deep Agents가 에이전트 프레임워크 생태계에서 어떤 위치를 차지하는지 객관적으로 평가한다. 각 프레임워크는 고유한 설계 철학과 강점을 가지므로, 절대적인 우열보다는 프로젝트 요구사항에 맞는 선택이 중요하다.

이 비교에서 다루는 세 프레임워크는 서로 다른 설계 축을 대표한다. Deep Agents는 모델 무관 + 플러그블 아키텍처, OpenCode는 터미널 네이티브 + 로컬 모델, Claude Agent SDK는 단일 모델 최적화 + 간결한 API를 지향한다. 이 장을 통해 각 프레임워크의 장단점을 균형 있게 이해하고, 프로젝트의 규모, 모델 전략, 배포 환경에 따른 합리적인 선택 기준을 세울 수 있다.

학습 목표

LEARNING OBJECTIVES

- Deep Agents, LangGraph, LangChain의 심화 차이를 이해한다
- OpenCode, Claude Agent SDK와 비교한다
- 아키텍처, 유연성, 생태계를 비교 분석한다
- 사용 사례별 추천 프레임워크를 안다
- 마이그레이션 고려사항을 이해한다

```
# 환경 설정
from dotenv import load_dotenv
import os

load_dotenv()
assert os.environ.get("OPENAI_API_KEY"), "OPENAI_API_KEY가 설정되지 않았습니다!"
print("환경 설정 완료")
```

환경 설정 완료

```
from langchain_openai import ChatOpenAI
```

```
model = ChatOpenAI(model="gpt-5.4")
print(f"모델 설정 완료: {model.model_name}")
```

모델 설정 완료: gpt-5.4

1. 비교 개요

AI 에이전트 프레임워크를 선택할 때는 모델 지원, 아키텍처, 생태계, 라이선스 등 여러 요소를 고려해야 합니다. 프레임워크를 선택하는 것은 단기적인 개발 속도뿐 아니라, 장기적인 유지보수성, 확장성, 벤더 종속(vendor lock-in) 위험에도 직접적인 영향을 미칩니다.

이 장에서는 세 가지 주요 프레임워크를 비교합니다:

- LangChain Deep Agents – 모델 무관(model-agnostic) 에이전트 하네스. LangGraph 기반의 그래프 실행 엔진과 플러그블 백엔드를 결합하여 프로덕션급 에이전트 앱을 구축합니다.
- OpenCode – 터미널/데스크톱/IDE 기반 코딩 에이전트. Go로 작성되어 가볍고 빠르며, Ollama를 통한 로컬 모델 지원이 특징입니다.
- Claude Agent SDK – Anthropic의 Claude 전용 에이전트 SDK. Claude 모델에 최적화된 간결한 API로 빠른 프로토타이핑에 적합합니다.

· NOTE

이 비교는 2026년 초 기준이며, 각 프레임워크는 활발하게 개발 중입니다. 특정 기능의 지원 여부는 최신 릴리즈 노트를 확인하세요. 여기서는 설계 철학과 아키텍처 차이에 초점을 맞추어, 시간이 지나도 유효한 비교 기준을 제공합니다.

2. Deep Agents vs OpenCode vs Claude Agent SDK

먼저 세 프레임워크의 기본 스펙을 비교표로 정리합니다. 이 표는 프로젝트 초기에 후보를 좁히는 데 유용합니다.

기본 비교

특성	LangChain Deep Agents	OpenCode	Claude Agent SDK
모델 지원	모델 무관 (Anthropic, OpenAI, 100+ 제공자)	75+ 제공자 (Ollama 포함 로컬)	Claude 모델 전용
라이선스	MIT	MIT	MIT (SDK), 독점 (Claude Code)

SDK	Python, TypeScript + CLI	터미널, 데스크톱, IDE 확장	Python, TypeScript
샌드박스	통합 도구로 사용 가능	미지원	미지원
상태 관리	타임 트래블 지원	미지원	타임 트래블 지원
Observability	LangSmith 네이티브	없음	없음

이 표에서 가장 두드러지는 차이는 모델 지원 범위입니다. Deep Agents는 100개 이상의 모델 제공자를 지원하여 벤더 종속 위험이 낮고, Claude Agent SDK는 Claude 전용으로 특정 모델에 최적화되어 있으며, OpenCode는 로컬 모델(Ollama)을 포함한 75개 이상의 제공자를 지원합니다. 다음 코드로 비교 테이블을 프로그래밍적으로 확인할 수 있습니다.

```
# 프레임워크 비교 테이블 출력
frameworks = {
    "LangChain Deep Agents": {
        "모델 지원": "100+ 제공자 (model-agnostic)",
        "라이선스": "MIT",
        "SDK": "Python, TypeScript, CLI",
        "샌드박스": "통합 지원",
        "타임 트래블": "지원",
    },
    "OpenCode": {
        "모델 지원": "75+ 제공자 (로컬 포함)",
        "라이선스": "MIT",
        "SDK": "터미널, 데스크톱, IDE",
        "샌드박스": "미지원",
        "타임 트래블": "미지원",
    },
    "Claude Agent SDK": {
        "모델 지원": "Claude 전용",
        "라이선스": "MIT (SDK)",
        "SDK": "Python, TypeScript",
        "샌드박스": "미지원",
        "타임 트래블": "지원",
    },
}

print("=== 프레임워크 비교 ===")
for name, features in frameworks.items():
    print(f"\n{name}")
    for key, value in features.items():
        print(f"  {key}: {value}")
```

```
=== 프레임워크 비교 ===

[LangChain Deep Agents]
모델 지원: 100+ 제공자 (model-agnostic)
라이선스: MIT
SDK: Python, TypeScript, CLI
샌드박스: 통합 지원
타임 트래블: 지원

[OpenCode]
모델 지원: 75+ 제공자 (로컬 포함)
라이선스: MIT
SDK: 터미널, 데스크톱, IDE
샌드박스: 미지원
타임 트래블: 미지원

[Claude Agent SDK]
모델 지원: Claude 전용
라이선스: MIT (SDK)
SDK: Python, TypeScript
샌드박스: 미지원
타임 트래블: 지원
```

3. 핵심 기능 비교

기본 스펙 비교에서 전체적인 윤곽을 파악했으니, 이제 각 프레임워크가 제공하는 핵심 기능을 세부적으로 비교합니다. 기능의 유무뿐 아니라 구현 깊이에도 주목하세요.

공통 기능

세 프레임워크 모두 코딩 에이전트의 기본 기능을 갖추고 있습니다. 이 공통 기능은 코딩 에이전트의 최소 요구 사항으로, 어떤 프레임워크를 선택하든 제공됩니다:

- 파일 작업 (읽기, 쓰기, 편집)
- 셸 명령 실행
- 검색 기능 (grep, glob)
- 계획 기능 (태스크 리스트)
- Human-in-the-Loop (권한 프레임워크는 상이)

진정한 차이는 이 공통 기능 위에 각 프레임워크가 추가하는 고유 기능에서 드러납니다.

차별화 기능

기능	Deep Agents	OpenCode	Claude Agent SDK
코어 도구	파일, 셸, 검색, 계획	파일, 셸, 검색, 계획	파일, 셸, 검색, 계획
샌드박스 통합	도구로 통합 가능	없음	없음
플러거블 백엔드	스토리지, 파일시스템	없음	없음
가상 파일시스템	플러거블 백엔드	없음	없음
네이티브 트레이싱	LangSmith	없음	없음

TIP

프레임워크 선택에서 가장 과소평가되는 요소가 Observability(관찰 가능성)입니다. 에이전트가 예상과 다르게 동작할 때, 내부 상태와 도구 호출 과정을 추적할 수 있는지가 디버깅 시간을 결정합니다. Deep Agents의 LangSmith 통합은 이 문제에 대한 성숙한 솔루션을 제공합니다.

3-1. 3가지 핵심 차별점

표 위 항목들을 더 깊이 들여다보면, Deep Agents가 다른 두 프레임워크와 갈리는 지점은 세 축에 모인다. 단순 기능 차이가 아니라 설계 결정의 결과이므로, 어느 축이 본인 프로젝트에 결정적인지를 먼저 짚어 보면 선택이 분명해진다.

Execution Model — 그래프 기반 상태 머신 vs 단방향 ReAct 루프

Deep Agents는 LangGraph의 `StateGraph` 위에서 동작한다. 노드·엣지·서브그래프로 조립되며, 분기·반복·인터럽트·재개가 모두 그래프 명세에 들어간다. 결과적으로 time travel(과거 시점으로 분기), interrupt/resume(HITL 표준 경로), subgraph 단위 스트리밍이 자연스럽게 따라온다. OpenCode와 Claude Agent SDK는 ReAct 루프 중심이라 상태 분기·subgraph 격리에서 별도 구현 비용이 든다.

Deployment & Multi-Tenancy — LangSmith Deployments 통합

Deep Agents는 LangSmith Deployments에 그대로 배포되어, `assistant_id / thread_id / user_id` 로 멀티 테넌시를 제공한다. cron 스케줄(consolidation agent), Store(영속 메모리), Checkpointer가 배포 시점에 자동 프로비저닝된다. OpenCode는 로컬 터미널 도구, Claude Agent SDK는 모델 호출 SDK에 가깝다 — 두 프레임워크 모두 별도 인프라 구축이 필요하다.

Per-Model Configuration — `HarnessProfile` / `ProviderProfile`

Deep Agents 0.5+는 provider/model별 기본값을 profile로 패키징한다. 같은 코드베이스에서 `openai:gpt-5.4` 와 `anthropic:claude-sonnet-4-6` 을 호출부 수정 없이 교체할 수 있다. Claude Agent SDK는 Claude 전용으로 모델 추상화 자체가 없고, OpenCode는 75+ 프로바이더를 지원하지만 provider 차이를 흡수하는 profile 개념이 없다.

4. 아키텍처 비교

기능 목록만으로는 프레임워크의 본질을 이해하기 어렵습니다. 기능 비교를 넘어, 각 프레임워크의 설계 철학을 이해하면 장기적인 기술 결정에 도움이 됩니다. 어떤 기능이 있는지보다, 왜 그렇게 설계되었는지를 아는 것이 향후 확장성과 유지보수에 더 중요합니다.

LangChain Deep Agents

Deep Agents의 아키텍처는 플러그블 추상화를 핵심 원칙으로 설계되었습니다. 이 철학의 핵심은 “교체 가능성”입니다. 모델, 스토리지, 파일시스템, 관찰 도구를 독립적인 추상화 레이어로 분리하여, 하나의 컴포넌트를 교체해도 나머지에 영향을 주지 않습니다:

- 플러그블 스토리지 백엔드 - 상태, 파일시스템, 스토어를 독립적으로 구성. 인메모리, SQLite, PostgreSQL 등 필요에 따라 교체 가능
- 가상 파일시스템 - 로컬, 인메모리, 샌드박스 백엔드 교체 가능. `BackendProtocol` 구현체만 바꾸면 에이전트 코드 변경 없이 전환
- LangGraph 기반 - 그래프 실행 엔진으로 복잡한 워크플로 지원. 노드와 엣지로 정의된 상태 머신이 도구 호출, 분기, 반복을 관리
- 미들웨어 시스템 - `ToDoListMiddleware`, `SummarizationMiddleware` 등으로 에이전트 동작을 세밀하게 커스텀

OpenCode

OpenCode는 개발자 경험(DX)을 최우선으로 설계된 프레임워크입니다:

- 터미널 네이티브 - Go 바이너리 하나로 설치 완료. 별도의 Python 환경 구성 없이 즉시 사용 가능
- 75+ 모델 제공자 - Ollama를 통한 로컬 모델 지원. 인터넷 없이도 에이전트 실행 가능
- LSP 통합 - Language Server Protocol을 활용하여 코드 자동 완성, 정의 이동 등 에디터 수준의 코드 이해 제공

Claude Agent SDK

Claude Agent SDK는 단일 모델 최적화라는 명확한 설계 선택을 했습니다:

- Claude 최적화 – Claude 모델의 고유 기능(extended thinking, tool use 패턴)에 특화된 추상화 제공
- 타임 트래블 – 상태 분기(branching) 지원. 에이전트 실행 중 특정 시점으로 되돌아가 다른 경로를 탐색 가능
- 간결한 API – 최소한의 코드로 에이전트를 구축할 수 있어 빠른 프로토타이핑에 적합

! WARNING

모델 종속(vendor lock-in)은 프레임워크 선택에서 반드시 고려해야 할 요소입니다. Claude Agent SDK는 Claude 모델에 특화된 만큼 강력한 최적화를 제공하지만, 향후 다른 모델로 전환이 필요할 때 상당한 리팩토링이 필요합니다. 반면 Deep Agents의 모델 무관 설계는 초기 설정이 다소 복잡하더라도 장기적인 유연성을 보장합니다.

5. 사용 사례별 추천

아키텍처를 이해했으니, 이제 구체적인 사용 사례에 따라 어떤 프레임워크를 선택해야 하는지 정리합니다. 가장 중요한 기준은 프로젝트의 규모와 수명입니다. 단기 프로토타입이라면 API의 간결함이 중요하고, 장기 프로젝트라면 확장성과 오픈서빌리티가 결정적입니다.

사용 사례	추천 프레임워크	이유
프로덕션 에이전트 앱	Deep Agents	플러그블 백엔드, 오픈서빌리티, 샌드박스
멀티 모델 에이전트	Deep Agents	100+ 모델 제공자 지원
터미널 코딩 어시스턴트	OpenCode	가볍고 빠른 시작, 로컬 모델
Claude 전용 앱	Claude Agent SDK	Claude 최적화, 간결한 API
빠른 프로토타이핑	Claude Agent SDK	간결한 API, 빠른 설정
복잡한 멀티 에이전트 시스템	Deep Agents	서브에이전트, 컨텍스트 관리
로컬 모델 사용	OpenCode	Ollama 네이티브 지원

📌 TIP

하나의 프레임워크가 모든 요구사항을 완벽하게 충족하기는 어렵습니다. 실무에서는 프로토타입 단계에서 Claude Agent SDK로 빠르게 검증한 후, 프로덕션 전환 시 Deep Agents로 마이그레이션하는 전략도 효과적입니다. 7절의 마이그레이션 고려사항을 함께 참고하세요.

다음 코드는 사용 사례를 입력하면 적합한 프레임워크를 추천하는 간단한 도우미 함수입니다.

```
# 사용 사례별 프레임워크 추천 도우미
def recommend_framework(use_case: str) -> str:
    """사용 사례에 따라 프레임워크를 추천합니다."""
    recommendations = {
        "production": ("Deep Agents", "플러그블 백엔드, 오픈버빌리티, 샌드박스"),
        "multi-model": ("Deep Agents", "100+ 모델 제공자 지원"),
        "terminal": ("OpenCode", "가볍고 빠른 시작, 로컬 모델"),
        "claude-only": ("Claude Agent SDK", "Claude 최적화, 간결한 API"),
        "prototyping": ("Claude Agent SDK", "간결한 API, 빠른 설정"),
        "multi-agent": ("Deep Agents", "서브에이전트, 컨텍스트 관리"),
        "local-model": ("OpenCode", "Ollama 네이티브 지원"),
    }
    if use_case in recommendations:
        fw, reason = recommendations[use_case]
        return f"{fw} - {reason}"
    return "해당 사용 사례를 찾을 수 없습니다."

# 테스트
test_cases = ["production", "terminal", "claude-only", "multi-agent"]
print("=== 프레임워크 추천 ===")
for case in test_cases:
    print(f" {case}: {recommend_framework(case)}")
```

```
=== 프레임워크 추천 ===
production: Deep Agents - 플러그블 백엔드, 오픈버빌리티, 샌드박스
terminal: OpenCode - 가볍고 빠른 시작, 로컬 모델
claude-only: Claude Agent SDK - Claude 최적화, 간결한 API
multi-agent: Deep Agents - 서브에이전트, 컨텍스트 관리
```

6. 생태계 비교

프레임워크 자체의 기능도 중요하지만, 주변 생태계의 성숙도는 실제 개발 경험에 큰 영향을 미칩니다. 문서 품질, 커뮤니티 규모, 서드파티 통합 수준을 비교합니다.

항목	Deep Agents	OpenCode	Claude Agent SDK
커뮤니티	LangChain 생태계 (대규모)	GitHub 커뮤니티	Anthropic 커뮤니티
문서	공식 문서 + LangSmith 연동	GitHub README	Anthropic 공식 문서
통합	LangChain, LangGraph, LangSmith	LSP, 터미널	Claude API
패키지 관리	pip/uv	go install / brew	pip/npm
에디터 통합	ACP (Zed, JetBrains, VS Code, Neovim)	자체 에디터	없음

Deep Agents의 가장 큰 생태계 장점은 LangChain 생태계와의 통합입니다. LangChain의 방대한 도구 라이브러리, LangGraph의 그래프 실행 엔진, LangSmith의 트레이싱 플랫폼을 모두 활용할 수 있습니다. 또한 ACP 프로토콜을 통해 주요 에디터와 통합할 수 있어, 개발자가 익숙한 도구에서 에이전트를 사용할 수 있습니다.

· NOTE

생태계 규모는 장점이자 단점이 될 수 있습니다. LangChain 생태계는 풍부한 서드파티 도구를 제공하지만, 버전 호환성 관리와 학습 곡선이 상대적으로 높습니다. OpenCode는 생태계가 작지만 단일 바이너리로 설치가 간단하고, Claude Agent SDK는 Anthropic이 직접 관리하므로 API 안정성이 높습니다.

7. 마이그레이션 고려사항

프레임워크는 한 번 선택하면 영원히 고정되는 것이 아닙니다. 프레임워크 선택 후에도 요구사항 변화에 따라 마이그레이션이 필요할 수 있습니다. 프레임워크 간 마이그레이션 시 고려할 핵심 사항을 정리합니다.

공통 고려사항

1. 모델 호환성 – 사용 중인 모델이 대상 프레임워크에서 지원되는지 확인
2. 도구 호환성 – 커스텀 도구의 인터페이스 변환 필요
3. 상태 관리 – 체크포인트/메모리 마이그레이션 방법 확인
4. 옵저버빌리티 – 트레이싱/로깅 솔루션 대체 방안

Deep Agents로 마이그레이션 시 장점

Deep Agents는 LangChain 생태계 위에 구축되어 있으므로, 기존 LangChain/LangGraph 사용자에게 마이그레이션이 가장 용이합니다:

- LangChain 도구 재사용 – 기존 LangChain 도구(`@tool` 데코레이터로 만든 함수)를 `tools` 파라미터에 그대로 전달 가능
- LangGraph 호환 – `CompiledSubAgent` 를 통해 기존 LangGraph 그래프를 서브에이전트로 연결 가능
- 점진적 마이그레이션 – 기존 코드를 단계적으로 전환하면서 Deep Agents 기능을 점진적으로 도입 가능

TIP

이미 LangChain이나 LangGraph를 사용 중이라면, Deep Agents로의 마이그레이션은 `create_deep_agent()` 래퍼를 추가하는 것만으로 시작할 수 있습니다. 기존 도구와 그래프를 재사용하면서 계획, 컨텍스트 관리, 메모리 등의 하네스 기능을 점진적으로 활용하세요.

주의사항

- Claude Agent SDK에서 마이그레이션 시 Claude 전용 기능(extended thinking, Claude 고유 도구 호출 패턴)은 대체 구현이 필요합니다
- OpenCode에서 마이그레이션 시 터미널 UI 로직과 에이전트 비즈니스 로직을 분리해야 합니다. OpenCode는 Go 기반이므로, Python 환경으로의 전환도 고려해야 합니다
- 모든 마이그레이션에서 가장 비용이 높은 부분은 프롬프트 엔지니어링의 재작업입니다. 각 프레임워크는 시스템 프롬프트를 처리하는 방식이 다르므로, 기존 프롬프트를 그대로 사용하면 성능이 저하될 수 있습니다

! WARNING

마이그레이션을 결정하기 전에, 현재 프레임워크에서 원하는 기능을 확장(extension)으로 구현할 수 있는지 먼저 검토하세요. Deep Agents의 미들웨어 시스템이나 커스텀 도구를 통해 많은 요구사항을 기존 프레임워크 내에서 해결할 수 있습니다. 전체 마이그레이션은 최후의 수단으로 고려하는 것이 좋습니다.

요약

주제	핵심 개념	핵심 API/도구
3-way 비교	Deep Agents, OpenCode, Claude Agent SDK	모델 지원, 라이선스, SDK
핵심 기능	공통 도구 + 차별화 기능	샌드박스, 플러그블 백엔드
아키텍처	플러그블 vs 네이티브 vs 최적화	LangGraph, LSP, Claude API
사용 사례	프로덕션, 터미널, 프로토타이핑 등	<code>recommend_framework()</code>
생태계	커뮤니티, 문서, 통합, 에디터	LangSmith, ACP
마이그레이션	모델/도구/상태 호환성 확인	점진적 전환

REFERENCES

- [Comparison with OpenCode and Claude Agent SDK](#)

10 샌드박스와의 ACP

에이전트가 코드를 실행하고 파일을 조작할 때, 호스트 시스템의 보안을 보장하는 것은 프로덕션 배포의 필수 조건이다. 이 장에서는 격리된 실행 환경인 샌드박스의 개념과 Modal, Daytona, Runloop 등 프로바이더별 특징을 비교하고, ACP (Agent Client Protocol)를 통해 에디터/IDE와 에이전트를 통합하는 방법을 다룬다. 샌드박스 와 ACP를 결합한 아키텍처는 안전한 코드 실행과 직관적인 개발자 경험을 동시에 제공한다.

7장에서 샌드박스와 ACP를 개념적으로 소개했다면, 이 장에서는 아키텍처 패턴, 프로바이더 선택 기준, 보안 가이드라인, 에디터 통합 설정까지 실무에 필요한 세부 사항을 심화한다. Deep Agents 기반 프로덕션 에이전트를 배포할 때 이 두 기술은 안전성(샌드박스)과 개발자 경험(ACP)이라는 두 축을 담당한다.

이 장을 관통하는 핵심 질문은 다음과 같다: “에이전트에게 코드 실행 능력을 부여하면서, 어떻게 호스트 시스템을 안전하게 보호하고, 동시에 개발자에게 자연스러운 사용 경험을 제공할 수 있는가?” 샌드박스는 이 질문의 보안 측면을, ACP는 사용성 측면을 해결한다.

학습 목표

LEARNING OBJECTIVES

- 샌드박스 격리 개념과 보안 원칙을 이해한다
- E2B, Modal, Docker 등 샌드박스 프로바이더를 비교한다
- ACP(Agent Communication Protocol)의 개요와 용도를 안다
- 에이전트-에디터 통합 패턴을 이해한다
- 샌드박스 + ACP 통합 아키텍처를 설계한다

핵심 질문

에이전트에게 코드 실행 권한을 주되 안전성과 개발자 경험을 동시에 지키려면, “어디에서 추론하고 어디에서 실행할 것인가?” 를 먼저 정해야 합니다.

한눈에 보는 핵심

- 샌드박스는 실행 권한을 격리하고, ACP는 에디터와의 연결을 담당합니다.
- 실무에서는 Agent-in-Sandbox보다 Sandbox-as-Tool이 보안 경계가 더 분명합니다.
- 프로바이더 선택은 GPU, 콜드 스타트, 비용 구조 같은 운영 요구와 연결됩니다.

```
# 환경 설정
from dotenv import load_dotenv
import os

load_dotenv()
assert os.environ.get("OPENAI_API_KEY"), "OPENAI_API_KEY가 설정되지 않았습니다!"
print("환경 설정 완료")
```

환경 설정 완료

```
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-5.4")

print(f"모델 설정 완료: {model.model_name}")
```

모델 설정 완료: gpt-5.4

1. 샌드박스 개념

샌드박스는 AI 에이전트가 코드를 실행하고, 파일을 관리하고, 셸 명령을 수행할 수 있는 격리된 실행 환경입니다. 일반적인 소프트웨어 개발에서도 컨테이너(Docker)로 실행 환경을 격리하지만, AI 에이전트의 경우 격리의 중요성이 훨씬 큼니다. 에이전트는 비결정적(non-deterministic)이기 때문입니다. 동일한 프롬프트에도 매번 다른 명령을 실행할 수 있으며, 모델의 할루시네이션으로 인해 의도치 않은 위험한 명령(예: `rm -rf /`)을 생성할 가능성이 항상 존재합니다.

왜 격리가 중요한가?

위험	격리 없을 때	샌드박스 사용 시
파일 시스템 접근	호스트 파일 변경/삭제 가능	격리된 파일시스템만 접근
네트워크 접근	무제한 외부 통신	제한된 네트워크 접근
자격 증명	환경 변수 유출 가능	시크릿 격리
시스템 영향	호스트 OS에 영향	호스트 시스템 보호

Deep Agents에서 샌드박스는 백엔드로 기능합니다. 4장에서 학습한 `BackendProtocol` 을 구현하므로, 기존 파일시스템 도구(`ls` , `read_file` , `write_file` , `edit_file` , `glob` , `grep`)를 그대로 사용할 수 있으며, 추가로 `execute` 도구가 노출됩니다. 에이전트 코드를 변경하지 않고 `backend` 파라미터만 교체하면 로컬 실행에서 샌드박스 실행으로 전환할 수 있습니다.



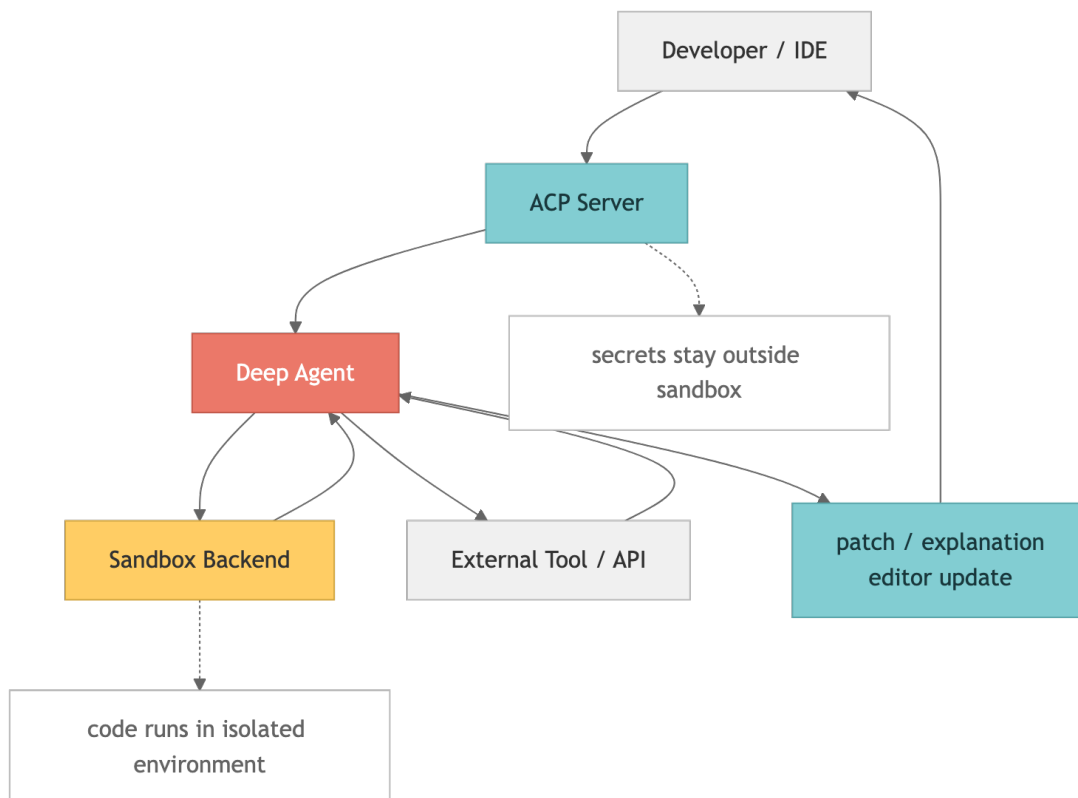
TIP

샌드박스를 도입할 때 가장 큰 장점은 코드 변경 없는 전환입니다. 개발 단계에서는 로컬 파일시스템 백엔드로 빠르게 반복하고, 프로덕션 배포 시에만 `backend=ModalSandbox(...)` 한 줄을 추가하면 됩니다. 이 플러그블 설계 덕분에 개발-배포 간의 마찰이 최소화됩니다.

2. 아키텍처 패턴

샌드박스의 필요성을 이해했으니, 이제 에이전트와 샌드박스를 어떻게 연결할 것인지 아키텍처를 살펴봅니다.

샌드박스를 에이전트와 통합하는 방식에는 두 가지 근본적으로 다른 접근법이 있습니다. 이 선택은 단순한 구현 방식의 차이가 아니라, 보안 경계(security boundary)를 어디에 설정하느냐의 문제입니다. 각 패턴의 보안 모델과 운영 특성을 이해하면 프로젝트에 맞는 올바른 선택을 할 수 있습니다.



위 시퀀스 다이어그램에서 핵심은 에이전트의 지능 과 실행 권한 이 분리된다는 점입니다. ACP는 개발자 경험을 담당하고, 샌드박스는 실제 실행을 격리합니다. 따라서 실무에서는 “어디서 추론하고, 어디서 실행하는가?”를 먼저 결정하면 패턴 선택이 훨씬 쉬워집니다.

Agent-in-Sandbox

에이전트가 샌드박스 내부에서 실행되며, 네트워크 프로토콜을 통해 외부와 통신합니다.

장점	단점
개발 환경과 동일한 경험	자격 증명 노출 위험
간단한 설정	인프라 복잡성 증가

Sandbox-as-Tool (권장)

에이전트가 외부(호스트)에서 실행되며, 샌드박스 API를 호출하여 코드를 실행합니다. Deep Agents의 기본 접근법이며, 보안상 이 패턴을 강력히 권장합니다. 에이전트의 두뇌(모델 호출, 상태 관리)와 손(코드 실행, 파일 조작)을 물리적으로 분리하는 것이 핵심입니다.

장점	단점
에이전트 상태와 실행 분리	네트워크 지연
시크릿을 샌드박스 외부에 유지	
병렬 태스크 실행 가능	

! WARNING

Agent-in-Sandbox 패턴에서는 에이전트가 자격 증명에 접근할 수 있어 보안 위험이 크게 증가합니다. 에이전트가 `os.environ` 으로 API 키를 읽거나, `cat ~/.ssh/id_rsa` 로 SSH 키를 확인하거나, 네트워크를 통해 외부로 데이터를 전송할 수 있습니다. 프로덕션 환경에서는 반드시 Sandbox-as-Tool 패턴을 사용하세요.

· NOTE

패턴 선택 요약

- Agent-in-Sandbox 는 로컬 개발 경험은 단순하지만 보안 경계가 약합니다.
- Sandbox-as-Tool 은 네트워크 호출이 추가되지만 시크릿과 실행 권한을 분리할 수 있습니다.
- ACP를 함께 쓰면 에디터 UX는 유지하면서도 실제 코드는 샌드박스에서만 실행되도록 설계할 수 있습니다.

다음 코드는 두 아키텍처 패턴의 구조적 차이를 ASCII 다이어그램으로 보여줍니다.

```
# 두 가지 아키텍처 패턴 비교 (참고용)
print("=== 패턴 1: Agent-in-Sandbox ===")
print("  [샌드박스]")
print("    |-- 에이전트 (내부 실행)")
print("    |-- 파일시스템")
print("    |-- 코드 실행")
print("  <---> 네트워크 프로토콜 <---> 외부 시스템")
```

```
print()
print("=== 패턴 2: Sandbox-as-Tool (권장) ===")
print("  [호스트]")
print("    |-- 에이전트 (외부 실행)")
print("    |-- 자격 증명 관리")
print("    |-- API 호출 --> [샌드박스]")
print("                                |-- 파일시스템")
print("                                |-- 코드 실행")
```

```
=== 패턴 1: Agent-in-Sandbox ===
[샌드박스]
|-- 에이전트 (내부 실행)
|-- 파일시스템
|-- 코드 실행
<---> 네트워크 프로토콜 <---> 외부 시스템

=== 패턴 2: Sandbox-as-Tool (권장) ===
[호스트]
|-- 에이전트 (외부 실행)
|-- 자격 증명 관리
|-- API 호출 --> [샌드박스]
                                |-- 파일시스템
                                |-- 코드 실행
```

3. 샌드박스 프로바이더 비교

아키텍처 패턴을 결정했다면, 다음 질문은 “어떤 프로바이더를 사용할 것인가?”입니다. Deep Agents는 여러 클라우드 샌드박스 프로바이더를 지원하며, 각 프로바이더는 고유한 강점을 가지고 있습니다. 프로바이더 선택의 핵심 기준은 워크로드 유형, 콜드 스타트 시간, 비용 구조입니다.

패키지 / 프로바이더	특징	적합한 용도
langchain-modal / Modal	GPU 지원, ML 워크로드	AI/ML 작업, 데이터 처리
langchain-daytona / Daytona	TypeScript/Python, 빠른 콜드 스타트	웹 개발, 빠른 반복
langchain-runloop / Runloop	일회용 devbox, 격리 실행	코드 테스트, 일회성 작업
langsmith[sandbox] / LangSmith	LangSmith Deployments 통합 (private beta)	LangSmith 위 운영 환경
langchain-agentcore-codeinterpreter / AgentCore	AWS Bedrock 기반 코드 인터프리터	AWS 생태계 통합

프로바이더 선택 기준을 정리하면:

- **GPU가 필요한 ML/AI 작업** - Modal이 유일한 선택입니다. 모델 학습, 추론, 대규모 데이터 처리에 적합합니다.
- **웹 개발 에이전트** - Daytona의 빠른 콜드 스타트가 유리합니다. TypeScript/Python 환경이 즉시 준비됩니다.
- **일회성 코드 검증** - Runloop의 일회용 devbox가 비용 효율적입니다. 테스트 실행 후 자동 삭제됩니다.

다음 코드는 Modal 프로바이더를 사용한 샌드박스 설정 예시입니다.

```
# Modal 샌드박스 연동 - langchain-modal 패키지 (canonical 패턴)
# pip install langchain-modal deepagents
import modal
from deepagents import create_deep_agent
from langchain_anthropic import ChatAnthropic
from langchain_modal import ModalSandbox

app = modal.App.lookup("your-app")
modal_sandbox = modal.Sandbox.create(app=app)
backend = ModalSandbox(sandbox=modal_sandbox)

agent = create_deep_agent(
    model=ChatAnthropic(model="claude-sonnet-4-6"),
    system_prompt="You are a Python coding assistant with sandbox access.",
    backend=backend,
)

try:
    result = agent.invoke({
        "messages": [{"role": "user", "content": "Create a small Python package and run pytest"}],
    })
finally:
    modal_sandbox.terminate() # 필수: 리소스 해제
```

4. 보안 가이드라인

프로바이더를 선택하고 샌드박스를 설정하더라도, 보안 관행을 따르지 않으면 격리의 의미가 퇴색됩니다. 이 섹션은 샌드박스 운영에서 반드시 지켜야 할 보안 원칙을 정리합니다.

! WARNING

절대 샌드박스 안에 시크릿을 넣지 마세요. 에이전트는 환경 변수를 읽거나(`os.environ`), 파일을 탐색하거나(`cat ~/.ssh/id_rsa`), 네트워크를 통해 외부로 데이터를 전송할 수 있습니다. API 키, 데이터베이스 자격 증명, SSH 키 등은 반드시 샌드박스 외부의 전용 도구에서만 관리하세요.

안전한 관행

1. 자격 증명은 외부 도구에서만 관리 - 샌드박스 외부의 전용 도구 사용
2. Human-in-the-Loop - 민감한 작업에 대해 사람 승인 요구
3. 네트워크 접근 차단 - 불필요한 아웃바운드 연결 차단
4. 아웃바운드 모니터링 - 예기치 않은 외부 연결 감시. 에이전트가 `curl` 등으로 외부 서버에 데이터를 전송하는지 모니터링
5. 출력 검토 - 샌드박스 출력을 애플리케이션에 적용하기 전 검토. 특히 에이전트가 생성한 파일이 악성 코드를 포함하지 않는지 확인
6. 최소 권한 원칙 - 샌드박스에 필요한 최소한의 권한만 부여. root 접근, 불필요한 파일 마운트, 과도한 메모리 할당을 제한

· NOTE

보안은 계층적(defense-in-depth) 접근이 효과적입니다. 샌드박스 격리만으로는 완벽한 보안을 보장할 수 없습니다. Human-in-the-Loop(8장), 네트워크 정책, 출력 검증을 함께 적용하여 여러 계층의 방어선을 구축하세요.

5. 파일 전송과 라이프사이클

보안 가이드라인을 확립했으니, 이제 샌드박스를 실제로 운영하는 방법을 다룹니다.

샌드박스를 실무에서 운영할 때는 파일 전송과 라이프사이클 관리가 중요합니다. 샌드박스 내부의 파일시스템은 격리되어 있으므로, 입력 데이터를 업로드하고 결과물을 다운로드하는 메커니즘이 필요합니다. 이 과정을 시딩(seeding)과 아티팩트 수집(artifact collection)이라고 합니다.

파일 접근 방법

방법	설명
에이전트 파일시스템 도구	<code>execute()</code> 를 통한 직접 파일 작업
파일 전송 API	<code>uploadFiles()</code> , <code>downloadFiles()</code> 로 시드/아티팩트 관리

라이프사이클 관리

샌드박스는 불필요한 비용을 방지하기 위해 명시적 종료が必要です. 운영 모드는 두 가지로 갈린다.

모드	수명	적합한 용도
Thread-scoped (기본)	대화 thread 하나당 샌드박스 하나. 첫 run에서 생성, 같은 thread의 다음 turn에서 재사용. idle TTL로 자동 정리	채팅·대화형 에이전트
Assistant-scoped	같은 assistant의 모든 thread가 샌드박스 하나를 공유. 파일·패키지·레포지토리가 대화 사이에 누적. 반드시 TTL 또는 주기적 스냅샷 필요	장기 코드 세션, 누적 작업 공간

운영 체크리스트: 단발성 스크립트 실행은 `try/finally` 로 즉시 종료. 채팅/장기 세션은 thread 당 고유 샌드박스 + TTL 자동 정리. LangSmith Deployments 운영 시 세션 종료 후에 종료 로직 연결.

TIP

대화형 에이전트에서는 대화 스레드마다 고유한 샌드박스를 할당하고, TTL을 30분 정도로 설정하는 것이 일반적입니다. 사용자가 대화를 재개하면 새 샌드박스를 생성하되, 이전 상태를 파일 전송 API로 복원할 수 있습니다.


```
# 파일 전송과 라이프사이클 설정 예시 (참고용)
lifecycle_config = {
    "ttl_seconds": 1800, # 30분
    "auto_shutdown": True,
    "thread_isolation": True,
}

file_operations = [
    "uploadFiles(['/local/data.csv'], '/sandbox/data/')",
    "downloadFiles(['/sandbox/output/result.json'], '/local/results/')",
]

print("=== 라이프사이클 설정 ===")
for key, value in lifecycle_config.items():
    print(f" {key}: {value}")

print("\n=== 파일 전송 예시 ===")
for op in file_operations:
    print(f" {op}")
```

```
=== 라이프사이클 설정 ===
ttl_seconds: 1800
auto_shutdown: True
thread_isolation: True

=== 파일 전송 예시 ===
uploadFiles(['/local/data.csv'], '/sandbox/data/')
downloadFiles(['/sandbox/output/result.json'], '/local/results/')
```

6. ACP 개요

여기서부터 이 장의 두 번째 주제인 ACP로 전환합니다. 샌드박스가 에이전트의 안전한 실행 환경을 제공했다면, ACP는 에이전트를 개발자의 일상 도구로 통합하는 역할을 합니다.

샌드박스가 에이전트의 실행 환경 안전성을 담당한다면, ACP는 에이전트와 개발자 워크플로를 연결하는 역할을 합니다. ACP(Agent Client Protocol)는 코딩 에이전트와 개발 환경(에디터/IDE) 간의 통신을 표준화하는 프로토콜로, 에이전트가 에디터 내에서 직접 파일을 편집하고, 변경 사항을 실시간으로 동기화할 수 있게 합니다.

MCP vs ACP

프로토콜	용도	대상
MCP (Model Context Protocol)	외부 도구 통합	에이전트 ↔ 외부 서비스
ACP (Agent Client Protocol)	에디터-에이전트 통합	에이전트 ↔ 에디터/IDE

ACP는 에이전트가 에디터와 직접 상호작용하여 코드 편집, 파일 탐색, 터미널 명령을 수행할 수 있게 합니다. stdio(표준 입출력) 기반 통신을 사용하므로, 에디터가 에이전트 프로세스를 자식 프로세스로 실행하고 stdin/stdout 파이프를 메시지를 주고받습니다. 이 방식은 네트워크 설정이 필요 없어 로컬 개발 환경에 특히 적합합니다.

· NOTE

ACP는 7장에서 소개한 MCP(Model Context Protocol)와 상호 보완적입니다. MCP로 외부 도구(데이터베이스, API)를 에이전트에 연결하고, ACP로 에이전트를 에디터에 연결하면, 완전한 개발자 도구 체인이 구성됩니다. 두 프로토콜의 역할을 혼동하지 마세요: MCP는 에이전트가 사용하는 도구를 확장하고, ACP는 에이전트를 사용하는 인터페이스를 확장합니다.

7. ACP 서버 구현

ACP의 개념을 이해했으니, 실제 구현을 살펴봅시다. ACP 서버는 Deep Agents 에이전트를 에디터가 소비할 수 있는 형태로 노출하는 역할을 합니다. `deepagents-acp` 패키지의 `AgentServerACP` 클래스가 이를 담당합니다. 다음 코드는 ACP 서버의 최소 구현 예시입니다. `create_deep_agent()` 로 에이전트를 생성하고, `AgentServerACP` 로 감싸면 에디터와 통신할 수 있는 ACP 서버가 완성됩니다.

```
# ACP 서버 - canonical 패턴 (asyncio + acp.run_agent)
# pip install deepagents-acp
import asyncio

from acp import run_agent
from deepagents import create_deep_agent
from langgraph.checkpoint.memory import MemorySaver

from deepagents_acp.server import AgentServerACP

async def main() -> None:
    agent = create_deep_agent(
        model="anthropic:claude-sonnet-4-6",
        system_prompt="You are a helpful coding assistant",
        checkpoint=MemorySaver(),
    )
    server = AgentServerACP(agent)
    await run_agent(server)

if __name__ == "__main__":
    asyncio.run(main())
```

8. ACP 지원 에디터

ACP 서버를 구현했다면, 이제 어떤 에디터에서 이 서버에 연결할 수 있는지 확인합니다. 현재 ACP를 지원하는 주요 에디터와 통합 방식은 다음과 같습니다.

에디터	통합 방식
Zed	네이티브 통합
JetBrains IDEs	빌트인 지원
Visual Studio Code	vscode-acp 플러그인

Neovim

ACP 호환 플러그인

Zed 설정 예시

```
// Zed settings.json
{
  "agent_servers": [
    {
      "command": "python",
      "args": ["acp_server.py"],
      "env": {
        "ANTHROPIC_API_KEY": "sk-..."
      }
    }
  ]
}
```

Toad CLI

Toad는 ACP 서버를 로컬 개발 도구로 실행하기 위한 프로세스 관리자입니다. 시작·종료·재시작을 자동으로 관리해 주므로 수동으로 프로세스를 다룰 필요가 없습니다.

```
# 설치
uv tool install -U batrachian-toad

# 실행 - ACP 서버 명령과 작업 디렉토리를 인자로 전달
toad acp "python path/to/your_server.py" .
```

· NOTE

에디터별 ACP 통합의 성숙도는 다릅니다. Zed와 JetBrains는 네이티브/빌트인 지원을 제공하여 설정이 간단하지만, VS Code와 Neovim은 별도 플러그인 설치가 필요합니다. 에디터 선택 시 ACP 지원 수준도 함께 고려하세요.

9. 샌드박스 + ACP 통합

마지막으로, 이 장의 두 주제를 하나로 결합합니다. 샌드박스과 ACP를 결합하면, 에디터에서 에이전트를 제어하면서 코드 실행은 격리된 환경에서 수행하는 완전한 프로덕션 아키텍처를 구현할 수 있습니다. 이것이 Deep Agents 기반 코딩 에이전트의 궁극적인 배포 형태입니다.

이 통합 아키텍처에서 각 컴포넌트의 역할은 명확하게 분리됩니다: 에디터는 사용자 인터페이스, 에이전트는 지능과 조율, 샌드박스는 안전한 실행을 담당합니다. 세 컴포넌트가 각자의 역할에 집중하므로, 개별 컴포넌트를 독립적으로 업그레이드하거나 교체할 수 있습니다.

통합 아키텍처

```
[에디터/IDE] <-- ACP --> [에이전트] <-- API --> [샌드박스]
```

코드 편집
파일 탐색
터미널 UI

태스크 관리
컨텍스트 관리
도구 호출

코드 실행
파일 격리
보안 환경

장점

- 에디터에서 직접 에이전트와 상호작용 - 개발자가 익숙한 도구를 떠나지 않아도 됩니다
- 코드 실행은 안전한 샌드박스에서 수행 - 호스트 시스템에 영향을 주지 않습니다
- 시크릿은 호스트(에이전트 측)에서만 관리 - API 키, 데이터베이스 자격 증명이 샌드박스에 노출되지 않습니다
- Human-in-the-Loop과 결합하면 민감한 작업에 대한 사람의 승인까지 추가할 수 있습니다

! WARNING

통합 아키텍처에서 ACP 서버와 샌드박스 프로바이더 모두에 장애가 발생할 수 있습니다. ACP 연결이 끊어지면 에디터에서 에이전트를 제어할 수 없고, 샌드박스가 중단되면 코드 실행이 불가능합니다. 프로덕션 환경에서는 양쪽 모두에 대한 오류 처리와 재연결 로직을 구현하세요.

다음 코드는 샌드박스 백엔드와 ACP 서버를 결합한 완전한 구성 예시입니다.

```
# 샌드박스 + ACP 통합 - canonical 패턴
import asyncio

import modal
from acp import run_agent
from deepagents import create_deep_agent
from langchain_anthropic import ChatAnthropic
from langchain_modal import ModalSandbox
from langgraph.checkpoint.memory import MemorySaver

from deepagents_acp.server import AgentServerACP

async def main() -> None:
    app = modal.App.lookup("your-app")
    modal_sandbox = modal.Sandbox.create(app=app)
    try:
        agent = create_deep_agent(
            model=ChatAnthropic(model="claude-sonnet-4-6"),
            system_prompt="You are a coding assistant.",
            backend=ModalSandbox(sandbox=modal_sandbox),
            checkpointinter=MemorySaver(),
            interrupt_on={"execute": True}, # 코드 실행 전 HITL 승인
        )
        server = AgentServerACP(agent)
        await run_agent(server)
    finally:
        modal_sandbox.terminate()

if __name__ == "__main__":
    asyncio.run(main())
```

요약

주제	핵심 개념	핵심 API/도구
샌드박스 개념	격리된 실행 환경으로 호스트 보호	<code>execute</code> , 파일시스템 도구
아키텍처 패턴	Agent-in-Sandbox vs Sandbox-as-Tool	Sandbox-as-Tool 권장
프로바이더	Modal(GPU), Daytona(빠른 시작), Runloop(일회용)	<code>ModalSandbox</code>
보안	시크릿 외부 관리, HITL, 네트워크 차단	<code>interrupt_on</code>
ACP 개요	에디터-에이전트 통신 표준화	<code>AgentServerACP</code>
ACP 서버	stdio 모드로 에이전트 노출	<code>deepagents-acp</code>
에디터 통합	Zed, JetBrains, VS Code, Neovim	ACP 프로토콜
통합 패턴	에디터 ↔ 에이전트 ↔ 샌드박스	ACP + Sandbox 결합

REFERENCES

- [Sandboxes](#)
- [Agent Client Protocol \(ACP\)](#)

11

비동기 서브에이전트

AsyncSubAgent · Non-blocking · Mid-flight steering

Deep Agents 0.5.0의 플래그십 기능 `AsyncSubAgentMiddleware` 는 슈퍼바이저가 블록되지 않고 서브에이전트를 백그라운드로 기동합니다. 장시간 리서치·코딩·병렬 서브에이전트 조율처럼 수 분에서 수 시간 단위 작업에서 기존 동기 서브에이전트의 한계를 넘는 인프라입니다. 단, 이 기능은 Agent Protocol 서버가 필요합니다 – LangSmith Deployments 또는 `langgraph dev` 같은 자체 호스트 환경에서만 동작합니다.

학습 목표

LEARNING OBJECTIVES

- 동기 서브에이전트와 `AsyncSubAgent` 의 실행 모델 차이를 설명한다
- `start_async_task` / `check_async_task` / `update_async_task` / `cancel_async_task` / `list_async_tasks` 5개 도구의 역할을 구분한다
- ASGI(co-deploy)와 HTTP(원격) 전송 모드를 비교하고 하이브리드 구성을 만든다
- `async_tasks` state 채널이 컨텍스트 압축을 넘어 상태를 보존하는 원리를 이해한다
- Mid-flight steering 패턴(update/cancel)과 `--n-jobs-per-worker` 슬롯 튜닝을 안다

11.1 기존 동기 서브에이전트의 한계

기존 서브에이전트는 동기였습니다. `task` 도구가 호출되면 슈퍼바이저는 서브에이전트가 끝날 때까지 멈추고, 사용자는 그 동안 새 지시를 줄 수 없었습니다. 분 단위 이상의 리서치 작업을 동기로 돌리면 프론트엔드가 응답 없이 굳어 있고, 사용자는 “아직 살아 있는가”를 확인할 방법이 없었습니다.

0.5.0의 `AsyncSubAgentMiddleware` 는 이 제약을 제거합니다.

- **Non-blocking 실행:** `start_async_task` 가 task id만 즉시 반환. 슈퍼바이저는 곧바로 사용자와 대화를 계속합니다.
- **Mid-flight steering:** 실행 중인 서브에이전트에 follow-up 지시를 보내거나 취소할 수 있습니다.
- **독립 스레드:** 각 서브에이전트 태스크는 자체 thread와 run을 가집니다. 슈퍼바이저의 컨텍스트 압축이 일어나도 상태가 손실되지 않습니다.

11.2 슈퍼바이저에게 주입되는 5개 도구

`AsyncSubAgentMiddleware` 는 슈퍼바이저에게 다섯 개 도구를 주입합니다.

도구	역할
<code>start_async_task</code>	서브에이전트 백그라운드 기동. task id 즉시 반환
<code>check_async_task</code>	현재 상태 조회, 완료되었으면 최종 출력 추출
<code>update_async_task</code>	실행 중인 태스크의 같은 스레드에 새 지시 주입 (interrupt 전략)
<code>cancel_async_task</code>	서버에 cancel 신호 전송, 태스크를 <code>cancelled</code> 로 마킹
<code>list_async_tasks</code>	추적 중인 모든 태스크의 현재 상태 일괄 조회

11.3 기본 사용

`AsyncSubAgent` 스펙 리스트를 `subagents` 에 전달하면 `create_deep_agent` 가 `AsyncSubAgentMiddleware` 를 자동 부착합니다.

```
from deepagents import AsyncSubAgent, create_deep_agent

async_subagents = [
    AsyncSubAgent(
        name="researcher",
        description="정보 수집과 종합이 필요한 리서치 작업",
        graph_id="researcher",
    ),
    AsyncSubAgent(
        name="coder",
        description="코드 생성/리뷰 작업",
        graph_id="coder",
        url="https://coder-deployment.langsmith.dev", # 원격 HTTP 호출
    ),
]

agent = create_deep_agent(
    model="google_genai:gemin-3.5-flash",
    subagents=async_subagents,
)
```

AsyncSubAgent 핵심 필드

필드	설명
<code>name</code>	슈퍼바이저가 참조하는 고유 식별자
<code>description</code>	어떤 태스크에 위임할지 판단 근거. 동기 서브에이전트와 동일하게 중요
<code>graph_id</code>	<code>langgraph.json</code> 의 graph 이름과 일치해야 함
<code>url</code>	선택. 없으면 ASGI(in-process), 있으면 원격 HTTP 호출
<code>headers</code>	선택. 자체 호스트 서버의 인증 헤더

11.4 `async_tasks` state 채널

태스크 메타데이터는 메시지 히스토리와 분리된 `async_tasks` state 채널에 저장됩니다. 레코드 하나에는 다음이 들어갑니다:

- task id
- agent name
- thread id, run id
- status (`pending` / `running` / `success` / `error` / `cancelled`)
- `created_at` , `updated_at`

이 분리 구조 덕분에 슈퍼바이저의 컨텍스트가 압축(summarization)되어 오래된 메시지가 요약으로 대체되어도 task id는 사라지지 않습니다. 압축 후에도 `check_async_task` / `update_async_task` 를 그대로 호출할 수 있습니다.

! WARNING

커스텀 state reducer나 middleware로 state를 재구성할 때 `async_tasks` 채널을 덮어쓰면 실행 중인 태스크 추적을 잃습니다. 병합 전략을 명시하세요.

11.5 전송 모드

ASGI (co-deploy, 기본 권장)

`url` 을 생략하면 서브에이전트가 슈퍼바이저와 같은 프로세스에서 함수 호출로 실행됩니다. 네트워크 레이턴시 없이, 동일 `langgraph.json` 에 두 그래프를 등록합니다.

```
// langgraph.json
{
  "dependencies": [".",],
  "graphs": {
    "supervisor": "./agent.py:supervisor",
    "researcher": "./subagents/researcher.py:graph",
    "coder": "./subagents/coder.py:graph"
  },
  "env": ".env"
}
```

```
AsyncSubAgent(
    name="researcher",
    description="...",
    graph_id="researcher", # url 없음 → ASGI
)
```

HTTP (원격)

독립 배포된 서브에이전트를 `url` 로 호출합니다. 리서치 전용 저가 모델, 코딩 전용 GPU 배포처럼 리소스 프로파일이 다른 서브에이전트를 따로 스케일하고 싶을 때 씁니다.


```
import os

AsyncSubAgent(
    name="coder",
    description="...",
    graph_id="coder",
    url="https://coder-deployment.langsmith.dev",
    headers={"x-api-key": os.environ["CODER_API_KEY"]},
)
```

하이브리드

둘을 섞어도 됩니다. 경량 서버에이전트는 ASGI, 리소스 집약 서버에이전트는 원격 HTTP로 배포 토폴로지를 워크로드에 맞게 나눕니다.

11.6 실행 라이프사이클

1. **Launch** — `start_async_task` 가 새 thread 생성, run 시작, task id 즉시 반환
2. **Check** — `check_async_task` 가 상태 조회, 완료 시 최종 출력 추출
3. **Update** — `update_async_task` 가 기존 thread에 history를 유지한 채 새 instruction으로 interrupt 기 반 새 run 기동
4. **Cancel** — `cancel_async_task` 가 `runs.cancel()` 호출 후 cancelled로 마킹
5. **List** — 비종결 태스크의 live status를 병렬 조회, 종결된 건 캐시 반환

11.7 Mid-flight steering 패턴

대화 도중 사용자가 방향을 바꾸면 슈퍼바이저가 `update_async_task` 를 호출합니다.

```
사용자: 경쟁사 리서치 시작해줘.
슈퍼바이저: [start_async_task(agent="researcher", ...)] → task_abc123

사용자: 아, Series A 이상만 봐줘.
슈퍼바이저: [update_async_task(task_id="task_abc123",
                               instruction="범위를 Series A 이상으로 좁혀줘")]

사용자: 그만, 대신 SaaS 쪽으로 다시 파줘.
슈퍼바이저: [cancel_async_task(task_id="task_abc123")]
            [start_async_task(agent="researcher",
                               description="SaaS 경쟁사 리서치")]
```

`update_async_task` 는 같은 스레드에 새 run을 만들므로, 서버에이전트는 이전 탐색 결과를 그대로 이어받고 새 지시만 엽니다. 완전히 새 태스크로 시작할 때만 cancel + start 조합을 씁니다.

11.8 로컬 개발: `--n-jobs-per-worker`

`langgraph dev` 의 기본 worker pool은 작아서 동시 서버에이전트 기동 시 큐잉이 발생합니다. 슬롯 수를 늘려 줍니다.

```
langgraph dev --n-jobs-per-worker 10
```

서브에이전트 3개를 동시에 돌리는 슈퍼바이저는 최소 4 슬롯이 필요합니다(슈퍼바이저 1 + 서브에이전트 3). 여유 있게 10 20으로 잡아 두세요.

11.9 v2 통합 스트리밍으로 라이프사이클 관찰

비동기 서브에이전트의 Pending/Running/Complete 신호는 별도 v3 projection이 아니라

`agent.stream(...)` 한 경로로 받습니다. `subgraphs=True` + `version="v2"` 로 메인/서브 이벤트를 통일하고, `async_tasks` 채널 변화는 `updates` 모드의 `data` 에 함께 흘러나옵니다.

```
async for chunk in agent.astream(
    {"messages": [{"role": "user", "content": "경쟁사 3개 동시 리서치"}]},
    stream_mode=["updates", "messages", "custom"],
    subgraphs=True,
    version="v2",
):
    t = chunk["type"]
    if t == "updates":
        # async_tasks 채널 변화, check_async_task / list_async_tasks 결과
        for node, data in chunk["data"].items():
            print(f"step={node}")
    elif t == "messages":
        # 슈퍼바이저 토큰 + tool_call_chunks
        ...
    elif t == "custom":
        # 도구 내부 get_stream_writer 진행률
        ...
```

신호	감지 지점
Pending	슈퍼바이저가 <code>start_async_task</code> <code>tool_call</code> 을 방출한 순간
Running	<code>async_tasks[task_id].status</code> 가 <code>running</code> 으로 바뀐 <code>updates</code> 청크
Complete	<code>status</code> 가 <code>success</code> / <code>error</code> / <code>cancelled</code> 중 하나로 종결된 <code>updates</code> 청크

UI 카드 상태를 이 세 신호에 매핑하면 별도 폴링 없이 라이프사이클이 자동으로 흐릅니다. 자세한 스트리밍 분기는 14장(스트리밍)을 참고합니다.

11.10 동기 vs 비동기 선택 기준

축	Sync SubAgent	AsyncSubAgent
실행	슈퍼바이저 블록, 완료까지 대기	즉시 task id 반환, 슈퍼바이저 계속 진행
결과 회수	자동으로 슈퍼바이저에 반환	<code>check_async_task</code> 로 폴링 필요
Mid-task 지시	불가	<code>update_async_task</code> 로 가능
취소	불가	<code>cancel_async_task</code> 로 가능
상태 유지	스테이트리스 (일회성)	자체 thread에 상태 유지, 상호작용 가능

인프라 요구	특별 요구 없음	Agent Protocol 서버 필요
적합 작업	수초 수십초, 결과만 필요한 작업	분 시간 단위 장시간, 중간 개입 가능한 작업

판단 흐름

- 작업이 수초 내 끝나고 슈퍼바이저가 결과를 바로 써야 한다 → **Sync**
- 작업이 수 분 이상이거나, 여러 서버에이전트를 병렬로 돌리고 싶다 → **Async**
- 사용자가 도중에 방향을 바꾸거나 중단시킬 수 있다 → **Async**
- LangSmith Deployments를 쓰지 않고 Agent Protocol 서버도 띄울 수 없다 → **Sync만 가능**

11.11 주의사항

- **Agent Protocol 의존성:** `AsyncSubAgent` 를 선언했는데 실행 환경이 Agent Protocol을 지원하지 않으면 초기화 단계에서 실패
- `async_tasks` 채널은 보존하라: 커스텀 state reducer나 middleware로 state를 재구성할 때 이 채널을 덮어쓰면 추적 상실
- **폴링 비용:** `check_async_task` 를 너무 자주 호출하지 않도록 시스템 프롬프트에 가이드를 넣음 (예: “한 번에 2 3개 태스크를 기동한 뒤 사용자 반응을 기다려라”)
- **장시간 태스크 정리:** 장기 미종결 태스크는 `list_async_tasks` + `cancel_async_task` 로 정기적으로 정리. LangSmith Deployments는 checkpoint 기반이라 비용이 남음
- **HTTP 모드 타임아웃:** 원격 URL은 헤더·네트워크 타임아웃·재시도 정책을 별도로 확인

핵심 정리

- `AsyncSubAgent` 는 슈퍼바이저를 블록하지 않고 서버에이전트를 백그라운드로 기동하는 0.5.0 플래그십 기능
- 5개 도구(`start` / `check` / `update` / `cancel` / `list`)로 라이프사이클을 제어
- ASGI는 co-deploy 기본, HTTP는 리소스 분리 시 사용, 하이브리드 가능
- `async_tasks` state 채널이 컨텍스트 압축에도 태스크 추적을 유지
- 수 분 이상 걸리거나 사용자 개입이 필요한 작업, 다중 병렬 서버에이전트에 적합

12

프로덕션 준비

Thread · User · Assistant 기반 10대 영역

로컬 프로토타입을 다중 사용자·다중 테넌트·장기 운영 가능한 프로덕션 에이전트로 전환할 때 짚어야 할 열 가지 영역을 다룹니다. Deep Agent를 실서비스에 올리기 직전, 또는 올렸는데 운영 이슈가 쌓일 때 이 장을 체크리스트로 씁니다.

학습 목표

LEARNING OBJECTIVES

- Thread·User·Assistant 세 추상을 프로덕션 설계의 출발점으로 이해한다
- `langgraph.json` + LangSmith Deployments가 자동 프로비저닝하는 인프라를 안다
- 멀티 테넌트 authz, 엔드유저 자격증명, 메모리 스코핑을 구분한다
- 샌드박스 thread-scoped vs assistant-scoped 패턴을 선택한다
- 내구성·레이트 리밋·에러 3층·PII·실시간 프런트엔드를 미들웨어와 SDK로 구성한다

12.1 세 가지 핵심 추상

프로덕션 Deep Agent는 세 가지 핵심 추상 위에서 운영됩니다.

- **Thread** – 한 번의 대화. 메시지·파일·체크포인트의 단위
- **User** – 인증된 신원. 자원 소유권과 접근 범위의 단위
- **Assistant** – 설정된 에이전트 인스턴스 (프롬프트·도구·모델 조합)

이 세 개념 위에서 관리할 영역이 열 개이며, 각 영역은 독립적으로 도입할 수 있고 리스크 프로파일에 따라 골라 적용합니다.

12.2 LangSmith Deployments

`deepagents deploy` CLI 또는 LangSmith Deployment로 배포하면 다음 인프라가 자동 프로비저닝됩니다.

- Assistants / Threads / Runs API
- Store + Checkpointer (퍼시스턴스)
- 인증, Webhook, Cron, Observability

– MCP/A2A 노출 옵션

```
// langgraph.json 최소 설정
{
  "dependencies": [".",],
  "graphs": {
    "agent": "./agent.py:agent"
  },
  "env": ".env"
}
```

12.3 멀티 테넌트 접근 제어

커스텀 auth/authz 핸들러

LangSmith Deployments는 커스텀 인증으로 사용자 신원을 확립하고, 별도 authorization 핸들러로 thread / assistant / store namespace 접근을 제어합니다. 핸들러는 리소스에 소유권 메타데이터를 태깅하고 사용자별 가시성을 필터링하며, HTTP 403으로 접근을 거부합니다.

Workspace RBAC

팀 단위 권한(LangSmith 자체 기능).

역할	권한
Workspace Admin	전체 권한
Workspace Editor	생성/수정, 삭제·멤버 관리 불가
Workspace Viewer	읽기 전용

12.4 엔드유저 자격증명 관리

에이전트가 사용자 대신 외부 서비스(GitHub, Slack, Gmail 등)를 호출할 때 자격증명을 에이전트 코드 밖에서 관리합니다.

Agent Auth (OAuth 2.0)

관리형 OAuth 플로우입니다. 첫 호출 때 사용자에게 consent URL을 interrupt로 제시하고, 토큰을 받으면 자동으로 resume·refresh합니다.

```
from langchain_auth import Client
from langchain.tools import tool, ToolRuntime

auth_client = Client()

@tool
async def github_action(runtime: ToolRuntime):
    """사용자 명의로 GitHub 작업 수행."""
    auth_result = await auth_client.authenticate(
        provider="github",
```

```
scopes=["repo", "read:org"],
user_id=runtime.server_info.user.identity,
)
# auth_result.token으로 GitHub API 호출
```

Sandbox Auth Proxy

샌드박스에서 실행되는 사용자 코드(또는 에이전트가 생성한 코드)가 외부 API를 호출할 때 프록시가 자격증명을 주입합니다. API 키가 샌드박스 안 코드에 노출되지 않습니다.

```
{
  "proxy_config": {
    "rules": [
      {
        "name": "openai-api",
        "match_hosts": ["api.openai.com"],
        "inject_headers": {
          "Authorization": "Bearer ${OPENAI_API_KEY}"
        }
      }
    ]
  }
}
```

`${SECRET_KEY}` 는 워크스페이스 시크릿에서 해석됩니다.

12.5 메모리 영속 스코핑

`StoreBackend` 의 `namespace` 함수로 메모리 범위를 정합니다. `CompositeBackend` 가 `/memories/` 만 `StoreBackend` 로 라우팅하고 나머지는 `StateBackend` 휘발성으로 두는 것이 기본 패턴입니다.

User-scoped (권장 기본값)

```
from deepagents import create_deep_agent
from deepagents.backends import CompositeBackend, StateBackend, StoreBackend

agent = create_deep_agent(
    model="google_genai:gemin-3.5-flash",
    backend=CompositeBackend(
        default=StateBackend(),
        routes={
            "/memories/": StoreBackend(
                namespace=lambda rt: (
                    rt.server_info.assistant_id,
                    rt.server_info.user.identity,
                ),
            ),
        },
    ),
    system_prompt=(
        "대화 시작 시 /memories/instructions.txt를 읽어라. "
        "지속 가치 있는 인사이트는 갱신한다."
    ),
)
```

Assistant-scoped / Organization-scoped

같은 assistant를 쓰는 모든 사용자가 공유(assistant-scoped)하거나 조직 전체에 공유(org-scoped)하는 구성도 가능합니다. 조직 공유는 read-only로 두는 것이 원칙입니다.

! WARNING

Prompt injection 경고: 공유 메모리는 프롬프트 인젝션 벡터입니다. 사용자가 조작 가능한 범위에 쓰기 권한을 주지 말 것. 자세한 정책은 Part IV ch15(권한 관리) 참고.

12.6 실행 격리 – Sandbox

호스트 파일시스템과 네트워크를 그대로 노출하지 말고 샌드박스를 씁니다.

Thread-scoped sandbox (가장 흔한 패턴)

```
from daytona import CreateSandboxFromSnapshotParams, Daytona
from deepagents import create_deep_agent
from langchain_core.runnables import RunnableConfig
from langchain_daytona import DaytonaSandbox

client = Daytona()

async def agent(config: RunnableConfig):
    thread_id = config["configurable"]["thread_id"]
    try:
        sandbox = await client.find_one(labels={"thread_id": thread_id})
    except Exception:
        sandbox = await client.create(
            CreateSandboxFromSnapshotParams(
                labels={"thread_id": thread_id},
                auto_delete_interval=3600, # TTL
            )
        )
    return create_deep_agent(
        model="google_genai:gemin-3.5-flash",
        backend=DaytonaSandbox(sandbox=sandbox),
    )
```

Assistant-scoped sandbox

모든 스레드가 한 샌드박스를 공유해 도구 체인 캐시와 설치물을 보존해야 할 때 씁니다.

12.7 내구성·Async I/O

매 스텝 체크포인트

LangSmith Deployments는 자동으로 체크포인트를 붙입니다. 매 스텝 상태가 저장되므로:

- **Indefinite interrupt:** HITL 승인 대기가 며칠이어도 정확히 멈춘 자리에서 재개
- **Time travel:** 임의 체크포인트 시점으로 되감아 분기 실행
- **Audit trail:** 결제·관리자 동작 같은 민감 연산 직전 상태 감사

```
await agent.ainvoke(
    {"messages": [...]},
    config={"configurable": {"thread_id": "thread-abc"}},
)
```

Async I/O

LLM 앱은 I/O 바운드입니다. 비동기 도구와 미들웨어 혹은 (`abefore_agent`, `astream`)을 쓰면 처리량이 늘어납니다.

12.8 레이트 리밋과 비용 제어

```
from deepagents import create_deep_agent
from langchain.agents.middleware import (
    ModelCallLimitMiddleware,
    ToolCallLimitMiddleware,
)

agent = create_deep_agent(
    model="google_genai:gemin-3.5-flash",
    middleware=[
        ModelCallLimitMiddleware(run_limit=50),
        ToolCallLimitMiddleware(run_limit=200),
    ],
)
```

`run_limit` 는 `invoke` 한 번마다 리셋, `thread_limit` 는 스레드 수명 동안 누적합니다. 폭주·무한 루프가 비용 폭탄이 되기 전에 잘라냅니다.

12.9 에러 핸들링 3층

분류	예시	전략	미들웨어
Transient	타임아웃, rate limit, 네트워크 일시 실패	자동 재시도 (backoff)	<code>ModelRetryMiddleware</code> , <code>ToolRetryMiddleware</code>
Recoverable	잘못된 도구 인자, 파싱 실패	모델에 피드백 → 재시도	도구 래퍼 에러 메시지
Human required	권한 없음, 불명확한 요청	에이전트 일시 정지	HITL <code>interrupt_on</code>

```
from langchain.agents.middleware import (
    ModelRetryMiddleware,
    ModelFallbackMiddleware,
    ToolRetryMiddleware,
)

agent = create_deep_agent(
    model="google_genai:gemin-3.5-flash",
    middleware=[
        ModelRetryMiddleware(max_retries=3, backoff_factor=2.0, initial_delay=1.0),
        ModelFallbackMiddleware("gpt-5.4"),
    ],
)
```



```
ToolRetryMiddleware(
  max_retries=2,
  tools=["search", "fetch_url"],
  retry_on=(TimeoutError, ConnectionError),
),
],
)
```

12.10 데이터 프라이버시와 실시간 프론트엔드

PIIMiddleware

이메일·카드 번호·주민번호 등 PII를 입출력 경계에서 가공합니다. 전략은 `redact` (삭제), `mask` (마스킹), `hash` (해시), `block` (차단)이며 커스텀 detector도 등록할 수 있습니다. PII가 로그에 섞이기 전 입력 단계에서 처리하는 게 핵심입니다. LangSmith 트레이스에도 마스킹된 채로 기록됩니다.

`useStream` **혹**

`@langchain/react` 의 `useStream` **혹**으로 실시간 스트리밍·재연결·히스토리 로드를 한 번에 처리합니다.

```
import { useStream } from "@langchain/react";

function App() {
  const stream = useStream<typeof agent>({
    apiUrl: "https://your-deployment.langsmith.dev",
    assistantId: "agent",
    reconnectOnMount: true,
    fetchStateHistory: true,
  });
}
```

서브에이전트를 많이 띄우는 Deep Agent는 서브그래프 이벤트까지 스트리밍해 UI에 서브에이전트 진행 카드를 보여줍니다(`streamSubgraphs: true`).

12.11 프로덕션 체크리스트

- [] `langgraph.json` 에 `graph·env`가 선언되어 있다
- [] 사용자별 authz 핸들러가 `thread/store`에 적용된다
- [] 외부 서비스 토큰은 Agent Auth/Proxy로 관리되고 코드에 하드코딩되지 않는다
- [] `/memories/` 는 `StoreBackend` 로 라우팅되고 스코프가 명시적이다(`user/assistant/org`)
- [] 공유 메모리는 `read-only`이거나 쓰기 권한이 엄격히 제한된다
- [] 호스트 파일시스템·셀에 직접 노출되지 않고 샌드박스를 경유한다
- [] `ModelCallLimitMiddleware` / `ToolCallLimitMiddleware` 로 비용 상한이 설정되어 있다
- [] 재시도·fallback·HITL이 전부 구성되어 있다
- [] PII가 입출력 경계에서 마스킹된다
- [] 프론트엔드가 `reconnectOnMount` + `fetchStateHistory` 를 사용한다

핵심 정리

- Thread·User·Assistant 세 추상 위에서 10대 운영 영역을 독립적으로 도입
- `langgraph.json` + LangSmith Deployments가 체크포인트·Store·인증·관측을 자동 프로비저닝
- 메모리 스코프는 user-scoped 기본, 공유 스코프는 반드시 read-only
- 예러는 3층(Transient/Recoverable/Human)으로 분류해 미들웨어와 HITL로 대응
- PII는 모델 입력 단계와 트레이스 전송 단계 두 레이어에서 이중 방어

13 컨텍스트 엔지니어링

Write big, read selective

Deep Agent가 모델의 제한된 컨텍스트 창 안에서 무엇을 보여주고, 언제 오프로드하고, 어떻게 다시 끌어올지 결정하는 설계를 정리합니다. 에이전트가 길어질수록 품질이 떨어지거나 요약에 삼켜질 때, 또는 서브에이전트·스킬·메모리를 한 파이프라인에서 정리하고 싶을 때 이 장을 씁니다.

학습 목표

LEARNING OBJECTIVES

- 컨텍스트 엔지니어링의 4층 구조(Layered / Dynamic / Compression / Retrieval)를 이해한다
- 시스템 프롬프트의 9단계 자동 합성 순서를 안다
- 자동 offloading(>20k 토큰)과 자동 summarization(85%)의 트리거를 구분한다
- `@dynamic_prompt` 와 `context_schema` 로 런타임 컨텍스트를 전파한다
- 서브에이전트 격리와 `/memories/` 라우팅으로 컨텍스트 오염을 방지한다

13.1 4층 구조

Deep Agents의 컨텍스트 엔지니어링은 네 층으로 구성됩니다.

1. **Layered input context** – system prompt, AGENTS.md, SKILL.md, tool 설명이 정해진 순서로 합성
2. **Dynamic / runtime context** – `@dynamic_prompt` 와 `ToolRuntime` 으로 요청 시점 데이터 주입
3. **Automatic compression** – offloading(>20k 토큰 → 디스크), summarization(85% → 구조화 요약)
4. **Filesystem-centric retrieval** – `read_file` / `grep` / 서브에이전트 격리로 선별적 재주입

각 층은 독립적으로 끄고 켤 수 있고, 기본값만으로도 장시간 대화의 90%는 처리하도록 맞춰져 있습니다.

13.2 Layered input context – 시스템 프롬프트 9단계 합성

시스템 프롬프트는 다음 순서로 자동 합성됩니다.

1. Custom system prompt (사용자가 준 지시)

2. Base agent prompt (planning/filesystem/subagent 기반 지시)
3. To-do list prompt
4. Memory prompt (AGENTS.md , 설정 시 항상 로드)
5. Skills prompt (스킬 위치와 frontmatter 목록만)
6. Filesystem prompt (도구 문서)
7. Subagent prompt (위임 가이드)
8. Middleware prompts (커스텀 추가분)
9. Human-in-the-loop prompt

AGENTS.md: 항상 로드되는 반영구 컨텍스트

프로젝트 규약·사용자 선호처럼 모든 대화에 적용해야 하는 내용을 넣습니다.

```
from deepagents import create_deep_agent

agent = create_deep_agent(
    model="google_genai:gemin-3.5-flash",
    memory=["/project/AGENTS.md", "~/.deepagents/preferences.md"],
)
```

항상 로드되므로 최소한으로 유지합니다. 상세 워크플로우는 SKILL.md로 빼는 것이 원칙입니다.

13.3 @dynamic_prompt — 요청 시점 데이터 주입

정적 문자열로 충분하지 않고 요청 시점 데이터(사용자 역할, 조직 ID, 현재 시각, store 내용 등)로 지시를 바꿔야 할 때 씁니다. 미들웨어는 `request.runtime.context` 와 `request.runtime.store` 를 읽어 동적 프롬프트를 만듭니다. 동적 프롬프트는 미들웨어 레이어에서 결정되고, 도구는 별도 설정 없이 `ToolRuntime` 으로 런타임 값을 받습니다.

13.4 Progressive skill loading

스킬은 2단계로 로드됩니다.

- **Startup:** SKILL.md 의 frontmatter(이름·설명)만 읽음 → 수백 토큰 수준
- **On demand:** 사용자 요청이 해당 스킬에 매칭될 때만 본문 전체 로드

```
agent = create_deep_agent(
    model="google_genai:gemin-3.5-flash",
    skills=["/skills/research/", "/skills/web-search/"],
)
```

스킬 수십 개를 등록해도 초기 토큰 비용은 frontmatter 분량뿐입니다.

13.5 Runtime context propagation

`context_schema` 로 선언된 컨텍스트는 에이전트뿐 아니라 모든 서브에이전트·도구에 자동 전달됩니다.

```

from dataclasses import dataclass
from deepagents import create_deep_agent
from langchain.tools import tool, ToolRuntime

@dataclass
class Context:
    user_id: str
    api_key: str

@tool
def fetch_user_data(query: str, runtime: ToolRuntime[Context]) -> str:
    """현재 사용자 데이터를 조회한다."""
    user_id = runtime.context.user_id
    return f>Data for user {user_id}: {query}</pre>
</div>
<div data-bbox="129 386 896 419" data-label="Text">
<p>이 구조 덕분에 도구는 메시지에 API 키를 담지 않고도 런타임 값을 읽고, 서브에이전트도 같은 값을 자동으로 상속합니다.</p>
</div>
<div data-bbox="152 448 206 463" data-label="Section-Header">
<h4>➡ TIP</h4>
</div>
<div data-bbox="152 474 870 506" data-label="Text">
<p>도구 문서화 품질이 곧 컨텍스트. 도구 선택은 모델이 설명만 보고 합니다. docstring + Args 섹션을 구체적으로 써야 토큰 낭비가 줄어듭니다. @tool(parse_docstring=True) 로 Args 자동 파싱을 활성화하세요.</p>
</div>
<div data-bbox="130 555 489 576" data-label="Section-Header">
<h2>13.6 자동 offloading (>20k 토큰)</h2>
</div>
<div data-bbox="129 597 546 612" data-label="Text">
<p>컨텍스트가 커지면 자동으로 두 가지 오프로딩이 발동됩니다.</p>
</div>
<div data-bbox="147 619 902 688" data-label="List-Group">
<ul>
<li>- Input offloading: 큰 파일 write/edit 결과는 컨텍스트가 85% 용량을 넘으면 실제 내용 대신 file pointer reference로 치환
<li>- Result offloading: 도구 응답이 20,000 토큰을 초과하면 스토리지에 저장되고, 컨텍스트에는 파일 경로 + 첫 10라인 프리뷰만 남음
</ul>
</div>
<div data-bbox="129 694 859 728" data-label="Text">
<p>큰 결과는 일단 디스크에 쓰고, 필요할 때 read_file / grep 으로 부분만 끌어옵니다. 이것이 filesystem-centric 설계의 핵심입니다.</p>
</div>
<div data-bbox="130 758 555 778" data-label="Section-Header">
<h2>13.7 자동 summarization (85% 트리거)</h2>
</div>
<div data-bbox="129 800 896 851" data-label="Text">
<p>오프로드할 것이 더 없는데 컨텍스트가 여전히 크면 구조화 요약이 발동됩니다. LLM이 현재 대화를 Session intent, Artifacts created, Next steps 세 요소로 압축합니다. 이 요약이 기존 대화 히스토리를 대체하고 에이전트는 계속 진행합니다.</p>
</div>
<div data-bbox="310 867 347 883" data-label="Text">
<table>
<tr>
<th>항목</th>
<th>기본값</th>
</tr>
</table>
</div>
<div data-bbox="417 867 470 883" data-label="Text">
</div>
<div data-bbox="863 917 902 932" data-label="Page-Footer">377</div>
```

트리거	모델 <code>max_input_tokens</code> 의 85%
최근 보존	10%
Fallback	170,000 토큰 트리거, 최근 6 메시지 보존
즉시 트리거	<code>ContextOverflowError</code> 발생 시

수동 summarization 도구

자동 트리거 외에 에이전트가 명시적으로 요약을 호출하게 하고 싶을 때 미들웨어를 추가합니다.

```
from deepagents import create_deep_agent
from deepagents.backends import StateBackend
from deepagents.middleware.summarization import create_summarization_tool_middleware

backend = StateBackend
model = "google_genai:gemin-3.5-flash"

agent = create_deep_agent(
    model=model,
    middleware=[create_summarization_tool_middleware(model, backend)],
)
```

스트리밍 시 요약 토큰은 UI에 노출하지 않도록 필터링할 수 있습니다

```
( metadata.get("lc_source") == "summarization" ).
```

13.8 Subagent isolation

서브에이전트는 자체 컨텍스트에서 실행되고 슈퍼바이저에게는 최종 리포트 하나만 돌려줍니다. 중간 도구 호출과 검색 결과는 서브에이전트 컨텍스트에 남아 슈퍼바이저 창이 깨끗하게 유지됩니다.

```
research_subagent = {
    "name": "researcher",
    "description": "특정 주제 리서치 수행",
    "system_prompt": (
        "You are a research assistant.\n"
        "IMPORTANT: Return only the essential summary (under 500 words).\n"
        "Do NOT include raw search results or detailed tool outputs."
    ),
    "tools": [web_search],
}
```

서브에이전트 프롬프트에 “최종 요약만 반환해라”를 명시하는 것이 컨텍스트 오염 방지의 첫 방어선입니다.

13.9 CompositeBackend로 `/memories/` 라우팅

일부 경로만 영속 스토어, 나머지는 휘발성 상태입니다. 에이전트는 같은 `write_file` / `read_file` 로 접근하지만 내부적으로는 다른 백엔드가 처리합니다.

```
from deepagents import create_deep_agent
from deepagents.backends import CompositeBackend, StateBackend, StoreBackend
```

```

from langgraph.store.memory import InMemoryStore

def make_backend(runtime):
    return CompositeBackend(
        default=StateBackend(runtime),
        routes={"/memories/": StoreBackend(runtime)},
    )

agent = create_deep_agent(
    model="google_genai:gemin-3.5-flash",
    store=InMemoryStore(),
    backend=make_backend,
    system_prompt=(
        "When users tell you their preferences, save them to "
        "/memories/user_preferences.txt so you remember them in future conversations."
    ),
)

```

13.10 Filesystem-centric 아키텍처

Deep Agents의 모든 컨텍스트 압축 전략은 결국 하나로 모입니다.

TIP

“Write big, read selective.” 큰 산출물은 디스크에 쓰고, 다시 쓸 땐 `read_file` / `grep` 으로 필요한 부분만 주입한다.

이 구조가 있기에:

- 도구 결과가 20k 토큰을 넘어도 컨텍스트는 파일 레퍼런스 + 10라인 프리뷰만 유지
- 서버에이전트는 원하는 만큼 탐색해도 슈퍼바이저 창에 영향 없음
- 메모리는 항상 로드가 아니라 필요할 때 `read_file("/memories/...")` 로 가져옴
- 스킬은 frontmatter만 보이고 본문은 on-demand

13.11 세 가지 패턴

패턴 1: 장시간 리서치

- `AGENTS.md` 는 조사 양식·인용 규칙만 (최소)
- 스킬 3 10개 등록, frontmatter만 노출
- 주제별 서버에이전트가 병렬로 탐색 → 최종 요약만 회수
- 큰 검색 결과는 자동 파일 오프로드 → `grep` 으로 다시 쿼리

패턴 2: 개인화 어시스턴트

- `/memories/` 라우팅으로 사용자별 선호 누적
- `@dynamic_prompt` 로 사용자 역할별 톤·가드레일 주입
- `context_schema` 에 `user_id` / `org_id` 선언해 전 서버에이전트에 전파
- 공유 정책은 `/policies/` (read-only) 네임스페이스

패턴 3: 비용 최적화

- 자동 summarization 트리거를 기본(85%)보다 조금 낮추기
- 서버에이전트 system_prompt에 “under 500 words” 명시
- 도구 응답이 큰 도구는 요약본 + 파일 포인터를 반환하도록 래핑

13.12 주의사항

-
- AGENTS.md 과적재 금지 – 항상 로드되므로 5KB 넘기면 체감됩니다
 - 스킴 frontmatter는 설명이 전부 – 매칭이 안 되면 본문이 영영 안 읽힘
 - Summarization은 파괴적 – 중요한 중간 산출물은 먼저 파일로 저장
 - 서버에이전트가 raw dump를 반환하면 슈퍼바이저 창이 오히려 더 빨리 터짐
 - 도구 설명 품질이 곧 토큰 효율 – 모호한 docstring은 재시도 비용을 낡음

핵심 정리

-
- 컨텍스트 엔지니어링은 Layered / Dynamic / Compression / Retrieval 4층 구조
 - 자동 offloading(20k 결과)과 자동 summarization(85%)이 기본 압축 메커니즘
 - @dynamic_prompt 와 context_schema 가 런타임 컨텍스트를 전 그래프에 전파
 - 서버에이전트 격리와 /memories/ 라우팅이 컨텍스트 오염 방지의 두 축
 - 전체 설계는 “Write big, read selective” 원칙 – 큰 산출물은 디스크, 재주입은 선별적

14 스트리밍

subgraphs=True + v2 네임스페이스

메인 에이전트는 물론 서브에이전트 내부 이벤트까지 실시간으로 UI에 노출하고, 커스텀 진행률 이벤트까지 방출하는 방법을 다룹니다. 장시간 실행 에이전트의 진행 상황을 사용자에게 보여주거나 서브에이전트 카드 UI를 만들 때 이 장을 씁니다.

학습 목표

LEARNING OBJECTIVES

- `subgraphs=True` 와 `version="v2"` 조합으로 서브에이전트 내부까지 관찰한다
- 네임스페이스(`ns`) 튜플로 메인/서브 이벤트를 라우팅한다
- `updates` · `messages` · `custom` 세 모드를 동시에 구독한다
- `get_stream_writer` 로 커스텀 진행률 이벤트를 방출한다
- 서브에이전트 라이프사이클(Pending/Running/Complete) 3신호를 UI 카드에 매핑한다

14.1 세 가지 핵심

Deep Agents 스트리밍의 핵심은 세 가지입니다.

- `subgraphs=True` + `version="v2"` - 서브에이전트 내부 이벤트까지 관찰
- 네임스페이스(`ns`) - 이벤트가 어느 서브그래프에서 왔는지 튜플로 구분
- Stream mode 조합 - `updates` (스텝), `messages` (토큰/도구), `custom` (커스텀 이벤트)

v2 포맷은 모든 청크를 `{"type", "ns", "data"}` 통일 구조로 돌려줍니다. v1의 중첩 튜플보다 라우팅이 단순합니다.

14.2 기본 사용: 서브에이전트 스트리밍 활성화

```
for chunk in agent.stream(
    {"messages": [{"role": "user", "content": "Research quantum computing advances"}]},
    stream_mode="updates",
    subgraphs=True,
    version="v2",
```

```

):
    print(chunk)

```

`subgraphs=True` 를 주지 않으면 서브에이전트 내부 동작은 슈퍼바이저 레벨에서 `task` 도구 호출 결과로만 보입니다. UI에서는 “블랙박스에서 뭔가 하다가 결과가 나오는” 경험이 됩니다.

14.3 네임스페이스로 이벤트 라우팅

모든 v2 청크는 `ns` 필드에 튜플을 담고 있으며, 이 튜플이 이벤트 발생 위치를 식별합니다.

튜플	의미
<code>()</code>	메인 에이전트 이벤트
<code>("tools:abc123",)</code>	메인 에이전트의 <code>task</code> 도구 호출로 스폰된 서브에이전트
<code>("tools:abc123", "model_request:def456")</code>	해당 서브에이전트 내부의 model request 노드

서브에이전트 이벤트 여부 판정:

```
is_subagent = any(segment.startswith("tools:") for segment in chunk["ns"])
```

메인 대화 패널과 서브에이전트 카드를 UI에서 나눌 때 이 판정이 기본 분기점입니다.

14.4 Stream mode: `updates`

노드 스텝 단위로 “지금 어느 단계를 실행 중인가”를 보여줍니다. 서브에이전트 라이프사이클 추적에 특히 유용합니다.

```

for chunk in agent.stream(
    {"messages": [...]},
    stream_mode="updates",
    subgraphs=True,
    version="v2",
):
    if chunk["type"] == "updates":
        for node_name, data in chunk["data"].items():
            print(f"Step: {node_name}")

```

14.5 Stream mode: `messages`

LLM 토큰을 조각 단위로 받습니다. 도구 호출이 발생하면 각 청크에 `tool_call_chunks` 가 붙어 도구 이름과 args 가 스트리밍으로 쌓입니다.

```

for chunk in agent.stream(
    {"messages": [...]},
    stream_mode="messages",

```

```

subgraphs=True,
version="v2",
):
    if chunk["type"] == "messages":
        token, metadata = chunk["data"]
        if token.tool_call_chunks:
            for tc in token.tool_call_chunks:
                print(f"Tool: {tc['name']}, Args: {tc.get('args')}")
        else:
            print(token.content, end="")

```

토큰 단위 출력과 도구 호출 조립을 같은 루프에서 처리할 수 있습니다.

14.6 커스텀 진행률 이벤트

도구 내부에서 스트리밍 작성자를 꺼내 임의의 구조체를 방출합니다. 업로드 진행률, 처리 건수, 중간 상태 등을 UI에 내보낼 때 씁니다.

```

from langchain.tools import tool
from langgraph.config import get_stream_writer

@tool
def analyze_data(topic: str) -> str:
    """주제에 대한 데이터를 분석한다."""
    writer = get_stream_writer()
    writer({"status": "starting", "progress": 0})
    # ... 분석 작업 ...
    writer({"status": "complete", "progress": 100})
    return "done"

```

수신 측에서는 `stream_mode="custom"` 을 구독합니다.

```

for chunk in agent.stream(
    {"messages": [...]},
    stream_mode="custom",
    subgraphs=True,
    version="v2",
):
    if chunk["type"] == "custom":
        print(chunk["data"])

```

14.7 다중 모드 동시 구독

리스트로 넘기면 루프 하나에서 세 종류 이벤트를 모두 받습니다.

```

for chunk in agent.stream(
    {"messages": [...]},
    stream_mode=["updates", "messages", "custom"],
    subgraphs=True,
    version="v2",
):
    t = chunk["type"]
    if t == "updates":
        # 스텝 진행
        ...
    elif t == "messages":

```

```
# 토큰 / 도구 호출
...
elif t == "custom":
    # 진행률 / 커스텀 이벤트
    ...
```

프로덕션 UI에서는 보통 이 세 모드를 함께 구독해 각자의 패널에 분배합니다.

14.8 서브에이전트 라이프사이클 추적

메인 에이전트 관점에서 서브에이전트는 세 이벤트로 식별합니다.

1. **Pending** — 메인인 `name="task"` 인 `tool_call`을 방출한 순간
2. **Running** — `("tools:<id>",)` 네임스페이스에서 첫 이벤트가 도착한 순간
3. **Complete** — 해당 `task` 호출의 `ToolMessage`가 메인 `tools` 노드에 돌아온 순간

UI 카드 상태를 이 세 신호에 매핑하면 “어떤 서브에이전트가 지금 탐색 중인지” 바로 보입니다.

14.9 v2 통합 포맷

모든 청크는 다음 세 필드를 가집니다.

```
{
  "type": "updates" | "messages" | "custom",
  "ns": tuple,
  "data": Any,
}
```

v1의 중첩 튜플 포맷은 `type` 마다 자료 구조가 달라 분기가 복잡했습니다. v2는 프론트엔드 라우팅을 `(type, ns prefix)` 하나로 단순화합니다.

14.10 프론트엔드 연동

React/Vue/Svelte/Angular는 `@langchain/react` 등의 `useStream` 훅으로 이 스트림을 자동 처리합니다. 서브 에이전트 카드 렌더링, 재연결, 히스토리 로드까지 내장돼 있습니다.

```
import { useStream } from "@langchain/react";

const stream = useStream<typeof agent>({
  apiUrl: "https://your-deployment.langsmith.dev",
  assistantId: "agent",
  reconnectOnMount: true,
  fetchStateHistory: true,
});

stream.submit(
  { messages: [{ type: "human", content: text }] },
  {
    streamSubgraphs: true,
    config: { recursionLimit: 10000 },
  },
);
```

14.11 주의사항

- `subgraphs=True` 없으면 서브에이전트 내부는 보이지 않는다 - UI 진행 카드에 필수
- `version="v2"` 를 명시하라 - 레거시 포맷으로 떨어지면 네임스페이스 처리가 달라짐
- 커스텀 이벤트 스키마는 고정 - 도구마다 제각각 쓰면 UI 라우팅이 깨짐 (예: `{"status", "progress", "message"}` 공통 필드)
- 요약(summarization) 토큰은 노출 여부를 결정해야 함 (`metadata.get("lc_source") = "summarization"` 로 필터링)
- 재시도 이벤트도 스트리밍됨 - `ModelRetryMiddleware` 가 붙어 있으면 실패-재시도 플로우가 그대로 보임으로 UI에서 집계

핵심 정리

- v2 포맷 + `subgraphs=True` 가 서브에이전트 관찰의 입구
- `ns` 튜플 prefix로 메인/서브 이벤트를 라우팅
- `updates / messages / custom` 세 모드를 리스트로 동시 구독하는 것이 프로덕션 기본
- `get_stream_writer` 로 도구에서 임의 진행률 이벤트 방출
- Pending/Running/Complete 3신호를 UI 카드 상태에 매핑

15 권한 관리

FilesystemPermission · first-match-wins

Deep Agents의 built-in 파일시스템 도구(`ls`, `read_file`, `glob`, `grep`, `write_file`, `edit_file`)에 선언적 allow/deny 규칙을 적용해 경로 기반 접근 제어를 강제합니다. 프롬프트 인젝션 방어, 읽기 전용 에이전트, 특정 디렉터리만 허용하는 워크스페이스 격리를 구성할 때 이 장을 씁니다.

학습 목표

LEARNING OBJECTIVES

- FilesystemPermission 의 3요소(`operations`, `paths`, `mode`)를 이해한다
- first-match-wins 평가 규칙과 기본값(allow)을 안다
- 읽기 전용·워크스페이스 격리·민감 파일 보호·read-only memory 4가지 패턴을 구분한다
- 서브에이전트 permission 상속과 전면 대체 규칙을 인지한다
- permission이 닿지 않는 커스텀 도구·MCP·샌드박스 셀의 보완 전략을 안다

15.1 적용 범위

`permissions` 는 built-in 파일시스템 도구에만 적용되는 경로 기반 규칙입니다. 다음은 우회됩니다:

- 커스텀 도구
- MCP 도구
- 샌드박스의 `execute` 셀 명령

보안 경계를 permission만으로 완성할 수 없습니다. 샌드박스에서 임의 셀 명령이 가능한 구성에서는 CompositeBackend의 라우팅 제약과 함께 설계해야 합니다. 커스텀 도구의 추가 검증·감사는 backend policy hooks가 담당합니다.

15.2 평가 규칙: first-match-wins

규칙은 리스트 순서대로 평가되고, `operations` 와 `paths` 가 현재 호출과 매치되는 첫 번째 규칙이 결과를 결정합니다. 어떤 규칙에도 매치되지 않으면 기본값은 **allow**입니다. 그러므로 구체적인 deny/allow를 앞에 두고 일반적인 fallback을 뒤에 놓아야 합니다.

15.3 규칙 3요소

각 `FilesystemPermission` 은 세 필드를 갖습니다.

필드	값	설명
operations	<code>list["read" "write"]</code>	<code>read = ls / read_file / glob / grep</code> , <code>write = write_file / edit_file</code>
paths	<code>list[str]</code>	글로브 패턴, <code>**</code> 재귀, <code>{a,b}</code> 선택자 지원
mode	<code>"allow" "deny"</code>	기본 <code>"allow"</code>

15.4 패턴 1: 읽기 전용 에이전트

모든 쓰기를 전역 차단합니다. 조사·감사·리포트 생성 에이전트에 알맞습니다.

```
from deepagents import create_deep_agent, FilesystemPermission

agent = create_deep_agent(
    model=model,
    backend=backend,
    permissions=[
        FilesystemPermission(
            operations=["write"],
            paths=["/**"],
            mode="deny",
        ),
    ],
)
```

15.5 패턴 2: 워크스페이스 격리

`/workspace/` 아래만 허용하고 나머지는 전면 거부합니다. `first-match-wins`이므로 `allow`를 먼저, `deny`는 `catch-all`로 뒤에 둡니다.

```
agent = create_deep_agent(
    model=model,
    backend=backend,
    permissions=[
        FilesystemPermission(
            operations=["read", "write"],
            paths=["/workspace/**"],
            mode="allow",
        ),
        FilesystemPermission(
            operations=["read", "write"],
            paths=["/**"],
            mode="deny",
        ),
    ],
)
```

15.6 패턴 3: 특정 파일만 보호

`/workspace/.env` 와 예제 디렉터리는 건드리지 못하게 막고, 나머지 `/workspace/` 는 자유롭게 허용합니다.

```
agent = create_deep_agent(
    model=model,
    backend=backend,
    permissions=[
        # 가장 구체적인 deny를 최상단에
        FilesystemPermission(
            operations=["read", "write"],
            paths=["/workspace/.env", "/workspace/examples/**"],
            mode="deny",
        ),
        FilesystemPermission(
            operations=["read", "write"],
            paths=["/workspace/**"],
            mode="allow",
        ),
        FilesystemPermission(
            operations=["read", "write"],
            paths=["/**"],
            mode="deny",
        ),
    ],
)
```

15.7 패턴 4: Read-only memory / policies

`/memories/` · `/policies/` 경로의 쓰기만 막습니다. 읽기는 자유입니다. 공유 메모리와 조직 정책이 프롬프트 인젝션으로 오염되는 것을 막는 기본 방어선입니다.

```
from deepagents import create_deep_agent, FilesystemPermission
from deepagents.backends import CompositeBackend, StateBackend, StoreBackend

agent = create_deep_agent(
    model=model,
    backend=CompositeBackend(
        default=StateBackend(),
        routes={
            "/memories/": StoreBackend(
                namespace=lambda rt: (rt.server_info.user.identity,),
            ),
            "/policies/": StoreBackend(
                namespace=lambda rt: (rt.context.org_id,),
            ),
        },
    ),
    permissions=[
        FilesystemPermission(
            operations=["write"],
            paths=["/memories/**", "/policies/**"],
            mode="deny",
        ),
    ],
)
```

메모리와 정책은 애플리케이션 코드로만 갱신하고, 에이전트는 읽기만 합니다.

15.8 Subagent 상속

기본 동작은 부모 에이전트의 permissions가 서브에이전트에 그대로 상속되는 것입니다. 서브에이전트 스펙에 permissions 를 주면 부모 규칙을 완전히 대체합니다(부분 오버라이드 아님). 대체할 때는 catch-all deny까지 직접 넣어야 안전합니다.

```
agent = create_deep_agent(
    model=model,
    backend=backend,
    permissions=[...parent_rules...],
    subagents=[
        {
            "name": "auditor",
            "description": "읽기 전용 코드 리뷰어",
            "system_prompt": "Review the code for issues.",
            "permissions": [
                # 쓰기 전역 차단
                FilesystemPermission(
                    operations=["write"], paths=["/*"], mode="deny",
                ),
                # /workspace만 읽기 허용
                FilesystemPermission(
                    operations=["read"], paths=["/workspace/*"], mode="allow",
                ),
                # 나머지 읽기 차단
                FilesystemPermission(
                    operations=["read"], paths=["/*"], mode="deny",
                ),
            ],
        },
    ],
)
```

감사 전용 서브에이전트는 읽기 권한만 주고 메인 에이전트는 넓은 권한을 유지하는 식으로 분리할 수 있습니다.

15.9 CompositeBackend 제약: sandbox-default

CompositeBackend 의 default가 sandbox일 때, 모든 permission path는 선언된 route prefix 안에 있어야 합니다. 샌드박스가 execute 도구로 임의 셸 명령을 돌릴 수 있기 때문입니다. 경로 기반 규칙은 셸 수준 파일 접근을 막지 못하므로, 라우팅 외부 경로에 permission을 다는 것은 가짜 안전감입니다.

```
from deepagents import create_deep_agent, FilesystemPermission
from deepagents.backends import CompositeBackend

composite = CompositeBackend(
    default=sandbox,
    routes={"/memories/": memories_backend},
)

# 유효: /memories/ 라우트 안에 있음
agent = create_deep_agent(
    model=model,
    backend=composite,
    permissions=[
        FilesystemPermission(
            operations=["write"], paths=["/memories/*"], mode="deny",
        ),
    ],
)
```

알려진 route 바깥 경로에 규칙을 걸면 `NotImplementedError` 가 발생합니다. 샌드박스 내부 파일시스템을 제어하려면 permission이 아니라 샌드박스 구성 자체(허용 바이너리, 네트워크 정책, 볼륨 마운트 범위)로 풀어야 합니다.

15.10 프롬프트 인젝션 방어 관점

공유 메모리, 조직 정책, 외부에서 주입되는 문서는 모두 인젝션 벡터입니다. 대응 계층은 다음과 같습니다.

1. **Read-only 강제:** `/memories/**`, `/policies/**` 에 write deny (패턴 4)
2. **Workspace 격리:** 에이전트가 건드릴 수 있는 경로를 `/workspace/**` 로 한정
3. **민감 파일 보호:** `.env` 류는 패턴 3처럼 개별 deny
4. **Subagent scope 축소:** 감사/조회 전용 서브에이전트는 read-only로 별도 구성
5. **커스텀 도구/샌드박스 경계:** permission이 닿지 않으므로 policy hook + 샌드박스 정책으로 보완

15.11 Permissions vs backend policy hooks

용도	사용 대상
Built-in FS 도구의 경로 기반 allow/deny	<code>permissions</code>
커스텀 검증(데이터 유효성, 로깅, rate limit)	backend policy hooks
커스텀 도구 / MCP 도구 제어	도구 자체 래퍼 / middleware
샌드박스 내부 파일·네트워크 통제	샌드박스 설정 (허용 바이너리·네트워크 정책)

Permission은 선언적 간이 규칙, policy hook은 로직이 필요한 통제입니다. 역할을 나눠 씁니다.

15.12 주의사항

- **first-match-wins:** 규칙 순서가 결과를 바꿈. 더 구체적인 deny/allow를 위로
- **매치 실패 = allow:** 의도치 않게 허용되는 걸 막으려면 마지막에 catch-all deny를 두는 것이 안전
- **operations 누락:** 하나의 규칙에 `"read"` 만 넣으면 쓰기는 별도 규칙이 없는 한 기본 허용
- **Subagent 오버라이드는 전면 대체:** 부분 수정 불가. 부모 규칙 중 유지할 것은 복사해 넣어야 함
- **Permission은 완전한 보안 경계가 아니다:** 샌드박스·네트워크·시크릿 관리와 함께 설계되어야 의미가 있음

핵심 정리

- `FilesystemPermission` 의 3요소(`operations` / `paths` / `mode`)와 first-match-wins 평가 규칙이 기본
- 4가지 전형 패턴(읽기 전용 / 워크스페이스 격리 / 특정 파일 보호 / read-only memory) 조합으로 대부분 대응
- Subagent permission 오버라이드는 전면 대체이므로 catch-all deny 복사에 주의

- CompositeBackend + sandbox-default 구성에서는 route prefix 밖 permission이 `NotImplementedError`
- Permission은 선언적 간이 규칙이며 커스텀 도구·MCP·샌드박스 셸은 policy hook과 샌드박스 정책으로 보완

P A R T



고급 패턴

Advanced Agent Patterns

이 파트에서 다루는 내용

- 0. v0 → v1 마이그레이션
 - 1. 미들웨어 심화
- 2. 멀티에이전트: Subagents
- 3. 멀티에이전트: Handoffs & Router
- 4. 컨텍스트 엔지니어링 & 메모리 심화
 - 5. Agentic RAG
 - 6. SQL 에이전트 심화
- 7. 데이터 분석 에이전트
 - 8. 보이스 에이전트
- 9. 프로덕션 배포

00 v0 → v1 마이그레이션 가이드

LangChain/LangGraph v0에서 v1으로 전환할 때 알아야 할 브레이킹 체인지와 코드 매핑을 다룹니다. v1은 단순한 버전 업데이트가 아니라, 에이전트 개발 패러다임의 근본적인 전환입니다. 기존의 `create_react_agent` 는 `create_agent` 로, `AgentExecutor` 는 LangGraph 네이티브로, `ConversationBufferMemory` 는 `InMemorySaver` 기반 체크포인트로 대체되었습니다. 이 장에서는 각 변경점의 배경과 마이그레이션 전략을 체계적으로 살펴봅니다.

v0에서 v1으로의 전환은 크게 세 가지 축을 중심으로 이루어집니다. 첫째, 패키지 구조가 모놀리식에서 모듈형으로 분리되었습니다. 둘째, 에이전트 생성 API가 통합되어 `create_agent` 하나로 에이전트를 정의합니다. 셋째, 횡단 관심사(cross-cutting concerns) 처리가 개별 훅에서 미들웨어 시스템으로 일원화되었습니다. 이 세 축을 이해하면 마이그레이션의 큰 그림이 명확해집니다.

학습 목표

LEARNING OBJECTIVES

- v1 패키지 구조 변경과 import 경로를 이해한다
- `create_react_agent` → `create_agent` 마이그레이션을 수행한다
- 미들웨어 기반 동적 프롬프트, 상태 관리, 컨텍스트 주입을 적용한다
- 표준 콘텐츠 블록과 구조화된 출력 전략을 활용한다

0.1 패키지 구조 변경

v0의 `langchain` 패키지는 Chains, Retrievers, Hub, 인덱싱 등 수십 개의 모듈을 포함하는 모놀리식 구조였습니다. 이 때문에 설치 용량이 크고 의존성 충돌이 잦았으며, 에이전트 개발에 불필요한 레거시 코드가 함께 설치되는 문제가 있었습니다. 예를 들어, 단순한 ReAct 에이전트를 만들기 위해 `langchain` 을 설치하면 Retriever, Indexing, Hub 등 사용하지 않는 모듈까지 함께 설치되어 가상환경 크기가 불필요하게 커지고, 패키지 간 버전 충돌이 발생하는 경우가 빈번했습니다.

v1에서는 이를 해결하기 위해 `langchain` 네임스페이스를 에이전트 구축에 필수적인 5개 핵심 모듈로 대폭 축소하고, 나머지는 `langchain-classic` 으로 분리했습니다. 이러한 분리는 “에이전트 퍼스트” 철학을 반영한 것으로, LangChain이 범용 LLM 프레임워크에서 에이전트 전용 프레임워크로 방향을 전환했음을 의미합니다.

특히 메시지·도구 import 경로가 바뀐 점을 주의해야 합니다. v0에서는 `langchain_core.messages` · `langchain_core.tools` 에서 가져왔지만, v1에서는 동일한 심볼을 `langchain.messages` · `langchain.tools` 로 재노출합니다. 코어 모듈은 그대로 유지되지만, v1 코드 베이스에서는 단축 경로를 표준으로 사용하는 것이 권장됩니다.

```
# v1 표준 import
from langchain.messages import HumanMessage, AIMessage, ToolMessage
from langchain.tools import tool, ToolRuntime, BaseTool

# v0 호환 import (`langchain_core.*`)도 계속 동작하지만 v1 코드에서는 위 경로를 선호
from langchain_core.messages import HumanMessage as _LegacyHumanMessage
```

v1 모듈	역할	주요 API
langchain.agents	에이전트 생성 및 상태 관리	create_agent , AgentState
langchain.messages	메시지 타입과 콘텐츠 블록	HumanMessage , AIMessage , content_blocks
langchain.tools	도구 정의	\@tool 데코레이터, BaseTool
langchain.chat_models	모델 초기화	init_chat_model
langchain.embeddings	임베딩 유틸리티	임베딩 모델 래퍼

레거시 코드 마이그레이션 — langchain-classic

기존에 langchain 패키지에서 사용하던 Chains, Retrievers, Hub, 인덱싱 API 등은 모두 langchain-classic이라는 별도 패키지로 분리되었습니다. 기존 코드를 유지해야 하는 경우 pip install langchain-classic 으로 설치 후 import 경로를 변경합니다:

```
# v0 - 기존 방식
from langchain.chains import LLMChain
from langchain.retrievers import MultiQueryRetriever
from langchain import hub

# v1 - langchain-classic으로 이전
from langchain_classic.chains import LLMChain
from langchain_classic.retrievers import MultiQueryRetriever
from langchain_classic import hub
```

이 분리를 통해 v1의 langchain 패키지는 에이전트 빌딩에만 집중하는 경량 구조가 되었으며, 레거시 기능은 독립적으로 유지보수됩니다. langchain-classic 은 기존 프로젝트의 호환성을 보장하면서도, 새 프로젝트에서는 최소한의 의존성으로 시작할 수 있게 합니다.

TIP

마이그레이션 시 기존 import 경로를 일괄 치환하려면 sed 또는 IDE의 전체 검색/치환 기능을 활용하세요.
from langchain.chains → from langchain_classic.chains 처럼 패턴이 일정합니다.

! WARNING

langchain 과 langchain-classic 을 동시에 설치할 수 있지만, 두 패키지의 내부 모듈이 이름 충돌을 일으킬 수 있습니다. 가상환경을 분리하거나, 점진적으로 langchain-classic 의존을 제거하는 것을 권장합니다.

패키지 구조가 정리되었으니, 이제 가장 중요한 변경점인 에이전트 생성 API를 살펴보겠습니다.

0.2 에이전트 생성 API 변경

v0에서 에이전트를 만들려면 `langgraph.prebuilt` 에서 `create_react_agent` 를 import하고, 별도의 `AgentExecutor` 를 감싸는 이중 구조가 필요했습니다. 이 구조는 에이전트 정의와 실행이 분리되어 있어 코드가 장황해지고, 미들웨어나 상태 관리를 추가할 때 복잡도가 급격히 증가했습니다.

v1에서는 이를 `langchain.agents.create_agent` 단일 함수로 통합하여, 에이전트 생성부터 실행까지 하나의 API 로 처리합니다. 파라미터명도 직관적으로 변경되었습니다. `prompt` 는 `system_prompt` 로, 혹은 시스템은 통합 `middleware` 로 일원화되었습니다. 이 통합 덕분에 에이전트 정의가 선언적(declarative)이 되어 코드 가독성이 크게 향상됩니다.

다음 코드는 v1에서 에이전트를 생성하는 기본 패턴을 보여줍니다. v0 대비 import 경로, 함수명, 파라미터명이 어떻게 변경되었는지 주석으로 비교합니다.

```
from dotenv import load_dotenv
load_dotenv(override=True)

from langchain_openai import ChatOpenAI
from langchain.tools import tool
from langchain.agents import create_agent

model = ChatOpenAI(model="gpt-5.4")

@tool
def add(a: int, b: int) -> int:
    """두 수를 더합니다."""
    return a + b

# v0: from langgraph.prebuilt import create_react_agent
# v1: from langchain.agents import create_agent
agent = create_agent(
    model=model,
    tools=[add],
    system_prompt="당신은 수학 어시스턴트입니다.", # v0: prompt=
)
print("\u2713 v1 에이전트 생성 완료")
```

✓ v1 에이전트 생성 완료

위 코드에서 주목할 점은 `create_agent` 가 에이전트를 즉시 실행 가능한 상태로 반환한다는 것입니다. v0처럼 `AgentExecutor` 로 감쌀 필요가 없으므로, `agent.invoke(...)` 또는 `agent.stream(...)` 을 바로 호출할 수 있습니다.

주요 변경점

v0	v1	비고
<code>from langgraph.prebuilt import create_react_agent</code>	<code>from langchain.agents import create_agent</code>	import 경로 + 함수명
<code>prompt=</code>	<code>system_prompt=</code>	파라미터명
ToolNode 지원	미지원	함수/BaseTool/dict만
<code>pre_hooks</code> , <code>post_hooks</code>	<code>middleware=[]</code>	통합 미들웨어
Pydantic/dataclass 상태	TypedDict only	상태 스키마

에이전트 생성 API가 단순해졌으므로, 이제 에이전트의 상태를 어떻게 정의하는지 살펴보겠습니다. 상태 스키마는 에이전트가 실행 중에 추적하는 데이터의 구조를 결정하는 중요한 요소입니다.

0.3 상태 스키마 — TypedDict only

v1에서 커스텀 상태는 반드시 `TypedDict` 기반 `AgentState` 를 상속해야 합니다. v0에서는 Pydantic `BaseModel` 이나 `@dataclass` 로도 상태를 정의할 수 있었지만, v1은 직렬화 일관성과 LangGraph 그래프 엔진과의 호환성을 위해 `TypedDict` 로 단일화했습니다. `TypedDict` 는 런타임 오버헤드가 없는 순수한 타입 힌팅 메커니즘이므로, 그래프 엔진이 상태를 직렬화/역직렬화할 때 예측 가능한 동작을 보장합니다.

기존에 Pydantic 상태를 사용하던 코드는 `TypedDict` 로 전환해야 하며, 검증 로직이 필요한 경우 미들웨어의 `before_model` 훅에서 수동으로 처리할 수 있습니다.

! WARNING

`TypedDict` 는 런타임 검증을 제공하지 않으므로, 상태 필드의 타입 안전성은 개발자 책임입니다. `mypy` 나 `pyright` 같은 정적 분석 도구를 병행 사용하는 것을 권장합니다.

상태 스키마가 단순해진 대신, v1은 런타임 컨텍스트라는 새로운 개념을 도입하여 실행 시점의 데이터를 체계적으로 관리합니다.

0.4 런타임 컨텍스트 주입 (신규)

v1에서는 `context_schema` 와 `context` 파라미터를 통해 불변 런타임 데이터를 에이전트에 전달할 수 있습니다. 이는 사용자 ID, 역할, 세션 정보 등 요청마다 달라지지만 실행 중에는 변하지 않는 데이터를 에이전트와 도구에 안전하게 전달하는 패턴입니다.

작동 방식:

1. `@dataclass` 로 컨텍스트 스키마를 정의합니다.
2. `create_agent(context_schema=...)` 로 스키마를 등록합니다.
3. `agent.invoke(..., context=ContextInstance(...))` 로 런타임에 값을 전달합니다.
4. 도구에서는 `ToolRuntime[ContextType]` 파라미터로 컨텍스트에 접근합니다.

컨텍스트는 에이전트 상태(`AgentState`)와 달리 도구 호출 간에 변경되지 않는 읽기 전용 데이터입니다. 상태는 에이전트 루프 중 업데이트되지만, 컨텍스트는 `invoke` 호출 시 고정됩니다.

➡ TIP

`context_schema` 는 멀티테넌트 에이전트에서 특히 유용합니다. 사용자 ID, 조직 정보, 권한 수준 등을 컨텍스트로 전달하면, 도구가 사용자별 맞춤 동작을 수행할 수 있습니다.

런타임 컨텍스트가 정적 데이터를 다룬다면, 동적 프롬프트는 실행 시점의 조건에 따라 에이전트의 지시사항을 변경하는 메커니즘입니다.

0.5 동적 프롬프트 — 미들웨어 방식 (신규)

v0에서는 `system_prompt` 가 에이전트 생성 시점에 고정되어, 런타임 조건에 따라 프롬프트를 변경할 수 없었습니다. 이 제약 때문에 개발자들은 프롬프트 내에 조건 분기 텍스트를 미리 넣어두거나, 에이전트를 여러 개 만들어 상황별로 전환하는 우회 방법을 사용해야 했습니다.

v1의 `@dynamic_prompt` 미들웨어는 매 모델 호출 직전에 상태와 컨텍스트를 분석하여 시스템 프롬프트를 동적으로 생성합니다. 이를 통해 사용자 역할, 시간대, 이전 도구 호출 결과 등에 따라 에이전트의 행동 지침을 실시간으로 조정할 수 있습니다. 예를 들어, 관리자 사용자에게는 삭제 권한에 대한 지침을 추가하고, 일반 사용자에게는 읽기 전용 지침만 포함하는 식입니다.

미들웨어는 에이전트 실행 루프의 6개 지점에 훅을 노출합니다. `before_agent` (루프 시작), `before_model` (매 모델 호출 직전), `wrap_model_call` (모델 호출 감싸기), `after_model` (매 모델 응답 직후), `wrap_tool_call` (도구 호출 감싸기), `after_agent` (루프 종료) 순으로 실행되며, 6개 훅 모두 동기/비동기(`a` 프리픽스) 두 형태로 작성할 수 있습니다. 자세한 동작은 Chapter 1에서 다룹니다.

TIP

동적 프롬프트 미들웨어는 Chapter 1에서 상세히 다룹니다. 여기서는 v0에서 고정 프롬프트를 사용하던 코드가 v1에서 어떻게 유연해졌는지만 이해하면 충분합니다.

동적 프롬프트와 마찬가지로, 도구 에러 처리도 v1에서는 미들웨어 패턴으로 통합되었습니다.

0.6 도구 에러 처리 — `@wrap_tool_call` (신규)

v0에서는 `handle_tool_errors=True` 파라미터로 도구 에러를 자동 처리했지만, 에러 메시지 형식이나 재시도 로직을 커스터마이징하기 어려웠습니다. 에러가 발생하면 기본 에러 메시지가 그대로 LLM에 전달되어, LLM이 에러의 맥락을 파악하지 못하는 경우가 많았습니다.

v1은 `@wrap_tool_call` 미들웨어로 도구 호출 전체를 감싸서, 에러 포매팅, 조건부 재시도, 폴백 로직 등을 자유롭게 구현할 수 있습니다. `wrap_tool_call` 은 도구 호출의 요청(request)과 핸들러(handler)를 받아, 핸들러 실행 전후로 임의의 로직을 삽입합니다. 핸들러를 호출하지 않으면 도구 실행 자체를 차단할 수도 있고, 여러 번 호출하여 재시도를 구현할 수도 있습니다.

다음 코드는 도구 실행 중 발생하는 모든 예외를 포착하여, LLM이 이해할 수 있는 형식의 `ToolMessage` 로 변환하는 에러 핸들링 미들웨어 예시입니다.

```
from langchain.agents.middleware import wrap_tool_call
from langchain.messages import ToolMessage

@wrap_tool_call
def handle_errors(request, handler):
    try:
        return handler(request)
    except Exception as e:
        return ToolMessage(
            content=f"도구 오류: {e}",
            tool_call_id=request.tool_call["id"],
        )

agent = create_agent(
    model=model,
```

```
tools=[add],
middleware=[handle_errors],
)
print("\u2713 예러 핸들링 미들웨어 적용")
```

✓ 예러 핸들링 미들웨어 적용

예러 처리 미들웨어를 통해 도구 실행의 안정성을 확보했습니다. 이제 에이전트가 생성하는 응답의 형식에 대한 변경점을 살펴보겠습니다.

0.7 표준 콘텐츠 블록 & 구조화된 출력 (신규)

v1에서 메시지는 프로바이더 무관한 `content_blocks` 를 지원합니다. 이전에는 OpenAI와 Anthropic의 응답 형식이 달라서 프로바이더 전환 시 파싱 로직을 수정해야 했지만, `content_blocks` 는 텍스트, 이미지, 도구 호출 등을 표준화된 블록으로 표현하여 프로바이더 간 이식성을 보장합니다. 이는 멀티 프로바이더 전략(예: OpenAI를 주 모델로, Anthropic을 폴백으로 사용)을 채택할 때 특히 중요합니다.

구조화된 출력은 `ToolStrategy` (도구 호출 기반)와 `ProviderStrategy` (네이티브) 두 가지로 분리되었습니다. `ToolStrategy` 는 모든 모델에서 동작하는 범용 방식이고, `ProviderStrategy` 는 OpenAI의 JSON mode나 Anthropic의 `tool_use` 같은 네이티브 기능을 활용하여 더 높은 정확도를 제공합니다.

```
from pydantic import BaseModel
from langchain.agents.structured_output import ToolStrategy, ProviderStrategy

class Weather(BaseModel):
    location: str
    temperature_c: float

# 1) 도구 호출 기반 - 모든 모델에서 동작
agent_tool = create_agent(
    model=model,
    tools=[add],
    response_format=ToolStrategy(schema=Weather),
)

# 2) 프로바이더 네이티브 - OpenAI JSON mode, Anthropic tool_use 등 활용
agent_provider = create_agent(
    model=model,
    tools=[add],
    response_format=ProviderStrategy(schema=Weather),
)
```

기존 OpenAI Responses API 동작이 필요하다면 `output_version="v0"` 으로 명시적으로 옵트아웃합니다. v1 신규 모델은 기본적으로 `output_version="v1"` 로, 표준 콘텐츠 블록과 `content_blocks` 프로퍼티가 자동 활성화됩니다.

```
from langchain_openai import ChatOpenAI

# 표준 블록 형식 (v1 기본값) - content_blocks / .text 프로퍼티 활성화
model_v1 = ChatOpenAI(model="gpt-5.4", output_version="v1")

# 레거시 호환 - v0 동작 복구
model_v0 = ChatOpenAI(model="gpt-5.4", output_version="v0")
```

! WARNING

`content_blocks` 를 사용할 때, 기존에 `message.content` 를 문자열로 가정하던 코드는 수정이 필요합니다. v1에서 `content` 는 문자열 또는 블록 리스트일 수 있으므로, `.text` 프로퍼티를 사용하여 텍스트를 추출하는 것이 안전합니다.

에이전트 응답 형식이 변경되었으므로, 스트리밍 처리 방식도 함께 달라졌습니다.

0.8 스트리밍 변경

v1에서 스트리밍 API의 두 가지 핵심 변경점이 있습니다. 에이전트 노드명이 `"agent"` 에서 `"model"` 로 변경되었고, 텍스트 접근 방식이 메서드 호출에서 프로퍼티 접근으로 바뀌었습니다. 이 두 변경은 상호 의존적이므로, 반드시 함께 적용해야 합니다.

항목	v0	v1
에이전트 노드명	<code>"agent"</code>	<code>"model"</code>
<code>.text</code>	메서드 <code>.text()</code>	프로퍼티 <code>.text</code>

스트리밍 변경 외에도, v1에는 여러 세부적인 브레이킹 체인지가 포함되어 있습니다. 각 항목은 개별적으로는 작지만, 놓치면 런타임 에러의 원인이 되므로 체계적으로 확인해야 합니다.

0.9 기타 브레이킹 체인지

변경	설명
Python 3.10+	모든 LangChain 패키지가 Python 3.10 이상을 요구합니다. 3.9 이하는 미지원됩니다.
반환 타입	채팅 모델의 반환 타입이 <code>BaseMessage</code> 에서 <code>AIMessage</code> 로 고정되었습니다.
OpenAI Responses API	메시지 콘텐츠가 기본적으로 표준 블록 형식입니다. <code>output_version="v0"</code> 으로 이전 동작을 복구할 수 있습니다.
Anthropic max_tokens	기본값이 1024에서 모델별 자동 설정으로 변경되었습니다.
AIMessage.example	<code>example</code> 파라미터가 제거되었습니다. <code>additional_kwargs</code> 를 사용하세요.
AIMessageChunk	<code>chunk_position</code> 속성이 추가되었습니다 (마지막 청크에 <code>'last'</code> 값).
<code>.text</code> 프로퍼티	<code>.text()</code> 메서드가 <code>.text</code> 프로퍼티로 변경되었습니다.
파일 인코딩	파일이 기본적으로 UTF-8 인코딩으로 열립니다.


TIP

스트리밍 코드를 마이그레이션할 때는 노드명("agent" → "model")과 `.text` 접근 방식(메서드 → 프로퍼티)을 동시에 변경해야 합니다. 둘 중 하나만 변경하면 런타임 에러가 발생합니다.

지금까지 살펴본 모든 변경점을 체크리스트로 정리합니다. 마이그레이션을 진행할 때 이 목록을 순서대로 확인하면서 작업하면 누락을 방지할 수 있습니다.

요약 — 마이그레이션 체크리스트

- [] Python 3.10+ 확인
- [] `create_react_agent` → `create_agent` 변경
- [] `prompt=` → `system_prompt=` 변경
- [] 상태 스키마를 `TypedDict` 기반 `AgentState` 로 전환
- [] `pre_hooks` / `post_hooks` → `middleware=[]`
- [] `ToolNode` → 함수/BaseTool로 교체
- [] `.text()` → `.text` 프로퍼티
- [] 스트리밍 노드명 "agent" → "model" 확인
- [] 레거시 import를 `langchain-classic` 으로 이전

v1 마이그레이션의 핵심은 에이전트 중심 아키텍처로의 전환입니다. Chains와 Retrievers 중심의 v0 패러다임에서, 미들웨어와 상태 관리 중심의 v1 패러다임으로 사고방식을 바꾸는 것이 중요합니다. 다음 장에서는 v1의 가장 강력한 신기능인 미들웨어 시스템을 심층적으로 다룹니다.

01 미들웨어 시스템 심화

v1 최대 신기능

미들웨어는 LangGraph v1에서 가장 주목할 만한 신기능으로, 에이전트 실행 루프의 각 단계에 모니터링, 변환, 거버넌스 로직을 비침습적으로 삽입할 수 있게 합니다. 이 장에서는 7가지 빌트인 미들웨어의 동작 원리를 파악하고, 데코레이터 및 클래스 기반으로 커스텀 미들웨어를 작성하는 방법을 심층적으로 다룹니다. 다중 미들웨어 조합 시 실행 순서 제어까지 익히면, 프로덕션 수준의 에이전트 파이프라인을 설계할 수 있습니다.

학습 목표

LEARNING OBJECTIVES

- 에이전트 루프에서 미들웨어 혹은 역할과 실행 흐름을 이해한다
- 7가지 빌트인 미들웨어의 설정과 실전 사용법을 익힌다
- 데코레이터/클래스 기반 커스텀 미들웨어를 작성할 수 있다
- 다중 미들웨어 조합 시 실행 순서를 정확히 예측할 수 있다

1.1 환경 설정

웹 프레임워크(Express, Django 등)의 미들웨어가 HTTP 요청/응답 파이프라인에 로직을 삽입하듯, LangGraph v1의 미들웨어는 에이전트 실행 루프에 횡단 관심사(cross-cutting concerns)를 삽입합니다. 횡단 관심사란 로깅, 인증, 에러 처리, PII 마스킹처럼 여러 모듈에 걸쳐 공통으로 필요하지만, 핵심 비즈니스 로직과는 분리되어야 하는 기능을 말합니다.

미들웨어 없이 이러한 기능을 구현하면, 도구 함수마다 로깅 코드를 넣고, 에이전트 호출마다 에러 핸들링을 감싸야 합니다. 미들웨어를 사용하면 이 로직을 에이전트 코어 로직과 분리하여, 단일 책임 원칙을 유지하면서도 강력한 프로덕션 파이프라인을 구성할 수 있습니다. `create_agent` 함수의 `middleware` 파라미터에 미들웨어 인스턴스 리스트를 전달하여 사용합니다.

다음 코드로 이 장에서 사용할 환경을 준비합니다.

```
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI

load_dotenv()

model = ChatOpenAI(model="gpt-5.4")
```

환경 설정이 완료되었으므로, 미들웨어가 에이전트 루프의 어느 지점에 개입하는지 전체 아키텍처를 살펴보겠습니다.

1.2 미들웨어 아키텍처 개요

에이전트 루프는 모델 호출 → 도구 선택 → 도구 실행 → 종료 판단의 반복 사이클입니다. 이 사이클이 한 번 돌 때마다 에이전트는 사용자의 요청에 한 걸음 더 가까워집니다. 미들웨어는 이 사이클의 각 단계에 **훅(hook)**을 삽입하여 세밀한 제어를 가능하게 합니다. 훅은 특정 시점에 자동으로 호출되는 콜백 함수로, 개발자는 훅에 원하는 로직만 구현하면 됩니다.

훅 유형

훅 유형	실행 시점	대표 용도
<code>before_model</code>	모델 호출 직전	프롬프트 수정, 로깅, 상태 업데이트
<code>after_model</code>	모델 응답 직후	응답 검증, 가드레일, 결과 변환
<code>before_agent</code>	에이전트 실행 시작 시	초기화, 전처리
<code>after_agent</code>	에이전트 실행 종료 시	정리, 후처리
<code>wrap_model_call</code>	모델 호출 감싸기	재시도, 캐싱, 폴백
<code>wrap_tool_call</code>	도구 호출 감싸기	도구 재시도, 감사 로그, 에러 핸들링

두 가지 훅 스타일

미들웨어 훅은 동작 방식에 따라 두 가지 스타일로 나뉩니다. 이 구분을 이해하는 것이 미들웨어 설계의 기초입니다.

- **Node-style 훅** (`before_*`, `after_*`): 순차적으로 실행되며, 로깅/검증/상태 업데이트에 적합합니다. 파이프라인의 특정 지점에서 “관찰”하거나 “검증”하는 용도입니다.
- **Wrap-style 훅** (`wrap_*`): 핸들러(`next_fn`) 호출 여부를 제어할 수 있습니다. 0회(차단), 1회(통과), 다회(재시도) 호출이 가능하여 재시도, 캐싱, 변환 로직에 적합합니다. Python의 데코레이터 패턴과 유사하게, 원래 함수를 감싸서 전후 처리를 추가합니다.

미들웨어는 에이전트의 핵심 로직을 변경하지 않고도 모니터링, 변환, 신뢰성, 거버넌스 등 횡단 관심사(cross-cutting concerns)를 깔끔하게 분리할 수 있게 해줍니다. 7가지 빌트인 미들웨어(`SummarizationMiddleware`, `HumanInTheLoopMiddleware`, `ModelCallLimitMiddleware`, `ToolCallLimitMiddleware`, `ModelFallbackMiddleware`, `PIIMiddleware`, `LLMToolSelectorMiddleware`)가 제공되며, 데코레이터 또는 클래스 기반으로 커스텀 미들웨어를 작성할 수도 있습니다.

TIP

빌트인 미들웨어는 프로덕션에서 반복적으로 필요한 패턴을 사전 구현한 것입니다. 커스텀 미들웨어를 작성하기 전에, 빌트인으로 해결 가능한지 먼저 확인하세요. 대부분의 프로덕션 요구사항은 빌트인의 조합만으로 충족됩니다.

다음 코드는 미들웨어를 에이전트에 적용하는 기본 패턴을 보여줍니다. `middleware` 파라미터에 인스턴스 리스트를 전달합니다.

이제 각 빌트인 미들웨어를 하나씩 살펴보겠습니다. 먼저 장시간 대화에서 가장 빈번하게 필요한

`SummarizationMiddleware` 부터 시작합니다.

```
from langchain.agents import create_agent
from langchain.agents.middleware import (
    SummarizationMiddleware,
    HumanInTheLoopMiddleware,
)

agent = create_agent(
    model="gpt-5.4", tools=[],
    middleware=[
        SummarizationMiddleware(model="gpt-5.4-mini", trigger=("messages", 50)),
        HumanInTheLoopMiddleware(interrupt_on={}),
    ],
)
```

1.3 SummarizationMiddleware

대화가 길어져 컨텍스트 윈도우를 초과할 때 자동으로 이전 대화를 요약하여 압축합니다. LLM의 컨텍스트 윈도우는 유한하므로, 수십 턴 이상의 대화에서는 오래된 메시지가 윈도우 밖으로 밀려나 맥락을 잃는 문제가 발생합니다. `SummarizationMiddleware` 는 이 문제를 트리거 조건에 따라 자동으로 해결합니다.

장시간 실행되는 대화, 다중 턴 대화, 전체 대화 맥락 보존이 필요한 애플리케이션에 필수적입니다.

주요 파라미터

파라미터	설명	예시
<code>model</code>	요약 생성에 사용할 경량 모델 (비용 절감)	"gpt-5.4-mini"
<code>trigger</code>	요약 트리거 조건	(<code>"tokens", 4000</code>), (<code>"messages", 50</code>), (<code>"fraction", 0.8</code>)
<code>keep</code>	요약 후 유지할 최근 컨텍스트	(<code>"messages", 20</code>)
<code>token_counter</code>	커스텀 토큰 카운팅 함수	선택적
<code>summary_prompt</code>	커스텀 요약 프롬프트 템플릿	선택적

`trigger` 는 토큰 수, 메시지 수, 윈도우 비율 중 하나를 기준으로 설정할 수 있으며, 조건에 도달하면 `keep` 에 지정된 최근 메시지를 제외한 나머지를 요약문으로 교체합니다.

TIP

요약 모델로 `gpt-5.4-mini` 같은 경량 모델을 사용하면 비용을 절감할 수 있습니다. 요약의 목적은 핵심 맥락 보존이므로 메인 모델만큼의 추론 능력은 필요하지 않습니다.

```
from langchain.agents.middleware import SummarizationMiddleware

summarizer = SummarizationMiddleware(
    model="gpt-5.4-mini",
    trigger=("tokens", 4000),
    keep=("messages", 20),
)
```

대화 컨텍스트 관리와 더불어, 프로덕션 에이전트에서 빠질 수 없는 요소가 인간 감독입니다. 다음 미들웨어는 고 위험 작업에 대한 인간 승인 게이트를 제공합니다.

1.4 HumanInTheLoopMiddleware

자율적으로 동작하는 에이전트가 위험한 작업을 수행하기 전에 “잠깐, 이거 해도 되나요?”라고 물어보는 메커니즘입니다. 고위험 도구 호출 전에 에이전트 실행을 중단하고 인간 승인을 기다립니다. 데이터베이스 쓰기, 금융 거래, 이메일 전송 등 고위험 작업이나 컴플라이언스 워크플로우에서 인간 감독이 필요한 경우에 사용합니다.

`checkpointer` 필수 — 중단 후 상태를 복원하기 위해 체크포인트가 반드시 필요합니다. 체크포인트 없이 HITL을 사용하면 중단 시점의 상태가 유실되어, 승인 후 에이전트가 처음부터 다시 실행됩니다.

! WARNING

HITL 미들웨어는 에이전트 실행을 완전히 중단합니다. 웹 애플리케이션에서 사용할 때는 중단 상태를 클라이언트에 적절히 전달하고, 사용자 응답을 비동기로 처리하는 UI 설계가 필요합니다.

결정 유형

결정	설명	사용 방법
approve	도구 호출 승인 및 실행	<code>Command(resume="approve")</code>
edit	도구 인자 수정 후 실행	<code>Command(resume={"type": "edit", "args": {...}})</code>
reject	도구 호출 거부	<code>Command(resume={"type": "reject", "reason": "..."})</code>

`interrupt_on` 딕셔너리에서 각 도구별로 승인 정책을 설정합니다. `False` 로 설정하면 해당 도구는 중단 없이 실행됩니다.

```
from langchain.agents.middleware import HumanInTheLoopMiddleware
from langgraph.checkpoint.memory import InMemorySaver

hitl = HumanInTheLoopMiddleware(
    interrupt_on={
        "send_email": {"allowed_decisions": ["approve", "edit", "reject"]},
        "read_email": False,
    }
)
```

```
agent = create_agent(
    model="gpt-5.4", tools=[],
    checkpointer=InMemorySaver(),
```



```
        middleware=[hitl],
    )
```

HITL이 인간의 판단을 에이전트 루프에 통합한다면, 다음 두 미들웨어는 에이전트의 자원 소비를 프로그래밍 방식으로 제한합니다. 에이전트가 무한 루프에 빠지거나 예상치 못한 고비용 경로를 탈 때 자동으로 제동을 거는 안전장치입니다.

1.5 ModelCallLimitMiddleware & ToolCallLimitMiddleware

에이전트가 복잡한 문제를 풀 때, 의도치 않게 무한 루프에 빠지거나 수백 번의 모델/도구 호출을 수행하여 예상치 못한 비용이 발생할 수 있습니다. 호출 제한 미들웨어는 이러한 상황에서 자동으로 제동을 거는 안전장치입니다.

ModelCallLimitMiddleware

에이전트가 모델을 호출하는 횟수를 제한합니다. 폭주하는 에이전트 방지, 프로덕션 비용 제어, 테스트 시 호출 예산 관리에 사용됩니다. `thread_limit` 은 동일 스레드의 전체 생명주기에 걸친 제한이고, `run_limit` 은 단일 `invoke()` 호출에 대한 제한입니다. 두 제한을 조합하면 세밀한 비용 관리가 가능합니다.

ToolCallLimitMiddleware

도구 호출 횟수를 전역적으로 또는 특정 도구별로 제한합니다. 비용이 높은 외부 API 호출 제한, 검색/DB 쿼리 빈도 제어, 특정 도구의 레이트 리밋 적용에 유용합니다.

공통 파라미터

파라미터	설명
<code>thread_limit</code>	전체 스레드(모든 <code>invoke</code>)에서의 최대 호출 수
<code>run_limit</code>	단일 <code>invoke</code> 실행에서의 최대 호출 수
<code>exit_behavior</code>	"end" (정상 종료), "error" (예외 발생), "continue" (에러 메시지와 함께 계속 – ToolCallLimit 전용)

ToolCallLimitMiddleware는 추가로 `tool_name` 파라미터를 받아 특정 도구에만 제한을 적용할 수 있습니다. 예를 들어, 외부 API 호출 도구에는 엄격한 제한을, 로컬 계산 도구에는 느슨한 제한을 적용하는 차별화된 정책을 구현할 수 있습니다.

다음 코드들은 모델 호출 제한과 도구 호출 제한을 각각 설정하는 예시입니다.

```
from langchain.agents.middleware import ModelCallLimitMiddleware

model_limit = ModelCallLimitMiddleware(
    thread_limit=10,
    run_limit=5,
    exit_behavior="end",
)
```

```
from langchain.agents.middleware import ToolCallLimitMiddleware

# 전역 제한
global_tool_limit = ToolCallLimitMiddleware(thread_limit=20, run_limit=10)

# 특정 도구 제한
search_limit = ToolCallLimitMiddleware(
    tool_name="search",
    thread_limit=5, run_limit=3,
    exit_behavior="continue",
)
```

호출 제한은 비용과 안정성을 보호합니다. 그러나 모델 자체가 장애를 일으킬 수도 있습니다. 다음 미들웨어는 모델 레벨의 장애 복구를 자동화합니다.

TIP

개발/테스트 환경에서는 `run_limit` 을 낮게 설정(예: 3~5)하여 의도치 않은 폭주를 빠르게 감지하세요. 프로덕션에서는 태스크 복잡도에 맞게 적절히 높여야 합니다.

호출 제한으로 비용과 안정성을 보호했습니다. 그러나 모델 서비스 자체가 장애를 일으키는 경우도 대비해야 합니다. 다음 미들웨어는 모델 레벨의 장애 복구를 자동화합니다.

1.6 ModelFallbackMiddleware

주 모델 실패 시 대체 모델 체인으로 자동 전환합니다. 프로덕션 장애 대응, 비용 최적화(비싼 모델 → 저렴한 모델 폴백), 멀티 프로바이더 중복성(OpenAI + Anthropic 등) 확보에 유용합니다. 단일 프로바이더에 의존하면 해당 서비스 장애 시 전체 시스템이 중단되므로, 프로덕션 환경에서는 폴백 전략이 필수적입니다.

생성자에 폴백 모델을 순서대로 전달하면, 주 모델 호출이 실패할 때 지정된 순서로 대체 모델을 시도합니다. 모든 폴백이 실패하면 최종 에러가 발생합니다. 폴백 체인은 비용 순서(고성능 → 저비용)로 구성하는 것이 일반적입니다.

```
from langchain.agents.middleware import ModelFallbackMiddleware

# gpt-5.4 실패 -> gpt-5.4-mini -> claude
fallback = ModelFallbackMiddleware(
    "gpt-5.4-mini",
    "claude-sonnet-4-6",
)
```

모델 안정성 확보 다음으로 중요한 것은 데이터 보안입니다. 에이전트가 처리하는 메시지에 개인정보가 포함될 수 있으며, 이를 로그나 외부 API에 노출하면 법적 문제가 발생합니다.

1.7 PIIMiddleware

에이전트가 처리하는 메시지에는 이메일 주소, 신용카드 번호, 전화번호 등 개인 식별 정보(PII)가 포함될 수 있습니다. 이러한 정보가 로그, 외부 API, 또는 LLM 프로바이더에 노출되면 개인정보보호법(GDPR, CCPA 등) 위반이 될 수 있습니다. `PIIMiddleware` 는 개인 식별 정보를 자동 탐지하고 설정된 전략에 따라 처리합니다. 의료/금융 컴플라이언스, 고객 서비스 에이전트의 로그 세정, 민감한 사용자 데이터 처리 등에 필수적입니다.

빌트인 PII 타입

email, credit_card, ip, mac_address, url

처리 전략

전략	동작	예시 (이메일)
block	예외 발생 – PII 발견 시 실행 중단	에러 발생
redact	[REDACTED_TYPE] 으로 교체	[REDACTED_EMAIL]
mask	부분 마스킹	u***\@example.com
hash	결정적 해싱	a1b2c3d4...

적용 범위

- apply_to_input : 사용자 입력 메시지 검사
- apply_to_output : AI 응답 메시지 검사
- apply_to_tool_results : 도구 실행 결과 검사

커스텀 탐지기

빌트인 PII 타입(email, credit_card, ip, mac_address, url)만으로는 도메인 특화된 민감 정보를 처리할 수 없습니다. 커스텀 탐지기는 세 가지 방식으로 만들 수 있으며, 복잡도에 따라 적절한 방식을 선택합니다:

1. 정규식 문자열: 간단한 패턴 매칭 (가장 간단)
2. 컴파일된 정규식 (`re.compile`): 복잡한 정규식 (중간)
3. 함수: 검증 로직이 필요한 고급 탐지 (반환: `list[dict]` – `text`, `start`, `end` 키 포함, 가장 유연)

다음 코드들은 각 방식의 예시를 순서대로 보여줍니다.

```
from langchain.agents.middleware import PIIMiddleware

email_pii = PIIMiddleware("email", strategy="redact", apply_to_input=True)
card_pii = PIIMiddleware("credit_card", strategy="mask", apply_to_input=True)
```

```
# 커스텀 탐지기: 정규식 문자열
api_key_pii = PIIMiddleware(
    "api_key",
    detector=r"sk-[a-zA-Z0-9]{32}",
    strategy="block",
)
```

```
import re

# 커스텀 탐지기: 컴파일된 정규식
phone_pii = PIIMiddleware(
    "phone_number",
    detector=re.compile(r"\+?\d{1,3}[\s.-]? \d{3,4}[\s.-]? \d{4}"),
    strategy="mask",
)
```

```
# 커스텀 탐지기: 함수 (SSN 예시)
def detect_ssn(content: str) -> list[dict]:
    matches = []
    for m in re.finditer(r"\d{3}-\d{2}-\d{4}", content):
        first = int(m.group(0)[:3])
        if first not in [0, 666] and not (900 <= first <= 999):
            matches.append({"text": m.group(0), "start": m.start(), "end": m.end()})
    return matches

ssn_pii = PIIMiddleware("ssn", detector=detect_ssn, strategy="hash")
```

PII 미들웨어로 데이터 보안을 확보했다면, 마지막 빌트인 미들웨어는 에이전트의 도구 선택 정확도를 높이는 최적화입니다.

! WARNING

PII 탐지는 정규식 기반이므로 100% 정확하지 않을 수 있습니다. 높은 보안 수준이 필요한 시스템에서는 PII 미들웨어를 방어 계층 중 하나로 사용하고, 별도의 DLP(Data Loss Prevention) 솔루션과 병행하는 것을 권장합니다.

PII 미들웨어로 데이터 보안을 확보했다면, 마지막 빌트인 미들웨어는 에이전트의 도구 선택 정확도를 높이는 최적화입니다.

1.8 LLMToolSelectorMiddleware

도구가 10개 이상일 때, 경량 LLM이 사용자 쿼리를 분석하여 관련 도구만 선별합니다. 도구 수가 많아지면 LLM이 모든 도구의 description을 처리해야 하므로 입력 토큰이 급증하고, 잘못된 도구를 선택할 확률도 높아집니다. 이 미들웨어는 사전 필터링을 통해 두 문제를 동시에 해결합니다. 비유하자면, 도서관에서 책을 찾을 때 전체 서가를 훑는 대신, 사서에게 먼저 관련 섹션을 물어보는 것과 같습니다.

주요 파라미터

파라미터	설명	기본값
model	도구 선택용 모델	에이전트의 메인 모델
system_prompt	커스텀 선택 지침	내장 프롬프트
max_tools	최대 선택 도구 수	전체
always_include	항상 포함할 도구 이름 리스트	[]

선택 모델로 `gpt-5.4-mini` 같은 경량 모델을 사용하면 비용을 절감하면서도 효과적인 도구 필터링이 가능합니다.

```
from langchain.agents.middleware import LLMToolSelectorMiddleware

tool_selector = LLMToolSelectorMiddleware(
    model="gpt-5.4-mini",
    max_tools=3,
    always_include=["search"],
)
```

7가지 빌트인 미들웨어만으로 대부분의 프로덕션 요구사항을 충족할 수 있지만, 컨텍스트 누적과 도구 호출 안정성 문제를 해결하는 두 가지 신규 빌트인 미들웨어도 살펴볼 가치가 있습니다.

ContextEditingMiddleware

긴 멀티턴 대화에서 누적된 도구 결과는 빠르게 컨텍스트 윈도우를 잠식합니다. `ContextEditingMiddleware` 는 누적 토큰이 임계치를 넘으면 오래된 도구 결과만 잘라내고 최근 N개를 보존합니다. `SummarizationMiddleware` 가 전체 요약물을 수행한다면, 이 미들웨어는 도구 결과 단위로 청소합니다.

```
from langchain.agents.middleware import ContextEditingMiddleware, ClearToolUsesEdit

agent = create_agent(
    model="gpt-5.4",
    tools=[search_tool],
    middleware=[
        ContextEditingMiddleware(
            edits=[
                ClearToolUsesEdit(
                    trigger=100_000, # 100K 토큰 누적 시 동작
                    keep=3,          # 가장 최근 도구 결과 3건만 보존
                    placeholder="[cleared]",
                ),
            ],
        ),
    ],
)
```

`ClearToolUsesEdit` 의 주요 파라미터는 `trigger` (토큰 임계치), `keep` (보존 개수), `clear_at_least` (최소 회수 토큰), `clear_tool_inputs` (도구 인자도 비울지 여부), `exclude_tools` (정리에서 제외할 도구), `placeholder` (자리 표시 문자열)입니다.

PatchToolCallsMiddleware (Deep Agents)

일부 모델은 도구 호출 JSON에 사소한 결함(따옴표 누락, 잘못된 인자 타입 등)을 자주 만듭니다.

`PatchToolCallsMiddleware` 는 이러한 결함을 자동으로 보정하여 도구 호출이 그대로 실패하지 않도록 합니다. Deep Agents에서 멀티턴 도구 사용 안정성을 높이기 위해 자주 사용됩니다.

```
from deepagents.middleware.patch_tool_calls import PatchToolCallsMiddleware

agent = create_agent(
    model="claude-sonnet-4-6",
    tools=[search_tool, write_file],
    middleware=[PatchToolCallsMiddleware()],
)
```

TIP

`PatchToolCallsMiddleware` 는 `deepagents` 패키지에서 제공되며, `wrap_model_call` 단계에서 도구 호출 페이로드를 후처리합니다. 도구 인자 검증 실패가 빈번한 모델에 우선 적용하세요.

7가지 빌트인 미들웨어만으로 대부분의 프로덕션 요구사항을 충족할 수 있지만, 비즈니스 고유의 로직이 필요한 경우 커스텀 미들웨어를 작성해야 합니다. 예를 들어, 도메인 특화 감사 로그, 커스텀 메트릭 수집, 비즈니스 규칙 기반 가드레일 등은 커스텀 미들웨어로 구현합니다.

1.9 커스텀 미들웨어 작성

두 가지 구현 방식이 있으며, 복잡도와 요구사항에 따라 선택합니다:

1. 데코레이터 방식

단일 혹은 간단한 로직에 적합합니다. `@before_model`, `@after_model`, `@wrap_model_call`, `@wrap_tool_call` 데코레이터를 사용합니다. 함수 하나로 미들웨어를 정의할 수 있어 빠른 프로토타이핑에 유리합니다.

2. 클래스 방식 (`AgentMiddleware`)

여러 혹은 조합하거나 설정이 필요한 경우 `AgentMiddleware` 를 상속합니다. `sync/async` 구현을 동시에 제공할 수 있습니다. 생성자에서 설정을 받고, 여러 혹은 메서드를 오버라이드하여 복합적인 동작을 정의할 수 있습니다.

커스텀 상태

미들웨어는 `NotRequired` 타입 힌트를 사용해 에이전트 상태를 확장할 수 있습니다. 이를 통해 실행 간 값 추적, 혹은 간 데이터 공유, 레이트 리밋이나 감사 로깅 같은 횡단 관심사 구현이 가능합니다.

에이전트 점프

미들웨어의 강력한 기능 중 하나는 에이전트의 실행 흐름 자체를 변경할 수 있다는 것입니다. `after_model` 등에서 딕셔너리를 반환하여 에이전트 흐름을 제어할 수 있습니다:

- `{"jump_to": "end"}` — 에이전트 즉시 종료 (가드레일 위반 시 강제 중단)
- `{"jump_to": "tools"}` — 도구 실행 단계로 이동 (모델 응답을 무시하고 특정 도구 강제 실행)
- `{"jump_to": "model"}` — 모델 호출 단계로 이동 (도구 결과를 바탕으로 재추론 요청)

v1.2부터는 점프 가능 노드를 정적으로 선언하는 `@hook_config(can_jump_to=[...])` 데코레이터가 추가되었습니다. 컴파일 타임에 점프 대상이 검증되므로, 그래프에 존재하지 않는 노드로 점프하는 실수를 방지할 수 있습니다.

```
from langchain.agents.middleware import after_model, hook_config, AgentState
from langgraph.runtime import Runtime
from langchain.messages import AIMessage

@after_model
@hook_config(can_jump_to=["end"])
def block_unsafe(state: AgentState, runtime: Runtime) -> dict | None:
    last = state["messages"][-1]
    if "BLOCKED" in last.content:
        return {
            "messages": [AIMessage("응답할 수 없는 요청입니다.")],
            "jump_to": "end",
        }
    return None
```

타입 어노테이션 — `ModelRequest` / `ToolCallRequest` / `ModelResponse`

`wrap_model_call` / `wrap_tool_call` 에 정확한 타입을 부여하면 IDE 자동완성과 정적 분석 도구의 도움을 받을 수 있습니다. v1은 다음 데이터 타입을 제공합니다.

- `ModelRequest` — `wrap_model_call` 입력. `messages`, `tools`, `model`, `system_message`, `response_format`, `state`, `runtime` 필드 보유.
- `ToolCallRequest` — `wrap_tool_call` 입력. `tool_call`, `state`, `runtime` 필드 보유.
- `ModelResponse` — 내부 핸들러의 반환 타입.
- `ExtendedModelResponse` — `ModelResponse` + `Command` 를 묶어 상태 업데이트를 영속화.

```

from typing import Callable
from langchain.agents.middleware import (
    wrap_model_call, wrap_tool_call,
    ModelRequest, ModelResponse, ToolCallRequest,
)
from langchain.messages import ToolMessage
from langgraph.types import Command

@wrap_model_call
def add_cache_marker(
    request: ModelRequest,
    handler: Callable[[ModelRequest], ModelResponse],
) -> ModelResponse:
    cached = request.override(system_message=f"{request.system_message}\n[CACHE]")
    return handler(cached)

@wrap_tool_call
def audit_tool(
    request: ToolCallRequest,
    handler: Callable[[ToolCallRequest], ToolMessage | Command],
) -> ToolMessage | Command:
    print(f"[audit] {request.tool_call['name']}")
    return handler(request)

```

request.override(...) — 불변 요청 복사본

`ModelRequest.override(...)` 는 원본을 변경하지 않고 수정된 사본을 반환합니다. 같은 `wrap_model_call` 안에서 여러 미들웨어가 안전하게 협력하도록 불변성을 보장하는 패턴입니다. `messages`, `tools`, `model`, `system_message`, `system_prompt`, `response_format` 등이 모두 override 가능합니다.

```

new_request = (
    request
    .override(system_prompt="새 시스템 프롬프트")
    .override(tools=[restricted_tool])
    .override(response_format=Weather)
)
return handler(new_request)

```

ExtendedModelResponse — 상태 업데이트 영속화

`wrap_model_call` 이 `Command` 기반 상태 업데이트를 영속적으로 적용해야 할 때 `ExtendedModelResponse` 를 반환합니다. 일반 `ModelResponse` 만 반환하면 모델 결과는 반영되지만 사이드 상태 변경은 누락될 수 있습니다.

```

from langchain.agents.middleware import ExtendedModelResponse

@wrap_model_call
def track_tokens(request, handler) -> ExtendedModelResponse:
    response = handler(request)
    tokens = response.message.usage_metadata.get("total_tokens", 0)
    return ExtendedModelResponse(
        model_response=response,
        command=Command(update={"total_tokens": tokens}),
    )

```

다음 코드들은 데코레이터 방식과 클래스 방식의 커스텀 미들웨어 구현 예시를 보여줍니다.

```

from langchain.agents.middleware import before_model

@before_model
def log_before(state, runtime):

```

```
"""모델 호출 전 메시지 수를 기록합니다."""
print(f"[LOG] 메시지 {len(state.get('messages', []))}개")
```

```
from langchain.agents.middleware import after_model

@after_model
def validate_output(state, runtime):
    """가드: 금지된 콘텐츠를 차단합니다."""
    last = state["messages"][-1].content
    if "FORBIDDEN" in last:
        return {"jump_to": "end"}
```

```
from langchain.agents.middleware import wrap_model_call

@wrap_model_call
def retry_on_error(request, handler):
    """실패 시 모델 호출을 최대 2회 재시도합니다."""
    for attempt in range(3):
        try:
            return handler(request)
        except Exception as e:
            if attempt == 2: raise
```

```
from langchain.agents.middleware import AgentMiddleware

class AuditMiddleware(AgentMiddleware):
    def __init__(self, log_file="audit.log"):
        self.log_file = log_file
    def before_model(self, state, config):
        print(f"[AUDIT] before -> {self.log_file}")
    def after_model(self, state, config):
        print(f"[AUDIT] after -> {self.log_file}")
```

커스텀 미들웨어를 작성할 수 있게 되었으니, 여러 미들웨어를 함께 사용할 때의 실행 순서를 이해하는 것이 중요합니다. 순서에 따라 미들웨어 간 상호작용이 달라지기 때문입니다.

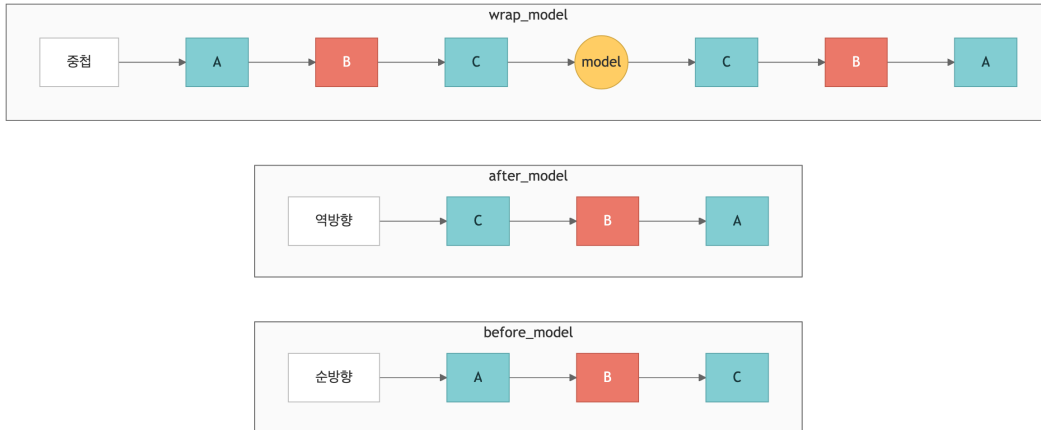
1.10 미들웨어 실행 순서

다중 미들웨어 등록 시 실행 순서를 정확히 이해해야 예상치 못한 동작을 방지할 수 있습니다.

! WARNING

`before_*` 혹은 등록 순서(A → B → C)로 실행되지만, `after_*` 혹은 역순(C → B → A)으로 실행됩니다. 이는 함수 호출 스택의 LIFO(Last In, First Out) 원리와 동일합니다. `wrap_*` 혹은 중첩 래핑(A가 B를 감싸고, B가 C를 감싸는) 방식으로 동작합니다.

`middleware=[A, B, C]` 등록 시:



실전 팁

- PII 검출은 로깅보다 먼저 등록해야 로그에 PII가 포함되지 않습니다.
- 폴백 미들웨어는 재시도 미들웨어보다 뒤에 배치하여, 재시도 실패 후 폴백이 작동하도록 합니다.
- `wrap` 혹은 `next_fn` 을 호출하지 않으면 이후 미들웨어와 실제 호출이 모두 건너뛰어집니다.

```
@before_model
def mw_a(state, runtime): print("before A")

@before_model
def mw_b(state, runtime): print("before B")

@before_model
def mw_c(state, runtime): print("before C")

# 실행: A -> B -> C (after_model이면 C -> B -> A)
```

실행 순서를 이해했다면, 이제 프로덕션에서 실제로 미들웨어를 어떻게 조합하는지 살펴보겠습니다.

1.11 미들웨어 조합 (Stacking)

프로덕션 환경에서는 여러 미들웨어를 함께 사용하여 종합적인 에이전트 거버넌스를 구현합니다. 미들웨어는 등록 순서에 따라 실행되므로, 배치 순서가 매우 중요합니다. 권장 순서는 다음과 같습니다:

1. **보안(PII):** 가장 먼저 실행하여 민감 정보가 후속 미들웨어나 로그에 노출되지 않도록 합니다
2. **신뢰성(폴백):** 모델 장애 시 대체 모델로 자동 전환합니다

3. **비용 제어(호출 제한)**: 호출 횟수를 제한하여 예산을 보호합니다
4. **컨텍스트 관리(요약)**: 긴 대화를 자동 요약하여 윈도우 초과를 방지합니다
5. **최적화(도구 선택)**: 관련 도구만 필터링하여 정확도를 높입니다
6. **감독(HITL)**: 고위험 작업에 대해 인간 승인을 요청합니다

이러한 조합을 통해 각 미들웨어는 단일 책임 원칙을 유지하면서도, 전체적으로 강력한 프로덕션 에이전트 파이프라인을 구성할 수 있습니다.

다음 코드는 프로덕션 환경에서 권장되는 미들웨어 스택 구성 예시입니다.

```
from langchain.agents import create_agent
from langchain.agents.middleware import (
    PIIMiddleware, ModelFallbackMiddleware,
    ModelCallLimitMiddleware, SummarizationMiddleware,
    HumanInTheLoopMiddleware, LLMToolSelectorMiddleware,
)
from langgraph.checkpoint.memory import InMemorySaver
```

```
middleware_stack = [
    PIIMiddleware("email", strategy="redact", apply_to_input=True),
    ModelFallbackMiddleware("gpt-5.4-mini", "claude-sonnet-4-6"),
    ModelCallLimitMiddleware(thread_limit=50, run_limit=10),
    SummarizationMiddleware(model="gpt-5.4-mini", trigger=("{tokens", 4000)),
]

production_agent = create_agent(
    model="gpt-5.4", tools=[], checkpointer=InMemorySaver(), middleware=middleware_stack,
)
```

1.12 공급자 특화 미들웨어

빌트인 7종이 프로바이더 무관 공통 패턴을 다룬다면, 공급자 특화 미들웨어는 각 LLM 프로바이더가 제공하는 고유 기능을 에이전트에 주입합니다. Anthropic의 프롬프트 캐시, Bedrock의 TTL 캐시, OpenAI의 Moderation API처럼 공급자 서버 측에서만 활성화할 수 있는 기능들은 별도 미들웨어로 노출됩니다.

Anthropic Prompt Caching

`langchain_anthropic.middleware.AnthropicPromptCachingMiddleware` 는 Claude 모델 호출 시 시스템 프롬프트 · 도구 정의 · 마지막 메시지 위치에 `cache_control` 마커를 자동 부착합니다. 긴 system prompt나 RAG 컨텍스트를 반복 전송하는 에이전트의 입력 토큰 비용을 최대 90% 절감합니다.

파라미터	기본값	설명
<code>type</code>	"ephemeral"	현재 Anthropic이 지원하는 유일한 타입
<code>tll</code>	"5m"	"5m" 또는 "1h" (1h는 Anthropic beta)
<code>min_messages_to_cache</code>	0	메시지 수가 이 값 이상일 때만 <code>cache_control</code> 부착
<code>unsupported_model_behavior</code>	"warn"	비 Anthropic 모델에 적용 시: "warn" / "ignore" / "raise"

```

from langchain.agents import create_agent
from langchain_anthropic.middleware import AnthropicPromptCachingMiddleware

agent = create_agent(
    model="anthropic:claude-sonnet-4-6",
    tools=[],
    middleware=[
        AnthropicPromptCachingMiddleware(
            ttl="1h",
            min_messages_to_cache=2,
            unsupported_model_behavior="warn",
        ),
    ],
)

```

연속 호출 시 `usage_metadata.input_token_details` 에서 `cache_creation_input_tokens` (1회차)와 `cache_read_input_tokens` (2회차 이후)로 캐시 적중 여부를 확인할 수 있습니다.

Claude 네이티브 도구 미들웨어

Claude 모델은 Anthropic 서버가 직접 해석하는 네이티브 도구 스키마(`bash_20250124` , `text_editor_20250728` , `memory_20250818`)를 학습해 두었습니다. 범용 `@tool` 데코레이터로 동일 기능을 구현하는 것보다 tool schema 토큰이 거의 0에 수렴하고 오류율이 낮습니다. LangChain은 네이티브 도구를 상태(State) 기반과 디스크(Filesystem) 기반 두 변형으로 래핑한 미들웨어를 제공합니다.

- `ClaudeBashToolMiddleware` — 네이티브 bash 실행 도구. `workspace_root` , `startup_commands` , `execution_policy` (`HostExecutionPolicy` / `DockerExecutionPolicy` / `CodexSandboxExecutionPolicy`) , `redaction_rules` 를 받습니다. Docker 정책이 기본 권장치입니다.
- `StateClaudeTextEditorMiddleware` / `FilesystemClaudeTextEditorMiddleware` — 네이티브 text editor로 `view` / `create` / `str_replace` / `insert` / `delete` / `rename` 을 지원합니다. State 변형은 `text_editor_files` 상태 키에, Filesystem 변형은 실제 `root_path` 디렉터리에 파일을 씁니다.
- `StateClaudeMemoryMiddleware` / `FilesystemClaudeMemoryMiddleware` — `/memories/*` 경로 계약을 따르는 네이티브 메모리 도구. State 변형은 체크포인트와 결합해 `thread_id` 재개 시 복원되고, Filesystem 변형은 디스크에 남기므로 프로세스 재시작에도 보존됩니다.
- `StateFileSearchMiddleware` — text editor/메모리가 쌓은 가상 파일을 `glob` / `grep` 으로 검색하는 네이티브 도구. `state_key="text_editor_files"` 가 기본이며 `"memory_files"` 로 바꾸면 메모리 쪽을 검색합니다.

```

from langchain_anthropic.middleware import (
    ClaudeBashToolMiddleware,
    StateClaudeTextEditorMiddleware,
    StateClaudeMemoryMiddleware,
    StateFileSearchMiddleware,
)
from langchain.agents.middleware import DockerExecutionPolicy

agent = create_agent(
    model="anthropic:claude-sonnet-4-6",
    middleware=[
        ClaudeBashToolMiddleware(execution_policy=DockerExecutionPolicy()),
        StateClaudeTextEditorMiddleware(allowed_path_prefixes=["/src"]),
        StateClaudeMemoryMiddleware(),
        StateFileSearchMiddleware(state_key="text_editor_files"),
    ],
)

```

! WARNING

LangChain 1.2의 `create_agent` 는 같은 미들웨어 클래스의 중복 인스턴스를 거부합니다. text editor 파일과 메모리 파일 두 저장소를 동시에 `StateFileSearchMiddleware` 로 검색하려면 서브클래싱으로 분리해야 합니다.

Bedrock Prompt Caching

AWS Bedrock 경유로 Claude/Nova를 호출하는 기업 환경에서는

`langchain_aws.middleware.BedrockPromptCachingMiddleware` 를 사용합니다. `AnthropicPromptCachingMiddleware` 와 파라미터 이름과 의미가 거의 같지만 대상 패키지와 모델별 제약이 다릅니다.

항목	ChatBedrockConverse + Anthropic	ChatBedrockConverse + Nova
시스템 프롬프트 캐시	O	O
도구 정의 캐시	O	X
메시지 캐시	O	O (tool result 메시지 제외)
TTL <code>"1h"</code>	O	X (5m 전용)

`type` 파라미터는 `ChatBedrock` (이전 세대 `invoke-model` 래퍼)에서만 실제로 반영되고, `ChatBedrockConverse`에서는 `"default"` 로 고정됩니다. 체크포인트가 실제로 작동하려면 캐시 대상 블록이 약 1,024 토큰 이상이어야 합니다. Nova 모델에서는 `ttl="5m"` 로 고정하고 `unsupported_model_behavior="warn"` 을 명시해 실수로 `"1h"` 를 지정했을 때 조용히 무시되지 않도록 하세요.

```
from langchain_aws.middleware import BedrockPromptCachingMiddleware

agent = create_agent(
    model="bedrock_converse:anthropic.claude-sonnet-4-v6:0",
    tools=[],
    middleware=[
        BedrockPromptCachingMiddleware(
            ttl="1h",
            min_messages_to_cache=2,
            unsupported_model_behavior="warn",
        ),
    ],
)
```

OpenAI Content Moderation (보강)

앞서 소개한 `OpenAIModerationMiddleware` 의 전체 파라미터를 정리합니다. `check_tool_results` 는 도구 결과를 모델에 넣기 전에 추가로 스캔하는 옵션으로, 웹 검색이나 이메일처럼 제3자가 작성한 콘텐츠가 섞이는 파이프라인에서만 켜는 것이 비용 대비 효율적입니다.

파라미터	기본값	설명
<code>model</code>	<code>"omni-moderation-latest"</code>	Moderation 모델. <code>"omni-moderation-2024-09-26"</code> 등 편 가능
<code>check_input</code>	<code>True</code>	사용자 메시지를 모델에 넘기기 전 검사

check_output	True	모델 생성 응답을 반환하기 전 검사
check_tool_results	False	도구 실행 결과를 모델 입력 전에 검사
exit_behavior	"end"	"end" (그래프 종료) / "error" (예외) / "replace" (메시지만 교체)
violation_message	기본 템플릿	{categories} · {category_scores} · {original_content} 변수 지원
client / async_client	None	사전 구성된 OpenAI 클라이언트 주입 (선택)

```

from langchain_openai.middleware import OpenAIModerationMiddleware

agent = create_agent(
    model="openai:gpt-5.4",
    tools=[search_tool],
    middleware=[
        OpenAIModerationMiddleware(
            model="omni-moderation-latest",
            check_input=True,
            check_output=True,
            check_tool_results=False,
            exit_behavior="replace",
            violation_message=(
                "안전 정책 위반이 감지되었습니다 "
                "(카테고리: {categories}). 원 내용은 기록되지 않습니다."
            ),
        ),
    ],
)

```

📌 TIP

프로덕션에서는 `check_input` 만 항상 켜고 `check_output` 은 샘플링 전략(예: 10% 랜덤 샘플)으로 운영하는 경우가 많습니다. Moderation API도 호출 비용과 레이턴시가 추가되기 때문입니다.

요약

항목	핵심 내용
아키텍처	4가지 훅: <code>before_model</code> , <code>after_model</code> , <code>wrap_model_call</code> , <code>wrap_tool_call</code>
빌트인 7종	Summarization, HITL, ModelCallLimit, ToolCallLimit, ModelFallback, PII, LLMToolSelector
커스텀	데코레이터(<code>\@before_model</code> 등) / <code>AgentMiddleware</code> 클래스
실행 순서	<code>before</code> : 순방향, <code>after</code> : 역방향, <code>wrap</code> : 중첩
프로덕션	PII → Fallback → Limit → Summarization → ToolSelector → HITL

공급자 특화	Anthropic(캐시·bash·editor·memory·search), Bedrock(캐시), OpenAI(Moderation)
--------	---

미들웨어는 단일 에이전트의 동작을 제어하는 강력한 도구입니다. 그러나 복잡한 도메인 문제를 해결하려면 여러 에이전트가 협력하는 멀티에이전트 아키텍처가 필요합니다. 다음 장에서는 감독자 패턴으로 서브에이전트를 조율하는 멀티에이전트 시스템을 다룹니다.

02 멀티에이전트: Subagents

감독자 패턴

단일 에이전트로 복잡한 도메인 요구사항을 충족하기 어렵습니다. Subagents 패턴은 중앙 감독자(Supervisor)가 전문화된 서브에이전트들을 @tool 로 래핑하여 호출하는 3계층 아키텍처로, 역할 분리와 확장성을 동시에 달성합니다. 이 장에서는 감독자 → 서브에이전트 → 도구의 계층 구조를 설계하고, HITL 및 비동기 디스패치 패턴까지 실전 예제로 구현합니다.

학습 목표

LEARNING OBJECTIVES

- 감독자 → 서브에이전트 → 도구의 3계층 아키텍처를 설계한다
- 서브에이전트를 @tool 로 래핑하여 감독자에게 도구로 노출한다
- HITL, ToolRuntime, 비동기/디스패치 패턴을 이해한다

2.1 환경 설정

이전 장에서 미들웨어로 단일 에이전트의 동작을 제어하는 방법을 배웠습니다. 그러나 실무에서는 캘린더 관리, 이메일 처리, CRM 조회 등 여러 도메인을 동시에 다루는 에이전트가 필요합니다. 하나의 에이전트에 모든 도구를 넣으면 시스템 프롬프트가 비대해지고 도구 선택 정확도가 떨어집니다. 실제로 도구가 15개 이상일 때 단일 에이전트의 도구 선택 정확도는 눈에 띄게 하락한다는 것이 경험적으로 알려져 있습니다.

Subagents 패턴은 중앙 감독자(Supervisor) 에이전트가 전문화된 서브에이전트들을 도구처럼 호출하여 작업을 위임하는 멀티에이전트 아키텍처입니다. 이 패턴은 소프트웨어 공학의 위임(delegation) 원칙을 에이전트 세계에 적용한 것으로, 각 서브에이전트는 자신의 도메인에만 집중하고, 감독자는 작업 분해와 결과 집계에 집중합니다. 이 노트북에서는 캘린더와 이메일 도메인을 처리하는 개인 비서 시스템을 구축합니다.

```
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI

load_dotenv()

model = ChatOpenAI(model="gpt-5.4")
```

2.2 Subagents 아키텍처 개요

Subagents 패턴은 3계층 아키텍처로 구성됩니다. 감독자가 모든 라우팅을 담당하고, 서브에이전트는 사용자와 직접 상호작용하지 않으며, 결과를 감독자에게 반환합니다.

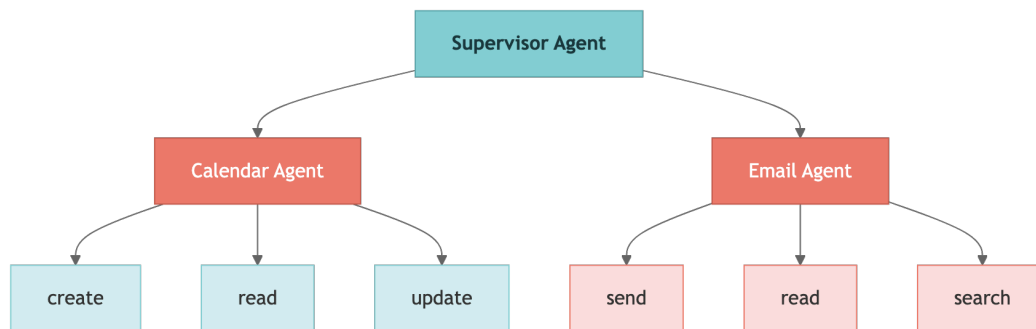


그림 읽는 법

위에서 아래로 읽으면 됩니다. 감독자는 사용자 요청을 보고 어떤 전문 서브에이전트를 호출할지 결정하고, 각 서브에이전트는 자신에게 연결된 저수준 도구만 사용해 결과를 다시 감독자에게 돌려줍니다.

계층	역할	특징
저수준 도구	외부 서비스 직접 호출 (Calendar API, Email API)	단순한 함수 래퍼
서브에이전트	도메인별 추론 + 도구 조합	전문 시스템 프롬프트, 독립적 도구 세트
감독자	작업 분해, 위임, 결과 집계	전체 대화 기억, 서브에이전트 = 도구로 취급

핵심 특성

이 아키텍처의 네 가지 핵심 특성을 이해하면, 설계 시 올바른 판단을 내릴 수 있습니다.

- 중앙 집중 제어: 모든 라우팅이 감독자를 통해 흐릅니다. 사용자는 감독자하고만 대화하며, 서브에이전트의 존재를 인지하지 못합니다.

- 컨텍스트 격리: 서브에이전트는 매번 깨끗한 컨텍스트 윈도우에서 실행되어 컨텍스트 비대화를 방지합니다. 캘린더 에이전트는 이메일 대화 내용을 알 필요가 없습니다.
- 병렬 실행: 여러 서브에이전트를 한 턴에서 동시에 호출할 수 있습니다. “미팅 잡고 이메일 보내줘”라는 요청에서 캘린더와 이메일 서브에이전트가 병렬로 동작합니다.
- 도구 기반 호출: 서브에이전트를 `@tool` 로 래핑하여 감독자에게 일반 도구처럼 노출합니다. 이를 통해 감독자는 서브에이전트와 일반 도구를 구별 없이 사용합니다.

사용 시점

서브에이전트 패턴은 여러 도메인(캘린더, 이메일, CRM 등)을 관리하면서 서브에이전트가 사용자와 직접 대화할 필요 없고, 중앙화된 워크플로 관리가 필요할 때 적합합니다. 도구가 적은 단순한 시나리오에서는 단일 에이전트로 충분합니다.

TIP

서브에이전트 패턴 vs. 단일 에이전트 판단 기준: 도구가 10개 이상이거나, 도메인별로 서로 다른 시스템 프롬프트가 필요하거나, 도구 간 의존성이 복잡한 경우 서브에이전트 패턴이 유리합니다.

아키텍처를 이해했으니, 이제 3계층의 최하층인 저수준 도구부터 Bottom-up으로 구현해 보겠습니다.

2.3 저수준 도구 정의

3계층 아키텍처의 최하층인 저수준 도구를 정의합니다. 이 도구들은 외부 서비스(Calendar API, Email API)와 직접 상호작용하는 단순한 함수 래퍼입니다. 실제 프로덕션에서는 Google Calendar API, Email Service 등과 연동하지만, 여기서는 학습을 위한 스텝(stub) 구현을 사용합니다.

도구를 설계할 때 중요한 점:

- 하나의 도구는 하나의 기능만 담당 (단일 책임 원칙)
- 동일한 도구를 여러 서브에이전트에 중복 할당하지 않아야 합니다
- docstring을 명확하게 작성하여 LLM이 도구를 올바르게 선택할 수 있게 합니다

```
from langchain_core.tools import tool

@tool
def create_calendar_event(
    title: str, start_time: str, end_time: str,
    attendees: list[str] = None,
) -> str:
    """새 캘린더 이벤트를 생성합니다."""
    return f"이벤트 '{title}' 생성됨: {start_time} ~ {end_time}"
```

```
@tool
def read_calendar_events(date: str) -> str:
    """날짜(YYYY-MM-DD)의 캘린더 이벤트를 조회합니다."""
    return f"{date}에 이벤트가 없습니다."
```

```
@tool
def send_email(to: str, subject: str, body: str) -> str:
    """이메일 메시지를 전송합니다."""
```

```

        return f"{to}에게 이메일 전송됨: '{subject}'"

@tool
def read_emails(folder: str = "inbox", limit: int = 10) -> str:
    """폴더에서 최근 이메일을 읽습니다."""
    return f"{folder}에 이메일 3개"

```

```

@tool
def search_emails(query: str, limit: int = 10) -> str:
    """검색어로 이메일을 검색합니다."""
    return f"'{query}' 검색 결과 2건"

```

도구가 준비되었으면, 이제 이 도구들을 조합하여 도메인별 서브에이전트를 생성합니다. 서브에이전트는 저수준 도구를 직접 사용하여 도메인 내의 복합적인 작업을 수행하는 중간 계층입니다.

! WARNING

저수준 도구의 docstring은 서브에이전트(LLM)가 도구를 선택하는 데 사용됩니다. 모호하거나 중복된 설명은 잘못된 도구 선택의 원인이 됩니다. 각 도구의 역할이 명확히 구분되도록 docstring을 작성하세요.

2.4 서브에이전트 생성

각 서브에이전트는 `create_agent()` 로 생성하며, 세 가지 핵심 요소를 갖습니다:

1. 전문화된 시스템 프롬프트: 도메인별 역할과 행동 지침을 정의합니다
2. 도메인별 도구 세트: 해당 도메인의 도구만 할당하여 관심사를 분리합니다
3. `name` **식별자**: 감독자가 서브에이전트를 구분하고 호출할 때 사용합니다

서브에이전트의 세분화 수준(granularity)은 도메인 단위(캘린더, 이메일 등)가 권장됩니다. 너무 세분화하면 감독자의 라우팅 부담이 증가하고, 너무 통합하면 컨텍스트 격리의 이점이 줄어듭니다.

```

from langchain.agents import create_agent

calendar_agent = create_agent(
    model="gpt-5.4",
    tools=[create_calendar_event, read_calendar_events],
    system_prompt="당신은 캘린더 어시스턴트입니다. ISO 8601 날짜 형식을 사용하세요.",
    name="calendar_agent",
)

```

```

email_agent = create_agent(
    model="gpt-5.4",
    tools=[send_email, read_emails, search_emails],
    system_prompt="당신은 이메일 어시스턴트입니다. 메시지를 전문적으로 작성하세요.",
    name="email_agent",
)

```

서브에이전트가 생성되었지만, 감독자 에이전트가 이를 호출하려면 도구 인터페이스로 래핑해야 합니다. 이것이 3계층 아키텍처의 핵심 연결 고리이며, Subagents 패턴의 가장 중요한 구현 단계입니다.

2.5 서브에이전트를 도구로 래핑

서브에이전트를 감독자에게 노출하는 표준 패턴은 `@tool` 데코레이터로 감싸는 것입니다. 래핑 함수 내부에서 `subagent.invoke()` 를 호출하고, 마지막 메시지의 `content` 를 반환합니다. 이 래핑 패턴은 어댑터(Adapter) 디자인 패턴과 유사합니다. 서브에이전트의 복잡한 인터페이스를 감독자가 이해할 수 있는 단순한 도구 인터페이스로 변환합니다.

이 패턴의 장점:

- 감독자 입장에서 서브에이전트는 일반 도구와 동일하게 취급됩니다
- 서브에이전트의 내부 구현 변경이 감독자에 영향을 주지 않습니다
- 입출력 형식을 래핑 함수에서 자유롭게 커스터마이징할 수 있습니다

입출력 전략 선택: 쿼리만 전달(간단)할 수도 있고, 전체 컨텍스트를 전달(정교)할 수도 있습니다. 결과 반환 시에도 최종 결과만 반환하거나 전체 히스토리를 반환하는 선택지가 있습니다.

TIP

래핑 함수의 docstring이 감독자의 도구 선택에 결정적인 영향을 미칩니다. “캘린더 관련 모든 작업을 처리합니다”처럼 포괄적인 설명을 작성하세요. 감독자는 이 설명만 보고 어떤 서브에이전트를 호출할지 결정합니다.

서브에이전트가 도구로 래핑되었으므로, 이제 이들을 조율할 감독자 에이전트를 조립할 차례입니다.

2.6 감독자 에이전트 조립

감독자는 래핑된 서브에이전트 도구들을 `tools` 에 전달받아 생성됩니다. 감독자의 시스템 프롬프트에는 작업 분해(task decomposition) 및 위임(delegation) 지침을 포함합니다. 감독자는 사용자의 복합 요청을 분석하여 어떤 서브에이전트에게 어떤 부분을 위임할지 결정하는 오케스트레이터 역할을 합니다.

감독자 설계 시 고려사항:

- 에러 처리: 서브에이전트 실패를 감독자가 우아하게 처리해야 합니다. “이메일 전송에 실패했지만 미팅은 예약되었습니다”와 같은 부분 성공 보고가 필요합니다.
- 결과 집계: 여러 서브에이전트의 결과를 통합하여 사용자에게 일관된 응답을 제공합니다
- 승인 범위: 상태를 변경하는 작업(이메일 전송, 이벤트 생성)에만 HITL을 적용합니다

```
supervisor = create_agent(
    model="gpt-5.4",
    tools=[call_calendar, call_email],
    system_prompt=(
        "당신은 개인 비서입니다. 복잡한 요청을 "
        "하위 작업으로 분해하고 적절한 에이전트에 위임하세요."
    ),
)
```

2.7 실행 테스트

```
User: "내일 2시 Sarah 미팅 잡고 초대 이메일 보내줘"
Supervisor → call_calendar → create_calendar_event
Supervisor → call_email → send_email
Supervisor: "미팅과 초대 이메일 완료"
```

기본 감독자-서브에이전트 구조가 동작하는 것을 확인했습니다. 프로덕션 환경에서는 에이전트가 수행하는 고위험 작업(이메일 전송, 일정 변경 등)에 대해 인간 승인이 필요합니다. HITL 미들웨어를 감독자 레벨에 적용하면, 서브에이전트의 도구 호출이 아닌 서브에이전트 호출 자체를 승인 대상으로 설정할 수 있습니다.

2.8 HITL (Human-in-the-Loop) 통합

`HumanInTheLoopMiddleware` 와 `checkpointer` 를 결합하면 고위험 도구 호출(이메일 전송, 일정 생성 등) 전에 사용자 승인을 요청할 수 있습니다. 에이전트가 보호된 도구를 호출하려 할 때 실행이 일시 중지되고, 사용자가 검토합니다.

승인 응답 유형

응답	설명	코드
승인(Approve)	도구 호출을 그대로 실행	<code>Command(resume="approve")</code>
편집(Edit)	도구 인자를 수정 후 실행	<code>Command(resume={"type": "edit", "args": {...}})</code>
거부(Reject)	도구 호출을 취소	<code>Command(resume={"type": "reject", "reason": "..."})</code>

HITL은 상태를 변경하는 작업(`send_email`, `create_calendar_event` 등)에만 적용하는 것이 권장됩니다. 읽기 전용 작업에는 불필요한 마찰을 줄이기 위해 적용하지 않습니다.

```
from langchain.agents.middleware import HumanInTheLoopMiddleware
from langgraph.checkpoint.memory import InMemorySaver

hitl = HumanInTheLoopMiddleware(interrupt_on={
    "schedule_event": {"allowed_decisions": ["approve", "edit", "reject"]},
    "manage_email": {"allowed_decisions": ["approve", "reject"]},
})
```

```
supervisor_hitl = create_agent(
    model="gpt-5.4",
    tools=[call_calendar, call_email],
    checkpointer=InMemorySaver(),
    middleware=[hitl],
    system_prompt="당신은 개인 비서입니다.",
)
```

HITL이 감독자 레벨에서 적용되면, 서브에이전트 호출 자체가 승인 대상이 됩니다. 즉, “이메일을 보내겠습니다”라는 서브에이전트 호출 전에 사용자 승인을 요청하므로, 서브에이전트 내부의 개별 도구 호출까지 일일이 승인할 필요가 없습니다. 이제 서브에이전트에 런타임 정보를 전달하는 방법을 살펴보겠습니다.

2.9 컨텍스트 전달 (ToolRuntime)

`ToolRuntime` 은 런타임 컨텍스트(사용자 ID, 이름, 타임존 등)를 메시지에 포함하지 않고도 도구에 전달하는 메커니즘입니다. 매 프롬프트마다 반복적인 텍스트를 넣는 대신, `ToolRuntime` 으로 공유 컨텍스트를 한 번만 설정합니다. 이는 프롬프트의 토큰 비용을 절감하면서도 도구가 필요한 모든 컨텍스트 정보에 접근할 수 있게 합니다.

도구 함수에서는 `runtime_context` 키워드 인자로 컨텍스트에 접근합니다. 이를 통해:

- 사용자 신원 정보를 도구가 활용할 수 있습니다 (예: 발신자 이메일 자동 설정)
- 환경 설정(타임존 등)을 일관되게 적용할 수 있습니다
- 프롬프트 길이를 줄여 토큰 비용을 절감합니다

```
# ToolRuntime은 LangChain v1 에이전트의 런타임 컨텍스트 전달 메커니즘입니다.
# 아직 릴리스되지 않은 경우를 대비하여 폴백 처리합니다.
try:
    from langchain.runtime import ToolRuntime
except ImportError:
    # ToolRuntime이 아직 릴리스되지 않은 경우 간단한 대체 구현
    class ToolRuntime:
        """사건 릴리스 버전을 위한 폴백 ToolRuntime."""
        def __init__(self, context: dict):
            self.context = context
        print("ToolRuntime 미출시 - 폴백 스텝 사용")

runtime = ToolRuntime(context={
    "user_email": "me@example.com",
    "user_name": "Alice",
    "timezone": "Asia/Seoul",
})
```

ToolRuntime 미출시 - 폴백 스텝 사용

```
supervisor_ctx = create_agent(
    model="gpt-5.4",
    tools=[call_calendar, call_email],
    system_prompt="당신은 개인 비서입니다.",
)
```

```
# 도구에서 런타임 컨텍스트 접근
@tool
def send_email_ctx(
    to: str, subject: str, body: str, *, runtime_context: dict
) -> str:
    """현재 사용자로 이메일을 전송합니다."""
    sender = runtime_context["user_email"]
    return f"{sender}에서 {to}로 이메일 전송됨: '{subject}'"
```

컨텍스트 전달 메커니즘을 이해했으니, 이제 서브에이전트의 실행 모드에 대해 알아보겠습니다. 기본적으로 서브 에이전트는 동기적으로 실행되지만, 일부 시나리오에서는 비동기 실행이 더 효율적입니다.

2.10 비동기 실행 패턴

서브에이전트의 실행 모드는 두 가지입니다. 작업의 성격과 의존 관계에 따라 적절한 모드를 선택합니다:

모드	동작	사용 시점
동기(Synchronous)	감독자가 서브에이전트 완료를 대기 후 다음 진행	결과가 다음 작업에 필요할 때 (기본 값)
비동기(Asynchronous)	즉시 Job ID 반환, 백그라운드 실행, 나중에 결과 조회	독립적인 작업, 장시간 소요 작업

비동기 패턴은 Job ID → Status → Result 구조를 따릅니다. 서브에이전트가 즉시 Job ID를 반환하면, 감독자는 다른 작업을 계속 진행하다가 나중에 결과를 조회합니다.

```
import uuid
job_store = {}

@tool("schedule_async", description="이벤트 예약 (비동기)")
def call_calendar_async(query: str) -> str:
    """비동기 캘린더 작업을 시작하고 작업 ID를 반환합니다."""
    job_id = str(uuid.uuid4())[:8]
    job_store[job_id] = {"status": "done", "result": "이벤트 생성됨"}
    return f"작업 시작됨: {job_id}"
```

```
@tool("check_job", description="비동기 작업 상태 확인")
def check_job(job_id: str) -> str:
    """비동기 작업의 상태를 확인합니다."""
    job = job_store.get(job_id, {"status": "not_found"})
    return f"상태: {job['status']}, 결과: {job.get('result')}"
```

! WARNING

비동기 패턴에서 `job_store` 는 메모리에 저장되므로, 서버 재시작 시 작업 상태가 유실됩니다. 프로덕션에서는 Redis나 데이터베이스 기반 작업 큐(Celery, RQ 등)를 사용하세요.

비동기 패턴까지 이해했다면, 마지막으로 서브에이전트 호출 방식의 확장성을 높이는 단일 디스패치 패턴을 살펴 보겠습니다.

2.11 단일 디스패치 도구 패턴

서브에이전트가 늘어날수록 각각을 개별 도구로 래핑하는 것이 번거로워집니다. 단일 디스패치 패턴은 이 문제를 해결하는 확장 가능한 접근법입니다. 도구 패턴에는 두 가지 접근법이 있습니다:

패턴	설명	장점
에이전트별 도구	서브에이전트마다 별도의 래핑 도구 생성	세밀한 제어, description 커스터마이징 용이
단일 디스패치 도구	하나의 파라미터화된 도구로 모든 서브에이전트 호출	확장성 우수, 서브에이전트 추가/제거가 독립적

단일 디스패치 패턴은 `agent_name` 파라미터로 호출 대상을 지정합니다. 에이전트 레지스트리에 등록된 서브에이전트를 이름으로 조회하여 호출하므로, 분산 팀에서 서브에이전트를 독립적으로 추가하거나 제거할 수 있어 확장성이 뛰어납니다.

단일 dispatch 도구 3가지 노출 방식

단일 디스패치 도구를 모델에게 어떻게 노출하느냐에 따라 세 가지 변형이 있습니다. 도메인 수와 라우팅 자유도에 맞춰 선택합니다.

노출 방식	특징	적합 시나리오
Literal enum	<code>agent_name: Literal["calendar", "email"]</code> — 모델이 enum에서 선택	도메인 ≤ 5, 안정적인 라우팅 필요
dict 레지스트리 + 동적 description	에이전트 추가/제거를 코드 수정 없이 dict로 관리	도메인 6 20, 잦은 추가/삭제
벡터 검색 라우팅	에이전트 설명을 임베딩하고 사용자 쿼리와 유사도 매칭	도메인 20+, 의미 기반 라우팅

```
from typing import Literal
from langchain.tools import tool

# 1) Literal enum - 가장 단순
@tool
def dispatch_enum(
    agent_name: Literal["calendar", "email", "crm"],
    query: str,
) -> str:
    """전문 에이전트에 작업을 라우팅합니다."""
    return AGENT_REGISTRY[agent_name].invoke(
        {"messages": [{"role": "user", "content": query}]}
    )["messages"][-1].content
```

SubAgentMiddleware — Deep Agents 통합 패턴

서브에이전트를 일일이 `@tool` 로 래핑하는 대신, `deepagents.SubAgentMiddleware` 를 사용하면 선언적 설정만으로 감독자가 서브에이전트를 호출할 수 있게 됩니다. 미들웨어가 자동으로 dispatch 도구를 등록하고, 서브에이전트 실행 결과를 메시지로 변환해 감독자 상태에 병합합니다.

```
from deepagents.middleware import SubAgentMiddleware

supervisor_with_mw = create_agent(
    model="gpt-5.4",
    tools=[], # 별도 래핑 불필요 - 미들웨어가 dispatch 도구 자동 등록
    middleware=[
        SubAgentMiddleware(subagents=[
            {"name": "calendar", "description": "일정 관리", "agent": calendar_agent},
            {"name": "email", "description": "이메일 전송/조회", "agent": email_agent},
        ]),
    ],
    system_prompt="당신은 개인 비서입니다.",
)
```

checkpointer=True — 서브에이전트 자동 체크포인트 공유

v1.2부터는 `create_agent(checkpointer=True)` 로 부모 그래프의 체크포인트를 자동으로 상속받을 수 있습니다. 명시적 체크포인트 인스턴스를 전달하지 않아도 서브에이전트가 부모와 같은 `thread_id` 에서 재개됩니다. HITL 미들웨어를 서브에이전트에 적용할 때 특히 편리합니다.

```
calendar_agent = create_agent(
    model="gpt-5.4",
    tools=[create_calendar_event],
    checkpointer=True, # 부모 그래프의 체크포인트 자동 상속
    middleware=[HumanInTheLoopMiddleware(interrupt_on={
        "create_calendar_event": {"allowed_decisions": ["approve", "reject"]},
    })],
    name="calendar_agent",
)
```

ToolRuntime[None, CustomState] — 컨텍스트 없이 커스텀 상태만 주입

컨텍스트(`@dataclass`)는 필요 없지만 커스텀 상태만 도구에 노출하고 싶을 때 `ToolRuntime[None, CustomState]` 타입 어노테이션을 사용합니다. 첫 번째 제네릭이 컨텍스트, 두 번째가 상태 스키마입니다.

```
from typing import NotRequired
from langchain.tools import tool, ToolRuntime
from langchain.agents import AgentState

class JobState(AgentState):
    pending_jobs: NotRequired[list[str]]

@tool
def list_pending(runtime: ToolRuntime[None, JobState]) -> str:
    """현재 진행 중인 비동기 작업 목록을 조회합니다."""
    return ", ".join(runtime.state.get("pending_jobs", []))
```

비동기 5-tool 디스패치 패턴

실제 프로젝트에서는 비동기 서브에이전트 호출에 5개 도구를 한 세트로 제공하는 것이 권장됩니다. 시작/조회/취소/목록/대기의 다섯 책임을 분리하면 모델이 작업 라이프사이클을 명확히 파악합니다.

도구	책임
<code>start_job(agent_name, query)</code>	비동기 잡 시작 후 <code>job_id</code> 반환
<code>check_job(job_id)</code>	잡 상태(<code>pending</code> / <code>running</code> / <code>done</code> / <code>failed</code>) 조회
<code>cancel_job(job_id)</code>	실행 중 잡을 취소
<code>list_jobs()</code>	현재 활성 잡 목록 반환
<code>wait_for(job_id, timeout_s)</code>	완료까지 폴링 — 짧은 작업에 유용

다섯 도구는 외부 작업 큐(Celery, Redis Streams, Cloud Tasks 등)나 데이터베이스 백엔드와 결합해 프로세스 재시작에도 잡 상태가 유지되도록 구성해야 합니다.

```
supervisor_dispatch = create_agent(
    model="gpt-5.4",
    tools=[dispatch],
    system_prompt=(
```



```

    "delegate 도구를 사용하여 작업을 라우팅하세요. "
    "에이전트: 'calendar'(일정), 'email'(이메일)."
```

요약

항목	핵심
3계층	도구 → 서브에이전트(<code>create_agent</code>) → 감독자
래핑	<code>\@tool</code> + <code>subagent.invoke()</code> → 마지막 content 반환
격리	서브에이전트 = 깨끗한 컨텍스트, 감독자만 전체 기억
HITL	<code>HumanInTheLoopMiddleware</code> + <code>InMemorySaver</code>
컨텍스트	<code>ToolRuntime(context={...})</code> → <code>runtime_context</code>
비동기	Job ID → Status → Result 패턴
디스패치	단일 <code>dispatch(agent_name, query)</code> 도구

Subagents 패턴은 감독자가 모든 라우팅을 제어하는 중앙 집중형 아키텍처입니다. 그러나 에이전트가 사용자와 직접 대화하면서 단계적으로 상태를 전이해야 하거나, 여러 소스에서 병렬로 정보를 수집해야 하는 경우에는 다른 패턴이 더 적합합니다. 다음 장에서는 `Command(goto=...)` 로 상태를 전이하는 Handoffs 패턴과, `Send` API 로 병렬 디스패치하는 Router 패턴을 학습합니다.

03 멀티에이전트: Handoffs & Router

상태 머신과 병렬 라우팅

멀티에이전트 시스템의 또 다른 핵심 패턴인 `Handoffs` 와 `Router` 를 다룹니다. `Handoffs` 는 단일 에이전트가 상태 변수에 따라 프롬프트와 도구를 동적으로 교체하는 상태 머신 방식이며, `Router` 는 쿼리를 분류하여 전문 에이전트들에게 `Send` API로 병렬 라우팅한 뒤 결과를 합성하는 방식입니다. 두 패턴을 비교하여 상황에 맞는 멀티에이전트 아키텍처를 선택할 수 있는 판단력을 기릅니다.

학습 목표

LEARNING OBJECTIVES

- `Handoffs` 패턴: 상태 변수 기반 동적 구성(프롬프트 + 도구 교체)을 구현한다
- `Command` 객체로 상태 전이를 도구에서 트리거한다
- `Router` 패턴: 구조화 출력 분류 → `Send` API 병렬 실행 → 결과 합성

Part A — Handoffs: Customer Support 상태 머신

3.1 환경 설정

이전 장의 Subagents 패턴은 감독자가 서브에이전트를 도구로 호출하는 구조였습니다. 이 장에서 다루는 두 패턴은 근본적으로 다른 접근법을 취합니다:

- Part A — Handoffs: 단일 에이전트가 상태 변수에 따라 동적으로 프롬프트와 도구를 교체하는 상태 머신 패턴. `Command(update={...})` 로 상태 전이를 트리거합니다. 고객 지원처럼 순차적 단계를 거치는 워크플로에 적합합니다.
- Part B — Router: 쿼리를 분류하여 전문 에이전트들에게 `Send` API로 병렬 라우팅하고 결과를 합성하는 패턴. 여러 지식 소스에서 동시에 정보를 수집해야 할 때 적합합니다.

이 두 패턴과 Subagents를 비교하여, 상황에 맞는 멀티에이전트 아키텍처를 선택하는 판단력을 기릅니다. 세 패턴의 핵심 차이는 제어 흐름에 있습니다: Subagents는 감독자가 제어하고, Handoffs는 상태가 제어하며, Router는 분류 결과가 제어합니다.

```

from dotenv import load_dotenv
from langchain_openai import ChatOpenAI

load_dotenv()

model = ChatOpenAI(model="gpt-5.4")

```

3.2 Handoffs 개요

Handoffs 패턴은 단일 에이전트가 상태 변수에 따라 동적으로 행동을 바꾸는 아키텍처입니다. 여러 에이전트를 전환하는 것이 아니라, 하나의 에이전트가 단계(step)에 따라 다른 시스템 프롬프트와 도구 세트를 사용합니다. 이를 유한 상태 머신(Finite State Machine)에 비유할 수 있습니다. 각 상태는 에이전트의 “페르소나”를 정의하고, 전이 조건은 도구가 반환하는 `Command` 객체로 결정됩니다.

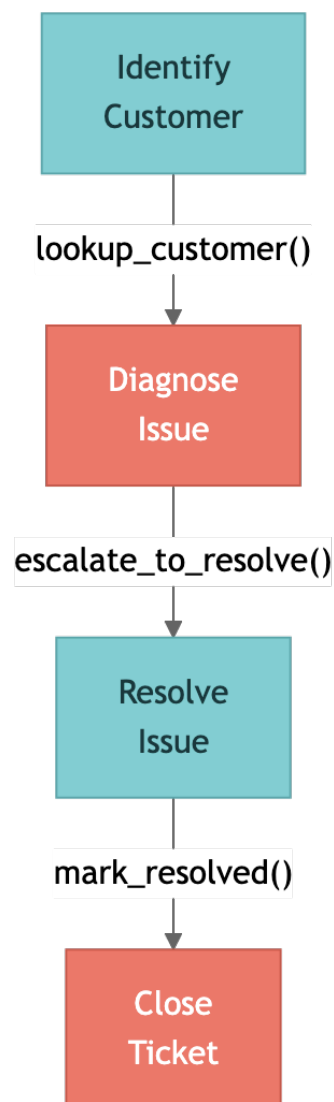


그림 읽는 법

이 그림은 단계 전이만 보면 됩니다. 각 상태는 현재 대화 단계이고, 상태 아래의 라벨은 다음 단계로 넘어가게 만드는 도구 호출 또는 `Command` 전이입니다.

핵심 메커니즘

메커니즘	설명
<code>current_step</code>	현재 단계를 추적하는 상태 변수. 이 값에 따라 에이전트의 행동이 결정됩니다
<code>Command(update={...})</code>	도구가 반환하여 상태 전이를 트리거. <code>current_step</code> 변경 + 추가 데이터 저장
<code>\@wrap_model_call</code>	미들웨어가 <code>current_step</code> 을 읽어 시스템 프롬프트와 사용 가능 도구를 동적으로 교체

Handoffs의 핵심 특성

- 상태 주도 행동(State-driven behavior): 추적된 상태 변수에 따라 설정이 조정됩니다
- 도구 기반 전이(Tool-based transitions): 도구가 `Command` 객체를 반환하여 상태를 업데이트합니다
- 직접 사용자 상호작용: 각 단계에서 독립적으로 메시지를 처리합니다
- 영속 상태(Persistent state): 상태가 대화 턴을 넘어 유지됩니다

! WARNING

도구가 `Command` 를 통해 메시지를 업데이트할 때, 매칭되는 `tool_call_id` 를 가진 `ToolMessage` 를 포함해야 합니다.

중요한 구현 세부사항

도구가 `Command` 를 통해 메시지를 업데이트할 때, 매칭되는 `tool_call_id` 를 가진 `ToolMessage` 를 포함해야 합니다. LLM은 도구 호출과 응답이 쌍을 이루길 기대하므로, 이를 누락하면 대화 히스토리가 잘못된 형태 (malformed)가 됩니다.

사용 시점

순차적 제약(sequential constraints)이 필요하거나, 각 상태에서 사용자와 직접 대화하거나, 다단계 플로우 (예: 고객 지원)에서 정보를 특정 순서로 수집해야 할 때 적합합니다. 반면, 단계 간 순서가 유연하거나 병렬 실행이 필요한 경우에는 Subagents나 Router 패턴이 더 적합합니다.

TIP

Handoffs vs. Subagents 선택 기준: 사용자가 각 단계에서 에이전트와 직접 대화해야 하면 Handoffs, 사용자가 감독자와만 대화하고 서브에이전트는 백그라운드에서 동작하면 Subagents를 선택하세요.

개념을 이해했으니, 이제 고객 지원 시나리오를 예제로 Handoffs 패턴을 단계별로 구현합니다. 먼저 상태 스키마를 정의합니다.

3.3 SupportState 정의

`AgentState` 를 상속하여 `current_step` 필드를 추가합니다. 이 필드가 상태 머신의 현재 노드를 결정하며, `Literal` 타입으로 유효한 단계를 제한합니다.

기본값은 `"identify_customer"` 로, 모든 대화가 고객 식별 단계에서 시작됩니다. 이후 도구가

`Command(update={"current_step": "..."})` 를 반환하면 자동으로 다음 단계로 전이됩니다.

```
from langchain.agents import AgentState
from typing import Literal

class SupportState(AgentState):
    current_step: Literal[
        "identify_customer", "diagnose_issue",
        "resolve_issue", "close_ticket",
    ] = "identify_customer"
```

상태 스키마가 정의되었으므로, 이제 각 단계에서 사용할 도구를 정의합니다. 도구는 Handoffs 패턴에서 상태 전이를 트리거하는 핵심 메커니즘입니다.

3.4 단계별 도구 정의

각 단계(step)에는 해당 단계의 역할에 맞는 도구들이 할당됩니다. 도구는 두 가지 유형으로 나뉩니다:

- 상태 전이 도구: `Command(update={...})` 를 반환하여 `current_step` 을 변경하고 추가 데이터를 상태에 저장합니다. `result` 필드로 LLM에게 보여줄 문자열 결과도 함께 전달합니다.
- 일반 도구: 문자열을 반환하며 상태를 변경하지 않습니다. 정보 조회 등에 사용됩니다.

설계 권장사항:

- 상태 전이는 `Command` 를 반환하는 도구를 통해서만 이루어져야 합니다
- 역방향 전이(이전 단계로 되돌아가기)도 필요 시 허용합니다
- 미들웨어에서 유효하지 않은 전이를 검증하여 단계 건너뛰기를 방지합니다

```
from langchain_core.tools import tool
from langgraph.types import Command

# --- Identify Customer ---
@tool
def lookup_customer(email: str) -> Command:
    """이메일로 고객을 조회합니다."""
    return Command(
        update={"customer": {"name": "Alice", "id": "C-1234"}, "current_step": "diagnose_issue"},
        result="고객 찾을: Alice (C-1234). 진단 단계로 이동합니다.",
    )
```

```
# --- Diagnose Issue ---
@tool
def check_service_status(service_name: str) -> str:
    """서비스의 현재 상태를 확인합니다."""
    return f"서비스 '{service_name}': 정상 (99.9% 가동률)"
```

```
@tool
def escalate_to_resolve(diagnosis: str) -> Command:
```

```

"""진단 후 해결 단계로 이동합니다."""
return Command(
    update={"diagnosis": diagnosis, "current_step": "resolve_issue"},
    result=f"진단 완료: {diagnosis}. 해결 단계로 이동합니다.",
)

```

```

# --- Resolve Issue ---
@tool
def apply_fix(fix_type: str, customer_id: str) -> Command:
    """고객 계정에 수정 사항을 적용합니다."""
    return Command(
        update={"resolution": {"type": fix_type}},
        result=f"수정 적용됨: {customer_id}에 {fix_type}",
    )

```

```

@tool
def mark_resolved(summary: str) -> Command:
    """이슈를 해결 완료로 표시하고 종료 단계로 이동합니다."""
    return Command(
        update={"current_step": "close_ticket", "resolution_summary": summary},
        result="해결됨. 종료 단계로 이동합니다.",
    )

```

```

# --- Close Ticket ---
@tool
def send_satisfaction_survey(customer_id: str) -> str:
    """만족도 설문을 전송합니다."""
    return "설문 전송 완료."

@tool
def close_ticket(ticket_id: str, notes: str) -> str:
    """지원 티켓을 종료합니다."""
    return f"티켓 {ticket_id} 종료됨."

```

모든 단계의 도구가 준비되었습니다. 이제 이 도구들을 `current_step` 상태에 따라 동적으로 할당하는 미들웨어를 구현합니다. 이 미들웨어가 Handoffs 패턴의 “두뇌” 역할을 합니다.

3.5 @wrap_model_call 미들웨어

`@wrap_model_call` 미들웨어는 Handoffs 패턴의 핵심입니다. LLM 호출을 가로채어 `current_step` 에 따라 시스템 프롬프트와 사용 가능 도구를 동적으로 교체합니다. Chapter 1에서 배운 `wrap_model_call` 의 실전 적용 사례입니다.

동작 순서:

1. 미들웨어가 상태에서 `current_step` 값을 읽습니다
2. `STEP_CONFIG` 디셔너리에서 해당 단계의 설정(프롬프트 + 도구)을 조회합니다
3. `config` 를 오버라이드하여 LLM에 전달합니다
4. `next_fn(state, config)` 으로 수정된 설정으로 LLM을 호출합니다

이것이 Handoffs의 핵심 메커니즘입니다: 단일 에이전트가 상태에 따라 완전히 다른 페르소나와 능력을 갖게 됩니다. 다중 에이전트를 만들 필요 없이, 미들웨어 하나로 동적 행동 변경을 달성합니다.

단일 에이전트 핸드오프 — `request.override(system_prompt=, tools=)`

복수 에이전트를 만들고 `Command(goto=...)` 로 그래프 노드 사이를 이동하는 다중 에이전트 핸드오프는 디버깅이 어렵고 상태 동기화 비용이 큼니다. v1에서는 `@wrap_model_call` 미들웨어 안에서

`request.override(system_prompt=, tools=)` 만 호출하면, 동일한 그래프 노드를 유지하면서 한 에이전트가 단계별로 페르소나와 도구를 바꿔 끼울 수 있습니다. 메시지 히스토리도 그대로 보존되므로, 사용자 입장에서는 “한 명의 상담원”이 흐름을 이어가는 것처럼 보입니다.

```
@wrap_model_call
def step_middleware(request, handler):
    """current_step에 따라 단일 에이전트의 system_prompt와 tools를 교체."""
    step = request.state.get("current_step", "identify_customer")
    cfg = STEP_CONFIG[step]
    request = request.override(
        system_prompt=cfg["system_prompt"],
        tools=cfg["tools"],
    )
    return handler(request)
```

! WARNING

`request.override(...)` 는 원본 `ModelRequest` 를 변경하지 않고 수정된 사본을 반환합니다. 반드시 반환값을 새 `request` 변수에 할당한 뒤 `handler(request)` 로 전달해야 합니다.

서브그래프 메시지 규칙 — `AIMessage + ToolMessage` 페어

도구가 `Command(update={"messages": [...]})` 로 메시지를 직접 업데이트할 때, 도구 호출과 응답이 짝을 이뤄야 합니다. 구체적으로는 다음 2개 메시지를 모두 포함해야 합니다.

1. `AIMessage` — `tool_calls=[{"id": ..., "name": ..., "args": ...}]` 형태로 도구 호출을 선언.
2. `ToolMessage` — 동일한 `tool_call_id` 를 가진 응답 메시지.

이 페어를 누락하면 후속 모델 호출이 `Unmatched tool_call_id` 오류를 발생시킵니다. LangChain v1은

`Command(update={...}, result=...)` 헬퍼로 이 페어를 자동 생성해 주지만, 직접 `messages` 키를 수정할 때는 수동으로 두 메시지를 함께 넣어야 합니다.

```
from langchain.messages import AIMessage, ToolMessage
from langgraph.types import Command

# Command.result를 쓰면 두 메시지가 자동 페어로 생성됨
return Command(
    update={"current_step": "diagnose_issue", "customer": {...}},
    result="고객을 찾았습니다. 진단 단계로 이동합니다.", # ToolMessage.content
)

# 직접 messages를 만들 때는 AIMessage + ToolMessage를 함께 넣어야 함
return Command(update={
    "current_step": "diagnose_issue",
    "messages": [
        AIMessage(content="", tool_calls=[{"id": "c1", "name": "handoff", "args": {}}]),
        ToolMessage(content="진단 단계로 이동", tool_call_id="c1"),
    ],
})
```

! WARNING

STEP_CONFIG 에 정의되지 않은 `current_step` 값이 설정되면 `KeyError` 가 발생합니다. 프로덕션 코드에서는 반드시 디폴트 처리나 유효성 검증을 추가하세요. 예:

```
cfg = STEP_CONFIG.get(step, STEP_CONFIG["identify_customer"])
```

다음 코드는 단계별 설정 딕셔너리와 이를 활용하는 미들웨어 구현입니다.

```
STEP_CONFIG = {
    "identify_customer": {
        "tools": [lookup_customer],
        "system_prompt": "고객을 식별하세요. 이메일 또는 계정 ID를 요청하세요.",
    },
    "diagnose_issue": {
        "tools": [check_service_status, escalate_to_resolve],
        "system_prompt": "이슈를 진단하세요. 도구를 사용한 후 escalate_to_resolve를 호출하세요.",
    },
}
```

```
STEP_CONFIG["resolve_issue"] = {
    "tools": [apply_fix, mark_resolved],
    "system_prompt": "이슈를 해결하세요. 수정을 적용한 후 mark_resolved를 호출하세요.",
}
STEP_CONFIG["close_ticket"] = {
    "tools": [send_satisfaction_survey, close_ticket],
    "system_prompt": "고객에게 감사를 전하고, 설문을 보내고, 티켓을 종료하세요.",
}
```

```
from langchain.agents.middleware import wrap_model_call

@wrap_model_call
def step_middleware(request, handler):
    """current_step에 따라 에이전트를 동적으로 구성합니다."""
    step = request.state.get("current_step", "identify_customer")
    cfg = STEP_CONFIG[step]
    request = request.override(
        system_prompt=cfg["system_prompt"],
        tools=cfg["tools"],
    )
    return handler(request)
```

3.6 에이전트 생성 및 실행 흐름

에이전트 생성 시 모든 도구를 등록하되, `state_schema=SupportState` 와 미들웨어를 지정합니다. 미들웨어가 런타임에 `current_step` 에 따라 도구를 필터링하므로, 각 단계에서는 해당 단계의 도구만 LLM에 노출됩니다.

실행 흐름 예시:

```
[identify_customer] User: "로그인이 안 돼요. 이메일: alice@example.com"
→ lookup_customer("alice@example.com")
← Command(update={customer: {...}, current_step: "diagnose_issue"})

[diagnose_issue] Agent: "계정을 찾았습니다. 어떤 문제가 있나요?"
→ check_service_status("auth-service") → "healthy"
→ escalate_to_resolve("3회 로그인 실패로 잠김")

[resolve_issue] → apply_fix("reset_password", "C-1234")
→ mark_resolved("비밀번호 재설정 완료")
```



```
[close_ticket] → send_satisfaction_survey("C-1234")
→ close_ticket("T-5678", notes="비밀번호 재설정")
```

각 단계 전이는 `Command` 를 반환하는 도구에 의해 자동으로 이루어집니다.

```
from langchain.agents import create_agent

all_tools = [
    lookup_customer, check_service_status,
    escalate_to_resolve, apply_fix, mark_resolved,
    send_satisfaction_survey, close_ticket,
]
support_agent = create_agent(
    model="gpt-5.4", tools=all_tools,
    state_schema=SupportState, middleware=[step_middleware],
)
```

```
# [identify_customer]
# User: "Can't log in. Email: alice@example.com"
# Agent -> lookup_customer("alice@example.com")
#     <- Command(update={current_step: "diagnose_issue"})
#
# [diagnose_issue] (auto transition)
# Agent -> check_service_status("auth-service")
# Agent -> escalate_to_resolve("Locked out")
#
# [resolve_issue] -> [close_ticket]
```

Part A에서는 단일 에이전트가 상태 전이를 통해 다단계 워크플로를 처리하는 Handoffs 패턴을 살펴보았습니다. Part B에서는 이와 대조적인 접근법인 Router 패턴을 다룹니다. Router는 병렬 실행이 핵심이며, 여러 지식 소스에서 동시에 정보를 수집하여 합성하는 데 최적화되어 있습니다.

Part B — Router: 병렬 라우팅과 결과 합성

3.7 Router 개요

Router 패턴은 입력을 분류하여 전문 에이전트들에게 라우팅하는 아키텍처입니다. Subagents 패턴과 달리, Router는 전용 분류 단계(단일 LLM 호출 또는 규칙 기반 로직)를 거쳐 쿼리를 배분합니다. 이 패턴은 검색 엔진의 fan-out/fan-in 아키텍처와 유사합니다. 쿼리를 여러 인덱스에 동시에 보내고(fan-out), 결과를 수집하여 통합합니다(fan-in).

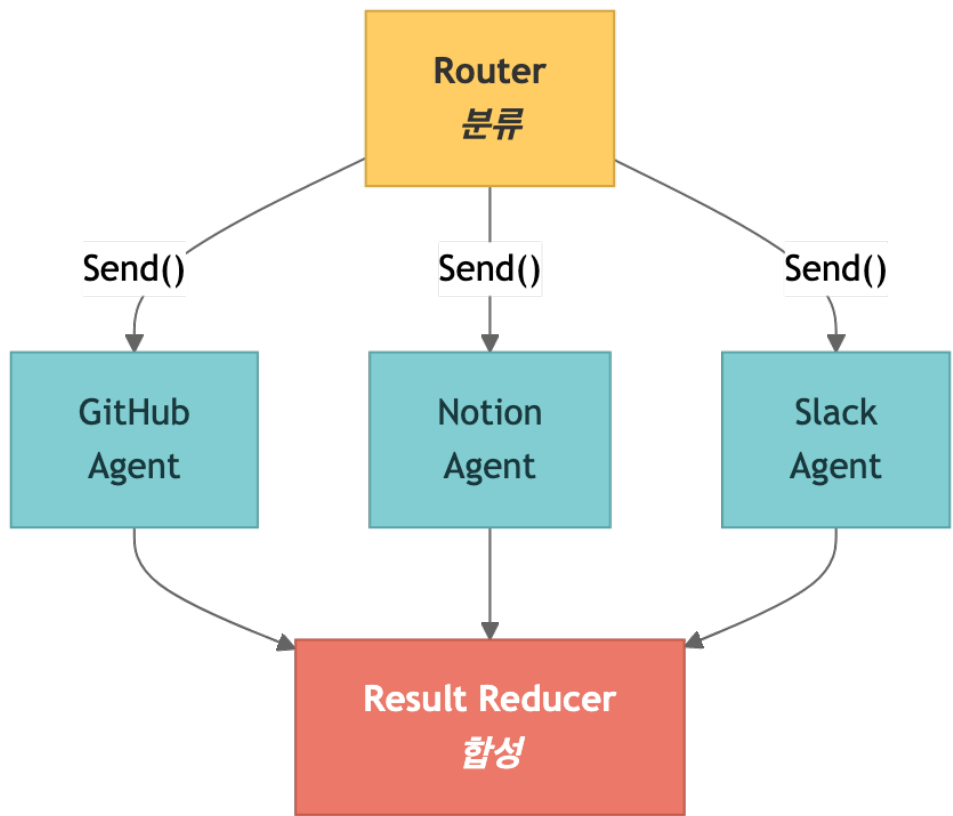


그림 읽는 법

위쪽 Router가 먼저 입력을 분류하고, 가운데 전문 에이전트들이 병렬로 작업한 뒤, 마지막 Reducer가 결과를 다시 하나로 합칩니다. 즉 **분류** → **병렬 실행** → **합성** 구조로 이해하면 됩니다.

파이프라인

단계	역할	구현
분류(Classification)	쿼리를 분석하여 관련 소스와 서브쿼리를 생성	<code>with_structured_output(QueryClassification)</code>
병렬 디스패치	분류된 각 소스에 동시에 서브쿼리 전달	<code>Send</code> API
결과 합성(Reduction)	모든 에이전트 결과를 수집하여 통합 응답 생성	Reducer 노드 + LLM

Router vs. Subagents (Supervisor) 비교

세 패턴은 제어 흐름 주체가 다릅니다. 잘못 선택하면 라우팅 비용이 폭증하거나 컨텍스트 격리가 깨집니다.

패턴	제어 주체	실행 모드	적합 시나리오
Router	별도 분류 단계 (LLM 또는 규칙)	병렬 fan-out + fan-in	지식 도메인 분리 + 병렬 검색
Supervisor (Subagents)	감독자 에이전트의 동적 도구 호출	순차 또는 병렬 (감독자 결정)	대화형 상호작용 + 여러 도메인 조율
Handoffs	상태 변수 (<code>current_step</code>)	단일 에이전트가 페르소나 교체	순차적 다단계 워크플로

Router는 “전용 라우팅 단계(분류)”가 있고, Subagents는 “감독자 에이전트가 동적으로” 호출 대상을 결정합니다. 별개의 지식 도메인(vertical)이 명확히 구분되어 있고 병렬 조치가 필요할 때 Router가 적합합니다. 반면, 여러 도메인을 조율하면서 대화형 상호작용이 필요하다면 Subagents가 더 적합합니다.

➡ TIP

Router의 분류 단계에서 `with_structured_output` 을 사용하면 분류 결과가 Pydantic 모델로 반환되어 타입 안전한 라우팅이 가능합니다. 규칙 기반 분류(if/else)는 빠르지만 유연성이 떨어지고, LLM 기반 분류는 느리지만 복잡한 쿼리도 처리할 수 있습니다.

아키텍처 모드

- Stateless: 각 요청이 독립적으로 라우팅됩니다 (메모리 없음)
- Stateful: 대화 히스토리를 유지하여 멀티턴 상호작용을 지원합니다. Stateless 라우터를 도구로 래핑하거나, 라우터 자체가 상태를 직접 관리하는 방식이 있습니다

Router의 전체 아키텍처를 이해했으니, 이제 분류 스키마와 상태를 정의합니다. 분류 스키마의 품질이 라우팅의 정확도를 직접적으로 결정하므로, 필드 설계에 신중해야 합니다.

3.8 RouterState 및 분류 스키마

`QueryClassification` 은 Pydantic 모델로, LLM의 `with_structured_output()` 을 통해 쿼리를 구조화된 형태로 분류합니다. `RouterState` 는 분류 결과, 소스 목록, 서브쿼리, 에이전트 결과를 추적합니다.

분류 스키마의 핵심 필드:

- `sources` : 어떤 지식 소스가 관련 있는지 (복수 선택 가능)
- `reasoning` : 왜 해당 소스를 선택했는지 설명
- `sub_queries` : 소스별로 최적화된 서브쿼리 (원래 쿼리를 각 소스에 맞게 재구성)

```
from pydantic import BaseModel, Field
from typing import Literal

class SubQuery(BaseModel):
    """소스별 하위 쿼리."""
    source: Literal["github", "notion", "slack"] = Field(description="지식 소스.")
```

```

query: str = Field(description="해당 소스에 최적화된 검색 쿼리.")

class QueryClassification(BaseModel):
    """사용자 쿼리의 분류 결과."""
    sources: list[Literal["github", "notion", "slack"]] = Field(
        description="관련 지식 소스."
    )
    reasoning: str = Field(description="해당 소스를 선택한 이유.")
    sub_queries: list[SubQuery] = Field(description="소스별 하위 쿼리.")

```

```

from langchain.agents import AgentState

class RouterState(AgentState):
    classification: QueryClassification = None
    sources: list[str] = []
    sub_queries: list[SubQuery] = []
    agent_results: list[dict] = []

```

3.9 분류 노드

`with_structured_output` 으로 쿼리를 소스별로 분류하고, 각 소스에 최적화된 서브쿼리를 생성합니다.

분류 예시

사용자 쿼리	분류 소스	이유
“auth 서비스 배포 방법”	["github", "notion"]	배포 코드는 GitHub, 절차 문서는 Notion에 존재
“API 변경 결정 경위”	["slack", "notion"]	논의는 Slack, 결정 문서는 Notion에 기록
“로그인 버그 PR”	["github"]	PR은 GitHub에만 존재
“온보딩 프로세스와 스타터 레포”	["github", "notion", "slack"]	레포는 GitHub, 프로세스는 Notion, 맥락은 Slack

서브쿼리 생성이 중요합니다: “auth 서비스 배포”라는 원본 쿼리를 GitHub에는

“auth service deployment scripts CI/CD pipeline”, Notion에는

“auth service deployment process procedure runbook” 으로 각각 최적화합니다.

분류가 완료되면, 각 소스에 서브쿼리를 동시에 전달하는 병렬 라우팅 단계로 진입합니다. 이 단계가 Router 패턴의 성능 이점을 실현하는 핵심입니다.

3.10 병렬 라우팅 (Send API)

`Send` API는 분류된 각 소스에 동시에 서브쿼리를 디스패치합니다. `Send(node_name, payload)` 형태로, 그래프의 특정 노드에 데이터를 병렬로 전달합니다. LangGraph의 `Send` 는 MapReduce 패턴의 Map 단계에 해당합니다.

이 병렬 실행이 Router 패턴의 핵심 강점입니다: 여러 지식 소스를 순차적으로 조회하면 지연 시간이 합산되지만, `Send` 를 통한 병렬 실행은 가장 느린 소스의 응답 시간만큼만 걸립니다. 예를 들어, GitHub(2초), Notion(1초), Slack(3초)을 순차 조회하면 6초가 걸리지만, 병렬 실행하면 3초면 충분합니다.

! WARNING

`Send` API로 병렬 실행할 때, 각 에이전트 노드의 상태는 독립적입니다. 에이전트 간에 상태를 공유하려면 Reducer 노드에서 결과를 수집한 후 처리해야 합니다.

새로운 소스 추가하기

Router 패턴은 확장이 간단합니다:

1. 소스별 도구를 정의합니다
2. 전문 에이전트를 생성합니다
3. `QueryClassification.sources` 에 새 소스를 추가합니다
4. 그래프에 에이전트 노드를 추가합니다
5. Reducer에 연결합니다

```
from langchain_core.tools import tool
from langchain.agents import create_agent

@tool
def search_github_code(query: str) -> str:
    """GitHub 저장소를 검색합니다."""
    return f"'{query}'에 대한 GitHub 결과"

@tool
def search_notion_pages(query: str) -> str:
    """Notion 워크스페이스를 검색합니다."""
    return f"'{query}'에 대한 Notion 결과"
```

```
@tool
def search_slack_messages(query: str) -> str:
    """Slack 메시지를 검색합니다."""
    return f"'{query}'에 대한 Slack 결과"
```

```
github_agent = create_agent(
    model="gpt-5.4", tools=[search_github_code],
    system_prompt="GitHub에서 코드와 PR을 검색합니다.",
    name="github_agent",
)
notion_agent = create_agent(
    model="gpt-5.4", tools=[search_notion_pages],
    system_prompt="Notion에서 문서를 검색합니다.",
    name="notion_agent",
)
```

```
slack_agent = create_agent(
    model="gpt-5.4", tools=[search_slack_messages],
    system_prompt="Slack에서 토론을 검색합니다.",
    name="slack_agent",
)
```

```
from langgraph.types import Send

def dispatch_to_agents(state):
    """하위 쿼리를 에이전트들에게 병렬로 전달합니다."""
    cls = state["classification"]
    sq_dict = {sq.source: sq.query for sq in cls.sub_queries}
    return [
        Send(src, {"messages": [{"role": "user", "content": sq_dict.get(src, "")}], "source": src})
        for src in cls.sources
    ]
```

병렬 라우팅의 핵심은 분류 결과의 각 항목마다 `Send(노드_이름, 페이로드)` 객체를 리스트로 반환하는 것입니다. LangGraph는 이 리스트의 길이만큼 동시에 노드를 실행하고, 모든 분기가 끝나면 다음 노드로 모입니다. 분류 결과를 `[{"agent": "github", "query": "..."}, ...]` 형태의 dict 리스트로 받는다면 다음과 같이 작성할 수 있습니다.

```
def dispatch_dict_form(state):
    """분류 결과가 dict 리스트일 때의 fan-out."""
    return [
        Send(c["agent"], {"messages": [{"role": "user", "content": c["query"]}]}))
        for c in state["classification"]
    ]
```

TIP

`Send` 는 같은 노드를 복수 번 호출할 수도 있습니다. 예를 들어 `[Send("github", q1), Send("github", q2)]` 처럼 같은 에이전트에 두 개의 서로 다른 쿼리를 동시에 보내고 결과를 합치는 패턴도 가능합니다.

3.11 결과 합성

모든 에이전트의 결과가 수집되면 Reducer 노드에서 통합 응답을 합성합니다. 이 단계는 MapReduce의 Reduce에 해당하며, 단순한 결과 나열이 아닌 의미 있는 통합이 목표입니다. Reducer는 LLM을 사용하여 통합된 응답을 합성합니다. 합성 시 각 정보의 출처(source)를 인용하여 사용자가 어디서 온 정보인지 파악할 수 있게 합니다.

합성 프롬프트에서는 소스를 명시하도록 지시합니다. 예를 들어, “배포 스크립트는 GitHub의 `payment-service` 레포에 있고(GitHub), 배포 절차는 Notion의 ‘Payment Service Ops’ 문서를 참고하세요(Notion)”와 같이 응답합니다.

```
from langgraph.graph import StateGraph, START, END

graph = StateGraph(RouterState)
graph.add_node("router", route_query)
graph.add_node("github", github_agent)
graph.add_node("notion", notion_agent)
graph.add_node("slack", slack_agent)
graph.add_node("reducer", reduce_results)
```

```
<langgraph.graph.state.StateGraph at 0x1bfb57ab230>
```

```
graph.add_edge(START, "router")
graph.add_conditional_edges("router", dispatch_to_agents)
graph.add_edge("github", "reducer")
graph.add_edge("notion", "reducer")
graph.add_edge("slack", "reducer")
graph.add_edge("reducer", END)

app = graph.compile()
```

요약

Part A – Handoffs

항목	핵심
패턴	단일 에이전트 + <code>current_step</code> 기반 동적 구성
전이	<code>Command(update={"current_step": "next"})</code>
동적 구성	<code>\@wrap_model_call</code> 로 프롬프트 + 도구 교체

Part B – Router

항목	핵심
분류	<code>with_structured_output(QueryClassification)</code>
병렬	<code>Send(source, payload)</code> API
합성	Reducer 노드에서 LLM 통합 응답

Handoffs, Router, Subagents 세 가지 멀티에이전트 패턴을 모두 학습했습니다. 패턴 선택의 핵심 기준을 정리하면: 순차적 다단계 워크플로에는 Handoffs, 중앙 집중 위임과 대화형 상호작용에는 Subagents, 병렬 지식 검색과 결과 합성에는 Router가 적합합니다. 실제 프로젝트에서는 이 패턴들을 조합하여 사용하는 경우도 많습니다. 예를 들어, 감독자(Subagents)가 Router를 서브에이전트로 호출하여 병렬 검색 후 결과를 통합하는 구성이 가능합니다.

다음 장에서는 이 모든 패턴의 기반이 되는 컨텍스트 엔지니어링과 장기 메모리 시스템을 심층적으로 다룹니다.

04 컨텍스트 엔지니어링 & 메모리 심화

- Static/Dynamic Context, InMemoryStore, Skills 패턴

앞선 장들에서 미들웨어, 서브에이전트, Handoffs, Router를 학습하며 에이전트의 실행 흐름을 제어하는 방법을 익혔습니다. 이 장에서는 에이전트 성능의 또 다른 축인 정보 관리에 집중합니다. LangGraph의 컨텍스트 시스템과 장기 메모리(Store)를 심층 학습합니다. 정적/동적 런타임 컨텍스트부터 시맨틱 검색 기반 장기 메모리, 그리고 Progressive Disclosure(Skills) 패턴까지 다룹니다.

학습 목표

LEARNING OBJECTIVES

- 컨텍스트 엔지니어링의 2차원(Mutability x Lifetime) 매트릭스를 이해한다
- `context_schema` + `@dataclass` 로 정적 런타임 컨텍스트를 구현한다
- `state_schema` 와 `AgentState` 커스텀으로 동적 런타임 컨텍스트를 관리한다
- `InMemoryStore` 의 `namespace`, `put`, `get`, `search` API를 활용한다
- 시맨틱 검색 기반 장기 메모리를 구축한다
- 메모리 3유형(Semantic, Episodic, Procedural)을 구분하여 설계한다
- Skills 패턴으로 Progressive Disclosure를 구현한다
- Hot path vs Background 메모리 쓰기 전략을 비교한다

4.1 환경 설정

```
from dotenv import load_dotenv
load_dotenv(override=True)

from langchain_openai import ChatOpenAI, OpenAIEmbeddings

model = ChatOpenAI(model="gpt-5.4")
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
print("환경 준비 완료.")
```

환경 준비 완료.

환경이 준비되었으므로, 컨텍스트 엔지니어링의 개념 프레임워크부터 학습합니다. 이 프레임워크는 이후 모든 구현의 이론적 토대가 됩니다.

4.2 컨텍스트 엔지니어링 개요

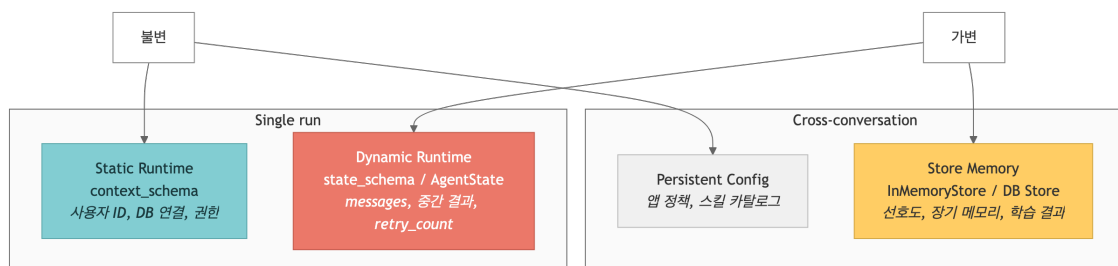
컨텍스트 엔지니어링은 “올바른 정보를, 올바른 형식으로, 올바른 시점에” AI에 제공하는 시스템 설계입니다. 단순한 프롬프트 엔지니어링을 넘어, 컨텍스트를 런타임에 프로그래밍 방식으로 조립하는 아키텍처적 접근입니다. 프롬프트 엔지니어링이 “무엇을 말할지”에 집중한다면, 컨텍스트 엔지니어링은 “어떤 정보를 언제, 어떻게 조립하여 제공할지”에 집중합니다.

에이전트가 실패하는 주된 원인은 두 가지입니다:

1. LLM 능력 부족
2. 컨텍스트 부족 또는 부적절한 컨텍스트 (더 빈번한 원인)

실제로 GPT-4 수준의 모델도 필요한 정보가 컨텍스트에 없으면 올바른 답을 생성할 수 없습니다. 반면, 적절한 컨텍스트가 주어지면 비교적 작은 모델도 놀라운 성능을 보입니다. 따라서 컨텍스트 엔지니어링은 AI 엔지니어의 핵심 역할이며, 에이전트 신뢰성의 근본적인 해결책입니다.

2차원 매트릭스: Mutability x Lifetime



이 다이어그램은 표의 행과 열을 “언제 바뀌는가”와 “얼마나 오래 유지되는가”라는 두 질문으로 다시 읽게 해 줍니다. 정리하면 다음 3줄 규칙으로 기억하면 됩니다:

1. Static Runtime – 한 번의 실행 동안 고정되어야 하는 정보
2. Dynamic Runtime – 실행 중 계속 변하는 작업 상태와 중간 결과
3. Store – 다음 대화에서도 다시 써야 하는 장기 메모리

Static (불변)	User ID, DB 연결, 도구 정의	설정 파일 등
Dynamic (가변)	대화 히스토리, 중간 결과	사용자 선호도, 학습된 메모리

3가지 컨텍스트 타입

타입	Mutability	Lifetime	예시	LangGraph 구현
Static Runtime	Static	Single run	User ID, DB conn	<code>context_schema</code>
Dynamic Runtime (State)	Dynamic	Single run	Messages, 중간결과	<code>state_schema</code>
Dynamic Cross-conv (Store)	Dynamic	Cross-conversation	선호도, 메모리	<code>InMemoryStore</code>

제어 가능한 3가지 컨텍스트 카테고리

카테고리	제어 대상	특성
Model Context	Instructions, 메시지 히스토리, 도구, 응답 형식	Transient (일시적)
Tool Context	도구 접근, 상태 읽기/쓰기, 런타임 컨텍스트	Persistent (영구적)
Life-cycle Context	단계 간 변환, 요약, 가드레일	Persistent (영구적)

LangChain은 미들웨어(middleware) 메커니즘으로 컨텍스트 엔지니어링을 구현합니다. `@dynamic_prompt`, `@wrap_model_call` 등의 미들웨어로 컨텍스트를 업데이트하거나 라이프사이클 단계 간 제어를 할 수 있습니다. Chapter 1에서 학습한 미들웨어가 여기서 컨텍스트 관리의 핵심 도구로 활용됩니다.

TIP

컨텍스트 엔지니어링의 3가지 타입(Static Runtime, Dynamic Runtime, Cross-conversation)은 상호 배타적이지 않습니다. 하나의 에이전트에서 세 가지를 모두 사용하는 것이 일반적이며, 각 타입은 서로 다른 종류의 정보를 담당합니다.

개념 프레임워크를 이해했으니, 이제 각 컨텍스트 타입을 구체적인 코드로 구현해 보겠습니다. 가장 단순한 정적 런타임 컨텍스트부터 시작합니다.

4.3 정적 런타임 컨텍스트 - context_schema + @dataclass

에이전트 실행 중 변하지 않는 정보를 context_schema 로 주입합니다. “변하지 않는다”는 것은 invoke() 호출 시 설정된 값이 해당 실행 전체에 걸쳐 고정된다는 의미입니다. 동일한 에이전트를 다른 사용자에게 대해 호출할 때는 다른 컨텍스트를 전달합니다. @dataclass 로 스키마를 정의하고, 도구에서 ToolRuntime[Context] 로 접근합니다.

다음 코드는 사용자 정보를 정적 컨텍스트로 정의하고, 도구에서 이를 활용하는 패턴입니다.

create_agent(context_schema=...) 로 스키마를 등록하고, 도구는 ToolRuntime[Context] 타입 어노테이션으로 runtime.context.user_id 같이 도트 접근합니다.

```
from dataclasses import dataclass
from langchain.tools import tool, ToolRuntime
from langchain.agents import create_agent

@dataclass
class Context:
    user_id: str
    role: str
    department: str
```

```
@tool
def get_permissions(runtime: ToolRuntime[Context]) -> str:
    """현재 사용자의 역할에 따른 권한을 조회합니다."""
    ctx = runtime.context
    perms = {"admin": "read,write,delete", "editor": "read,write"}
    return f"사용자 {ctx.user_id} ({ctx.department}): {perms.get(ctx.role, 'read')}"

agent = create_agent(
    model=model,
    tools=[get_permissions],
    context_schema=Context, # 스키마 등록
)
# invoke 시 context= 파라미터로 인스턴스 전달
agent.invoke(
    {"messages": [{"role": "user", "content": "내 권한은?"}]},
    context=Context(user_id="u42", role="admin", department="eng"),
)
```

runtime.execution_info / runtime.server_info

runtime 은 컨텍스트 외에도 실행 메타데이터를 제공합니다. runtime.execution_info 는 현재 invoke의 thread_id/run_id/step을 포함하고, runtime.server_info 는 LangGraph 서버 환경 정보(deployment id, host 등)를 제공합니다. 감사 로그나 분산 추적에 유용합니다.

```
@tool
def audited_action(action: str, runtime: ToolRuntime[Context]) -> str:
    """도구 호출 시점의 실행 정보를 함께 기록합니다."""
    info = runtime.execution_info # thread_id, run_id 등
    print(f"[audit] user={runtime.context.user_id} "
          f"thread={info.thread_id} run={info.run_id} action={action}")
    return f"실행됨: {action}"
```

핵심 포인트

- context_schema 에 @dataclass 를 전달하면 타입 안전한 컨텍스트를 사용할 수 있습니다
- 도구 함수에서 runtime: ToolRuntime[Context] 타입힌트로 자동 주입됩니다
- 실행 중에는 읽기 전용이며 변경되지 않습니다

- 적합한 데이터: User ID, DB 연결, API 키, 세션 메타데이터

정적 컨텍스트는 실행 중 불변이므로 안전하고 예측 가능합니다. 그러나 에이전트가 도구를 호출하면서 중간 결과를 추적해야 하는 경우에는 동적 상태가 필요합니다.

4.4 동적 런타임 컨텍스트 - `state_schema`, `AgentState` 커스텀

정적 컨텍스트가 “누가 요청했는가”를 다룬다면, 동적 런타임 컨텍스트는 “지금까지 무엇이 일어났는가”를 추적합니다. 에이전트가 메시지를 처리하고 도구를 호출하면서 변화하는 상태입니다. `AgentState` 를 상속하여 커스텀 필드를 추가합니다.

! WARNING

`AgentState` 의 기본 필드(`messages` , `jump_to` , `structured_response`)는 시스템이 사용하는 예약 필드입니다. 커스텀 필드를 추가할 때 이 이름들과 충돌하지 않도록 주의하세요.

```
from langchain.agents import AgentState

class RAGState(AgentState):
    """동적 검색 컨텍스트를 포함한 상태."""
    retrieved_docs: list[str]
    query_count: int

print(f"상태 키: {list(RAGState.__annotations__.keys())}")
```

상태 키: ['messages', 'jump_to', 'structured_response', 'retrieved_docs', 'query_count']

Static vs Dynamic 비교

구분	Static Runtime (<code>context_schema</code>)	Dynamic Runtime (<code>state_schema</code>)
변경 여부	불변 (읽기 전용)	가변 (노드가 업데이트)
전달 방식	<code>context=</code> 파라미터	invoke 입력 dict
접근 방법	<code>runtime.context.field</code>	<code>state["field"]</code>
적합한 데이터	인증 정보, 설정	대화 히스토리, 중간 결과

정적 컨텍스트와 동적 상태는 모두 단일 실행(run) 범위입니다. 하지만 사용자의 선호도, 과거 상호작용 패턴, 학습된 규칙 등은 여러 대화 세션에 걸쳐 지속되어야 합니다. 이를 위해 LangGraph는 `InMemoryStore` 를 제공합니다.

정적 컨텍스트와 동적 상태는 모두 단일 실행(run) 범위입니다. 에이전트를 종료하면 상태가 사라집니다. 그러나 “이 사용자는 Python을 선호한다”, “지난번에 pytest 관련 질문을 했다” 같은 정보는 세션을 넘어 지속되어야 합니다. 이를 위해 LangGraph는 장기 메모리 시스템을 제공합니다.

4.5 장기 메모리 – InMemoryStore 기본 API

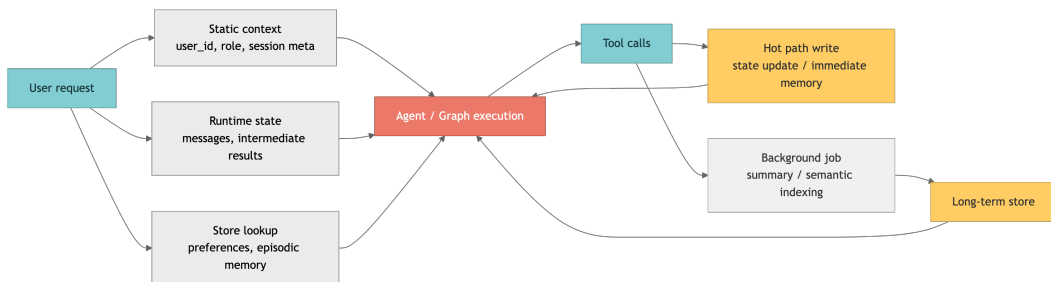
Cross-conversation 컨텍스트를 위해 `InMemoryStore` 를 사용합니다. 장기 메모리는 세션과 스레드를 초월하여 지속되는 사용자별 또는 앱 수준의 데이터입니다. 이는 인간의 장기 기억에 해당하며, 에이전트가 사용자에 대해 “학습”한 내용을 영구적으로 보존합니다.

저장 구조

메모리는 JSON 문서 형태로 저장되며, 계층적 namespace로 조직화됩니다:

- namespace: 메모리를 분류하는 폴더 역할 (예: `(user_id, "preferences")`)
- key: 각 메모리의 고유 식별자 (예: `"theme"`)
- 네임스페이스에는 보통 사용자 ID나 조직 ID를 포함하여 정보 관리를 용이하게 합니다

기본 API



핵심은 읽기 경로와 쓰기 경로를 분리해서 보는 것입니다. 현재 답변에 즉시 필요한 정보는 hot path에서 state로 반영하고, 장기적으로 축적할 내용만 background 경로를 통해 Store에 기록하면 토큰 비용과 지연 시간을 함께 관리할 수 있습니다.

API	설명
<code>store.put(namespace, key, value)</code>	메모리 저장 (upsert)
<code>store.get(namespace, key)</code>	특정 키로 메모리 조회
<code>store.search(namespace)</code>	네임스페이스 내 전체 검색

```
store.search(namespace, filter={...})
```

필터 조건으로 검색

! WARNING

`InMemoryStore` 는 프로세스 메모리에 데이터를 저장하므로, 서버 재시작 시 모든 데이터가 유실됩니다. 프로덕션 환경에서는 반드시 DB 기반 Store (예: PostgreSQL)를 사용해야 합니다. `InMemoryStore` 는 개발과 테스트 용도로만 사용하세요.

PostgreSQL Store – 프로덕션 영속 백엔드

`langgraph-checkpoint-postgres` 패키지의 `PostgresStore` 는 `InMemoryStore` 와 동일한 `put` / `get` / `search` API를 제공하면서 PostgreSQL에 영속화합니다. `pgvector` 확장과 결합하면 시맨틱 검색도 그대로 지원됩니다. 프로세스 재시작이나 멀티 워커 환경에서도 메모리가 유지되므로 프로덕션의 기본 선택지입니다.

```
# pip install langgraph-checkpoint-postgres
from langgraph.store.postgres import PostgresStore

DB_URI = "postgresql://user:pass@localhost:5432/langgraph?sslmode=disable"

with PostgresStore.from_conn_string(DB_URI) as store:
    store.setup() # 최초 1회 - 테이블 생성

    # 시맨틱 검색 활성화
    store_with_index = PostgresStore.from_conn_string(
        DB_URI,
        index={"embed": embeddings, "dims": 1536},
    )

    agent = create_agent(
        model=model,
        tools=[get_user_info, save_preference],
        store=store_with_index,
        context_schema=Context,
    )
```

TIP

`PostgresStore` 는 동기 컨텍스트 매니저이고, 비동기 환경에서는 `AsyncPostgresStore` 를 사용합니다. 둘 다 동일한 API를 제공하므로 `await store.aput(...)` / `await store.asearch(...)` 형태로 호출하면 됩니다.

다음 코드는 `InMemoryStore` 의 기본 CRUD 작업(`put`, `get`, `search`)을 보여줍니다.

```
from langgraph.store.memory import InMemoryStore

store = InMemoryStore()
user_id = "user_42"
store.put((user_id, "preferences"), "theme", {"value": "dark"})
store.put((user_id, "preferences"), "language", {"value": "ko"})

item = store.get((user_id, "preferences"), "theme")
print(f"테마: {item.value}")
```

```
테마: {'value': 'dark'}
```

```
items = store.search((user_id, "preferences"))
for item in items:
    print(f" [{item.key}] = {item.value}")

filtered = store.search(
    (user_id, "preferences"), filter={"value": "dark"}
)
print(f"필터 결과: {len(filtered)}건")
```

```
[theme] = {'value': 'dark'}
[language] = {'value': 'ko'}
필터 결과: 1건
```

키 기반 조회는 정확한 키를 알아야만 메모리에 접근할 수 있습니다. 그러나 에이전트가 “이 사용자는 어떤 테스트 방법을 좋아하는가?”처럼 의미 기반으로 관련 메모리를 찾아야 하는 경우, 시맨틱 검색이 필요합니다.

키 기반 조회는 정확한 키를 알아야만 메모리에 접근할 수 있습니다. 그러나 에이전트가 “이 사용자는 어떤 테스트 방법을 좋아하는가?”처럼 의미 기반으로 관련 메모리를 찾아야 하는 경우, 정확한 키를 미리 알 수 없습니다. 이때 시맨틱 검색이 필요합니다.

4.6 장기 메모리 – 시맨틱 검색

임베딩 함수를 설정하면 `InMemoryStore` 가 시맨틱 검색을 지원합니다. 시맨틱 검색은 텍스트를 벡터로 변환한 후 코사인 유사도로 관련 메모리를 찾는 방식입니다. `query` 파라미터로 의미 기반 유사도 검색을 수행합니다. 키워드가 정확히 일치하지 않아도, 의미적으로 관련 있는 메모리를 찾을 수 있습니다.

다음 코드에서 “testing preferences”라는 쿼리로 검색하면, “pytest를 unittest보다 선호”라는 메모리가 반환됩니다. “testing”이나 “preferences”라는 단어가 메모리에 없어도, 의미적 유사성으로 매칭됩니다.

```
semantic_store = InMemoryStore(
    index={"embed": embeddings, "dims": 1536}
)
ns = ("user_42", "memories")
semantic_store.put(ns, "mem1", {"content": "pytest를 unittest보다 선호"})
semantic_store.put(ns, "mem2", {"content": "모든 함수에 타입 힌트 사용"})
semantic_store.put(ns, "mem3", {"content": "좋아하는 음식은 초밥"})
semantic_store.put(ns, "mem4", {"content": "ML 인프라 팀에서 근무"})
print("임베딩과 함께 메모리 4개 저장 완료.")
```

임베딩과 함께 메모리 4개 저장 완료.

```
results = semantic_store.search(
    ("user_42", "memories"), query="testing preferences", limit=2
)
for r in results:
    print(f" [{r.key}] {r.value['content']}")
```

```
[mem1] pytest를 unittest보다 선호
[mem2] 모든 함수에 타입 힌트 사용
```

```
results2 = semantic_store.search(
    ("user_42", "memories"), query="machine learning work", limit=2
```

```
)
for r in results2:
    print(f" [{r.key}] {r.value['content']}")
```

[mem4] ML 인프라 팀에서 근무
[mem2] 모든 함수에 타입 힌트 사용

기본 Store vs 시맨틱 Store 비교

기능	InMemoryStore()	InMemoryStore(index={...})
정확 키 조회	get(ns, key)	get(ns, key)
필터 검색	search(ns, filter={...})	search(ns, filter={...})
시맨틱 검색	불가	search(ns, query="...", limit=N)
프로덕션	InMemoryStore 대신 DB 백엔드 사용	PostgreSQL 기반 Store 권장

시맨틱 검색의 원리를 이해했으니, 이제 실제 에이전트의 도구에서 Store를 활용하는 방법을 살펴보겠습니다. 에이전트가 대화 중에 사용자 정보를 읽고 학습하는 핵심 메커니즘입니다.

4.7 도구에서 Store 읽기/쓰기 – ToolRuntime.store

에이전트의 도구 내에서 Store에 접근하여 사용자 정보를 읽고 쓸 수 있습니다. `create_agent(store=...)` 로 Store를 연결하면 `runtime.store` 로 자동 주입됩니다. 이를 통해 도구는 “이 사용자에게 대해 이전에 무엇을 배웠는가?”를 조회하고, “새로 알게 된 정보”를 저장할 수 있습니다.

읽기 패턴

도구에서 `runtime.store` 를 통해 저장된 사용자 정보를 조회합니다. `ToolRuntime[Context]` 타입힌트로 컨텍스트와 Store 모두에 접근할 수 있습니다.

쓰기 패턴

도구 파라미터로 사용자 입력을 받아 `store.put()` 으로 메모리를 저장합니다. 이를 통해 에이전트가 대화 중 학습한 정보를 영구 저장할 수 있습니다.

핵심 사항

- `runtime.store` : Store 인스턴스에 접근
- `runtime.context` : 정적 런타임 컨텍스트에 접근
- Store와 Context를 결합하면 “누구의(context) 어떤 정보(store)”를 체계적으로 관리할 수 있습니다

```
@tool
def get_user_info(runtime: ToolRuntime[Context]) -> str:
    """현재 사용자의 저장된 정보를 조회합니다."""
    store = runtime.store
    user_id = runtime.context.user_id
    info = store.get((user_id, "memories"), "profile")
```



```
# info.value는 dict-like 객체이므로 dict()로 명시적 캐스트
return str(dict(info.value)) if info else "사용자 정보 없음."
```

TIP

`store.get(...)` 이 반환하는 `Item.value` 는 외부 백엔드(PostgreSQL/Redis 등)에 따라 dict 서브클래스이거나 lazy 객체일 수 있습니다. JSON 직렬화나 키 순회 전에 `dict(info.value)` 로 명시적으로 캐스트하는 것이 안전합니다.

```
@tool
def save_preference(key: str, value: str, runtime: ToolRuntime[Context]) -> str:
    """사용자 선호도를 저장합니다."""
    store = runtime.store
    user_id = runtime.context.user_id
    # 권장 네임스페이스: (user_id, "memories") - 시맨틱 검색 활성 store와 호환
    store.put((user_id, "memories"), key, {"value": value})
    return f"선호도 저장됨: {key}={value}"
```

Store의 읽기/쓰기 메커니즘을 이해했으니, 이제 장기 메모리를 어떤 구조로 조직할 것인지에 대한 체계를 살펴 보겠습니다. 무작정 메모리를 저장하면 관리가 어려워지므로, 인지과학에서 영감을 받은 분류 체계를 적용합니다.

4.8 메모리 3유형: Semantic, Episodic, Procedural

장기 메모리는 인지과학에서 영감을 받은 세 가지 유형으로 분류됩니다. 인간의 기억이 “아는 것(Semantic)”, “경험한 것(Episodic)”, “할 줄 아는 것(Procedural)”으로 나뉘듯, 에이전트의 메모리도 동일하게 분류합니다. 각 유형에 따라 저장 구조와 활용 방식이 다릅니다.

TIP

세 유형의 메모리를 네임스페이스로 분리하면 관리가 용이해집니다. `(user_id, "profile")`, `(user_id, "episodes")`, `(user_id, "procedures")` 처럼 유형별 네임스페이스를 사용하세요. 검색할 때도 관련 유형의 네임스페이스만 조회하면 되므로 효율적입니다.

유형	설명	예시	구조
Semantic	엔티티에 대한 사실적 지식	사용자 선호도, 프로필 정보	Profile 또는 Collection
Episodic	과거 경험과 이벤트 기억	Few-shot 예시, 과거 액션 로그	Collection
Procedural	수행 방법에 대한 규칙/지침	시스템 프롬프트 수정, 가이드 라인	Profile (규칙 목록)

Semantic Memory – Profile vs Collection

Semantic 메모리는 저장 전략에 따라 두 가지 접근법이 있습니다:

접근법	구조	적합한 경우	예시
Profile	단일 JSON 문서, 지속 업데이트	소수의 잘 알려진 속성	<pre>{"name": "Alice", "language": "Python", "preferred_style": "concise"}</pre>
Collection	다수의 좁은 범위 문서, 높은 리콜	오픈엔드 또는 대규모 지식	<pre>[{"topic": "testing", "content": "Prefers pytest"}, ...]</pre>

Episodic Memory

과거에 유사한 상황에서 어떻게 행동했는지를 기록합니다. Few-shot 예시로 활용되어 에이전트가 과거 경험에서 학습할 수 있게 합니다.

Procedural Memory

에이전트의 행동 규칙을 저장합니다. 시스템 프롬프트를 동적으로 수정하는 효과를 가져, 에이전트가 사용자별 맞춤 지침을 따르도록 합니다.

```
mem_store = InMemoryStore(index={"embed": embeddings, "dims": 1536})
uid = "user_42"

# Semantic -- Profile (single JSON)
mem_store.put((uid, "profile"), "main", {
    "name": "Alice", "language": "Python",
    "preferred_style": "concise",
})
# Semantic -- Collection (multiple docs)
mem_store.put((uid, "facts"), "f1", {"content": "pytest 선호"})
```

```
# Episodic -- past experiences (few-shot)
mem_store.put((uid, "episodes"), "ep1", {
    "content": "SQL 최적화 -> EXPLAIN ANALYZE 사용",
})

# Procedural -- rules/guidelines
mem_store.put((uid, "procedures"), "rules", {
    "content": "항상 에러 처리를 포함. logging 사용.",
})
print("3가지 메모리 유형 모두 저장 완료.")
```

3가지 메모리 유형 모두 저장 완료.

```
# Episodic search: find similar past experiences
episodes = mem_store.search(
    (uid, "episodes"), query="database query help", limit=1
)
for ep in episodes:
    print(f"관련 에피소드: {ep.value['content']}")
```

관련 에피소드: SQL 최적화 -> EXPLAIN ANALYZE 사용

지금까지 컨텍스트의 저장과 검색을 다뤘습니다. 마지막으로 중요한 질문은 “저장된 컨텍스트를 언제, 얼마나 에이전트에게 노출할 것인가”입니다. Skills 패턴은 이 문제에 대한 우아한 해결책입니다.

지금까지 컨텍스트의 저장과 검색을 다뤘습니다. 마지막으로 중요한 질문은 “저장된 컨텍스트를 언제, 얼마나 에이전트에게 노출할 것인가”입니다. 모든 정보를 항상 프롬프트에 넣는 것은 비효율적이고 역효과를 낼 수 있습니다.

4.9 Progressive Disclosure – Skills 패턴

모든 컨텍스트를 프롬프트에 넣으면 토큰 비용이 증가하고 정확도가 떨어집니다. 이를 “컨텍스트 과부하 (context overload)”라고 합니다. 연구에 따르면 프롬프트가 길어질수록 LLM의 주의력이 분산되어, 중간에 위치한 정보를 놓치는 “Lost in the Middle” 현상이 발생합니다. Skills 패턴은 필요할 때만 관련 정보를 로드하는 Progressive Disclosure 방식으로, 이 문제를 우아하게 해결합니다.

Skill의 구조

Skill은 {name, description, content} 로 구성된 지식 단위입니다:

- name: 스킬 식별자 (예: "customers_schema")
- description: 짧은 설명 (시스템 프롬프트에 포함됨)
- content: 상세 내용 (`load_skill` 도구로 온디맨드 로드)

크기별 전략

크기	전략	예시
\<1K tokens	시스템 프롬프트에 직접 포함	테이블 이름, 고수준 관계
1–10K tokens	<code>load_skill</code> 도구로 온디맨드 로드	테이블 스키마, 쿼리 패턴, 베스트 프랙티스
\>10K tokens	페이지네이션으로 온디맨드 로드	대규모 참조 데이터, 과거 쿼리 로그

동작 흐름

1. 미들웨어가 모든 스킬의 이름과 설명을 시스템 프롬프트에 주입
2. 에이전트가 질문을 분석하고 필요한 스킬을 판단
3. `load_skill` 도구를 호출하여 상세 내용을 로드
4. 로드된 내용을 바탕으로 작업 수행

장점

장점	설명
토큰 효율성	현재 쿼리에 필요한 정보만 로드
확장성	수백 개의 테이블이 있는 DB도 지원
정확도	필요한 시점에 상세 스키마를 제공
비용 절감	요청당 입력 토큰 감소

```
skills = [
    {"name": "db_overview",
     "description": "모든 테이블의 고수준 개요",
     "content": "테이블: customers, orders, products"},
    {"name": "customers_schema",
     "description": "customers 테이블의 전체 스키마",
     "content": "CREATE TABLE customers (id INT PK, name VARCHAR)"},
]
SKILL_MAP = {s["name"]: s for s in skills}
print(f"스킬 {len(skills)}개 정의됨.")
```

스킬 2개 정의됨.

```
from langchain_core.tools import tool

@tool
def load_skill(skill_name: str) -> str:
    """데이터베이스 스킬에 대한 상세 정보를 로드합니다."""
    skill = SKILL_MAP.get(skill_name)
    if skill is None:
        return f"찾을 수 없음. 사용 가능: {' '.join(SKILL_MAP.keys())}"
    return f"## {skill['name']}\n\n{skill['content']}"
```

Skills 패턴으로 컨텍스트를 필요한 시점에 로드하는 방법을 배웠습니다. 마지막으로 남은 설계 결정은, 새로운 메모리를 언제 저장할 것인가입니다. 이 결정은 사용자 경험(응답 속도)에 직접적인 영향을 미칩니다.

· NOTE

실무에서는 먼저 `_이 정보가 다음 턴에도 필요할까?` 를 묻고, 그렇다면 Store로, 아니면 state로 둡니다.

`_지금 답변 전에 꼭 필요할까?` 까지 예라고 답하면 hot path, 아니면 background 작업으로 미루는 것이 가장 안전한 기본 전략입니다.

4.10 Hot Path vs Background 메모리 쓰기

메모리를 언제 쓰느냐에 따라 사용자 응답 지연에 영향을 미칩니다. 이는 분산 시스템에서의 일관성 (Consistency) vs 가용성(Availability) 트레이드오프와 유사합니다.

! WARNING

Hot path 쓰기는 응답 지연을 증가시키므로, 반드시 필요한 경우에만 사용하세요. “사용자가 방금 알려준 이름을 다음 응답에서 바로 사용해야 하는 경우”처럼, 즉시 리콜이 필수적인 상황으로 한정합니다.

방식	타이밍	즉시 사용 가능?	지연 영향
Hot path	대화 루프 내 실시간	즉시 (다음 턴에 사용 가능)	응답 지연 증가
Background	별도 비동기 태스크	지연됨 (Eventual Consistency)	지연 영향 없음

Hot Path 쓰기

에이전트 루프 내 인라인으로 메모리를 저장합니다. 바로 다음 턴에서 해당 메모리를 사용해야 할 때 적합합니다. 예: 사용자가 방금 알려준 선호도를 즉시 반영해야 하는 경우.

Background 쓰기

별도의 프로세스나 비동기 태스크로 메모리를 저장합니다. Eventual Consistency가 허용되는 경우에 사용하며, 응답 지연에 영향을 주지 않습니다. 예: 대화 패턴 분석, 장기 학습 데이터 축적.

선택 기준

- 즉시 리콜이 필요한가? -> Hot path
- 지연 감소가 우선인가? -> Background
- 대부분의 경우 Background 쓰기를 선호합니다

```
from langgraph.store.base import BaseStore

# Hot path: write inline (adds latency)
def reflect_node(state, store: BaseStore):
    """메모리를 인라인으로 추출하고 저장합니다."""
    last_msg = state["messages"][-1].content
    store.put(("user", "reflections"), "latest", {"content": last_msg})
    return state

print("Hot path: 즉시 저장, 다음 턴에 사용 가능.")
```

Hot path: 즉시 저장, 다음 턴에 사용 가능.

```
import asyncio

# Background: write in separate async task
async def background_memory_writer(state, store: BaseStore):
    """백그라운드에서 메모리를 저장합니다 (지연 없음)."""
    last_msg = state["messages"][-1].content
    await store.aput(
        ("user", "reflections"), "latest", {"content": last_msg}
    )
print("Background: 최종 일관성, 지연 없음.")
```

Background: 최종 일관성, 지연 없음.

요약

컨텍스트 엔지니어링 3요소

요소	구현	API
정적 런타임	context_schema + @dataclass	runtime.context.field
동적 런타임	state_schema + AgentState	state["field"]

장기 메모리	InMemoryStore + store=	store.put/get/search
--------	------------------------	----------------------

메모리 3유형

유형	용도	Namespace 예시
Semantic	사용자 프로필/사실	(user_id, "profile") , (user_id, "facts")
Episodic	과거 경험 (few-shot)	(user_id, "episodes")
Procedural	규칙/프롬프트 수정	(user_id, "procedures")

Best Practices

원칙	설명
정적 컨텍스트 최소화	현재 태스크에 필요한 것만 포함
Namespace 구조화	충돌 방지를 위해 계층적 namespace 사용
시맨틱 검색 우선	정확 매칭보다 임베딩 기반 검색이 확장성 우수
Background 쓰기 선호	즉시 리콜 불필요시 background로 지연 감소
Skills 패턴 적용	대규모 컨텍스트는 Progressive Disclosure

컨텍스트 엔지니어링은 에이전트의 지능의 기반입니다. 올바른 정보를 적시에 제공하는 것이 모델 선택보다 더 큰 영향을 미칩니다. 다음 장에서는 이 컨텍스트 엔지니어링을 실전에 적용하는 첫 번째 사례로, RAG 파이프라인을 심층적으로 다룹니다.

05 Agentic RAG

- LangGraph로 직접 구축

이전 장에서 학습한 컨텍스트 엔지니어링의 가장 대표적인 실전 응용이 RAG입니다. Retrieval-Augmented Generation(RAG)을 세 가지 방법으로 구현합니다: LangChain RAG Agent, LangChain RAG Chain, 그리고 LangGraph StateGraph 기반 커스텀 RAG. 문서 관련성 평가, 쿼리 리라이트, 조건부 라우팅 등 심화 패턴을 다룹니다.

학습 목표

LEARNING OBJECTIVES

- RAG 파이프라인(인덱싱 -> 검색 -> 생성)의 전체 구조를 이해한다
- `RecursiveCharacterTextSplitter` 로 문서를 청킹한다
- `InMemoryVectorStore` 로 벡터 스토어를 구축한다
- LangChain `create_agent` + `@tool` 로 RAG Agent를 구현한다
- `@dynamic_prompt` 미들웨어로 RAG Chain(단일 LLM 호출)을 구현한다
- LangGraph `StateGraph` 로 커스텀 RAG 에이전트를 구축한다
- `GradeDocuments` 구조화 출력으로 문서 관련성을 평가한다
- 쿼리 리라이트와 조건부 라우팅을 구현한다

핵심 질문

이 장의 중심 질문은 “검색을 한 번 하고 바로 답할 것인가, 에이전트에게 검색을 맡길 것인가, 아니면 검색 이후의 판단 흐름까지 그래프로 직접 통제할 것인가?” 입니다.

한눈에 보는 핵심

- RAG는 오프라인 인덱싱과 온라인 질의 처리를 분리해서 보면 구조가 훨씬 단순해집니다.
- 단순 Q&A는 Chain, 유연한 검색은 Agent, 평가·재작성·재시도는 Graph가 잘 맞습니다.
- 검색 품질 문제는 보통 모델보다 청킹, 검색, 관련성 판단 설계에서 먼저 발생합니다.

5.1 환경 설정

RAG 파이프라인을 구축하기 위해 LLM과 임베딩 모델을 초기화합니다. `ChatOpenAI` 는 텍스트 생성을, `OpenAIEmbeddings` 는 문서를 벡터로 변환하는 역할을 담당합니다. 두 모델은 RAG의 서로 다른 단계에서 사용되므로 모두 필요합니다.

```
from dotenv import load_dotenv
load_dotenv(override=True)

from langchain_openai import ChatOpenAI, OpenAIEmbeddings

llm = ChatOpenAI(model="gpt-5.4")
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
print("환경 준비 완료.")
```

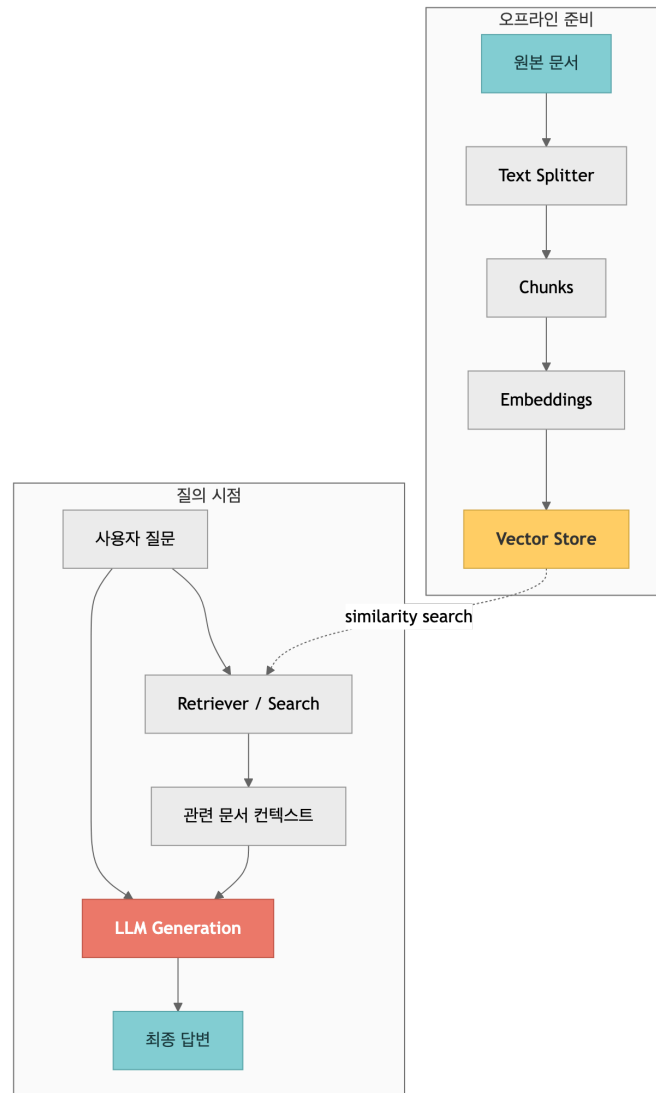
환경 준비 완료.

5.2 RAG 개요

RAG(Retrieval-Augmented Generation)는 외부 지식을 검색하여 LLM 응답의 정확도를 높이는 패턴입니다. LLM은 두 가지 핵심 제약을 가집니다:

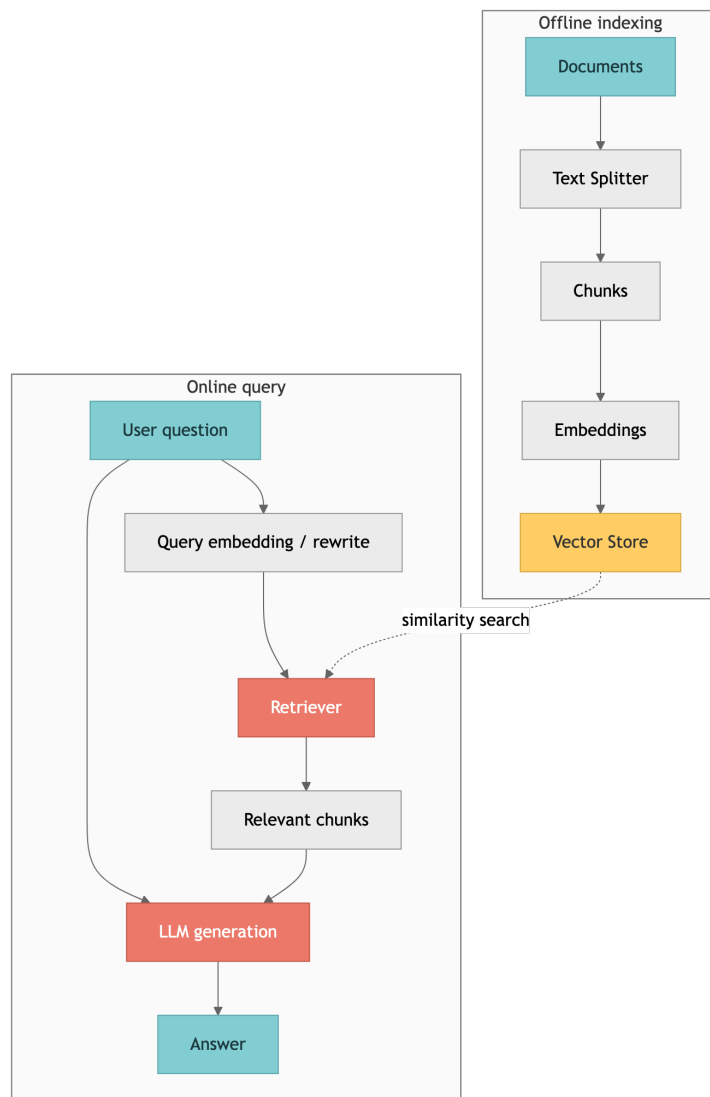
- 유한한 컨텍스트: 전체 코퍼스를 한 번에 처리할 수 없음
- 정적 지식: 학습 데이터가 시간이 지나면 구식이 됨

RAG는 쿼리 시점에 관련 외부 정보를 가져와 이 제약을 극복합니다.



RAG를 처음 볼 때는 오프라인 인덱싱 과 온라인 질의 처리 를 분리해서 보는 것이 가장 중요합니다. 문서를 청킹하고 임베딩하는 단계는 미리 준비해 두는 백오피스 작업이고, 사용자가 질문했을 때는 이미 준비된 벡터 스토어에서 관련 문서를 찾은 뒤 그 결과만 LLM에 넣어 답을 생성합니다.

파이프라인: 인덱싱 -> 검색 -> 생성



파이프라인을 두 개의 시간 축으로 나눠 보면 이해가 쉬워집니다. 인덱싱 단계는 미리 준비하는 오프라인 작업이고, 질의 단계는 사용자의 질문이 들어올 때마다 반복되는 온라인 작업입니다. 이 구분을 기억하면 청킹/임베딩 최적화와 검색/생성 최적화를 분리해서 설계할 수 있습니다.

5가지 핵심 구성 요소

구성 요소	역할
Document Loaders	외부 소스(Google Drive, Notion 등)에서 데이터를 표준 Document 객체로 수집
Text Splitters	대규모 문서를 컨텍스트 윈도우에 맞는 청크로 분할
Embedding Models	텍스트를 의미적으로 유사한 내용이 가까이 모이는 벡터로 변환
Vector Stores	임베딩을 저장하고 유사도 검색을 수행하는 전문 데이터베이스

Retrievers

비정형 쿼리를 기반으로 관련 문서를 반환

세 가지 RAG 아키텍처

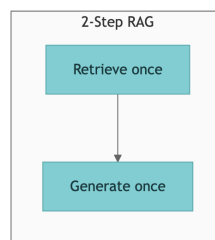
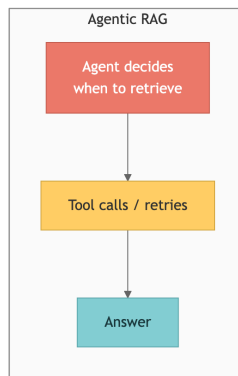
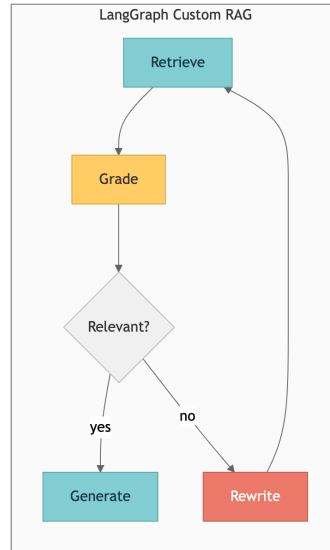


그림 읽는 법

아래에서 위로 비교하지 말고, 각 박스를 독립 패턴으로 보면 됩니다. **2-Step RAG**는 가장 단순하고, **Agentic RAG**는 검색 시점과 횟수를 에이전트가 결정하며, **LangGraph Custom RAG**는 검색 이후의 평가와 재시도 규칙까지 명시합니다.

접근법	아키텍처	LLM 호출	유연성	적합한 경우
2-Step RAG	검색 후 즉시 생성	단일	낮음	FAQ, 문서 봇 (빠르고 예측 가능)
Agentic RAG	에이전트가 검색 시점/방법 결정	다중	높음	복잡한 리서치, 다중 도구 접근
Hybrid RAG	쿼리 강화 + 검색 검증 + 답변 품질 체크	다중	높음	반복적 정제가 필요한 경우

Agent vs Chain 접근 (LangChain 구현)

접근법	아키텍처	LLM 호출	적합한 경우
RAG Agent	에이전트 + retriever 도구	다중	복잡한 쿼리, 쿼리 재구성 필요
RAG Chain	미들웨어 주입 컨텍스트	단일	단순 Q&A, 예측 가능한 비용
LangGraph 커스텀	StateGraph + 커스텀 노드	다중	관련성 평가, 리라이트 등 세밀한 제어

· NOTE

선택 기준 요약

- Agent 는 검색 횟수나 도구 사용을 스스로 결정해야 할 때 적합합니다.
- Chain 은 비용과 지연 시간을 예측 가능하게 유지해야 할 때 가장 단순합니다.
- Graph 는 관련성 평가, 재작성, 재시도 규칙을 명시적으로 통제해야 할 때 선택합니다.

· NOTE

언제 쓰지 않을까?

- **RAG Chain**으로 충분한 질문을 Agent로 만들면 비용과 동작 편차가 불필요하게 늘어납니다.
- **RAG Agent**는 유연하지만, 검색 횟수와 도구 선택을 통제해야 하는 요구가 강하면 오히려 Graph가 더 안전합니다.
- **LangGraph 커스텀 RAG**는 가장 강력하지만, 단순 문서 Q&A에 적용하면 구현/운영 복잡도가 빠르게 커 집니다.

5.3 문서 로딩 & 청킹

문서 로더 (Document Loaders)

문서 로더는 다양한 소스에서 원시 콘텐츠를 읽어 `page_content` 와 `metadata` 필드를 가진 `Document` 객체로 반환합니다.

로더	소스	패키지
PyPDFLoader	PDF 파일	pypdf
TextLoader	텍스트 파일	내장
CSVLoader	CSV 파일	내장
WebBaseLoader	웹 페이지	beautifulsoup4
DirectoryLoader	디렉토리 내 파일들	내장

텍스트 분할 (Text Splitting)

`RecursiveCharacterTextSplitter` 는 `\n\n` -> `\n` -> -> `" "` 순으로 재귀적으로 분할하여 의미적 연관성을 유지합니다. 가장 범용적인 분할기로 권장됩니다. 분할 순서가 중요한 이유는, 가능한 한 단락 단위로 분할하여 의미가 끊기지 않는 청크를 만들기 위해서입니다. 단락 경계(`\n\n`)에서 분할할 수 없을 만큼 긴 텍스트만 문장 경계(`\n`)로 넘어가고, 그래도 안 되면 공백 경계로 내려갑니다.

TIP

`chunk_size` 와 `chunk_overlap` 의 최적값은 데이터와 사용 사례에 따라 달라집니다. 일반적으로 FAQ 봇에는 작은 청크(500자)가 정밀한 검색에 유리하고, 보고서 생성에는 큰 청크(1500 2000자)가 맥락 보존에 효과적입니다. 시작 시 `chunk_size=1000`, `chunk_overlap=200`을 권장하며, 검색 품질 평가를 통해 조정하세요.

파라미터	설명	권장값
<code>chunk_size</code>	청크 최대 문자 수	500-2000 (작으면 정밀 검색, 크면 맥락 보존)
<code>chunk_overlap</code>	인접 청크 공유 문자 수	<code>chunk_size</code> 의 10-20% (경계 정보 손실 방지)

기타 분할기

분할기	적합한 경우
<code>MarkdownHeaderTextSplitter</code>	마크다운 문서
<code>HTMLHeaderTextSplitter</code>	HTML 문서
<code>TokenTextSplitter</code>	토큰 예산 기반 분할
<code>CodeTextSplitter</code>	소스 코드 (언어 인식)

```
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_core.documents import Document

raw_docs = [
    Document(page_content="LangGraph는 LLM으로 상태 기반 멀티 액터 "
        "애플리케이션을 구축하기 위한 프레임워크입니다.",
        metadata={"source": "langgraph-docs"}),
    Document(page_content="에이전트는 도구를 사용하여 외부 시스템과 "
```

```

"상호작용합니다. ReAct 패턴은 추론과 행동을 번갈아 수행합니다.",
metadata={"source": "agent-guide"}),
]
print(f"문서 {len(raw_docs)}개 로드됨.")

```

문서 2개 로드됨.

```

text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000, chunk_overlap=200,
)
splits = text_splitter.split_documents(raw_docs)

for i, doc in enumerate(splits):
    print(f"청크 {i}: {doc.page_content[:60]}...")
print(f"총 청크 수: {len(splits)}")

```

청크 0: LangGraph는 LLM으로 상태 기반 멀티 액터 애플리케이션을 구축하기 위한 프레임워크입니다....
 청크 1: 에이전트는 도구를 사용하여 외부 시스템과 상호작용합니다. ReAct 패턴은 추론과 행동을 번갈아 수행합니다....
 총 청크 수: 2

문서가 청크로 분할되었으니, 다음 단계는 이 청크들을 벡터로 변환하여 검색 가능한 형태로 저장하는 것입니다. 이 과정이 인덱싱이며, RAG 파이프라인의 오프라인 준비 단계에 해당합니다.

5.4 벡터 스토어 구축

벡터 스토어는 임베딩을 인덱싱하고 유사도 검색을 수행하는 전문 데이터베이스입니다. `InMemoryVectorStore` 는 개발/테스트용으로 적합합니다. 벡터 스토어의 핵심 원리는 의미적 유사도 검색입니다. 전통적인 키워드 검색이 정확한 단어 매칭에 의존하는 반면, 벡터 검색은 임베딩 공간에서의 거리(코사인 유사도, 유클리드 거리 등)를 기준으로 의미적으로 가까운 문서를 찾습니다. 예를 들어 “LLM 애플리케이션 구축”이라는 쿼리는 “AI 시스템 개발”이라는 문서와도 높은 유사도를 보일 수 있습니다.

! WARNING

`InMemoryVectorStore` 는 프로세스 종료 시 모든 데이터가 소멸합니다. 프로덕션에서는 반드시 영속적 벡터 스토어(Chroma, FAISS, Pinecone 등)를 사용하세요. 또한 대규모 데이터셋(수만 건 이상)에서는 인프로세스 벡터 스토어의 메모리 사용량이 급격히 증가하므로, 클라이언트-서버 아키텍처의 벡터 스토어를 고려해야 합니다.

주요 벡터 스토어 비교

벡터 스토어	유형	적합한 경우
<code>InMemoryVectorStore</code>	인프로세스	개발, 소규모 데이터셋
<code>Chroma</code>	임베디드/클라이언트-서버	프로토타이핑, 중규모 데이터셋
<code>FAISS</code>	인프로세스	고성능 로컬 검색
<code>Pinecone</code>	매니지드 클라우드	프로덕션, 확장성

PGVector	PostgreSQL 확장	기존 PostgreSQL 인프라 활용
----------	---------------	----------------------

검색 유형

검색 타입	설명
"similarity"	표준 최근접 이웃 검색
"mmr"	Maximal Marginal Relevance – 관련성과 다양성의 균형 (중복 감소)
"similarity_score_threshold"	최소 유사도 점수 이상인 문서만 반환

```
from langchain_core.vectorstores import InMemoryVectorStore

vector_store = InMemoryVectorStore.from_documents(
    documents=splits, embedding=embeddings,
)
test_results = vector_store.similarity_search("LangGraph", k=2)
for doc in test_results:
    print(f" [{doc.metadata['source']}] {doc.page_content[:80]}")
print(f"벡터 스토어 준비 완료. 문서 {len(splits)}개.")
```

[langgraph-docs] LangGraph는 LLM으로 상태 기반 멀티 액터 애플리케이션을 구축하기 위한 프레임워크입니다.
[agent-guide] 에이전트는 도구를 사용하여 외부 시스템과 상호작용합니다. ReAct 패턴은 추론과 행동을 번갈아 수행합니다.
벡터 스토어 준비 완료. 문서 2개.

벡터 스토어가 구축되었으니, 에이전트가 이를 활용하여 문서를 검색할 수 있도록 도구(tool) 형태로 래핑해야 합니다. 에이전트는 도구를 통해서만 외부 시스템과 상호작용하므로, 벡터 스토어를 도구로 정의하는 것이 RAG 통합의 핵심입니다.

5.5 검색 도구 정의

`response_format="content_and_artifact"` 를 사용하면 도구 출력을 두 부분으로 분리합니다:

- Content: 모델에 전달되는 문자열 표현 (추론에 사용)
- Artifact: 원본 Document 객체 (프로그래밍 방식으로 접근 가능하지만 모델에 전송되지 않음)

이 분리를 통해 모델에는 읽기 쉬운 텍스트를, 후속 처리에는 메타데이터가 포함된 원본 객체를 사용할 수 있습니다.

```
from langchain_core.tools import tool

@tool(response_format="content_and_artifact")
def retrieve(query: str):
    """지식 베이스에서 관련 문서를 검색합니다."""
    docs = vector_store.similarity_search(query, k=4)
    serialized = "\n\n".join(
        f"출처: {d.metadata.get('source', '?')}\n{d.page_content}"
        for d in docs
    )
    return serialized, docs
```

5.6 LangChain RAG Agent – `create_agent` + `@tool`

가장 간단한 방법: retriever를 도구로 등록하고 에이전트가 필요할 때 호출합니다.

다중 단계 검색 흐름

RAG Agent는 자동으로 다중 검색 단계를 실행할 수 있습니다:

1. 초기 검색 – 사용자 질문 기반 쿼리 생성
2. 결과 평가 – 검색된 문서가 질문에 충분한지 판단
3. 재구성 및 재검색 – 결과가 부족하면 쿼리를 수정하여 재검색
4. 통합 – 모든 검색 결과를 결합하여 최종 답변 생성

이 접근법은 복잡한 리서치 질문에 적합하지만, 여러 번의 LLM 호출로 비용과 지연이 증가합니다.

5.7 LangChain RAG Chain – `@dynamic_prompt` 미들웨어

단일 LLM 호출로 RAG를 구현합니다. `@dynamic_prompt` 가 LLM 호출 전에 문서를 검색하고 시스템 프롬프트에 자동으로 주입합니다. 미들웨어 방식이므로 에이전트 루프 없이 단일 패스로 동작합니다.

특성	RAG Agent	RAG Chain
LLM 호출 수	다중 (에이전트 결정)	단일
검색 횟수	1회 이상 (에이전트 제어)	정확히 1회 (미들웨어 제어)
쿼리 재구성	자동	미지원
지연	높음 (여러 왕복)	낮음 (단일 패스)
비용	높음 (더 많은 토큰)	낮음 (더 적은 토큰)
투명성	에이전트 추론이 메시지에 노출	컨텍스트 주입이 암묵적

고급 활용: `@dynamic_prompt` 로 기본 컨텍스트를 주입하면서 동시에 retriever 도구를 제공하여 두 접근법을 결합할 수도 있습니다.

```
from langchain.agents.middleware import dynamic_prompt

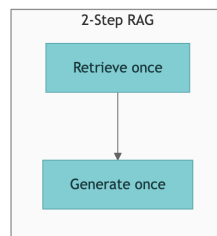
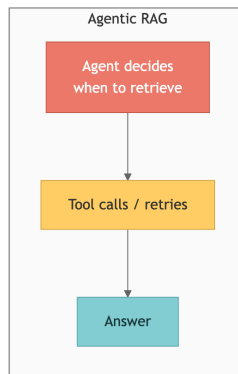
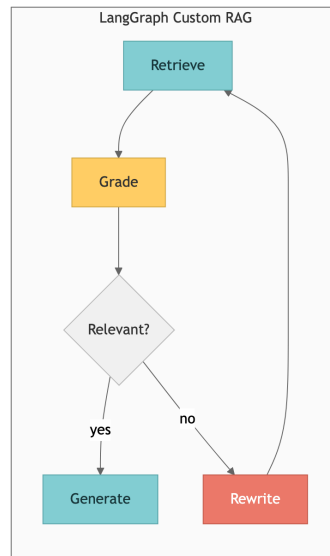
@dynamic_prompt
def rag_prompt(request):
    """문서를 검색하여 시스템 프롬프트에 주입합니다."""
    user_msg = request.state["messages"][-1].content
    docs = vector_store.similarity_search(user_msg, k=4)
    ctx = "\n\n".join(d.page_content for d in docs)
    return f"컨텍스트를 기반으로 답변하세요:\n\n{ctx}"
```

RAG Agent와 RAG Chain은 각각 유연성과 단순성에서 장점이 있습니다. 그러나 검색 결과의 품질을 자동으로 평가하고, 부적합한 경우 쿼리를 리라이트하여 재검색하는 적응형 RAG를 구현하려면 LangGraph `StateGraph`가 필요합니다. 이것이 Agentic RAG의 핵심입니다: retrieve → grade → generate → hallucination check의 순환 루프를 통해 답변 품질을 보장합니다.

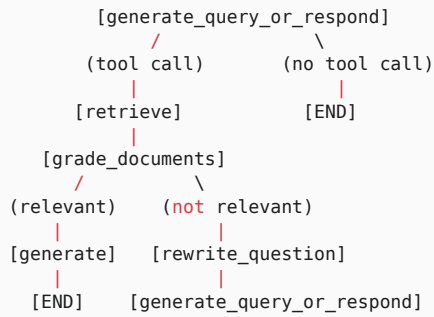
5.8 LangGraph 커스텀 RAG – StateGraph 구축

LangGraph `StateGraph` 로 세밀한 제어가 가능한 RAG 에이전트를 직접 구축합니다. 이 방식의 핵심 장점은 조건부 라우팅을 통해 검색 결과의 관련성을 평가하고, 관련 없는 경우 쿼리를 리라이트하여 재검색하는 등의 세밀한 흐름 제어가 가능하다는 것입니다.

아키텍처



이 장의 LangGraph 구현은 위 세 번째 패턴에 해당합니다. 즉, 검색 자체보다도 검색 이후의 판단 – 관련성 평가, 재작성, 종료 조건 – 을 그래프로 명시하는 데 의미가 있습니다.



각 노드의 역할

노드	역할
generate_query_or_respond	진입 노드. 검색할지 직접 응답할지 결정
retrieve	<code>ToolNode</code> 로 검색 실행
grade_documents	구조화 출력(<code>GradeDocuments</code>)으로 문서 관련성 평가
rewrite_question	관련 없는 결과 시 더 구체적인 쿼리로 리라이트
generate_answer	관련 문서 기반 최종 답변 생성

! WARNING

`rewrite_question` → `generate_query_or_respond` 순환이 발생할 수 있습니다. `retry_count` 를 State에 추가하여 최대 재시도 횟수를 제한하는 것이 권장됩니다. 기본적으로 2 3회 재시도 후 가용한 정보로 답변하는 전략이 효과적입니다.

```

from langgraph.graph import MessagesState

class AgentState(MessagesState):
    """커스텀 RAG 에이전트 상태."""
    relevance: str # "relevant" or "not_relevant"

print(f"AgentState 키: {list(AgentState.__annotations__)}")

```

```
AgentState 키: ['messages', 'relevance']
```

5.9 generate_query_or_respond 노드

진입 노드입니다. LLM이 `retrieve` 도구를 호출할지, 직접 응답할지 결정합니다.

5.10 `grade_documents` 노드 – 구조화 출력으로 관련성 평가

Agentic RAG에서 가장 핵심적인 노드입니다. 검색된 문서가 사용자 질문과 실제로 관련이 있는지를 LLM이 평가합니다. 이 평가가 없으면 에이전트는 관련 없는 문서를 기반으로 환각(hallucination)을 생성할 위험이 있습니다. `GradeDocuments` 스키마로 LLM이 문서 관련성을 평가하며, `with_structured_output` 으로 구조화된 응답을 받아 프로그래밍적으로 후속 처리를 결정합니다.

TIP

`GradeDocuments` 에 `reasoning` 필드를 포함하면 LLM이 평가 이유를 명시하게 됩니다. 이는 디버깅에 유용할 뿐 아니라, Chain-of-Thought 효과로 평가 정확도 자체도 향상시킵니다. 프로덕션에서는 이 이유를 로깅하여 검색 품질 분석에 활용할 수 있습니다.

```
from pydantic import BaseModel, Field
from typing import Literal

class GradeDocuments(BaseModel):
    """검색된 문서의 이진 관련성 점수."""
    relevance: Literal["relevant", "not_relevant"] = Field(
        description="문서가 관련이 있는지 여부."
    )
    reasoning: str = Field(description="간략한 설명.")

grader = llm.with_structured_output(GradeDocuments)
```

```
def grade_documents(state: AgentState):
    """
    검색된 문서의 관련성을 평가합니다.
    """

    msgs = state["messages"]

    user_q = next(
        (m.content for m in msgs if m.type == "human"),
        ""
    )

    tool_content = msgs[-1].content

    grade = grader.invoke(
        f"질문: {user_q}\n문서:\n{tool_content}\n"
        f"이 문서들이 관련이 있습니까?"
    )

    return {
        "relevance": grade.relevance,
        "messages": msgs
    }
```

문서 관련성 평가에서 “not_relevant”이 반환되면, 단순히 실패로 처리하지 않고 쿼리를 개선하여 재검색하는 전략을 취합니다. 이것이 Agentic RAG가 단순 RAG와 차별화되는 핵심 메커니즘입니다.

5.11 `rewrite_question` 노드

검색된 문서가 관련 없을 때, 원래 질문을 더 구체적으로 리라이트하여 검색 품질을 향상시킵니다. 리라이트 전략은 다양합니다: 모호한 용어를 구체화하거나, 질문의 범위를 좁히거나, 동의어를 사용하여 다른 각도에서 검색할 수 있습니다. LLM이 원래 질문의 의도를 파악하여 벡터 검색에 더 적합한 형태로 변환합니다.

5.12 `generate_answer` 노드

관련 문서가 확인되면, 검색 결과와 원본 질문을 결합하여 최종 답변을 생성합니다.

개별 노드가 모두 정의되었으니, 이제 이들을 하나의 그래프로 조립하는 마지막 단계입니다. LangGraph의 핵심 강점은 이러한 노드들을 선언적으로 연결하여 복잡한 흐름을 명확하게 표현할 수 있다는 것입니다.

5.13 그래프 조립 & 실행

모든 노드를 `StateGraph`에 등록하고, 조건부 엣지(`tools_condition`, `relevance_router`)로 연결합니다. 그래프 조립 과정은 세 단계로 이루어집니다: (1) 노드 등록 - 각 함수를 이름과 함께 그래프에 추가, (2) 엣지 연결 - 노드 간의 고정 경로 설정, (3) 조건부 엣지 - 상태에 따른 동적 라우팅 설정. `compile()` 호출 후에는 그래프가 실행 가능한 상태가 됩니다.

```
from langgraph.graph import StateGraph, START, END
from langgraph.prebuilt import ToolNode, tools_condition

def relevance_router(state: AgentState):
    if state.get("relevance") == "relevant":
        return "generate_answer"
    return "rewrite_question"

graph = StateGraph(AgentState)
graph.add_node("gen_query", generate_query_or_respond)
```

```
<langgraph.graph.state.StateGraph at 0x29f10457080>
```

```
graph.add_node("retrieve", ToolNode([retrieve]))
graph.add_node("grade_documents", grade_documents)
graph.add_node("rewrite_question", rewrite_question)
graph.add_node("generate_answer", generate_answer)

graph.add_edge(START, "gen_query")
graph.add_conditional_edges(
    "gen_query", tools_condition,
    {"tools": "retrieve", "__end__": END},
)
```

```
<langgraph.graph.state.StateGraph at 0x29f10457080>
```

```
graph.add_edge("retrieve", "grade_documents")
graph.add_conditional_edges(
    "grade_documents", relevance_router,
    {"generate_answer": "generate_answer",
```

```

        "rewrite_question": "rewrite_question"},
    )
    graph.add_edge("rewrite_question", "gen_query")
    graph.add_edge("generate_answer", END)

    app = graph.compile()
    print("그래프 컴파일 성공.")

```

그래프 컴파일 성공.

요약

세 가지 RAG 접근법 비교

특성	RAG Agent	RAG Chain	LangGraph 커스텀
LLM 호출 수	다중	단일	다중
검색 횟수	에이전트 결정	정확히 1회	커스텀
쿼리 재구성	자동	미지원	명시적 노드
관련성 평가	암묵적	없음	GradeDocuments
제어 수준	낮음	낮음	높음
구현 복잡도	낮음	최저	높음

핵심 LangGraph 패턴

패턴	구현
조건부 라우팅	<code>add_conditional_edges</code> + <code>tools_condition</code>
구조화 출력	<code>llm.with_structured_output(GradeDocuments)</code>
도구 노드	<code>ToolNode([retrieve])</code>
루프 제어	<code>rewrite_question</code> -> <code>gen_query</code> 순환

Agentic RAG는 비정형 문서에서 정보를 검색하는 패턴입니다. 그러나 기업 데이터의 상당 부분은 정형 데이터베이스에 저장되어 있습니다. 다음 장에서는 자연어를 SQL로 변환하여 데이터베이스에 질의하는 SQL 에이전트를 구축합니다.

06 SQL 에이전트 심화

- LangChain & LangGraph

이전 장의 Agentic RAG가 비정형 문서를 다뤘다면, SQL 에이전트는 정형 데이터베이스에 자연어로 질의하는 패턴입니다. 자연어를 SQL 쿼리로 변환하는 에이전트를 두 가지 방법으로 구축합니다: LangChain

`create_agent` + `SQLDatabaseToolkit` (간단 버전)과 LangGraph `StateGraph` (커스텀 버전). Human-in-the-Loop, `interrupt()`, `Command(resume=...)` 패턴을 다룹니다.

학습 목표

LEARNING OBJECTIVES

- SQL 에이전트의 8단계 워크플로우를 이해한다
- `SQLDatabase` 와 `SQLDatabaseToolkit` 의 4개 도구를 활용한다
- LangChain `create_agent` 로 ReAct 기반 SQL Agent를 구현한다
- `HumanInTheLoopMiddleware` 로 쿼리 실행 전 승인을 추가한다
- LangGraph `StateGraph` 로 커스텀 SQL Agent를 구축한다
- `bind_tools` 와 `tool_choice` 로 강제 도구 호출을 설정한다
- `interrupt()` 와 `Command(resume=...)` 로 쿼리 리뷰를 구현한다

핵심 질문

SQL 에이전트의 본질은 “자연어를 SQL로 바꾸는 것” 자체보다, “그 쿼리를 어떤 순서로 검증하고 누가 실행을 승인할 것인가?” 에 있습니다.

한눈에 보는 핵심

- 실무 SQL 에이전트는 생성보다 검증과 실행 통제가 더 중요합니다.
- 읽기 전용 권한, 접근 테이블 제한, HITL은 시스템 프롬프트보다 더 근본적인 안전장치입니다.
- 순서를 강제하고 싶으면 LangChain 기본 루프보다 LangGraph 커스텀 그래프가 더 적합합니다.

6.1 환경 설정 (SQLite + Chinook DB)

SQL 에이전트를 구축하기 위해 LLM과 데이터베이스를 연결합니다. Chinook DB는 디지털 음악 스토어의 샘플 데이터베이스로, Artist, Album, Track, Invoice 등 11개 테이블을 포함합니다. `SQLDatabase` 래퍼는 SQLAlchemy를 기반으로 데이터베이스 메타데이터(테이블 목록, 스키마, 샘플 데이터)에 프로그래밍 방식으로 접근하는 인터페이스를 제공합니다.

```
# %pip install langchain langchain-openai langchain-community langgraph sqlalchemy python-dotenv

from dotenv import load_dotenv
load_dotenv(override=True)

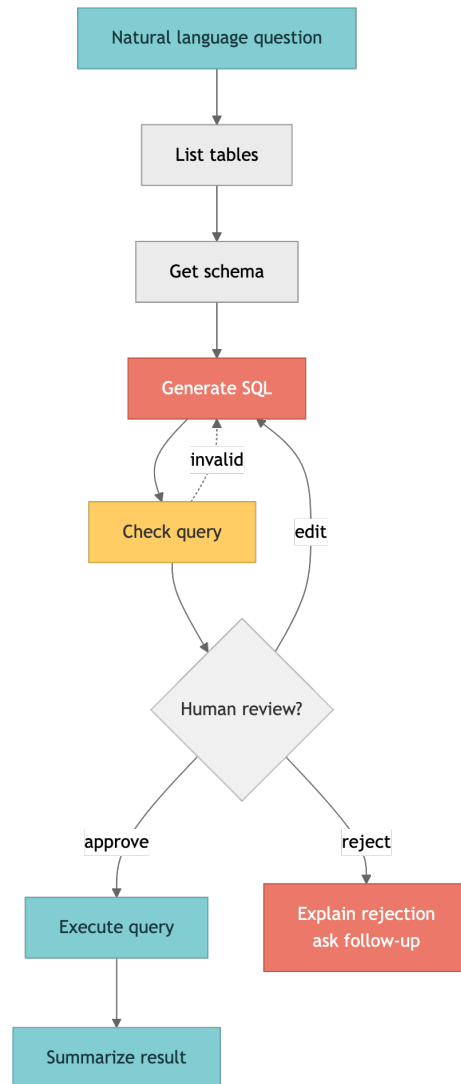
from langchain_openai import ChatOpenAI
from langchain_community.utilities import SQLDatabase

llm = ChatOpenAI(model="gpt-5.4")
db = SQLDatabase.from_uri("sqlite:///Chinook.db")
print(f"Dialect: {db.dialect}")
```

```
Dialect: sqlite
```

6.2 SQL 에이전트 개요

SQL 에이전트는 자연어 질문을 SQL 쿼리로 변환하는 8단계 프로세스를 따릅니다:



실무에서는 이 흐름을 두 갈래로 기억하면 됩니다. 정상 경로는 생성 → 검증 → 승인 → 실행으로 이어지고, 보호 경로는 검증 실패나 사람 거절 시 즉시 다시 작성하거나 사용자에게 추가 정보를 요구합니다.

왜 에이전트가 필요한가?

단순 text-to-SQL과 달리 에이전트 방식은 스키마 탐색 → 쿼리 생성 → 검증 → 실행의 반복 루프를 수행합니다. 잘못된 쿼리가 생성되면 에이전트가 오류를 분석하고 쿼리를 재작성할 수 있어 정확도가 크게 향상됩니다. 또한 에이전트는 필요한 테이블의 스키마만 선택적으로 로드하므로 컨텍스트 윈도우를 효율적으로 사용합니다.

에이전트 실행 트레이스 예시

User: "지난달 매출 상위 5개 제품은?"

Agent -> `sql_db_list_tables()`
 <- "customers, orders, order_items, products, categories"

Agent -> `sql_db_schema("orders, order_items, products")`
 <- `CREATE TABLE orders (id INT, order_date DATE, ...)`
`CREATE TABLE order_items (order_id INT, product_id INT, quantity INT, price DECIMAL, ...)`


```

Agent -> sql_db_query_checker("SELECT p.name, SUM(oi.quantity * oi.price) ...")
      <- "The query looks correct."

Agent -> sql_db_query(validated_query)
      <- [("Widget Pro", 45230.00), ("Gadget X", 38100.00), ...]

```

안전 수칙

우려사항	대응
SQL Injection	파라미터화된 쿼리 사용, Toolkit이 자동 처리
DML 실행	시스템 프롬프트에서 INSERT/UPDATE/DELETE 금지, DB 레벨 읽기 전용 권한 설정
비용 높은 쿼리	LIMIT 강제, Human-in-the-Loop으로 실행 전 승인
민감 데이터	<code>include_tables</code> / <code>exclude_tables</code> 로 접근 가능 테이블 제한, 컬럼 레벨 권한 설정
데이터 노출	데이터베이스 뷰(view) 또는 제한된 사용자 권한 활용

! WARNING

SQL 에이전트는 데이터베이스에 직접 쿼리를 실행하므로, 반드시 읽기 전용 계정을 사용하고 `include_tables` 로 접근 범위를 제한해야 합니다. DML(INSERT/UPDATE/DELETE) 실행을 시스템 프롬프트로만 방지하는 것은 불충분합니다 – DB 레벨 권한 설정이 근본적인 안전장치입니다.

접근 가능 테이블 제한

프로덕션에서는 에이전트가 접근할 수 있는 테이블을 명시적으로 제한하는 것이 좋습니다:

```

db = SQLiteDatabase.from_uri(
    "sqlite:///company.db",
    include_tables=["products", "orders", "order_items"], # 허용 목록
    # exclude_tables=["users", "credentials"],           # 또는 차단 목록
)

```

SQL 에이전트의 전체 구조와 안전 수칙을 이해했으니, 이제 실제 구현의 핵심인 도구(tool) 레이어를 살펴봅시다. `SQLiteDatabaseToolkit` 은 데이터베이스 연결 하나로 SQL 에이전트에 필요한 모든 도구를 자동으로 생성해 주는 유틸리티입니다.

6.3 SQLiteDatabaseToolkit

`SQLiteDatabaseToolkit` 은 `SQLiteDatabase` 인스턴스와 LLM을 받아 4개의 도구를 자동 생성합니다. 이 도구들은 SQL 에이전트 워크플로의 각 단계에 1:1로 매핑됩니다:



TIP

`sql_db_query_checker` 는 내부적으로 LLM을 사용하여 쿼리를 검사합니다. 따라서 `SQLDatabaseToolkit(db=db, llm=llm)` 에서 LLM을 전달하는 것이 필수입니다. 쿼리 검증에 사용되는 LLM은 에이전트의 메인 LLM과 동일하거나, 비용 절감을 위해 더 가벼운 모델을 지정할 수 있습니다.

도구	기능
<code>sql_db_list_tables</code>	데이터베이스의 모든 테이블 이름 반환
<code>sql_db_schema</code>	CREATE TABLE 문 + 샘플 행 반환
<code>sql_db_query</code>	SQL 쿼리를 실행하고 결과 반환
<code>sql_db_query_checker</code>	LLM이 쿼리의 오류를 사전 검사

```
from langchain_community.agent_toolkits import SQLDatabaseToolkit

toolkit = SQLDatabaseToolkit(db=db, llm=llm)
tools = toolkit.get_tools()

for t in tools:
    print(f" {t.name}: {t.description[:60]}...")
print(f"총 도구 수: {len(tools)}")
```

```
sql_db_query: Input to this tool is a detailed and correct SQL query, outp...
sql_db_schema: Input to this tool is a comma-separated list of tables, outp...
sql_db_list_tables: Input is an empty string, output is a comma-separated list o...
sql_db_query_checker: Use this tool to double check if your query is correct befor...
총 도구 수: 4
```

6.4 LangChain SQL Agent – `create_agent` + ReAct

`create_agent` 는 LangChain의 고수준 API로, 모델과 도구를 받아 ReAct(Reasoning + Acting) 루프를 자동으로 구성합니다. 에이전트는 시스템 프롬프트에 정의된 워크플로우를 따라 도구를 순서대로 호출합니다.

ReAct 루프 동작 원리

1. LLM이 사용자 질문과 대화 이력을 분석하여 다음에 호출할 도구를 결정합니다
2. 도구가 실행되고 결과가 대화 이력에 추가됩니다
3. LLM이 결과를 확인하고, 추가 도구 호출이 필요하면 1단계로 돌아갑니다
4. 최종 답변이 준비되면 텍스트 응답을 반환합니다

시스템 프롬프트의 역할

시스템 프롬프트는 에이전트의 행동 지침을 정의합니다. SQL 에이전트에서는 특히 다음을 명시해야 합니다:

- 도구 호출 순서: `list_tables` → `schema` → `query_checker` → `query` 순서 강제
- 안전 규칙: `LIMIT` 사용, DML 금지, 필요한 컬럼만 조회
- 오류 처리: 쿼리 오류 발생 시 재작성 지시
- SQL 방언: 현재 DB의 dialect(SQLite, PostgreSQL 등) 명시

```
system_prompt = (
    "당신은 SQL 에이전트입니다. 단계:\n"
    "1. sql_db_list_tables\n2. sql_db_schema\n"
    "3. 쿼리 작성 + sql_db_query_checker\n"
    "4. sql_db_query\n5. 결과를 해석하세요.\n"
    f"규칙: LIMIT 10 사용. DML 금지. Dialect: {db.dialect}"
)
```

```
from langchain.agents import create_agent

sql_agent = create_agent(
    model=llm, tools=tools, system_prompt=system_prompt,
)
print("LangChain SQL 에이전트 생성됨.")
```

LangChain SQL 에이전트 생성됨.

6.5 실행 테스트

에이전트가 생성되었으니, 실제 자연어 질문을 던져 SQL 에이전트의 동작을 확인합니다. 에이전트가 도구를 호출하는 순서와 생성하는 SQL 쿼리를 주의 깊게 관찰하세요. 시스템 프롬프트에 지시한 워크플로(list_tables -> schema -> query_checker -> query)를 따르는지 확인하는 것이 핵심입니다.

LangChain `create_agent` 기반의 SQL 에이전트는 간단하게 프로토타입을 만들 수 있지만, 프로덕션 환경에서는 SQL 쿼리 실행 전 반드시 사람의 검토가 필요합니다. 다음 절에서 이를 구현합니다.

6.6 HITL – HumanInTheLoopMiddleware

프로덕션 환경에서는 SQL 쿼리 실행 전 사람의 승인이 필수입니다. 에이전트가 생성한 쿼리가 비용이 높거나, 예상치 못한 테이블에 접근하거나, 의도와 다른 결과를 반환할 수 있기 때문입니다.

`HumanInTheLoopMiddleware` 는 지정된 도구(`sql_db_query`) 호출을 가로채서 실행을 일시 중단하고, 사람이 리뷰할 수 있도록 합니다.

리뷰 옵션 3가지

에이전트가 `sql_db_query` 를 호출하려 할 때, 실행이 일시 중단되고 사람은 다음 중 하나를 선택합니다:

옵션	Command(resume=...) 값	설명
승인	<code>{"decisions": [{"type": "approve"}]}</code>	생성된 쿼리를 그대로 실행
수정	<code>{"decisions": [{"type": "edit", "args": {...}]}]</code>	쿼리를 수정한 후 실행
거부	<code>{"decisions": [{"type": "reject", "message": "..."}]}</code>	쿼리를 실행하지 않고 사유를 전달



TIP

v1 `HumanInTheLoopMiddleware` 는 동일 인터럽트에서 여러 도구 호출을 한 번에 검토할 수 있도록 `decisions` 배열을 받습니다. 각 항목은 도구 호출 순서와 1:1로 대응하며 `type` 은 `approve` / `edit` / `reject` 중 하나입니다.

왜 HITL이 중요한가?

리뷰 단계는 단순 승인 버튼이 아니라 분기점입니다.

- 승인 경로 - 생성된 쿼리를 그대로 실행하여 빠른 응답 제공
- 수정 경로 - WHERE 조건, LIMIT, 집계 방식만 사람 손으로 보정
- 거절 경로 - 쿼리를 실행하지 않고 질문을 더 좁히거나 정책 위반 사유를 설명
- 비용 제어: `LIMIT` 없는 대규모 테이블 풀 스캔 방지
- 데이터 보호: 민감한 컬럼 접근 사전 차단
- 정확성 검증: 에이전트가 질문 의도를 잘못 해석한 경우 수정 가능
- 감사 추적(Audit Trail): 모든 실행된 쿼리에 대한 승인 기록 유지

```
from langchain.agents.middleware import HumanInTheLoopMiddleware

hitl = HumanInTheLoopMiddleware(
    interrupt_on={"sql_db_query": True},
)
sql_agent_hitl = create_agent(
    model=llm, tools=tools,
    system_prompt=system_prompt, middleware=[hitl],
)
print("HITL이 적용된 SQL 에이전트 생성됨.")
```

HITL이 적용된 SQL 에이전트 생성됨.

DELETE / UPDATE 같은 파괴적 쿼리에 대한 재개 패턴

읽기 전용 SELECT는 `approve` 만으로 충분하지만, DELETE/UPDATE/INSERT 같은 파괴적 쿼리는 보통 `edit` 으로 `WHERE` 조건이나 `LIMIT` 을 보강한 뒤 실행합니다. 인터럽트가 발생하면 `Command(resume=...)` 의 `decisions` 배열로 의사결정을 한 번에 전달합니다.

```
from langgraph.types import Command

# 위험한 쿼리 - 사람 검토 후 LIMIT를 추가하여 승인
result = sql_agent_hitl.invoke(
    Command(resume={
        "decisions": [{
            "type": "edit",
            "args": {
                "query": (
                    "UPDATE invoices SET status='void' "
                    "WHERE customer_id=42 AND total < 1 LIMIT 10"
                )
            },
        }],
    }),
    config={"configurable": {"thread_id": "sql-session-1"}},
)
```

`create_agent` 기반 SQL 에이전트는 빠르게 프로토타입을 만들 수 있지만, 도구 호출 순서가 LLM의 자율 판단에 의존합니다. 프로덕션에서는 스키마 조회 → 쿼리 생성 → 검증 → 실행의 순서를 강제하고, 쿼리 리뷰를 위한 정확한 중단점을 설정해야 합니다. LangGraph `StateGraph` 가 이를 가능하게 합니다.

6.7 LangGraph 커스텀 SQL Agent – StateGraph

LangChain `create_agent` 는 빠르게 프로토타입을 만들 수 있지만, 노드 단위의 세밀한 제어가 필요하면 LangGraph `StateGraph` 를 사용합니다. 각 단계를 독립적인 노드로 정의하여 다음을 실현할 수 있습니다:

- 조건부 분기: 쿼리 검증 실패 시 재생성 노드로 라우팅
- 강제 도구 호출: `bind_tools(tool_choice=...)` 로 특정 노드에서 반드시 특정 도구 호출
- 세밀한 중단점: `interrupt()` 로 원하는 노드에서 정확히 실행 중단
- 커스텀 상태: 쿼리 이력, 재시도 횟수 등을 상태에 추가

그래프 구조

```
START -> list_tables -> get_schema -> generate_query
      -> check_query -> execute_query -> END
```

각 노드는 공유 `State` 객체를 받아 메시지를 추가하며, 에이전트가 워크플로우를 진행하는 동안 대화 이력이 누적됩니다. `tools_condition` 을 사용하면 `check_query` 결과에 따라 쿼리를 재생성하거나 실행으로 진행하는 조건부 분기를 구현할 수 있습니다.

LangChain `create_agent` 대비 장점

측면	<code>create_agent</code>	<code>StateGraph</code>
도구 호출 순서	LLM 자율 결정	그래프 엣지로 강제
오류 시 재시도	시스템 프롬프트에 의존	조건부 엣지로 명시적 구현
사람 리뷰	미들웨어 기반	<code>interrupt()</code> 기반, 위치 자유
디버깅	블랙박스	노드별 상태 확인 가능

```
from typing import Annotated
from typing_extensions import TypedDict
from langgraph.graph.message import add_messages

class SQLState(TypedDict):
    messages: Annotated[list, add_messages]

print(f"SQLState 키: {list(SQLState.__annotations__)}")
```

```
SQLState 키: ['messages']
```

6.8 전용 노드 - `list_tables`, `get_schema`, `generate_query`, `check_query`

각 노드는 SQL 에이전트 워크플로우의 한 단계를 담당합니다.

6.9 `bind_tools` with `tool_choice` - 강제 도구 호출

`tool_choice` 파라미터로 특정 도구를 강제 호출하도록 설정합니다. 이는 `create_agent` 의 ReAct 루프에서 LLM 이 자율적으로 도구를 선택하는 것과 대비됩니다. `StateGraph` 에서는 특정 노드에서 반드시 특정 도구를 호출하도록 강제할 수 있어, 워크플로의 일관성이 보장됩니다. 예를 들어, `list_tables` 노드에서는 반드시 `sql_db_list_tables` 도구를 호출하고, `get_schema` 노드에서는 반드시 `sql_db_schema` 를 호출하도록 설정할 수 있습니다.

➡ TIP

`bind_tools(tools, tool_choice="sql_db_list_tables")` 는 LLM에게 해당 도구를 반드시 호출하도록 강제합니다. 이는 OpenAI API의 `tool_choice` 파라미터를 활용하며, LLM이 도구 호출 없이 텍스트 응답만 생성하는 것을 방지합니다. 특정 노드에서 확실한 도구 호출이 필요할 때 사용하세요.

6.10 `interrupt()` 로 쿼리 리뷰

LangGraph의 `interrupt()` 함수는 그래프 실행을 일시 중단하고 외부 입력(사람의 리뷰)을 기다립니다.

`HumanInTheLoopMiddleware` 와 달리 `interrupt()` 는 노드 내부 코드의 정확한 위치에서 중단할 수 있어 더 유연합니다.

동작 원리

1. 노드 함수 내에서 `interrupt(payload)` 를 호출하면 그래프 실행이 즉시 중단됩니다
2. `payload` 는 클라이언트에게 전달되어 리뷰 UI에 표시됩니다 (예: 생성된 SQL 쿼리)
3. 클라이언트가 `Command(resume=value)` 로 그래프를 재개하면, `interrupt()` 가 `value` 를 반환합니다
4. 노드 함수는 반환된 값에 따라 쿼리를 실행, 수정, 또는 거부합니다

`interrupt()` VS `HumanInTheLoopMiddleware`

특성	<code>interrupt()</code>	<code>HumanInTheLoopMiddleware</code>
적용 범위	노드 내 코드 레벨	도구 호출 레벨
유연성	임의 로직 구현 가능	도구 호출 가로채기만 가능
상태 접근	전체 State 접근 가능	도구 인자만 접근 가능
체크포인트	필수 (상태 저장 필요)	선택적

6.11 Command(resume=...) 패턴

`interrupt()` 로 중단된 그래프를 재가하려면 `Command(resume=...)` 를 사용합니다. 이 패턴은 에이전트 실행의 비 동기적 중단과 재개를 가능하게 합니다. 웹 애플리케이션에서는 사용자가 쿼리를 리뷰하는 동안 에이전트 상태가 체크포인트에 저장되며, 사용자의 결정(승인/수정/거부)이 `Command(resume=...)` 를 통해 전달되면 정확히 중단된 지점에서 실행이 재개됩니다.

! WARNING

`interrupt()` 사용 시 반드시 체크포인트(`InMemorySaver`, `SqliteSaver` 등)를 설정해야 합니다. 체크포인트 없이 `interrupt()` 를 호출하면 그래프 상태가 소실되어 재개가 불가능합니다. 또한 `thread_id` 를 통해 세션을 식별하므로, 같은 `thread_id` 로 `Command(resume=...)` 를 전달해야 올바른 세션이 재개됩니다.

```
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import InMemorySaver

builder = StateGraph(SQLState)
builder.add_node("list_tables", list_tables_node)
builder.add_node("get_schema", get_schema_node)
builder.add_node("generate_query", generate_query_node)
builder.add_node("check_query", check_query_node)
builder.add_node("execute_query", execute_query_node)
```

```
<langgraph.graph.state.StateGraph at 0x1f6f9914b00>
```

```
builder.add_edge(START, "list_tables")
builder.add_edge("list_tables", "get_schema")
builder.add_edge("get_schema", "generate_query")
builder.add_edge("generate_query", "check_query")
builder.add_edge("check_query", "execute_query")
builder.add_edge("execute_query", END)

checkpointer = InMemorySaver()
sql_graph = builder.compile(checkpointer=checkpointer)
print("LangGraph SQL 에이전트 컴파일됨.")
```

```
LangGraph SQL 에이전트 컴파일됨.
```

요약

두 가지 SQL Agent 비교

특성	LangChain <code>create_agent</code>	LangGraph <code>StateGraph</code>
구현 복잡도	낮음 (5줄)	높음 (전용 노드)
제어 수준	ReAct 자동	노드 단위 커스텀

HITL	HumanInTheLoopMiddleware	<code>interrupt() + Command(resume=...)</code>
강제 도구 호출	미지원	<code>bind_tools(tool_choice=...)</code>
적합한 경우	빠른 프로토타입	프로덕션, 세밀한 제어

HITL 패턴

액션	<code>Command(resume=...)</code>
Approve	<code>{"decisions": [{"type": "approve"}]}</code>
Edit	<code>{"decisions": [{"type": "edit", "args": {...}]}</code>
Reject	<code>{"decisions": [{"type": "reject", "message": "..."}]}</code>

SQLDatabaseToolkit 4개 도구

도구	단계	용도
<code>sql_db_list_tables</code>	2	테이블 목록 확인
<code>sql_db_schema</code>	3	DDL + 샘플 데이터 조회
<code>sql_db_query_checker</code>	5	쿼리 사전 검증
<code>sql_db_query</code>	7	쿼리 실행

SQL 에이전트는 정형 데이터에 자연어 인터페이스를 제공합니다. 다음 장에서는 코드 실행 샌드박스를 활용하여 비정형 데이터(CSV, 이미지 등)를 프로그래밍 방식으로 분석하는 데이터 분석 에이전트를 구축합니다.

07 데이터 분석 에이전트

Deep Agents + 샌드박스

이전 장의 SQL 에이전트가 데이터베이스에 질의했다면, 데이터 분석 에이전트는 코드 실행 샌드박스에서 Python 코드를 작성하고 실행하여 CSV 데이터를 분석합니다. pandas로 데이터를 탐색하고, matplotlib으로 시각화하며, 분석 결과를 Slack으로 공유하는 자율 에이전트를 구축합니다.

Deep Agents SDK의 `create_deep_agent`, 백엔드 시스템, 빌트인 도구, 체크포인트를 활용합니다. 에이전트가 코드를 생성하고 실행하는 과정을 스트리밍으로 실시간 관찰하는 것이 이 장의 핵심입니다.

학습 목표

이 노트북을 완료하면 다음을 수행할 수 있습니다:

1. 백엔드 선택 — `LocalShellBackend` (개발)과 `DaytonaSandbox` / `Modal` / `Runloop` (운영) 간 차이를 이해하고 선택할 수 있다
2. 커스텀 도구 정의 — `@tool` 데코레이터로 Slack 연동 등 외부 서비스 도구를 만들 수 있다
3. 에이전트 생성 — `create_deep_agent()` 로 모델, 도구, 백엔드, 체크포인트를 조합한 에이전트를 구성할 수 있다
4. 스트리밍 관찰 — 에이전트의 분석 과정을 실시간으로 모니터링할 수 있다
5. 빌트인 도구 활용 — `write_todos`, `ls`, `read_file`, `write_file`, `edit_file`, `glob`, `grep` 도구의 역할을 이해할 수 있다
6. 체크포인트 — `InMemorySaver` 로 대화 상태를 유지하고 이어서 분석을 수행할 수 있다

7.1 환경 설정

Deep Agents SDK와 Tavily(웹 검색), Slack SDK를 설치합니다. 실행 환경에 따라 `pip` 또는 `uv` 를 사용합니다. Deep Agents는 LangGraph 위에 구축된 올인원 에이전트 SDK로, 코드 실행 백엔드, 빌트인 도구, 서버 에이전트 시스템을 내장하고 있어 데이터 분석 에이전트 구축에 최적화되어 있습니다.

```
from dotenv import load_dotenv

load_dotenv()
```

```
True
```

7.2 데이터 분석 에이전트 개요

Deep Agents의 데이터 분석 에이전트는 다음 파이프라인을 자율적으로 실행합니다:

CSV 입력 → 계획 수립(write_todos) → 파일 읽기(read_file) → 코드 생성 & 실행 → 반복 분석 → 결과 전달(Slack)

실행 흐름 상세

단계	설명	사용 도구
Planning	write_todos 로 구조화된 작업 계획을 수립하고, 분석 진행에 따라 TODO를 업데이트	write_todos
File Reading	CSV의 구조, 컬럼명, 데이터 타입, 행 수를 파악. 이미지 파일도 멀티모달로 읽기 가능	read_file
Code Execution	pandas, matplotlib 등 Python 코드를 작성하여 백엔드에서 격리 실행	Backend execute
Iterative Analysis	초기 결과를 기반으로 추가 분석 수행, 웹 검색으로 도메인 맥락 확보	Tavily, edit_file
Result Delivery	분석 결과를 포매팅하여 Slack 채널에 전송	slack_send_message

자율 에이전트의 핵심 특성

Deep Agents의 에이전트는 단순한 도구 호출을 넘어 자율적 계획 수립 능력을 갖추고 있습니다:

1. 계획 수립: write_todos 를 통해 분석 작업을 세분화하고 진행 상황을 추적합니다
2. 적응적 실행: 분석 중 오류가 발생하면 코드를 수정(edit_file)하고 재실행합니다
3. 서브에이전트 위임: 복잡한 작업은 전문 서브에이전트를 생성하여 병렬 처리합니다
4. 컨텍스트 관리: 파일 시스템 도구로 중간 결과를 저장하고 필요할 때 다시 참조합니다

데이터 분석 에이전트의 가장 중요한 설계 결정은 코드를 어디서 실행할 것인가입니다. 로컬 환경에서 직접 실행하면 간편하지만 보안 위험이 있고, 클라우드 샌드박스는 안전하지만 설정이 필요합니다.

7.3 백엔드 선택

Deep Agents는 플러그형 백엔드 아키텍처를 통해 파일시스템 및 코드 실행 환경을 제공합니다. 모든 백엔드는 동일한 BackendProtocol 을 구현하므로, 코드를 변경하지 않고 백엔드만 교체할 수 있습니다.

백엔드	용도	보안 수준	설정 난이도
LocalShellBackend	로컬 개발/테스트	낮음 (호스트 시스템 전체 접근)	설정 불필요
Daytona	클라우드 샌드박스 환경	높음 (격리된 컨테이너)	API 키 필요
Modal	서버리스 GPU/CPU 연산	높음	Modal 계정 필요

RunLoop	관리형 클라우드 실행	높음	API 키 필요
---------	-------------	----	----------

백엔드 유형별 상세

- `StateBackend` (기본값): 파일을 LangGraph 에이전트 상태에 저장합니다. 스크래치패드 용도로 적합하며, 큰 출력물은 자동 제거됩니다.
- `FilesystemBackend`: `root_dir` 설정으로 로컬 디스크 접근을 제공합니다. `virtual_mode=True` 옵션으로 경로 제한 및 디렉터리 탐색 방식이 가능합니다.
- `LocalShellBackend`: 파일시스템 접근에 더해 `execute` 도구를 통한 무제한 셸 명령 실행을 제공합니다. 호스트 시스템에 대한 전체 사용자 권한으로 실행됩니다.
- `CompositeBackend`: 경로별로 다른 백엔드를 라우팅합니다. 예: 임시 파일은 `StateBackend`, `/memories/`는 `StoreBackend`.

샌드박스 보안 원칙

샌드박스는 에이전트가 코드를 안전하게 실행할 수 있는 격리된 환경을 제공합니다. 핵심 보안 원칙:

- 절대로 시크릿을 샌드박스 안에 넣지 마세요 - 컨텍스트 주입 공격으로 에이전트가 환경 변수나 마운트된 파일의 자격 증명을 읽어 유출할 수 있습니다
- 자격 증명은 샌드박스 외부의 도구에 보관하세요
- 민감한 작업에는 Human-in-the-Loop 승인을 사용하세요
- 불필요한 네트워크 접근을 차단하세요

! WARNING

주의: `LocalShellBackend`는 호스트 시스템에 대한 무제한 셸 실행 권한을 가집니다. 프로덕션에서는 반드시 샌드박스 백엔드를 사용하세요.

```
from deepagents.backends import LocalShellBackend

# 개발용 - 로컬 셸 백엔드
dev_backend = LocalShellBackend(virtual_mode=True)

# 운영용 - 클라우드 샌드박스 (택 1)
# prod_backend = ... # 프로덕션에서는 클라우드 샌드박스 백엔드를 사용하세요
```

백엔드가 구성되었으니, 에이전트가 분석할 데이터를 준비합니다. 실제 프로덕션에서는 사용자가 파일을 업로드하거나 에이전트가 API에서 데이터를 가져오겠지만, 여기서는 학습 목적으로 직접 CSV를 생성합니다.

7.4 샘플 데이터 업로드

에이전트가 분석할 CSV 파일을 백엔드의 작업 디렉터리에 생성합니다. `create_deep_agent`의 백엔드는 `write_file` 도구를 내장하고 있어 에이전트가 직접 파일을 쓸 수 있지만, 여기서는 미리 데이터를 준비합니다.



TIP

에이전트에게 데이터를 제공하는 방식은 크게 세 가지입니다: (1) 백엔드의 작업 디렉터리에 미리 파일을 배치, (2) 에이전트의 `write_file` 빌트인으로 런타임에 파일 생성, (3) 커스텀 도구로 외부 API/DB에서 데이터 로드. 분석 시나리오에 따라 적합한 방식을 선택하세요.

```
import os

workspace = "/tmp/analysis"
os.makedirs(workspace, exist_ok=True)

csv_content = (
    "region,quarter,revenue,units_sold\n"
    "Seoul,Q1,120000,340\nSeoul,Q2,135000,380\n"
    "Seoul,Q3,128000,355\nSeoul,Q4,150000,420\n"
    "Busan,Q1,85000,240\nBusan,Q2,92000,260\n"
    "Busan,Q3,88000,250\nBusan,Q4,105000,300"
)
```

```
with open(f"{workspace}/sales_2025.csv", "w") as f:
    f.write(csv_content)
print(f"CSV 저장됨: {workspace}/sales_2025.csv")
```

CSV 저장됨: /tmp/analysis/sales_2025.csv

7.5 커스텀 도구 – Slack 연동

`@tool` 데코레이터를 사용해 에이전트가 호출할 수 있는 커스텀 도구를 정의합니다. 도구의 docstring이 에이전트에게 사용법을 알려주는 역할을 합니다.

아래는 분석 결과를 Slack 채널로 전송하는 도구입니다.

```
from langchain_core.tools import tool
try:
    from slack_sdk import WebClient
except ImportError:
    WebClient = None
    print("slack_sdk 미설치 -- Slack 도구는 스텝으로 작동합니다")

slack_client = WebClient(token=os.environ.get("SLACK_BOT_TOKEN", "xoxb-placeholder")) if WebClient else None

@tool
def slack_send_message(message: str) -> str:
    """분석 결과를 Slack 채널로 전송합니다."""
    resp = slack_client.chat_postMessage(
        channel=os.environ.get("SLACK_CHANNEL_ID", "general"), text=message)
    return f"전송 완료. 타임스탬프: {resp['ts']}"
```

웹 검색 도구 (Tavily)

에이전트가 분석 중 도메인 맥락이 필요할 때 웹 검색을 수행할 수 있도록 Tavily 도구를 준비합니다.

```
from tavily import TavilyClient
```

```
tavily_client = TavilyClient(api_key=os.environ["TAVILY_API_KEY"])

def tavily_search(query: str) -> str:
    """분석에 관련된 정보를 웹에서 검색합니다."""
    results = tavily_client.search(query, max_results=5)
    return "\n".join(
        [r["content"] for r in results["results"]]
    )
```

pandas / scikit-learn @tool 통합

데이터 분석 에이전트는 종종 도구가 곧 작은 분석 셸입니다. @tool 데코레이터는 모델이 호출하는 시점에 pandas / scikit-learn 코드를 그대로 실행하고, 결과를 텍스트로 다시 모델에 돌려줍니다. 파일시스템 접근 (read_file) 과 분리해 두면, 입력 검증·캐싱·테스트가 쉬워집니다.

```
import pandas as pd
from sklearn.cluster import KMeans
from langchain_core.tools import tool

@tool
def summarize_csv(path: str, top_n: int = 5) -> str:
    """CSV 파일의 기초 통계와 상위 N개 행을 요약합니다."""
    df = pd.read_csv(path)
    return (
        f"shape={df.shape}\n"
        f"dtypes={df.dtypes.to_dict()}\n"
        f"head=\n{df.head(top_n).to_string()}"
    )

@tool
def cluster_customers(path: str, k: int = 3) -> str:
    """수치 컬럼만 사용해 KMeans 클러스터링을 수행합니다."""
    df = pd.read_csv(path).select_dtypes("number").dropna()
    labels = KMeans(n_clusters=k, n_init="auto").fit_predict(df)
    return f"cluster_sizes={pd.Series(labels).value_counts().to_dict()}"
```

CodeInterpreterMiddleware 와 Skills 패턴

@tool 만으로 작성하기 어려운 임시 코드 — 예를 들어 “이 컬럼 분포를 박스플롯으로 그려줘” 같은 자유 형식 분석 — 은 CodeInterpreterMiddleware 가 자체적인 샌드박스 위에서 실행합니다. 그리고 자주 쓰는 분석 절차는 skills/ 디렉터리에 마크다운 + 코드 스니펫으로 묶어 두면 에이전트가 Skill 단위로 호출합니다.

```
from deepagents.middleware import CodeInterpreterMiddleware
from deepagents.skills import load_skills

skills = load_skills("./skills/data-analysis") # 마크다운 + 코드 스니펫

agent = create_deep_agent(
    model="gpt-5.4",
    tools=[summarize_csv, cluster_customers, slack_send_message],
    middleware=[CodeInterpreterMiddleware(backend=backend)],
    skills=skills,
    backend=backend,
    checkpointer=checkpointer,
    system_prompt="당신은 데이터 분석가입니다.",
)
```



TIP

Skills 디렉터리에는 재현 가능한 분석 레시피만 두고, 일회성 탐색 코드는 `CodeInterpreterMiddleware` 가 처리하도록 분리하세요. 이렇게 하면 검증된 절차와 즉흥 코드가 섞이지 않습니다.

커스텀 도구까지 정의했으니, 이제 모든 구성 요소를 하나로 조립하여 데이터 분석 에이전트를 생성합니다.

`create_deep_agent()` 는 모델, 도구, 백엔드, 체크포인트를 받아 완전한 에이전트를 반환합니다.

7.6 에이전트 생성

`create_deep_agent()` 는 모델, 도구, 백엔드, 체크포인트, 시스템 프롬프트를 조합하여 에이전트를 생성합니다. 에이전트는 전달된 커스텀 도구(`tavily_search` , `slack_send_message`)와 백엔드가 제공하는 빌트인 도구(`read_file` , `write_file` , `execute` 등)를 모두 사용할 수 있습니다. 빌트인 도구는 백엔드 유형에 따라 자동으로 결정됩니다.

파라미터	타입	설명
<code>model</code>	<code>str</code>	모델 식별자 (기본값: <code>claude-sonnet-4-6</code>)
<code>tools</code>	<code>list</code>	에이전트가 사용할 도구 함수 목록
<code>backend</code>	<code>Backend</code>	코드 실행 및 파일시스템 백엔드
<code>checkpointer</code>	<code>Checkpointter</code>	상태 영속화 메커니즘
<code>system_prompt</code>	<code>str</code>	에이전트 행동 지침

```
from deepagents import create_deep_agent
from deepagents.backends import LocalShellBackend
from langgraph.checkpoint.memory import InMemorySaver
```

```
backend = LocalShellBackend(virtual_mode=True)
checkpointer = InMemorySaver()
```

```
agent = create_deep_agent(
    model="gpt-5.4",
    tools=[tavily_search, slack_send_message],
    backend=backend,
    checkpointer=checkpointer,
    system_prompt="당신은 데이터 분석가입니다.",
)
```

7.7 실행 — 분석 요청

`agent.invoke()` 로 분석 요청을 전달하면 에이전트가 자율적으로 계획 수립 -> 파일 읽기 -> 코드 실행 -> 결과 전달 파이프라인을 수행합니다. 에이전트의 자율성이 이 단계에서 드러납니다: 사용자는 “매출 추이를 분석해 줘”라는 고수준 요청만 하면, 에이전트가 어떤 파일을 읽을지, 어떤 pandas 코드를 실행할지, 어떤 시각화를 생성할지 스스로 결정합니다.

에이전트가 `invoke()` 로 분석을 완료하면 최종 결과만 반환됩니다. 그러나 데이터 분석에서는 과정이 결과만큼 중요합니다. 에이전트가 어떤 코드를 작성했는지, 중간 결과가 어떤지, 오류를 어떻게 수정했는지 실시간으로 확인할 수 있어야 합니다.

7.8 스트리밍으로 분석 과정 관찰

`agent.stream()` 을 사용하면 에이전트의 실행 과정을 실시간으로 관찰할 수 있습니다. Deep Agents는 LangGraph의 스트리밍 인프라를 기반으로 서브에이전트의 실행까지 추적할 수 있는 강력한 모니터링을 제공합니다.

스트림 모드

모드	설명	사용 사례
Updates	각 단계(노드) 완료 시 이벤트 수신	진행 상황 대시보드, 로그
Messages	개별 토큰 스트리밍, 소스 에이전트 메타데이터 포함	실시간 채팅 UI
Custom	<code>get_stream_writer()</code> 로 커스텀 진행 이벤트 발행	분석 진행률 표시, 커스텀 알림

네임스페이스 시스템

스트리밍 이벤트에는 소스를 식별하는 네임스페이스가 포함됩니다:

- `()` (빈 튜플) = 메인 에이전트
- `("tools:abc123",)` = 도구 호출로 생성된 서브에이전트
- `("tools:abc123", "model_request:def456")` = 서브에이전트 내부의 모델 요청 노드

`subgraphs=True` 를 설정하면 서브에이전트의 실행까지 추적할 수 있으며, 여러 스트림 모드를 동시에 사용할 수도 있습니다:

```
for namespace, chunk in agent.stream(
    {"messages": [...]},
    stream_mode=["updates", "messages", "custom"],
    subgraphs=True,
):
    mode, data = chunk
    # 각 모드별로 다르게 처리
```

서브에이전트 라이프사이클 추적

서브에이전트는 세 단계를 거칩니다:

1. Pending - 메인 에이전트의 `model_request` 에 태스크 도구 호출이 포함될 때 감지
2. Running - `tools:UUID` 네임스페이스에서 이벤트가 발생할 때 시작
3. Complete - 메인 에이전트의 `tools` 노드가 결과를 반환할 때 완료

7.9 빌트인 도구 활용

Deep Agents는 백엔드를 통해 다음 빌트인 도구를 자동으로 에이전트에게 제공합니다. 에이전트는 분석 과정에서 이 도구들을 자율적으로 선택하여 사용합니다.

도구	설명	사용 예시
<code>write_todos</code>	구조화된 작업 계획 수립 및 추적	분석 단계별 TODO 리스트 생성
<code>ls</code>	디렉터리 내용 나열 (<code>ls_info()</code>)	CSV 파일 존재 여부 확인
<code>read_file</code>	파일 읽기 (이미지 멀티모달 지원)	CSV 구조 파악, 차트 이미지 확인
<code>write_file</code>	새 파일 생성 (create-only)	분석 스크립트, 결과 파일 저장
<code>edit_file</code>	기존 파일 수정 (find-and-replace)	코드 수정, 설정 파일 업데이트
<code>glob</code>	glob 패턴 기반 파일 탐색	<code>*.csv</code> 패턴으로 데이터 파일 검색
<code>grep</code>	패턴 매칭 검색	특정 컬럼명이나 값 검색

`read_file`의 멀티모달 지원

`read_file`은 모든 백엔드에서 이미지 파일을 멀티모달 콘텐츠로 지원합니다. 에이전트가 matplotlib으로 차트를 생성한 후, `read_file`로 해당 이미지를 읽으면 차트의 내용을 시각적으로 해석할 수 있습니다. 이를 통해 “차트가 올바르게 생성되었는지”, “추가적으로 어떤 시각화가 필요한지” 등을 자율적으로 판단합니다.

커스텀 백엔드 구현

필요에 따라 `BackendProtocol`을 직접 구현할 수 있습니다. 필수 메서드:

- `ls_info()` - 디렉터리 내용 나열
- `read()` - 줄 번호와 함께 파일 읽기
- `grep_raw()` - 구조화된 매치를 반환하는 패턴 매칭
- `glob_info()` - glob 기반 파일 매칭
- `write()` - 파일 생성 (create-only)
- `edit()` - 유일성을 보장하는 find-and-replace

빌트인 도구를 이해했으니, 마지막으로 에이전트의 장기 실행을 지원하는 체크포인트를 살펴봅시다. 데이터 분석은 한 번의 요청으로 끝나지 않습니다 - “지역별 매출 분석” 후 “그 중 서울만 더 자세히 분석해”와 같은 멀티턴 대화가 자연스럽습니다.

7.10 체크포인트로 대화 유지

체크포인트는 에이전트 상태를 저장하여 중단 후 재개가 가능하게 합니다.

! WARNING

클라우드 샌드박스(Daytona, Modal 등)를 사용할 때는 반드시 샌드박스의 수명(lifetime)을 관리해야 합니다. 체크포인트가 에이전트 상태를 저장하더라도, 샌드박스의 실행 환경(설치된 패키지, 생성된 파일 등)은 샌드박스

종료 시 소멸합니다. TTL(Time-to-Live)을 설정하여 비활성 샌드박스를 자동 정리하고, 중요한 분석 결과는 외부 스토리지에 저장하세요.

`thread_id` 를 통해 대화 세션을 식별하며, 동일한 `thread_id` 로 이어서 요청하면 이전 대화 맥락이 유지됩니다.

체크포인트	용도	영속성
InMemorySaver	개발/테스트	프로세스 종료 시 소멸
SQLiteSaver	로컬 영속	디스크에 저장 (<code>./agent_checkpoints.db</code>)
PostgresSaver	프로덕션	데이터베이스에 저장, 다중 인스턴스 지원

체크포인트의 역할

체크포인트는 단순한 대화 이력 저장을 넘어 다음을 가능하게 합니다:

- 중단 복구: 네트워크 오류나 타임아웃 시 마지막 완료된 단계부터 재개
- `interrupt()` 지원: Human-in-the-Loop에서 그래프 상태를 저장하여 사람의 응답 후 정확한 위치에서 재개
- 멀티턴 분석: 동일 세션에서 여러 분석 요청을 연속으로 처리하면서 이전 결과를 참조

샌드박스 수명 관리

샌드박스를 사용할 때는 명시적인 종료(shutdown)가 필요합니다. 종료하지 않으면 불필요한 비용이 발생합니다. 채팅 애플리케이션에서는 대화 스레드별로 고유한 샌드박스를 사용하고, TTL(Time-to-Live) 설정으로 자동 정리를 구성하세요.

```
from langgraph.checkpoint.memory import InMemorySaver

checkpointer = InMemorySaver()
config = {"configurable": {"thread_id": "analysis-session-1"}}

agent_with_memory = create_deep_agent(
    model="gpt-5.4",
    tools=[tavily_search, slack_send_message],
    backend=LocalShellBackend(virtual_mode=True),
    checkpointer=checkpointer,
)
```

요약

이 노트북에서 다룬 내용을 정리합니다:

주제	핵심 내용
백엔드	LocalShell (개발)은 호스트 셸 접근, 프로덕션에서는 Daytona / Modal / Runloop 샌드박스 사용
커스텀 도구	<code>\@tool</code> 데코레이터 + docstring으로 정의, 에이전트가 자율 호출

에이전트 생성	<code>create_deep_agent(model, tools, backend, checkpointer, system_prompt)</code>
스트리밍	<code>agent.stream(stream_mode="updates", subgraphs=True)</code> 로 실시간 관찰
빌트인 도구	<code>write_todos</code> , <code>ls</code> , <code>read_file</code> , <code>write_file</code> , <code>edit_file</code> , <code>glob</code> , <code>grep</code>
체크포인트	<code>InMemorySaver</code> (개발) → <code>SQLiteSaver</code> → <code>PostgresSaver</code> (프로덕션)

데이터 분석 에이전트는 텍스트 입출력을 기반으로 합니다. 다음 장에서는 텍스트를 넘어 음성 입출력을 다루는 보이스 에이전트를 구축합니다. STT → Agent → TTS의 3-레이어 파이프라인으로 sub-700ms 레이턴시의 실시간 음성 인터페이스를 구현합니다.

08 보이스 에이전트

STT/Agent/TTS 파이프라인

이전 장까지의 에이전트는 모두 텍스트 인터페이스를 사용했습니다. 이 장에서는 인간에게 가장 자연스러운 소통 수단인 음성으로 에이전트와 상호작용하는 방법을 다룹니다. 음성 입력을 텍스트로 변환(STT)하고, LangChain 에이전트가 처리한 뒤, 텍스트를 음성으로 합성(TTS)하여 실시간으로 돌려주는 보이스 에이전트를 구축합니다.

이 아키텍처는 Sandwich 패턴 (STT → Agent → TTS)으로, 각 계층이 스트리밍으로 연결되어 sub-700ms 레이턴시를 목표로 합니다. 스트리밍이 핵심입니다 – 각 레이어가 이전 레이어의 완전한 출력을 기다리지 않고, 부분 결과가 생성되는 즉시 다음 레이어로 전달합니다. WebSocket 기반의 양방향 실시간 통신으로 이를 구현합니다.

보이스 에이전트는 텍스트 기반 에이전트와 동일한 추론 능력을 유지하면서 음성이라는 가장 자연스러운 인터페이스를 제공합니다. 이전 장까지의 에이전트와 근본적으로 다른 점은 실시간성에 대한 요구입니다. 텍스트 채팅에서는 몇 초의 지연이 허용되지만, 음성 대화에서 700ms 이상의 침묵은 사용자에게 “끊어진 대화”로 느껴집니다.

핵심 설계 원칙

Sandwich 아키텍처의 핵심은 스트리밍 체이닝입니다. 각 레이어가 이전 레이어의 완전한 출력을 기다리지 않고, 부분 결과가 생성되는 즉시 다음 레이어로 전달합니다:

- STT: 부분 전사(partial transcript)를 실시간으로 생성하고, 발화 완료(end-of-speech) 감지 시 최종 전사를 emit
- Agent: `astream()` 을 통해 토큰 단위로 응답을 스트리밍 – 전체 응답 생성 완료를 기다리지 않음
- TTS: WebSocket 기반으로 텍스트 청크가 도착하는 즉시 오디오 합성 시작

이 구조 덕분에 전체 파이프라인의 지연 시간은 각 단계의 합이 아닌, 각 단계의 첫 출력까지의 시간 합으로 줄어듭니다.

학습 목표

이 노트북을 완료하면 다음을 수행할 수 있습니다:

1. Sandwich 아키텍처 – STT → Agent → TTS 파이프라인의 구조와 데이터 흐름을 설명할 수 있다
2. 아키텍처 비교 – Sandwich 방식과 Speech-to-Speech(S2S) 방식의 장단점을 비교할 수 있다
3. STT 단계 – AssemblyAI 실시간 전사의 Producer-Consumer 패턴을 이해할 수 있다
4. 에이전트 단계 – LangChain `create_agent` 로 스트리밍 응답을 생성하는 에이전트를 구축할 수 있다
5. TTS 단계 – Cartesia WebSocket 기반 스트리밍 음성 합성의 동작 원리를 이해할 수 있다
6. RunnableGenerator – 비동기 제너레이터 체이닝으로 파이프라인을 조합할 수 있다
7. 성능 최적화 – 레이턴시 목표 달성을 위한 각 단계별 최적화 기법을 이해할 수 있다

핵심 질문

보이스 에이전트의 핵심은 정답 자체보다 “얼마나 빨리 첫 오디오를 돌려줄 수 있는가?” 입니다. 이 장은 그 지연 시간을 어떻게 분해하고 줄일지에 집중합니다.

한눈에 보는 핵심

- Sandwich 아키텍처는 STT, Agent, TTS를 겹쳐 실행해 체감 지연을 줄입니다.
- 성능 목표는 전체 완료 시간이 아니라 첫 출력까지의 시간(TTFT, TTFA)입니다.
- 스트리밍을 끊는 병목은 종종 모델보다 파이프라인 연결 방식에서 발생합니다.

· NOTE

참고: STT(AssemblyAI)와 TTS(Cartesia)는 외부 유료 서비스입니다. 해당 셀은 개념 설명용 마크다운으로 제공되며, 에이전트 생성 부분만 실제 실행 가능한 코드입니다.

8.1 환경 설정

보이스 에이전트에 필요한 패키지와 각 역할입니다:

패키지	역할
langchain , langchain-openai	에이전트 생성 및 LLM 연결
assemblyai	실시간 음성-텍스트 변환 (STT)
cartesia	저지연 텍스트-음성 합성 (TTS)
websockets	실시간 양방향 통신 서버
pyaudio	마이크 오디오 캡처 (선택)

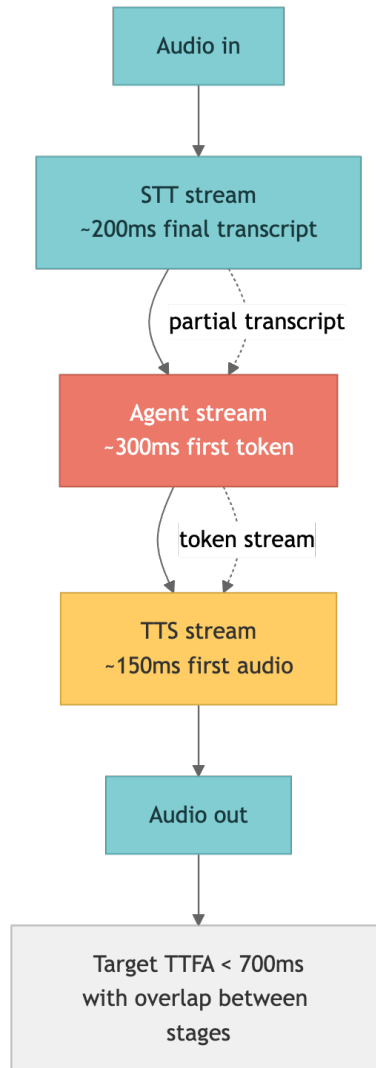
```
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI

load_dotenv()

model = ChatOpenAI(model="gpt-5.4")
```

8.2 보이스 에이전트 아키텍처 개요

보이스 에이전트는 3-레이어 파이프라인으로 구성됩니다:



이 파이프라인은 세 단계가 겹쳐서 실행될 때 비로소 자연스럽게 동작합니다. STT가 문장을 완전히 마치기 전에도 부분 전사를 넘기고, Agent는 첫 토큰이 나오자마자 TTS로 전달하여 전체 체감 지연을 줄입니다.

레이어	제공자	역할
STT	AssemblyAI	사용자 음성을 텍스트로 변환
Agent	LangChain	텍스트 쿼리를 처리하고 응답 생성
TTS	Cartesia	에이전트의 텍스트 응답을 음성으로 변환

각 레이어가 스트리밍으로 연결되어, 이전 단계의 완전한 출력을 기다리지 않고 부분 결과를 즉시 다음 단계로 전달합니다:

1. STT – 부분 전사 결과를 스트리밍 (발화 완료 감지 시 최종 전사)
2. Agent – 토큰 단위 스트리밍 응답 생성 (`astream()`)
3. TTS – 전체 응답 완료 전에 음성 합성 시작 (WebSocket 기반)

스트리밍이 중요한 이유

스트리밍 없이 동기적으로 처리하면, 각 단계가 완전히 끝나야 다음 단계가 시작됩니다. 예를 들어 에이전트가 200 토큰 응답을 생성하는 데 3초가 걸린다면, TTS는 3초 후에야 시작됩니다. 반면 스트리밍에서는 에이전트의 첫 토큰이 나오는 즉시 TTS가 음성 합성을 시작하므로, 사용자는 에이전트가 아직 응답을 생성하는 동안에도 이미 음성을 듣기 시작합니다.

8.3 아키텍처 비교표

음성 에이전트를 구축하는 두 가지 접근법을 비교합니다.

Sandwich 아키텍처 (STT -\> Agent -\> TTS)

항목	내용
장점	각 컴포넌트 독립 교체 가능, 텍스트 기반 도구 활용 용이, 디버깅 편리 (중간 텍스트 로깅)
단점	3단계 직렬 처리로 레이턴시 누적, 감정/억양 등 비언어 정보 손실
적합 상황	도구 호출이 많은 복잡한 에이전트, 다국어 지원 필요 시

Speech-to-Speech (S2S)

항목	내용
장점	낮은 레이턴시, 음성 특성(감정, 강세) 보존, 자연스러운 대화
단점	도구 호출 통합 어려움, 모델 선택지 제한, 디버깅 어려움
적합 상황	단순 대화형 인터페이스, 감정 인식이 중요한 경우

· NOTE

이 노트북에서는 도구 호출이 가능한 Sandwich 아키텍처에 집중합니다.

아키텍처의 전체 구조를 이해했으니, 각 레이어를 하나씩 구현합니다. 첫 번째는 사용자 음성을 텍스트로 변환하는 STT 단계입니다. 이 단계의 성능이 전체 파이프라인의 시작점을 결정합니다.

8.4 STT 단계 – AssemblyAI 실시간 전사

➡ TIP

이 셀은 AssemblyAI API 키가 필요합니다. 개념 이해를 위한 참조 코드입니다.

AssemblyAI의 `RealtimeTranscriber` 는 Producer-Consumer 패턴으로 동작합니다:

- Producer: 마이크에서 캡처한 오디오 청크를 WebSocket으로 전송
- Consumer: 부분 전사(partial) 및 최종 전사(final) 결과를 콜백으로 수신

두 작업이 동시에 진행되므로, 음성 전송 중에도 이전 발화의 전사 결과를 받을 수 있습니다.

전사 결과의 두 가지 유형

유형	클래스	설명
Partial	<code>RealtimeTranscript</code>	아직 발화 중인 단어들의 임시 전사. 사용자가 말하는 동안 실시간으로 업데이트됨
Final	<code>RealtimeFinalTranscript</code>	발화 종료(엔드포인팅)가 감지된 후의 확정 전사. 이 결과만 에이전트로 전달

AssemblyAI의 내장 VAD(Voice Activity Detection)가 발화 종료 시점을 자동으로 감지하므로, 별도의 무음 감지 로직이 필요하지 않습니다.

```
import assemblyai as aai

aai.settings.api_key = "your-assemblyai-key"

transcriber = aai.RealtimeTranscriber(
    sample_rate=16000,
    encoding=aai.AudioEncoding.pcm_s16le,
    on_data=on_transcription_data,
    on_error=on_transcription_error,
)

def on_transcription_data(transcript: aai.RealtimeTranscript):
    if isinstance(transcript, aai.RealtimeFinalTranscript):
        process_user_input(transcript.text)

def on_transcription_error(error: aai.RealtimeError):
    print(f"Transcription error: {error}")

transcriber.connect()
```

마이크 오디오 캡처

PCM 16-bit, 16kHz 단일 채널 오디오를 캡처하여 실시간으로 전사기에 전송합니다:

```
import pyaudio

audio = pyaudio.PyAudio()
stream = audio.open(
    format=pyaudio.paInt16,
    channels=1, rate=16000,
    input=True, frames_per_buffer=1024,
)

while True:
    data = stream.read(1024)
    transcriber.stream(data)
```

STT 단계에서 텍스트가 생성되면, 이제 에이전트가 이 텍스트를 처리하여 응답을 생성합니다. 에이전트 단계는 Sandwich 아키텍처의 중심부이며, 기존 텍스트 기반 에이전트를 그대로 재사용할 수 있다는 것이 핵심 장점입니다.

8.5 에이전트 단계 – LangChain 에이전트 활용

파이프라인의 핵심인 에이전트 단계입니다.

! WARNING

보이스 에이전트의 시스템 프롬프트에서 응답 길이 제한은 매우 중요합니다. 텍스트 채팅과 달리 음성에서는 긴 응답이 사용자에게 큰 부담입니다. “1 2문장으로 간결하게”, “목록 대신 핵심만”, “마크다운 서식 사용 금지” 등의 지시를 반드시 포함하세요. 또한 괄호, URL, 특수문자 등 음성으로 자연스럽게 읽을 수 없는 표현을 피하도록 지시해야 합니다.

`create_agent` 로 생성한 에이전트는 텍스트 입력을 받아 도구 호출과 추론을 수행한 뒤 텍스트 응답을 스트리밍합니다.

보이스 에이전트용 시스템 프롬프트의 핵심은 간결하고 대화체인 응답을 유도하는 것입니다. 음성 출력은 텍스트와 달리 긴 응답이 사용자에게 부담이 되므로, 1 2문장 이내의 짧은 응답을 생성하도록 지시합니다.

비동기 스트리밍의 역할

에이전트의 `astream()` 메서드는 응답 토큰을 생성되는 즉시 yield합니다. 이것이 보이스 에이전트에서 특히 중요한 이유:

- TTS 초기 시작: 전체 응답 완료를 기다리지 않고 첫 토큰부터 음성 합성 가능
- 체감 지연 감소: 사용자가 에이전트의 응답을 더 빨리 듣기 시작
- 파이프라인 효율성: Agent와 TTS가 동시에 실행되어 총 처리 시간 단축

```
from langchain.agents import create_agent

def search_tool(query: str) -> str:
    """웹에서 최신 정보를 검색합니다."""
    return f"검색 결과: {query}"

def calendar_tool(action: str, details: str) -> str:
    """캘린더 이벤트를 관리합니다."""
    return f"캘린더 {action}: {details}"
```

```
agent = create_agent(
    model="gpt-5.4",
    tools=[search_tool, calendar_tool],
    system_prompt=(
        "당신은 유용한 음성 어시스턴트입니다. "
        "응답을 간결하고 대화체로 유지하세요."
    ),
)
```

비동기 스트리밍 응답 생성

`astream()` 은 에이전트의 응답 토큰을 생성되는 즉시 yield합니다. TTS 단계가 전체 응답 완료를 기다리지 않고 음성 합성을 시작할 수 있게 합니다.

LangChain의 스트리밍은 크게 두 가지 방식을 제공합니다:

메서드	설명
-----	----

<code>astream()</code>	에이전트 실행의 각 스텝별 출력을 스트리밍. 메시지, 도구 호출 결과 등을 포함
<code>astream_events()</code>	더 세분화된 이벤트 단위 스트리밍. 개별 토큰, LLM 시작/종료 등 상세 이벤트 제공

보이스 에이전트에서는 `astream()` 으로 메시지 청크를 수신하고, 각 청크의 `content` 를 추출하여 TTS로 전달합니다.

```
async def stream_agent_response(user_text: str):
    """에이전트 응답 토큰을 하나씩 스트리밍합니다."""
    async for chunk in agent.astream(
        {"messages": [{"role": "user",
                        "content": user_text}]}
    ):
        if "messages" in chunk:
            for msg in chunk["messages"]:
                if hasattr(msg, "content") and msg.content:
                    yield msg.content
```

멀티모달 content blocks 와 `output_version="v1"`

보이스 에이전트는 입력이 텍스트만이 아닐 수 있습니다 — 사용자가 사진을 함께 보내거나, 도구가 이미지 차트를 반환할 수 있습니다. LangChain v1 메시지는 content blocks 배열을 받기 때문에 텍스트·이미지·오디오를 한 메시지에 섞어 보낼 수 있습니다. 또한 `output_version="v1"` 로 모델을 만들면 응답이

`AIMessage.content_blocks` 표준 스키마로 직렬화되어 STT/Agent/TTS 사이에서 안전하게 주고받을 수 있습니다.

```
from langchain.chat_models import init_chat_model

llm_v1 = init_chat_model("gpt-5.4", output_version="v1")

response = llm_v1.invoke([
    {"role": "user", "content": [
        {"type": "text", "text": "이 차트에서 이상 구간을 찾아 음성으로 설명해줘"},
        {"type": "image", "source_type": "base64",
         "mime_type": "image/png", "data": "..."},
    ]},
])

# v1 표준 - content_blocks 로 안전하게 직렬화
for block in response.content_blocks:
    if block["type"] == "text":
        tts_input_queue.put_nowait(block["text"])
```

TIP

`output_version="v1"` 은 응답을 `content_blocks` 표준 스키마로 정규화합니다. WebSocket 으로 다른 프로세스(예: TTS 서버)에 메시지를 넘길 때 JSON 직렬화/역직렬화가 깨지지 않으므로, 보이스 파이프라인처럼 경계를 자주 넘는 시스템에서는 사실상 필수 옵션입니다.

에이전트가 텍스트 응답을 스트리밍하면, 마지막 레이어인 TTS가 이를 음성으로 변환합니다. TTS 단계의 설계 목표는 첫 오디오 출력까지의 시간을 최소화하는 것입니다.

8.6 TTS 단계 – Cartesia 스트리밍 음성 합성

TIP

이 셀은 Cartesia API 키가 필요합니다. 개념 이해를 위한 참조 코드입니다.

Cartesia는 WebSocket 기반 저지연 TTS를 제공합니다. 에이전트의 텍스트 스트림을 받아 부분 텍스트마다 즉시 오디오 청크를 생성합니다.

Cartesia TTS의 동작 방식

1. WebSocket 연결: `AsyncCartesia` 클라이언트로 WebSocket 세션을 열어 지속적인 연결 유지
2. 텍스트 청크 전송: 에이전트가 생성한 토큰/문장 단위로 텍스트를 전송
3. 오디오 청크 수신: 각 텍스트 청크에 대해 즉시 PCM 오디오 바이트를 수신
4. 클라이언트 전달: 수신된 오디오 바이트를 WebSocket으로 클라이언트에 스트리밍

WebSocket 기반이므로 HTTP 요청/응답 오버헤드 없이 양방향 실시간 통신이 가능합니다. `sonic-2` 모델은 자연스러운 음성과 낮은 지연 시간을 동시에 제공합니다.

```
import cartesia

cartesia_client = cartesia.AsyncCartesia(
    api_key="your-cartesia-key"
)

async def text_to_speech_stream(text_stream):
    ws = await cartesia_client.tts.websocket()
    async for text_chunk in text_stream:
        audio_chunks = ws.send(
            model_id="sonic-2",
            transcript=text_chunk,
            voice_id="your-voice-id",
            stream=True,
            output_format={
                "container": "raw",
                "encoding": "pcm_s16le",
                "sample_rate": 24000,
            },
        )
    async for audio in audio_chunks:
        yield audio["audio"]
    await ws.close()
```

세 개의 레이어(STT, Agent, TTS)가 각각 완성되었습니다. 이제 이들을 하나의 스트리밍 파이프라인으로 연결하는 것이 남았습니다. 핵심 과제는 비동기 스트림 간의 데이터 흐름을 자연스럽게 체이닝하는 것입니다.

8.7 파이프라인 조합 – RunnableGenerator

LangChain의 `RunnableGenerator` 를 사용하면 비동기 제너레이터를 Runnable 파이프라인에 통합할 수 있습니다.

TIP

`RunnableGenerator` 는 기존 비동기 제너레이터 함수를 LangChain 생태계에 통합하는 브릿지 역할을 합니다. STT나 TTS와 같이 LangChain 외부의 스트리밍 서비스를 에이전트 파이프라인에 연결할 때 특히 유용합니다.

- | 연산자로 체이닝이 가능해지므로, 복잡한 비동기 로직을 선언적으로 표현할 수 있습니다.

이를 통해 STT 출력 -> Agent 처리 -> TTS 입력이라는 전체 데이터 흐름을 하나의 파이프라인으로 구성할 수 있습니다.

RunnableGenerator란?

`RunnableGenerator` 는 비동기 제너레이터 함수를 LangChain의 `Runnable` 인터페이스로 래핑합니다. 이렇게 하면:

- | (파이프) 연산자로 다른 `Runnable`과 체이닝 가능
- LangChain의 `batch()`, `stream()` 등 표준 메서드 사용 가능
- 입력 스트림을 받아 변환된 출력 스트림을 생성하는 패턴에 적합

```
from langchain_core.runnables import RunnableGenerator

async def transform_input(input_stream):
    async for text in input_stream:
        async for token in stream_agent_response(text):
            yield token

agent_runnable = RunnableGenerator(transform_input)
```

전체 파이프라인 연결 (개념 코드)

`asyncio.Queue` 를 사용하여 STT의 최종 전사 결과를 에이전트로 전달하고, 에이전트의 스트리밍 응답을 TTS로 중계합니다. `Queue` 는 비동기 Producer-Consumer 패턴의 핵심 구성 요소입니다.

```
async def voice_pipeline(audio_input_stream):
    transcript_queue = asyncio.Queue()

    def on_final(transcript):
        if isinstance(transcript, aai.RealtimeFinalTranscript):
            transcript_queue.put_nowait(transcript.text)

    transcriber = aai.RealtimeTranscriber(
        sample_rate=16000, on_data=on_final
    )
    transcriber.connect()

    async for audio_chunk in audio_input_stream:
        transcriber.stream(audio_chunk)
        if not transcript_queue.empty():
            user_text = await transcript_queue.get()
            text_stream = stream_agent_response(user_text)
            async for audio in text_to_speech_stream(text_stream):
                yield audio
```

8.8 WebSocket 서버 – 실시간 양방향 통신

TIP

이 셀의 코드를 실행하면 서버가 시작됩니다. 개념 이해를 위한 참조 코드입니다.

WebSocket을 통해 클라이언트와 양방향 오디오 스트리밍을 처리합니다. `asyncio.gather` 로 수신과 송신을 동시에 실행합니다.

WebSocket을 사용하는 이유

보이스 에이전트는 양방향 실시간 통신이 필수입니다:

- 수신 (Receive): 클라이언트가 마이크 오디오를 서버로 지속적으로 전송
- 송신 (Send): 서버가 합성된 음성 오디오를 클라이언트로 지속적으로 전송

HTTP의 요청-응답 모델은 이런 양방향 스트리밍에 적합하지 않습니다. WebSocket은 단일 TCP 연결 위에서 양방향 데이터 흐름을 지원하며, `asyncio.gather` 를 사용하면 수신과 송신이 서로를 블록하지 않고 동시에 실행됩니다.

```
import websockets
import asyncio

async def handle_client(websocket):
    transcriber = create_transcriber()
    transcriber.connect()

    async def receive_audio():
        async for message in websocket:
            if isinstance(message, bytes):
                transcriber.stream(message)

    async def send_audio():
        async for transcript in transcription_queue:
            text_stream = stream_agent_response(transcript)
            async for audio in text_to_speech_stream(text_stream):
                await websocket.send(audio)

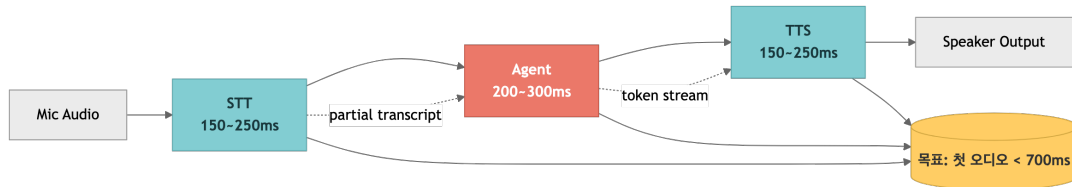
    await asyncio.gather(receive_audio(), send_audio())

async def main():
    async with websockets.serve(handle_client, "0.0.0.0", 8765):
        print("Voice agent server on ws://0.0.0.0:8765")
        await asyncio.Future()
```

8.9 성능 목표 – Sub-700ms 레이턴시

보이스 에이전트의 핵심 성능 지표는 발화 종료부터 첫 오디오 출력까지의 시간(Time to First Audio, TTFA)입니다. 자연스러운 대화 경험을 위해 이 지표가 700ms 미만이어야 합니다.

레이턴시 분석



레이턴시 버짓 예시: STT 180ms + Agent TTFT 320ms + TTS 첫 오디오 140ms = 약 640ms 입니다. 이때 핵심은 각 단계를 개별 최적화하는 것보다, 다음 단계가 이전 단계의 완료 가 아니라 첫 결과를 받아 시작하도록 만드는 것입니다.

단계	목표 시간	설명
STT 최종 전사	200ms	발화 종료 감지 -\> 최종 텍스트 (AssemblyAI 내장 엔드포인팅)
Agent 첫 토큰	300ms	텍스트 입력 -\> 첫 응답 토큰 생성 (TTFT: Time to First Token)
TTS 첫 오디오	150ms	첫 텍스트 토큰 -\> 첫 오디오 청크 (WebSocket 기반)
합계	\<700ms	발화 종료 -\> 첫 오디오 출력

최적화 기법

기법	효과	구현 방법
스트리밍 STT	전체 전사 대기 제거	<code>RealtimeTranscriber</code> + 부분 결과
스트리밍 Agent	전체 응답 완료 전 TTS 시작	<code>astream()</code> 사용 - <code>ainvoke()</code> 대신
스트리밍 TTS	첫 오디오까지 시간 단축	WebSocket 기반 합성
커넥션 풀링	연결 설정 지연 제거	WebSocket 재사용 (요청마다 새 연결 생성 방지)
VAD	무음 구간 처리 방지	AssemblyAI 내장 엔드포인팅
응답 캐싱	빈번한 질문 즉시 응답	자주 묻는 질문 캐싱

TIP

팁: Agent의 TTF(첫 토큰까지 시간)가 전체 레이턴시에서 가장 큰 비중을 차지합니다. 짧은 시스템 프롬프트와 경량 모델 선택이 레이턴시 최적화의 핵심입니다.

8.10 에이전트에 도구 추가

보이스 에이전트의 강점은 Sandwich 아키텍처 덕분에 텍스트 기반 도구를 그대로 활용할 수 있다는 점입니다. 검색, 일정 관리, 날씨 조회 등 다양한 도구를 추가할 수 있습니다.

이것이 S2S(Speech-to-Speech) 방식과의 가장 큰 차별점입니다. S2S 모델은 오디오를 직접 처리하므로 텍스트 기반 도구 호출이 어렵지만, Sandwich 아키텍처는 에이전트가 텍스트 영역에서 동작하므로 기존의 모든 LangChain 도구를 그대로 사용할 수 있습니다.

도구 설계 팁

음성 에이전트의 도구는 빠른 응답이 중요합니다. 도구 실행 시간이 길어지면 전체 파이프라인의 레이턴시가 증가하므로:

- 가벼운 API 호출 위주로 설계
- 타임아웃 설정 필수
- 가능하면 캐싱 적용

```
def weather_tool(city: str) -> str:
    """도시의 현재 날씨를 조회합니다."""
    return f"{city} 날씨: 15도, 구름 조금"

def reminder_tool(time: str, message: str) -> str:
    """지정된 시간에 알람을 설정합니다."""
    return f"{time}에 알람 설정됨: {message}"
```

```
voice_agent = create_agent(
    model="gpt-5.4",
    tools=[search_tool, calendar_tool,
           weather_tool, reminder_tool],
    system_prompt=(
        "당신은 음성 어시스턴트입니다. "
        "1-2문장으로 간결하고 대화체로 응답하세요."
    ),
)
```

요약

주제	핵심 내용
Sandwich 아키텍처	STT -> Agent -> TTS, 각 레이어 독립 교체 가능, 도구 호출 지원
STT (AssemblyAI)	RealtimeTranscriber, Producer-Consumer 패턴, WebSocket 스트리밍

Agent (LangChain)	<code>create_agent</code> + <code>astream()</code> , 토큰 단위 스트리밍, 도구 통합
TTS (Cartesia)	WebSocket 기반 저지연 합성, 부분 텍스트 즉시 오디오 변환
파이프라인 조합	<code>RunnableGenerator</code> , 비동기 제너레이터 체이닝
성능 목표	Sub-700ms (STT 200ms + Agent 300ms + TTS 150ms)
도구 확장	텍스트 기반 도구를 그대로 음성 에이전트에 적용 가능

보이스 에이전트까지 구현할 수 있게 되었습니다. 지금까지 학습한 모든 에이전트 패턴을 실제 서비스로 배포하려면 테스트, 관측성, 배포 인프라가 필요합니다. 마지막 장에서는 에이전트를 프로덕션에 안전하게 배포하기 위한 전체 파이프라인을 다룹니다.

09 프로덕션 배포

- 테스트, 관측성, 배포

Part 5의 마지막 장입니다. 앞선 장들에서 미들웨어, 멀티에이전트, 컨텍스트 엔지니어링, RAG, SQL, 데이터 분석, 보이스 에이전트를 학습했습니다. 이 모든 에이전트를 실제 사용자에게 제공하려면 테스트 → 관측성 → 배포의 프로덕션 파이프라인이 필수입니다. 에이전트를 프로덕션에 배포하기 위한 전체 파이프라인을 다룹니다. 단위 테스트와 LangSmith 평가로 품질을 보장하고, 트레이싱으로 관측성을 확보한 뒤, LangGraph Platform으로 배포합니다.

개발 -> 테스트 (단위 + 평가) -> 관측성 (트레이싱) -> 배포 (LangGraph Platform)

왜 에이전트에 특화된 프로덕션 파이프라인이 필요한가?

에이전트는 전통적인 소프트웨어와 다른 특성을 갖습니다:

- 비결정적 실행: 동일한 입력에도 다른 도구 호출 순서, 다른 응답을 생성할 수 있음
- 상태 유지: 대화 기록과 체크포인트를 관리하는 장기 실행 프로세스
- 다중 컴포넌트: LLM, 도구, 메모리, 체크포인트 등 여러 시스템이 연동

따라서 단순한 단위 테스트를 넘어 궤적(trajecory) 평가, LLM-as-Judge, 트레이스 기반 모니터링 등 에이전트 전용 품질 보장 기법이 필요합니다.

학습 목표

이 노트북을 완료하면 다음을 수행할 수 있습니다:

1. 단위 테스트 - `GenericFakeChatModel` 로 LLM 응답을 모킹하여 결정적 테스트를 작성할 수 있다
2. LangSmith 평가 - 데이터셋 생성, 평가자 정의, 자동화된 에이전트 평가를 수행할 수 있다
3. 트레이스 분석 - LangSmith 트레이싱으로 레이턴시, 토큰 사용량, 에러를 추적할 수 있다
4. LangGraph Studio - 시각적 디버깅 도구로 에이전트 실행 흐름을 분석할 수 있다
5. 배포 옵션 - LangGraph Platform, 셀프호스트, Cloud 배포 간 차이를 이해할 수 있다
6. `langgraph.json` - 배포 설정 파일을 구성하고 배포 명령어를 실행할 수 있다

핵심 질문

이 장의 질문은 “에이전트가 잘 동작한다는 것을 어떻게 증명하고, 운영 중 문제를 어떻게 다시 개발 프로세스로 되돌릴 것인가?” 입니다.

한눈에 보는 핵심

- 에이전트 프로덕션은 개발→평가→트레이싱→배포→운영 피드백의 닫힌 루프로 봐야 합니다.
- 단위 테스트만으로는 부족하고, trajectory 평가와 tracing이 함께 필요합니다.
- 배포는 끝이 아니라 다음 개선 사이클의 시작점입니다.

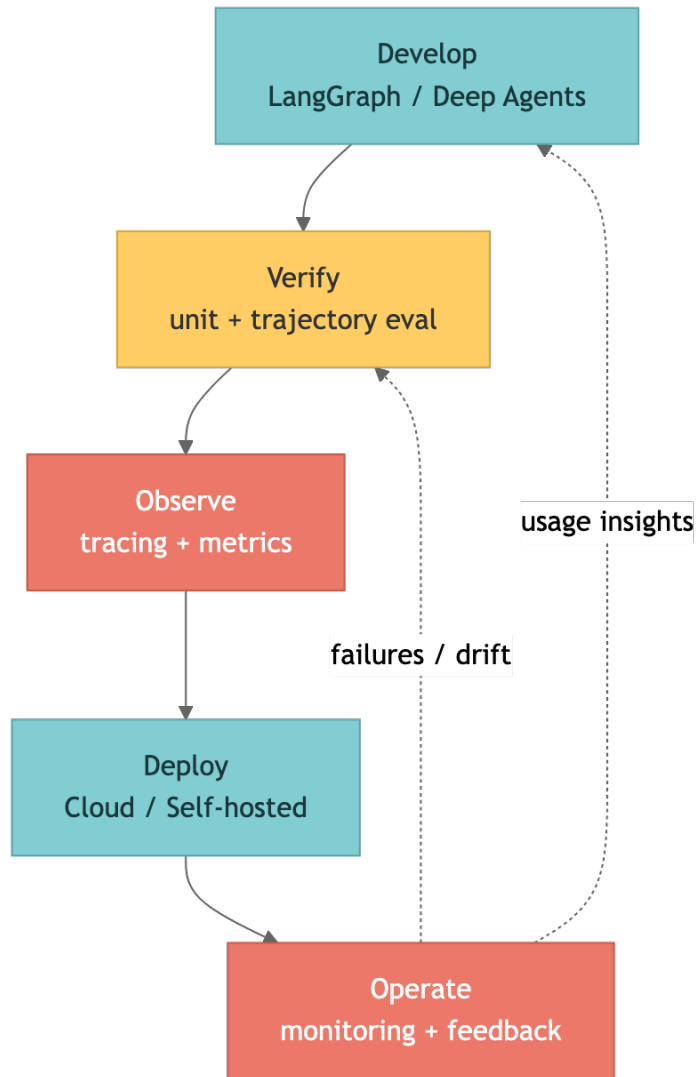
9.1 환경 설정

테스트, 관측성, 배포에 필요한 패키지를 설치합니다. 이 장에서는 `pytest`, `agentevals`, `langsmith`, `langgraph-cli`, `langgraph-sdk` 를 활용합니다. 각 패키지는 프로덕션 파이프라인의 서로 다른 단계를 담당합니다.

에이전트를 실제 서비스로 운영하기 위해서는 “동작한다”는 것만으로는 부족합니다. 얼마나 잘 동작하는지, 언제 문제가 발생하는지, 어떻게 확장할 것인지를 체계적으로 관리해야 합니다. 이것이 프로덕션 파이프라인의 핵심 목적입니다.

9.2 프로덕션 파이프라인 개요

에이전트의 비결정적 특성 때문에 전통적인 소프트웨어 테스트만으로는 품질을 보장할 수 없습니다.



이 흐름도는 프로덕션 운영을 개발 → 검증 → 관측 → 배포 → 운영 의 닫힌 루프로 보여줍니다. 에이전트는 배포 후에도 계속 프롬프트, 도구, 평가셋이 수정되므로, 마지막 운영 단계가 다시 테스트와 개발로 되돌아가는 구조를 의도적으로 가져가야 합니다.

단계	목적	도구
개발	에이전트 구현 및 로컬 테스트	LangGraph, LangGraph Studio
테스트	단위 테스트 + LLM 기반 평가	pytest, agentevals, LangSmith
관측성	트레이싱, 메트릭, 에러 추적	LangSmith Tracing
배포	프로덕션 환경 배포	LangGraph Platform

테스트 전략

에이전트 테스트는 두 가지 접근을 병행해야 합니다:

1. 개발 단계 — 로컬 재현성과 회귀 검증 확보

2. 평가 단계 – 실제 궤적과 답변 품질을 데이터셋으로 측정
3. 추적 단계 – 운영 중 실패 사례와 비용 상승을 트레이스로 발견
4. 배포 단계 – 설정, 체크포인트, 환경 변수까지 함께 검증

유형	특징	적합 대상
Unit	<code>GenericFakeChatModel</code> 로 LLM 응답을 모킹해 격리된 결정적 테스트 수행. API 호출 없이 빠르게 실행	개별 도구, 파서, 프롬프트 포매팅, 상태 변환 로직
Integration	실제 네트워크 호출로 컴포넌트 간 협업 검증. <code>pytest</code> + <code>vcrpy</code> 카세트로 재현성 확보	전체 에이전트 흐름, 도구 호출 시퀀스, 외부 API 연동
Evals	LangSmith 데이터셋과 <code>agentevals</code> 궤적 평가자로 응답 품질·도구 호출 순서를 정량화	응답 정확도, trajectory match, LLM-as-Judge 루브릭

TIP

테스트는 세 종류 모두 필요합니다 – Unit 은 회귀 방지, Integration 은 외부 API 계약 검증, Evals 는 비결정적 응답 품질 측정. 셋 다 CI 에서 돌리되 Evals 만 샘플 데이터셋 으로 자주, 풀 데이터셋 으로 야간에 돌리는 식의 이원화가 일반적입니다.

에이전트 시스템은 다중 컴포넌트가 체이닝되어 동작하므로, 일반적으로 통합 테스트의 비중이 더 높습니다. 단위 테스트로 개별 컴포넌트의 정확성을 확인하고, 통합 테스트로 전체 흐름의 품질을 평가합니다.

단위 테스트로 개별 컴포넌트를 검증한 다음, 전체 에이전트의 행동 패턴을 평가하는 통합 테스트가 필요합니다. LangSmith는 에이전트 전용 평가 인프라를 제공합니다.

단위 테스트가 개별 컴포넌트의 정확성을 검증한다면, LangSmith 평가는 에이전트의 전체 행동 패턴을 평가합니다. 전통적인 소프트웨어에서는 입출력 비교로 충분하지만, 에이전트는 같은 결과에 다른 경로로 도달할 수 있으므로 궤적 기반 평가가 필수적입니다.

9.3 에이전트 테스트 – LangSmith 평가

`agentevals` 패키지는 에이전트 궤적(trajectory) 전용 평가자를 제공합니다.

TIP

LangSmith 평가를 CI/CD 파이프라인에 통합하면, 코드 변경이나 프롬프트 수정 후 자동으로 에이전트 품질을 검증할 수 있습니다. 데이터셋에 다양한 시나리오(정상 케이스, 엣지 케이스, 다국어 입력 등)를 포함하여 회귀 테스트를 구성하세요. 평가 결과의 점수 추이를 모니터링하면 품질 저하를 조기에 감지할 수 있습니다.

궤적이란 에이전트가 최종 응답에 도달하기까지의 모든 단계(도구 호출, 중간 추론, 의사결정)를 의미합니다. 전통적인 입출력 비교 테스트와 달리, 궤적 평가는 에이전트가 올바른 과정을 거쳤는지까지 검증합니다. 궤적이란 에이전트가 최종 응답에 도달하기까지의 모든 단계(도구 호출, 중간 추론, 의사결정)를 의미합니다.

전략	설명	장점	단점
Trajectory Match	기대 시퀀스와 단계별 비교. 미리 정의한 참조 궤적과 에이전트의 실제 궤적을 매칭	정확한 검증, 재현 가능	구체적 기대값 필요, 유연성 낮음
LLM-as-Judge	LLM이 궤적을 루브릭(평가 기준) 기반으로 정성 평가. 도구 사용 적절성, 응답 정확성 등을 자동 판단	유연한 평가, 복잡한 시나리오 대응	추가 LLM 비용, 평가 자체의 비결정성

Trajectory Match 모드

에이전트의 도구 호출 순서에 대한 기대 수준을 4단계로 조절할 수 있습니다:

모드	설명	사용 시점
strict	메시지와 도구 호출 순서가 참조와 완전히 동일해야 통과	순서가 중요한 워크플로 (예: 인증 -\> 조회 -\> 업데이트)
unordered	동일한 도구들을 호출했으면 순서 무관하게 통과	독립적인 도구 호출이 여러 개인 경우
subset	에이전트가 참조 도구만 호출 (추가 도구 호출 없음)	불필요한 도구 호출을 방지하고 싶을 때
superset	에이전트가 참조 도구를 최소한 포함 (추가 호출 허용)	핵심 도구 호출만 보장하고 싶을 때

```
try:
    from langsmith import Client

    ls_client = Client()

    dataset = ls_client.create_dataset("agent-eval-v1")
    ls_client.create_examples(
        inputs=[{"query": "LangGraph란 무엇인가요?"}],
        outputs=[{"expected": "에이전트를 위한 프레임워크"}],
        dataset_id=dataset.id,
    )
    print("데이터셋 생성됨:", dataset.name)
except Exception as e:
    print(f"LangSmith 미설정 (건너뛸): {e}")
    ls_client = None
    dataset = None
```

LangSmith 미설정 (건너뛸): Authentication failed for /datasets. HTTPError('401 Client Error: Unauthorized for url: https://api.smith.langchain.com/datasets', '{"detail": "Invalid token"}')

```
try:
    from agentevals.trajectory import create_trajectory_llm_as_judge
```

```

evaluator = create_trajectory_llm_as_judge(
    rubric=(
        "에이전트가 적절한 도구를 사용했습니까? "
        "최종 답변이 정확했습니까?"
    ),
)
print("평가자 생성됨:", type(evaluator).__name__)
except ImportError:
    print("agentevals 미설치. 설치: pip install agentevals")
    evaluator = None
except Exception as e:
    print(f"평가자 생성 건너뛸 (LLM API 키 필요): {e}")
    evaluator = None

```

평가자 생성 건너뛸 (LLM API 키 필요): create_trajectory_llm_as_judge() got an unexpected keyword argument 'rubric'

LangSmith 평가가 통합 테스트라면, 단위 테스트는 에이전트의 개별 구성 요소를 격리하여 검증합니다. 특히 도구 함수, 상태 변환 로직, 프롬프트 포매팅은 단위 테스트로 빠르게 검증해야 하는 핵심 대상입니다.

9.4 단위 테스트 패턴

`GenericFakeChatModel` 을 사용하면 API 호출 없이 LLM 응답을 모킹하여 결정적 테스트를 작성할 수 있습니다. 응답 인터레이터를 받아 호출마다 하나씩 반환합니다.

! WARNING

`GenericFakeChatModel` 은 도구 호출 응답도 모킹할 수 있지만, 이 경우 `AIMessage`의 `tool_calls` 필드를 올바르게 구성해야 합니다. 도구 호출 형식이 맞지 않으면 에이전트 루프가 정상적으로 동작하지 않으므로, 실제 LLM 응답의 형식을 먼저 확인한 후 모킹 데이터를 작성하세요.

왜 모킹이 필요한가?

에이전트 테스트에서 실제 LLM API를 호출하면 다음 문제가 발생합니다:

- 비결정적 결과: 동일 입력에도 매번 다른 응답이 올 수 있어 테스트 재현이 어려움
- 비용: 테스트를 실행할 때마다 API 비용 발생
- 속도: 네트워크 지연으로 테스트 속도 저하
- 가용성: API 장애 시 테스트 실패

`GenericFakeChatModel` 은 미리 정의한 응답을 순서대로 반환하므로, 결정적이고 빠르며 무료인 테스트를 작성할 수 있습니다. 스트리밍 패턴도 지원하여 `astream()` 기반 코드도 테스트 가능합니다.

상태 지속성 테스트

`InMemorySaver` 체크포인트를 사용하면 여러 대화 턴에 걸친 상태 의존적 행동을 테스트할 수 있습니다.

`thread_id` 를 지정하여 같은 대화 컨텍스트를 유지하면서 여러 호출의 누적 상태를 검증합니다.

```

def search_tool(query: str) -> str:
    """웹에서 정보를 검색합니다."""
    return f"검색 결과: {query}"

```

```
def test_search_tool():
    """검색 도구가 예상 형식을 반환하는지 테스트합니다."""
    result = search_tool("test query")
    assert isinstance(result, str) and len(result) > 0
    print("통과: test_search_tool")

test_search_tool()
```

통과: test_search_tool

HTTP 요청 녹화/재생

CI/CD에서 API 비용을 줄이기 위해 `vcrpy` 와 `pytest-recording` 으로 HTTP 요청을 녹화하고 재생할 수 있습니다. 첫 실행 시 실제 API 호출을 녹화(cassette 파일로 저장)하고, 이후 실행에서는 녹화된 응답을 재생하여 네트워크 호출 없이 테스트합니다.

이 방식의 장점:

- 첫 실행: 실제 API와의 통합을 검증 (통합 테스트 역할)
- 이후 실행: 녹화된 응답으로 빠르고 결정적인 테스트 (단위 테스트 역할)
- CI/CD: API 키 없이도 테스트 실행 가능

```
import pytest

@pytest.fixture(scope="module")
def vcr_config():
    return {"record_mode": "once"}

@pytest.mark.vcr()
def test_agent_with_recorded_responses():
    result = agent.invoke("What is LangGraph?")
    assert "framework" in result.lower()
```

테스트는 배포 전의 품질 보장입니다. 배포 후에는 실제 사용자 트래픽에 대한 지속적 모니터링이 필요합니다. 이를 관측성(Observability)이라 합니다.

테스트는 배포 전의 품질 보장 수단입니다. 하지만 에이전트는 실제 사용자의 다양한 입력에 노출되면 예상치 못한 행동을 보일 수 있습니다. 배포 후 지속적으로 에이전트의 건강 상태를 모니터링하는 것이 관측성(Observability)입니다.

9.5 관측성 – LangSmith 트레이싱

LangSmith 트레이싱은 에이전트 실행의 모든 단계를 기록합니다.

TIP

트레이싱은 환경 변수 2개만으로 활성화되며, 코드 수정이 전혀 필요 없습니다. 프로덕션에서는 항상 트레이싱을 켜두되, `tracing_context` 로 특정 실행만 상세 트레이싱하는 것이 비용과 성능의 균형을 맞추는 좋은 전략입니다. 예를 들어, 정상 요청은 기본 트레이싱만, 오류 발생 시에만 상세 트레이싱을 활성화할 수 있습니다.

트레이스(trace)는 초기 사용자 입력부터 최종 응답까지, 모든 모델 호출, 도구 사용, 의사결정 포인트를 포함하는 에이전트 실행의 완전한 기록입니다.

트레이스에 기록되는 정보

항목	설명
입력/출력	각 단계의 입력 데이터와 출력 결과
모델 호출	프롬프트, 응답, 모델 파라미터
도구 호출	호출된 도구, 인자, 반환값
레이턴시	각 단계별 소요 시간
토큰 사용량	입력/출력 토큰 수
에러	실패한 단계와 에러 메시지

활성화 방법

환경 변수 2개만 설정하면 추가 코드 없이 자동 트레이싱됩니다:

```
export LANGSMITH_TRACING=true
export LANGSMITH_API_KEY=<your-api-key>
```

`create_agent` 로 생성한 에이전트는 환경 변수 설정 시 자동으로 실행 데이터를 LangSmith에 전송합니다. LangChain의 모든 컴포넌트(LLM, 체인, 도구 등)가 내장 계측(instrumentation)을 포함하고 있어 별도의 코드 수정이 필요 없습니다.

선택적 트레이싱 & PII 익명화 (`tracing_context` + `LangChainTracer`)

`tracing_context` 는 특정 코드 블록만 선택적으로 트레이싱하거나, 프로젝트·태그·메타데이터를 런타임에 동적으로 바꿔 끼울 수 있습니다. 프로덕션에서는 보통 `LangChainTracer` 를 직접 인스턴스화해서 `anonymizer` 콜백을 끼워 넣어, 트레이스로 전송되기 전 단계에서 PII(이메일·전화·주민번호 등)를 마스킹합니다.

```
import re
from langsmith import tracing_context
from langchain.callbacks.tracers import LangChainTracer

EMAIL_RE = re.compile(r"[\w.+-]+@[\w-]+\.[\w.-]+")
PHONE_RE = re.compile(r"\b\d{2,3}-\d{3,4}-\d{4}\b")

def anonymize(payload: dict) -> dict:
    """트레이스 전송 전에 PII를 마스킹합니다."""
    text = str(payload)
    text = EMAIL_RE.sub("<email>", text)
    text = PHONE_RE.sub("<phone>", text)
    return {**payload, "redacted": text}

tracer = LangChainTracer(
    project_name="production-agent",
    client_kwargs={"hide_inputs": anonymize,
                  "hide_outputs": anonymize},
)

with tracing_context(
    project_name="production-agent",
    tags=["v2.1", "experiment-A"],
    metadata={"user_tier": "premium"},
):
    agent.invoke({"messages": [...]}, config={"callbacks": [tracer]})
```

! WARNING

PII 마스킹은 트레이스가 LangSmith 로 전송되기 전 의 단계에서 처리해야 합니다. 모델에는 원본을 그대로 전달하고, 외부에 기록되는 트레이스에만 마스킹된 사본이 남도록 분리하는 것이 핵심입니다.

9.6 트레이스 분석

LangSmith 대시보드에서 트레이스를 분석하여 프로덕션 에이전트의 품질을 모니터링합니다.

메트릭	설명	확인 포인트
Latency	각 단계별 소요 시간	병목 구간 식별 (어느 노드가 가장 느린지)
Token Usage	입력/출력 토큰 수	비용 최적화 (프롬프트 길이 조절, 불필요한 컨텍스트 제거)
Error Rate	실패한 실행 비율	안정성 모니터링 (특정 도구의 실패율, LLM 타임아웃 등)
Tool Call Frequency	도구별 호출 빈도	에이전트 행동 패턴 분석 (과도한 도구 호출, 미사용 도구 식별)

태그와 메타데이터 활용

`config` 파라미터나 `tracing_context` 로 커스텀 태그와 메타데이터를 추가하여 트레이스를 분류하고 필터링할 수 있습니다. 프로덕션 환경에서 유용한 태그/메타데이터 예시:

태그/메타데이터	용도
버전 태그 (<code>v2.1</code>)	A/B 테스트, 버전별 성능 비교
실험 태그 (<code>experiment-A</code>)	프롬프트 변경 등 실험 추적
사용자 티어 (<code>premium</code>)	사용자 그룹별 품질 모니터링
리전 (<code>kr</code>)	지역별 레이턴시 분석

프로젝트 관리

프로젝트는 두 가지 방식으로 설정할 수 있습니다:

- 정적: `LANGSMITH_PROJECT` 환경 변수로 기본 프로젝트 지정
- 동적: `tracing_context(project_name=...)` 으로 코드 블록별 프로젝트 분리

```
try:
    from langsmith import tracing_context

    with tracing_context(
        project_name="production-agent",
        tags=["v2.1", "experiment-A"],
        metadata={"user_tier": "premium", "region": "kr"},
    ):
```



```
print("태그된 트레이싱 활성화됨")
except Exception as e:
    print(f"LangSmith 트레이싱 사용 불가: {e}")
```

태그된 트레이싱 활성화됨

9.7 LangGraph Studio – 시각적 디버깅

LangGraph Studio는 에이전트의 실행 흐름을 시각적으로 디버깅할 수 있는 무료 도구입니다. 로컬 머신에서 에이전트를 개발하고 테스트하는 데 최적화되어 있습니다.

기능	설명
그래프 시각화	에이전트의 노드/엣지 구조를 실시간 확인. 현재 실행 중인 노드를 하이라이트
단계별 실행	각 노드의 입출력 데이터를 검사하며 디버깅. 프롬프트, 도구 호출, 결과를 단계별로 확인
상태 검사 & 편집	메시지 히스토리·체크포인트를 탐색하고, 노드 입력값을 즉석에서 수정 한 뒤 재실행
Time Travel	이전 체크포인트로 되돌아가 다른 분기/도구 호출을 시도하는 분기 디버깅
HITL 인터럽트 처리	<code>interrupt()</code> 로 멈춘 그래프에 Studio UI 에서 직접 <code>Command(resume=...)</code> 입력
스레드 관리	<code>thread_id</code> 별 대화 트리를 좌측 패널에 표시, 과거 실행 비교 가능
실시간 스트리밍	에이전트 실행 과정을 실시간으로 관찰. 토큰/레이턴시 메트릭 제공

로컬 개발 서버 설정

Studio를 사용하려면 LangGraph CLI로 로컬 개발 서버를 시작합니다:

```
# LangGraph CLI 설치 (Python 3.11+ 필요)
pip install --upgrade "langgraph-cli[inmem]"

# 개발 서버 시작
langgraph dev
```

서버가 시작되면 <https://smith.langchain.com/studio/?baseUrl=http://127.0.0.1:2024> 에서 Studio UI에 접근할 수 있습니다.

Studio에서 확인 가능한 정보

- 에이전트에게 전송되는 프롬프트
- 각 도구 호출과 그 결과
- 최종 출력
- 중간 상태 (검사 및 수정 가능)
- 토큰 사용량과 레이턴시 메트릭

채팅 UI 스캐폴드 - `npx create-agent-chat-app`

Studio가 개발자의 디버깅 도구라면, 최종 사용자를 위한 채팅 UI는 별도의 프론트엔드가 필요합니다. LangChain 팀이 제공하는 `create-agent-chat-app` 템플릿은 배포된 LangGraph 에이전트에 곧바로 연결되는 Next.js 채팅 앱을 한 줄로 생성합니다.

```
npx create-agent-chat-app@latest my-agent-ui
cd my-agent-ui
# .env.local 에 NEXT_PUBLIC_API_URL, NEXT_PUBLIC_ASSISTANT_ID, LANGSMITH_API_KEY 설정
pnpm dev
```

이 템플릿은 `/runs/stream` SSE 스트리밍, `thread_id` 기반 대화 이력, `interrupt()` 인터럽트 UI 까지 기본 탑재하고 있어 배포된 그래프의 외피로 빠르게 활용할 수 있습니다.

테스트와 관측성이 준비되었다면, 마지막 단계는 실제 배포입니다. 에이전트 배포는 전통적인 웹 애플리케이션 배포와 다른 고려사항이 있습니다.

9.8 배포 옵션

에이전트는 상태를 유지하는 장기 실행 프로세스이므로 일반 웹앱 호스팅과 다른 접근이 필요합니다.

TIP

배포 직전 체크포인트는 4가지만 확인하면 됩니다: 1) 상태 저장소가 실제 영속적인가, 2) 트레이싱이 켜져 있는가, 3) 재시도/타임아웃 정책이 정의되었는가, 4) 배포 후 동일한 평가셋으로 스모크 테스트를 다시 돌렸는가.

체크포인트를 통한 상태 영속화, 백그라운드에서의 장시간 실행, WebSocket 기반 스트리밍 등은 전통적인 요청-응답 모델의 웹 호스팅에서는 지원이 어렵습니다. LangGraph Platform은 이러한 에이전트 특유의 요구사항을 처음부터 고려하여 설계된 배포 인프라입니다.

WARNING

LangGraph Cloud에 배포할 때 `.env` 파일에 포함된 API 키는 Cloud 환경의 환경 변수로 별도로 설정해야 합니다. `langgraph.json`의 `env` 필드는 로컬 개발 시 사용되며, 클라우드 배포 시에는 LangSmith 대시보드의 환경 변수 설정에서 관리합니다. API 키를 GitHub 리포지토리에 커밋하지 않도록 주의하세요.

전통적인 stateless 웹 애플리케이션 호스팅(예: Vercel, Heroku)은 에이전트의 지속적 상태 관리, 백그라운드 실행, 체크포인트링에 적합하지 않습니다.

옵션	설명	적합 대상
LangGraph Cloud	LangSmith 관리형 호스팅. GitHub 연결만으로 자동 빌드/배포	빠른 프로토타이핑, 소규모 팀
Self-Hosted	자체 인프라에서 Docker 컨테이너로 실행	데이터 주권, 엔터프라이즈, 규제 환경

Hybrid	클라우드 관리 + 자체 런타임	관리 편의성 + 데이터 제어 양립
--------	------------------	--------------------

배포 요구사항

항목	설명
GitHub 저장소	코드 호스팅 (공개/비공개 모두 지원)
LangSmith 계정	무료 가입 가능 (smith.langchain.com)
langgraph.json	의존성, 그래프, 환경 변수를 정의하는 배포 설정 파일

LangGraph Cloud 배포 프로세스

1. LangSmith에 로그인 후 Deployments 페이지로 이동
2. "+ New Deployment" 클릭
3. GitHub 계정 연결 (비공개 저장소의 경우)
4. 저장소 선택 후 제출 (약 15분 소요)
5. 배포 완료 후 API URL 복사하여 클라이언트에서 사용

9.9 langgraph.json 설정

langgraph.json 은 배포의 핵심 설정 파일입니다. 의존성, 그래프 엔트리포인트, 환경 변수를 정의합니다. 이 파일이 프로젝트 루트에 있어야 LangGraph CLI와 Cloud 배포가 정상 동작합니다.

```
import json

langgraph_config = {
    "dependencies": [".",],
    "graphs": {"agent": "./src/agent.py:graph"},
    "env": ".env",
}

print(json.dumps(langgraph_config, indent=2))
```

```
{
  "dependencies": [
    "."
  ],
  "graphs": {
    "agent": "./src/agent.py:graph"
  },
  "env": ".env"
}
```

설정 항목 상세

항목	타입	설명
dependencies	list[str]	Python 패키지 의존성. . 은 현재 디렉터리의 pyproject.toml 을 참조
graphs	dict	그래프 이름과 모듈 경로 매핑. "모듈경로:변수명" 형식

env	str	환경 변수 파일 경로 (<code>.env</code> 형식). API 키 등 민감 정보 포함
-----	-----	---

그래프 엔트리포인트

`graphs` 필드의 값은 `"./경로/파일.py:변수명"` 형식입니다. 변수는 `CompiledGraph` 인스턴스여야 합니다. LangGraph의 Graph API(`StateGraph`)와 Functional API(`@entrypoint`) 모두 사용 가능합니다.

Graph API 예시

```
# src/agent.py
from langgraph.prebuilt import create_react_agent

graph = create_react_agent(
    model="claude-sonnet-4-6",
    tools=[search_tool],
    checkpointers=True,
)
```

Functional API 예시

LangGraph의 Functional API를 사용하면 기존 Python 코드에 최소한의 변경으로 연속성, 메모리, 스트리밍을 통합할 수 있습니다. `@entrypoint` 데코레이터가 워크플로의 시작점을 정의하고, `@task` 데코레이터가 개별 작업 단위를 나타냅니다.

```
# src/agent.py
from langgraph.func import entrypoint, task

@task
def process_query(query: str) -> str:
    return f"처리 완료: {query}"

@entrypoint(checkpointer=checkpointer)
def graph(inputs: dict) -> str:
    result = process_query(inputs["query"])
    return result.result()
```

Functional API의 핵심 특징:

- 표준 Python 제어 흐름 사용 (if/for 등) - 명시적 그래프 구조 불필요
- 함수 스코프 상태 관리 - 별도의 State 선언이나 reducer 설정 불필요
- 태스크 결과 체크포인트링 - 재실행 시 이전에 완료된 태스크 결과를 자동 재사용
- 입출력 JSON 직렬화 필수 - 체크포인트 사용 시 모든 데이터가 직렬화 가능해야 함

9.10 배포 명령어

LangGraph CLI를 사용한 빌드, 로컬 서버, 클라우드 배포 명령어입니다.

1. Docker 이미지 빌드

`langgraph.json` 설정을 기반으로 Docker 이미지를 생성합니다:

```
langgraph build -t my-agent:latest
```

2. 로컬 서버 실행

로컬에서 에이전트를 실행하여 테스트합니다. Studio UI와 연동하여 시각적 디버깅이 가능합니다:

```
# 프로덕션 모드 (Docker 기반)
langgraph up --config langgraph.json

# 개발 모드 (인메모리, 빠른 시작)
langgraph dev
```

3. 클라우드 배포

LangSmith 대시보드에서:

1. Deployments -> “+ New Deployment”
2. GitHub 저장소 연결
3. 리포지토리 선택 후 제출 (약 15분 소요)
4. 배포 완료 후 API URL 복사

Python SDK로 배포된 에이전트 접근

langgraph-sdk 를 사용하면 배포된 에이전트와 프로그래밍 방식으로 통신할 수 있습니다. 스트리밍, 스레드 관리, 상태 조회 등 모든 기능을 SDK로 제어합니다.

```
try:
    from langgraph_sdk import get_sync_client

    api_key = os.environ.get("LANGSMITH_API_KEY", "")
    if not api_key:
        print("LANGSMITH_API_KEY 미설정. 클라이언트 연결 건너뛸.")
    else:
        client = get_sync_client(
            url="https://your-deployment.langsmith.com",
            api_key=api_key,
        )
        print("클라이언트 연결됨:", type(client).__name__)
except Exception as e:
    print(f"LangGraph SDK 클라이언트 사용 불가: {e}")
```

LANGSMITH_API_KEY 미설정. 클라이언트 연결 건너뛸.

client.runs.stream(...) **으로 스트리밍 호출**

SDK 의 runs.stream(...) 는 배포된 에이전트로부터 토큰·메시지·도구 호출을 Server-Sent Events 로 받습니다. stream_mode 는 "values", "updates", "messages", "events" 를 조합할 수 있으며, thread_id 를 함께 넘기면 동일 세션의 대화 이력이 유지됩니다.

```
from langgraph_sdk import get_client

client = get_client(url="https://your-deployment.langsmith.com",
                    api_key=api_key)

async for chunk in client.runs.stream(
    thread_id="user-42",
    assistant_id="agent",
    input={"messages": [
        {"role": "user", "content": "최근 5분 매출 요약"}
    ]},
    stream_mode=["updates", "messages"],
```

```
    ):
        print(chunk.event, chunk.data)
```

REST API로 접근

배포된 에이전트는 REST API로도 접근할 수 있습니다. 이를 통해 어떤 프로그래밍 언어/프레임워크에서도 에이전트와 통신 가능합니다:

```
curl --request POST \
  --url <DEPLOYMENT_URL>/runs/stream \
  --header 'Content-Type: application/json' \
  --header 'X-API-Key: <LANGSMITH_API_KEY>' \
  --data '{
    "assistant_id": "agent",
    "input": {
      "messages": [
        { "role": "user", "content": "안녕하세요!" }
      ]
    },
    "stream_mode": "updates"
  }'
```

주요 엔드포인트:

엔드포인트	메서드	설명
/runs/stream	POST	스트리밍 실행
/runs	POST	동기 실행
/threads	POST	새 스레드 생성
/threads/{id}/state	GET	스레드 상태 조회

요약

주제	핵심 내용
테스트 전략	단위 테스트(GenericFakeChatModel) + 통합 테스트(agentevals) 병행
LangSmith 평가	Trajectory Match (strict/unordered/subset/superset), LLM-as-Judge
관측성	LANGSMITH_TRACING=true + LANGSMITH_API_KEY 로 자동 트레이싱
트레이스 분석	Latency, Token Usage, Error Rate, Tool Call Frequency
LangGraph Studio	시각적 그래프 디버깅, 단계별 상태 검사
배포 옵션	Cloud (관리형), Self-Hosted (Docker), Hybrid
langgraph.json	dependencies , graphs , env 3가지 핵심 설정

배포 명령어	<code>langgraph build -\> langgraph up -\> LangSmith Deploy</code>
--------	--

Part 5를 통해 v1 마이그레이션부터 프로덕션 배포까지, 에이전트 개발의 전체 라이프사이클을 학습했습니다. Part 6에서는 이 모든 지식을 종합하여 RAG, SQL, 데이터 분석, 머신러닝, 딥 리서치 에이전트를 실전 프로젝트로 구현합니다.

P A R T

VI

LangSmith

Tracing · Evaluation · Monitoring

이 파트에서 다루는 내용

- 1. Quickstart: 첫 트레이스
- 2. 에이전트 트레이스 구조
- 3. 데이터셋과 평가 루프
- 4. Prompt Hub 버전 관리
- 5. 프로덕션 모니터링

01 LangSmith Quickstart

첫 트레이스부터 UI 투어까지

LangSmith는 LangChain 팀이 만든 LLM 애플리케이션 관측 플랫폼입니다. 에이전트 코드를 고칠 필요 없이 환경변수 세 줄만 추가하면 모든 LLM 호출과 도구 실행이 자동으로 트레이스로 기록됩니다. 이 장은 API 키 발급부터 UI에서 첫 트레이스를 확인하고, `@traceable` 데코레이터와 `langsmith.Client` 로 순수 Python 함수까지 트레이스에 편입하는 흐름을 다룹니다.

학습 목표

LEARNING OBJECTIVES

- LangSmith API 키를 발급하고 `.env` 에 주입한다
- `LANGSMITH_TRACING=true` 설정만으로 기존 에이전트가 자동 트레이싱되는 것을 확인한다
- `run_name` · `tags` · `metadata` 로 트레이스를 검색 가능하게 만든다
- UI에서 트레이스 트리, latency, 토큰 사용량, 비용을 읽는다
- `@traceable` 로 일반 함수를 트레이스에 편입한다

1.1 API 키와 환경 변수

LangSmith는 코드 변경 없이 환경 변수 세 줄만으로 트레이싱을 활성화합니다. `.env` 파일에 다음을 추가합니다.

```
LANGSMITH_API_KEY=lsv2_pt_XXXXXXX
LANGSMITH_TRACING=true
LANGSMITH_PROJECT=langsmith-quickstart
```

`LANGSMITH_PROJECT` 는 UI 좌측 Projects 메뉴에서 구분되는 네임스페이스입니다. 설정하지 않으면 모든 트레이스가 `default` 프로젝트에 기록됩니다. API 키 발급은 한 번만 표시되므로, 생성 직후 바로 `.env` 에 복사해야 합니다.

`langsmith ≥ 0.3` 패키지만 추가하면 `langchain` · `langgraph` · `deepagents` 의 자동 계측이 모두 이 환경 변수를 공유합니다.

```
pip install -U langsmith
```

온보딩 플로우 (최초 1회)

최초 로그인 시 역할 선택 → 모드 선택 → 홈 → 퀵스타트 다이얼로그 순서로 4단계 온보딩을 거칩니다.
Technical 역할과 LangSmith(code-first) 모드를 선택해 개발자용 흐름으로 진입합니다.

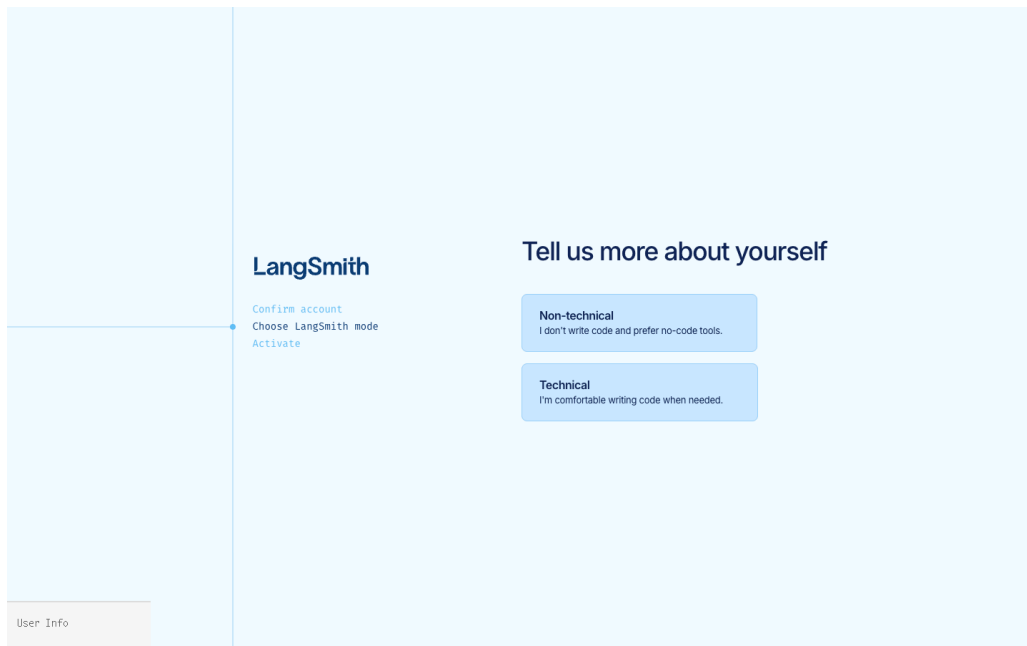


그림 1: 역할 선택 — Technical 선택 시 개발자용 code-first 흐름으로 진입

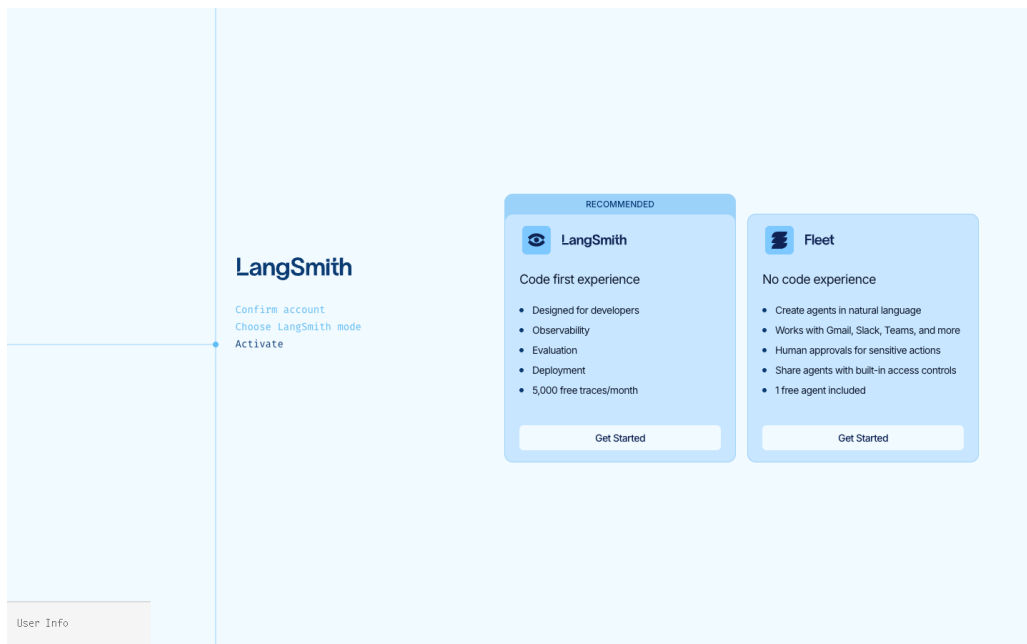


그림 2: LangSmith(code-first) vs Fleet(no-code) 분기 — 이 장은 LangSmith 경로

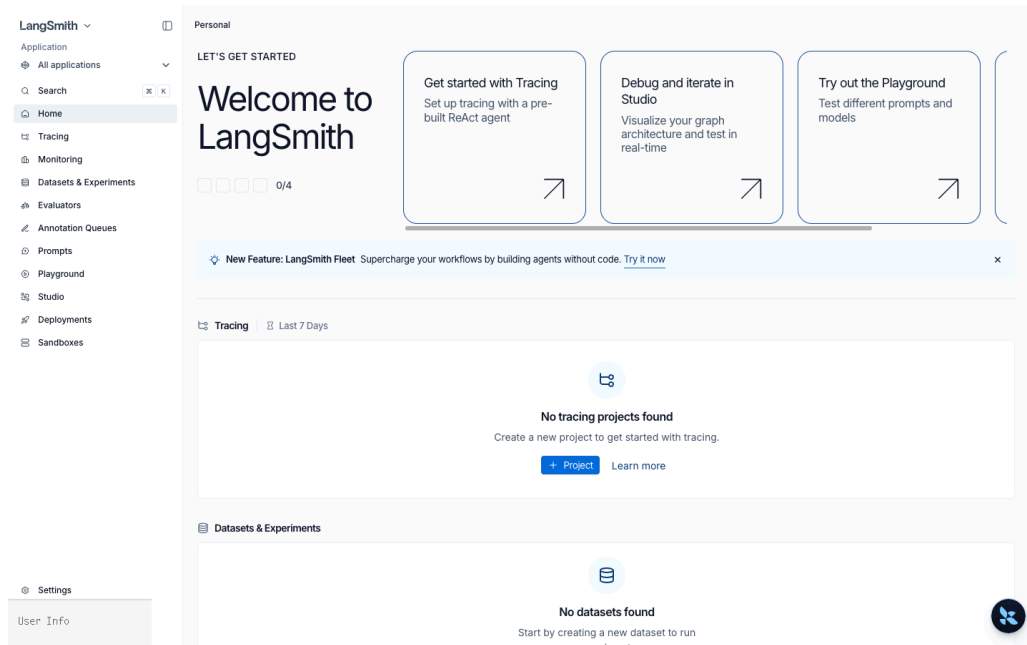


그림 3: 온보딩 완료 직후 Home – Tracing/Datasets/Prompts 모두 초기 상태

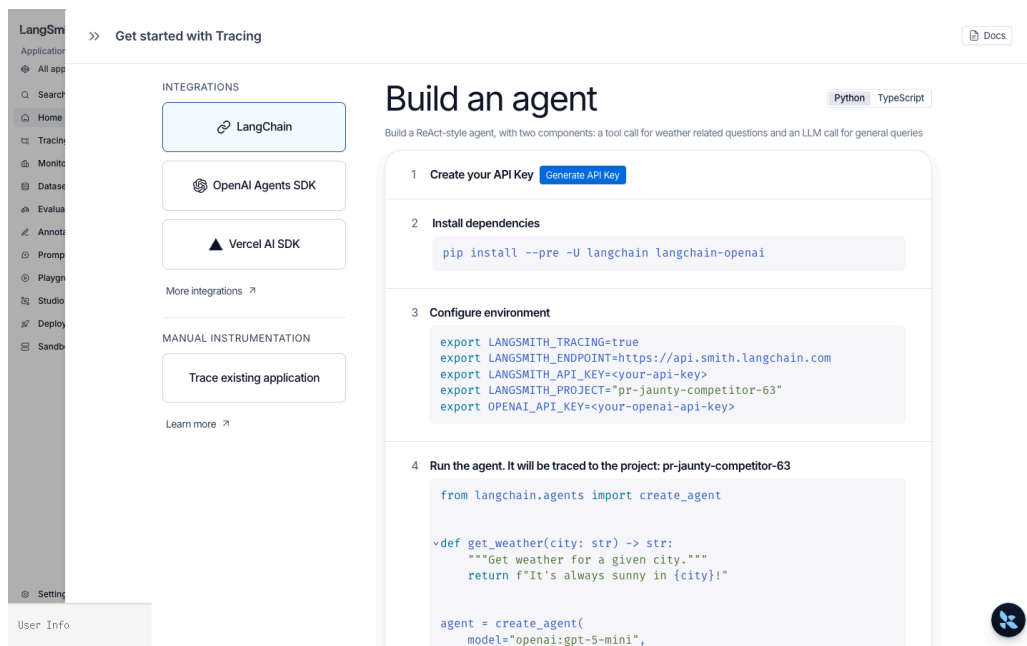


그림 4: Home의 첫 카드로 열리는 4단계 퀵스타트 다이얼로그

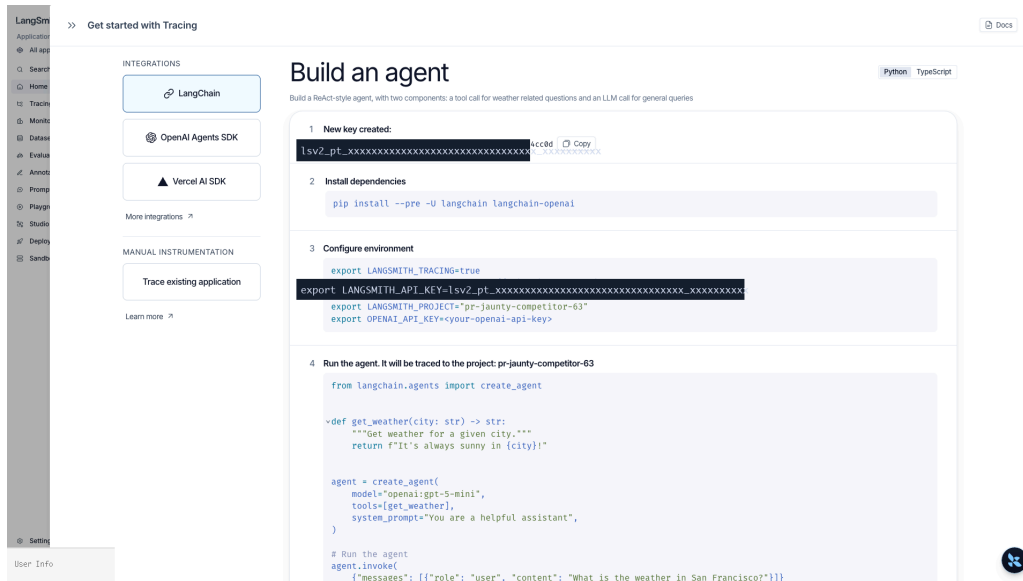


그림 5: Generate API Key 클릭 시 lsv2_pt_... 키 생성 — 한 번만 표시되므로 즉시 .env 로 복사

1.2 첫 트레이스 — LangChain 에이전트

`create_agent` 는 자동 계측되어 있습니다. 환경변수만 설정되어 있으면 아래 코드가 실행되는 순간 LangSmith 에 기록됩니다.

```
from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain.tools import tool

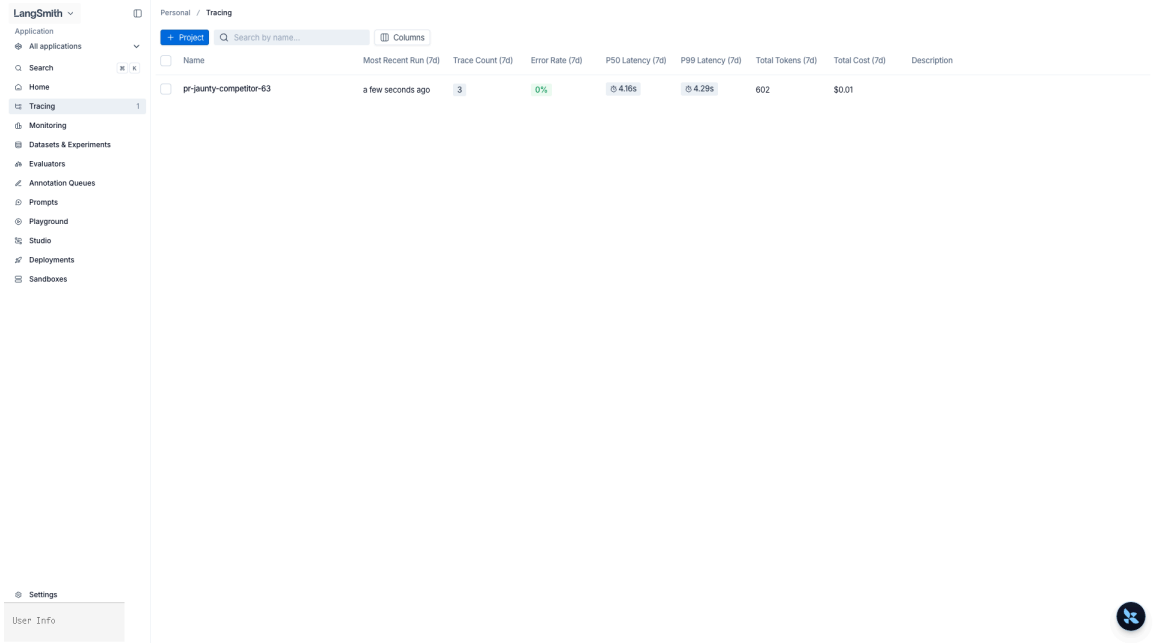
load_dotenv()

@tool
def get_weather(city: str) -> str:
    """도시의 날씨를 조회한다."""
    return f"{city} 맑음"

agent = create_agent(
    model="openai:gpt-5.4",
    tools=[get_weather],
)

agent.invoke({"messages": [{"role": "user", "content": "서울 날씨 알려줘"}]})
```

실행 후 UI의 Projects → langsmith-quickstart 에서 방금 실행이 한 행으로 보이고, 클릭하면 LLM 호출 → 도구 호출 → LLM 호출 트리가 시각적으로 재구성됩니다.



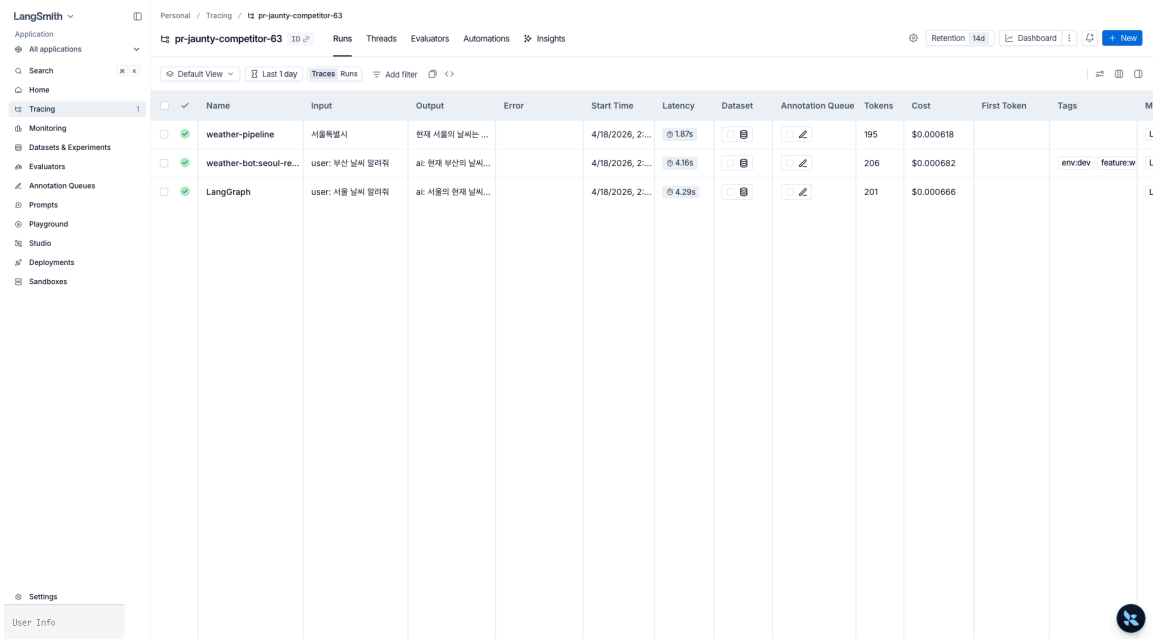
Personal / Tracing

+ Project Search by name... Columns

Name	Most Recent Run (7d)	Trace Count (7d)	Error Rate (7d)	P50 Latency (7d)	P99 Latency (7d)	Total Tokens (7d)	Total Cost (7d)	Description
pr-jaunty-competitor-63	a few seconds ago	3	0%	4.16s	4.29s	602	\$0.01	

Settings User Info

그림 6: Projects 리스트 – Trace Count, P50/P99 Latency, Total Tokens, Total Cost 자동 집계



Personal / Tracing / pr-jaunty-competitor-63

pr-jaunty-competitor-63 ID Runs Threads Evaluators Automations Insights

Retention 14d Dashboard New

Default View Last 1 day Traces Runs Add filter

	Name	Input	Output	Error	Start Time	Latency	Dataset	Annotation Queue	Tokens	Cost	First Token	Tags	Metadata
☐	weather-pipeline	서울특별시	현재 서울의 날씨는 ...		4/18/2026, 2...	1.87s			195	\$0.000618			
☐	weather-bot-seoul-re...	user: 부산 날씨 알려줘	ai: 현재 부산의 날씨...		4/18/2026, 2...	4.16s			206	\$0.000682		env:dev feature:w	
☐	LangGraph	user: 서울 날씨 알려줘	ai: 서울의 현재 날씨...		4/18/2026, 2...	4.29s			201	\$0.000666			

Settings User Info

그림 7: 프로젝트 상세의 Run 테이블 – Input/Output/Latency/Tokens/Cost/Tags/Metadata 한 화면

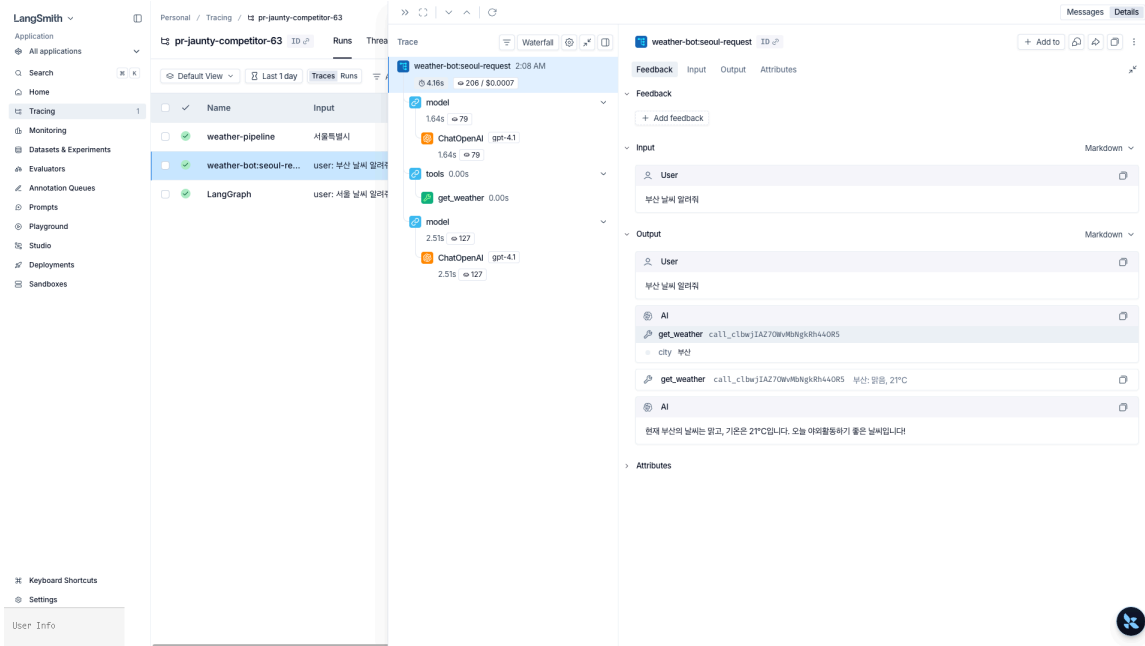


그림 8: Trace Tree — model → tools → model 순서와 각 단계의 토큰/지연이 waterfall로 표시

1.3 run_name · tags · metadata

`invoke(..., config=...)` 로 트레이스에 검색·필터용 메타 정보를 부착합니다. 운영에서 “이 사용자가 이 세션에서 낸 실패만 보겠다” 같은 질의를 쓸 수 있게 해 주는 핵심 장치입니다.

```
agent.invoke(
  {"messages": [{"role": "user", "content": "부산 날씨"}]},
  config={
    "run_name": "weather-query-demo",
    "tags": ["env:dev", "feature:weather"],
    "metadata": {
      "user_id": "u_00123",
      "session_id": "s_demo",
      "app_version": "0.1.0",
    },
  },
)
```

UI에서 `Filter → Tags contains env:dev` 또는 `Metadata.user_id = u_00123` 으로 필터링되는 것을 확인합니다.

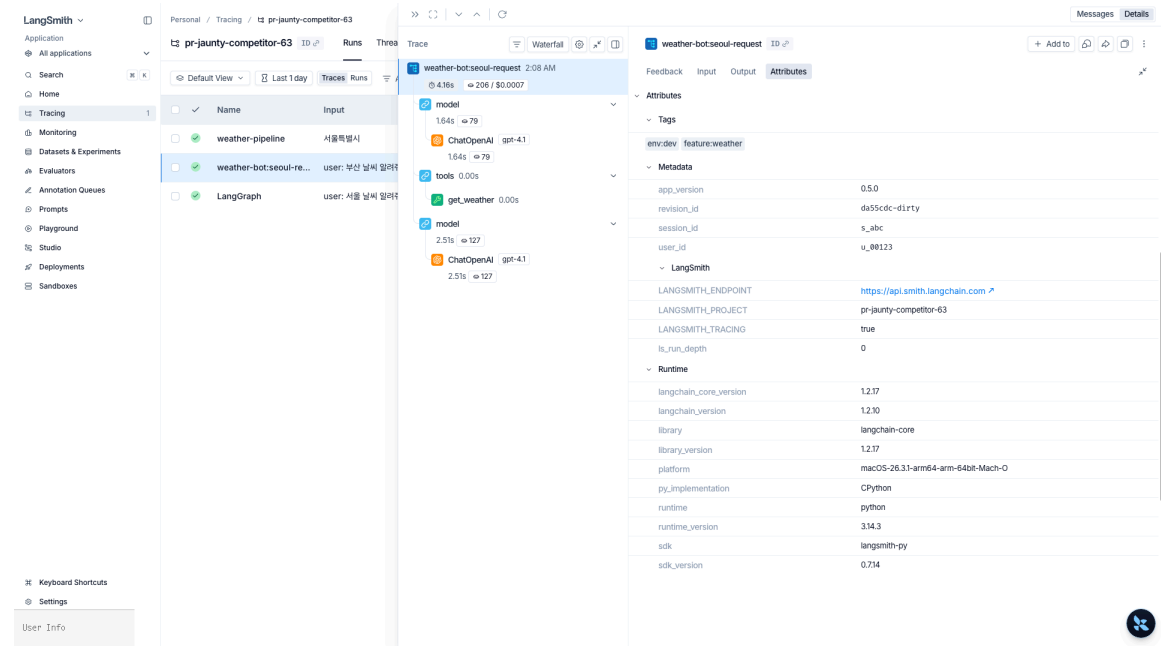


그림 9: Attributes 탭 – 부착한 Tags/Metadata가 그대로 보이고 환경·런타임 정보도 자동 캡처

1.4 순수 Python 함수 트레이싱 — @traceable

LangChain을 거치지 않는 전처리/후처리 유틸리티 함수도 `@traceable` 로 트레이싱에 편입합니다. 부모 트레이스 아래 자식 span으로 묶이므로 “왜 이 도시로 쿼리를 보냈는가” 같은 추적이 됩니다.

```
from langsmith import traceable

@traceable(run_type="chain", name="normalize_city")
def normalize_city(raw: str) -> str:
    return raw.strip().replace("시", "")

@traceable(run_type="chain", name="weather-pipeline")
def run_weather(user_text: str) -> str:
    city = normalize_city(user_text)
    result = agent.invoke(
        {"messages": [{"role": "user", "content": f"{city} 날씨"}]},
    )
    return result["messages"][-1].content

run_weather("부산시")
```

UI에서 `weather-pipeline` 이 루트, 그 아래 `normalize_city` 와 `AgentExecutor` 가 자식으로 묶인 하나의 트리로 보입니다.

`tracing_context` 로 환경 변수 없이 켜기

환경 변수 없이 일시적으로 트레이싱을 켜고 끄려면 `langsmith as ls` 의 `tracing_context` 를 컨텍스트 매니저로 씁니다. 노트북·테스트 코드에서 특정 블록만 기록하고 싶을 때 유용합니다.

```
import langsmith as ls

with ls.tracing_context(enabled=True, project_name="langsmith-quickstart"):
    agent.invoke({"messages": [{"role": "user", "content": "서울 날씨"}]})
```

`enabled=False` 로 호출하면 전역 설정이 켜져 있어도 해당 블록만 트레이스가 비활성화됩니다 — 민감 데이터 처리 블록을 격리할 때 씁니다.

1.5 트레이스를 코드에서 다시 조회

`langsmith.Client` 로 UI 없이도 트레이스를 프로그램적으로 꺼냅니다. 회귀 테스트 입력, 야간 배치 리포트 원천, 데이터셋 시드로 씁니다.

```
from langsmith import Client

client = Client()

runs = list(
    client.list_runs(
        project_name="langsmith-quickstart",
        filter='and(has(tags, "env:dev"), eq(is_root, true))',
        limit=10,
    )
)

for r in runs:
    print(r.id, r.name, r.total_tokens, r.total_cost)
```

`filter` 표현식은 UI의 `Add filter` 가 만들어내는 것과 동일한 DSL을 그대로 씁니다.

1.6 비용 · 토큰 집계

UI 프로젝트 페이지 우상단의 Analytics 탭에서 프로젝트 단위 총 비용/토큰 시계열을 봅니다. 모델별 단가는 LangSmith가 자동으로 계산해 `total_cost` 필드에 채우고, 자체 호스팅 모델이라면 Settings → Model pricing 에서 단가를 직접 지정할 수 있습니다.

1.7 API 키 재발급

온보딩 이후 키를 재발급하거나 revoke하려면 Settings → Access and Security → API Keys 로 이동합니다. 기존 키는 Last Used At이 자동으로 기록되므로, 유출이 의심되면 즉시 revoke하고 새 키를 발급합니다.

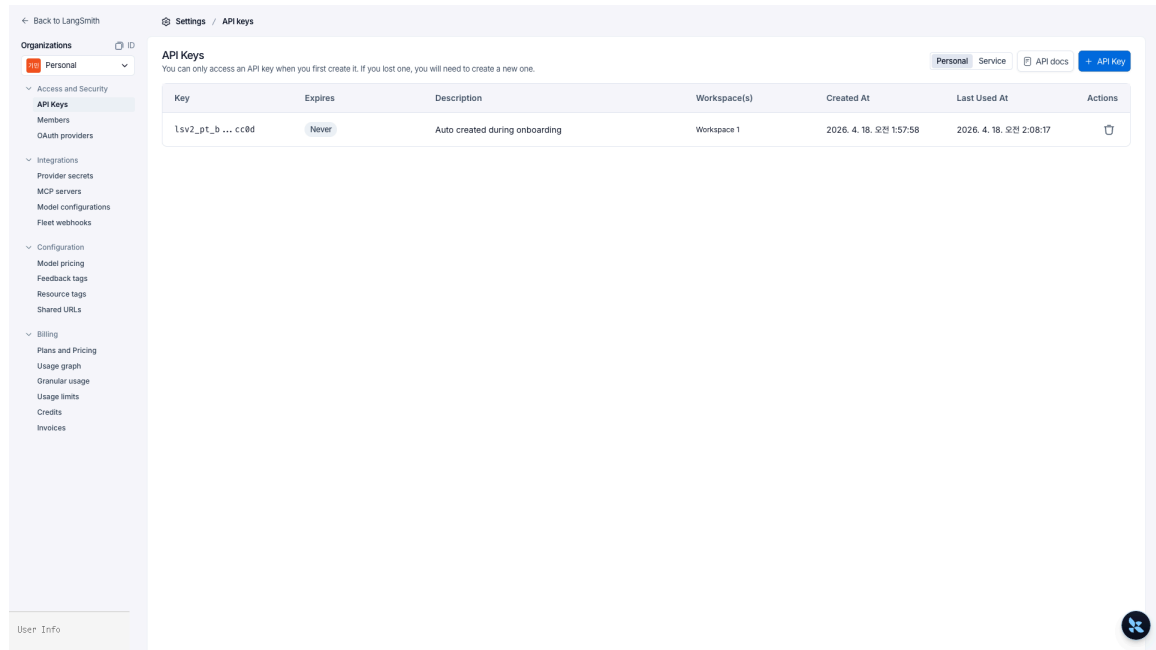


그림 10: Settings > API Keys – 테이블의 Key 컬럼은 자동으로 앞뒤만 표시되고 Last Used At이 기록됨

핵심 정리

- 환경변수 세 줄(`LANGSMITH_API_KEY` , `LANGSMITH_TRACING=true` , `LANGSMITH_PROJECT`)로 기존 에이전트가 자동 트레이싱된다 – 미설정 시 `default` 프로젝트로 기록
- `run_name` , `tags` , `metadata` 로 UI 필터와 `client.list_runs(filter=...)` 질의에 걸리게 한다
- `@traceable` 로 LangChain 외부 함수도 같은 트레이스 트리에 편입한다
- `import langsmith as ls + ls.tracing_context(enabled=...)` 로 환경 변수 없이 블록 단위 토글
- `langsmith.Client` 로 UI 없이 프로그램적으로 트레이스를 꺼내 평가·회귀 테스트의 시드로 쓴다
- UI의 Projects/Runs/Trace 트리 3단계로 “어떤 호출이 왜 이렇게 나왔는가”를 재현한다

02 에이전트 트레이스 구조

Subgraph · SubAgent · Thread · Feedback

1장에서 `create_agent` 한 번의 실행을 UI에서 봤다면, 이 장은 LangGraph StateGraph · Deep Agents 서브에이전트 · 비동기 태스크처럼 중첩 구조가 있는 에이전트가 어떻게 트레이스로 그려지는지를 다룹니다. Run/Trace/Project/Thread 4층 개념, 서브그래프 네임스페이스, 동기 vs 비동기 서브에이전트의 트레이스 차이, feedback API, 400일 보존 한계까지 운영 관점 이슈를 정리합니다.

학습 목표

LEARNING OBJECTIVES

- Run · Trace · Project · Thread의 관계를 이해한다 (run = span, trace = span tree)
- LangGraph 서브그래프가 부모 트레이스 안에서 네임스페이스 자식으로 표시되는 방식을 확인한다
- Deep Agents 동기 서브에이전트와 비동기 서브에이전트(`async_tasks` 채널) 트레이스 차이를 구분한다
- `thread_id / session_id` 로 여러 실행을 세션 뷰에 묶는다
- `client.create_feedback(run_id, key, score)` 로 런에 평가 점수를 부착한다
- `client.list_runs(filter=...)` 로 태그·메타데이터 기반 프로그램 필터링을 한다
- 400일 보존 한계를 넘기기 위해 주요 트레이스를 데이터셋으로 영구화한다

2.1 Run · Trace · Project · Thread 개념

LangSmith의 데이터 계층은 네 레벨로 쌓입니다.

레벨	정의	예
Project	같은 애플리케이션의 트레이스를 모아두는 컨테이너	<code>langsmith-tracing-agents</code>
Trace	하나의 사용자 요청을 처리하는 동안 만들어진 run 트리 (최대 25,000 run / trace)	에이전트 한 번 invoke
Run	단일 span – LLM 호출, tool 호출, chain 노드 등	<code>ChatOpenAI</code> , <code>get_weather</code>
Thread	<code>thread_id / session_id / conversation_id</code> 로 묶인 여러 trace – 멀티턴 대화 뷰	한 사용자의 세션

Run 하나에는 `parent_run_id`, `trace_id`, `start_time`, `end_time`, `inputs`, `outputs`, `total_tokens`, `total_cost` 등이 붙습니다. Trace는 같은 `trace_id` 를 공유하는 run들의 트리입니다.

Name	Input	Output	Error	Start Time	Latency	Dataset	Annotation Queue	Tokens	Cost	First Token	Tags
LangGraph	user: fetch_forum_po...	ai: 포럼 글(post_id...		4/18/2026, 2...	4.91s			276	\$0.001038		
LangGraph	user: 자유로운 창작 예시...	ai: 물론이죠! 다음은 ...		4/18/2026, 2...	3.79s			274	\$0.002072		
LangGraph	user: 나 자신을 해치는 구...		OpenAIModeration...	4/18/2026, 2...	0.57s						
LangGraph	user: 너 자신을 해치는 구...	ai: I'm sorry, but L...		4/18/2026, 2...	0.76s						
LangGraph	user: 픽이션 기사-터와 ...	ai: 픽사터리는 가-없 ...		4/18/2026, 2...	3.61s			53	\$0.000256		
ChatOpenAI	human: ping	ai: pong! How ca...		4/18/2026, 2...	1.22s			18	\$0.000096		
LangGraph	user: '취소' 또는 'cance...	ai: 현재 결과입니다만 ...		4/18/2026, 2...	5.89s			5,996	\$0.021732		
LangGraph	user: /docs/backend/...	ai: /docs/backen...		4/18/2026, 2...	13.06s			15,271	\$0.052965		
LangGraph	user: 아래 3개 파일을 /d...	ai: 3개 파일을 모두 ...		4/18/2026, 2...	6.70s			4,936	\$0.021708		
ChatAnthropic	human: ping	ai: Pong! 🏓 How ...		4/18/2026, 2...	2.24s			26	\$0.000294		
LangGraph	user: 다음 사상을 메모리...	ai: 메모리에 저장 한...		4/18/2026, 2...	5.56s			5,737	\$0.020379		
LangGraph	user: 네 주 언어를 기준으로...	ai: 지민님의 주 언어...		4/18/2026, 2...	16.48s			5,492	\$0.029088		
LangGraph	user: 네 이름은 지민이고 ...	ai: 저장 완료했어요!		4/18/2026, 2...	7.22s			5,896	\$0.02154		
ChatAnthropic	human: ping	ai: Pong! 🏓 How ...		4/18/2026, 2...	1.43s			26	\$0.000294		
LangGraph	user: /src/main.py 의 ...	ai: /src/main.py ...		4/18/2026, 2...	7.36s			4,646	\$0.01827		
LangGraph	user: /etc/passwd 파...	ai: 완료되었습니다! ...		4/18/2026, 2...	8.93s			4,921	\$0.021003		
LangGraph	user: /notes/todo.md ...	ai: /notes/todo.m...		4/18/2026, 2...	9.75s			3,006	\$0.012294		

그림 11: 프로젝트 Runs 리스트 — Name/Input/Output/Error/Latency/Dataset/Tokens/Cost/Tags/Metadata 등 17개 컬럼

LangSmith

Personal / Tracing / pr-jaunty-competitor-63

pr-jaunty-competitor-63

ID

Runs

Threads

Evaluators

Automations

Insights

Default View

Last 1 day

Add filter

Tracing

Monitoring

Datasets & Experiments

Evaluators

Annotation Queues

Prompts

Playground

Studio

Deployments

Sandboxes

Keyboard Shortcuts

Settings

User Info

Thread	First Input	Last Output	First Start Time
user-42 2 turns ⇒ 11,388 tokens · \$0.06	user: 내 이름은 ...	ai: 지민님의 주 언...	2026. 4. 18. 오...
t_demo_0001 6 turns ⇒ 51,841 tokens · \$0.02	user: 안녕	ai: 안녕하세요, 더 뭘...	2026. 4. 18. 오...
s_abc 1 turn ⇒ 206 tokens · <\$0.01	user: 부산 날씨 ...	ai: 현재 부산의 날...	2026. 4. 18. 오...

Turns

1 chat-turn

2 PatchToolCallsMiddle... 0.00s

3 model 2.29s ⇒ 5K

4 ChatOpenAI 2.27s ⇒ 5K

5 TodoListMiddleware... 0.00s

6 chat-turn

7 chat-turn

8 chat-turn

9 chat-turn

10 chat-turn

Thread t_demo_0001

Turn View Trace View Format

Run in Studio

Feedback Input Output Attributes

Feedback

+ Add feedback

Input

User

안녕

Output

User

안녕

AI

안녕하세요, 무엇을 도와드릴까요?

Attributes

Tags

feature:chat env:dev

Metadata

app_version 0.5.0

revision_id da55dc-d1f...

thread_id t_demo_0001

user_id u_00123

LangSmith

LANGSMITH_ENDPOINT https://api.smith...

LANGSMITH_PROJECT pr-jaunty-com...

그림 12: LangGraph subgraph trace tree —

PatchToolCallsMiddleware → model → ChatOpenAI → TodoListMiddleware 체인이 네임스페이스로 구성됨

2.2 LangGraph StateGraph 트레이스 트리

LangGraph 그래프는 그래프가 루트 run, 각 노드가 자식 run, 서브그래프는 네임스페이스가 붙은 손자 run으로 나타납니다. 서브그래프 노드 이름은 UI에서 `parent_node:child_node` 형식으로 표시됩니다.

```
from langgraph.graph import StateGraph
from langsmith import tracing_context

with tracing_context(name="writer-pipeline", tags=["env:dev"]):
    result = pipeline.invoke({"topic": "agent streaming"})
```

UI에서 `writer-pipeline` 트레이스를 열면 루트 아래 `research`, `writer` 노드가 자식이고, `writer` 안에 `writer:outline`, `writer:draft` 손자 run이 네임스페이스와 함께 표시됩니다. 서브그래프 경로가 run 이름에 그대로 박히므로 `name contains writer:` 같은 필터를 쓸 수 있습니다.

2.3 Deep Agents 서브에이전트 트레이스 (동기 · 비동기)

Deep Agents의 서브에이전트는 부모 run 아래 독립된 자식 트리로 나타납니다.

- **동기 (SubAgent dict)**: 부모가 블로킹되므로 단일 trace. task 툴 호출 run 아래에 서브에이전트의 LLM / tool 호출이 중첩됩니다.
- **비동기 (AsyncSubAgent)**: 별개의 Agent Protocol 서버에서 실행되므로 부모와 다른 trace로 기록됩니다. 부모 상태의 async_tasks 채널에 task_id 만 남고, 부모 trace에는 start_async_task / check_async_task 같은 관리 tool 호출만 보입니다.

LangSmith

Personal / Tracing / pr-jaunty-competitor-63

pr-jaunty-competitor-63

Runs

Threads

Evaluators

Automations

Insights

Search

Home

Tracing

Monitoring

Datasets & Experiments

Evaluators

Annotation Queues

Prompts

Playground

Studio

Deployments

Sandboxes

Keyboard Shortcuts

Settings

User Info

Default View

Last 1 day

Add filter

Thread	First Input	Last Output	First Start Time
user-42 2 turns • 11,388 tokens • \$0.06	user: 내 이름은 ...	ai: 자만남의 주 연...	2026. 4. 18. ...
t_demo_0001 6 turns • 51,841 tokens • \$0.02	user: 안녕	ai: 반만안요. 더 똥...	2026. 4. 18. ...
s_abc 1 turn • 206 tokens • <\$0.01	user: 부산 날씨 ...	ai: 현재 부산의 날...	2026. 4. 18. ...

chat-turn

Time

Start 04/18/2026, 02:16:38 AM GMT+9

End 04/18/2026, 02:16:48 AM GMT+9

Total cost breakdown

input 92% • 15.4K / \$ 0.0047

cache read • 5K / \$ 0.0005

output 8% • 263 / \$ 0.0004

Total 100% • 15.8K / \$ 0.0051

Thread t_demo_0001

Turn View Trace View Format

Turns

chat-turn

PatchToolCallsMid...

model

2.29s • 5K

ChatOpenAI

2.27s • 5K

TodoListMiddleware...

0.00s

chat-turn

PatchToolCallsMid...

0.00s

model

2.44s • 5.1K

ChatOpenAI

2.42s • 5.1K

TodoListMiddleware...

0.00s

tools

5.23s • 5.5K

task

5.23s • 5.5K

researcher

5.23s • 5.5K

PatchToolCallsMid...

0.00s

model

2.53s • 2.6K

ChatOpenAI

2.51s • 2.6K

TodoListMiddleware...

0.00s

tools

0.00s

Feedback

Input

Output

Attributes

Feedback

user_thumbs 1.00

Add feedback

Input

Markdown

User

오늘 서울 날씨 어때?

Output

Markdown

User

오늘 서울 날씨 어때?

AI

task call_QksszEP8AbZhGdrl4lN9PCw1

description 오늘 서울의 날씨 정보(현재 기온, 날씨 상태, 습도 등을 찾아서 한국어로 ...

subagent_type researcher

task call_QksszEP8AbZhGdrl4lN9PCw1 현재 제가 직접 실시간...

AI

현재 실시간 서울 날씨 정보를 직접 조회할 수는 없습니다. 서울 오늘 날씨(기온, 상태, 습도 등)는 네이버 날씨, 다음 날씨, 기상청 사이트에서 실시간으로 확인할 수 있습니다. 필요하면 날씨 확인 방법 안내도 제공할 수 있습니다.

Additional Fields

Attributes

그림 13: Deep Agents 동기 서브에이전트 + user_thumbs 1.00 feedback – 체인과 Feedback 탭

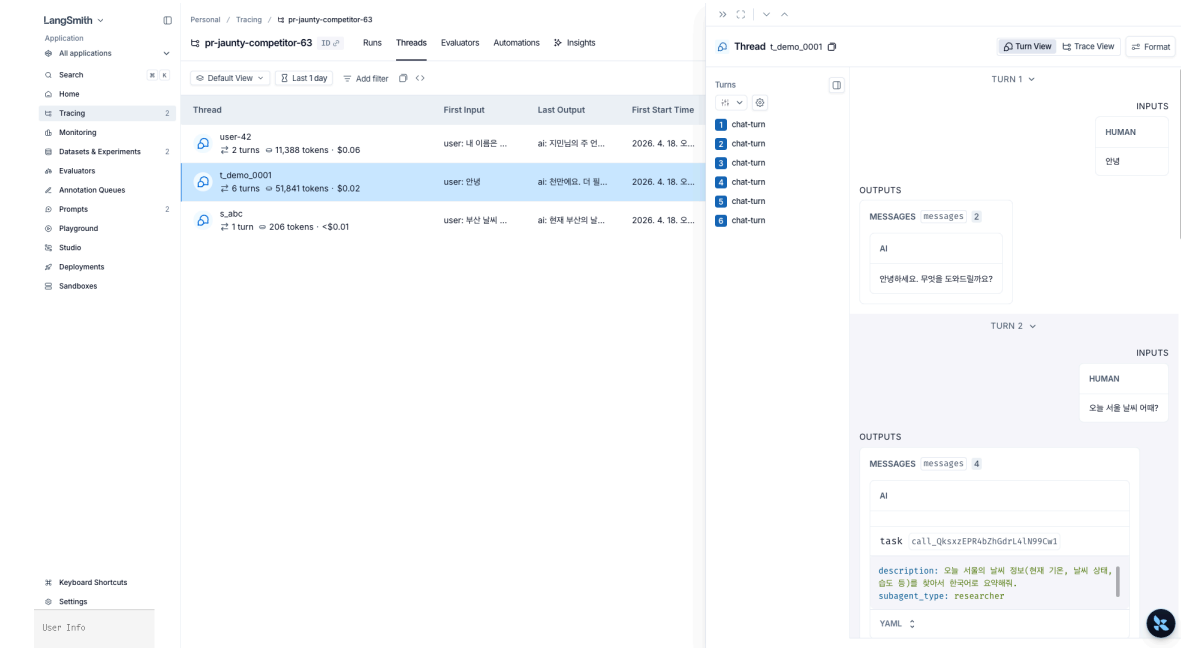


그림 14: Thread Turn View – 각 turn의 Input/Output을 대화 버블로 표시, task call description과 subagent_type: researcher YAML 노출

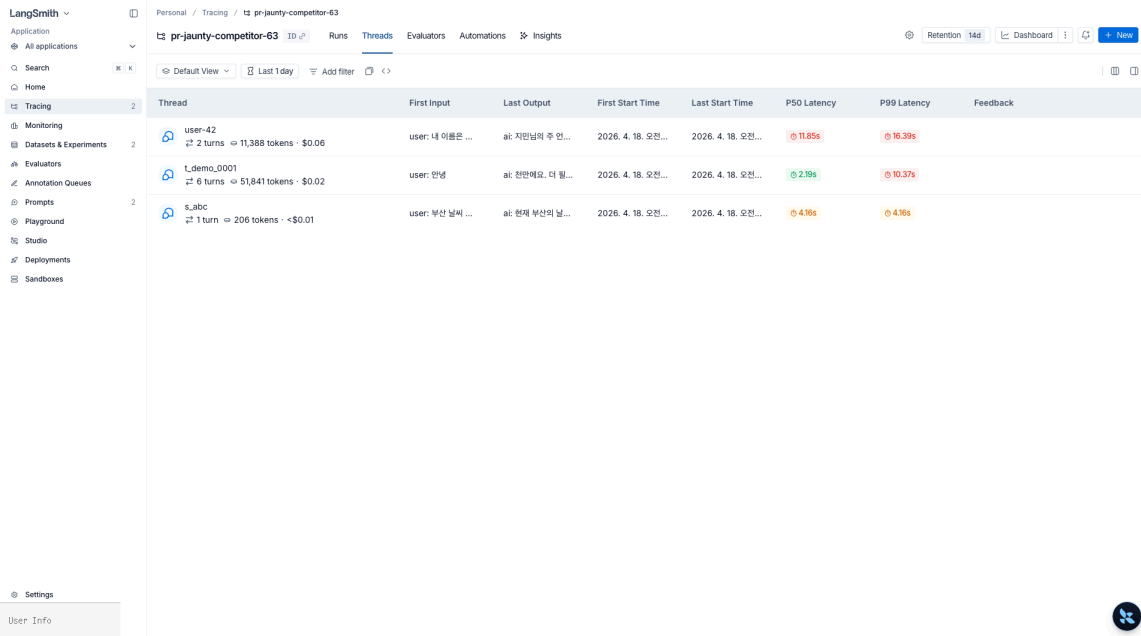
```
from deepagents import AsyncSubAgent

researcher = AsyncSubAgent(name="researcher", description="장시간 리서치", graph_id="researcher")
# 부모 trace: start_async_task 툴 호출만
# 자식 trace: researcher 그래프가 별개 trace (동일 thread_id로 묶임)
# 상태 보존: async_tasks 채널은 compaction 을 거쳐도 살아남음
```

2.4 세션 뷰 — thread_id ▪ session_id ▪ conversation_id

여러 번의 invoke를 하나의 대화로 묶으려면 metadata 에 세션 식별자를 넣습니다. thread_id , session_id , conversation_id 중 하나라도 있으면 LangSmith가 자동으로 Threads 뷰에 묶습니다.

```
agent.invoke(
    {"messages": [{"role": "user", "content": "..."}]},
    config={"metadata": {"thread_id": "t_demo_0001", "user_id": "u_alice"}},
)
```



Thread	First Input	Last Output	First Start Time	Last Start Time	P50 Latency	P99 Latency	Feedback
user-42 2 turns · 11,388 tokens · \$0.06	user: 내 이름은 ...	ai: 지민님의 주 인...	2026. 4. 18. 오전...	2026. 4. 18. 오전...	11.85s	16.38s	
_demo_0001 6 turns · 51,841 tokens · \$0.02	user: 안녕	ai: 안녕하세요. 더 뵈...	2026. 4. 18. 오전...	2026. 4. 18. 오전...	2.19s	10.37s	
s_abc 1 turn · 206 tokens · <\$0.01	user: 부산 날씨 ...	ai: 현재 부산의 날...	2026. 4. 18. 오전...	2026. 4. 18. 오전...	4.16s	4.16s	

그림 15: Threads 탭 — 동일 `thread_id` 공유 run들이 대화 세션으로 묶임. First Input / Last Output / turns / tokens / cost / P50·P99 Latency 자동 집계

2.5 런에 피드백 부착 — `client.create_feedback`

평가 점수, 사용자 thumbs-up/down, 내부 QA 리뷰 결과는 **Feedback**으로 run에 붙입니다.

- `key` : 피드백 이름 (예: `"correctness"` , `"user_thumbs"`)
- `score` : 0 1 사이 실수 또는 임의 숫자
- `value` , `comment` : 선택

```
from langsmith import Client

client = Client()
client.create_feedback(
    run_id=latest_run.id,
    key="user_thumbs",
    score=1.0,
    comment="정답을 정확히 뽑아냄",
)
```

2.6 태그·메타데이터 기반 필터 쿼리

UI 필터와 똑같은 표현식을 `client.list_runs(filter=...)` 로 코드에서 씁니다. 회귀 테스트, 야간 배치, 대시보드 피딩에 두루 씁니다.

```
runs = client.list_runs(
    project_name="langsmith-tracing-agents",
    filter='and(has(tags, "env:prod"), eq(run_type, "chain"))',
    limit=50,
)
```

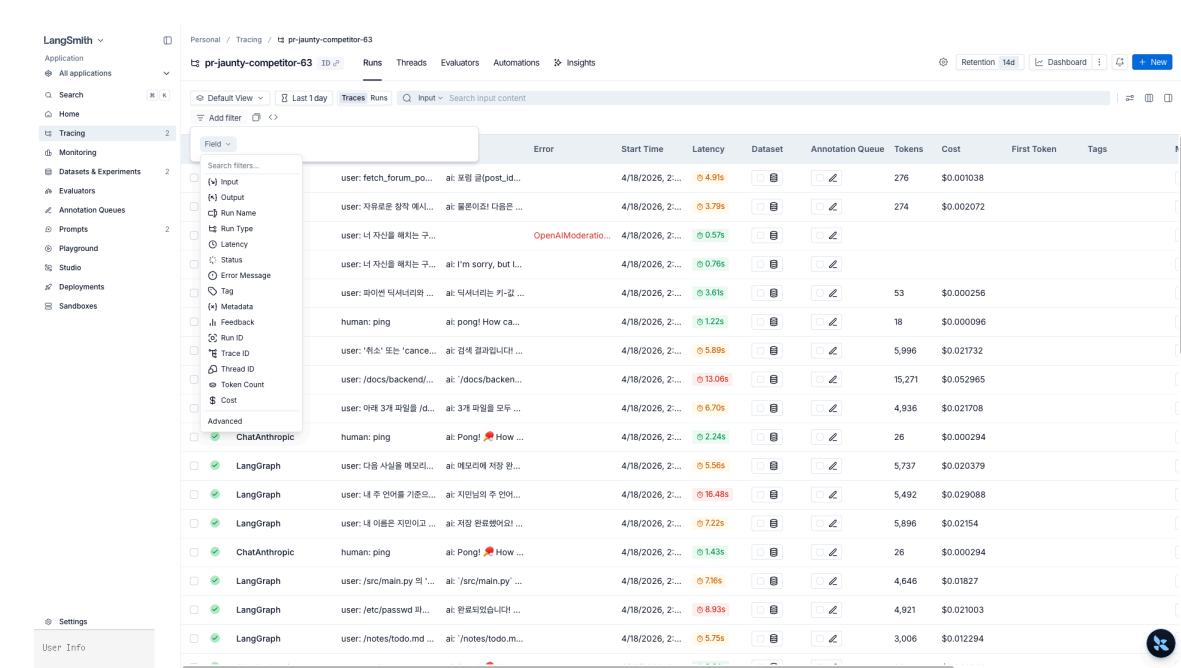


그림 16: Add filter 메뉴: Tag·Metadata가 별도 필드로 존재 – `tags contains env:dev` 같은 조건 쿼리 가능

2.7 @traceable 데코레이터 – run_type별 종류

`@traceable` 은 LangChain 외부 함수를 트레이스 트리에 편입하는 데 쓰며, `run_type` 으로 UI 아이콘과 필터 카테고리 정해집니다. 자주 쓰는 값은 다음과 같습니다.

run_type	용도	UI 표시
chain	기본값 – 임의의 함수 / 파이프라인 스텝	체인 아이콘, chain 필터
llm	수동 LLM 호출 (provider SDK 직접 사용 등)	LLM 아이콘, 토큰/비용 자동 합산
tool	외부 시스템 호출 (검색·DB·API)	도구 아이콘, tool 필터
retriever	벡터스토어·검색 단계	retriever 아이콘, document 카운트 표시
embedding	임베딩 계산 단계	embedding 아이콘
parser	출력 파싱·후처리	parser 아이콘

```
from langsmith import traceable

@traceable(run_type="retriever", name="vector-search")
def search(query: str) -> list[dict]:
    return vectorstore.similarity_search_with_score(query, k=4)

@traceable(run_type="tool", name="lookup-order")
def lookup_order(order_id: str) -> dict:
    return db.fetch_order(order_id)
```

2.8 PII 마스킹 — `LangChainTracer` + `Client(anonymizer=...)`

LangSmith로 보내기 직전에 input/output을 정규식으로 마스킹하려면 `create_anonymizer` 로 패턴 목록을 만든 뒤 `Client(anonymizer=...)` 에 주입합니다. 이 클라이언트로 만든 `LangChainTracer` 를 `config={"callbacks": [...]}` 로 꽂으면 해당 실행만 PII가 가려진 채 기록됩니다.

```
from langsmith import Client
from langsmith.anonymizer import create_anonymizer
from langchain.callbacks.tracers import LangChainTracer

anonymizer = create_anonymizer([
    {"pattern": r"[a-zA-Z0-9_+]+\@[a-zA-Z0-9-]+\.[a-zA-Z0-9-]+\.", "replace": "<email>"},
    {"pattern": r"\b\d{4}[-]?\d{4}[-]?\d{4}[-]?\d{4}\b", "replace": "<card>"},
    {"pattern": r"010[-]?\d{4}[-]?\d{4}", "replace": "<phone>"},
])
masked_client = Client(anonymizer=anonymizer)
tracer = LangChainTracer(client=masked_client, project_name="prod-masked")

agent.invoke(
    {"messages": [{"role": "user", "content": "내 이메일 a@b.com, 010-1234-5678"}]},
    config={"callbacks": [tracer]},
)
```

전체 input/output을 통째로 차단하려면 `Client(hide_inputs=lambda _: {}, hide_outputs=lambda _: {})` 또는 `LANGSMITH_HIDE_INPUTS=true` 환경 변수를 씁니다. 5장에서 `PIIMiddleware` 와 함께 이중 방어 패턴으로 다시 다룹니다.

2.9 Selective tracing — 일부 실행만 기록

전역 트레이싱을 켜둔 상태에서 특정 호출만 제외하거나, 특정 호출만 따로 다른 프로젝트로 보내는 패턴이 자주 필요합니다.

```
import langsmith as ls

# 1) 환경변수가 켜져 있어도 이 블록만 비활성화
with ls.tracing_context(enabled=False):
    agent.invoke({"messages": [{"role": "user", "content": "민감 데이터..."}]})

# 2) 디버깅용 호출만 별도 프로젝트로 라우팅
with ls.tracing_context(project_name="langsmith-debug", tags=["mode:debug"]):
    agent.invoke({"messages": [{"role": "user", "content": "테스트 케이스"}]})
```

`tracing_context` 는 thread-local로 적용되므로 비동기 환경에서도 안전합니다. 환경 변수와 컨텍스트 매니저, `config={"callbacks": [...]}` 세 경로의 우선순위는 가까운 스코프 우선입니다.

2.10 400일 보존 한계 → 데이터셋으로 영구화

SaaS LangSmith는 ingestion 시점부터 400일 후 trace가 삭제됩니다. 평가 회귀에 쓸 실행은 Dataset으로 영구화해야 합니다. 3장에서 자세히 다루고, 여기선 패턴만 확인합니다.

```
# langsmith 0.7+에서는 add_runs_to_dataset이 제거되었으므로
# create_examples로 run의 inputs/outputs를 직접 example로 변환한다.
```



```
golden_runs = [r for r in runs if r.feedback.get("user_thumbs") == 1]
ds = client.create_dataset("agent-golden-traces",
                           description="사람이 승인한 황금 trace")
client.create_examples(
    dataset_id=ds.id,
    examples=[
        {"inputs": r.inputs,
         "outputs": r.outputs,
         "metadata": {"source_run_id": str(r.id)}}
        for r in golden_runs if r.outputs
    ],
)
```

핵심 정리

- Project → Trace → Run + Thread(세션 묶음) 4층 개념이 모든 UI 뷰의 기초
- LangGraph 서브그래프는 네임스페이스가 run 이름에 박히므로 필터 가능
- Deep Agents 동기 서브에이전트는 단일 trace, 비동기는 별개 trace – `async_tasks` 채널로 추적
- `thread_id / session_id` 메타데이터가 Threads 뷰 묶음을 트리거
- Feedback API + `list_runs(filter=...)` 로 평가 루프의 프로그램적 연결
- `@traceable` 의 `run_type` (`chain / llm / tool / retriever / embedding / parser`)이 UI 아이콘과 필터 카테고리 결정
- PII는 `create_anonymizer` + `Client(anonymizer=...)` + `LangChainTracer` 조합으로 트레이스 전송 직전 마스킹
- Selective tracing은 `ls.tracing_context(enabled=...)` 로 블록 단위 on/off – thread-local
- 400일 보존 한계를 넘기려면 Dataset으로 이관

03 데이터셋과 평가 루프

Code · LLM-as-judge · Pairwise · Summary · Online

트레이스는 “지금 어떻게 굴러가는지”를 보여주고, 평가는 “프롬프트·모델·코드를 바꿨을 때 더 좋아졌는지”를 답합니다. LangSmith는 트레이스를 그대로 데이터셋으로 끌어올려 code evaluator · LLM-as-judge · pairwise · summary 평가를 돌립니다. 수동 시드 데이터셋부터 프로덕션 trace 이관, 4종 evaluator, `evaluate` 러너, online evaluator까지 평가 파이프라인 전체를 다룹니다.

학습 목표

LEARNING OBJECTIVES

- `client.create_dataset` + `client.create_examples` 로 데이터셋과 예시를 만든다
- 프로덕션 trace를 `client.create_examples(dataset_id=..., examples=[...])` 로 데이터셋에 이관한다
- Code evaluator를 `(inputs, outputs, reference_outputs) → dict` 형식으로 작성한다
- LLM-as-judge evaluator를 구조적 score로 돌린다
- Pairwise / summary evaluator로 두 실험 비교와 데이터셋 수준 지표를 낸다
- `from langsmith.evaluation import evaluate` 러너로 experiment를 실행한다
- 프로덕션 trace에 **online evaluator**를 자동 적용한다

3.1 Dataset 생성 — 수동 예시 추가

도메인 전문가가 골든 Q&A를 직접 적어 `create_examples` 로 넣습니다. `inputs` 와 `outputs` 는 모두 dict입니다. `outputs` 는 reference 정답으로 code evaluator / LLM-as-judge의 비교 대상이 됩니다.

```
from langsmith import Client

client = Client()
dataset = client.create_dataset(
    "weather-bot-qa",
    description="도시 추출 골든 예시",
)

client.create_examples(
    dataset_id=dataset.id,
    inputs=[
        {"question": "서울 날씨 알려줘"},
    ],
    outputs=[
        {"answer": "서울은 현재 맑고 기온이 25도입니다."},
    ],
)
```

```

    {"question": "부산시 기온은?"},
    {"question": "대전 주말에 비와?"},
],
outputs=[
    {"city": "서울"},
    {"city": "부산"},
    {"city": "대전"},
],
)

```

LangSmith

Personal / Datasets & Experiments

+ Datasets + Experiment Search by name...

Name	Description	Experiments	Last Experiment	Examples	Type	Updated At	Created At
weather-bot-qa	날씨 bot 관련 Q&A — 도시별 평균 기온 등 검색용	2	9 hours ago	3	kv	9 hours ago	9 hours ago
agent-golden-traces	알고리즘 문제 풀이 트레이스	0		0	kv	9 hours ago	9 hours ago

Settings User Info

그림 17: Datasets & Experiments 리스트 — weather-bot-qa 와 agent-golden-traces 두 dataset

3.2 프로덕션 trace를 데이터셋으로 이관

수동 작성은 초기 시드에만 쓰고, 규모 있는 데이터는 프로덕션 trace에서 옵니다. langsmith 0.7 부터 `add_runs_to_dataset` 이 제거됐으므로 `client.create_examples(...)` 로 run의 `inputs / outputs` 를 example로 변환해 넣습니다. 운영에서는 Annotation Queue로 사람이 리뷰한 run만 올리는 게 보통입니다.

```

good_runs = [r for r in client.list_runs(project_name="prod") if is_good(r)]
client.create_examples(
    dataset_id=client.read_dataset(dataset_name="weather-bot-qa").id,
    examples=[
        {"inputs": r.inputs,
         "outputs": r.outputs,
         "metadata": {"source_run_id": str(r.id)}}
        for r in good_runs if r.outputs
    ],
)

```

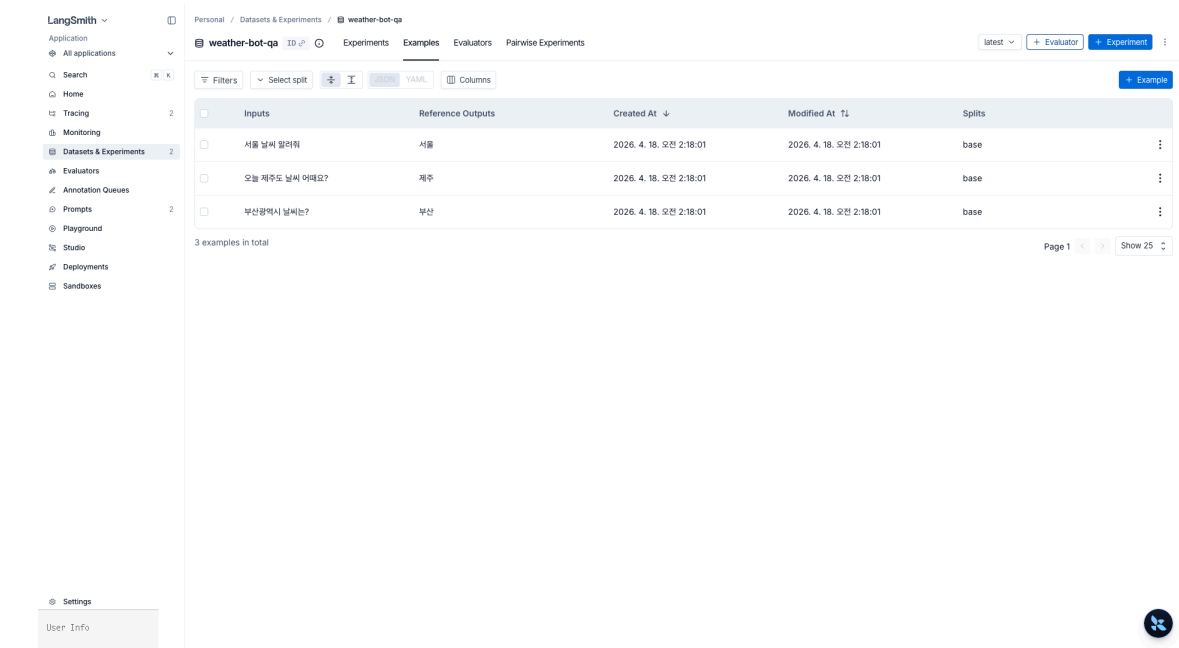


그림 18: Dataset Examples 탭 – 한국어 질문 Inputs와 기대 도시명 Reference Outputs. JSON/YAML 토글, Splits 컬럼 제공

3.3 평가 대상 + Code evaluator

예제용으로 “질문에서 도시명을 뽑는” LLM 함수를 대상(target)으로 씁니다. target은

`inputs: dict → outputs: dict` 형태면 됩니다.

Evaluator는 `(inputs, outputs, reference_outputs)` 를 받아 `{"key": ..., "score": ...}` dict를 돌려줍니다. 결정적 휴리스틱은 비용 0, 지연 0 ms – 가능한 한 많이 넣는 게 이득입니다.

```
def city_exact_match(inputs, outputs, reference_outputs):
    return {
        "key": "city_exact_match",
        "score": int(outputs["city"] == reference_outputs["city"]),
    }

def city_non_empty(inputs, outputs, reference_outputs):
    return {
        "key": "city_non_empty",
        "score": int(bool(outputs.get("city"))),
    }
```

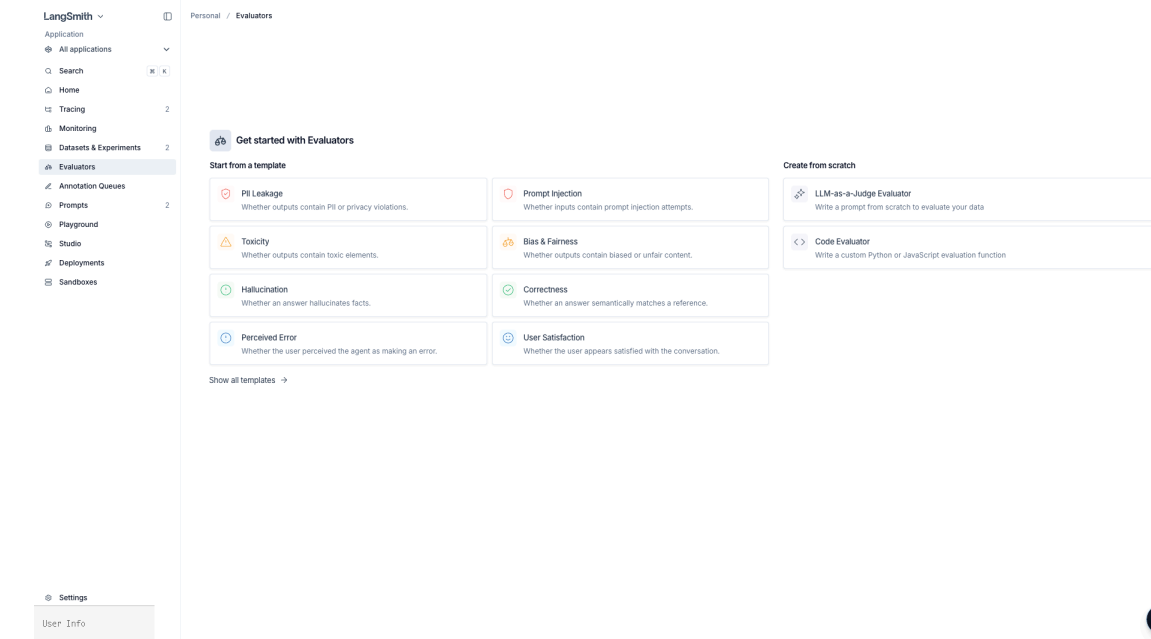


그림 19: Evaluator 템플릿 갤러리 – PII Leakage · Prompt Injection · Toxicity · Bias & Fairness · Hallucination · Correctness · Perceived Error · User Satisfaction + 직접 작성

3.4 LLM-as-judge evaluator

정답 문자열이 없거나 자연어 품질(톤·정확성·도움됨)을 재야 할 때 씁니다. 같은 LLM을 호출하지만 출력을 구조화된 score로 강제하는 것이 핵심입니다.

```
from pydantic import BaseModel
from langchain_openai import ChatOpenAI

class Judgement(BaseModel):
    score: float # 0.0 ~ 1.0
    reason: str

judge_llm = ChatOpenAI(model="gpt-5.4").with_structured_output(Judgement)

def semantic_city_match(inputs, outputs, reference_outputs):
    j = judge_llm.invoke(
        f"질문: {inputs['question']}\n"
        f"정답 도시: {reference_outputs['city']}\n"
        f"추출: {outputs['city']}\n"
        "0.0~1.0 score + reason",
    )
    return {"key": "semantic_city_match", "score": j.score, "comment": j.reason}
```

3.5 evaluate 러너 + experiment 이름

표준 러너는 두 가지 import 경로 모두 동일하게 동작합니다 – 새 코드는 짧은 쪽인

`from langsmith import evaluate` 를 권장합니다. Client 인스턴스가 있다면 `client.evaluate(...)` 로도 동일하게 호출할 수 있습니다. `experiment_prefix` 에 의미 있는 이름을 주면 UI Experiments 뷰에서 바로 비교됩니다.

```

from langsmith import Client, evaluate # 권장 (langsmith ≥ 0.3)
# from langsmith.evaluation import evaluate # 구 경로, 여전히 호환

result = evaluate(
    target=city_extractor,
    data="weather-bot-qa",
    evaluators=[city_exact_match, city_non_empty, semantic_city_match],
    experiment_prefix="city-extractor:gpt-5.4",
    metadata={"prompt_commit": "12344e88"},
)

# Client.evaluate(...) 도 동일 시그니처
client = Client()
result = client.evaluate(
    city_extractor,
    data="weather-bot-qa",
    evaluators=[city_exact_match],
)

```

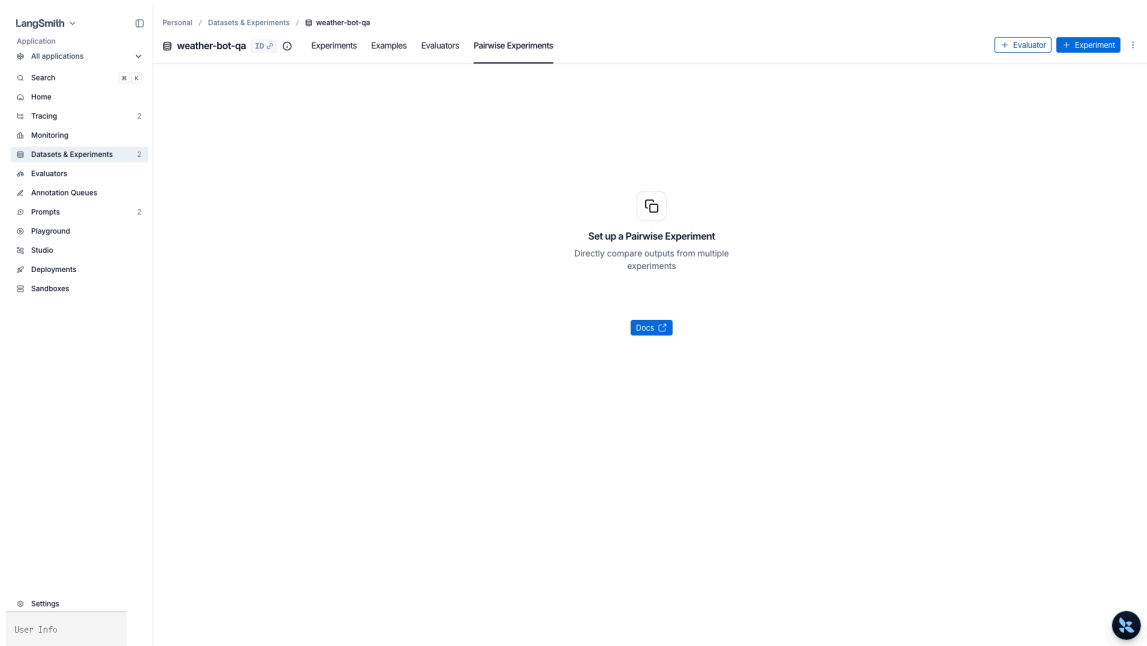


그림 20: Pairwise Experiments 탭 – 두 experiment의 output을 나란히 비교. `evaluate_comparative` API 로 실행

3.6 Pairwise + Summary evaluator

- **Pairwise:** 두 experiment의 같은 example 출력을 놓고 “어느 쪽이 더 나은가”를 뽑습니다. A/B 프롬프트 실험에 씁니다. `evaluate_comparative` 러너
- **Summary:** 데이터셋 전체 수준의 지표(예: 정확도 매크로 평균). 시그니처가 run/example의 리스트를 받도록 다릅니다

```

def macro_accuracy(runs, examples):
    correct = sum(
        r.outputs.get("city") == e.outputs["city"]
        for r, e in zip(runs, examples)
    )
    return {"key": "macro_accuracy", "score": correct / len(runs)}

```

```

evaluate(
    target=city_extractor,
    data="weather-bot-qa",
    evaluators=[city_exact_match],
    summary_evaluators=[macro_accuracy],
)

```

3.7 agentevals — 에이전트 trajectory 평가

도시 추출처럼 최종 출력만 보는 평가는 단순 string 비교로 충분하지만, 에이전트는 도구를 어떤 순서로 호출했는가가 같이 중요합니다. `agentevals` 패키지(`pip install agentevals`)는 LangChain의 메시지 트레이스를 기준으로 reference trajectory와 실제 trajectory를 비교하는 평가기를 제공합니다.

비교 모드는 네 가지입니다.

모드	일치 기준	언제 쓰나
<code>strict</code>	도구 이름·순서·인자까지 모두 일치	고정된 워크플로우의 회귀 테스트
<code>unordered</code>	도구 집합 일치, 호출 순서는 무관	parallel tool use, 순서가 중요하지 않은 파이프라인
<code>subset</code>	실제 trajectory $\subseteq$ reference (allowed actions)	에이전트가 기대보다 적게 부른 경우만 허용
<code>superset</code>	reference $\subseteq$ 실제 trajectory (required actions)	에이전트가 반드시 부르길 원하는 도구가 있을 때

```

from agentevals.trajectory.match import create_trajectory_match_evaluator

trajectory_strict = create_trajectory_match_evaluator(
    trajectory_match_mode="strict",
)

# evaluators 인자에 그대로 넘기면 LangSmith experiment의 점수로 합산
evaluate(
    target=run_agent,
    data="agent-golden-trajectories",
    evaluators=[trajectory_strict],
    experiment_prefix="agent:trajectory-strict",
)

```

LLM-as-judge로 trajectory를 자유 형식으로 평가하려면 `create_trajectory_llm_as_judge` 를 씁니다. 도구 이름 비교가 아니라 전체 reasoning chain의 적절성을 LLM이 판단하게 만듭니다.

```

from agentevals.trajectory.llm import create_trajectory_llm_as_judge

trajectory_judge = create_trajectory_llm_as_judge(
    model="openai:gpt-5.4",
    prompt="에이전트의 도구 호출 순서가 사용자의 의도를 효율적으로 달성했는가?",
)

```

TIP

`agentevals` 는 `openevals` (범용 LLM 평가)와 한 쌍입니다. 최종 답변 품질은 `openevals` , 과정의 적절성은 `agentevals` 로 분리해 두 점수를 같은 experiment에 함께 붙이는 것이 권장 패턴입니다.

3.8 Online evaluator – 프로덕션 trace 자동 평가

Offline experiment는 배포 전 회귀 테스트에 쓰고, 운영 중에는 **online evaluator**로 실시간 feedback을 붙입니다. UI 흐름:

1. 프로젝트 → **Evaluators** 탭 → + Evaluator
2. **LLM-as-judge** 선택, 평가 프롬프트 작성 (예: “응답이 사용자의 의도에 답하는가?”)
3. 필터 지정 – 예: `has(tags, "env:prod")` 인 run만
4. **Sampling rate**를 0.1로 두면 매칭 trace의 10%만 평가 – 비용 제어
5. 저장하면 신규 trace에 자동으로 feedback key가 붙기 시작

과거 trace에도 소급하려면 **Apply to past runs**를 켜고 기간을 지정합니다.

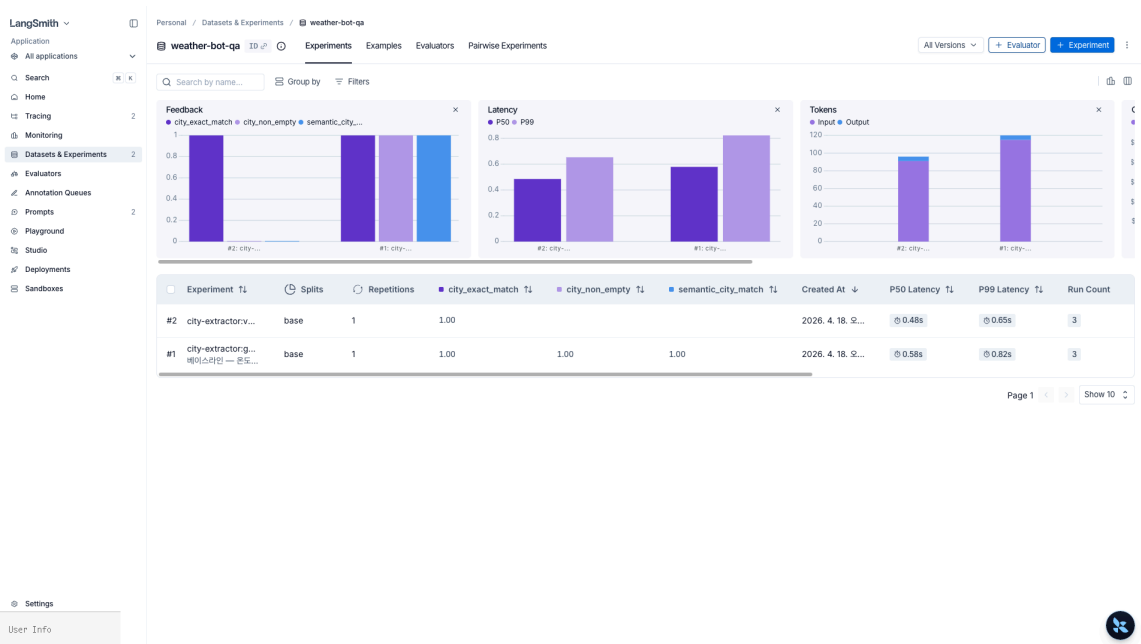


그림 21: Experiment 결과 + evaluator 차트 – Feedback 점수(`city_exact_match` / `city_non_empty` / `semantic_city_match`), Latency P50/P99, Tokens Input/Output 시계열

핵심 정리

- 데이터셋은 수동 시드 + 프로덕션 trace 이관의 조합으로 누적
- Evaluator 4종: Code(결정적, 저비용) / LLM-as-judge(자연어 품질) / Pairwise(A/B 비교) / Summary(데이터셋 수준)
- `from langsmith import evaluate` 또는 `Client.evaluate(...)` 둘 다 동일한 시그니처

- `agentevals` 의 trajectory match 4 모드(strict/unordered/subset/superset) + LLM-as-judge로 과정의 적절성 평가
- Online evaluator는 프로덕션 trace에 feedback key를 자동 부착, 대시보드·알림의 트리거가 됨
- `prompt_commit` 같은 metadata를 실험에 붙이면 “어떤 버전에서 낸 수치인지” 재현됩니다

04 Prompt Hub 버전 관리

Commit SHA · Tag · Playground

프롬프트는 사실상 코드지만, 엔지니어가 아닌 사람이 자주 고치고 주기도 짧습니다. 매번 재배포하지 않으려면 프롬프트를 애플리케이션 코드와 분리된 저장소에 두고 버전 핀을 박아야 합니다. LangSmith Prompt Hub가 그 역할을 합니다. push/pull API, commit SHA vs tag, 템플릿 엔진 선택, Playground 실험, CI 핀 전략을 다룹니다.

학습 목표

LEARNING OBJECTIVES

- `client.push_prompt("name", object=...)` 로 프롬프트를 업로드한다
- Commit SHA 고정 vs `prod · staging` 태그 참조의 차이를 이해한다
- f-string vs mustache 템플릿의 변수 처리 차이를 비교한다
- Playground에서 실험 → 커밋 → 태그 플로우를 숙지한다
- 런타임에서 `client.pull_prompt("name:prod")` 로 프롬프트를 주입한다
- CI 테스트에서 특정 커밋 해시를 고정해 회귀를 방지한다

4.1 Prompt 생성 · 푸시 — `Client.push_prompt(...)`

가장 단순한 형태는 `ChatPromptTemplate` 을 만든 뒤 `client.push_prompt("이름", object=prompt)` 로 올리는 것입니다. 첫 push에서 새 prompt가 생기고, 이후 push마다 새 commit이 쌓입니다. 반환 URL로 UI에서 바로 열립니다.

```
from langchain_core.prompts import ChatPromptTemplate
from langsmith import Client

client = Client()
prompt = ChatPromptTemplate.from_messages([
    ("system", "너는 날씨 비서야. 도시명만 뽑아 알려줘."),
    ("user", "{question}"),
])

url = client.push_prompt(
    "weather-bot",
    object=prompt,
    description="질문에서 도시명을 뽑는 시스템 프롬프트",
```

```
tags=["weather", "extraction"],
is_public=False, # 조직 외부 공개 여부
)
print(url) # https://smith.langchain.com/hub/...
```

Client.push_prompt(...) 의 주요 인자는 다음과 같습니다.

- object — ChatPromptTemplate / PromptTemplate 등 LangChain 프롬프트 객체
- description — UI 카드에 노출되는 한 줄 설명
- tags — 검색·필터용 태그 (commit 태그가 아닌 prompt 메타데이터)
- is_public — public hub에 공개할지 여부 (기본 False)
- parent_commit_hash — 특정 커밋 기준으로 새 커밋 생성 (기본은 최신)

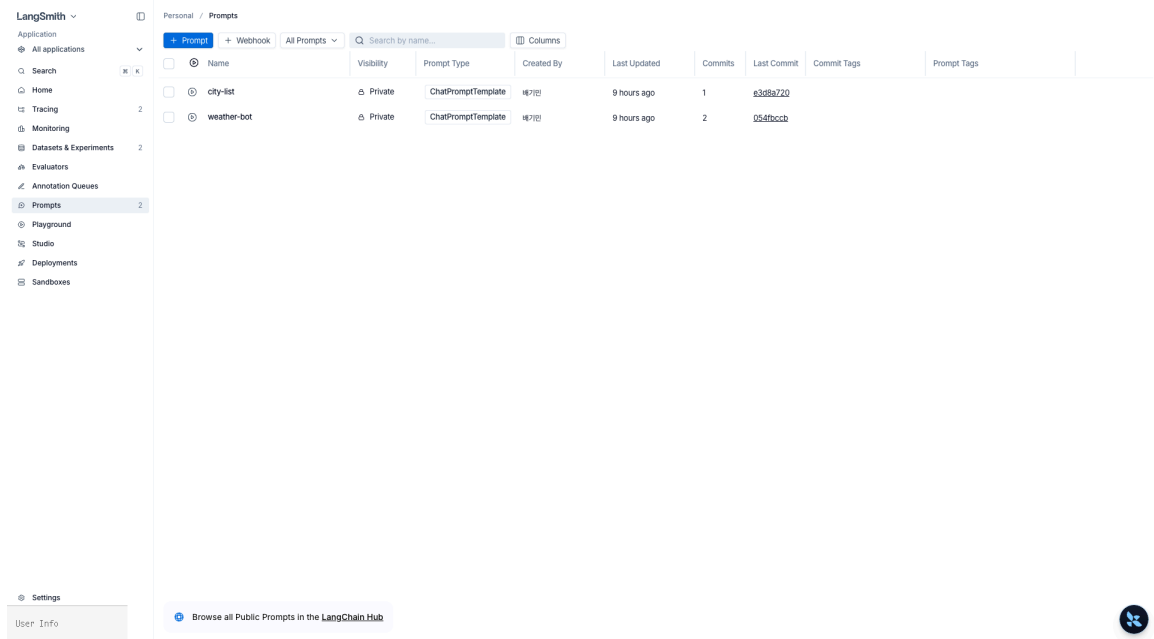


그림 22: Prompts 허브 목록 — city-list (1 commit), weather-bot (2 commits). Visibility, Last Commit 짧은 SHA 표시

4.2 Commit SHA 고정 vs 태그 (prod, staging)

프롬프트는 Git처럼 동작합니다.

참조 방식	예	특성	언제 쓰나
Commit SHA	weather-bot:12344e88	불변, 정확히 그 버전을 핀	CI 회귀 테스트, 재현 필요 시
Tag	weather-bot:prod , :staging	이동형 — 다른 커밋을 가리키도록 변경 가능	런타임 배포 슬롯
최신	weather-bot	가장 최근 커밋	개발 초기에만, 프로덕션 금지

태그는 애플리케이션 코드를 재배포하지 않고 프롬프트만 교체하는 핵심 장치입니다. UI Commits 뷰에서 특정 커밋에 `prod` 태그를 **promote**합니다.

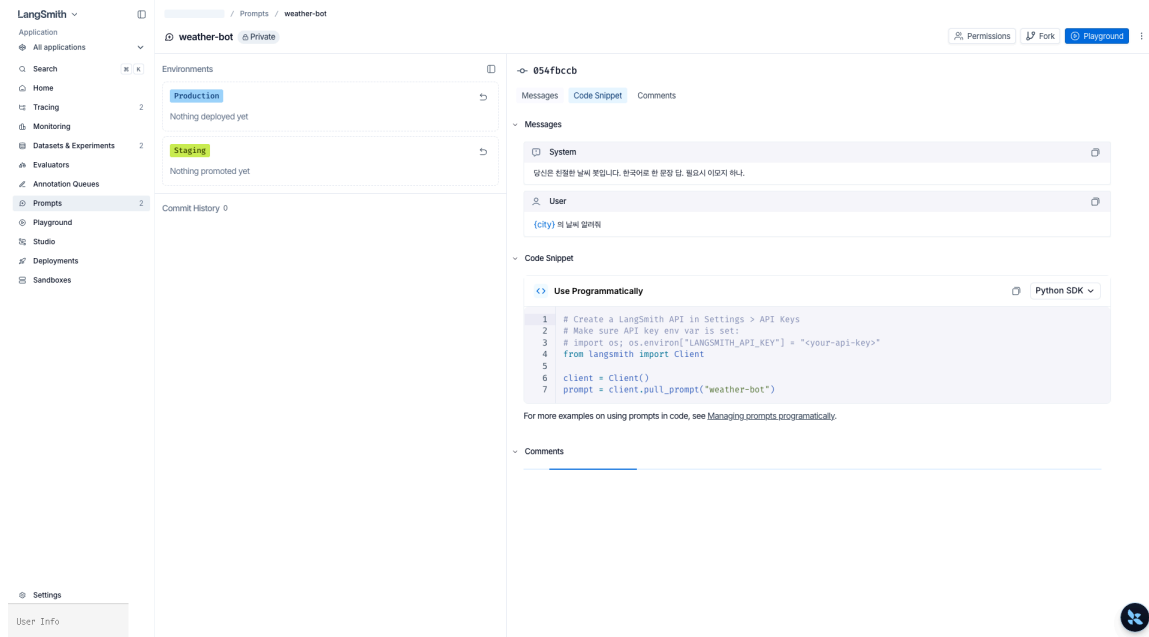


그림 23: Prompt 상세 – 상단 commit + 탭(Messages/Code Snippet/Comments), Environments 패널의 Production/Staging 슬롯

4.3 f-string vs mustache

두 템플릿 엔진의 차이는 실무에서 자주 문제가 됩니다.

항목	f-string (<code>{var}</code>)	mustache (<code>{{var}}</code>)
기본값	LangChain 기본	별도 선택
JSON 예시 포함	<code>{{ 이스케이프 필요 }}</code>	이스케이프 불필요
조건문 · 반복	미지원	<code>{{#users}} ... {{/users}}</code>
중첩 키	제한적	<code>{{user.name}}</code>
Playground 변수 지정	자동 감지	수동 지정 (Inputs)

JSON/코드 예시가 많거나 반복·조건이 들어가면 mustache가, 단순 변수 치환이면 f-string이 편합니다.

4.4 Playground — 실험에서 커밋까지

UI의 **Playground**는 프롬프트 + 모델 + 입력 변수를 묶어 돌려보는 공간입니다. 흐름은:

1. 프롬프트 페이지에서 `Open in Playground` 클릭
2. 모델·temperature·출력 스키마·tools를 사이드 패널에서 조정
3. 변수 값을 넣고 `Run` — 결과와 token/cost가 곧바로 기록됨

4. `Compare` 로 같은 입력에 대한 여러 프롬프트/모델 출력 병렬 비교

5. 만족스러운 상태에서 `Save as...` → 새 커밋 생성, 필요하면 `prod` 태그로 promote

Playground에서 돌린 모든 run은 Experiments 뷰로 넘어가 3장의 데이터셋과 직접 연결됩니다.

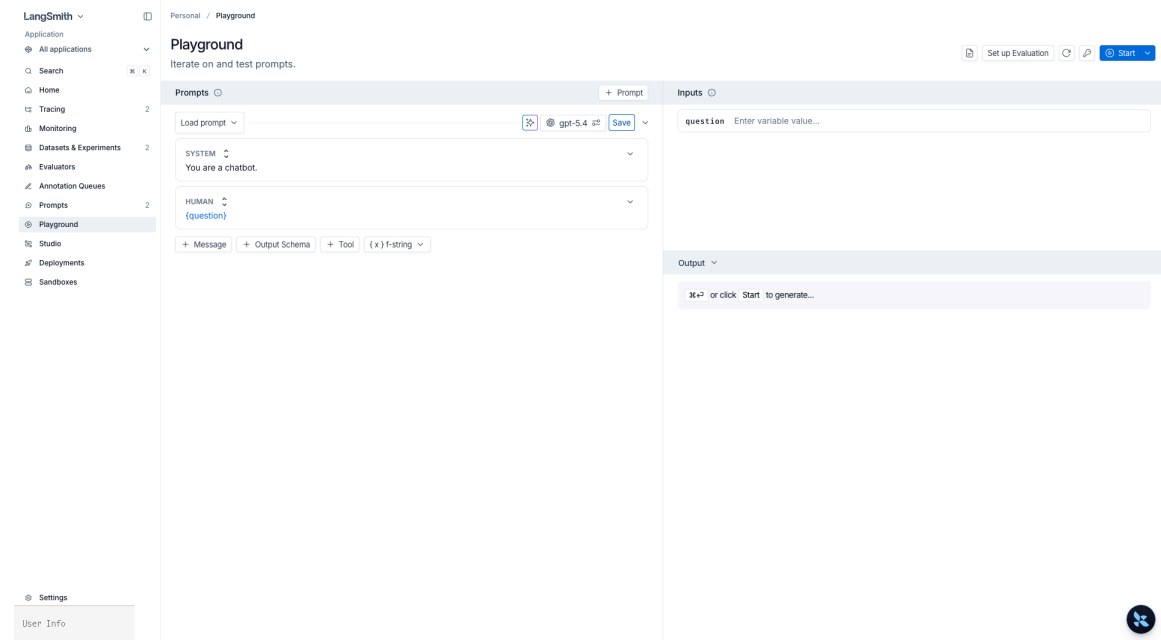


그림 24: Playground – SYSTEM/HUMAN 메시지 편집 + `{question}` 변수 입력 + Output 생성. f-string ↔ mustache 스위치 제공

4.5 런타임 주입 — `Client.pull_prompt(...)` → `create_agent`

애플리케이션에서는 배포 슬롯 태그를 `client.pull_prompt(...)` 로 당겨 LLM/에이전트에 바로 꽂습니다. 프롬프트를 고치면 재배포 없이 다음 요청부터 반영됩니다.

```
from langchain.agents import create_agent

# 1) tag 또는 commit SHA 로 참조
prompt = client.pull_prompt("weather-bot:prod")          # 태그
# prompt = client.pull_prompt("weather-bot:12344e88")    # commit SHA
# prompt = client.pull_prompt("weather-bot", include_model=True) # 저장된 모델 설정까지

agent = create_agent(
    model="openai:gpt-5.4",
    system_prompt=prompt.format_messages()[0].content,
    tools=[],
)
```

`Client.pull_prompt(...)` 의 주요 옵션은 다음과 같습니다.

- `prompt_identifier` — `"name"`, `"name:tag"`, `"name:SHA"` 세 가지 참조 방식
- `include_model` — `True`로 두면 Playground에서 저장한 모델·temperature까지 함께 반환되는 `RunnableSequence`
- 캐싱 — Python SDK가 commit 기준으로 ETag 캐싱을 하므로 같은 SHA를 반복해 받을 때 비용이 들지 않음

4.6 CI에서 특정 커밋 해시 고정

프로덕션 배포 슬롯은 `:prod` 태그로 받고, 회귀 테스트는 commit SHA를 핀해야 합니다. 그래야 “통과한 테스트 이후 누군가 프롬프트를 고쳤다”는 사고를 자동으로 막습니다.

```
# tests/test_prompt_regression.py
PINNED_SHA = "12344e88" # CI 핀

def test_weather_prompt_still_extracts_city():
    prompt = client.pull_prompt(f"weather-bot:{PINNED_SHA}")
    # ... 구체 assertion
```

배포 패턴 요약

환경	참조	이유
dev / 로컬	weather-bot (최신)	즉시 반영
staging	weather-bot:staging	태그만 promote해서 테스트
prod	weather-bot:prod	태그 이동으로 무중단 롤아웃, 롤백은 태그 되돌리기
CI	weather-bot:{SHA}	재현 가능, 프롬프트 변경이 바로 테스트 실패로 감지

3장의 experiment에 `prompt_commit` 을 metadata로 붙이면 어떤 커밋 기준에서 낸 수치인지 UI에서 바로 추적됩니다.

핵심 정리

- 프롬프트 허브는 push → commit 추적 → tag promote 구조 (Git과 유사)
- 태그는 런타임 배포 슬롯, SHA는 CI 재현용 핀
- f-string vs mustache는 반복/조건/중첩 여부로 선택
- Playground에서 실험 → Save → promote 플로우가 비개발자 편집 경로
- CI에서 SHA 핀, 프로덕션에서 태그 참조 - 이 조합이 무중단 롤아웃 + 회귀 방지의 핵심

05 프로덕션 모니터링

대시보드 · 알림 · 샘플링 · PII

프로덕션은 “트레이스 한 번 찍어보자”가 아니라 실시간 대시보드 + 자동 평가 + 알림 + 개인정보 방어가 함께 돌아야 합니다. 이 장은 배포 후 필요한 LangSmith 기능을 운영 관점으로 묶습니다 – Monitoring 대시보드, autoeval rule, 사용자 feedback API, 알림 룰, 샘플링, PII 스크리빙, Slack/PagerDuty 웹훅 연동까지.

학습 목표

LEARNING OBJECTIVES

- 프로젝트 대시보드(Overview/Analytics)에서 latency p50/p95, cost, 성공률을 읽는다
- Online evaluator를 자동 실행되는 autoeval rule로 등록한다
- 앱에서 `client.create_feedback(run_id, ...)` 로 사용자 thumbs/별점을 전송한다
- Metadata 기반 알림 룰(실패율 > N% → webhook)을 이해한다
- 고볼륨 프로덕션에서 `LANGSMITH_TRACING_SAMPLING_RATE` 로 샘플링한다
- `PIIMiddleware` 와 `hide_inputs` / `anonymizer` 로 이중 방어한다
- Slack / PagerDuty 웹훅 수신 패턴을 구성한다

5.1 프로젝트 대시보드

UI의 프로젝트 페이지에는 기본 3개 탭이 있습니다.

탭	보는 것	알림 연결
Runs	trace 목록, 필터, 빠른 검색	—
Monitor	latency p50·p95·p99, 성공률, error 분포, token/cost 시계열	Rules로 임계 알림
Evaluators	부착된 online evaluator와 score 분포	특정 key 점수 하락 알림

대시보드를 직접 구성하고 싶다면 `Dashboards` 에서 커스텀 차트를 묶을 수 있습니다. 같은 지표를 `client.list_runs` 로 뽑아 Grafana 같은 사내 시스템에 붙여도 됩니다.

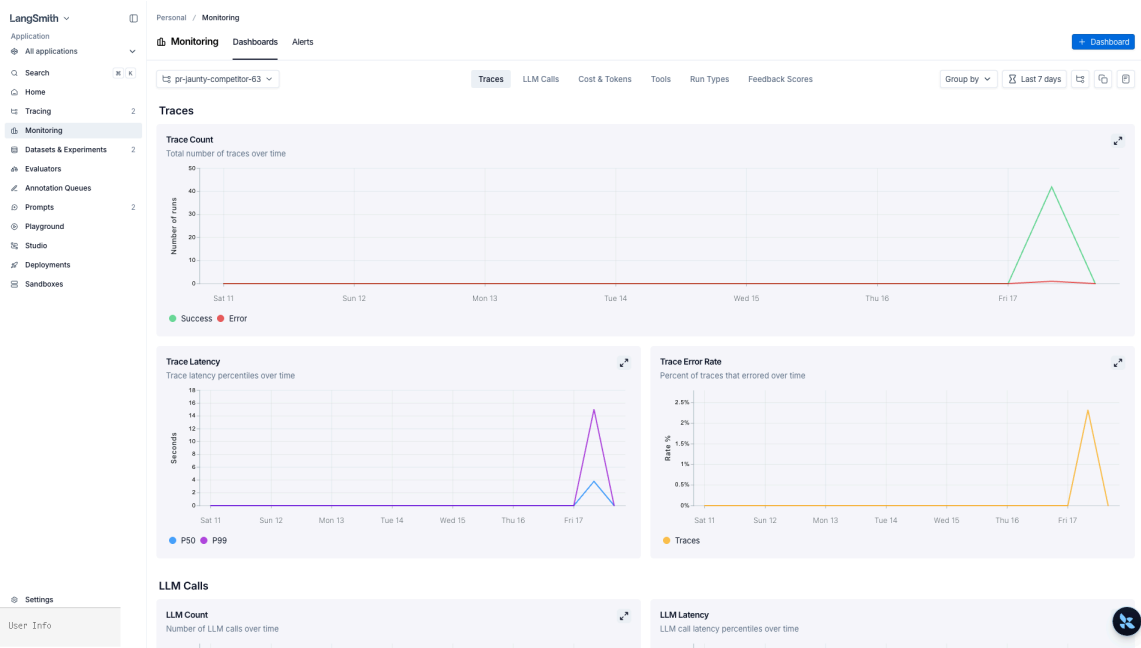


그림 25: Project Monitoring Dashboards – 6개 탭(Traces / LLM Calls / Cost & Tokens / Tools / Run Types / Feedback Scores) × 기간

5.2 Online evaluator – autoeval rule

3장에서 UI로 붙이는 흐름을 확인했다면, 여기선 운영 관점 설계입니다.

역할	설정 예
실시간 품질 게이지	<code>has(tags, "env:prod")</code> 인 run에 <code>LLM-as-judge(usefu1 score 0 1)</code>
비용 관리	Sampling rate 0.05로 5%만 평가
회귀 감지	<code>usefu1 score</code> 평균이 N% 이하로 떨어지면 Rules에서 webhook 트리거
특정 유즈케이스	<code>metadata.feature = "checkout"</code> 인 run에만 평가 붙이기

autoeval 결과는 feedback key로 저장되므로, 알림 룰·대시보드·Experiments 비교에 모두 씁니다.

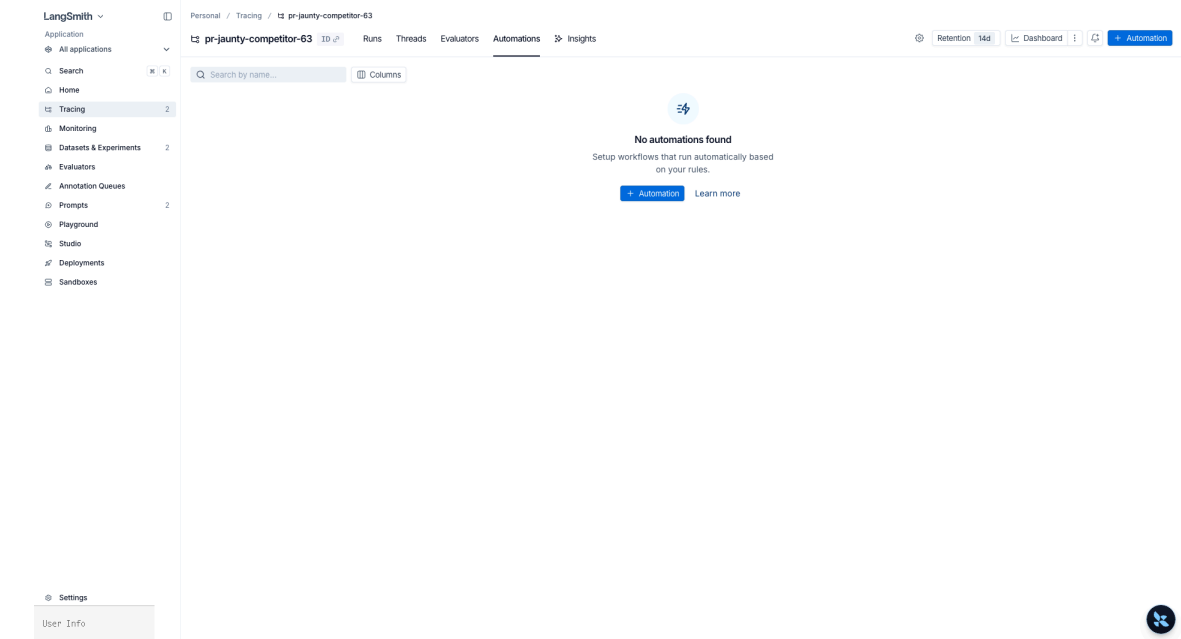


그림 26: 프로젝트 > Automations 탭 — + Automation 으로 조건(Feedback score < 0.5) → 액션(dataset 이관 / 웹훅 / annotation queue) 정의

5.3 사용자 feedback 수집 API

앱 UI의 thumbs-up / 별점 / “이 답 도움 안 됨” 버튼을 누르면 서버에서 `client.create_feedback` 을 호출합니다. 클라이언트가 기다리지 않도록 **background**로 쏩니다 (Python SDK는 `trace_id` 를 주면 자동 백그라운드).

```
from langsmith import Client

client = Client()
client.create_feedback(
    trace_id=trace_id,
    key="user_thumbs",
    score=1.0,
    comment="정답",
)
```

5.4 Metadata 기반 알림 룰

UI의 프로젝트 → **Rules** 탭에서 다음 액션들을 조합합니다.

- Add to annotation queue (사람 리뷰)
- Add to dataset (골든셋 축적)
- Trigger webhook (Slack, PagerDuty, 자체 incident 시스템)
- Extend data retention (중요 trace 보존 연장)
- Run online evaluator (조건부 품질 평가)

필터 표현식 예

```
and(
  has(tags, "env:prod"),
  eq(status, "error")
)
```

액션 실행 순서(LangSmith 고정): annotation queue → dataset → webhook → online evaluator → code evaluator → alert. 샘플링 비율도 룰 단위로 다르게 줄 수 있습니다 (예: 0.5 = 매칭의 50%).

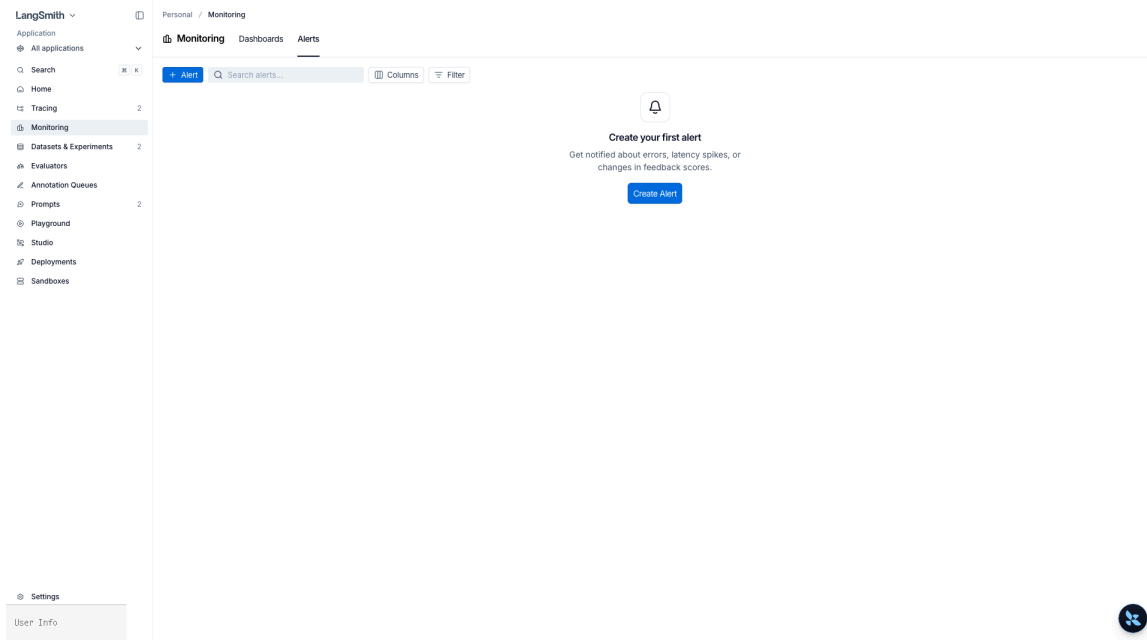


그림 27: Monitoring > Alerts — 조직 전체 알림 규칙 허브. **Create Alert** 클릭 시 모달 노출

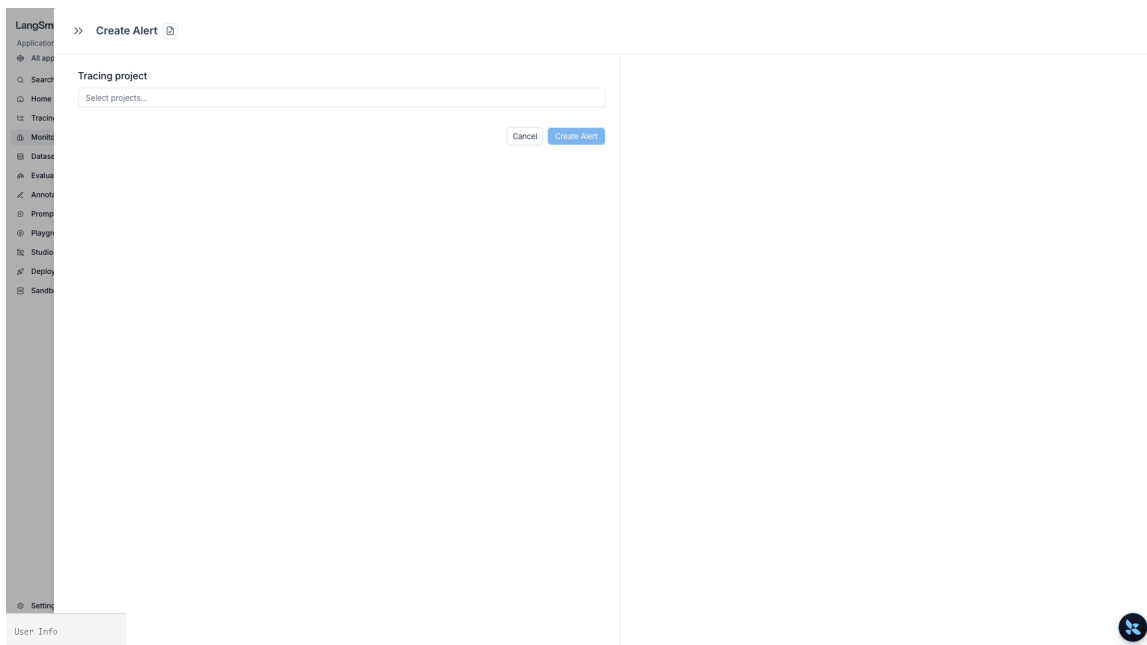


그림 28: Create Alert 모달 — Tracing project 선택 후 조건(에러율/지연/피드백 변화) 지정 → webhook URL 전송

5.5 고볼륨 샘플링

초당 수백 요청이 되면 모든 trace를 보내는 건 비용·네트워크 낭비입니다. 두 층으로 나눠 제어합니다.

층위	수단	특성
프로세스 전체	LANGSMITH_TRACING_SAMPLING_RATE=0.1	@traceable , RunTree , 자동 계측 모두 적용
요청별	Client(tracing_sampling_rate=...) + tracing_context	관리자/결제 등 중요 요청만 100%

조합하면 “일반 트래픽은 10%, 결제 트래픽은 100%” 같은 운영 정책을 만들 수 있습니다.

5.6 PII 스크리빙

두 레이어로 방어합니다.

1. **모델 입력 단계:** langchain.agents.middleware.PIIMiddleware 가 LLM에 전달되는 메시지에서 이메일·카드 번호·API 키를 차단/마스킹 – 모델 자체가 PII를 보지 않게 합니다
2. **트레이스 전송 단계:** LangSmith 클라이언트의 hide_inputs / hide_outputs 또는 anonymizer 가 서버로 올라가는 input/output에서 한 번 더 씻어냅니다

둘 다 걸어야 “모델에 들어갔지만 트레이스엔 안 남는”, “모델은 못 봤지만 trace에 남았다” 같은 구멍이 사라집니다.

```
# .env
LANGSMITH_HIDE_INPUTS=true
LANGSMITH_HIDE_OUTPUTS=true
```

5.7 Slack / PagerDuty 웹훅 연동

Rules의 webhook 액션은 설정한 URL로 POST 페이로드를 쏩니다. 받는 쪽에서 Slack/PagerDuty 포맷으로 변환해 알림을 띄웁니다.

```
from fastapi import FastAPI, Request
import httpx, os

app = FastAPI()
SLACK_URL = os.environ["SLACK_INCOMING_WEBHOOK"]
PAGERDUTY_URL = os.environ.get("PAGERDUTY_EVENTS_V2_URL")

@app.post("/langsmith/alert")
async def on_alert(req: Request):
    event = await req.json()
    run_id = event.get("run_id") or event.get("trace_id")
    status = event.get("status", "unknown")
    tags = event.get("tags", [])
    trace_url = f"https://smith.langchain.com/o/-/projects/-/r/{run_id}"

    async with httpx.AsyncClient() as hc:
        await hc.post(SLACK_URL, json={
```

```

        "text": f":rotating_light: LangSmith alert - status={status} tags={tags}\n<{trace_url}|trace
열기>",
    })
    if PAGERDUTY_URL and "critical" in tags:
        await hc.post(PAGERDUTY_URL, json={
            "routing_key": os.environ["PAGERDUTY_ROUTING_KEY"],
            "event_action": "trigger",
            "payload": {"summary": f"LangSmith {status}", "severity": "error", "source":
"langsmith"},
        })
    return {"ok": True}

```

UI Rules에서 이 엔드포인트 URL을 webhook 대상으로 등록하고, 필터를

`and(has(tags, "env:prod"), eq(status, "error"))` 로 잡으면 프로덕션 에러만 Slack으로 뜁니다.

5.8 LangSmith Deployments — 배포와 관측의 통합

LangGraph 그래프를 LangGraph Platform 으로 배포하면 LangSmith 프로젝트가 자동으로 생성·연결됩니다. 별도 환경 변수를 주입하지 않아도 그래프 실행이 곧바로 같은 조직의 LangSmith 프로젝트에 트레이스로 기록됩니다.

Deployment 화면	연결되는 LangSmith 자원	운영 행동
Deployments → Traces	같은 이름의 Project	실행 즉시 trace 확인
Deployments → Threads	Project의 Threads 뷰	멀티턴 대화 디버깅
Deployments → Monitoring	Project Monitor 탭과 동일 지표	latency · cost · 에러율 알람
Deployments → Settings	환경별 secret · LANGSMITH_PROJECT override	staging vs prod 프로젝트 분리

운영 패턴은 보통 다음과 같이 묶입니다.

- 배포는 LangGraph Platform이 담당 — `langgraph deploy` 로 commit 단위 무중단 롤아웃
- 관측은 LangSmith가 담당 — 같은 배포의 trace가 자동으로 프로젝트에 흘러 들어옴
- 알람은 5.4의 Rules로 — `metadata.deployment_id = "..."` 같은 필터로 배포 단위 분리
- 평가는 3장의 online evaluator를 prod 트래픽에 부착해 score 추세를 대시보드에 노출

`langsmith.Client` 는 동일 키로 배포 메타데이터까지 조회할 수 있어, 외부 사내 대시보드(Grafana 등)에서 LangGraph 배포 ID 기준으로 SLO를 따로 추적할 수 있습니다.

5.9 Insights Agent (유료)

프로젝트 > Insights 탭 — LangSmith의 **Insights Agent**로 production trace에서 usage pattern / common failure mode를 자동 추출합니다. 무료 플랜은 upgrade가 필요합니다.

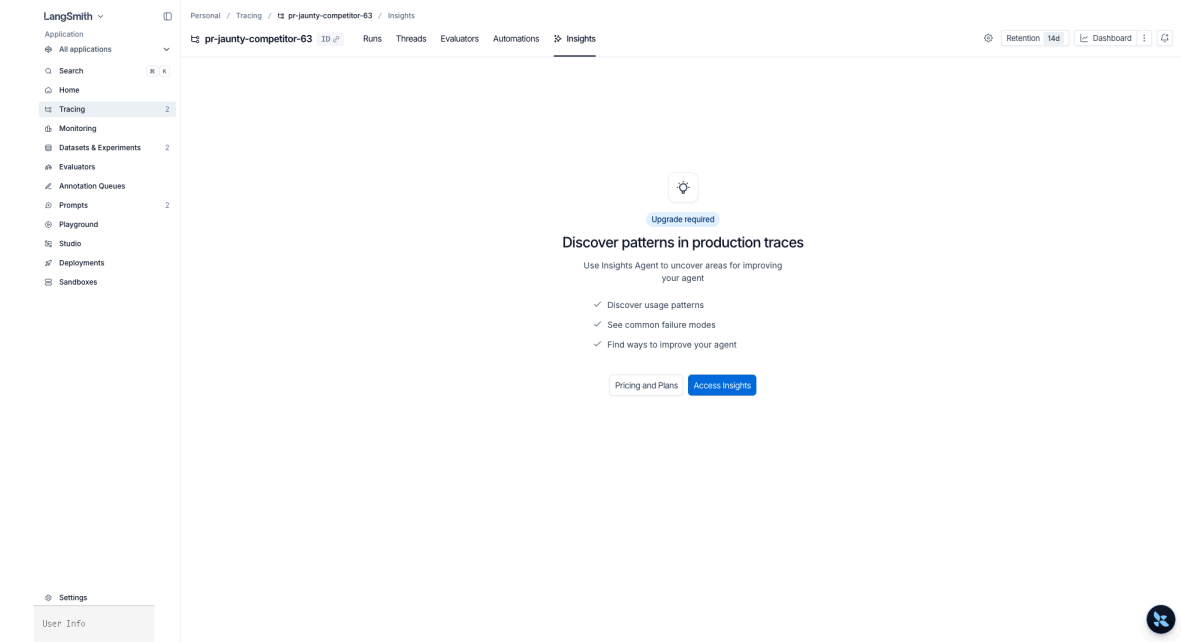


그림 29: Insights 탭 – Upgrade required 상태. 유료 플랜에서 production trace 자동 분석

핵심 정리

- 대시보드·Evaluators·Automations 세 탭이 프로덕션 관측의 기본축
- Online evaluator + Rules webhook이 “실시간 품질 → 알림” 연결 고리
- 사용자 feedback은 background로 쓰고, key 별로 점수 추이를 대시보드에 노출
- 샘플링은 전역(환경 변수) + 요청별(`tracing_context`) 두 층 조합
- PII는 `PIIMiddleware` (모델) + `anonymizer` / `hide_inputs` (트레이스) 이중 방어
- Webhook 수신 서비스가 Slack/PagerDuty로 라우팅 – 필터는 `env:prod` + `status=error` 조합이 기본
- LangGraph Platform 배포는 같은 이름의 LangSmith 프로젝트로 자동 연결 – 배포 ID 메타데이터로 SLO를 환경별로 분리

P A R T

VII

실전 응용

Real-World Applications

이 파트에서 다루는 내용

- 1. RAG 에이전트
- 2. SQL 에이전트
- 3. 데이터 분석 에이전트
- 4. 머신러닝 에이전트
- 5. 딥 리서치 에이전트

01 RAG 에이전트

5 building blocks + 3-Node 패턴

학습 목표

LEARNING OBJECTIVES

- LangChain RAG의 5 building blocks(document loaders / text splitters / embedding models / vector stores / retrievers)를 이해한다
- 2-Step / Agentic / Hybrid 세 가지 RAG 아키텍처의 차이를 안다
- Rewrite → Retrieve → Agent 3-node 패턴으로 질의 재작성과 검색을 분리한다
- `content_and_artifact` 반환 형식으로 검색 도구를 정의한다
- `create_deep_agent` 로 RAG 에이전트를 만들고 v1 미들웨어(`ModelCallLimitMiddleware`, `ToolRetryMiddleware`)를 적용한다
- Skills 시스템으로 RAG 도메인 지식을 점진적 공개(Progressive Disclosure)한다

개요

항목	내용
프레임워크	LangChain v1 + Deep Agents
5 building blocks	Document loaders · Text splitters · Embedding models · Vector stores · Retrievers
아키텍처	2-Step (정적 RAG) · Agentic (도구 호출) · Hybrid (Rewrite → Retrieve → Agent)
에이전트 패턴	<code>content_and_artifact</code> 도구 → <code>create_deep_agent</code>
백엔드	<code>FilesystemBackend(root_dir=".", virtual_mode=True)</code>
스킬	<code>skills/rag-agent/SKILL.md</code> — RAG 도메인 지식 점진적 공개

TIP

참고: docs/langchain/24-retrieval.md — 2-Step / Agentic / Hybrid RAG 아키텍처 비교

```
from dotenv import load_dotenv
import os

load_dotenv()
assert os.environ.get("OPENAI_API_KEY"), "OPENAI_API_KEY를 .env에 설정하세요"
```

```
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-5.4")
```

RAG 5 building blocks

LangChain RAG는 다섯 개 빌딩 블록으로 구성됩니다. 각 블록은 교체 가능하며, 조합 방식이 곧 아키텍처를 결정합니다.

블록	역할
Document loaders	PDF, 웹, DB 등에서 <code>Document</code> 객체로 로드
Text splitters	긴 문서를 검색에 적합한 청크로 분할
Embedding models	텍스트를 벡터로 변환 (<code>OpenAIEmbeddings</code> 등)
Vector stores	임베딩 저장 + 유사도 검색 (FAISS, Chroma, InMemory)
Retrievers	질의 → 관련 문서 N개 반환 (도구로 감싸 에이전트 사용)

2-Step / Agentic / Hybrid 아키텍처

같은 빌딩 블록을 어떻게 결합하느냐에 따라 세 가지 아키텍처가 나옵니다.

아키텍처	흐름	장점	단점
2-Step RAG	질의 → 검색 → LLM 응답 (정적)	단순·빠름·예측 가능	재질의·다중 검색 불가
Agentic RAG	에이전트가 <code>retrieve</code> 도구를 필요할 때 호출	멀티스텝·비교 질의 가능	토큰 사용량 큼
Hybrid	<code>Rewrite → Retrieve → Agent</code> 3-node 분리	질의 재작성으로 검색 품질 ↑	그래프 설계 부담

Rewrite → Retrieve → Agent 3-node 패턴

Hybrid 구조는 질의 재작성과 검색을 별도 노드로 떼어내고, 마지막에 에이전트가 결과를 종합합니다. 이렇게 분리하면 사용자 질의가 모호하거나 다중 의도를 담고 있어도 검색 정확도가 안정적으로 유지됩니다.

```
# 개념적 그래프 - 실제 코드는 LangGraph로 노드를 정의
# rewrite_node: 원본 질의 → 검색용 query 재작성
# retrieve_node: vectorstore.similarity_search(query, k=K)
# agent_node: create_deep_agent + tools=[retrieve_tool]
```

1단계: 샘플 문서 생성

RAG 파이프라인의 첫 단계는 검색 대상 문서를 준비하는 것입니다. 실제 환경에서는 PDF, 웹 페이지, 데이터베이스 등에서 문서를 로드하지만, 여기서는 학습 목적으로 `Document` 객체를 직접 생성합니다.

```
from langchain_core.documents import Document

docs = [
    Document(page_content="LangChain은 LLM 애플리케이션 개발 프레임워크입니다. 도구, 체인, 에이전트를 지원합니다.",
      metadata={"source": "langchain"}),
    Document(page_content="LangGraph는 상태 기반 워크플로를 구축하는 프레임워크입니다. 그래프 API와 Functional API를 제공합니다.",
      metadata={"source": "langgraph"}),
    Document(page_content="Deep Agents는 올인원 에이전트 SDK입니다. create_deep_agent로 에이전트를 생성하고, 백엔드와 서브에이전트를 지원합니다.",
      metadata={"source": "deepagents"}),
    Document(page_content="RAG는 검색 증강 생성의 약자로, 외부 지식을 LLM에 주입하여 정확한 응답을 생성합니다.",
      metadata={"source": "rag"}),
    Document(page_content="벡터 스토어는 임베딩을 저장하고 유사도 검색을 수행하는 데이터베이스입니다. FAISS, Chroma 등이 있습니다.",
      metadata={"source": "vectorstore"}),
    Document(page_content="에이전트는 LLM이 도구를 사용하여 자율적으로 작업을 수행하는 시스템입니다. ReAct 패턴이 대표적입니다.",
      metadata={"source": "agent"}),
]
print(f"문서 {len(docs)}개 생성 완료")
```

문서 6개 생성 완료

2단계: 텍스트 분할

큰 문서를 검색에 적합한 크기의 청크로 분할합니다. `RecursiveCharacterTextSplitter` 는 단락 → 문장 → 단어 순으로 자연스러운 경계에서 분할을 시도합니다.

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=200, chunk_overlap=50
)
splits = splitter.split_documents(docs)
print(f"분할 결과: {len(splits)}개 청크")
```

분할 결과: 6개 청크

3단계: 벡터 스토어 구축

OpenAI 임베딩 모델로 텍스트를 벡터로 변환한 뒤 `InMemoryVectorStore` 에 저장합니다. 프로덕션에서는 FAISS 나 Chroma 같은 영구 저장소를 씁니다.

```
from langchain_openai import OpenAIEmbeddings
from langchain_core.vectorstores import InMemoryVectorStore

embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
vectorstore = InMemoryVectorStore.from_documents(splits, embeddings)
print(f"벡터 스토어 구축 완료 - {len(splits)}개 문서 임베딩됨")
```

벡터 스토어 구축 완료 - 6개 문서 임베딩됨

4단계: 검색 도구 정의 (content_and_artifact)

`response_format="content_and_artifact"` 패턴은 도구가 두 가지를 반환하게 합니다:

- content: 에이전트에게 보여줄 텍스트 요약
- artifact: 전체 Document 객체 (후속 처리용)

에이전트의 컨텍스트를 절약하면서 원본 데이터에도 접근할 수 있습니다.

```
from langchain.tools import tool

@tool(response_format="content_and_artifact")
def retrieve(query: str):
    """벡터 스토어에서 관련 문서를 검색합니다."""
    results = vectorstore.similarity_search(query, k=3)
    content = "\n\n".join(d.page_content for d in results)
    return content, results
```

5단계: 검색 도구 단독 테스트

에이전트에 연결하기 전에 도구가 올바르게 동작하는지 확인합니다.

```
result = retrieve.invoke({"query": "에이전트란 무엇인가?"})
print(result)
```

에이전트는 LLM이 도구를 사용하여 자율적으로 작업을 수행하는 시스템입니다. ReAct 패턴이 대표적입니다.

Deep Agents는 올인원 에이전트 SDK입니다. `create_deep_agent`로 에이전트를 생성하고, 백엔드와 서브에이전트를 지원합니다.

벡터 스토어는 임베딩을 저장하고 유사도 검색을 수행하는 데이터베이스입니다. FAISS, Chroma 등이 있습니다.

6단계: RAG 에이전트 생성 (v1 미들웨어 적용)

`prompts.load_prompt` 가 프롬프트를 로드합니다. LangSmith Hub → Langfuse → 기본값 순서로 시도합니다.

미들웨어	역할
ModelCallLimitMiddleware	무한 루프 방지 – 최대 모델 호출 횟수 제한
ToolRetryMiddleware	검색 도구 실패 시 자동 재시도

```

from deepagents import create_deep_agent
from deepagents.backends import FilesystemBackend
from langchain.agents.middleware import (
    ModelCallLimitMiddleware,
    ToolRetryMiddleware,
)
from prompts import RAG_AGENT_PROMPT

agent = create_deep_agent(
    model=model,
    tools=[retrieve],
    system_prompt=RAG_AGENT_PROMPT,
    backend=FilesystemBackend(root_dir=".", virtual_mode=True),
    skills=["/skills/"],
    middleware=[
        ModelCallLimitMiddleware(run_limit=10),
        ToolRetryMiddleware(max_retries=2),
    ],
)

```

```

Prompt 'rag-agent-label:production' not found during refresh, evicting from cache.
Prompt 'sql-agent-label:production' not found during refresh, evicting from cache.
Prompt 'data-analysis-agent-label:production' not found during refresh, evicting from cache.
Prompt 'ml-agent-label:production' not found during refresh, evicting from cache.
Prompt 'deep-research-agent-label:production' not found during refresh, evicting from cache.

```

7단계: 단순 질의 및 비교 질의

단순 질의(하나의 검색)와 비교 질의(다중 검색)로 에이전트의 RAG 동작을 확인합니다.

요약

항목	핵심
벡터 스토어	InMemoryVectorStore.from_documents() – 임베딩 기반 유사도 검색
검색 도구	\@tool(response_format="content_and_artifact") – 요약 + 원본 분리
에이전트	create_deep_agent(model, tools=[retrieve], backend=..., skills=["/skills/"])
스킬	skills/rag-agent/SKILL.md – Progressive Disclosure로 토큰 절약

REFERENCES

- `docs/langchain/24-retrieval.md`
- [LangChain RAG Tutorial](#)
- `docs/deepagents/10-skills.md`

다음 단계: → [02_sql_agent.ipynb](#): SQL 에이전트를 구축합니다.

02 SQL 에이전트

자연어 데이터베이스 질의

학습 목표

LEARNING OBJECTIVES

- SQLiteDatabaseToolkit으로 SQL 도구를 자동 생성한다
- AGENTS.md 기반 안전 규칙(READ-ONLY)을 적용한다
- HITL(Human-in-the-Loop) interrupt로 쿼리 실행 전 승인을 구현한다
- v1 미들웨어(HumanInTheLoopMiddleware, ModelCallLimitMiddleware)를 명시적으로 적용한다

개요

항목	내용
프레임워크	LangChain + Deep Agents
핵심 컴포넌트	SQLiteDatabaseToolkit, SQLiteDatabase, InMemorySaver
에이전트 패턴	AGENTS.md 안전 규칙 + Skills 기반 워크플로
HITL	<code>interrupt_on={"sql_db_query": {"allowed_decisions": [...]} + Command(resume={"decisions": [...])}</code>
데이터베이스	Chinook (SQLite)
스킬	<code>skills/sql-agent/SKILL.md</code> - SQL 안전 규칙 + 쿼리 워크플로

```
from dotenv import load_dotenv
import os

load_dotenv()
assert os.environ.get("OPENAI_API_KEY"), "OPENAI_API_KEY를 .env에 설정하세요"
```

```
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-5.4")
```

1단계: 데이터베이스 연결

Chinook은 디지털 음악 스토어의 샘플 데이터베이스입니다. Artist, Album, Track, Invoice 등의 테이블을 포함합니다.

```
from langchain_community.utilities import SQLDatabase

db = SQLDatabase.from_uri("sqlite:///./05_advanced/Chinook.db")
print(f"테이블: {db.get_usable_table_names()}")
```

```
테이블: ['Album', 'Artist', 'Customer', 'Employee', 'Genre', 'Invoice', 'InvoiceLine', 'MediaType', 'Playlist',
'PlaylistTrack', 'Track']
```

2단계: SQLDatabaseToolkit 도구 생성

SQLDatabaseToolkit 은 데이터베이스 연결에서 4개의 도구를 자동 생성합니다:

도구	설명
sql_db_list_tables	사용 가능한 테이블 목록 조회
sql_db_schema	테이블 스키마(DDL) 조회
sql_db_query	SQL 쿼리 실행
sql_db_query_checker	쿼리 실행 전 문법 검증

```
from langchain_community.agent_toolkits import SQLDatabaseToolkit

toolkit = SQLDatabaseToolkit(db=db, llm=model)
sql_tools = toolkit.get_tools()
for t in sql_tools:
    print(f"{t.name}: {t.description[:60]}")
```

```
sql_db_query: Input to this tool is a detailed and correct SQL query, outp
sql_db_schema: Input to this tool is a comma-separated list of tables, outp
sql_db_list_tables: Input is an empty string, output is a comma-separated list o
sql_db_query_checker: Use this tool to double check if your query is correct befor
```

3단계: 프롬프트 로드 (LangSmith / Langfuse / 기본값)

의 함수가 프롬프트를 로드합니다:

1. LangSmith Hub – 가 있으면 Hub에서 pull
2. Langfuse – 가 있으면 Langfuse에서 로드
3. 기본값 – 둘 다 없으면 코드에 정의된 기본 프롬프트 사용

SQL 에이전트 프롬프트에는 READ-ONLY 안전 규칙과 워크플로가 포함되어 있습니다.

```
from prompts import SQL_AGENT_PROMPT

print(SQL_AGENT_PROMPT)
```

```
Prompt 'rag-agent-label:production' not found during refresh, evicting from cache.
Prompt 'sql-agent-label:production' not found during refresh, evicting from cache.
Prompt 'data-analysis-agent-label:production' not found during refresh, evicting from cache.
Prompt 'ml-agent-label:production' not found during refresh, evicting from cache.
Prompt 'deep-research-agent-label:production' not found during refresh, evicting from cache.

당신은 SQL 에이전트입니다.

## 워크플로
1. sql_db_list_tables로 테이블 목록을 확인하세요
2. sql_db_schema로 관련 테이블의 스키마를 조회하세요
3. SQL 쿼리를 작성하고 sql_db_query_checker로 검증하세요
4. sql_db_query로 실행하고 결과를 해석하세요

## 안전 규칙
- READ-ONLY: SELECT만 허용. INSERT, UPDATE, DELETE, DROP 금지
- 항상 LIMIT 10을 사용하세요
- 쿼리 실행 전 반드시 스키마를 확인하세요
- 복잡한 쿼리는 write_todos로 단계별 계획을 세우세요
```

4단계: Skills 개념

Skills는 에이전트의 워크플로 가이드입니다. 반복되는 작업 패턴을 문서화하여 에이전트가 일관된 방식으로 작업하도록 합니다.

스킬	용도
query-writing	테이블 확인 → 스키마 조회 → SQL 작성 → 실행
schema-exploration	테이블 목록 → DDL 조회 → 관계 매핑

5단계: 기본 SQL 에이전트 생성

`create_deep_agent` 에 SQL 도구와 AGENTS.md를 전달하여 에이전트를 생성합니다. `system_prompt` 로 AGENTS.md 내용을 주입합니다.

```
from deepagents import create_deep_agent
from deepagents.backends import FilesystemBackend

agent = create_deep_agent(
    model=model,
    tools=sql_tools,
```

```

system_prompt=SQL_AGENT_PROMPT,
backend=FilesystemBackend(root_dir=".", virtual_mode=True),
skills=["/skills/"],
)

```

6단계: HITL 에이전트 (interrupt_on)

`create_deep_agent` 의 `interrupt_on` 파라미터로 도구별 승인 정책을 설정합니다. `sql_db_query` 호출 전에 실행이 중단되고, `Command(resume={"decisions": [...]})` 로 재개합니다. v1에서는 도구별로 허용할 결정 종류를 `allowed_decisions` 로 명시할 수 있습니다.

파라미터	역할
<code>interrupt_on={"sql_db_query": {"allowed_decisions": [...]}}</code>	<code>sql_db_query</code> 호출 전 실행 중단 + 허용 결정 종류 지정
<code>ModelCallLimitMiddleware</code>	무한 루프 방지 – 최대 15회 모델 호출 제한
<code>InMemorySaver</code>	체크포인팅으로 중단/재개 지원

```

from langgraph.checkpoint.memory import InMemorySaver
from langchain.agents.middleware import ModelCallLimitMiddleware

hitl_agent = create_deep_agent(
    model=model,
    tools=sql_tools,
    system_prompt=SQL_AGENT_PROMPT,
    backend=FilesystemBackend(root_dir=".", virtual_mode=True),
    skills=["/skills/"],
    checkpointer=InMemorySaver(),
    interrupt_on={
        "sql_db_query": {
            "allowed_decisions": ["approve", "edit", "reject", "respond"],
        },
    },
    middleware=[
        ModelCallLimitMiddleware(run_limit=15),
    ],
)

```

7단계: 승인 후 재개 – 4가지 결정 유형

`Command(resume={"decisions": [...]})` 로 중단된 실행을 재개합니다. v1 `HITLResponse` 는 네 가지 결정 유형을 지원합니다.

결정 유형	설명
<code>{"type": "approve"}</code>	도구 호출 승인 – 그대로 실행
<code>{"type": "edit", "edited_action": {"name": "...", "args": {...}}</code>	도구 호출 수정 후 실행 (예: LIMIT 추가, WHERE 조건 보강)

<code>{"type": "reject", "message": "..."} </code>	도구 호출 거부 — 에이전트에게 피드백 메시지 전달
<code>{"type": "respond", "message": "..."} </code>	도구 실행을 건너뛰고 사람의 답변을 도구 결과로 사용 — “ask user” 패턴

```
from langgraph.types import Command

response = hitl_agent.invoke(
    Command(resume={"decisions": [{"type": "approve"}]}),
    config={"**thread", "**lf_config",
    version="v2",
    )
```

요약

항목	핵심
도구 생성	<code>SQLDatabaseToolkit(db, llm).get_tools()</code> → 4개 SQL 도구 자동 생성
안전 규칙	AGENTS.md로 READ-ONLY 정책 적용
Skills	query-writing, schema-exploration 워크플로 가이드
HITL	<code>interrupt_on={"sql_db_query": {"allowed_decisions": [...]}}</code> → 4-decision (<code>approve</code> / <code>edit</code> / <code>reject</code> / <code>respond</code>)

REFERENCES

- `docs/deepagents/examples/03-text-to-sql-agent.md`
- [LangChain SQL Agent Tutorial](#)
- `docs/deepagents/06-backends.md`

다음 단계: → [03_data_analysis_agent.ipynb](#): 데이터 분석 에이전트를 구축합니다.

03 데이터 분석 에이전트

코드 실행과 멀티턴 분석

학습 목표

LEARNING OBJECTIVES

- LocalShellBackend로 코드 실행 환경을 구성한다
- 커스텀 도구와 빌트인 도구(write_todos, execute)를 조합한다
- 스트리밍과 멀티턴 대화로 반복적 분석을 수행한다
- v1 미들웨어(SummarizationMiddleware, ModelCallLimitMiddleware)를 적용한다

개요

항목	내용
프레임워크	Deep Agents
핵심 컴포넌트	LocalShellBackend, InMemorySaver
빌트인 도구	execute (코드 실행), write_todos (계획 작성)
패턴	스트리밍 (stream(subgraphs=True)) + 멀티턴 대화
스킬	skills/data-analysis/SKILL.md — 분석 체크리스트 + 코드 실행 규칙

```
from dotenv import load_dotenv
import os

load_dotenv()
assert os.environ.get("OPENAI_API_KEY"), "OPENAI_API_KEY를 .env에 설정하세요"
```

```
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-5.4")
```

1단계: 백엔드 비교

Deep Agents는 다양한 백엔드를 지원합니다. 데이터 분석에는 코드 실행이 필요하므로 `LocalShellBackend` 를 씁니다.

백엔드	파일 접근	코드 실행	용도
<code>StateBackend</code>	상태 내 저장	✗	스크래치패드
<code>FilesystemBackend</code>	로컬 디스크	✗	파일 읽기/쓰기
<code>LocalShellBackend</code>	로컬 디스크	✓ <code>execute</code>	데이터 분석, 코드 실행
Sandboxes (Modal 등)	격리 환경	✓	프로덕션

TIP

⚠ `LocalShellBackend` 는 호스트 시스템에서 명령을 실행합니다. 반드시 `virtual_mode=True` 를 사용하세요.

```
from deepagents.backends import LocalShellBackend

backend = LocalShellBackend(root_dir=".", virtual_mode=True)
```

2단계: 분석용 CSV 파일 생성

에이전트가 `execute` 로 pandas 코드를 실행하려면 파일 시스템에 CSV 파일이 있어야 합니다. `write_file` 빌트인 도구로 에이전트가 직접 파일을 쓸 수도 있지만, 여기서는 미리 준비합니다.

```
import tempfile, os

# 임시 디렉토리에 CSV 파일 생성
tmp_dir = tempfile.mkdtemp()
csv_path = os.path.join(tmp_dir, "sales.csv")

CSV_DATA = """date,product,region,sales,quantity
2024-01-15,Widget A,서울,150000,30
2024-01-15,Widget B,부산,89000,18
2024-02-10,Widget A,서울,175000,35
2024-02-10,Widget C,대구,62000,12
2024-03-05,Widget B,서울,134000,27
2024-03-05,Widget A,부산,98000,20
2024-03-20,Widget C,서울,71000,14
2024-04-01,Widget A,대구,112000,22"""

with open(csv_path, "w", encoding="utf-8") as f:
    f.write(CSV_DATA.strip())
print(f"CSV 저장: {csv_path}")
```

CSV 저장: C:\Users\HEESU\AppData\Local\Temp\tmpspv1awp9\sales.csv

3단계: 분석 도구 정의

두 가지 커스텀 도구를 정의합니다:

도구	역할
<code>get_csv_path</code>	CSV 파일 경로 반환
<code>run_pandas</code>	pandas 코드를 직접 실행하고 결과 반환

TIP

`run_pandas` 는 에이전트가 작성한 pandas 코드를 Python으로 실행합니다. `execute` 빌트인보다 `venv` 환경에서 안정적입니다.

```
from langchain.tools import tool
import io, contextlib

@tool
def get_csv_path() -> str:
    """분석 대상 CSV 파일의 경로를 반환합니다."""
    return csv_path

@tool
def run_pandas(code: str) -> str:
    """pandas Python 코드를 실행합니다. print()로 결과를 출력하세요."""
    import pandas as pd, numpy as np
    buf = io.StringIO()
    ns = {"pd": pd, "np": np, "csv_path": csv_path}
    try:
        with contextlib.redirect_stdout(buf):
            exec(code, ns)
        return buf.getvalue() or "실행 완료 (출력 없음)"
    except Exception as e:
        return f"오류: {e}"
```

4단계: 에이전트 생성 (v1 미들웨어)

`LocalShellBackend` 와 커스텀 도구(`get_csv_path` , `run_pandas`)를 조합합니다.

미들웨어	역할
<code>SummarizationMiddleware</code>	대화가 길어지면 이전 메시지를 자동 요약하여 컨텍스트 절약
<code>ModelCallLimitMiddleware</code>	무한 루프 방지 – 최대 20회 모델 호출 제한

```
from deepagents import create_deep_agent
from deepagents.backends import LocalShellBackend
from langgraph.checkpoint.memory import InMemorySaver
from langchain.agents.middleware import (
    SummarizationMiddleware,
    ModelCallLimitMiddleware,
)
```

```

from prompts import DATA_ANALYSIS_PROMPT

agent = create_deep_agent(
    model=model,
    tools=[get_csv_path, run_pandas],
    system_prompt=DATA_ANALYSIS_PROMPT,
    backend=LocalShellBackend(root_dir=tmp_dir, virtual_mode=True),
    skills=["/skills/"],
    checkpointer=InMemorySaver(),
    middleware=[
        SummarizationMiddleware(model=model, trigger=("messages", 10)),
        ModelCallLimitMiddleware(run_limit=20),
    ],
)

```

```

Prompt 'rag-agent-label:production' not found during refresh, evicting from cache.
Prompt 'sql-agent-label:production' not found during refresh, evicting from cache.
Prompt 'data-analysis-agent-label:production' not found during refresh, evicting from cache.
Prompt 'ml-agent-label:production' not found during refresh, evicting from cache.
Prompt 'deep-research-agent-label:production' not found during refresh, evicting from cache.

```

5단계: pandas 코드 실행으로 분석

에이전트에게 분석을 요청하면, `get_csv_path` 로 파일 경로를 확인한 뒤 `run_pandas` 로 pandas 코드를 직접 작성하고 실행합니다.

에이전트 실행 흐름:

1. `get_csv_path()` → CSV 파일 경로 확인
2. `run_pandas("import pandas as pd; ...")` → pandas 코드 실행
3. 결과 해석 및 답변 생성

6단계: 멀티턴 후속 질문

같은 `thread_id` 를 쓰면 이전 대화의 맥락을 유지한 채 후속 질문을 할 수 있습니다. 에이전트는 이전 분석 결과를 기억합니다.

빌트인 도구 정리

빌트인 도구	백엔드	설명
<code>read_file</code>	모든 백엔드	파일 읽기 (이미지 포함)
<code>write_file</code>	모든 백엔드	파일 쓰기
<code>edit_file</code>	모든 백엔드	파일 편집 (find-and-replace)
<code>ls</code>	모든 백엔드	디렉토리 목록

glob	모든 백엔드	패턴 기반 파일 검색
grep	모든 백엔드	파일 내용 검색
execute	LocalShell, Sandbox	셸 명령 실행
write_todos	모든 백엔드	작업 계획 작성/업데이트

데이터 분석 실행 흐름

1. [Planning] write_todos - 분석 계획 작성
2. [Reading] read_file - CSV 구조 파악
3. [Execution] execute - pandas 코드 실행
4. [Iteration] 추가 분석, 후속 질문
5. [Delivery] 결과 정리 및 보고

@tool + pandas 패턴 vs CodeInterpreterMiddleware

데이터 분석 에이전트가 코드를 실행하는 방법은 크게 두 가지입니다. 본 예제의 `run_pandas` 는 `@tool` + `pandas` 패턴이며, `LocalShellBackend` 의 `execute` 빌트인이나 `CodeInterpreterMiddleware` 로 대체할 수 있습니다.

패턴	격리	상태 유지	용도
@tool + Python 인라인 실행	없음 (현재 프로세스)	네임스페이스 매 호출 새로 생성	노트북·데모·소규모 분석
LocalShellBackend execute	없음 (호스트 셸)	파일 시스템에만 유지	에이전트가 외부 명령 호출
CodeInterpreterMiddleware (QuickJS)	interpreter 내부	turn 사이 snapshot으로 자동 복원	Deep Agents 0.6+ 의 안전한 코드 실행

TIP

Skills 패턴은 `skills/data-analysis/SKILL.md` 에 분석 체크리스트와 코드 실행 규칙을 적어두고, 에이전트가 필요할 때 점진적으로 로드합니다. 모델 호출마다 전체 프롬프트에 옥여넣지 않아 토큰을 절약합니다.

요약

항목	핵심
----	----

백엔드	<code>LocalShellBackend(virtual_mode=True)</code> — 코드 실행 가능
빌트인 도구	<code>execute</code> (코드 실행) + <code>write_todos</code> (계획)
스트리밍	<code>stream(subgraphs=True)</code> — 실행 과정 실시간 관찰
멀티턴	<code>InMemorySaver</code> + 동일 <code>thread_id</code> — 대화 맥락 유지

REFERENCES

- [docs/deepagents/tutorials/data-analysis.md](#)
- [docs/deepagents/06-backends.md](#)

다음 단계: → [04_ml_agent.ipynb](#): 머신러닝 에이전트를 구축합니다.

04 머신러닝 에이전트

CSV 기반 자유 ML 워크플로

학습 목표

LEARNING OBJECTIVES

- `FileSystemBackend` 로 데이터 디렉토리를 설정하고, 에이전트가 자유롭게 파일을 탐색한다
- NB03의 `run_pandas` 패턴을 확장하여 `sklearn`을 포함하는 `run_ml_code` 도구를 만든다
- 에이전트가 빌트인 도구 (`ls`, `read_file`, `glob`)로 데이터를 탐색하고, `run_ml_code` 로 분석한다
- 멀티턴 대화로 EDA → 전처리 → 모델 선택 → 학습 → 평가를 수행한다

개요

항목	NB03 (데이터 분석)	NB04 (머신러닝)
백엔드	<code>LocalShellBackend</code>	<code>FileSystemBackend</code>
데이터	매출 CSV (8행)	사용자 지정 CSV (데모: 유방암 569행)
커스텀 도구	<code>get_csv_path</code> + <code>run_pandas</code>	<code>run_ml_code</code> (<code>sklearn</code> 추가)
빌트인 도구	—	<code>ls</code> , <code>read_file</code> , <code>glob</code> (파일 탐색)
목적	집계, 통계, 추이 분석	EDA → 전처리 → 모델 학습 → 비교

```
from dotenv import load_dotenv
import os

load_dotenv()
assert os.environ.get("OPENAI_API_KEY"), "OPENAI_API_KEY를 .env에 설정하세요"
```

```
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-5.4")
```


NB03 vs NB04: 백엔드와 도구 확장

NB03에서는 `LocalShellBackend` + `run_pandas` 로 pandas 코드를 실행했습니다. NB04에서는 두 가지를 확장합니다:

1. 백엔드: `FilesystemBackend(root_dir=DATA_DIR)` — 에이전트가 빌트인 도구(`ls`, `read_file`, `glob`)로 데이터 디렉토리를 자유롭게 탐색
2. 도구: `run_ml_code` — sklearn을 네임스페이스에 추가하여 ML 파이프라인 실행

```
# NB03: LocalShellBackend + run_pandas
backend = LocalShellBackend(root_dir=tmp_dir, virtual_mode=True)
ns = {"pd": pd, "np": np, "csv_path": csv_path}

# NB04: FilesystemBackend + run_ml_code
backend = FilesystemBackend(root_dir=DATA_DIR, virtual_mode=True)
ns = {"pd": pd, "np": np, "sklearn": sklearn, "DATA_DIR": DATA_DIR}
```

TIP

`FilesystemBackend` 는 `execute` 없이 파일 접근만 제공하므로 `LocalShellBackend` 보다 안전합니다.

1단계: 데이터 디렉토리 설정

`DATA_DIR` 을 변경하면 자신의 CSV 데이터를 사용할 수 있습니다. 에이전트가 `ls`, `glob`, `read_file` 빌트인으로 디렉토리를 탐색하여 어떤 파일이 있는지 파악합니다.

```
# 예시: 자신의 데이터 디렉토리 사용
DATA_DIR = "/path/to/your/data"
```

```
import tempfile
import pandas as pd
from sklearn.datasets import load_breast_cancer

# — 데이터 디렉토리 설정 —————
# 자신의 CSV가 있는 디렉토리로 변경하세요.
# 아래는 데모용으로 breast_cancer 데이터를 CSV로 저장합니다.
DATA_DIR = tempfile.mkdtemp()

# 데모 데이터 생성 (자신의 CSV가 있으면 이 블록을 제거하세요)
data = load_breast_cancer()
df = pd.DataFrame(data.data, columns=data.feature_names)
df["target"] = data.target
df.to_csv(os.path.join(DATA_DIR, "breast_cancer.csv"), index=False)

print(f"DATA_DIR: {DATA_DIR}")
print(f"파일 목록: {os.listdir(DATA_DIR)}")
```

2단계: FilesystemBackend 생성

`FilesystemBackend` 는 `root_dir` 아래의 파일에 대해 빌트인 도구를 제공합니다:

빌트인 도구	역할
ls	디렉토리 목록 조회
read_file	파일 내용 읽기
glob	패턴 기반 파일 검색
write_file	파일 쓰기 (결과 저장)

TIP

virtual_mode=True 로 디렉토리 탈출(.. , ~)을 방지합니다.

```
from deepagents.backends import FilesystemBackend

backend = FilesystemBackend(root_dir=DATA_DIR, virtual_mode=True)
```

3단계: run_ml_code 도구 정의

NB03의 run_pandas 를 확장하여 sklearn 을 네임스페이스에 추가합니다. DATA_DIR 을 네임스페이스에 전달하여, 에이전트가 디렉토리 내 어떤 CSV든 로드할 수 있습니다.

TIP

파일 탐색은 빌트인 ls / read_file 로, 코드 실행은 run_ml_code 로 — 역할 분리

```
from langchain.tools import tool
import io, contextlib

@tool
def run_ml_code(code: str) -> str:
    """sklearn/pandas Python 코드를 실행합니다. print()로 결과를 출력하세요.
    사용 가능: pd, np, sklearn, os. DATA_DIR 변수로 데이터 디렉토리에 접근하세요."""
    import pandas as pd, numpy as np, sklearn
    buf = io.StringIO()
    ns = {"pd": pd, "np": np, "sklearn": sklearn, "os": os, "DATA_DIR": DATA_DIR}
    try:
        with contextlib.redirect_stdout(buf):
            exec(code, ns)
        return buf.getvalue() or "실행 완료 (출력 없음)"
    except Exception as e:
        return f"오류: {e}"
```

4단계: 에이전트 생성

에이전트의 워크플로:

1. 빌트인 `ls / glob` 으로 `DATA_DIR` 내 CSV 파일 탐색
2. 빌트인 `read_file` 로 데이터 미리보기
3. `run_ml_code` 로 EDA → 전처리 → 모델 학습/비교

미들웨어	역할
<code>ToolRetryMiddleware</code>	도구 실패 시 자동 재시도 (최대 2회)
<code>ModelCallLimitMiddleware</code>	무한 루프 방지 — 최대 20회 모델 호출 제한

```

from deepagents import create_deep_agent
from langgraph.checkpoint.memory import InMemorySaver
from langchain.agents.middleware import (
    ToolRetryMiddleware,
    ModelCallLimitMiddleware,
)
from prompts import ML_AGENT_PROMPT

ml_agent = create_deep_agent(
    model=model,
    tools=[run_ml_code],
    system_prompt=ML_AGENT_PROMPT,
    backend=backend,
    skills=["/skills/"],
    checkpointer=InMemorySaver(),
    middleware=[
        ToolRetryMiddleware(max_retries=2),
        ModelCallLimitMiddleware(run_limit=20),
    ],
)

```

5단계: 파일 탐색 + EDA 분석

에이전트에게 데이터 디렉토리를 탐색하고 분석하도록 요청합니다. 에이전트는 빌트인 `ls` 로 파일 목록을 확인한 뒤, `run_ml_code` 로 EDA를 수행합니다.

6단계: 모델 학습 + 비교

에이전트에게 적절한 모델 3개 이상을 학습하고 교차 검증으로 성능을 비교하도록 요청합니다. 에이전트가 스스로 알고리즘을 선택합니다.

7단계: 멀티턴 후속 — Feature Importance 분석

같은 `thread_id` 를 써서 이전 대화의 맥락을 유지한 채 후속 분석을 요청합니다.

8단계: 스트리밍 — 추가 분석

`stream(subgraphs=True)` 으로 에이전트의 실행 과정을 실시간으로 관찰합니다.

@tool + scikit-learn 패턴

`run_ml_code` 는 단일 `@tool` 안에 `pandas`, `numpy`, `sklearn` 네임스페이스를 모두 노출합니다. 에이전트는 도구 호출 한 번으로 EDA부터 학습·평가까지 연속된 `sklearn` 파이프라인을 실행할 수 있습니다.

```
@tool
def run_ml_code(code: str) -> str:
    """pandas + numpy + sklearn 코드를 실행합니다."""
    import pandas as pd, numpy as np, sklearn
    ns = {"pd": pd, "np": np, "sklearn": sklearn, "os": os, "DATA_DIR": DATA_DIR}
    # ... print 캡처 후 결과 반환
```

TIP

- 빌트인 `ls` / `read_file` → 데이터 탐색
- `run_ml_code` → 코드 실행
- 책임을 분리하면 에이전트가 어떤 도구를 언제 호출해야 하는지 명확해집니다.

요약

항목	핵심
백엔드	<code>FilesystemBackend(root_dir=DATA_DIR)</code> — 사용자 데이터 디렉토리 설정
빌트인 도구	<code>ls</code> , <code>read_file</code> , <code>glob</code> — 파일 탐색
커스텀 도구	<code>run_ml_code</code> (<code>pandas</code> + <code>numpy</code> + <code>sklearn</code>) — ML 코드 실행
워크플로	파일 탐색 → EDA → 전처리 → 모델 선택 → 교차 검증 비교
멀티턴	<code>InMemorySaver</code> + 동일 <code>thread_id</code> — 대화 맥락 유지

자신의 데이터 사용하기

```
# 1단계 셀에서 DATA_DIR만 변경하면 됩니다
DATA_DIR = "/path/to/your/data" # CSV 파일이 있는 디렉토리
```

에이전트가 `ls` 로 파일을 탐색하고, `run_ml_code` 로 자유롭게 분석합니다.

REFERENCES

- `docs/deepagents/06-backends.md`
- `docs/deepagents/tutorials/data-analysis.md`
- [scikit-learn 공식 문서](#)

다음 단계: → `05_deep_research_agent.ipynb`: 딥 리서치 에이전트를 구축합니다.

05 딥 리서치 에이전트

병렬 서브에이전트와 5단계 워크플로

학습 목표

LEARNING OBJECTIVES

- 병렬 서브에이전트 3개(researcher-1, researcher-2, fact-checker)를 구성한다
- `think_tool` 로 전략적 반성(strategic reflection)을 구현한다
- 5단계 워크플로(Plan → Delegate → Synthesize → Verify → Report)를 설계한다
- v1 미들웨어(SummarizationMiddleware, ModelCallLimitMiddleware, ModelFallbackMiddleware)를 적용한다

개요

항목	내용
프레임워크	Deep Agents
핵심 컴포넌트	병렬 서브에이전트 3개, <code>think_tool</code>
워크플로	5단계: Plan → Delegate → Synthesize → Verify → Report
백엔드	<code>FilesystemBackend(root_dir=".", virtual_mode=True)</code>
빌트인 도구	<code>write_todos</code> (계획), <code>task</code> (서브에이전트 호출)
스킬	<code>skills/deep-research/SKILL.md</code> — 리서치 방법론 + 인용 규칙

```
from dotenv import load_dotenv
import os

load_dotenv()
assert os.environ.get("OPENAI_API_KEY"), "OPENAI_API_KEY를 .env에 설정하세요"
```

```
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-5.4")
```

1단계: think_tool — 전략적 반성 도구

`think_tool` 은 에이전트가 행동하기 전에 “생각”을 기록하는 도구입니다. 의사결정 품질을 높이는 데 씁니다:

- 검색 결과를 분석하고 다음 행동을 계획
- 수집된 정보의 충분성을 평가
- 서브에이전트에게 위임할 작업을 구체화

```
from langchain.tools import tool

@tool
def think_tool(thought: str) -> str:
    """전략적 반성 - 현재 상황을 분석하고 다음 행동을 계획합니다."""
    return f"Reflection recorded: {thought}"
```

2단계: web_search 도구 (간소화)

실제 딥 리서치에서는 Tavily API를 사용하지만, 여기서는 학습 목적으로 간소화된 검색 도구를 정의합니다.

```
@tool
def web_search(query: str) -> str:
    """웹 검색을 수행합니다 (시뮬레이션)."""
    results = {
        "AI agent": "AI 에이전트는 자율적으로 작업을 수행하는 시스템입니다. 2024년 이후 급성장 중입니다.",
        "LangGraph": "LangGraph는 상태 기반 워크플로 프레임워크입니다. Graph API와 Functional API를 지원합니다.",
        "Deep Agents": "Deep Agents는 올인원 에이전트 SDK입니다. 서브에이전트, 백엔드, 스킬을 지원합니다.",
    }
    for key, val in results.items():
        if key.lower() in query.lower():
            return val
    return f"'{query}'에 대한 검색 결과: 관련 정보를 찾을 수 없습니다."
```

3단계: 5단계 리서치 워크플로 프롬프트

`prompts.load_prompt` 가 프롬프트를 로드합니다 (LangSmith Hub → Langfuse → 기본값).

단계	이름	설명
1	Plan	<code>write_todos</code> 로 리서치 계획 작성
2	Delegate	서브에이전트에게 병렬 조사 위임 (최대 3개 동시)
3	Synthesize	수집된 정보를 통합
4	Verify	fact-checker가 사실 검증

5	Report	최종 보고서 작성
---	--------	-----------

```
from prompts import RESEARCH_AGENT_PROMPT

print(RESEARCH_AGENT_PROMPT)
```

Prompt 'rag-agent-label:production' not found during refresh, evicting from cache.

Prompt 'sql-agent-label:production' not found during refresh, evicting from cache.

Prompt 'data-analysis-agent-label:production' not found during refresh, evicting from cache.

Prompt 'ml-agent-label:production' not found during refresh, evicting from cache.

Prompt 'deep-research-agent-label:production' not found during refresh, evicting from cache.

당신은 박사급 딥 리서치 에이전트입니다.

워크플로

1. **Plan**: write_todos로 리서치 계획을 세우세요
2. **Delegate**: 서브에이전트에게 조사를 위임하세요 (비교 분석 시 병렬)
3. **Synthesize**: 수집된 정보를 통합하세요
4. **Verify**: fact-checker에게 사실 검증을 요청하세요
5. **Report**: 최종 보고서를 작성하세요

규칙

- 검색 후 반드시 think_tool로 반성하세요
- 서브에이전트는 최대 3개까지 병렬 실행
- 인용은 [1], [2] 형식으로, 출처 섹션을 포함하세요
- 단순 주제는 서브에이전트 1개, 비교 분석은 2-3개 사용하세요

4단계: 서브에이전트 3개 정의

딥 리서치 에이전트는 3개의 전문 서브에이전트를 씁니다:

서브에이전트	역할	도구
researcher-1	주제 조사 담당	web_search, think_tool
researcher-2	비교/보완 조사	web_search, think_tool
fact-checker	사실 검증 담당	web_search

```
researcher_1 = {
    "name": "researcher-1",
    "description": "주제에 대한 심층 조사를 수행합니다",
    "system_prompt": "당신은 리서치 전문가입니다. 주제를 깊이 조사하고 핵심 정보를 요약하세요. 검색 후 think_tool로 반성하세요.",
    "tools": [web_search, think_tool],
}
```

```
researcher_2 = {
    "name": "researcher-2",
    "description": "보완적 관점에서 추가 조사를 수행합니다",
    "system_prompt": "당신은 보완 리서처입니다. 다른 관점에서 추가 정보를 수집하세요. 검색 후 think_tool로 반성하세요.",
    "tools": [web_search, think_tool],
}
```

```
fact_checker = {
    "name": "fact-checker",
    "description": "수집된 정보의 사실 여부를 검증합니다",
    "system_prompt": "당신은 팩트체커입니다. 제공된 정보의 정확성을 검증하고, 오류가 있으면 지적하세요.",
    "tools": [web_search],
}
```

5단계: 딥 리서치 에이전트 생성 (v1 미들웨어)

모든 도구와 서브에이전트를 조합해 최종 에이전트를 생성합니다. v1 미들웨어로 안정성과 신뢰성을 끌어올립니다.

미들웨어	역할
SummarizationMiddleware	긴 리서치 대화를 자동 요약해 컨텍스트 절약
ModelCallLimitMiddleware	리서치 루프 방지 – 최대 30회 모델 호출 제한
ModelFallbackMiddleware	주 모델 실패 시 백업 모델로 자동 전환

`InMemorySaver` 로 체크포인트를 활성화하면 중단된 리서치를 재개할 수 있습니다.

```
from deepagents import create_deep_agent
from deepagents.backends import FilesystemBackend
from langgraph.checkpoint.memory import InMemorySaver
from langchain.agents.middleware import (
    SummarizationMiddleware,
    ModelCallLimitMiddleware,
    ModelFallbackMiddleware,
)

research_agent = create_deep_agent(
    model=model,
    tools=[web_search, think_tool],
    subagents=[researcher_1, researcher_2, fact_checker],
    system_prompt=RESEARCH_AGENT_PROMPT,
    backend=FilesystemBackend(root_dir=".", virtual_mode=True),
    skills=["/skills/"],
    checkpointer=InMemorySaver(),
    middleware=[
        SummarizationMiddleware(model=model, trigger=("messages", 15)),
        ModelCallLimitMiddleware(run_limit=30),
        ModelFallbackMiddleware("gpt-5.4-mini"),
    ],
)
```

DeepAgents 패턴 – TodoList + dispatch + 5-tool async

본 챕터의 딥 리서치 에이전트는 DeepAgents의 핵심 세 가지 패턴을 모두 사용합니다.

패턴	설명
TodoList	빌트인 <code>write_todos</code> 로 Plan 단계에서 작업 목록을 명시 – 진행 상태가 메시지 히스토리에 박힘

dispatch	<code>task</code> 빌트인으로 서브에이전트에 위임 – 메인 에이전트는 결과 요약만 받아 컨텍스트 분리
5-tool async	<code>web_search</code> · <code>think_tool</code> · <code>write_todos</code> · <code>task</code> · <code>read/write_file</code> 다섯 도구로 비동기 워크플로 구성

TIP

`AsyncSubAgent` 스펙을 `subagents`에 넘기면 `create_deep_agent`가 `AsyncSubAgentMiddleware`를 자동 부착해 서브에이전트를 병렬 비동기로 실행합니다 ([docs/deepagents/12-async-subagents.md](#)).

6단계: 리서치 실행

에이전트에게 리서치 주제를 부여하면 5단계 워크플로를 자동으로 수행합니다.

7단계: 스트리밍 — 네임스페이스 추적

`stream(subgraphs=True)`로 메인 에이전트와 서브에이전트의 실행 과정을 네임스페이스별로 추적합니다. 어떤 서브에이전트가 언제 호출되는지 실시간으로 확인할 수 있습니다.

서브에이전트 설계 모범 사례

원칙	설명
명확한 설명	<code>description</code> 을 구체적으로 작성 – 메인 에이전트가 위임 대상을 선택하는 기준
전문 프롬프트	<code>system_prompt</code> 에 출력 형식, 제약, 워크플로 포함
최소 도구	필요한 도구만 할당 – 불필요한 도구는 혼란 유발
간결한 결과	서브에이전트가 요약을 반환하도록 지시 – 원시 데이터 전달 금지

요약

항목	핵심
<code>think_tool</code>	전략적 반성 – 검색 후 분석, 다음 행동 계획
서브에이전트	<code>researcher-1</code> , <code>researcher-2</code> , <code>fact-checker</code> 병렬 실행

워크플로	Plan → Delegate → Synthesize → Verify → Report
컨텍스트 관리	서브에이전트 결과만 메인에 전달 – 중간 과정 격리

REFERENCES

- docs/deepagents/examples/02-deep-research.md
- docs/deepagents/07-subagents.md
- docs/deepagents/06-backends.md

이전 단계: ← [04_ml_agent.ipynb](#): 머신러닝 에이전트

P A R T

VIII

Integrations

LangChain 생태계 통합

이 파트에서 다루는 내용

- 1. 통합 카테고리 개요
- 2. Anthropic Prompt Caching
 - 3. Claude Bash Tool
 - 4. Claude Text Editor
 - 5. Claude Memory
- 6. Anthropic File Search
- 7. Bedrock Prompt Caching
- 8. OpenAI Moderation

01 통합 카테고리 개요

LangChain 생태계 지도

LangChain·LangGraph·Deep Agents는 공통 에이전트 인터페이스 아래 프로바이더별 구현을 플러그인 형태로 연결합니다. Part II Part IV가 “에이전트를 어떻게 쓰는가”를 다뤘다면, 이 Part는 “에이전트를 무엇과 연결하는가”를 살핍니다. 13개 카테고리의 통합 표면을 한눈에 파악하고, 각 카테고리의 대표 패키지와 선택 기준을 정리합니다.

학습 목표

LEARNING OBJECTIVES

- LangChain 생태계의 13개 통합 카테고리를 조감한다
- Provider Middleware가 왜 별도 카테고리로 분리되는지 이해한다
- 벤더 선택이 필요한 영역(Chat Models, Vector Stores)과 표준화된 영역(Middleware, Checkpointers)을 구분한다
- 이 Part에서 다루는 7개 Provider Middleware 장의 맵을 그린다

1.1 기준 버전 스냅샷

이 Part의 코드는 다음 버전을 전제로 합니다. 최신 릴리스는 [docs/skills/langchain-dependencies.md](#) 로 확인하세요.

- `langchain` 1.3+ (event streaming v3)
- `langgraph` 1.2+ (per-node timeout · `DeltaChannel` · graceful shutdown)
- `deepagents` 0.6.1+ (`CodeInterpreterMiddleware` , async subagents, interpreter snapshots)

! WARNING

LangChain 1.2 `create_agent` 는 같은 미들웨어 클래스의 중복 인스턴스를 거부합니다.

`AssertionError: Please remove duplicate middleware instances.` 가 발생하면 서브클래싱으로 두 인스턴스를 서로 다른 타입으로 분리하세요.

1.2 13개 통합 카테고리

카테고리	대표 패키지	선택 기준
Chat Models	langchain-openai , langchain-anthropic , langchain-google-genai , langchain-aws	모델 성능·비용·리전 요건
Embeddings	langchain-openai , langchain-cohere , langchain-voyageai	도메인 벤치마크·라이선스·다국어 지원
Vector Stores	langchain-chroma , langchain-pinecone , langchain-pgvector	운영 규모·메타데이터 필터링·관리형 여부
Document Loaders	langchain-community.document_loaders	소스 포맷(PDF, Notion, Slack 등)
Retrievers	langchain.retrievers , 벤더 특화 리트리버	하이브리드 검색·MMR·재랭커 통합
Text Splitters	langchain-text-splitters	언어·구조(코드 vs 프로즈)·오버랩 전략
Tools	langchain-community.tools , MCP 도구	외부 API 연동·인증 방식·호출 제약
Checkpointers	langgraph-checkpoint-postgres , -sqlite	내구성 요구·멀티 테넌트·백업 전략
Stores	langgraph-store-postgres , 벤더 관리형	영속 메모리 규모·검색 방식(키/벡터)
Sandboxes	langchain-daytona , langchain-modal , langchain-runloop	격리 수준·콜드 스타트·비용
Provider Middleware	langchain-anthropic , langchain-aws , langchain-openai	프로바이더 고유 기능(캐시·네이티브 도구·정책)
Observability	langsmith , langfuse , OpenTelemetry	SaaS vs 자체 호스팅·PII 정책
Providers (Hosted Runtime)	LangGraph Platform, Anthropic Skills, OpenAI Agents	관리형 런타임 vs 자체 호스팅·SDK 호환성

Provider Middleware는 프로바이더 서버 측에서 활성화되는 기능(프롬프트 캐시, 네이티브 도구, 콘텐츠 정책)을 LangChain 미들웨어 포맷으로 감싼 것입니다. 공식 문서에는 한 줄 정도만 언급되는 경우가 많아 실행 가능한 코드로 정리해 두는 편이 실용적입니다. 이 Part의 ch2 ch8에서 7개 미들웨어를 각각 1장씩 다룹니다.

1.3 Provider Middleware 로드맵

장	미들웨어	효과
2	<code>AnthropicPromptCachingMiddleware</code>	긴 시스템 프롬프트/도구 정의 서버 측 캐시 (5m/1h)
3	<code>ClaudeBashToolMiddleware</code>	Claude 네이티브 bash 도구 + 실행 정책 3종
4	<code>StateClaudeTextEditorMiddleware</code> / <code>FilesystemClaudeTextEditorMiddleware</code>	네이티브 text editor 6개 오퍼레이션
5	<code>StateClaudeMemoryMiddleware</code> / <code>FilesystemClaudeMemoryMiddleware</code>	<code>/memories/*</code> 경로 계약으로 모델 자기 메모
6	<code>StateFileSearchMiddleware</code>	상태 안 가상 파일의 glob/grep 검색
7	<code>BedrockPromptCachingMiddleware</code>	AWS Bedrock 경유 Claude/Nova 캐시
8	<code>OpenAIModerationMiddleware</code>	OpenAI Moderation API로 사전/사후 검사

1.4 공통 패턴

이 Part의 모든 장은 세 단계 루프를 공유합니다.

1. **환경 설정** — `.env` 에 해당 프로바이더 키를 넣고 패키지를 설치합니다.
2. **기본 사용** — `create_agent(..., middleware=[Middleware()])` 한 줄로 기능을 켭니다.
3. **검증** — `usage_metadata`, `tool_calls`, 또는 Moderation 응답을 읽어 기능이 실제로 동작하는지 확인합니다.

TIP

프로바이더 특화 미들웨어는 모델을 같이 정해야 합니다. Anthropic 미들웨어를 OpenAI 모델에 붙이면 `unsupported_model_behavior` 설정에 따라 경고만 나오거나 예외가 발생합니다. 멀티 프로바이더 파이프라인에서는 `ModelFallbackMiddleware` 와 조합할 때 순서에 주의하세요.

1.5 Observability 환경변수 표준화

LangChain 1.1부터 트레이싱 관련 환경변수는 `LANGCHAIN_*` → `LANGSMITH_*` 로 표준화되었습니다. 옛 변수도 호환을 위해 일정 기간 동작하지만, 신규 코드는 새 이름만 씁니다.

변수	역할	비고
LANGSMITH_TRACING	트레이스 활성화 (<code>true</code> / <code>false</code>)	모든 <code>create_agent</code> 실행 자동 계측
LANGSMITH_API_KEY	LangSmith 계정 키	<code>ls__</code> 접두사
LANGSMITH_PROJECT	트레이스 분류 프로젝트 이름	<code>tracing_context</code> 로 호출 구간별 <code>override</code> 가능
LANGSMITH_ENDPOINT	self-host LangSmith용 엔드포인트	선택 – SaaS 사용 시 불필요

핵심 정리

- 통합은 12개 카테고리로 정리되며, 각 카테고리는 “프로바이더 플러그인 + 공통 인터페이스” 구조를 공유한다
- Provider Middleware는 프로바이더 서버 측 고유 기능을 LangChain 미들웨어로 감싼 영역
- 이 Part의 2 8장이 7개 Provider Middleware를 1장씩 실행 가능한 코드로 다룬다
- 중복 인스턴스 제약, 모델 매칭, 멀티 프로바이더 폴백은 Provider Middleware 공통 주의점

02 Anthropic Prompt Caching

서버 측 캐시로 입력 토큰 90% 절감

Claude 모델 전용 프롬프트 캐시 미들웨어 `AnthropicPromptCachingMiddleware` 는 긴 system prompt, 도구 정의, 초기 대화 컨텍스트를 Anthropic 서버 측에 5분 1시간 캐시해 입력 토큰 비용과 지연을 크게 줄입니다. 같은 에이전트가 같은 시스템 프롬프트로 여러 번 호출될 때, 두 번째 호출부터는 입력 토큰 단가가 약 90% 할인됩니다.

학습 목표

LEARNING OBJECTIVES

- 4개 파라미터(`type` , `tll` , `min_messages_to_cache` , `unsupported_model_behavior`)를 이해한다
- `usage_metadata.input_token_details` 에서 `cache_creation` / `cache_read` 를 읽어 캐시 적중을 확인한다
- 1시간 TTL beta와 5분 기본값의 선택 기준을 안다
- 비(非) Anthropic 모델에 적용할 때 `warn` / `ignore` / `raise` 동작을 구분한다

2.1 언제 쓰나

- 긴 system prompt(수천 토큰의 기업 정책, 스타일 가이드)를 매 턴 전송하는 에이전트
- 큰 tool 정의 목록(20개 이상의 JSON schema)을 반복 전송하는 구성
- RAG 컨텍스트 재사용: 같은 문서 묶음을 여러 질의에 걸쳐 사용
- 멀티턴 대화에서 앞부분 컨텍스트가 안정적인 경우

2.2 환경 설정

필요 패키지: `langchain` , `langchain-anthropic` . `.env` 에 `ANTHROPIC_API_KEY` 가 있어야 합니다.

```
from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain_anthropic.middleware import AnthropicPromptCachingMiddleware

load_dotenv()
```


2.3 기본 사용

미들웨어를 `create_agent` 의 `middleware` 리스트에 넣으면 LangChain이 system prompt · tool 정의 · 마지막 메시지 위치에 `cache_control: {"type": "ephemeral"}` 마커를 자동 부착합니다.

```
long_system_prompt = (
    "You are a senior software engineer... " * 200 # 약 2,000 토큰 가정
)

agent = create_agent(
    model="anthropic:claude-sonnet-4-6",
    system_prompt=long_system_prompt,
    tools=[],
    middleware=[AnthropicPromptCachingMiddleware()],
)
```

2.4 캐시 적중 확인

같은 에이전트를 연속 두 번 호출하고 `usage_metadata` 를 읽으면 캐시 적중 여부가 드러납니다. 1회차는 `cache_creation_input_tokens` 가, 2회차 이후는 `cache_read_input_tokens` 가 커집니다.

```
for i in range(2):
    result = agent.invoke(
        {"messages": [{"role": "user", "content": f"요청 {i}"}]},
    )
    last = result["messages"][-1]
    print(i, last.usage_metadata.get("input_token_details"))
```

```
0 {'cache_creation': 2048, 'cache_read': 0}
1 {'cache_creation': 0, 'cache_read': 2048}
```

2.5 파라미터 전체

파라미터	기본값	설명
<code>type</code>	"ephemeral"	현재 Anthropic이 지원하는 유일한 타입
<code>ttr</code>	"5m"	캐시 유지 시간. "5m" 또는 "1h" . 1시간은 Anthropic beta
<code>min_messages_to_cache</code>	0	메시지 수가 이 값 이상일 때만 <code>cache_control</code> 부착
<code>unsupported_model_behavior</code>	"warn"	비 Anthropic 모델 적용 시 "warn" / "ignore" / "raise"

1시간 TTL 예

```
AnthropicPromptCachingMiddleware(
    ttl="1h",
    min_messages_to_cache=2,
)
```

1시간 캐시는 긴 상담 세션이나 일간 배치 분석처럼 같은 시스템 프롬프트로 수 시간 반복 호출되는 경우에 맞습니다. 5분 기본값은 실시간 챗봇의 연속 입력용입니다.

2.6 비 Anthropic 모델 동작

이 미들웨어는 내부적으로 `cache_control` 블록을 주입하므로, 다른 공급자 모델에서는 무시되거나 에러로 이어집니다. 모델을 교체할 수 있는 파이프라인이라면 `unsupported_model_behavior` 를 명시하세요.

```
# 경고만 남기고 계속
AnthropicPromptCachingMiddleware(unsupported_model_behavior="warn")

# 조용히 스킵
AnthropicPromptCachingMiddleware(unsupported_model_behavior="ignore")

# 즉시 실패 (CI/테스트용)
AnthropicPromptCachingMiddleware(unsupported_model_behavior="raise")
```

2.7 UsageMetadataCallbackHandler 로 집계

스레드 전체의 캐시 적중률을 한 번에 집계하려면 `langchain.callbacks.UsageMetadataCallbackHandler` 를 붙입니다. 매 LLM 호출의 `usage_metadata` 가 누적되어 `cache_creation` / `cache_read` 까지 그대로 보존됩니다.

```
from langchain.callbacks import UsageMetadataCallbackHandler

handler = UsageMetadataCallbackHandler()
agent.invoke(
    {"messages": [{"role": "user", "content": "..."}]},
    config={"callbacks": [handler]},
)
print(handler.usage_metadata) # {model_name: {input/output/cache_creation/cache_read}}
```

TIP

Anthropic 응답을 `output_version="v1"` (`ChatAnthropic(..., output_version="v1")`)로 받으면 `cache_creation` / `cache_read` 키가 메시지 본문이 아닌 표준화된 `usage_metadata.input_token_details` 안으로 정리됩니다 – 멀티 프로바이더 집계 코드가 단순해집니다.

2.8 Bedrock과의 차이

AWS Bedrock 경유로 Claude를 호출할 때는 `BedrockPromptCachingMiddleware` (ch7)를 씁니다. 파라미터 이름과 의미는 거의 같지만, 대상 패키지과 TTL 제약(Nova 모델은 5m만 지원, tool 정의 캐시 미지원)이 다릅니다.

핵심 정리

- 긴 system prompt·tool 정의를 반복 전송하는 에이전트에서 입력 토큰 비용을 90% 절감
- `cache_creation_input_tokens` → `cache_read_input_tokens` 이동으로 캐시 적중 검증
- TTL `"5m"` 은 실시간 대화, `"1h"` 는 장시간 배치·상담 세션용
- 멀티 프로바이더 파이프라인은 `unsupported_model_behavior="warn"` 을 명시해 조용한 실패 방지

03 Claude Bash Tool

네이티브 bash_20250124 + 실행 정책

ClaudeBashToolMiddleware 는 Claude 모델에 Anthropic 네이티브 bash_20250124 도구를 주입합니다. 범용 ShellToolMiddleware 와 달리 Anthropic 서버가 bash 호출 스키마를 직접 관리하므로 프롬프트가 간결하고 tool schema 토큰이 거의 0에 수렴합니다. 이 장에서는 3종 실행 정책(Host/Docker/CodexSandbox)의 트레이드오프를 비교하고, 민감정보 마스킹을 위한 redaction_rules 까지 다룹니다.

학습 목표

LEARNING OBJECTIVES

- ClaudeBashToolMiddleware 의 4개 주요 파라미터를 이해한다
- HostExecutionPolicy / DockerExecutionPolicy / CodexSandboxExecutionPolicy 의 격리 수준을 구분한다
- RedactionRule 로 출력에서 토큰·키를 마스킹한다
- Claude 네이티브 bash와 범용 shell 도구의 차이를 안다

3.1 언제 쓰나

- Claude에게 실제 셸 명령을 실행시켜 코드 실행, 파일 조사, 빌드를 맡길 때
- Deep Agents처럼 장시간 작업 공간에서 세션 상태(cwd, 환경변수)를 유지해야 할 때
- 격리된 Docker 컨테이너에서 안전하게 임의 코드를 실행하고 싶을 때
- 셸 출력에 API 키·자격증명이 섞일 가능성이 있어 출력 리다이렉션이 필요할 때

3.2 환경 설정

필요 패키지: langchain , langchain-anthropic . Docker 정책 예시를 실행하려면 로컬에 Docker 데몬이 떠 있어야 합니다.

```
from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain_anthropic.middleware import ClaudeBashToolMiddleware

load_dotenv()
```

3.3 호스트 실행 정책 (가장 단순)

`HostExecutionPolicy` 는 로컬 프로세스에서 명령을 실행합니다. 빠르고 설정이 간단하지만 격리가 없다 — 네트워크, 파일시스템, 환경변수에 그대로 접근하므로 신뢰할 수 있는 스크립트나 로컬 개발 용도로만 씁니다.

```
from langchain.agents.middleware import HostExecutionPolicy

agent = create_agent(
    model="anthropic:claude-sonnet-4-6",
    middleware=[
        ClaudeBashToolMiddleware(
            workspace_root="/tmp/work",
            startup_commands=["export PATH=$HOME/.local/bin:$PATH"],
            execution_policy=HostExecutionPolicy(),
        ),
    ],
)
```

- `workspace_root` : 셸 세션의 기본 디렉터리
- `startup_commands` : 세션 시작 시 자동 실행되는 명령들 (PATH 설정, venv 활성화 등)

3.4 Docker 실행 정책 (권장, 격리)

`DockerExecutionPolicy` 는 명령을 컨테이너 안에서 실행합니다. 호스트 파일시스템과 네트워크에서 완전히 분리되므로, 모델이 만든 임의 코드를 실행할 때는 사실상 이 정책을 기본으로 써야 합니다.

```
from langchain.agents.middleware import DockerExecutionPolicy

agent = create_agent(
    model="anthropic:claude-sonnet-4-6",
    middleware=[
        ClaudeBashToolMiddleware(
            execution_policy=DockerExecutionPolicy(image="python:3.11"),
            startup_commands=["pip install requests numpy"],
        ),
    ],
)
```

컨테이너는 첫 `bash` 호출 시 생성되고 세션이 끝나면 정리됩니다. 패키지가 필요하다면 `startup_commands` 에 `pip install ...` 을 넣습니다.

3.5 Codex 샌드박스 정책

`CodexSandboxExecutionPolicy` 는 Anthropic이 제공하는 샌드박스 러너에서 실행합니다. 로컬 Docker 없이 격리가 필요할 때 쓰는 대안입니다. 네트워크 화이트리스트와 리소스 한도가 기본값으로 강하게 잡혀 있습니다.

3.6 출력 리다이렉션 (`redaction_rules`)

셸 출력에 API 키·토큰·이메일 같은 민감정보가 흘러나올 수 있습니다. `RedactionRule` 을 리스트로 넘겨 도구 응답을 에이전트 컨텍스트에 들어가기 전에 마스킹합니다. LangChain 1.2부터 `RedactionRule` 은 `PIIMiddleware`

와 같은 (pii_type, strategy, detector) 시그니처를 씁니다 – 과거 pattern= / replacement= 인자는 제거되었습니다.

```
from langchain.agents.middleware import RedactionRule

agent = create_agent(
    model="anthropic:claude-sonnet-4-6",
    middleware=[
        ClaudeBashToolMiddleware(
            execution_policy=DockerExecutionPolicy(),
            redaction_rules=[
                RedactionRule(
                    pii_type="openai_api_key",
                    strategy="redact",
                    detector=r"sk-[a-zA-Z0-9]{32,}",
                ),
                RedactionRule(
                    pii_type="github_token",
                    strategy="redact",
                    detector=r"ghp_[a-zA-Z0-9]{36,}",
                ),
            ],
        ),
    ],
)
```

3.7 일반 @tool 로 대체할 때

Claude 외 모델에서도 같은 흐름을 쓰려면 LangChain 1.3의 @tool 데코레이터로 직접 셀 함수를 정의합니다. extras 필드(LangChain 1.2부터 도입)에 Anthropic programmatic tool calling 메타데이터를 주입하면 동일 함수를 Claude 전용 도구와 같은 톤으로 호출할 수 있습니다.

```
from langchain_core.tools import tool

@tool(extras={"anthropic_cache_control": {"type": "ephemeral"}})
def run_bash(command: str) -> str:
    """격리된 컨테이너에서 bash 명령을 실행하고 stdout/stderr를 반환합니다."""
    return run_in_container(command) # 자체 구현체
```

➡ TIP

@tool(extras=...) 은 멀티 프로바이더 파이프라인의 부분 캐싱에 유용합니다. Claude는 cache_control 을 해석하고, 다른 프로바이더는 무시하므로 한 코드베이스를 그대로 양쪽에 노출할 수 있습니다.

3.8 네이티브 vs 범용 비교

항목	ClaudeBashToolMiddleware	범용 ShellToolMiddleware
도구 타입	Anthropic 네이티브 bash_20250124	일반 @tool 함수
지원 모델	Claude 전용	모든 모델

tool schema 토큰	서버 측 → 거의 0	매 턴 전송
세션 상태	cwd, env 누적 유지	구현에 따라 다름
실행 정책 API	공유 (<code>HostExecutionPolicy</code> 등)	공유

선택 기준: Claude만 쓴다면 네이티브가 싸고 깔끔합니다. 멀티 프로바이더 파이프라인이라면 범용 `ShellToolMiddleware` 로 통일합니다.

핵심 정리

- Claude 네이티브 bash 도구는 tool schema 토큰을 거의 0으로 만든다
- 실행 정책은 Host(격리 없음) → Docker(권장) → CodexSandbox(Anthropic 관리) 세 층으로 선택
- `redaction_rules` 로 도구 응답의 민감정보를 에이전트 컨텍스트 진입 전에 마스킹
- 멀티 프로바이더 파이프라인에서는 범용 `ShellToolMiddleware` 로 통일하는 것이 안전

04 Claude Text Editor

State vs Filesystem 변형

Claude 네이티브 `text_editor_20250728` 도구는 `view` / `create` / `str_replace` / `insert` / `delete` / `rename` 6가지 오퍼레이션을 지원합니다. LangChain은 이 도구를 두 변형 미들웨어로 제공합니다 – State 변형은 LangGraph 상태 안의 가상 파일로, Filesystem 변형은 실제 디렉터리에 씁니다. 산출물이 스레드 내에서만 쓰이는지, 디스크에 남아야 하는지로 선택합니다.

학습 목표

LEARNING OBJECTIVES

- Claude 네이티브 text editor의 6가지 오퍼레이션을 사용한다
- State 변형과 Filesystem 변형의 수명·공유 범위 차이를 구분한다
- `allowed_path_prefixes` / `allowed_prefixes` 로 접근 경로를 제한한다
- `root_path` , `max_file_size_mb` 로 디스크 변형의 경계를 설정한다

4.1 언제 쓰나

- 코드/문서 편집을 모델이 멀티스텝으로 수행해야 할 때 (diff를 한 번에 뽑기 어려운 큰 변경)
- 결과물이 그래프 상태 안에서만 살아 있으면 되는 경우: State 변형
- 결과물을 실제 리포지터리/디렉터리에 남겨야 하는 경우: Filesystem 변형
- 일반 `@tool` 로 `read_file` / `write_file` 을 짜는 대신 Claude가 학습해둔 도구 스키마를 재사용하고 싶을 때

4.2 환경 설정

필요 패키지: `langchain` , `langchain-anthropic` . `.env` 에 `ANTHROPIC_API_KEY` .

```
from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain_anthropic.middleware import (
    StateClaudeTextEditorMiddleware,
    FilesystemClaudeTextEditorMiddleware,
)
```



```
load_dotenv()
```

4.3 State 변형 — 그래프 상태 안 가상 파일

`StateClaudeTextEditorMiddleware` 는 파일 내용을 LangGraph 상태의 `text_editor_files` 키에 저장합니다. 디스크에 쓰지 않으므로 스레드가 끝나면 사라지는 임시 작업에 어울립니다.

```
agent = create_agent(
    model="anthropic:claude-sonnet-4-6",
    middleware=[
        StateClaudeTextEditorMiddleware(
            allowed_path_prefixes=["/src"], # 가상 경로 제한
        ),
    ],
)

result = agent.invoke(
    {"messages": [{"role": "user", "content": "/src/hello.py에 hello world를 써줘"}]},
)
print(result.get("text_editor_files"))
```

- `allowed_path_prefixes` : 접근 허용 가상 경로 접두사. 미지정 시 전체 허용

TIP

`allowed_path_prefixes` 밖 경로로 쓰려고 하면 도구가 에러를 반환하고, 모델은 그 에러를 읽은 뒤 허용 경로로 재시도합니다. 경로 제한은 에이전트 자율성과 안전의 타협점입니다.

4.4 Filesystem 변형 — 실제 디스크

`FilesystemClaudeTextEditorMiddleware` 는 실제 디렉터리를 루트로 삼아 파일을 읽고 씁니다.

파라미터	기본값	설명
<code>root_path</code>	(필수)	파일 오퍼레이션의 실제 루트 디렉터리
<code>allowed_prefixes</code>	<code>["/"]</code>	허용 가상 경로 접두사 (<code>root_path</code> 기준 상대 경로)
<code>max_file_size_mb</code>	10	읽기/쓰기 허용 최대 파일 크기

```
agent = create_agent(
    model="anthropic:claude-sonnet-4-6",
    middleware=[
        FilesystemClaudeTextEditorMiddleware(
            root_path="/tmp/editor-demo",
            allowed_prefixes=["/drafts", "/reports"],
            max_file_size_mb=5,
        ),
    ],
)
```

```
1,  
)
```

4.5 State vs Filesystem 선택 기준

축	State 변형	Filesystem 변형
저장소	LangGraph 상태 dict	실제 디렉터리
수명	스레드와 함께 소멸	영구 (디스크에 남음)
체크포인트 호환	상태에 포함 → 자동 저장	경로만 저장, 파일은 별도 관리
동시성	스레드별 완전 격리	같은 <code>root_path</code> 공유 시 충돌 가능
적합한 용도	임시 draft, 1회성 분석	실제 코드베이스 수정, 보고서 산출물

규칙: 결과물을 스레드 종료 후에도 보존해야 하면 Filesystem, 그렇지 않으면 State를 씁니다.

4.6 일반 파일 도구와의 관계

Deep Agents의 `FilesystemMiddleware` 나 커스텀 `@tool` 로 짠 `read_file` / `write_file` 도 같은 목적을 달성합니다. 차이는 Claude가 이 도구를 학습 단계에서 이미 봤다는 점 – 도구 스키마 토큰이 줄고 오류가 적습니다. Claude 전용 파이프라인이라면 이 미들웨어를 먼저 쓰고, 멀티 프로바이더라면 일반 파일 도구로 내리는 것이 기본 전략입니다.

핵심 정리

- Claude 네이티브 text editor는 tool schema 토큰을 거의 0으로 만들고 6가지 오퍼레이션을 지원
- 산출물 수명으로 State(스레드와 함께 소멸) vs Filesystem(디스크 영구 보존) 선택
- `allowed_path_prefixes` / `allowed_prefixes` 로 접근 경계를 강제
- Filesystem 변형은 `max_file_size_mb` 로 메모리 안전성 확보

05 Claude Memory

`/memories/\*` 경로 계약

Claude 네이티브 `memory_20250818` 도구는 모델이 스스로 메모를 쓰고 다시 읽는 장기 기억을 구현합니다. 주입되는 시스템 프롬프트가 매 턴 “먼저 `/memories` 를 확인하라”고 지시하므로, 같은 사용자와 여러 세션에 걸쳐 취향이나 사실이 자연스럽게 쌓입니다. State와 Filesystem 두 변형의 지속성 차이를 파악하는 것이 이 장의 핵심입니다.

학습 목표

LEARNING OBJECTIVES

- Claude 메모리 도구의 `/memories/` 경로 계약을 이해한다
- State vs Filesystem 변형의 지속성 범위 차이를 구분한다
- 주입되는 `system_prompt` 가 모델에게 메모리 활용을 지시하는 방식을 안다
- Deep Agents `StoreBackend` 와의 역할 분담을 정리한다

5.1 언제 쓰나

- 같은 사용자와 여러 세션에 걸쳐 취향/사실을 기억해야 하는 챗봇
- 에이전트가 장시간 작업 중 중간 결과를 본인이 쓰고 본인이 다시 읽어야 하는 경우
- RAG와 달리 모델이 직접 메모 내용을 관리(쓰고 지우고 수정)하도록 맡기고 싶을 때

5.2 환경 설정

필요 패키지: `langchain`, `langchain-anthropic`, `.env` 에 `ANTHROPIC_API_KEY`.

```
from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain_anthropic.middleware import (
    StateClaudeMemoryMiddleware,
    FilesystemClaudeMemoryMiddleware,
)
from langgraph.checkpoint.memory import MemorySaver

load_dotenv()
```

5.3 State 변형 — 스레드 내 지속

`StateClaudeMemoryMiddleware` 는 메모 내용을 LangGraph 상태(`memory_files`)에 저장합니다. `MemorySaver` / `PostgresCheckpointner` 등 체크포인트를 쓰면 같은 `thread_id`로 재개할 때마다 자동 복원됩니다.

파라미터	기본값	설명
<code>allowed_path_prefixes</code>	<code>["/memories"]</code>	메모 저장 허용 경로. 보통 그대로 둔다
<code>system_prompt</code>	Anthropic 기본	매 턴 시작 전 “/memories를 먼저 확인하라”를 지시

```
agent = create_agent(
    model="anthropic:claude-sonnet-4-6",
    checkpointer=MemorySaver(),
    middleware=[StateClaudeMemoryMiddleware()],
)

cfg = {"configurable": {"thread_id": "user-42"}}

agent.invoke(
    {"messages": [{"role": "user", "content": "내 이름은 지훈이야."}]},
    config=cfg,
)

# 같은 thread_id로 재호출 → 메모 재사용
result = agent.invoke(
    {"messages": [{"role": "user", "content": "내 이름 알지?"}]},
    config=cfg,
)
print(result["messages"][-1].content)
```

모델은 두 번째 호출에서 이름을 묻지 않고 `/memories` 를 읽어 “지훈”으로 답합니다.

5.4 Filesystem 변형 — 프로세스 경계를 넘어

`FilesystemClaudeMemoryMiddleware` 는 메모를 실제 디스크 디렉터리에 남깁니다. 프로세스가 종료되거나 체크포인트 저장소가 바뀌어도 디스크 파일이 있으면 그대로 다시 읽습니다.

파라미터	기본값	설명
<code>root_path</code>	(필수)	메모리를 저장할 실제 디렉터리
<code>allowed_prefixes</code>	<code>["/memories"]</code>	메모 작성 허용 가상 경로
<code>max_file_size_mb</code>	10	한 메모 파일 최대 크기
<code>system_prompt</code>	기본 제공	메모리 활용 지시 프롬프트

```
agent = create_agent(
    model="anthropic:claude-sonnet-4-6",
    middleware=[
        FilesystemClaudeMemoryMiddleware(
            root_path="/var/data/claude-memory",
            max_file_size_mb=2,
        ),
    ],
)
```

```
1,
)
```

5.5 영속 체크포인트로 확장

`MemorySaver` 는 프로세스 내 사전(dict) 저장이므로 재시작 시 휘발됩니다. 운영 환경에서는 `langgraph-checkpoint-postgres` 또는 `-sqlite` 의 컨텍스트 매니저 패턴을 씁니다 — `from_conn_string()` 으로 풀을 얻고, 처음 한 번 `setup()` 을 호출해 스키마를 생성합니다.

```
from langgraph.checkpoint.postgres import PostgresSaver

DB = "postgresql://user:pass@localhost/agent"

with PostgresSaver.from_conn_string(DB) as checkpointer:
    checkpointer.setup() # 최초 1회 - 테이블/인덱스 생성

    agent = create_agent(
        model="anthropic:claude-sonnet-4-6",
        checkpointer=checkpointer,
        middleware=[StateClaudeMemoryMiddleware()],
    )
```

TIP

비동기 그래프는 `AsyncPostgresSaver.from_conn_string(DB)` 를 `async with` 로 받습니다. `setup()` 도 `await` 이 필요합니다. SQLite도 동일한 컨텍스트 매니저 API(`SqliteSaver` / `AsyncSqliteSaver`)를 따릅니다.

5.6 `runtime.context.user_id` 로 멀티 테넌트 분리

같은 디스크 디렉터리(또는 같은 Store)를 여러 사용자가 공유할 때는 네임스페이스로 격리해야 합니다. LangGraph 1.2부터 `runtime.context.user_id` 를 직접 읽어 Store의 namespace 키로 쓰는 패턴이 표준입니다.

```
from langgraph.store.postgres import PostgresStore
from langgraph.runtime import get_runtime

with PostgresStore.from_conn_string(DB) as store:
    store.setup()

    def upsert_profile(profile: dict) -> str:
        rt = get_runtime()
        ns = ("profiles", rt.context.user_id) # 사용자별 네임스페이스
        store.put(ns, "self", profile)
        return "ok"
```

TIP

Claude Memory Middleware의 `/memories/*` 경로 계약과 LangGraph Store의 `(namespace, key)` 계약은 층위가 다릅니다. 전자는 Claude가 자율적으로 메모를 관리, 후자는 코드가 명시적으로 키를 부여 – 멀티 테넌트 격리는 Store 쪽이 자연스럽습니다.

5.7 Deep Agents `StoreBackend` 와의 관계

Deep Agents 0.5는 영구 메모리를 위한 `StoreBackend` 를 별도로 제공합니다. 역할은 비슷하지만 적용 범위가 다릅니다.

축	Claude Memory Middleware	Deep Agents <code>StoreBackend</code>
제공자	Anthropic 네이티브 도구	LangGraph Store API 래퍼
지원 모델	Claude 전용	모든 모델
저장 구조	<code>/memories/*</code> 파일	<code>(namespace, key) → dict</code>
검색 방식	파일 목록 + 내용 읽기	임베딩 벡터 검색 지원
적합한 용도	Claude가 자유롭게 메모 관리	구조화된 프로필·사실 저장

조합 팁: 둘을 함께 써도 됩니다. Claude가 단기 작업 노트를 `/memories` 에 남기고, 장기 프로필은 Deep Agents `StoreBackend` 에 넣어 다른 프로바이더 에이전트와 공유하는 패턴이 현실적입니다.

핵심 정리

- Claude 네이티브 메모리 도구는 `/memories/*` 경로 계약 + 자동 시스템 프롬프트로 “먼저 확인, 필요 시 갱신” 흐름을 강제한다
- State 변형은 체크포인트로 thread 재개 시 복원, Filesystem 변형은 디스크에 영구 보존
- `max_file_size_mb` 로 모델이 한 메모에 몰아 쓰는 것을 방지
- Deep Agents `StoreBackend` 는 벡터 검색·멀티 프로바이더 공유 시 보완재

06 Anthropic File Search

가상 파일의 glob + grep

`StateFileSearchMiddleware` 는 그래프 상태 안에 있는 가상 파일(예: `text_editor_files` , `memory_files`)을 glob + grep으로 검색하는 Claude 네이티브 도구를 제공합니다. 텍스트 에디터/메모리 미들웨어가 “파일을 만들고 편집”한다면, 이 미들웨어는 그 위에서 “파일을 찾고 읽는” 역할입니다.

학습 목표

LEARNING OBJECTIVES

- `StateFileSearchMiddleware(state_key=...)` 로 검색 대상 저장소를 선택한다
- `StateClaudeTextEditorMiddleware` 와 조합해 “쓰고 → 찾고 → 읽는” 루프를 완성한다
- `state_key="memory_files"` 로 메모리 파일도 검색 대상으로 돌린다
- 중복 미들웨어 제약과 서브클래싱 회피 패턴을 안다

6.1 언제 쓰나

- 에이전트가 수십 수백 개의 가상 파일을 상태에 쌓아놓고 탐색해야 할 때
- 이름만 기억나는 파일을 패턴(`**/*.md`)으로 찾거나, 특정 키워드를 포함한 파일만 골라야 할 때
- 메모리에 쌓인 과거 노트를 모델이 스스로 grep 해서 참조하게 하고 싶을 때

6.2 환경 설정

필요 패키지: `langchain` , `langchain-anthropic` . `.env` 에 `ANTHROPIC_API_KEY` .

```
from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain_anthropic.middleware import import (
    StateClaudeTextEditorMiddleware,
    StateFileSearchMiddleware,
)

load_dotenv()
```

6.3 텍스트 에디터 + 파일 검색 조합

검색은 단독으로 쓸 수 없습니다 – 파일이 먼저 상태에 있어야 합니다. 가장 많이 쓰는 조합은 text editor로 파일을 만들고 file search로 찾는 패턴입니다.

파라미터	기본값	설명
state_key	"text_editor_files"	검색할 파일 dict가 들어 있는 상태 키. "memory_files" 로 바꾸면 메모리 쪽 검색

```
agent = create_agent(
    model="anthropic:claude-sonnet-4-6",
    middleware=[
        StateClaudeTextEditorMiddleware(),
        StateFileSearchMiddleware(state_key="text_editor_files"),
    ],
)
```

6.4 1단계 – 여러 파일을 상태에 심는다

모델에게 `/docs/` 아래에 노트 파일 몇 개를 만들어 달라고 요청합니다. 이후 검색 쿼리에서 이 파일들이 대상이 됩니다.

```
cfg = {"configurable": {"thread_id": "search-demo"}}
agent.invoke(
    {
        "messages": [
            {
                "role": "user",
                "content": (
                    "/docs/architecture.md, /docs/api.md, /docs/release-notes.md 세 파일을 "
                    "간단한 초안으로 만들어 줘."
                ),
            },
        ],
    },
    config=cfg,
)
```

6.5 2단계 – 같은 에이전트가 glob/grep으로 검색

같은 스레드에서 이어 호출하면 이전 파일들이 상태에 남아 있습니다. 모델은 `StateFileSearchMiddleware` 가 제공하는 `glob` 과 `grep` 도구로 파일을 찾고 내용을 읽습니다.

```
result = agent.invoke(
    {
        "messages": [
            {
                "role": "user",
                "content": "/docs/ 아래 md 파일 중 'API'가 언급된 파일만 알려줘",
            },
        ],
    },
)
```



```

    },
    config=cfg,
)
print(result["messages"][-1].content)

```

6.6 메모리 파일도 검색 대상으로

`state_key="memory_files"` 로 바꾸면 `StateClaudeMemoryMiddleware` 가 쌓아둔 메모를 검색할 수 있습니다.

! WARNING

중복 미들웨어 제약 – LangChain 1.2 `create_agent` 는 같은 미들웨어 클래스의 중복 인스턴스를 거부합니다 (`AssertionError: Please remove duplicate middleware instances.`). 텍스트 에디터 파일과 메모리 파일 두 저장소를 동시에 검색하려면 한 번에 하나만 등록하거나, `StateFileSearchMiddleware` 를 서브클래싱해 별도 클래스로 분리해야 합니다.

```

class MemoryFileSearchMiddleware(StateFileSearchMiddleware):
    """`memory_files` 전용 검색 미들웨어."""

agent = create_agent(
    model="anthropic:claude-sonnet-4-6",
    middleware=[
        StateClaudeTextEditorMiddleware(),
        StateClaudeMemoryMiddleware(),
        StateFileSearchMiddleware(state_key="text_editor_files"),
        MemoryFileSearchMiddleware(state_key="memory_files"),
    ],
)

```

6.7 Retrieval 5단계 building block에서의 위치

LangChain RAG는 Document Loaders → Text Splitters → Embeddings → Vector Stores → Retrievers 다섯 단계로 표준화되어 있습니다(08_integration의 04 05 카테고리).

`StateFileSearchMiddleware` 는 이 다섯 단계를 우회하는 lite 경로입니다 – 임베딩 비용 없이 현재 상태에 있는 파일을 리터럴 grep으로 뒤집니다.

축	State File Search	5단계 RAG 파이프라인
전처리	없음 – 상태에 들어온 그대로	Loader + Splitter + Embed + Index
검색 방식	literal glob + grep	vector similarity + filter
지연	밀리초 단위	임베딩 호출 + ANN
적합한 규모	수십 수백 파일	수만 수백만 청크

```

# 5단계 RAG로 확장할 때
from langchain.embeddings import init_embeddings

```

```
from langchain_chroma import Chroma

embeddings = init_embeddings("openai:text-embedding-3-small")
vectorstore = Chroma(collection_name="docs", embedding_function=embeddings)
retriever = vectorstore.as_retriever(search_kwargs={"k": 5})
```

TIP

`init_embeddings("provider:model")` 헬퍼는 `init_chat_model` 과 같은 패턴 – provider prefix 문자열로 임베딩 백엔드를 한 줄로 교체합니다. State File Search에서 벡터스토어 RAG로 넘어갈 때 코드 변경이 최소화됩니다.

6.8 vectorstore와의 선택 기준

- **정확한 파일명/패턴**이 중요하고 리터럴 grep으로 충분할 때
- 문서 수가 수십 수백 개 수준이고 매 턴 임베딩을 다시 돌릴 비용이 아까울 때
- 파일이 현재 세션에서 방금 만들어진 것이라 벡터 인덱스가 없을 때

대규모 코퍼스나 의미 검색이 필요하다면 Part VIII Chat Models/Vector Stores 영역(향후 확장)으로 넘어갑니다.

핵심 정리

- `StateFileSearchMiddleware` 는 상태 안 가상 파일을 glob + grep으로 검색한다
- text editor + file search 조합이 “쓰고 → 찾고 → 읽는” 루프의 기본
- `state_key` 를 바꿔 메모리 파일도 검색 대상으로 돌리되, 중복 미들웨어 제약은 서브클래싱으로 우회
- 리터럴 검색이 충분할 때는 벡터스토어보다 이 미들웨어가 저비용

07 Bedrock Prompt Caching

AWS 경유 Claude/Nova 캐시

AWS Bedrock 경유로 Claude/Nova 모델을 호출할 때 `BedrockPromptCachingMiddleware` 로 시스템 프롬프트 · 도구 정의 · 마지막 메시지 위치에 캐시 체크포인트를 자동으로 박아 입력 토큰 비용을 낮춥니다.

`AnthropicPromptCachingMiddleware` (ch2)와 파라미터 이름과 의미는 거의 같지만 대상 패키지와 모델별 제약이 다릅니다.

학습 목표

LEARNING OBJECTIVES

- `BedrockPromptCachingMiddleware` 의 4개 파라미터를 이해한다
- `usage_metadata.input_token_details` 에서 `cache_creation` / `cache_read` 를 읽어 적중을 검증한다
- `ChatBedrock` VS `ChatBedrockConverse` 에서 `type` 파라미터 처리 차이를 안다
- Nova 모델의 제약(5m TTL 전용, tool 정의 캐시 미지원)을 구분한다

7.1 언제 쓰나

- AWS 계정·VPC 경계 안에서만 LLM을 호출해야 하는 기업 환경 (Bedrock 경유)
- Claude를 Anthropic 직접 API가 아닌 Bedrock 요금제로 이용할 때
- Amazon Nova 모델을 쓰면서 긴 시스템 프롬프트를 반복 사용할 때

7.2 환경 설정

필요 패키지: `langchain`, `langchain-aws`. AWS 자격증명(`AWS_ACCESS_KEY_ID` 등)과 리전이 `.env` 또는 환경에 설정돼 있어야 합니다. 모델별로 Bedrock 콘솔에서 모델 액세스를 먼저 허용해야 호출이 됩니다.

```
from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain_aws.middleware import BedrockPromptCachingMiddleware

load_dotenv()
```

7.3 ChatBedrockConverse + Claude (권장)

Converse API는 Bedrock의 통합 인터페이스로, 다양한 모델을 같은 API로 호출합니다.

`BedrockPromptCachingMiddleware` 를 붙이면 `system/tool/last-message` 위치에 캐시 체크포인트가 자동으로 삽입됩니다.

! WARNING

체크포인트가 실제로 작동하려면 캐시 대상 블록이 약 1,024 토큰 이상이어야 합니다. 짧은 system prompt에 서는 `cache_creation` 이 잡히지 않을 수 있습니다.

```
long_prompt = (
    "You are an enterprise policy assistant. Always cite section numbers. "
    * 200 # ~2,000 tokens
)

agent = create_agent(
    model="bedrock_converse:anthropic.claude-sonnet-4-20250514-v2:0",
    system_prompt=long_prompt,
    tools=[],
    middleware=[BedrockPromptCachingMiddleware(ttl="1h")],
)
```

7.4 캐시 적중 검증

두 번 연속 호출한 뒤 `usage_metadata.input_token_details` 를 읽어 캐시 생성/적중을 확인합니다.

```
for i in range(2):
    result = agent.invoke(
        {"messages": [{"role": "user", "content": f"요청 {i}"}]},
    )
    last = result["messages"][-1]
    print(i, last.usage_metadata.get("input_token_details"))
```

`cache_creation` 은 1회차에 채워지고, `cache_read` 는 2회차 이후에 늘어납니다.

7.5 파라미터 전체

파라미터	기본값	설명
<code>type</code>	"ephemeral"	ChatBedrock 전용. ChatBedrockConverse 는 이 값을 무시하고 "default" 사용
<code>ttl</code>	"5m"	"5m" 또는 "1h". Nova 계열은 "5m" 만 지원
<code>min_messages_to_cache</code>	0	메시지 수가 이 이상일 때만 cache 체크포인트 부착
<code>unsupported_model_behavior</code>	"warn"	캐시 미지원 모델에 대해 "ignore" / "warn" / "raise"

7.6 ChatBedrock (Invoke 모델) 예시

ChatBedrock 은 이전 세대 invoke-model 래퍼입니다. BedrockPromptCachingMiddleware 는 이쪽도 지원하며 type="ephemeral" 파라미터가 여기서는 실제로 반영됩니다. ChatBedrock 은 Anthropic 모델 한정으로 tool 정의 캐시와 확장 TTL(1h)을 모두 지원합니다.

```
from langchain_aws import ChatBedrock

model = ChatBedrock(model_id="anthropic.claude-sonnet-4-20250514-v2:0")

agent = create_agent(
    model=model,
    system_prompt=long_prompt,
    tools=[],
    middleware=[
        BedrockPromptCachingMiddleware(type="ephemeral", ttl="1h"),
    ],
)
```

7.7 Reasoning · Profiling

Claude 4.6/4.7 모델은 Bedrock에서도 extended thinking(reasoning)이 지원됩니다. 캐싱은 reasoning이 켜진 호출에도 적용되지만, reasoning 블록 자체는 캐시 미스로 매번 새로 계산되니 토큰 절감 효과가 무 reasoning 케이스보다 작습니다.

```
from langchain_aws import ChatBedrockConverse

model = ChatBedrockConverse(
    model_id="anthropic.claude-sonnet-4-20250514-v2:0",
    additional_model_request_fields={
        "reasoning_config": {"type": "enabled", "budget_tokens": 4096},
    },
)
print(model.profile.reasoning_output) # True → 응답에 reasoning 블록 포함
```

TIP

ChatBedrockConverse(...).profile (LangChain 1.1+)에서 tool_calling, reasoning_output, max_input_tokens 같은 모델 능력치를 확인할 수 있습니다 – 멀티 모델 라우팅 로직에서 capability check로 쓰면 깨끗합니다.

7.8 Amazon Nova 제약

항목	ChatBedrockConverse + Anthropic	ChatBedrockConverse + Nova
시스템 프롬프트 캐시	O	O

도구 정의 캐시	O	X
메시지 캐시	O	O (tool result 메시지 제외)
TTL "1h"	O	X (5m 전용)

Nova 모델을 쓸 때는 `ttl="5m"` 으로 고정하고, `unsupported_model_behavior="warn"` 을 설정해 실수로 `"1h"` 를 지정했을 때 조용히 넘어가지 않도록 경고를 남기세요.

핵심 정리

- AWS 경계 안에서 Claude/Nova 사용 시 Bedrock 미들웨어로 입력 토큰 비용 절감
- `ChatBedrock` 과 `ChatBedrockConverse` 의 `type` 파라미터 처리 차이를 인지
- Nova 모델은 5m TTL + tool 정의 캐시 미지원 – 설정 시 `ttl="5m"` 고정
- 캐시 대상 블록은 약 1,024 토큰 이상이어야 실제 적용 발생

08 OpenAI Moderation

정책 위반 콘텐츠 사전/사후 차단

`OpenAIModerationMiddleware` 는 OpenAI Moderation API로 사용자 입력 · 모델 출력 · 도구 결과를 자동 스캔하고, 정책 위반이 감지되면 대화를 차단·교체·예외로 처리합니다. 공개 챗봇의 혐오·폭력·자해 카테고리를 선제 차단하거나, 외부 도구 결과의 2차 오염을 막을 때 씁니다.

학습 목표

LEARNING OBJECTIVES

- `check_input` / `check_output` / `check_tool_results` 세 스캔 지점의 의미와 비용 트레이드오프를 안다
- `exit_behavior` 세 모드("end" / "error" / "replace")의 동작을 구분한다
- 커스텀 `violation_message` 템플릿으로 사용자 응답을 다듬는다
- 사전 구성된 OpenAI 클라이언트를 `client` / `async_client` 로 주입한다

8.1 언제 쓰나

- 공개 챗봇에서 혐오·폭력·자해 같은 Moderation 카테고리를 선제 차단할 때
- 도구 결과(예: 웹 검색, 이메일 본문)에 유해 콘텐츠가 섞여 모델에 들어가는 것을 막을 때
- 로그/감사용으로 위반 메타데이터를 응답 흐름에 남겨 보고서를 만들 때

8.2 환경 설정

필요 패키지: `langchain`, `langchain-openai`, `.env` 에 `OPENAI_API_KEY` 가 있어야 합니다.

```
from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain_openai.middleware import OpenAIModerationMiddleware

load_dotenv()
```

8.3 기본 사용 — 입출력 모두 검사, 위반 시 종료

기본 설정은 `check_input=True`, `check_output=True`, `exit_behavior="end"`. 사용자 입력이 Moderation API에 걸리면 모델 호출 없이 바로 위반 메시지로 대화를 종료합니다.

파라미터	기본값	설명
<code>model</code>	"omni-moderation-latest"	사용할 Moderation 모델
<code>check_input</code>	True	사용자 메시지를 모델에 넘기기 전 검사
<code>check_output</code>	True	모델 생성 응답을 사용자에게 반환하기 전 검사
<code>check_tool_results</code>	False	도구 실행 결과를 모델 입력 전에 검사
<code>exit_behavior</code>	"end"	"end" / "error" / "replace"
<code>violation_message</code>	기본 템플릿	위반 시 사용자에게 보여줄 텍스트
<code>client</code> / <code>async_client</code>	None	사전 구성된 OpenAI 클라이언트 주입 (선택)

```
agent = create_agent(
    model="openai:gpt-5.4",
    tools=[],
    middleware=[OpenAIModerationMiddleware()],
)
```

8.4 `exit_behavior="end"` — 위반 입력 즉시 종료

유해 카테고리(예: 자해 유도)가 감지되면 모델 호출 없이 위반 메시지로 바로 마감합니다. 응답 마지막 메시지는 기본 위반 메시지가 담깁니다.

8.5 `exit_behavior="error"` — 예외로 빠르게 실패

테스트/배치 환경에서 위반을 조용히 넘기지 않고 예외로 띄울 때 씁니다. `OpenAIModerationError` 가 발생하며 상위 파이프라인에서 catch해 감사 로그에 남깁니다.

```
from langchain_openai.middleware import OpenAIModerationError

agent = create_agent(
    model="openai:gpt-5.4",
    tools=[],
    middleware=[OpenAIModerationMiddleware(exit_behavior="error")],
)

try:
    agent.invoke({"messages": [{"role": "user", "content": "..."}]})
except OpenAIModerationError as e:
    # 감사 로그에 기록
    print("Moderation violation:", e.categories)
```


8.6 `exit_behavior="replace"` — 메시지만 교체하고 계속

출력 검사에서 위반이 잡히면 해당 응답만 위반 메시지로 교체하고 그래프 실행은 이어집니다. 멀티턴 대화에서 흐름은 끊지 않고 특정 응답만 정리할 때 쓰는 모드입니다.

```
agent = create_agent(
    model="openai:gpt-5.4",
    tools=[],
    middleware=[
        OpenAIModerationMiddleware(
            exit_behavior="replace",
            violation_message=(
                "이 응답은 안전 정책에 따라 표시할 수 없습니다 "
                "(카테고리: {categories}). 다른 질문을 해 주세요."
            ),
        ),
    ],
)
```

`violation_message` 는 `{categories}`, `{category_scores}`, `{original_content}` 변수를 지원하는 템플릿 문자열입니다.

8.7 도구 결과 스캔 (`check_tool_results=True`)

웹 검색·크롤링·DB 조회 결과에 제3자 유해 콘텐츠가 섞여 모델에 들어가는 것을 막습니다. 비용은 도구 호출 수에 비례해 늘어나므로 신뢰할 수 없는 외부 소스를 쓰는 파이프라인에서만 켭니다.

```
agent = create_agent(
    model="openai:gpt-5.4",
    tools=[web_search_tool, fetch_url_tool],
    middleware=[
        OpenAIModerationMiddleware(
            check_input=True,
            check_output=True,
            check_tool_results=True,
        ),
    ],
)
```

8.8 LangSmith로 위반 흐름 관측

Moderation 차단은 드물지만 감사 로그에는 반드시 남아야 하는 사건입니다. LangSmith의 자동 트레이싱 (`LANGSMITH_TRACING=true` + `LANGSMITH_API_KEY`)을 켜면 `OpenAIModerationMiddleware` 가 트리거된 노드도 트레이스 트리에 정상적으로 박힙니다.

```
import os
from langsmith.run_helpers import tracing_context

os.environ["LANGSMITH_TRACING"] = "true"
# os.environ["LANGSMITH_API_KEY"] = "ls_..."
# os.environ["LANGSMITH_PROJECT"] = "moderation-audit"

with tracing_context(
    project_name="moderation-audit",
```

```
tags=["moderation", "input-only"],
metadata={"region": "kr"},
):
    agent.invoke({"messages": [{"role": "user", "content": "..."}]})
```

TIP

`tracing_context` 는 특정 호출 구간만 다른 프로젝트/태그로 분리해 라우팅합니다 – 평소 트레이스는 `production` 에, Moderation 차단 케이스만 `moderation-audit` 에 모아두는 식의 운영이 자연스럽습니다. `LANGCHAIN_*` 환경변수는 1.1부터 `LANGSMITH_*` 로 표준화됐습니다 – 옛 변수도 한동안 동작하지만 신규 코드에서는 `LANGSMITH_*` 만 씁니다.

8.9 비용·지연 트레이드오프

설정	비용	지연	주요 효과
<code>check_input=True</code>	Mod 1 + Model 1	+1 RT	유해 입력이 모델에 들어가기 전 차단
<code>check_output=True</code>	Model 1 + Mod 1	+1 RT	모델이 잘못된 응답을 냈을 때 최종 사용자 보호
<code>check_tool_results=True</code>	도구 수 × Mod	도구당 +1	외부 오염 데이터 차단

TIP

실전 가이드: 프로덕션에서는 `check_input` 만 항상 켜고 `check_output` 은 샘플링(예: 10%) 전략으로 운영하는 경우가 많습니다. `check_tool_results` 는 외부 신뢰할 수 없는 소스 전용으로 제한합니다.

핵심 정리

- 세 스캔 지점을 비용/지연 트레이드오프로 선택: input 항상 켜기, output 샘플링, tool_results 조건부
- `exit_behavior` 로 대화 종료(`end`)/즉시 예외(`error`)/메시지 교체(`replace`) 선택
- `violation_message` 템플릿으로 사용자 친화적 안내 구성
- `client` / `async_client` 로 사전 구성된 OpenAI 클라이언트를 재사용해 초기화 비용 절감

A

부록 A. 용어집

Book 전체에서 반복 등장하는 핵심 용어

이 부록은 책 전반에서 자주 등장하는 핵심 용어를 한 번에 다시 찾아볼 수 있도록 정리한 참고 섹션입니다. 처음부터 모두 외울 필요는 없으며, 낯선 용어가 나올 때마다 다시 찾아보는 방식으로 활용하면 됩니다.

· NOTE

용어집의 목적은 API 문서를 대체하는 것이 아니라, 책을 읽을 때 자주 마주치는 개념을 빠르게 재확인할 수 있게 하는 데 있습니다. 구현 세부사항은 각 장의 본문과 참고 문서를 함께 보세요.

프레임워크와 제품

LangChain 모델, 도구, 프롬프트, 미들웨어를 조합해 에이전트와 LLM 애플리케이션을 빠르게 만드는 상위 프레임워크입니다.

LangGraph 상태, 노드, 엣지, 체크포인트를 기반으로 복잡한 워크플로와 장기 실행형 에이전트를 오케스트레이션하는 런타임/프레임워크입니다.

Deep Agents LangGraph 위에 계획, 파일시스템, 서브에이전트, 메모리, 샌드박스 같은 하네스 기능을 엮은 올인원 에이전트 SDK입니다.

LangSmith 트레이싱, 평가, 데이터셋 관리, 디버깅을 지원하는 관측성과 품질 관리 플랫폼입니다.

Langfuse 트레이싱과 관측성 수집을 위한 오픈소스 계열의 observability 도구입니다.

MCP Model Context Protocol의 약자로, 외부 도구와 리소스를 모델/에이전트에 표준 방식으로 연결하는 프로토콜입니다.

ACP Agent Client Protocol의 약자로, 에이전트를 에디터나 IDE 같은 클라이언트에 연결하기 위한 통신 프로토콜입니다.

에이전트와 실행 모델

Agent LLM이 도구를 사용하고 결과를 관찰하며 작업이 끝날 때까지 반복적으로 행동하는 실행 단위입니다.

ReAct Reasoning + Acting의 줄임말로, 모델이 추론과 도구 사용을 반복하며 문제를 해결하는 대표적인 에이전트 패턴입니다.

Workflow 단계와 순서가 비교적 명확하게 정의된 실행 흐름입니다. 에이전트보다 결정론적 성격이 강합니다.

Orchestrator 여러 단계나 여러 워커의 작업을 조정하고 전체 흐름을 관리하는 상위 제어자입니다.

Worker 오케스트레이터에게 위임받은 특정 작업을 수행하는 실행 단위입니다.

Subagent 메인 에이전트가 세부 작업을 위임하기 위해 호출하는 보조 에이전트입니다. 독립된 컨텍스트를 갖는 경우가 많습니다.

Handoff 현재 단계나 상태에 따라 다른 역할/에이전트로 제어를 넘기는 패턴입니다.

Router 입력이나 상태를 분류하여 적절한 처리 경로나 전문 에이전트로 보내는 패턴입니다.

Human-in-the-Loop 민감한 도구 실행이나 중요한 분기 지점에서 사람의 승인·수정·거부를 끼워 넣는 안전 장치입니다.

Interrupt 그래프나 에이전트 실행을 중간에서 멈추고 외부 입력을 기다리게 만드는 메커니즘입니다.

Resume 중단된 실행을 이전 상태에서 다시 이어가는 동작입니다. 보통 같은 `thread_id` 와 체크포인트가 필요합니다.

Durable Execution 실행 상태를 저장해 두었다가 장애나 중단 이후 마지막 성공 지점부터 다시 시작할 수 있게 하는 실행 방식입니다.

Pregel LangGraph 내부 실행 모델에 영향을 준 메시지 패싱 기반 계산 모델입니다. 노드가 슈퍼스텝 단위로 동작합니다.

Superstep Pregel 모델에서 여러 노드가 한 라운드로 실행되는 병렬 계산 단위를 뜻합니다.

상태와 메모리

State 에이전트나 그래프가 실행되는 동안 유지·업데이트하는 현재 작업 정보 묶음입니다.

AgentState LangChain/에이전트 실행에서 사용하는 기본 상태 스키마입니다. `messages` 등 예약 필드를 포함합니다.

MessagesState 메시지 리스트를 중심으로 상태를 관리하는 LangGraph의 편의 상태 타입입니다.

Checkpoint 실행 상태를 저장하고 복구하는 컴포넌트입니다. 멀티턴 대화, interrupt/resume, durable execution의 핵심입니다.

Thread ID 하나의 대화/실행 흐름을 식별하는 고유 ID입니다. 같은 `thread_id` 를 사용해야 이전 상태를 이어서 불러올 수 있습니다.

Runtime Context 실행 시점에만 주입되는 메타데이터·권한·사용자 정보 같은 컨텍스트입니다.

`context_schema` 실행 중 변하지 않는 정적 컨텍스트의 구조를 정의하는 스키마입니다.

`state_schema` 실행 중 계속 변하는 동적 상태의 구조를 정의하는 스키마입니다.

Short-term memory 하나의 스레드나 대화 안에서만 유지되는 메모리입니다. 주로 메시지 히스토리와 최근 작업 상태를 뜻합니다.

Long-term memory 대화가 끝난 뒤에도 남아 다음 스레드에서 재사용되는 메모리입니다. 사용자 선호도, 학습 결과 등에 적합합니다.

Store 스레드 바깥의 장기 데이터를 저장하고 검색하는 저장 계층입니다.

InMemoryStore 메모리 기반의 간단한 Store 구현입니다. 개발과 테스트에는 편리하지만 재시작 시 데이터가 사라집니다.

Semantic memory 사실·선호도·개념처럼 의미 기반으로 다시 찾아 쓸 수 있는 장기 기억입니다.

Episodic memory 특정 사건이나 상호작용 기록처럼 시간 순서가 중요한 기억입니다.

Procedural memory 에이전트가 따라야 할 규칙, 절차, 행동 방식에 해당하는 기억입니다.

도구와 인터페이스

Tool 에이전트가 호출할 수 있는 함수나 외부 작업 인터페이스입니다. 검색, 계산, 파일 읽기, API 호출 등이 여기에 해당합니다.

ToolRuntime 도구 실행 중 현재 상태, 컨텍스트, Store 등에 접근할 수 있게 해 주는 런타임 객체입니다.

`create_agent()` LangChain에서 기본 에이전트를 빠르게 생성하는 핵심 API입니다.

`create_deep_agent()` Deep Agents에서 계획, 파일, 서브에이전트 등을 포함한 하네스형 에이전트를 생성하는 API입니다.

`StateGraph` LangGraph에서 상태 기반 그래프 워크플로를 정의하는 빌더 클래스입니다.

Graph API 노드와 엣지를 명시적으로 선언해 그래프를 조립하는 LangGraph 프로그래밍 방식입니다.

Functional API Python 함수와 `@entrypoint`, `@task` 로 워크플로를 표현하는 LangGraph 프로그래밍 방식입니다.

`@entrypoint` Functional API에서 워크플로의 시작 함수를 정의하는 데코레이터입니다.

`@task` 내구성 보장이 필요한 작업 단위를 감싸는 Functional API 데코레이터입니다.

`Send` LangGraph에서 동적으로 여러 워커 실행을 fan-out하는 데 쓰이는 API입니다.

`Command` 상태 업데이트, resume, 부모 그래프 전이 등을 표현하는 LangGraph의 제어 객체입니다.

Structured output 모델의 응답을 Pydantic 스키마나 명시된 구조로 강제하는 출력 방식입니다.

백엔드와 실행 환경

Backend Deep Agents에서 파일 읽기/쓰기와 실행 환경을 추상화하는 저장·실행 계층입니다.

StateBackend 대화 스레드 안에서만 유지되는 에페메럴 파일 저장 백엔드입니다.

FilesystemBackend 로컬 디스크에 접근하는 백엔드입니다. `virtual_mode=True` 로 경로 제한을 두는 것이 일반적입니다.

StoreBackend LangGraph Store를 사용해 스레드 간 영속 저장을 제공하는 백엔드입니다.

CompositeBackend 경로별로 서로 다른 백엔드에 요청을 라우팅하는 혼합형 백엔드입니다.

LocalShellBackend 로컬 파일 접근에 더해 셸 명령 실행까지 제공하는 강력하지만 위험한 백엔드입니다.

Sandbox 호스트와 격리된 환경에서 코드 실행과 파일 작업을 수행하게 하는 실행 환경입니다.

Modal GPU와 AI/ML 워크로드에 강한 샌드박스/서버리스 실행 환경입니다.

Daytona 빠른 devbox provisioning과 개발 환경 중심 사용 사례에 적합한 샌드박스 제공자입니다.

Runloop 일회성 devbox와 격리 실행에 적합한 샌드박스 제공자입니다.

검색, 스트리밍, 품질 관리

RAG Retrieval-Augmented Generation의 약자로, 외부 지식을 검색해 모델 응답에 주입하는 패턴입니다.

Retriever 질문과 관련된 문서나 청크를 검색해 반환하는 컴포넌트입니다.

Embedding 텍스트를 의미 공간의 벡터로 바꿔 유사도 검색을 가능하게 만드는 표현입니다.

Vector store 임베딩을 저장하고 유사도 검색을 수행하는 저장소입니다.

Chunking 긴 문서를 검색과 컨텍스트 주입에 적합한 작은 단위로 분할하는 작업입니다.

SQL Agent 자연어 질문을 SQL 쿼리로 변환하고 실행 결과를 해석하는 에이전트입니다.

Streaming 모델 응답이나 실행 상태를 완료 전에 점진적으로 전달하는 방식입니다.

StreamEvent 스트리밍 중 전달되는 이벤트 단위입니다. 토큰, 도구 시작/종료, 상태 변경 등이 포함됩니다.

TTFT Time to First Token의 약자로, 모델이 첫 토큰을 내놓기까지 걸리는 시간입니다.

TTFA Time to First Audio의 약자로, 보이스 에이전트에서 첫 오디오 출력까지 걸리는 시간입니다.

Tracing 모델 호출, 도구 실행, 상태 전이, 에러를 기록해 실행 흐름을 추적하는 관측 기법입니다.

Evaluation 에이전트 출력 또는 실행 궤적의 품질을 측정하는 과정입니다.

LLM-as-Judge 다른 LLM이 응답 품질이나 실행 결과를 평가자 역할로 채점하는 방식입니다.

Trajectory 에이전트가 최종 답변에 도달하기까지 거친 도구 호출, 중간 추론, 상태 변화의 전체 흐름입니다.

Guardrail 입력·도구·출력 단계에서 안전성, 정책, 품질을 검증하는 제어 장치입니다.

PII Personally Identifiable Information의 약자로, 이메일·전화번호·주민번호처럼 개인을 식별할 수 있는 민감 정보입니다.